UNISYS

Universal Compiling System (UCS)

COBOL Compiler
Programming Reference Manual

Volume 1: COBOL Statements

Level 12R2

February 2015 7831 0448–005

NO WARRANTIES OF ANY NATURE ARE EXTENDED BY THIS DOCUMENT. Any product or related information described herein is only furnished pursuant and subject to the terms and conditions of a duly executed agreement to purchase or lease equipment or to license software. The only warranties made by Unisys, if any, with respect to the products described in this document are set forth in such agreement. Unisys cannot accept any financial or other responsibility that may be the result of your use of the information in this document or software material, including direct, special, or consequential damages.

You should be very careful to ensure that the use of this information and/or software material complies with the laws, rules, and regulations of the jurisdictions with respect to which it is used.

The information contained herein is subject to change without notice. Revisions may be issued to advise of such changes and/or additions.

Notice to U.S. Government End Users: This is commercial computer software or hardware documentation developed at private expense. Use, reproduction, or disclosure by the Government is subject to the terms of Unisys standard commercial license for the products, and where applicable, the restricted/limited rights provisions of the contract data rights clauses.

Unisys and ClearPath are registered trademarks of Unisys Corporation in the United States and other countries.
All other brands and products referenced in this document are acknowledged to be the trademarks or registered trademarks of their respective holders.

Contents

Section 1. Introduction

1.1. About this Manual 1–1
1.2. Documentation Updates 1–3
1.3. COBOL X3.23–1974,–1985 Differences Potentially Not Affecting Existing Programs 1–3
1.4. COBOL X3.23–1974,–1985 Differences Potentially Affecting Existing Programs 1–12

Section 2. Programming Considerations

2.1. General 2–1
2.2. Files 2–1
2.2.1. File Attributes 2–1
2.2.2. File Processing 2–2
2.3. Table Handling 2–5
2.3.1. Table Definition 2–5
2.3.2. Initial Values of Tables 2–6
2.3.3. Reference to Table Items 2–7
2.3.4. Subscripting 2–7
2.4. Language Concepts 2–9
2.4.1. Notation Used in General Formats 2–10
2.4.2. Rules 2–11
2.4.3. Character Set 2–12
2.4.4. Language Structure 2–12
2.4.5. Separators 2–12
2.4.6. Character Strings 2–13
2.4.7. COBOL Words 2–13
2.4.8. Literals 2–18
2.4.9. PICTURE Character Strings 2–22
2.4.10. Comment Entries 2–22
2.4.11. Logical Record Concept 2–22
2.4.12. Physical Aspects of a File 2–23
2.4.13. Conceptual Characteristics of a File 2–23
2.4.14. Record Concepts 2–23
2.4.15. Concept of Levels 2–24
2.4.16. Level Numbers 2–24
2.4.17. Concept of Classes of Data 2–25
2.4.18. Algebraic Signs 2–25
2.4.19. Standard Alignment Rules 2–26
2.4.20. Uniqueness of Reference 2–26
2.4.21. Explicit and Implicit Specifications 2–33
2.4.22. External Switch 2–36

2.5. Source Program Construction 2–37
2.5.1. Identification Division 2–37
2.5.2. Environment Division 2–38
2.5.3. Data Division 2–39
2.5.4. Procedure Division 2–40
2.6. Reference Format 2–44
2.6.1. Representation 2–45
2.6.2. Sequence Numbers 2–45
2.6.3. Continuation of Lines 2–46
2.6.4. Blank Lines 2–46
2.6.5. Comment Lines 2–46
2.6.6. Pseudotext 2–46
2.6.7. Division Header 2–46
2.6.8. Section Header 2–47
2.6.9. Paragraph Header, Paragraph Name, and Paragraph 2–47
2.6.10. Data Division Entries 2–47
2.6.11. Declaratives 2–48
2.6.12. End Program Header 2–48

Section 3. Nucleus

3.1. General 3–1
3.1.1. Level Characteristics 3–1
3.1.2. Level Restrictions on Overall Language 3–1
3.2. A COBOL Source Program 3–2
3.2.1. General Description 3–2
3.2.2. Organization 3–3
3.2.3. Structure 3–3
3.2.4. End Program Header 3–4
3.3. Identification Division 3–4
3.3.1. General Description 3–4
3.3.2. Organization 3–4
3.3.3. Structure 3–5
3.3.4. The PROGRAM-ID Paragraph 3–5
3.3.5. The DATE-COMPILED Paragraph 3–6
3.4. Environment Division 3–6
3.4.1. General Description 3–6
3.4.2. Configuration Section 3–6
3.4.3. The SOURCE-COMPUTER Paragraph 3–7
3.4.4. The OBJECT-COMPUTER Paragraph 3–8
3.4.5. The SPECIAL-NAMES Paragraph 3–9
3.5. Data Division 3–14
3.5.1. General Description 3–14
3.5.2. Working-Storage Section 3–14
3.5.3. Record Description Structure 3–15
3.5.4. Initial Values 3–15
3.5.5. The Data Description Entry 3–15
3.5.6. The BLANK WHEN ZERO Clause 3–18
3.5.7. The Data Name or FILLER Clause 3–18
3.5.8. The JUSTIFIED Clause 3–19

3.5.9. Level Number 3–20
3.5.10. The OCCURS Clause 3–21
3.5.11. The PICTURE Clause 3–25
3.5.12. The REDEFINES Clause 3–40
3.5.13. The RENAMES Clause 3–42
3.5.14. The SIGN Clause 3–44
3.5.15. The SYNCHRONIZED Clause 3–45
3.5.16. The USAGE Clause 3–47
3.5.17. The VALUE Clause 3–53
3.5.18. The USING POINTER Clause 3–55
3.5.19. USAGE POINTER pointers 3–58
3.6. Procedure Division 3–69
3.6.1. General Description 3–69
3.6.2. Arithmetic Expressions 3–69
3.6.3. Conditional Expressions 3–73
3.6.4. Order of Evaluation of Condition 3–87
3.6.5. Common Options and Rules for Statements 3–92
3.6.6. The ACCEPT Statement 3–102
3.6.7. The ADD Statement 3–105
3.6.8. The ALTER Statement 3–107
3.6.9. The COMPUTE Statement 3–107
3.6.10. The CONTINUE Statement 3–108
3.6.11. The DISPLAY Statement 3–108
3.6.12. The DIVIDE Statement 3–111
3.6.13. The EVALUATE Statement 3–114
3.6.14. The EXIT Statement 3–118
3.6.15. The GO TO Statement 3–119
3.6.16. The IF Statement 3–120
3.6.17. The INITIALIZE Statement 3–122
3.6.18. The INSPECT Statement 3–124
3.6.19. The MOVE Statement 3–133
3.6.20. The MULTIPLY Statement 3–137
3.6.21. The PERFORM Statement 3–138
3.6.22. The SEARCH Statement 3–148
3.6.23. The SET Statement 3–152
3.6.24. The STOP Statement 3–156
3.6.25. The STRING Statement 3–157
3.6.26. The SUBTRACT Statement 3–161
3.6.27. The UNSTRING Statement 3–163
3.7. Debugging 3–168
3.7.1. General Description 3–168
3.7.2. Compile-Time Switch 3–169
3.7.3. Debugging Lines 3–169

Section 4. Sequential I-O

4.1. Level Characteristics 4–1
4.2. Language Concepts 4–1
4.2.1. Organization 4–1
4.2.2. Access Mode 4–1
4.2.3. Current Volume Pointer 4–2

4.2.4. File Position Indicator 4–2
4.2.5. I-O Status 4–2
4.2.6. The At-End Condition 4–4
4.2.7. The File Attribute Conflict Condition 4–4
4.2.8. Special Register LINAGE-COUNTER 4–5
4.3. Environment Division/Input-Output Section 4–5
4.3.1. The FILE-CONTROL Paragraph 4–5
4.3.2. The File Control Entry 4–5
4.3.3. The I-O-CONTROL Paragraph 4–13
4.4. Data Division/File Section 4–16
4.4.1. General Format 4–16
4.4.2. File Description Entry 4–17
4.4.3. Record Description Structure 4–17
4.4.4. Initial Values 4–17
4.4.5. The File Description - Complete Entry Skeleton 4–17
4.5. Procedure Division 4–29
4.5.1. General Description 4–29
4.5.2. The CLOSE Statement 4–29
4.5.3. The OPEN Statement 4–33
4.5.4. The READ Statement 4–38
4.5.5. The REWRITE Statement 4–41
4.5.6. The USE Statement 4–42
4.5.7. The WRITE Statement 4–44
4.6. Example of Sequential I-O 4–48
4.6.1. Source Program 4–48
4.6.2. Generated Output 4–54

Section 5. Relative I-O

5.1. Level Characteristics 5–1
5.2. Language Concepts 5–1
5.2.1. Organization 5–1
5.2.2. Access Modes 5–2
5.2.3. File Position Indicator 5–2
5.2.4. I-O Status 5–2
5.2.5. The Invalid Key Condition 5–5
5.2.6. The At-End Condition 5–6
5.2.7. The File Attribute Conflict Condition 5–6
5.3. Environment Division/Input-Output Section 5–7
5.3.1. The FILE-CONTROL Paragraph 5–7
5.3.2. The File Control Entry 5–7
5.3.3. The I-O-CONTROL Paragraph 5–10
5.4. Data Division/File Section 5–11
5.4.1. The File Description - Complete Entry Skeleton 5–11
5.5. Procedure Division 5–12
5.5.1. General Description 5–12
5.5.2. The CLOSE Statement 5–13
5.5.3. The DELETE Statement 5–15
5.5.4. The OPEN Statement 5–16
5.5.5. The READ Statement 5–19
5.5.6. The REWRITE Statement 5–23

5.5.7. The START Statement 5–25
5.5.8. The USE Statement 5–26
5.5.9. The WRITE Statement 5–28

Section 6. Indexed I-O

6.1. Level Characteristics 6–1
6.2. Language Concepts 6–1
6.2.1. Organization 6–1
6.2.2. Access Modes 6–2
6.2.3. File Position Indicator 6–2
6.2.4. I-O Status 6–2
6.2.5. The Invalid Key Condition 6–6
6.2.6. The At-End Condition 6–7
6.2.7. The File Attribute Conflict Condition 6–7
6.2.8. Statistics 6–7
6.3. Environment Division/Input-Output Section 6–7
6.3.1. The FILE-CONTROL Paragraph 6–7
6.3.2. The File Control Entry 6–7
6.3.3. The I-O-CONTROL Paragraph 6–14
6.4. Data Division/File Section 6–14
6.4.1. The File Description - Complete Entry Skeleton 6–15
6.5. The Procedure Division 6–16
6.5.1. General Description 6–16
6.5.2. The CLOSE Statement 6–17
6.5.3. The DELETE Statement 6–18
6.5.4. The OPEN Statement 6–19
6.5.5. The READ Statement 6–23
6.5.6. The REWRITE Statement 6–27
6.5.7. The START Statement 6–29
6.5.8. The USE Statement 6–31
6.5.9. The WRITE Statement 6–33

Section 7. Interprogram Communication

7.1. Level Characteristics 7–1
7.2. Language Concepts 7–1
7.2.1. Transfer of Control 7–1
7.2.2. Shared Data 7–2
7.2.3. Program and Run Unit Organization 7–2
7.3. Nested Source Programs 7–13
7.3.1. General Description 7–13
7.3.2. Organization 7–14
7.3.3. Structure 7–14
7.3.4. Initial State of a Program 7–15
7.3.5. End Program Header 7–15
7.4. Identification Division 7–16
7.5. Environment Division 7–17
7.6. Data Division 7–18
7.6.1. Linkage Section 7–18
7.6.2. Common-Storage Section 7–20

7.6.3. File Description Entry 7–21
7.6.4. The Data Description Entry 7–23
7.6.5. Report Description Entry 7–25
7.6.6. The EXTERNAL Clause 7–26
7.6.7. The GLOBAL Clause 7–27
7.7. Procedure Division 7–28
7.7.1. The Procedure Division Header 7–28
7.7.2. The CALL Statement 7–29
7.7.3. The CANCEL Statement 7–33
7.7.4. The ENTER Statement 7–34
7.7.5. The EXIT PROGRAM Statement 7–38
7.7.6. The GOBACK Statement 7–39
7.7.7. The USE Statement 7–39
7.7.8. The USE BEFORE REPORTING Statement 7–40

Section 8. Sort-Merge Module

8.1. General 8–1
8.2. Language Concepts 8–1
8.2.1. Sort File 8–1
8.2.2. Merge File 8–2
8.3. Environment Division/Input-Output Section 8–2
8.3.1. The FILE-CONTROL Paragraph 8–2
8.3.2. The File Control Entry 8–2
8.3.3. The I-O-CONTROL Paragraph 8–3
8.3.4. The SAME RECORD/SORT/SORT-MERGE AREA Clause 8–3
8.4. The Data Division/File Section 8–5
8.4.1. Sort-Merge File Description Entry 8–5
8.4.2. Record Description Structure 8–5
8.4.3. Initial Values 8–5
8.4.4. The Sort-Merge File Description Complete Entry Skeleton 8–6
8.5. Procedure Division 8–6
8.5.1. The MERGE Statement 8–6
8.5.2. The RELEASE Statement 8–12
8.5.3. The RETURN Statement 8–13
8.5.4. The SORT Statement 8–14
8.6. Sort-Merge Considerations 8–20
8.6.1. Main Storage Size 8–20
8.6.2. Sort-Merge Files 8–20
8.6.3. Suppression of SORT Console and Log Messages 8–20
8.7. Error Messages from a Sort or a Merge 8–21
8.8. Internationalization Considerations 8–21
8.8.1. Culturally Sensitive Sort/Merge 8–21
8.8.2. Code Type and Locale of Sort/Merge Files 8–22
8.9. Examples 8–22

Section 9. Source Text Manipulation

9.1. General 9–1
9.1.1. Libraries and Source Programs 9–1
9.1.2. Level Characteristics 9–1
9.2. The COPY Statement 9–2
9.3. The REPLACE Statement 9–7

Section 10. Report Writer

10.1. General 10–1
10.1.1. Function 10–1
10.1.2. Report File 10–1
10.1.3. Report Section 10–2
10.1.4. Report Structure 10–2
10.1.5. Report Procedure 10–4
10.1.6. PAGE-COUNTER 10–5
10.1.7. LINE-COUNTER 10–5
10.1.8. Subscripting 10–5
10.2. Environment Division 10–5
10.2.1. Input-Output Section 10–5
10.2.2. The FILE-CONTROL Paragraph 10–6
10.2.3. The File Control Entry 10–6
10.2.4. The I-O-CONTROL Paragraph 10–7
10.3. Data Division 10–7
10.3.1. File Section 10–8
10.3.2. Report Section 10–10
10.4. Procedure Division in the Report Writer Module 10–51
10.4.1. General Description 10–51
10.4.2. The CLOSE Statement 10–51
10.4.3. The GENERATE Statement 10–54
10.4.4. The INITIATE Statement 10–56
10.4.5. The OPEN Statement 10–57
10.4.6. The SUPPRESS Statement 10–59
10.4.7. The TERMINATE Statement 10–60
10.4.8. The USE AFTER STANDARD EXCEPTION PROCEDURE Statement 10–61
10.4.9. The USE BEFORE REPORTING Statement 10–62
10.5. Report Format on Files Other Than Printer Files 10–63
10.6. Examples 10–64
10.6.1. Program 1 10–64
10.6.2. Program 2 10–70

Section 11. Segmentation Module

11.1. Introduction 11–1
11.1.1. Level Characteristics 11–1
11.1.2. Scope 11–1
11.1.3. Organization 11–1
11.1.4. Segmentation Classification 11–3
11.1.5. Segmentation Control 11–3

11.2. Environment Division in the Segmentation Module 11–3
11.2.1. Configuration Section 11–3
11.2.2. The OBJECT-COMPUTER Paragraph 11–3
11.2.3. The SEGMENT-LIMIT Clause 11–4
11.3. Procedure Division in the Segmentation Module 11–5
11.3.1. General Description 11–5
11.3.2. Segment Numbers 11–5
11.3.3. Restrictions on Program Flow 11–6

Section 12. Intrinsic Functions

12.1. Language Concepts 12–3
12.1.1. Function Name 12–3
12.1.2. Value Returned by a Function 12–3
12.1.3. Function Identifier 12–3
12.2. Function Definition and Returned Value 12–3
12.3. Arguments 12–4
12.4. Types of Functions 12–5
12.5. UCS COBOL Intrinsic Functions 12–6
12.5.1. The ACOS Function 12–9
12.5.2. The ANNUITY Function 12–9
12.5.3. The ASIN Function 12–10
12.5.4. The ATAN Function 12–11
12.5.5. The CHAR Function 12–11
12.5.6. The COS Function 12–11
12.5.7. The CURRENT-DATE Function 12–12
12.5.8. The DATE-OF-INTEGER Function 12–13
12.5.9. The DAY-OF-INTEGER Function 12–14
12.5.10. The FACTORIAL Function 12–15
12.5.11. The INTEGER Function 12–15
12.5.12. The INTEGER-OF-DATE Function 12–16
12.5.13. The INTEGER-OF-DAY Function 12–16
12.5.14. The INTEGER-PART Function 12–17
12.5.15. The LENGTH Function 12–18
12.5.16. The LOG Function 12–19
12.5.17. The LOG10 Function 12–19
12.5.18. The LOWER-CASE Function 12–19
12.5.19. The MAX Function 12–20
12.5.20. The MEAN Function 12–20
12.5.21. The MEDIAN Function 12–21
12.5.22. The MIDRANGE Function 12–21
12.5.23. The MIN Function 12–22
12.5.24. The MOD Function 12–23
12.5.25. The NUMVAL Function 12–23
12.5.26. The NUMVAL-C Function 12–24
12.5.27. The ORD Function 12–25
12.5.28. The ORD-MAX Function 12–26
12.5.29. The ORD-MIN Function 12–26
12.5.30. The PRESENT-VALUE Function 12–27
12.5.31. The RANDOM Function 12–27
12.5.32. The RANGE Function 12–28

12.5.33. The REM Function 12–29
12.5.34. The REVERSE Function 12–29
12.5.35. The SIN Function 12–30
12.5.36. The SQRT Function 12–30
12.5.37. The STANDARD-DEVIATION Function 12–31
12.5.38. The SUM Function 12–32
12.5.39. The TAN Function 12–32
12.5.40. The UPPER-CASE Function 12–33
12.5.41. The VARIANCE Function 12–33
12.5.42. The WHEN-COMPILED Function 12–34

Appendix A. Reserved Words

Appendix B. Syntax Skeleton

B.1. General Description B–1
B.2. Identification Division B–1
B.3. Environment Division B–2
B.4. Data Division B–5
B.4.1. Data Description Entry B–8
B.4.2. Report Group Description Entry B–9
B.5. Procedure Division B–10
B.5.1. The ACCEPT Statement B–11
B.5.2. The ADD Statement B–11
B.5.3. The ALTER Statement B–12
B.5.4. The CALL Statement B–12
B.5.5. The CANCEL Statement B–12
B.5.6. The CLOSE Statement B–12
B.5.7. The COMPUTE Statement B–13
B.5.8. The CONTINUE Statement B–13
B.5.9. The DELETE Statement B–13
B.5.10. The DISPLAY Statement B–13
B.5.11. The DIVIDE Statement B–13
B.5.12. The ENTER Statement B–14
B.5.13. The EVALUATE Statement B–15
B.5.14. The EXIT Statement B–15
B.5.15. The EXIT PROGRAM Statement B–15
B.5.16. The GENERATE Statement B–15
B.5.17. The GOBACK Statement B–15
B.5.18. The GO TO Statement B–16
B.5.19. The IF Statement B–16
B.5.20. The INITIALIZE Statement B–16
B.5.21. The INITIATE Statement B–16
B.5.22. The INSPECT Statement B–16
B.5.23. The MERGE Statement B–17
B.5.24. The MOVE Statement B–18
B.5.25. The MULTIPLY Statement B–18
B.5.26. The OPEN Statement B–18
B.5.27. The PERFORM Statement B–19
B.5.28. The READ Statement B–20
B.5.29. The RELEASE Statement B–20

B.5.30. The RETURN Statement B–20
B.5.31. The REWRITE Statement B–21
B.5.32. The SEARCH Statement B–21
B.5.33. The SET Statement B–21
B.5.34. The SORT Statement B–22
B.5.35. The START Statement B–23
B.5.36. The STOP Statement B–23
B.5.37. The STRING Statement B–23
B.5.38. The SUBTRACT Statement B–23
B.5.39. The SUPPRESS Statement B–24
B.5.40. The TERMINATE Statement B–24
B.5.41. The UNSTRING Statement B–24
B.5.42. The USE Statement B–24
B.5.43. The USE BEFORE REPORTING Statement B–25
B.5.44. The WRITE Statement B–25
B.6. General Format for Nested Source Programs B–25
B.7. General Format for Nested–Source–Program B–26
B.8. General Format for a Sequence of Source Programs B–26
B.9. Conditions B–26
B.9.1. Relation Condition B–26
B.9.2. Class Condition B–26
B.9.3. Sign Condition B–27
B.9.4. *condition–name* Condition B–27
B.9.5. Switch–Status Condition B–27
B.9.6. Negated Condition B–27
B.9.7. Combined Condition B–27
B.9.8. Abbreviated Combined Relation Condition B–27
B.9.9. Data Pointer Relation Condition B–27
B.10. Miscellaneous Formats B–27
B.10.1. Qualification B–27
B.10.2. Subscripting B–28
B.10.3. Reference Modification B–28
B.10.4. Identifier B–28
B.10.5. Function Identifier B–29
B.11. The COPY Statement B–29
B.12. The REPLACE Statement B–29
B.13. Function Format B–30
B.14. DMS 2200 Skeletons B–30
B.14.1. Complete Entry Skeleton for Statements in the Data-Division B–31
B.14.2. Complete Entry Skeleton for Statements in the Procedure Division B–32
B.15. RDMS 2200 Skeletons B–36
B.15.1. Embedded SQL Skeletons B–36
B.15.2. ENTER MASM Skeleton B–37
B.16. UREP Skeletons B–37
B.16.1. Inserting SELECT Statement Information with INVOKE B–37
B.16.2. Inserting File Descriptions with INVOKE B–37
B.16.3. Inserting Data Descriptions with INVOKE B–37

Index 1

Figures

3–1. PICTURE Character Precedence Chart 3–38
3–2. Collating Sequence Example 3–78
3–3. Evaluation of the Hierarchical Level condition-1 AND condition-2 AND ... condition-n 3–88
3–4. Evaluation of the Hierarchical Level condition-1 OR condition-2 OR … condition-n 3–89
3–5. Evaluation of condition-1 OR condition-2 AND condition-3 3–90
3–6. Evaluation of (condition-1 OR NOT condition-2) AND condition-3 AND condition-4 3–91
3–7. Execution Flowchart 3–122

10–2. Body Group Presentation Rules 10–27
10–3. Report Footing Presentation Rules 10–34

Tables

2–1. Class and Category of Data Items 2–25

3–1. Internal Representation of Overpunched Operational Signs 3–29
3–2. Examples of Nonediting PICTURE Strings 3–39
3–3. Examples of Editing PICTURE Strings 3–40
3–4. Meaning and Allocation of Data Types by USAGE 3–50
3–5. Alignment of Data Types by USAGE 3–52
3–6. Combination of Symbols in Arithmetic Expressions 3–71
3–7. Combinations of Conditions, Logical Operators, and Parentheses 3–85
3–8. Arithmetic Statements and Expressions 3–100

4–1. Relationship of Categories of Files and the Formats of the CLOSE Statement 4–30
4–2. Availability of a File 4–34
4–3. Permissible Statements 4–34

5–1. Availability of a File 5–16
5–2. Permissible Statements 5–17

6–1. Availability of a File 6–20
6–2. Permissible Statements 6–21

10–1. Permissible Clause Combinations in Format 3 Entries 10–20
10–2. Page Heading Group Presentation Rules 10–25
10–3. Page Footing Presentation Rules 10–31
10–4. Relationship of Categories of Files and the Formats of the CLOSE Statements 10–52
10–5. Availability of a File 10–57

12–1. Table of Functions 12–6

Section 1
Introduction

The COBOL source language implemented by the Universal Compiling System (UCS) COBOL, commonly known as UCOB, includes all required features of the American National Standards Institute (ANSI) COBOL X3.23–1985 and of the Federal Information Processing Standards Publication 21–2 (FIPS PUB 21–2). In addition, it includes numerous compatibility features for migration from ASCII COBOL to UCS COBOL.

The UCS COBOL compiler uses the ASCII and ISO character sets in processing source text and object time character data. The compiler is fully reentrant and produces optimized reentrant code usable in multibanked, common-banked, or transaction program environments.

1.1. About this Manual

Purpose

This manual provides instruction in using the Universal Compiling System (UCS) COBOL, commonly known as UCOB, on OS 2200 systems. UCS COBOL generates code for execution in extended mode on 2200 Series systems.

Scope

UCS COBOL includes all required features of the American National Standards Institute (ANSI) COBOL X3.23–1985 and FIPS PUB 21–2 (Federal Information Processing Standards Publication 21–2). In addition, it includes numerous compatibility features for migration from ASCII COBOL to UCS COBOL.

Audience

This manual is intended for systems and applications programmers and analysts who are already familiar with the COBOL programming language.

Prerequisites

To use this manual effectively, you should have knowledge of the UCS COBOL programming language as well as familiarity with basic OS 2200 job control language concepts.

Notations Conventions

This manual uses the following conventions to illustrate COBOL syntax:

UPPERCASE represents COBOL keywords that are required in the syntax. Keywords must be spelled exactly as shown.

UPPERCASE in brackets represents keywords that you can optionally use to make your program more readable. Keywords must be spelled exactly as shown.

italic lowercase represents terms such as

- UCS COBOL words
- Literals
- PICTURE character strings
- Comment entries
- Complete syntactical entries that you supply

Where a term may be repeated in a general format, a number is appended to the term to identify that term for explanation or discussion (for example, identifier –1).

[ ]

indicate a choice of one or more of the enclosed options that can either be specified or omitted.

{ }

indicate a choice of one or more of the enclosed options. Unlike options enclosed in brackets, you must explicitly or implicitly specify one of the options enclosed in braces.

represents the omission of a word or words, generally to illustrate that portions of syntax can be repeated.

01, 02,...77 or level-number

represent specific level numbers that are required for the data description formats in which they appear. If a specific level is not identified, level-number is specified and the legal level numbers are explained in the rules for the format.

separators

include the comma, semicolon, and space. These separators are interchangeable within your source program; however, you must use the separator period when it is used in formats.

special character words

such as +, –, < , > , and =, must be specified when using portions of formats that contain them.

1.2. Documentation Updates

This document contains all the information that was available at the time of publication. Changes identified after release of this document are included in problem list entry (PLE) 19046851. To obtain a copy of the PLE, contact your Unisys representative or access the currently PLE from the Unisys Product Support website:

http://www.support.unisys.com/all/19046851

1.3. COBOL X3.23–1974,–1985 Differences Potentially Not Affecting Existing Programs

The COBOL 85 standard contains many new features that were not in COBOL 74 and should not have any impact on existing programs. These new features include new verbs as well as additional capabilities for existing verbs. The following is a list of changes of substance included in the COBOL 85 standard:

- Lowercase letters

  When the computer character set includes lowercase letters, they can be used in character strings. Except when used in nonnumeric literals, each is equivalent to the corresponding uppercase letter.

- Colon (:) character

  The COBOL character set has been expanded to include the colon (:) character that is used in reference modification.

- Punctuation characters

  The separators comma, semicolon, and space are interchangeable within a source program.

- User-defined words and system names

  The same COBOL word can be used as a system name and as a user-defined word within a source program; the context in which a COBOL word occurs determines what it is.

- Symbolic characters

  A symbolic character is a user-defined word that specifies a user-defined figurative constant.

- Nonnumeric literal

  A nonnumeric literal has an upper limit of 160 characters in length. The upper limit was 120 characters in the COBOL 74 standard.

- Figurative constant ZERO

  The figurative constant ZERO is allowed in arithmetic expressions.

- Uniqueness of reference

  A user-defined word need not be unique or be capable of being made unique unless referenced.

- Qualification

  An implementor must provide the capability to handle 50 levels of qualification. Five levels of qualification were required in the COBOL 74 standard.

- Subscripting

  A table can have up to seven dimensions. Up to 3 dimensions were allowed in the COBOL 74 standard.

- Relative subscripting

  Relative subscripting allows a subscript to be followed by the operator + or –, which is followed by an integer.

- Mixing subscripts and indexes

  Indexes and data-name subscripts can both be written in a single set of subscripts used to reference an individual occurrence of a multidimensional table.

- Reference modification

  Reference modification is a new method of referencing data by specifying a leftmost character and length for the data item.

- Sequence number

  The sequence number can contain any character in the computer's character set. In COBOL 74 the sequence number contained only digits.

- Data Division reference format

  The word following a level indicator, level number 01, or level number on the same line can begin in area A.

- End program header

  The end program header indicates the end of the named COBOL source program.

- Nested source programs

  Programs can be contained in other programs.

- INITIAL clause in PROGRAM-ID

  The INITIAL clause specifies a program whose state is initialized, whenever the program is called, to the same state as when that program was first called in the run unit.

- COMMON clause in PROGRAM-ID

  The COMMON clause specifies a program that, despite being directly contained within another program, can be called from any program directly or indirectly contained in that other program.

- Environment Division

  The Environment Division is optional.

- SPECIAL–NAMES paragraph

  If implementor name is a switch, condition name need not be a switch.

- SPECIAL–NAMES paragraph

  The reserved word IS has been made optional in the SPECIAL–NAMES paragraph to be consistent with the use of IS throughout the COBOL language.

- STANDARD–2 option

  The STANDARD–2 option within the ALPHABET clause of the SPECIAL–NAMES paragraph allows the specification of the ISO 7–bit character set for a character code set or collating sequence.

- ASSIGN clause

  A nonnumeric literal can be specified in the ASSIGN clause.

- OPTIONAL phrase

  The OPTIONAL phrase within the file control entry applies to sequential files, relative files and indexed files opened in the input, I–O, or extend mode.

- ORGANIZATION clause

  Within the ORGANIZATION clause of the file control entry, the words ORGANIZATION IS have been made optional.

- PADDING CHARACTER clause

  The PADDING CHARACTER clause in the file control entry specifies the character that is to be used for block padding on sequential files.

- RECORD DELIMITER clause

  The RECORD DELIMITER clause in the file control entry indicates the method of determining the length of a variable length record.

- I–O–CONTROL paragraph

  The order of clauses is immaterial in the I–O–CONTROL paragraph.

- Data Division

  The Data Division is optional.

- BLOCK CONTAINS clause

  Omission of the BLOCK CONTAINS clause is permitted if the number of records contained in a block is specified by the operating environment.

- CODE–SET clause

  The CODE–SET clause can be specified for all files with sequential organization.

- LABEL RECORDS clause

  The LABEL RECORDS clause is optional; if not specified, then the clause LABEL RECORDS ARE STANDARD is assumed.

- LINAGE clause

  Data names within the LINAGE clause can be qualified.

- EXTERNAL clause

  The EXTERNAL clause specifies that a data item or a file connector is external and can be accessed and processed by any program run unit.

- GLOBAL clause

  The GLOBAL clause specifies that a data name or a file name is a global name that is available to every program contained within the program that declares it.

- FILLER clause

  The use of the word FILLER is optional for data description entries. The word FILLER can appear in a data description entry containing a REDEFINES clause. The word FILLER can be used in a data description entry of a group item.

- OCCURS clause

  The data item specified in the DEPENDING ON phrase can have a zero value.

- PICTURE character string

  A PICTURE character string can be continued between coding lines.

- PICTURE clause

  The insertion character '.' (period) or ',' (comma) can be used as the last character of a PICTURE character string, provided it is immediately followed by the separator period terminating the data description entry.

- RECORD clause

  The VARYING phrase of the RECORD clause is used to specify variable length records. The DEPENDING phrase associated with the VARYING phrase specifies a data item containing the number of character positions in a record.

- REDEFINES clause

  The size of the item associated with the REDEFINES clause can be less than or equal to the size of the redefines item.

- SIGN clause

  Multiple SIGN clauses can be specified in the hierarchy of a data description entry.

- SIGN clause

  The SIGN clause is allowed in a report group description entry.

- USAGE clause

  BINARY and PACKED–DECIMAL are two new features of the USAGE clause.

- VALUE clause

  The VALUE clause can be specified in a data descriptions entry that contains an OCCURS clause. The VALUE clause can be specified in a data description entry that is subordinate to an entry containing an OCCURS clause.

- LINE NUMBER clause

  The integer 0 can be specified as the relative line number in the PLUS phrase of the LINE NUMBER clause.

- Procedure Division

  The Procedure Division is optional.

- Procedure Division header

  A Linkage Section item that redefines, or is subordinate to one that redefines, an item appearing in the Procedure Division header can be referenced in the Procedure Division.

- Scope terminators

  Scope terminators serve to delimit the scope of certain procedural statements.

- Relational operators

  The relational operator IS GREATER THAN OR EQUAL TO (>=) is equivalent to the relational operator IS NOT LESS THAN. The relational operator IS LESS THAN OR EQUAL TO (<=) is equivalent to the relational operator IS NOT GREATER THAN.

- Class conditions

  Class name is associated with a set of characters specified by the user in the CLASS clause within the SPECIAL–NAMES paragraph.

- DAY–OF–WEEK phrase of ACCEPT statement

  The DAY–OF–WEEK phrase of the ACCEPT statement provides access to an integer representing the day of week.

- ADD statement

  The word TO is an optional word in the format: ADD identifier/literal TO identifier/literal GIVING identifier.

- NOT ON SIZE ERROR phrase of ADD statement

  The NOT ON SIZE ERROR phrase provides the programmer with the capability to specify procedures to be executed when a size error condition does not exist for the ADD statement.

- CALL statement

  The BY CONTENT phrase indicates that the called program cannot change the value of a parameter in the CALL statement's USING phrase, but the called program can change the value of the corresponding data item in the called program's Procedure Division header.

- CALL statement

  The parameters passed in a CALL statement can be other than a 01– or 77–level data item. The parameters passed in a CALL statement can be subscripted or reference modified.

- ON EXCEPTION and NOT ON EXCEPTION phrases of CALL statement

  The ON EXCEPTION phrase of the CALL statement is equivalent to the ON OVERFLOW phrase of the CALL statement. The NOT ON EXCEPTION phrase provides the programmer with the capability to specify procedures to be executed when the program specified by the CALL statement has been made available for execution.

- REEL/UNIT phrase of the CLOSE statement

  The REEL/UNIT phrase of the CLOSE statement can be applied to a single reel/unit file and is specifically permitted for a report file.

- FOR REMOVAL phrase of the CLOSE statement

  The FOR REMOVAL phrase of the CLOSE statement is allowed for a sequential single reel/unit file.

- NOT ON SIZE ERROR phrase of COMPUTE statement

  The NOT ON SIZE ERROR phrase provides the programmer with the capability to specify procedures to be executed when a size error condition does not exist for the COMPUTE statement.

- CONTINUE statement

  The CONTINUE statement indicates that there is no executable statement present and causes an implicit transfer of control to the next executable statement.

- NOT INVALID KEY phrase of DELETE statement

  The NOT INVALID KEY phrase provides the programmer with the capability to specify procedures to be executed when an invalid key condition does not exist for the DELETE statement.

- DISPLAY statement

  The figurative constant ALL literal is permitted in the DISPLAY statement.

- NOT ON SIZE ERROR phrase of DIVIDE statement

  The NOT ON SIZE ERROR phrase provides the programmer with the capability to specify procedures to be executed when a size error condition does not exist for the DIVIDE statement.

- WITH NO ADVANCING phrase of the DISPLAY statement

  The WITH NO ADVANCING phrase of the DISPLAY statement provides interaction with a hardware device having vertical positioning.

- EVALUATE statement

  The EVALUATE statement describes a multibranch, multijoin structure in which multiple conditions are evaluated to determine the subsequent action of the object program.

- EXIT PROGRAM statement

  The EXIT PROGRAM statement need not be the only statement in a paragraph.

- GO TO DEPENDING statement

  The number of procedure names required in a GO TO DEPENDING statement has been reduced to one.

- IF statement

  The optional word THEN has been added to the general format of the IF statement.

- INITIALIZE statement

  The INITIALIZE statement provides the ability to set selected types of data to predetermined values.

- INSPECT statement

  Multiple occurrences of the BEFORE/AFTER phrase allow the TALLYING/REPLACING operation to be initiated after the beginning of the inspection of the data begins and/or terminated before the end of the inspection of the data ends.

- INSPECT statement.

  The ALL/LEADING adjective can be distributed over multiple occurrences of the identifier/literal and there can be multiple occurrences of the REPLACING CHARACTERS phrase.

- INSPECT CONVERTING statement

  The CONVERTING phrase provides a new variation of the INSPECT statement.

- MERGE statement

  Multiple file names are allowed in the GIVING phrase of the MERGE statement. A file named in a MERGE statement can contain variable length records. A file named in either the USING or GIVING phrase of a MERGE statement can be a relative file or an indexed file.

- MOVE statement

  A numeric edited data item can be moved to a numeric or numeric edited data item; thus de–editing takes place.

- NOT ON SIZE ERROR phrase of MULTIPLY statement

  The NOT ON SIZE ERROR phrase provides the programmer with the capability to specify procedures to be executed when an on size error condition does not exist for the MULTIPLY statement.

- EXTEND phrase of the OPEN statement

  The EXTEND phrase of the OPEN statement can be used with a relative file or an indexed file.

- PERFORM statement

  Procedure name can be omitted resulting in an in–line PERFORM of the imperative statements preceding the END–PERFORM phrase terminating the PERFORM statement.

- PERFORM statement

  The TEST AFTER phrase causes the condition to be tested after the specified set of statements has been executed. The TEST BEFORE phrase causes the condition to be tested before the specified set of statements is executed.

- PERFORM statement

  At least six AFTER phrases must be permitted in the VARYING phrase of the PERFORM statement.

- READ statement

  Variable length records are allowed when the READ statement has an INTO phrase. The next phrase is allowed in a READ statement referencing a file with sequential organization.

- NOT AT END phrase of READ statement

  The NOT AT END phrase provides the programmer with capability to specify procedures to be executed when the end condition does not exist for the READ statement.

- NOT INVALID KEY phrase of READ statement

  The NOT INVALID KEY phrase provides the programmer with the capability to specify procedures to be executed when an invalid key condition does not exist for the READ statement.

- REPLACE statement

  The REPLACE statement causes each occurrence of specified text in the source program to be replaced by the corresponding text specified in the REPLACE statement.

- RETURN statement

  Variable length records are allowed when the RETURN statement has an INTO phrase.

- NOT AT END phrase of RETURN statement

  The NOT AT END phrase provides the programmer with the capability to specify procedures to be executed when an “at end” condition does not exist for the RETURN statement.

- REWRITE statement

  A record of a different length can replace a record within either a relative or an indexed file.

- NOT INVALID KEY phrase of REWRITE statement

  The NOT INVALID KEY phrase provides the programmer with the capability to specify procedures to be executed when an invalid key condition does not exist for the REWRITE statement.

- SET statement

  Index names and identifiers can now be mixed in a series of operands preceding the word TO in a SET statement. Two new variations of the SET statement permit the setting of an external switch to be changed and to permit the value of a conditional variable to be changed.

- SORT statement

  Multiple file names are allowed in the GIVING phrase of the SORT statement. A file named in a SORT statement can contain variable length records. A file named in either the USING or GIVING phrase of a SORT statement can be a relative file or an indexed file.

The files names in the USING and GIVING phrases can reside on the same physical reel. If the DUPLICATES phrase is specified, records whose key values are identical remain in the same order as they were when they were input to the sort process after the sort process is completed.

- SORT and MERGE statements

  The input and output procedures of a SORT or MERGE statement can contain explicit transfers of control to points outside the input or output procedure. The remainder of the Procedure Division can contain transfers of control to points inside the input or output procedure. A paragraph name can be specified in the INPUT PROCEDURE phrase or the OUTPUT PROCEDURE phrase.

- NOT INVALID KEY phrase of START statement

  The NOT INVALID KEY phrase provides the programmer with the capability to specify procedures to be executed when an invalid key condition does not exist for the START statement.

- STRING statement

  The identifier in the INTO phrase of the STRING statement can be a group item.

- NOT ON OVERFLOW phrase of STRING statement

  The NOT ON OVERFLOW phrase provides the programmer with the capability to specify procedures to be executed when an overflow condition does not exist for the STRING statement.

- NOT ON SIZE ERROR phrase of the SUBTRACT statement

  The NOT ON SIZE ERROR phrase provides the programmer with the capability to specify procedures to be executed when a size error condition does not exist for the SUBTRACT statement.

- NOT ON OVERFLOW phrase of UNSTRING statement

  The NOT ON OVERFLOW phrase provides the programmer with the capability to specify procedures to be executed when an overflow condition does not exist for the UNSTRING statement.

- USE statement

  A USE AFTER EXCEPTION/ERROR declarative statement specifying the name of a file takes precedence over a declarative statement specifying the open mode of the file.

- USE statement

  The GLOBAL phrase specifies that the associated declarative procedures are invoked during the execution of any program contained within the program that includes the USE statement.

- USE BEFORE REPORTING statement

  The GLOBAL phrase specifies that the associated declarative procedures are invoked during the execution of any program contained within the program that includes the USE BEFORE REPORTING statement.

- NOT END–OF–PAGE phrase of WRITE statement

  The NOT END–OF–PAGE phrase provides the programmer with the capability to specify procedures to be executed when an end-of-page condition does not exist for the WRITE statement.

- NOT INVALID KEY phrase of WRITE statement

  The NOT INVALID KEY phrase provides the programmer with the capability to specify procedures to be executed when an invalid key condition does not exist for the WRITE statement.

1.4. COBOL X3.23–1974,–1985 Differences Potentially Affecting Existing Programs

The COBOL 85 standard contains some new features that were not in COBOL 74 that could have an impact on existing programs. For a complete list and discussion of changes that have a potential impact on migration from ASCII COBOL, refer to Volume 2, Appendix E.

Section 2
Programming Considerations

2.1. General

0055CS COBOL offers many features that allow you to obtain a necessary function without programming the function in detail. For information on programming in detail, see the appropriate section for specific information. This section provides general information about various language concepts.

2.2. Files

A file is a collection of records that can be placed into or retrieved from a storage medium. You can choose the file organization and the file processing method and sequence. Although the file organization and processing method are restricted for sequential media, no such restrictions exist for mass storage media.

When describing the capabilities of UCS COBOL programs to manipulate files, the following conventions are used:

- The term “file name” means the user-defined word used in the UCS COBOL source program to refer to a file.
- The terms “file referred to by file name” and “file” mean the physical file regardless of the file name used in the COBOL program.
- The term “file connector” means the entity containing information concerning the file.
- All accesses to physical files occur through file connectors.

2.2.1. File Attributes

A file has several attributes that apply to the file at the time it is created and cannot be changed during the lifetime of the file. The primary attribute is the organization of the file, which describes its logical structure. There are three organizations: sequential, relative, and indexed. Other fixed attributes of the file provided by the UCS COBOL program are prime record key, alternate record keys, code set, the minimum and maximum logical record size, the record type (fixed or variable), the collating sequence of the keys for indexed files, the blocking factor, the padding character, and the record delimiter.

A logical record is the unit of data that is retrieved from or stored in a file. The number of records that can exist in a file is limited only by the capacity of the storage media.

There are two types of records: fixed length and variable length. When a file is created, it is declared to contain either fixed-length or variable-length records.

In either case, the content of the record area does not reflect any control information added to the record on the physical storage medium (such as record length headers), nor does the length of the record you use reflect these additions.

2.2.2. File Processing

A file can be processed by performing operations on individual records or on the file as a unit. Unusual conditions that occur during processing are communicated back to the program.

Record Operations

The ACCESS MODE clause of the file control entry specifies the manner in which the object program operates on records within a file. The access mode can be sequential, random, or dynamic.

For files that are organized as relative or indexed, any of the three access modes can be used to access the file regardless of the access mode used to create the file. A file with sequential organization can be accessed in sequential mode only.

Open Mode

The open mode of the file is related to the actions to be performed on records in the file and is specified in the OPEN statement. The open modes and purposes are:

- Input to retrieve records
- Output to place records into a file
- Extend to append records to an existing file
- I-O to retrieve and update records

When the open mode is input, a file can be accessed by the READ statement. The START statement can also be used for files organized as indexed or relative that is in sequential or dynamic access modes.

When the open mode is output, the records are placed into the file by issuing WRITE, GENERATE, or TERMINATE statements.

When the open mode is extended, new records are added to the logical end of a file by issuing WRITE statements.

Only mass storage files can be referred to in the open I–O mode. The additional capabilities of mass storage devices permit updating in place, therefore READ and REWRITE statements can always be used. A mass storage file can be updated in the same manner as a file on a sequential medium by transcribing the entire file into another file (perhaps in a separate area of mass storage) using READ and WRITE statements. However, it is sometimes more efficient to update a mass storage file in place. This mass storage file maintenance technique uses the REWRITE statement to return to their previous locations on the storage medium only those records that have changed. READ and REWRITE statements are the only operations allowed while updating in place sequentially organized files.

However, for indexed or relative organized files, the following additional functions can be applied: Use the START statement in sequential or dynamic access mode to alter the sequence of record retrieval; use the DELETE statement with any access mode to remove a record logically from a file; and use the WRITE statement in random or dynamic access mode to insert a new record into the file.

LINAGE

Use the LINAGE clause when specifying an output report. It facilitates definition of a logical page and the positioning within that logical page of top and bottom margins and a footing area. Use of the LINAGE clause implicitly defines an associated special register, the LINAGE-COUNTER that acts as a pointer to a line within the page body.

Exception Handling

During the execution of any input or output operation, unusual conditions can arise that preclude normal completion of the operation. There are three methods by which these conditions are communicated to the object program: status keys, exception declaratives, and optional phrases associated with the imperative statement.

I–O Status

I–O status is a conceptual entity used in this document to facilitate exact specification of the status of the execution of an input-output operation. The setting of I-O status is affected only by the CLOSE, DELETE, OPEN, READ, REWRITE, START, and WRITE statements. The I-O status value for a given file is made available to the program through the data name specified in the FILE STATUS clause of the file control entry for that file. The I-O status value is placed into this data item during the execution of the input-output statement and prior to the execution of any imperative statement associated with that input-output statement or prior to the execution of any exception declarative.

Exception Declaratives

A USE AFTER EXCEPTION procedure, when one is specified for the file, is executed whenever an input or output condition arises that results in an unsuccessful input-output operation. However, the exception declarative is not executed if the condition is invalid key and the INVALID KEY phrase is specified or if the condition is at end and the AT END phrase is specified.

Optional Phrases

The INVALID KEY phrase can be associated with the DELETE, READ, REWRITE, START, or WRITE statements. Some of the conditions that give rise to an invalid key condition are where:

- A requested key does not exist in the file (DELETE, READ, or START statements).
- A key is already in a file and duplicates are not allowed (WRITE statement).
- A key does not exist in the file or when it was not the last key read (REWRITE statement).

If the invalid key condition occurs during the execution of a statement for which the INVALID KEY phrase has been specified, the statement identified by that INVALID KEY phrase is executed.

The AT END phrase can be associated with a READ statement. The at–end condition occurs in a sequentially accessed file when:

- No next logical record exists in the file.
- The number of significant digits in the relative record number is larger than the size of the relative key data item.
- An optional file is not present.
- READ statement is attempted and the at–end condition already exists.

If the at-end condition occurs during the execution of a statement for which the AT END phrase has been specified, the statement identified by that AT END phrase is executed.

Rerun

The rerun feature of COBOL provides a facility for checkpoint/restart. That is, executing a rerun takes a snapshot (checkpoint) of the program status and stores the information. It is then possible to restart the program from a rerun point. The RERUN clause protects the user from having to start a program over from the beginning if a hardware failure occurs while the job is running.

There are two basic parts to the RERUN clause. You must designate a medium to receive the data and a criterion from which the frequency of checkpoints can be determined. The receiving medium can be specified by designating a separate hardware device. The determination of frequency of the dump is made on the basis of a number of records of a particular file having been processed.

2.3. Table Handling

Tables of data are common components of business data processing problems. Although the repeating items that make up a table could be described otherwise by a series of separate data description entries all having the same level number and all subordinate to the same group item, there are two reasons why this approach is not satisfactory. First, from a documentation standpoint, the underlying homogeneity of the items would not be readily apparent. Second, the problem of making an individual element of such a table available would be severe when there is a decision as to which element is to be made available at object time.

Tables of data items are defined in COBOL by including the OCCURS clause in their data description entries. This clause specifies that the item is to be repeated as many times as stated. The item is a table element, and its name and description apply to each repetition or occurrence. Since each occurrence of a table element does not have a unique data name assigned to it, reference to a desired occurrence can be made only by specifying the data name of the table element together with the occurrence number of the desired table element. The occurrence number is known as a subscript. The number of occurrences of a table element can be specified as fixed or variable.

2.3.1. Table Definition

To define a one-dimensional table, you use an OCCURS clause as part of the data description of the table element, but the OCCURS clause must not appear in the description of group items that contains the table element. The following example shows a one-dimensional table defined by the item TABLE-ELEMENT.

Example

```
01  TABLE-1.
   02  TABLE-ELEMENT OCCURS 20 TIMES.
       03    DOG . . .
       03    FOX . . .
```

Example

In this example, TABLE–ELEMENT defines a one–dimensional table, but DOG does not since there is an OCCURS clause in the description of the group item (TABLE-ELEMENT) that contains DOG.

```
02  TABLE-1.
    03  TABLE-ELEMENT OCCURS 20 TIMES.
        04  DOG OCCURS 5 TIMES.
            05  EASY . . .
            05  FOX . . .
```

In both examples, the complete set of occurrences of TABLE-ELEMENT has been assigned the name TABLE-1. However, it is not necessary to give a group name to the table unless it is desired to refer to the complete table as a group item.

None of the three one-dimensional tables that appear in the following two examples has a group name.

Example

```
01  ABLE.
    02  BAKER . . .
    02  CHARLIE OCCURS 20 TIMES . . .
    02  DOG . . .
```

Example

```
01  ABLE.
    02  BAKER OCCURS 20 TIMES . . .
    02  CHARLIE . . .
    02  DOG OCCURS 5 TIMES . . .
```

Defining a one-dimensional table within each occurrence of an element of another one-dimensional table gives rise to a two-dimensional table. To define a two-dimensional table, an OCCURS clause must appear in the data description of the element of the table, and in the description of only one group item that contains that table element.

Therefore, in the following example, DOG is an element of a two-dimensional table; it occurs five times within each element of the item BAKER, which itself occurs 20 times. BAKER is an element of a one-dimensional table.

Example

```
02  BAKER OCCURS 20 TIMES . . .
    03  CHARLIE . . .
    03  DOG OCCURS 5 TIMES . . .
```

In general, to define an *n*–dimensional table, the OCCURS clause must appear in the data description of the element of the table and in the descriptions of (*n*–1) group items that contain the element.

2.3.2. Initial Values of Tables

In the working storage section, initial values of elements within tables are specified in one of the following ways:

- The table can be described as a series of separate data description entries all subordinate to the same group item, each of which specifies the value of an element, or part of an element, of the table. In defining the record and its elements, any data description clause (USAGE, PICTURE, etc.) can be used to complete the definition, where required. The hierarchical structure of the table is then shown by use of the REDEFINES entry and its associated subordinate entries. The subordinate entries, following the REDEFINES entry, which is repeated due to OCCURS clauses, must not contain VALUE clauses.

- All the dimensions of a table can be initialized by associating the VALUE clause with the description of the entry defining the entire table. The lower-level entries show the hierarchical structure of the table; lower-level entries must not contain VALUE clauses.
- The value of selected table elements can be specified using VALUE clauses.

2.3.3. Reference to Table Items

Whenever you refer to a table element or a condition name associated with a table element, the reference must indicate which occurrence of the element is intended, except in a SEARCH statement. For access to a one-dimensional table, the occurrence number of the desired element provides complete information. For tables of more than one dimension, an occurrence number must be supplied for each dimension of the table accessed. In the last example of Section 2.3.1, a reference to the fourth BAKER or the fourth CHARLIE would be complete, whereas a reference to the fourth DOG would not be complete. For example, to refer to DOG, which is an element of a two-dimensional table, you must refer to the fourth DOG within the fifth BAKER.

2.3.4. Subscripting

Occurrence numbers are specified by appending one or more subscripts to the data name.

The subscript can be represented either by an integer, a data name that refers to an integer numeric elementary item, or an index name associated with the table. A data name or index name can be followed by either the operator + or the operator - and an integer, which is used as an increment or decrement, respectively. It is permissible to mix integers, data names, and index names.

The subscripts, enclosed in parentheses are written immediately following any qualification for the name of the table element. The number of subscripts in such a reference must equal the number of dimensions in the table whose element is being referenced. That is, there must be a subscript for each OCCURS clause in the hierarchy containing the data name including the data name itself.

When more than one subscript is required, they are written in order of successively less inclusive dimensions of the data organization. If a multidimensional table is thought of as a series of nested tables and the most inclusive or outermost table in the nest is considered to be the major table with the innermost or least inclusive table being the minor table, the subscripts are written from left to right in the order major, intermediate, and minor.

A reference to an item must not be subscripted if the item is not a table element or an item or condition name within a table element.

The lowest permissible occurrence number is 1. The highest permissible occurrence number in any particular case is the maximum number of occurrences of the item as specified in the OCCURS clause.

Subscripting Using Integers or Data Names

When an integer or data name is used to represent a subscript, it can be used to refer to items within different tables. These tables need not have elements of the same size. The same integer or data name can appear as the only subscript with one item and as one of two or more subscripts with another item.

Subscripting Using Index Names

In order to facilitate such operations as table searching and manipulating specific items, a technique called indexing is available. To use this technique, you assign one or more index names to an item whose data description entry contains an OCCURS clause. An index associated with an index name acts as a subscript, and its value corresponds to an occurrence number for the item to which the index name is associated.

The INDEXED BY phrase, by which the index name is identified and associated with its table, is an optional part of the OCCURS clause. There is no separate entry to describe the index associated with the index name since its definition is completely hardware oriented. At object time, the contents of the index correspond to an occurrence number for that specific dimension of the table with which the index is associated.

When a reference is made to a table element, or to an item within a table element, and the name of the item is followed by its related index name or names in parentheses, then each occurrence number required to complete the reference is obtained from the respective index name. The index name acts as a subscript whose value is used in any table reference that specifies indexing.

The initial value of an index at object time is undefined, and the index must be initialized before use. The initial value of an index is assigned using the PERFORM statement with the VARYING phrase, the SEARCH statement with the ALL phrase, or the SET statement.

The use of an integer or data name as a subscript referencing a table element or an item within a table element does not cause the alteration of any index associated with that table.

An index name can be used to reference only the table to which it is associated through the INDEXED BY phrase.

Data that is arranged in the form of a table is often searched. The SEARCH statement provides facilities for producing serial and binary searches. It is used to search a table for a table element that satisfies a specific condition and to adjust the value of the associated index to indicate that table element.

Relative indexing is an additional option for making references to a table element or to an item within a table element. When the name of a table element is followed by a subscript of the form (index name + or - integer), the occurrence number required to complete the reference is the same as if index name were set up or down by integer through the SET statement before the reference. The use of relative indexing does not cause the object program to alter the value of the index.

The value of an index can be made accessible to an object program by storing the value in an index data item. Index data items are described in the program by a data description entry containing a USAGE IS INDEX clause. The index value is moved to the index data item by the execution of a SET statement.

Subscripting Example

Assume the following data definition:

```
02  XCOUNTER . . .

02  BAKER OCCURS 20 TIMES INDEXED BY BAKER-INDEX . . .
    03  CHARLIE . . .
    03  DOG OCCURS 5 TIMES . . .
        04  EASY . . .
        88  MAX VALUE IS . . .
        04  FOX . . .
            05  GEORGE OCCURS 10 TIMES . . .
                06  HARRY . . .
                06  JIM . . .
```

References to BAKER and CHARLIE require only one subscript; references to DOG, EASY, MAX, and FOX require two; and references to GEORGE, HARRY, and JIM require three.

To illustrate the requirement of order from major to minor, HARRY (18, 2, 7) means the HARRY in the seventh GEORGE, in the second DOG, in the eighteenth BAKER.

Mixing arithmetic expressions and index names is illustrated by HARRY (BAKER-INDEX, 4, XCOUNTER + 5).

2.4. Language Concepts

The language considerations and rules specified in this section apply to the highest level of UCS COBOL. When a particular level of a module does not allow all of these language concepts, the restrictions is pointed out in the section describing that language element. Note that restrictions contained in one module might affect other modules. For example, qualification is not allowed in level 1 of the Nucleus; therefore, any module that is combined with level 1 of the Nucleus would have the same restriction.

2.4.1. Notation Used in General Formats

A general format is the specific arrangement of the elements of a clause or a statement. A clause or a statement consists of elements as defined in the following subsections. Throughout this manual, a format is shown adjacent to information defining the clause or statement. When more than one specific arrangement is permitted, the general format is separated into numbered formats. Clauses must be written in the sequence given in the general formats. (Clauses that are optional must appear in the sequence shown if they are used.) In certain cases, stated explicitly in the rules associated with a given format, the clauses can appear in sequences other than that shown. Applications, requirements, or restrictions are shown as rules (see Section 2.4.2).

Elements

The elements that make up a clause or a statement consist of uppercase words, lowercase words, level numbers, brackets, braces, connectives, and special characters.

Words

All underlined uppercase words are called keywords and are required when the functions of which they are a part are used. Uppercase words that are not underlined are optional to the user and need not be written in the source program. Uppercase words, whether underlined or not, must be spelled correctly.

Lowercase words (in italics), in a general format, are generic terms used to represent UCS COBOL words, literals, PICTURE character strings, comment entries, or a complete syntactical entry that must be supplied by the user. Where generic terms are repeated in a general format, a number appendage to the term serves to identify that term for explanation or discussion.

Level Numbers

When specific level numbers appear in data description entry formats, those specific level numbers are required when such entries are used in an UCS COBOL program. In this manual, the form 01, 02, . . . , 09 is used to indicate level numbers 1 through 9.

Brackets, Braces, and Choice Indicators

When brackets, [ ], enclose a portion of a general format, one of the options contained within the brackets can be explicitly specified or that portion of the general format can be omitted.

When braces, { }, enclose a portion of a general format, one of the options contained within the braces *must* be either explicitly specified or implicitly selected. If one of the options contains only reserved words that are not keywords, then that option is the default option. This option is implicitly selected unless one of the options is explicitly specified.

When choice indicators {| |} enclose a portion of a general format, one or more of the unique options contained within the choice indicators *must* be specified, but a single option can be specified only once.

Options are indicated in a general format or a portion of a general format by vertically stacking alternative possibilities, by a series of brackets, braces, or choice indicators or by a combination of both. An option is selected by specifying one of the possibilities from a stack of alternative possibilities or by specifying a unique combination of possibilities from a series of brackets, braces, or choice indicators.

The Ellipsis

In text, other than general formats, the ellipsis (...) shows the omission of a word or words when such omission does not impair comprehension.

In general formats, the ellipsis represents the position at which the user elects repetition of a portion of a format. The portion of the format that can be repeated is determined as follows:

Given an ellipsis (...) in a clause or statement format, find the right bracket (]) or right brace (}) immediately to the left of the ellipsis. Find the logically matching left bracket ([) or left brace ({). The ellipsis applies to the words between the pair of delimiters.

Format Punctuation

The separators comma and semicolon can be used anywhere the separator space is used in the formats (see Section 2.4.5). In the source program, these separators are interchangeable. The separator period, when used in the formats, has the status of a required word.

Use of Special Character Words in Formats

The special character words +, -, >, <, =, <=, and >=, when appearing in formats, although not underlined, are required when such portions of the formats are used.

2.4.2. Rules

This subsection describes syntax rules and general rules you must follow when programming in COBOL.

Syntax Rules

Syntax rules define or clarify the order in which words or elements are arranged to form larger elements such as phrases, clauses, or statements. Syntax rules also either can impose restrictions on individual words or elements or can relax restrictions implied by words or elements.

These rules are used to define or clarify how the statement must be written; that is, the order of the elements of the statement and restrictions or amplifications of what each element can represent.

General Rules

A general rule defines or clarifies the meaning or relationship of meanings of an element or set of elements. It is used to define or clarify the semantics of the statement and the effect that it has on either execution or compilation.

2.4.3. Character Set

The most basic and indivisible unit of the language is the character. The set of characters used to form UCS COBOL character strings and separators includes the letters of the alphabet, digits, and special characters. The character set consists of the characters as defined under COBOL character set in the Glossary. In the case of nonnumeric literals, comment entries, and comment lines, the character set is expanded to include the computer's entire character set. The characters allowable in each type of character string and as separators are defined in Section 2.4.5.

2.4.4. Language Structure

The individual characters of the language are concatenated to form character strings and separators. A separator can be concatenated with another separator or with a character string. A character string can only be concatenated with a separator. The concatenation of character strings and separators forms the text of a source program.

2.4.5. Separators

A separator is a character or two contiguous characters formed according to the following rules:

- The punctuation character space is a separator. Anywhere a space is used as a separator or part of a separator, more than one space can be used. All spaces immediately following the separators comma, semicolon, and period are considered part of that separator and are not considered to be the separator space.
- Except when the comma is used in a PICTURE character string, the punctuation characters comma and semicolon, immediately followed by a space, are separators that can be used anywhere the separator space is used. They can be used to improve program readability.
- The punctuation character period, when followed by a space, is a separator. It must be used to indicate the end of a sentence, or as shown in formats.
- The punctuation characters right and left parentheses are separators. Parentheses can appear only in balanced pairs of left and right parentheses delimiting subscripts, reference modifiers, function arguments, arithmetic expressions, or conditions.
- The punctuation character quotation mark is a separator. An opening quotation mark must be immediately preceded by a space or left parenthesis; a closing quotation mark, when paired with an opening quotation mark, must be immediately followed by one of the separators space, comma, semicolon, period, or right parenthesis.

- Pseudotext delimiters are separators. An opening pseudotext delimiter must be immediately preceded by a space; a closing pseudotext delimiter must be immediately followed by one of the separators space, comma, semicolon, or period. Pseudotext delimiters can appear only in balanced pairs delimiting pseudo–text.
- The punctuation character colon is a separator and is required when shown in the general formats.
- The separator space can immediately precede all separators except:
  - As specified by reference format rules (see Section 2.4.6).
  - The separator closing quotation mark. In this case, a preceding space is considered as part of the nonnumeric literal and not as a separator. The opening pseudo–text delimiter, where the preceding space is required.
- The separator space can immediately follow any separator except the opening quotation mark. In this case, a following space is considered as part of the nonnumeric literal and not as a separator.

Any punctuation character that appears as part of the specification of a PICTURE character string or numeric literal is not considered as a punctuation character, but rather as a symbol used in the specification of that PICTURE character string or numeric literal. PICTURE character strings are delimited only by the separators space, comma, semicolon, or period.

The rules established for the formation of separators do not apply to the characters that comprise the contents of nonnumeric literals, comment entries, or comment lines.

2.4.6. Character Strings

A character string is a character or a sequence of contiguous characters that forms a COBOL word, a literal, a PICTURE character string, or a comment entry. A character string is delimited by separators.

2.4.7. COBOL Words

A COBOL word is a character string of not more than 30 characters that form a user-defined word, a system name, a reserved word, or a function name. Each character of a COBOL word is selected from the set of letters, digits, and the hyphen. The hyphen cannot appear as the first or last character. Each lowercase letter is considered equivalent to its corresponding uppercase letter. Within a source program, reserved words and user-defined words form disjoint sets; reserved words and system names form disjoint sets; reserved words and function names form disjoint sets; function names, system names, and user-defined words form intersecting sets.

The same COBOL word can be used as a function name, as a system name, and as a user-defined word within a source program. The class of a specific occurrence of this COBOL word is determined by the context of the clause or phrase in which it occurs.

User-Defined Words

A user-defined word is a COBOL word that the user must supply to satisfy the format of a clause or statement. Each character of a user-defined word is selected from the set of characters A through Z, 0 through 9, and –, except that - cannot appear as the first or last character.

The types of user-defined words are:

- Alphabet name
- Class name
- Condition name
- Data name
- File name
- Index name
- Level number
- Library name
- Mnemonic name
- Paragraph name
- Program name
- Record name
- Report name
- Routine name
- Section name
- Segment number
- Symbolic character
- Text name

Within a given source program, but not in any contained program, the user-defined words are grouped into the following disjoint sets:

- Alphabet names
- Class names
- Condition names, data names, and record names
- File names
- Index names
- Library names
- Mnemonic names
- Paragraph names

- Program names
- Report names
- Routine names
- Section names
- Symbolic characters
- Text names

All user-defined words, except segment numbers and level numbers, can belong to only one of these disjoint sets. Further, all user-defined words within a given disjoint set must be unique, except as specified in the rules for uniqueness of reference. (See Section 2.4.20.)

With the exception of section names, paragraph names, segment numbers, and level numbers, all user-defined words must contain at least one alphabetic character. Level numbers and segment numbers need not be unique. A given specification of a level number or segment number can be identical to any other level number.

- Condition names

  A condition name is a name assigned to a specific value, set of values, or range of values, within a complete set of values that a data item can assume. The data item itself is called a conditional variable.

  Condition names can be defined in the Data Division or in the SPECIAL-NAMES paragraph of the Environment Division where a condition name must be assigned to the on status or off status, or both, of implementor-defined switches.

  A condition name is used in conditions as an abbreviation for the relation condition. This relation condition defines the associated conditional variable as equal to one of the set of values to which that condition name is assigned. A condition name is also used in a SET statement, indicating that the associated value is to be moved to the conditional variable.

- Mnemonic name

  A mnemonic name assigns a user-defined word to an implementor name. These associations are established in the SPECIAL-NAMES paragraph of the Environment Division. (See Section 3.4.5.)

- Paragraph name

  A paragraph name is a word that names a paragraph in the Procedure Division. Paragraph names are equivalent only if they are composed of the same sequence of the same number of digits and characters.

- Section name

  A section name is a word that names a section in the Procedure Division. Section names are equivalent only if they are composed of the same sequence of the same number of digits and characters.

- Other user-defined names

  See the Glossary for definitions of all other types of user-defined words.

System Names

A system name is a COBOL word that is used to communicate with the operating environment. Each character used in the formation of a system name must be selected from the set of characters A through Z, 0 through 9, and minus (-), except that - cannot appear as the first or last character.

There are three types of system names:

- Computer names
- Implementor names
- Language names

These three types of system names form disjoint sets. A given system name can belong to one and only one of them.

Computer name and implementor name are individually defined in the Glossary.

Reserved Words

A reserved word is a COBOL word from a specified list of words that can be used in COBOL source programs, but which must not appear in the programs as user-defined words or system names. Reserved words can be used only as specified in the general formats. (See Appendix A.)

- Reserved words satisfy the following conditions:
- Reserved words do not begin with the characters 0 through 9, X, Y, or Z, except for the reserved words ZERO, ZEROES, and ZEROS.
- Reserved words do not contain only one alphabetic character.
- Reserved words do not start with one or two characters followed by "-" except for the reserved words I-O, I-O-CONTROL, and reserved words that begin with 'B-' or 'DB-'.
- Reserved words do not contain two or more contiguous hyphens.

There are three types of reserved words:

- Required words
- Optional words
- Special-purpose words

Required Words

A required word is a word whose presence is required when the format in which the word appears is used in a source program.

Required words are of two types:

- Keywords. Within each format, such words are uppercase and underlined.
- Special-character words. These are the arithmetic operators and relation characters.

Optional Words

Within each format, uppercase words that are not underlined are called optional words and can be specified at the user's option with no effect on the semantics of the format.

Special–Purpose Words

There are two types of special-purpose words:

- Special registers

  Certain reserved words are used to name and refer to special registers. Special registers are certain compiler-generated storage areas primarily used to store information produced in conjunction with the use of specific COBOL features. Unless noted otherwise in these specifications, one special register of each type is allocated for each program. In the general formats of this specification, a special register can be used, unless otherwise restricted, wherever data name or identifier is specified, provided that the special register is the same category as the data name or identifier. If qualification is allowed, special registers can be qualified as necessary to provide uniqueness. (See Section 2.4.20)

  There are three special registers:

  – LINAGE-COUNTER (see Section 4.2.8)

  – LINE-COUNTER (see Section 10.1.5)

  – PAGE-COUNTER (see Section 10.1.4)

- Figurative constants

  Certain reserved words are used to name and refer to specific constant values. These reserved words are specified in Section 2.4.8.

Function Names

A function name is a word that is one of a specified list of words that can be used in COBOL source programs. The same word, in a different context, can appear in a program as a user-defined word or a system name. (See Section 2.4.20).

2.4.8. Literals

A literal is a character string whose value is implied by an ordered set of characters, of which the literal is composed, or by specification of a reserved word that refers to a figurative constant. Every literal belongs to one of two types: nonnumeric or numeric.

Nonnumeric Literals

A nonnumeric literal is a character string delimited on both ends by the separator quotation mark or the separator apostrophe. The nonnumeric literal must be delimited on each end by the same character. The length of a nonnumeric literal applies to its representation in the object program.

A nonnumeric literal can be 1 through 160 characters in length.

General Format

```
"{character-1}... "
```

Syntax Rules

1. *Character-1* can be any character in the computer character set.
2. If *character-1* is to represent the quotation mark, and the literal is delimited by quotation marks, two contiguous quotation mark characters must be used to represent a single occurrence of that character.
3. If *character-1* is to represent an apostrophe, and the literal is delimited by an apostrophe, two contiguous apostrophe characters must be used to represent a single occurrence of that character.

General Rules

1. The value of a nonnumeric literal in the object program is the value represented by *character-1.*
2. The separator quotation mark or apostrophe that delimits the nonnumeric literal is not part of the value of the nonnumeric literal.
3. All nonnumeric literals are of category alphanumeric.

Numeric Literals

There are two types of numeric literals: fixed-point and floating-point.

Fixed–point literal

A fixed–point literal is a character string whose characters are selected from the digits 0 through 9, the plus sign, the minus sign, and the decimal point. Fixed-point literals of 1 through 18 digits in length are allowed. The rules for the formation of fixed-point literals are:

- A literal must contain at least one digit.
- A literal must not contain more than one sign character. If a sign is used, it must appear as the leftmost character of the literal. If the literal is unsigned, the literal is nonnegative.
- A literal must not contain more than one decimal point. The decimal point is treated as an assumed decimal point and can appear anywhere within the literal except as the rightmost character. If the literal contains no decimal point, the literal is an integer.

  If a literal conforms to the rules for the formation of numeric literals but is enclosed in quotation marks, it is a nonnumeric literal and is treated as such by the compiler.
- The value of a numeric literal is the algebraic quantity represented by the characters in the numeric literal. Every numeric literal is category numeric. (See Section 3.5.11.) The size of a numeric literal in standard data format characters is equal to the number of digits specified by the user.

Floating–point literal

A floating-point literal is a character string whose characters are selected from the numerals 0 through 9, the plus sign, the minus sign, the decimal point, and the character E.

A floating-point literal must have the following format:

```
[±] mantissa E [±] exponent
```

where

±

are the plus or minus signs (optional).

mantissa

consists of 1 to 18 digits with a required decimal point; the decimal point can appear in any position. If the mantissa is signed, the sign must be the leftmost character in the literal string. If the mantissa is unsigned, the literal value is positive.

E

must immediately follow the mantissa, and the exponent must appear immediately to the right of the character E.

exponent

consists of one to three digits and an optional sign. If the sign is present, it must be immediately to the right of the character E and immediately to the left of the digits. Unsigned exponents are positive. The exponent cannot contain a decimal point.

The largest number that can be represented by a floating-point literal for double precision is 0.89884656743115795 X 10^{308} and the smallest that can be represented is

0.27813423231340018 x 10^{-308} . For single precision, the largest number is 0.17014118 x 10^{39} and the smallest number is 0.14693680 x 10^{-38}.

The value of a floating-point literal is the algebraic quantity product of the mantissa and 10 raised to the power given by the exponent.

Example

```
+1.5E-2 = 1.5 x 10-2
```

Figurative Constant Values

Figurative constant values are generated by the compiler and are referred to through the use of the reserved words that follow. These words must not be bounded by quotation marks when used as figurative constants. The singular and plural forms of figurative constants are equivalent and can be used interchangeably.

The figurative constant values and the reserved words used to refer to them are

1. [ALL] ZERO, [ALL] ZEROS, [ALL] ZEROES

   Represent the numeric value 0, or one or more of the character 0 from the computer's character set.

2. [ALL] SPACE, [ALL] SPACES

   Represent one or more of the character spaces from the computer’s character set.

3. [ALL] HIGH–VALUE, [ALL] HIGH–VALUES

   Except in the SPECIAL–NAMES paragraph, represent one or more of the characters that have the highest ordinal position in the program collating sequence.

4. [ALL] LOW–VALUE, [ALL] LOW–VALUES

   Except in the SPECIAL–NAMES paragraph, represent one or more of the characters that have the lowest ordinal position in the program collating sequence.

5. [ALL] QUOTE, [ALL] QUOTES

   Represent one or more of the character quotation mark (") or apostrophe (’), depending upon the keyword option used. (See Table 4.2.) The word QUOTE or QUOTES cannot be used in place of a quotation mark in a source program to bind a nonnumeric literal. Therefore, QUOTE ABD QUOTE is incorrect as a way of stating the nonnumeric literal “ABD”.

6. ALL *literal*

   Represents all or part of the string generated by successive concatenations of the characters comprising the literal. The literal must be a nonnumeric literal. The literal must not be a figurative constant.

7. [ALL] *symbolic-character*

   Represents one or more of the character specified as the value of this symbolic character in the SYMBOLIC CHARACTERS clause of the SPECIAL–NAMES paragraph (see Section 3.4.5).

8. [ALL] *NULL*, [ALL] *NULLS*

   Represents the value used to indicate that the data items defined with USAGE POINTER or the ADDRESS OF phrase do not contain a valid address. NULL can be used only where explicitly allowed in the syntax formats. NULL has a value of two words of binary zero.

When a figurative constant represents a string of one or more characters, the length of the string is determined by the compiler from context according to the following rules:

1. When a figurative constant is specified in a VALUE clause or when a figurative constant is associated with another data item, as when the figurative constant is moved to or compared with another data item, the string of characters specified by the figurative constant is repeated character by character on the right until the size of the resultant string is greater than or equal to the number of character positions in the associated data item. The resultant string is then truncated from the right until it is equal to the number of character positions in the associated data item. This is done prior to, and independent of, the application of any JUSTIFIED clause that can be associated with the data item.
2. When a figurative constant, other than ALL *literal,* is not associated with another data item, as when the figurative constant appears in a DISPLAY, STOP, STRING, or UNSTRING statement, the length of the string is one character.
3. When the figurative constant ALL *literal* is not associated with another data item, the length of the string is the length of the literal.

A figurative constant can be used wherever *literal* appears in a format with the following exceptions:

1. If the literal is restricted to a numeric literal, the only figurative constant permitted is ZERO (ZEROS, ZEROES).
2. Associating the figurative constant ALL literal where the length of the literal is greater than one with a data item that is numeric or numeric edited is an obsolete feature in this revision of American National Standard COBOL. This obsolete feature is to be deleted from the next revision of American National Standard COBOL.
3. When a figurative constant other than ALL literal is used, the word ALL is redundant and is used for readability only.

Except in the SPECIAL-NAMES paragraph, when the figurative constants HIGH-VALUE(S) or LOW-VALUE(S) are used in the source program, the actual characters associated with each figurative constant depend upon the program collating sequence specified. (See Section 3.4.4 and Section 3.4.5.)

Each reserved word referring to a figurative constant value is a distinct character string with the exception of the constructs using the word ALL, such as ALL *literal,* ALL SPACES, and so forth, which are composed of two distinct character strings.

2.4.9. PICTURE Character Strings

A PICTURE character string is a combination of the currency symbol and certain characters in the UCS COBOL character set. See Section 3.5.11 for an explanation of the PICTURE character string and for the rules that govern its use.

Any punctuation character that appears as part of the specification of a PICTURE character string is not considered as a punctuation character, but rather as a symbol used in the specification of that PICTURE character string.

2.4.10. Comment Entries

A comment entry is an entry in the Identification Division that can be any combination of characters from the computer's character set. Comment entry is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted from the next revision of American National Standard COBOL.

2.4.11. Logical Record Concept

Separate clauses or phrases are used to separate the logical characteristics of data from the physical characteristics of the data storage media. See Section 2.4.12 through Section 2.4.14 for a discussion of the characteristics of files.

2.4.12. Physical Aspects of a File

The physical aspects of a file are the data as it appears on the input or output media and include such features as

- The grouping of logical records within the physical limitations of the file medium.
- The means by which the file can be identified.

2.4.13. Conceptual Characteristics of a File

The conceptual characteristics of a file are the explicit definition of each logical entity within the file itself. In a COBOL program, the input or output statements refer to one logical record.

It is important to distinguish between a physical record and a logical record. A COBOL logical record is a group of related information, uniquely identifiable and treated as a unit.

A physical record is a physical unit of information whose size and recording mode is convenient to a particular computer for the storage of data on an input or output device. The size of a physical record is hardware-dependent and bears no direct relationship to the size of the file of information contained on a device.

A logical record can be contained within a single physical unit, or several logical records can be contained within a single physical unit, or a logical record can require more than one physical unit to contain it. There are several source language methods available for describing the relationship of logical records and physical units. When a permissible relationship has been established, control of the accessibility of logical records as related to the physical unit must be provided by the interaction of the object program on the hardware and software system. In this manual, a reference to records means to logical records, unless the term “physical record” is specifically used.

The concept of a logical record is not restricted to file data but is carried over into the definition of working storage. Therefore, working storage is grouped into logical records and defined by a series of record description entries.

When a logical record is transferred to or from a physical unit, any translation required by the presence of a CODE–SET clause is accomplished. Padding characters are added or deleted as necessary. None of the clauses used to describe the data in the logical record has any effect on this transfer.

2.4.14. Record Concepts

The record description consists of a set of data description entries that describe the characteristics of a particular record. Each data description entry consists of a level number followed by a data name, if required, followed by a series of independent clauses, as required.

2.4.15. Concept of Levels

A level concept is inherent in the structure of a logical record. This concept arises from the need to specify subdivisions of a record for the purpose of data reference. Once a subdivision has been specified, it can be further subdivided to permit more detailed data referral.

The most basic subdivisions of a record, that is, those not further subdivided, are called elementary items. Consequently, a record is said to consist of a sequence of elementary items, or the record itself can be an elementary item.

In order to refer to a set of elementary items, the elementary items are combined into groups. Each group consists of a named sequence of one or more elementary items. Groups, in turn, can be combined into groups of two or more groups, etc. Therefore, an elementary item can belong to more than one group.

2.4.16. Level Numbers

A system of level numbers shows the organization of elementary items and group items. Since records are the most inclusive data items, level numbers for records start at 01. Less inclusive data items are assigned higher (not necessarily successive) level numbers not greater in value than 49. There are special level numbers (66, 77, and 88) that are exceptions to this rule. Separate entries are written in the source program for each level number used.

A group includes all groups and elementary items following it until a level number less than or equal to the level number of that group is encountered. All items that are immediately subordinate, to a given group item must be described using identical level numbers. These numbers must be greater than the level number used to describe that group item.

Three types of entries exist for which there is no true concept of level:

- Entries that specify elementary items or groups introduced by a RENAMES clause. Entries describing items by means of RENAMES clauses for the purpose of regrouping data items have been assigned the special level number 66.
- Entries that specify noncontiguous working-storage and linkage data items. Entries that specify noncontiguous data items that are not subdivisions of other items and are not themselves subdivided have been assigned the special level number 77.
- Entries that specify condition names. Entries that specify condition names to be associated with particular values of a conditional variable have been assigned the special level number 88.

2.4.17. Concept of Classes of Data

The five categories of data items (see Section 3.5.11) are grouped into three classes: alphabetic, numeric, and alphanumeric. For alphabetic and numeric, the classes and categories are synonymous. The class alphanumeric includes the categories of alphanumeric edited, numeric edited and alphanumeric (without editing). Every elementary item belongs to one of the classes and, further, to one of the categories. The class of a group item is treated at object time as alphanumeric regardless of the class of elementary items subordinate to that group item. Every data item that is a function is an elementary item. It belongs to one of the alphanumeric or numeric categories and to the corresponding class. The category of each function is determined by the definition of the function. This definition is made in these specifications. (See Volume 2.)

Table 2–1 depicts the relationship of the class and categories of data items.

Table 2–1. Class and Category of Data Items

Class	Category
Alphabetic	Alphabetic
Numeric	Numeric
Alphanumeric	Numeric Edited Alphanumeric Edited Alphanumeric

2.4.18. Algebraic Signs

Algebraic signs fall into two categories: operational signs that are associated with signed numeric data items and signed numeric literals to indicate their algebraic properties; and editing signs, which appear, for example, on edited reports to identify the sign of the item.

The SIGN clause permits you to explicitly state the location of the operational sign. The clause is optional; if it is not used; operational signs are represented as if SIGN IS TRAILING without the SEPARATE option was specified (see Section 3.5.14).

Editing signs are inserted into a data item through the use of the sign control symbols of the PICTURE clause.

2.4.19. Standard Alignment Rules

The standard rules for positioning data within an elementary item depend on the category of the receiving item. These rules are:

1. If the receiving data item is described as numeric
   a. The data is aligned by decimal point and is moved to the receiving digit positions with zero-fill or truncation on either end, as required.
   b. When an assumed decimal point is not explicitly specified, the data item is treated as if it has an assumed decimal point immediately following its rightmost digit and is aligned as in rule 1a.
2. If the receiving data item is a numeric edited data item, the data moved to the edited data item is aligned by decimal point with zero-fill or truncation at either end as required within the receiving character positions of the data item, except where editing requirements cause replacement of the leading zeros.
3. If the receiving data item is alphanumeric (other than a numeric edited data item), alphanumeric edited, or alphabetic, the sending data is moved to the receiving character positions and aligned at the leftmost character position in the data item with space-fill or truncation to the right, as required.

If the JUSTIFIED clause is specified for the receiving item, these standard rules are modified as described in Section 3.5.8.

2.4.20. Uniqueness of Reference

Every user-defined name in an UCS COBOL program is assigned to name a resource (for example, data item) that is to be used in solving a data processing problem. (See Section 2.4.7) In order to use a resource, a statement in an UCS COBOL program must contain a reference that uniquely identifies that resource. In order to ensure uniqueness of reference, a user-defined name can be qualified, subscripted, or reference modified as described in the following paragraphs.

When the same name has been assigned in separate programs to two or more occurrences of a resource of a given type, and when qualification by itself does not allow the reference in one of those programs to differentiate between the two identically named resources, then certain conventions that limit the scope of names apply. These conventions ensure that the resource identified is that described in the program containing the reference. (See Section 7.2.3)

When the resource named is a function, the function definition can require the user to specify in the reference to the function a value or set of values for one or more parameters that determine the value of the function for that particular reference.

This is accomplished through the specification of arguments as described in the following paragraphs.

Unless otherwise specified by the rules for a statement, any subscripting, reference modification, and function identifiers are evaluated only once as the first operation of the execution of that statement.

Qualification

Every user-defined name explicitly referred to in a COBOL source program must be uniquely referable because

- No other name has the identical spelling (including hyphens).
- It is unique within the context of a REDEFINES clause (see Section 3.5.12).
- The name exists within a hierarchy of names such that reference to the name can be made unique by mentioning one or more of the higher level names in the hierarchy.

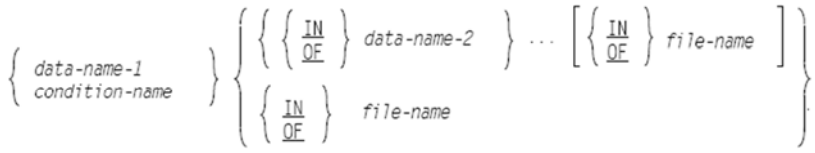
  These higher level names are called qualifiers and this process that specifies uniqueness is called qualification. Identical user-defined names can appear in a source program; however, uniqueness must then be established through qualification for each user-defined name explicitly referenced, except in the case of redefinition. All available qualifiers need not be specified so long as uniqueness is established. Reserved words naming the special registers require qualification to provide uniqueness of reference whenever a source program would result in more than one occurrence of any of these special registers. A paragraph name or section name appearing in a program cannot be referred to from any other program.
- A program is contained within a program or contains another program (see Section 7.2.3).

The data name must not be the same as the name of an external record and as the name of any other external data item described in any program contained within or containing the program that describes the external data record. The data name must not be the same as the name of an item possessing the global attribute and as the name of any other data item described in the program that describes the global data item. When a program is contained within a program or contains another program, specific conventions apply. (See Section 7.2.3).

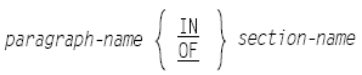
In the hierarchy of qualification for data names defined in the Data Division, names associated with a level indicator are the most significant, then those names associated with level number 01 (record names), then names associated with level number 02, 03, ..., 49.

The general formats for qualification are

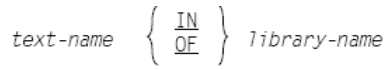
Format 1



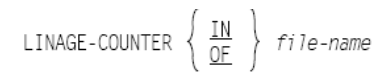
Format 2



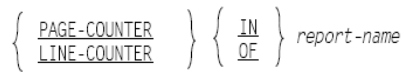
Format 3



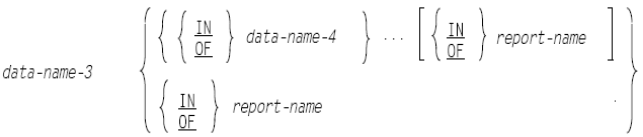
Format 4



Format 5



Format 6



The rules for qualification are:

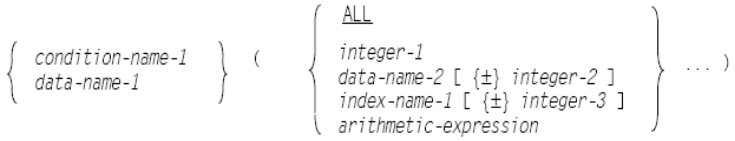
1. For each nonunique user-defined name that is explicitly referred to, uniqueness must be established through a sequence of qualifiers that precludes any ambiguity of reference.
2. A name can be qualified even though it does not need qualification. If there is more than one combination of qualifiers that ensures uniqueness, any such set can be used.
3. IN and OF are logically equivalent.
4. In format 1, each qualifier must be the name associated with a level indicator, the name of a group item to which the item being qualified is subordinate, or the name of the conditional variable with which the condition–name being qualified is associated. Qualifiers are specified in the order of successively more inclusive levels in the hierarchy.
5. In format 1, *data–name–1* or *data–name–2* can be a record name but only record names defined in the Data Division can be qualified.
6. If explicitly referred to, a paragraph name must not be duplicated within a section. When a paragraph name is qualified by a section name, the word SECTION must not appear. A paragraph name need not be qualified when referred to from within the same section. A paragraph name or section name appearing in a program cannot be referred to from any other program, except when the paragraph or section name appears on the WITH ENTRY POINTS clause of the Procedure Division header.
7. If more than one UCS COBOL library is available to the compiler during compilation, *text–name* must be qualified each time it is referred to unless the desired text can be found using the library search sequence (see Table 4.2).
8. LINAGE-COUNTER must be qualified each time it is referred to if more than one file description entry containing a LINAGE clause has been specified in the source program.

9. LINE-COUNTER must be qualified each time it is referred to in the Procedure Division if more than one report description entry is specified in the source program. In the Report Section, an unqualified reference to LINE-COUNTER is qualified implicitly by the report in whose report description entry the reference is made. Whenever the LINE-COUNTER of a different report is referenced, LINE-COUNTER must be qualified explicitly by the report name associated with the different report.
10. PAGE-COUNTER must be qualified each time it is referred to in the Procedure Division if more than one report description entry is specified in the source program. In the Report Section, an unqualified reference to the PAGE-COUNTER is qualified implicitly by the name of the report in whose report description entry the reference is made. Whenever the PAGE-COUNTER of a different report is referred to, PAGE-COUNTER must be qualified explicitly by the report name associated with the different report.

Subscripting

Subscripts are used when reference is made to an individual element within a table of like elements that have not been assigned individual data names (see Section 3.5.10).

General Format



Syntax Rules

1. The data description entry containing *data-name-1* or the data name associated with *condition-name-1* must contain an OCCURS clause or must be subordinate to a data description entry that contains an OCCURS clause.
2. Except as defined in syntax rule 4, when a reference is made to a table element, the number of subscripts must equal the number of OCCURS clauses in the description of the table element being referred to. When more than one subscript is required, the subscripts are written in the order of successively less inclusive dimensions of the table.
3. *Index-name-1* must correspond to a data description entry in the hierarchy of the table being referred to that contains an INDEXED BY phrase specifying that index name.
4. Each table element reference must be subscripted except when such reference appears
   a. As the subject of a SEARCH statement
   b. In a REDEFINES clause
   c. In the KEY IS phrase of an OCCURS clause
   d. As an argument in a function identifier when the definition of the referenced function specifically allows such an argument

5. *Data-name-2* can be qualified and must be a numeric elementary item representing an integer.
6. *Integer-1* can be signed and, if signed, must be positive.
7. The subscript ALL can be used only in a function reference. (See Section 12.)
8. If an arithmetic expression is used as a subscript, it must not be enclosed by parentheses or begin with a unary plus or minus.

General Rules

1. The value of the subscript must be a positive integer. The lowest possible occurrence number represented by a subscript is one. The first element of any given dimension of a table is referred to by an occurrence number of 1. Each successive element within that dimension of the table is referred to by occurrence numbers of 2, 3, ... The highest permissible occurrence number for any given dimension of the table is the maximum number of occurrences of the item as specified in the associated OCCURS clause.
2. The value of the index referred to by *index-name-1* corresponds to the occurrence number of an element in the associated table.
3. The value of the index referred to by *index-name-1* must be initialized before it is used as a subscript. An index can be given an initial value by either a PERFORM statement with the VARYING phrase, a SEARCH statement with the ALL phrase, or a SET statement. An index can be modified only by the PERFORM, SEARCH, and SET statements.
4. If *integer-2* or *integer-3* are specified, the value of the subscript is determined by incrementing or decrementing one of these values:
   a. The occurrence number represented by the value of the index *index-name-1.*
   b. The value of the data item *data-name-2*.

When the + operator is used, the appropriate value is incremented by the value *integer-2* or *integer-3* to get the subscript. When the - operator is used, the appropriate value is decremented by *integer-2 integer-3* to get the subscript.

Function Identifier

A function identifier is a syntactically correct combination of character strings and separators that uniquely references a resource available at object time. This is done to determine the value of a data item or of a condition.

General Format

```
FUNCTION function-name-1 [ ( { argument -1 }. . . )] [ reference-modifier ]
```

Syntax Rules

1. *Argument-1* must be an identifier, a literal, or an arithmetic expression. The definition of each function contains specific rules for governing the number, class, and category of *argument-1*. (See Section 12.)
2. *Reference-modifier* can be specified only for alphanumeric category functions.

3. A function identifier that references an alphanumeric function can be specified anywhere in the general formats that an 'identifier' is permitted, and where the rules associated with the general formats do not specifically prohibit reference to functions. The following exceptions apply:
   a. As a receiving operand of any statement.
   b. Where the rules associated with the general formats require the data item being referenced to have particular characteristics (such as class and category, usage, size, sign, and permissible values). The evaluation of the function according to its definition and the particular arguments specified would not have these characteristics.
4. A function identifier that references an integer or numeric function can be used only in an arithmetic expression.

General Rules

1. The function definition determines the class and other characteristics of the function being referenced. (See Section 12).
2. At the time reference is made to a function, its arguments are evaluated individually in the order specified in the list of arguments, from left to right. An argument being evaluated can itself be a function-identifier or can be an expression containing function identifiers. The function referenced in evaluating an argument can be the same function as that for which the argument is specified.

Reference Modifier

Reference modification defines a data item by specifying a leftmost character and length for the data item.

General Format

```
{ data-name-1                                                    }
{ FUNCTION function-name-1  [ ( { argument-1 } ... ) ]          }
  ( leftmost-character-position :  [ length ] )
```

Note: *Data-name-1 and FUNCTION function-name-1 (argument-1) are shown in the above format to provide context, and are not part of the reference modifier.*

Syntax Rules

1. *Data-name-1* must refer to a data item whose usage is DISPLAY or DISP–I18N.
2. *Leftmost-character-position* and *length* must be arithmetic expressions.
3. Unless otherwise specified, reference modification is allowed anywhere an identifier referring to a data item of the class alphanumeric is permitted.
4. *Data-name-1* can be qualified or subscripted.
5. The function referenced by *function-name-1* must be an alphanumeric function.

General Rules

1. Each character of a data item referred to by *data-name-1* or a function identifier is assigned an ordinal number incrementing by one from the leftmost position to the rightmost position. The leftmost position is assigned the ordinal number one. If the data description entry for *data-name-1* contains a SIGN IS SEPARATE clause, the sign position is assigned an ordinal number within that data item.
2. If the data item referred to by *data-name-1* is described as numeric, numeric edited, alphabetic, or alphanumeric edited, it is operated upon for reference modification as if it were redefined as an alphanumeric data item of the same size as the data item referred to by *data-name-1*.
3. Reference modification for an operand is evaluated as follows:
   a. If subscripting is specified for the operand, the reference modification is evaluated immediately after evaluation of subscripts.
   b. If subscripting is not specified for the operand, the reference modification is evaluated at the time subscripting would be evaluated if subscripts had been specified.
   c. If reference modification is specified in a function reference, the reference modification is evaluated immediately after evaluation of the function.
4. Reference modification creates a unique data item that is a subset of the data item referred to by *data-name-1* or *function-name-1*. This unique data item is defined as follows:
   a. The evaluation of *leftmost-character-position* specifies the ordinal position of the leftmost character of the unique data item in relation to the leftmost character of the data item referred to by *data-name-1* or *function-name-1*. Evaluation of *leftmost-character-position* must result in a positive nonzero integer less than or equal to the number of characters in the data item referred to by *data-name-1* or *function-name-1*.
   b. The evaluation of length specifies the size of the data item to be used in the operation. The evaluation of length must result in a positive nonzero integer. The sum of *leftmost-character-position* and *length* minus the value 1 must be less than or equal to the number of characters in the data item referred to by *data-name-1* or *function-name-1*. If *length* is not specified, the unique data item extends from and includes the character identified by *leftmost-character-position* up to and including the rightmost character of the data item referred to by *data-name-1* or *function-name-1*.
5. The unique data item is considered an elementary data item without the JUSTIFIED clause. When a function is referenced, it has the class and category of alphanumeric. When *data-name-1* is specified, the unique data item has the same class and category as that defined for the data item referred to by *data-name-1* except that the categories numeric, numeric edited, and alphanumeric edited are considered class and category alphanumeric.

Identifier

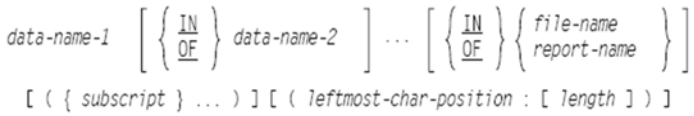
An identifier is a syntactically correct sequence of character strings and separators used to reference data uniquely.

When a data item other than a function is being referenced, "identifier" is a term used to reflect that a data name that, if not unique to a program, must be followed by a syntactically correct combination of qualifiers, subscripts, or reference modifiers necessary for uniqueness of reference. (See Section 7.2.3)

Format 1

```
function- identifier -1
```

Format 2



Condition Name

If explicitly referred to, a condition name must be unique or be made unique through qualification, subscripting, or both except when the scope of name conventions by themselves ensures unique references.

If qualification is used to make a condition name unique, the associated conditional variable can be used as the first qualifier. The hierarchy of names associated with the conditional variable itself is also available to make the condition name unique.

If references to a conditional variable require subscripting, references to any of its condition names also require the same combination of subscripting.

The format and restrictions on the combined use of qualification and subscripting of condition names are exactly those of "identifier" except that *data-name-1* is replaced by *condition-name-1.*

In the general formats that appear throughout this manual, *condition-name* refers to a condition name qualified, or subscripted, as necessary.

2.4.21. Explicit and Implicit Specifications

There are four types of explicit and implicit specifications that occur in COBOL source programs:

1. Explicit and implicit Procedure Division references
2. Explicit and implicit transfers of control
3. Explicit and implicit attributes
4. Explicit and implicit scope terminators

Procedure Division References

A UCS COBOL source program can refer to data items either explicitly or implicitly in Procedure Division statements. An explicit reference occurs when the name of the referenced item is written in a Procedure Division statement or when the name of the referenced item is copied into the Procedure Division by the processing of a COPY statement. An implicit reference occurs when the item is referenced by a Procedure Division statement without the name of the referenced item being written in the source statement. An implicit reference also occurs during the execution of a PERFORM statement when the index or data item referenced by the index name or identifier specified in the VARYING, AFTER, or UNTIL phrase is initialized, modified, or evaluated by the control mechanism associated with that PERFORM statement. Such an implicit reference occurs only if the data item contributes to the execution of the statement.

Transfers of Control

The mechanism that controls program flow transfers control from statement to statement in the sequence in which they were written in the source program, unless an explicit transfer of control overrides this sequence or there is no next executable statement to which control can be passed. The transfer of control from statement to statement occurs without the writing of an explicit Procedure Division statement and is, therefore, an implicit transfer of control.

UCS COBOL provides both explicit and implicit means of altering the implicit control transfer mechanism.

In addition to the implicit transfer of control between consecutive statements, implicit transfer of control also occurs when the normal flow is altered without the execution of a procedure branching statement.

UCS COBOL provides the following types of implicit control flow alterations that override the statement-to-statement transfers of control:

- If a paragraph is being executed under control of another UCS COBOL statement (for example, PERFORM, USE, SORT, and MERGE) and the paragraph is the last paragraph in the range of the controlling statement, then an implied transfer of control occurs from the last statement in the paragraph to the control mechanism of the last executed controlling statement. Furthermore, if a paragraph is being executed under the control of a PERFORM statement that causes iterative execution and that paragraph is the first paragraph in the range of that PERFORM statement, an implicit transfer of control occurs between the control mechanism associated with that PERFORM statement and the first statement in that paragraph for each iterative execution of the paragraph.
- When a SORT or MERGE statement is executed, an implicit transfer of control occurs to any associated input or output procedures.
- When any COBOL statement is executed that result in the execution of a declarative section, an implicit transfer of control to the declarative section occurs. Note that another implicit transfer of control occurs after execution of the declarative section, as described in the first item above.

An explicit transfer of control consists of an alteration of the implicit control transfer mechanism by the execution of a procedure branching or conditional statement. An explicit transfer of control can be caused only by the execution of a procedure branching or a conditional statement. The execution of the procedure branching statement ALTER does not, in itself, constitute an explicit transfer of control, but affects the explicit transfer of control that occurs when the associated GO TO statement is executed.

The procedure branching statement EXIT PROGRAM causes an explicit transfer of control only when the statement is executed in a called program.

In this manual, the term "next executable statement" is used to refer to the next COBOL statement to which control is transferred according to the rules just described and the rules associated with each language element.

There is no next executable statement when the program contains no Procedure Division or one of the following:

- The last statement in a declarative section when the paragraph in which it appears is not being executed under the control of some other COBOL statement.
- The last statement in a declarative section when the statement is in the range of an active PERFORM statement executed in a different section, and this last statement of the declarative section is not also the last statement of the procedure that is the exit of the active PERFORM statement.
- The last statement in a program when the paragraph in which it appears is not being executed under the control of some other COBOL statement in that program.
- A STOP RUN statement or EXIT PROGRAM statement that transfers control outside the COBOL program.
- The end program header.

When there is no next executable statement and control is not transferred outside the COBOL program, the program flow of control is undefined unless the program execution is in the nondeclarative procedures portion of the program under control of a CALL statement, in which case an implicit EXIT PROGRAM statement is executed.

Attributes

Attributes can be implicitly or explicitly specified. Any attribute that has been explicitly specified is called an explicit attribute. If an attribute has not been specified explicitly, the attribute takes on the default specification. Such an attribute is known as an implicit attribute.

For example, the usage of a data item need not be specified, in which case a data items usage is DISPLAY.

Scope Terminators

Scope terminators serve to delimit the scope of certain Procedure Division statements. (See Section 2.5.4) Scope terminators are of two types: explicit and implicit.

The explicit scope terminators are

```
END-ADD          END-IF           END-SEARCH
END-CALL         END-MULTIPLY     END-START
END-COMPUTE      END-PERFORM      END-STRING
END-DELETE       END-READ         END-SUBTRACT
END-DIVIDE       END-RETURN       END-UNSTRING
END-EVALUATE     END-REWRITE      END-WRITE
```

The implicit scope terminators are

- At the end of any sentence, the separator period that terminates the scope of all previous statements not yet terminated.
- The scope of any unterminated statement that is contained within another statement is terminated by the next phrase of the containing statement. Examples of such phrases are ELSE, WHEN, and NOT AT END.

When statements are nested within other statements that allow optional conditional phrases, any optional conditional phrase encountered is considered to be the next phase of the nearest preceding unterminated statement with which that phrase is permitted to be associated but with which no such phrase has already been associated. An unterminated statement is one that has not been previously terminated, either explicitly or implicitly.

2.4.22. External Switch

An external switch is a hardware or software device you define and name that is used to indicate that one of two alternate states exists. These alternate states are referred to as the on status and the off status of the associated external switch.

The status of an external switch can be interrogated by testing condition names associated with that switch. The association of a condition name with an external switch and the association of a user-specified mnemonic name with the implementor name that names an external switch is established in the SPECIAL-NAMES paragraph of the Environment Division. (See Section 3.4.5.)

You define the scope (program, run unit, etc.) of each external switch and any facility external to UCS COBOL that can be used to modify the status of an external switch. For example, if the scope of an external switch is the run unit, each implementor name that names such an external switch refers to only one such switch, the status of which is available to each object program functioning within that run unit.

The status of certain switches can be altered by the SET statement. (See Section 3.6.23.)

2.5. Source Program Construction

A UCS COBOL source program is a syntactically correct set of UCS COBOL statements.

With the exception of COPY and REPLACE statements and the end program header, the statements, entries, paragraphs, and sections of a UCS COBOL source program are grouped into four divisions that are sequenced in the following order:

1. Identification Division
2. Environment Division
3. Data Division
4. Procedure Division

The end of a UCS COBOL source program is indicated by either the end program header, if specified, or by the absence of additional source program lines.

The following gives the general format and order of presentation of the entries and statements that constitute a UCS COBOL source program.

General Format

```
identification-division

[environment-division]

[data-division]

[procedure-division]

[source-program] . . .

[end-program-header]
```

2.5.1. Identification Division

The Identification Division identifies the program. In addition, you can include the date the program is written, the date of the compilation of the source program, and other identification information.

Organization

Paragraph headers identify the type of information contained in the paragraph. The name of the program must be given in the first paragraph, which is the PROGRAM-ID paragraph. The other is paragraphs are optional and can be included in this division at the user's choice, in the order of presentation shown in Section 2.5.1.

Structure

The following general format of the paragraphs in the Identification Division defines the order of presentation in the source program:

```
IDENTIFICATION DIVISION.

PROGRAM-ID.program-identification-entry

[AUTHOR.[ comment-entry]...]

[INSTALLATION. [ comment-entry]...]

[DATE-WRITTEN.[comment-entry]...]

[DATE-COMPILED. [ comment-entry]... ]

[SECURITY. [ comment-entry]...]
```

2.5.2. Environment Division

The Environment Division specifies a standard method of expressing those aspects of a data processing problem that depend on the physical characteristics of a specific computer. This division allows specification of the configuration of the compiling computer and the object computer. In addition, information relating to input-output control, special hardware characteristics, and control techniques can be given.

Organization

Two sections make up the Environment Division: the Configuration Section and the Input-Output Section.

The Configuration Section deals with the characteristics of the source computer and the object computer. This section is divided into three paragraphs:

- The SOURCE-COMPUTER paragraph, which describes the computer configuration on which the source program is compiled
- The OBJECT-COMPUTER paragraph, which describes the computer configuration on which the object program produced by the compiler is to be run
- The SPECIAL-NAMES paragraph, which provides a means for specifying the currency sign, choosing the decimal point, specifying symbolic characters, relating implementor names to user-specified mnemonic names, relating alphabet names to character sets or collating sequences, and relating class names to sets of characters.

The Input-Output Section deals with the information needed to control transmission and handling of data between external media and the object program. This section is divided into two paragraphs: the FILE-CONTROL paragraph, which names and associates the files with external media; and the I-O-CONTROL paragraph, which defines special control techniques to be used in the object program.

Structure

The general format of the sections and paragraphs in the Environment Division defines the order of presentation in the source program.

```
ENVIRONMENT DIVISION.

[CONFIGURATION SECTION.

[SOURCE-COMPUTER. source-computer-entry] ]

[OBJECT-COMPUTER. [object-computer-entry] ]

[SPECIAL-NAMES. [ special-name-entry] ]

[INPUT-OUTPUT SECTION.

FILE-CONTROL.{ file-control-entry}...

[I-O-CONTROL.[ input-output-control-entry] ] ]
```

2.5.3. Data Division

The Data Division describes the data that the object program is to accept as input, to manipulate, to create, or to produce as output.

Organization

The Data Division is subdivided into the following sections: File, Working-Storage, Common-Storage, Linkage, and Report Sections.

The File Section defines the structure of data files. Each file is defined by a file description entry and one or more record descriptions, or by a file description entry and one or more report description entries. Record descriptions are written immediately following the file description entry. Report description entries appear in a separate section of the Data Division, the Report Section.

The Working-Storage Section describes records and noncontiguous data items that are not part of external data files but are developed and processed internally. It also describes data items whose values are assigned in the source program.

The Common-Storage Section describes records and noncontiguous data items that are to be shared by all programs that define a common Common-Storage Section.

The Linkage Section appears in the called program and describes data items that are to be referred to by the calling program and the called program. Its structure is the same as the Working-Storage Section.

The Report Section describes the content and format of reports that are to be generated.

Structure

The following general format of the sections in the Data Division defines the order of their presentation in the source program:

```
DATA DIVISION.
[ FILE SECTION.
  [ file-description-entry [ record-description-entry ] ...
    sort-merge-file-description-entry { record-description-entry } ... ] ... ]
[ WORKING-STORAGE SECTION.
  [ 77-level-description-entry
    record-description-entry ] ... ]
[ COMMON-STORAGE SECTION.
  [ 77-level-description-entry
    record-description-entry ] ... ]
[ LINKAGE SECTION.
  [ 77-level-description-entry
    record-description-entry ] ... ]
[ REPORT SECTION.
  [ report-description-entry { report-group-description-entry } ... ] ... ]
```

2.5.4. Procedure Division

This division can contain declaratives and nondeclarative procedures.

Declaratives

Declarative sections must be grouped at the beginning of the Procedure Division preceded by the keyword DECLARATIVES and followed by the keywords END DECLARATIVES. (See Section 4.5.6, 5.5.8, 6.5.8, 10.4.9, and Volume 2.)

Procedures

A procedure is composed of a paragraph, a group of successive paragraphs, a section, or a group of successive sections within the Procedure Division. If one paragraph is in a section, all paragraphs must be in sections. A procedure name is a word used to refer to a paragraph or section in the source program in which it occurs. It consists of a paragraph name, which can be qualified, or a section name.

A section consists of a section header followed by zero, one, or more successive paragraphs. A section ends immediately before the next section or at the end of the Procedure Division or, in the declaratives portion of the Procedure Division, at the keywords END DECLARATIVES.

A paragraph consists of a paragraph name followed by a period and a space and by zero, one, or more successive sentences. A paragraph ends immediately before the next paragraph name or section name or at the end of the Procedure Division or, in the declarative portion of the Procedure Division, at the keywords END DECLARATIVES.

A sentence consists of one or more statements and is terminated by the separator period.

A statement is a syntactically valid combination of words, literals, and separators beginning with a COBOL verb.

The term "identifier" is defined as the word or words necessary to make unique reference to a data item.

Execution

Execution begins with the first statement of the Procedure Division, excluding declaratives. Statements are then executed in the order in which they are presented for compilation, except where the rules indicate some other order.

Structure

Header

The Procedure Division is identified by and must begin with the following header:

```
PROCEDURE DIVISION  [ USING  { data-name-1 }  ...  ]
   [ WITH ENTRY POINTS  { procedure-name-1 } ...  ]
```

Body

The body of the Procedure Division must conform to one of the following formats:

Format 1

```
[ DECLARATIVES.
{ section-name SECTION  [ segment-number ] . declarative-sentence
[ paragraph-name. [ sentence ] ...  ] ...  } ...
END DECLARATIVES. ]
{ section-name SECTION  [ segment-number ] .
[ paragraph-name . [ sentence ] ...  ] ...  } ...
```

Format 2

```
{ paragraph-name . [ sentence ] ...  } ...
```

Statements

There are four types of statements: conditional statements, compiler directing statements, imperative statements, and delimited scope statements.

Conditional

A conditional statement specifies that the truth value of a condition is to be determined and that the subsequent action of the object program depends on this truth value.

A conditional statement is one of the following:

- An EVALUATE, IF, SEARCH, or RETURN statement.
- A READ statement that specifies the AT END, NOT AT END, INVALID KEY, or NOT INVALID KEY phrase.
- A WRITE statement that specifies the INVALID KEY, NOT INVALID KEY, END-O-FPAGE, or NOT END-OF-PAGE phrase.
- A DELETE, REWRITE, or START statement that specifies the INVALID KEY or NOT INVALID KEY phrase.
- An arithmetic statement (ADD, COMPUTE, DIVIDE, MULTIPLY, SUBTRACT) that specifies the ON SIZE ERROR or NOT ON SIZE ERROR phrase.
- A STRING or UNSTRING statement that specifies the ON OVERFLOW or NOT ON OVERFLOW phrase.
- A CALL statement that specifies the ON OVERFLOW, ON EXCEPTION, or NOT ON EXCEPTION phrase.

Compiler Directing

A compiler directing statement consists of a compiler directing verb and its operands. The compiler directing verbs are COPY, REPLACE, and USE (see Section 9.2, Section 9.3, Section 4.5.6, Section 5.5.8, and Section 10.4.9). A compiler directing statement causes the compiler to take a specific action during compilation.

Imperative

An imperative statement begins with an imperative verb. It specifies an unconditional action to be taken by the object program or is a conditional statement that is delimited by its explicit scope terminator (delimited scope statement). An imperative statement can consist of a sequence of imperative statements, each possibly separated from the next by a separator. The imperative verbs are:

```
ACCEPT            EXIT             RELEASE
ADD       (1)     GENERATE         REWRITE     (2)
ALTER             GO TO            SET
CALL      (6)     INITIALIZE       SORT
CANCEL            INITIATE         START       (2)
CLOSE             INSPECT          STOP
COMPUTE   (1)     MERGE            STRING      (3)
CONTINUE          MOVE             SUBTRACT    (1)
```

```
DELETE    (2)     MULTIPLY   (1)   SUPPRESS
DISPLAY           OPEN             TERMINATE
DIVIDE    (1)     PERFORM          UNSTRING    (3)
ENTER             READ       (4)   WRITE       (5)
```

Explanation

1. Without the optional ON SIZE ERROR and NOT ON SIZE ERROR phrases.
2. Without the optional INVALID KEY and NOT INVALID KEY phrases.
3. Without the optional ON OVERFLOW and NOT ON OVERFLOW phrases.
4. Without the optional AT END, NOT AT END, INVALID KEY, and NOT INVALID KEY phrases.
5. Without the optional INVALID KEY, NOT INVALID KEY, END-OF-PAGE, and NOT END-OF-PAGE phrases.
6. Without the optional ON OVERFLOW, ON EXCEPTION, and NOT ON EXCEPTION phrases.

When *imperative-statement* appears in the general format of statements, *imperative-statement* refers to that sequence of consecutive imperative statements that must be ended by a period or an ELSE phrase associated with a previous IF statement or a WHEN phrase associated with a previous SEARCH statement.

Delimited Scope

A delimited scope statement is any statement that includes its explicit scope terminator. (See Section 2.4.21)

Sentences

There are three types of sentences: conditional sentences, compiler directing sentences, and imperative sentences.

Conditional

A conditional sentence is a conditional statement, optionally preceded by an imperative statement, terminated by the separator period.

Compiler Directing

A compiler directing sentence is a single compiler directing statement terminated by the separator period.

Imperative

An imperative sentence is an imperative statement terminated by the separator period.

Conditional Phrases

A conditional phrase specifies the action to be taken upon determination of the truth value of a condition resulting from the execution of a conditional statement.

A conditional phrase is one of the following:

- AT END or NOT AT END phrase when specified within a READ statement
- INVALID KEY or NOT INVALID KEY phrase when specified within a DELETE, READ, REWRITE, START, or WRITE statement
- END-OF-PAGE or NOT END-OF-PAGE phrase when specified within a WRITE statement
- SIZE ERROR or NOT ON SIZE ERROR phrase when specified within an ADD, COMPUTE, DIVIDE, MULTIPLY, or SUBTRACT statement
- ON OVERFLOW or NOT ON OVERFLOW phrase when specified within a STRING or UNSTRING statement
- ON OVERFLOW, ON EXCEPTION, or NOT ON EXCEPTION phrase when specified within a CALL statement

2.6. Reference Format

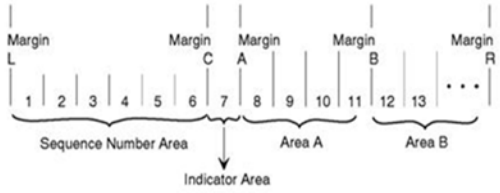
The reference format, which provides a standard method for describing UCS COBOL source programs, is described in terms of character positions in a line on an input/output medium. A line consists of the character positions 1 through 72 of a source program card image. The UCS COBOL compiler accepts source programs written in reference format and produces an output listing of the source program input in reference format.

The rules for spacing given in the discussion of the reference format take precedence over all other rules for spacing.

The divisions of a source program must be ordered as follows: the Identification Division, then the Environment Division, then the Data Division, then the Procedure Division. Each division must be written according to the rules for the reference format.

2.6.1. Representation

The reference format for a line is shown in the following figure:



Margin L is immediately to the left of the leftmost character position of a line.

Margin C is between the 6th and 7th character positions of a line.

Margin A is between the 7th and 8th character positions of a line.

Margin B is between the 11th and 12th character positions of a line.

Margin R is immediately to the right of the 72nd character position of a line.

The sequence number area occupies six character positions (1–6) and is between margin L and margin C.

The indicator area is the 7th character position of a line.

Area A occupies character positions 8, 9, 10, and 11 and is between margin A and margin B.

Area B occupies 61 character positions. It begins immediately to the right of margin B and terminates immediately to the left of margin R.

2.6.2. Sequence Numbers

A sequence number, consisting of six digits in the sequence area, can be used to label a source program line. The content of the sequence number area you define can consist of any character in the computer's character set. There is no requirement that the content of the sequence number area appear in any particular sequence or be unique.

2.6.3. Continuation of Lines

Any sentence, entry, phrase, or clause can be continued by starting subsequent lines in area B. These subsequent lines are called the continuation lines. The line being continued is called the continued line. Any word, literal, or PICTURE character string can be broken in such a way that part of it appears on a continuation line.

A hyphen in the indicator area of a line indicates that the first nonblank character in area B of the current line is the successor of the last nonblank character of the preceding line, excluding intervening comment lines or blank lines, without any intervening space. However, if the continued line contains a nonnumeric literal without closing quotation mark, the first nonblank character in area B on the continuation line must be a quotation mark, and the continuation starts with the character immediately after that quotation mark. All spaces at the end of the continued line are considered part of the literal. Area A of a continuation line must be blank.

If there is no hyphen in the indicator area of a line, it is assumed that the first nonblank character in the line is preceded by a space.

Both characters composing the "= =" separator must be on the same line.

2.6.4. Blank Lines

A blank line is one that is blank from margin C to margin R, inclusive. A blank line can appear anywhere in the source program.

2.6.5. Comment Lines

A comment line is any line with an asterisk or slant in the indicator area of the line. A comment line can appear as any line in a source program after the Identification Division header and as any line in library text of a UCS COBOL library. Any combination of the characters from the computer's character set can be included in area A and area B of that line.

The asterisk or slant and the characters in area A and area B is produced on the listing but serve as documentation only and is not be checked syntactically. The slant in the indicator area causes page ejection prior to printing the comment line in the listing of the source program; an asterisk in the indicator area causes printing of the line at the next available line position in the listing.

2.6.6. Pseudotext

The character strings and separators comprising pseudotext can start in either area A or area B. If, however, there is a hyphen in the indicator area of a line that follows the opening pseudotext delimiter, area A of the line must be blank and the normal rules for continuation of lines apply to the format of text words. (See Section 2.6.3.)

2.6.7. Division Header

The division header must start in area A.

2.6.8. Section Header

The section header must start in area A.

A section consists of zero, one, or more paragraphs in the Environment or Procedure Divisions or zero, one, or more entries in the Data Division.

2.6.9. Paragraph Header, Paragraph Name, and Paragraph

A paragraph consists of a paragraph name followed by a separator period and by zero, one, or more sentences, or a paragraph header followed by one or more entries.

The paragraph header or paragraph name starts in area A of any line following the first line of a division or a section.

The first sentence or entry in a paragraph begins either on the same line as the paragraph header or paragraph name or in area B of the next nonblank line that is not a comment line. Successive sentences or entries either begin in area B of the same line as the preceding sentence or entry or in area B of the next nonblank line that is not a comment line.

When the sentences or entries of a paragraph require more than one line, they can be continued as described in Section 2.6.3.

2.6.10. Data Division Entries

Each Data Division entry begins with a level indicator or a level number, followed by a space, followed by the name of the subject of entry, if specified, followed by a sequence of independent clauses describing the item. The last clause is always terminated by a separator period.

There are two types of Data Division entries: those that begin with a level indicator and those that begin with a level number.

The level indicator can be any among FD, SD, or RD.

In those Data Division entries that begin with a level indicator, the level indicator begins in area A followed by at least one space, and then followed by the name of the subject of entry and appropriate descriptive information. Those Data Division entries that begin with level numbers are called data description entries.

A level number has a value taken from the set of values 1 through 49, 66, 77 and 88. Level numbers in the range 1 through 9 can be written either as a single digit or as a zero followed by a significant digit. At least one space must separate a level number from the word following the level number.

In those data description entries that begin with a level number 01 or 77, the level number begins in area A, followed by at least one space, and then followed by its associated record name or item name, if specified, and appropriate descriptive information.

Data description entries can be indented. Any indentation is with respect to margin A. Each new data description entry can begin any number of positions to the right of margin A, except data description entries that begin with level number 01 or 77 must begin in area A.

2.6.11. Declaratives

The keyword DECLARATIVES and the pair of keywords END DECLARATIVES that precede and follow, respectively, the declaratives portion of the Procedure Division must each appear on a separate line. Each must begin in area A and be followed by the separator period.

2.6.12. End Program Header

The end program header must start in area A.

Section 3
Nucleus

3.1. General

The Nucleus module provides a language capability for the internal processing of data within the structure of the four divisions of a program. The Nucleus also provides a capability for defining tables of contiguous data items and accessing an item relative to its position in a table. The Nucleus provides a debugging capability consisting of a compile time switch and debugging lines.

3.1.1. Level Characteristics

Nucleus level 1 provides limited capabilities for the SPECIAL-NAMES paragraph and the data description entry. Within the Procedure Division, the Nucleus level 1 provides limited capabilities for the ACCEPT, ADD, ALTER, DISPLAY, DIVIDE, IF, MOVE, MULTIPLY, PERFORM, and SUBTRACT statements and full capabilities for the CONTINUE, ENTER, EXIT, GO TO, and STOP statements. Nucleus level 1 does not provide full COBOL capabilities for qualification, data name formation, and figurative constants. Nucleus level 1 provides a capability for accessing items in up to three-dimensional fixed-length tables. Nucleus level 1 provides a debugging capability consisting of a compile time switch and debugging lines.

Nucleus level 2 provides full capabilities for the SPECIAL-NAMES paragraph and the data description entry. Within the Procedure Division, the Nucleus level 2 provides full capabilities for the ACCEPT, ADD, ALTER, COMPUTE, DISPLAY, DIVIDE, EVALUATE, IF, INITIALIZE, INSPECT, MOVE, MULTIPLY, PERFORM, SEARCH, SET, STRING, SUBTRACT, and UNSTRING statements. Nucleus level 2 provides full capabilities for qualification, data name formation, and figurative constants. Nucleus level 2 provides a capability for accessing items in up to seven-dimensional tables.

3.1.2. Level Restrictions on Overall Language

This subsection describes the following level restrictions on the overall language:

- character set
- name characteristics
- figurative constants
- subscripts
- reference modification
- reference format

Character Set

The COBOL character colon (:) is not included in level 1. The COBOL character colon (:) is permitted in level 2.

Name Characteristics

Qualification is not included in level 1. In level 1, all user-defined words except level numbers and segment numbers must be unique if referenced. Fifty qualifiers are permitted in level 2. User-defined words need not be unique in level 2.

Figurative Constants

The figurative constants that can be used in level 1 are ZERO, ZEROS, ZEROES, SPACE, SPACES, HIGH-VALUE, HIGH-VALUES, LOW-VALUE, LOW-VALUES, QUOTE, and QUOTES.

The figurative constants that can be used in level 2 are ZERO, ZEROS, ZEROES, SPACE, SPACES, HIGH-VALUE, HIGH-VALUES, LOW-VALUE, LOW-VALUES, QUOTE, QUOTES, *symbolic-character,* ALL *literal,* ALL *figurative constant,* and ALL *symbolic-character.*

Subscripts

One, two, or three subscripts are permitted in level 1. Up to seven subscripts are permitted in level 2.

Reference Modification

Reference modification is permitted in level 2 only.

Reference Format

In level 1, a word, numeric literal, or PICTURE character string cannot be broken in such a way that part of it appears on a continuation line. In level 2, a word, numeric literal, or PICTURE character string can be broken in such a way that part of it appears on a continuation line.

3.2. A COBOL Source Program

This subsection describes the general organization and structure of a COBOL source program.

3.2.1. General Description

A COBOL source program is a syntactically correct set of COBOL statements.

3.2.2. Organization

With the exception of COPY and REPLACE statements and the end program header, the statements, entries, paragraphs, and sections of a COBOL source program are grouped into four divisions sequenced in the following order:

1. Identification Division
2. Environment Division
3. Data Division
4. Procedure Division

The end of a COBOL source program is indicated by either the end program header, if specified, or by the absence of additional source program lines.

3.2.3. Structure

The following gives the general format and order of presentation of the entries and statements that constitute a COBOL source program.

General Format

```
identification-division
[environment-division]
[data-division]
[procedure-division]
[end-program-header]
```

Syntax Rule

The generic terms identification-division, environment-division, data-division, procedure-division, and end-program-header represent a COBOL Identification Division, a COBOL Environment Division, a COBOL Data Division, a COBOL Procedure Division, and a COBOL end program header.

General Rules

1. The beginning of a division in a program is indicated by the appropriate division header. The end of a division is indicated by one of the following:
   a. The division header of a succeeding division in that program
   b. The end program header
   c. That physical position after which no more source program lines occur
2. All separately compiled source programs in a sequence of programs must be terminated by an end program header except for the last program in the sequence.

3.2.4. End Program Header

Function

The end program header indicates the end of the named COBOL source program.

General Format

```
END PROGRAM program-name.
```

Syntax Rules

1. The *program-name* must conform to the rules for forming a user-defined word.
2. The *program-name* must be identical to a program name declared in a preceding PROGRAM-ID paragraph. (See Section 3.3.4.)

General Rules

1. The end program header indicates the end of the specified COBOL source program.
2. If the next source statement after the program terminated by the end program header is a COBOL statement, it must be the Identification Division header of a program to be compiled separately from that program terminated by the end program header.

3.3. Identification Division

This subsection describes the Identification Division and its associated structure.

3.3.1. General Description

The Identification Division identifies the program and the program attributes. The Identification Division is required in a COBOL source program. The user can include the date the program is written and other information as desired using the general format shown in Section 3.3.3, “Structure”.

3.3.2. Organization

Paragraph headers identify the type of information contained in the paragraph. The name of the program must be given in the first paragraph, which is the PROGRAM-ID paragraph. The other paragraphs are optional and can be included in this division at the user’s choice, in the order of presentation shown by the general format in Section 3.3.3, “Structure”.

The AUTHOR paragraph, INSTALLATION paragraph, DATE-WRITTEN paragraph, DATE-COMPILED paragraph, and SECURITY paragraph are obsolete elements in this revision of American National Standard COBOL and are to be deleted from the next revision of American National Standard COBOL.

3.3.3. Structure

The following is the general format of the paragraphs in the Identification Division and it defines the order of presentation in the source program. See 3.3.4 and 3.3.5 which defines the PROGRAM-ID paragraph and the DATE-COMPILED paragraph. While the other paragraphs are not defined, each general format is formed in the same manner.

General Format

```
IDENTIFICATION DIVISION

PROGRAM-ID.program-name

           [ AUTHOR.[ comment.entry]. . . ]

           [ INSTALLATION.[ comment.entry]. . .  ]

           [ DATE-WRITTEN.[ comment.entry]. . .  ]

           [ DATE COMPILED.[ comment.entry]. . . ]

           [ SECURITY.[ comment.entry]. . .  ]
```

Syntax Rule

The *comment-entry* can be any combination of the characters from the computer's character set. The continuation of the *comment-entry* by the use of the hyphen in the indicator area is not permitted; however, the *comment-entry* can be contained on one or more lines.

3.3.4. The PROGRAM-ID Paragraph

Function

The PROGRAM-ID paragraph specifies the name by which a program is identified.

General Format

```
PROGRAM-ID.program-name
```

Syntax Rules

1. The *program-name* must conform to the rules for formation of a user-defined word.
2. Additional syntax rules are found in Section 7.4 on COMMON and INITIAL programs.

General Rules

1. The *program-name* identifies the source program, the object program, and all listings pertaining to a particular program.
2. Additional general rules are found in Section 7.4 on COMMON and INITIAL programs.

3.3.5. The DATE-COMPILED Paragraph

Function

The DATE-COMPILED paragraph provides the compilation date in the Identification Division source program listing.

The DATE-COMPILED paragraph is an obsolete element in this revision of American National Standard COBOL and is to be deleted from the next revision of American National Standard COBOL.

General Format

```
DATE-COMPILED . [ comment-entry ]. . .
```

Syntax Rule

The *comment-entry* can be any combination of the characters from the computer's character set. The continuation of the *comment-entry* by the use of the hyphen in the indicator area is not permitted; however, the *comment-entry* can be contained on one or more lines.

General Rule

The DATE-COMPILED paragraph inserts the current date during program compilation. If a DATE-COMPILED paragraph is present, it is replaced during compilation with a paragraph of the form:

```
DATE-COMPILED.   current-date.
```

3.4. Environment Division

This subsection describes the Environment Division and its associated paragraphs.

3.4.1. General Description

The Environment Division specifies a standard method of expressing those aspects of a data processing problem that depend on the physical characteristics of a specific computer. The Environment Division is optional in a COBOL source program. This division allows specification of information relating to input/output control, special hardware characteristics, and control techniques.

3.4.2. Configuration Section

The Configuration Section is located in the Environment Division of a source program. The Configuration Section deals with the characteristics of the source computer and the object computer. This section also provides a means for specifying the currency sign, choosing the decimal point, specifying symbolic characters, relating implementor names to user-specified mnemonic names, relating alphabet names to character sets or collating sequences, and relating class names to sets of characters. The Configuration Section is optional in the Environment Division of a COBOL source program.

General format

```
[ CONFIGURATION SECTION.]

[ SOURCE-COMPUTER.  [ source-computer-entry ] ]

[ OBJECT-COMPUTER.  [ object-computer-entry ] ]

[ SPECIAL-NAMES. [ special-names-entry ] ]
```

The Configuration Section must not be stated in a program contained directly or indirectly within another program.

The entries explicitly or implicitly stated in the Configuration Section of a program containing other programs apply to each contained program.

3.4.3. The SOURCE-COMPUTER Paragraph

Function

The SOURCE-COMPUTER paragraph provides a means of describing the computer upon which the program is to be compiled.

General Format

```
SOURCE-COMPUTER.  [computer-name [ WITH DEBUGGING MODE ] . ]
```

Syntax Rule

Computer-name is a system name.

General Rules

1. All clauses of the SOURCE-COMPUTER paragraph apply to the program in which they are explicitly or implicitly specified and to any program contained within that program.
2. When the SOURCE-COMPUTER paragraph is not specified and the program is not contained within a program including a SOURCE-COMPUTER paragraph, the computer on which the source program is being compiled is the source computer.
3. When the SOURCE-COMPUTER paragraph is specified but the source-computer-entry is not specified, the computer upon which the source program is being compiled is the source computer. The SOURCE-COMPUTER *computer-name* is for documentation only.
4. If the WITH DEBUGGING MODE clause is specified in a program, all debugging lines are compiled as specified in this presentation of the Nucleus module. (See Section 3.7)
5. If the WITH DEBUGGING MODE is not specified in a program and the program is not contained within a program including a WITH DEBUGGING MODE clause, any debugging lines are compiled as if they are comment lines.

3.4.4. The OBJECT-COMPUTER Paragraph

Function

The OBJECT-COMPUTER paragraph provides a means of describing the computer on which the program is to be executed. The MEMORY SIZE clause is an obsolete element in this revision of American National Standard COBOL and it is deleted from the next revision of American National Standard COBOL.

General Format

```
[ OBJECT-COMPUTER.  [ computer-name
[ MEMORY SIZE integer-1 { WORDS      } ]
                        { CHARACTERS }
                        { MODULES    }
[ PROGRAM COLLATING SEQUENCE IS alphabet-name-1 ] . ]
```

Syntax Rule

Computer-name is a system name.

General Rules

1. All clauses of the OBJECT-COMPUTER paragraph apply to the program in which they are explicitly or implicitly specified and to any program contained within that program.
2. *Computer-name* is for documentation only. Any name is allowed. The type of object code generated by the compiler is determined by the EXTENDED and EXTENDED/*n* keyword option (see the *UCS COBOL Programming Reference Manual, Volume 2*).
3. When the OBJECT-COMPUTER paragraph is not specified and the program is not contained within a program including an OBJECT-COMPUTER paragraph, the computer on which the source program is being compiled is the object computer.
4. When the OBJECT-COMPUTER paragraph is specified, but the object computer entry is not specified, the computer upon which the source program is being compiled is the object computer and the effect is as though the OBJECT-COMPUTER paragraph is replaced in the source program with a paragraph of the form:

   ```
   OBJECT-COMPUTER. computer-name.
   ```

5. The MEMORY SIZE clause is for documentation only. The compiler checks the syntax only for correctness.
6. If the PROGRAM COLLATING SEQUENCE clause is specified, the program collating sequence is the collating sequence associated with *alphabet-name-1* specified in that clause.

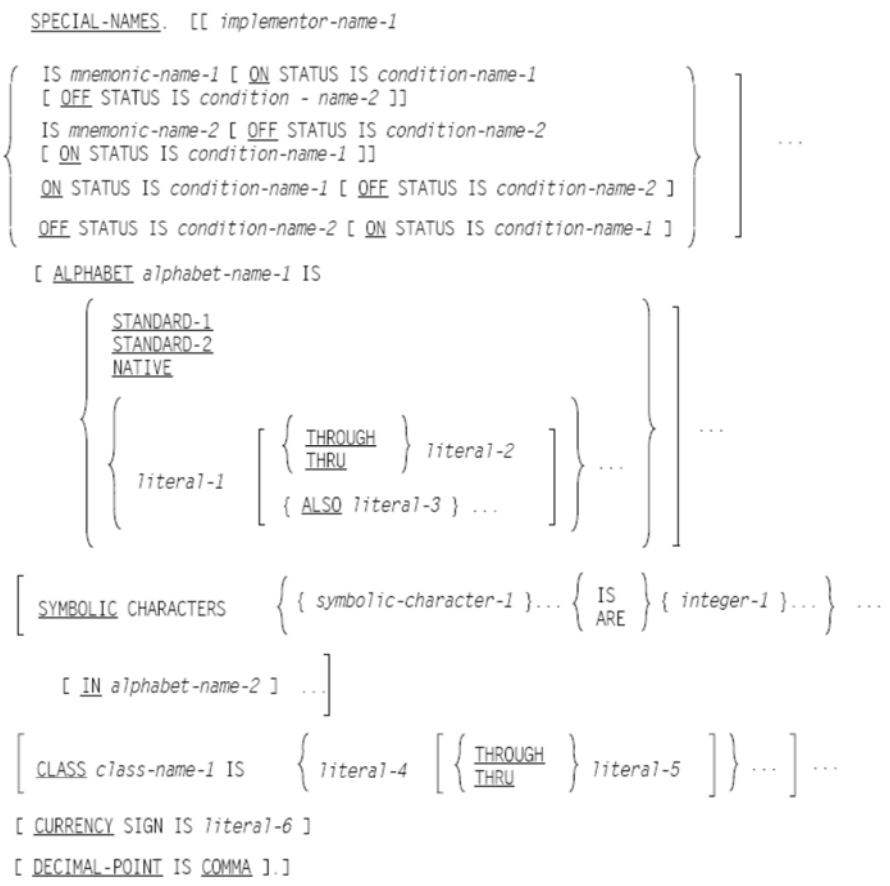
7. If the PROGRAM COLLATING SEQUENCE clause is not specified, the program collating sequence is the native collating sequence.
8. The program collating sequence established in the OBJECT-COMPUTER paragraph is used to determine the truth value of any nonnumeric comparisons that are
   a. Explicitly specified in relation conditions (See Section a.)
   b. E xplicitly specified in condition name conditions (See Section a.)
   c. I mplicitly specified by the presence of a CONTROL clause in a report description entry. (See Section 10.3.2.)
9. The program collating sequence established in the OBJECT-COMPUTER paragraph is applied to any nonnumeric merge or sort keys unless the COLLATING SEQUENCE phrase is specified in the respective SORT or MERGE statement. (See Section 8.5.4 and Section 8.5.1)

3.4.5. The SPECIAL-NAMES Paragraph

Function

The SPECIAL-NAMES paragraph provides a means for specifying the currency sign, choosing the decimal point, specifying symbolic-characters, relating implementor-names to user-specified mnemonic names, relating alphabet names to character sets or collating sequences, and relating class names to sets of characters.

General Format



Syntax Rules

1. If *implementor-name-1* refers to an external switch, the associated mnemonic name can be specified only in the SET statement. The allowable values for *implementor-name-1* are as follows:

```
CARD-PUNCH    PAGE
CARD-READER PRINTER
CONSOLE     SWITCH-n (where n = 1 - 24)
ANS74-PRINTER
```

2. If *implementor-name-1* does not refer to an external switch, the associated mnemonic name can be specified only in the ACCEPT, DISPLAY, SEND, or WRITE statements. A condition name cannot be associated with such an implementor name.
3. If the literal phrase of the ALPHABET clause is specified, a given character must not be specified more than once in that clause.

4. The literals specified in the literal phrase of the ALPHABET clause:
   a. If numeric, must be unsigned integers and must have a value within the range of one through the maximum number of characters in the native character set.
   b. If nonnumeric and associated with a THROUGH or ALSO phrase, must each be one character in length.
5. *Literal-1, literal-2, literal-3, literal-4,* and *literal-5* must not specify a symbolic-character figurative constant.
6. The words THRU and THROUGH are equivalent.
7. The same *symbolic-character-1* must appear only once in a SYMBOLIC CHARACTERS clause.
8. The relationship between each *symbolic-character-1* and the corresponding *integer-1* is by position in the SYMBOLIC CHARACTERS clause. The first *symbolic-character-1* is paired with the first *integer-1*; the second *symbolic-character-1* is paired with the second *integer-1*; and so on.
9. There must be a one-to-one correspondence between occurrences of *symbolic-character-1* and occurrences of *integer-1.*
10. The ordinal position specified by *integer-1* must exist in the native character set. If the IN phrase is specified, the ordinal position must exist in the character set named by *alphabet-name-2*.
11. The literals specified in the *literal-4* phrase: If numeric, must be unsigned integers and must have a value within the range of one through the maximum number of characters in the native character set. If nonnumeric and associated with a THROUGH phrase, must each be one character in length.
12. *Literal-6* must not be a figurative constant.

General Rules

1. All clauses specified in the SPECIAL-NAMES paragraph for a program also apply to programs contained within that program. The condition names specified in the containing program's SPECIAL-NAMES paragraph can be referenced from any contained program.
2. If *implementor-name-1* refers to an external switch, the on or off status of that switch can be associated with condition names. The status of that switch can be interrogated by testing these condition names (see Section a).
3. I f *implementor-name-1* refers to an external switch, the status of that switch can be altered by execution of a format 3 SET statement that specifies as its operand the mnemonic name associated with that switch (see Section a).

4. T he ALPHABET clause provides a means for relating a name to a specified character code set or collating sequence. When *alphabet-name-1* is referred to in the PROGRAM COLLATING SEQUENCE clause (see Section 3.4.4) or the COLLATING SEQUENCE phrase of a SORT or MERGE statement (see Section 8.5.4 and Section 8.5.1) the ALPHABET clause specifies a collating sequence. When alphabet-name-1 is referred to in the SYMBOLIC CHARACTERS clause or in a CODE-SET clause in a file description entry (see Section 4.4.5), the ALPHABET clause specifies a character code set.

   a. If the STANDARD-1 phrase is specified, the character code set or collating sequence identified is that defined in American National Standard X3.4-1977, Code for Information Interchange. If the STANDARD-2 phrase is specified, the character code set identified is the International Reference version of the ISO 7-bit code defined in International Standard 646, 7-Bit Coded Character Set for Information Processing Interchange. Each character of the standard character set is associated with its corresponding character of the native character set.

   b. If the NATIVE phrase is specified, the native character code set is that defined in American National Standard X3.4-1977, Code for Information Interchange.

   c. If the literal phrase is specified, the alphabet name cannot be referred to in a CODE-SET clause (see Section 4.4.5). The collating sequence identified is that defined according to the following rules:

Rule 1

The value of each literal specifies:

1. The ordinal number of a character within the native character set, if the literal is numeric. This value must not exceed the value that represents the number of characters in the native character set.
2. The actual character within the native character set, if the literal is nonnumeric. If the value of the nonnumeric literal contains multiple characters, each character in the literal, starting with the leftmost character, is assigned successive ascending positions in the collating sequence being specified.

Rule 2

The order in which the literals appear in the ALPHABET clause specifies, in ascending sequence, the ordinal number of the character within the collating sequence being specified.

Rule 3

Any characters within the native collating sequence, which are not explicitly specified in the literal phrase, assume a position, in the collating sequence being specified, greater than any of the explicitly specified characters. The relative order within the set of these unspecified characters is unchanged from the native collating sequence.

Rule 4

If the THROUGH phrase is specified, the set of contiguous characters in the native character set beginning with the character specified by the value of *literal-1* and ending with the character specified by the value of *literal-2* is assigned a successive ascending position in the collating sequence being specified.

In addition, the set of contiguous characters specified by a given THROUGH phrase can specify characters of the native character set in either ascending or descending sequence.

Rule 5

If the ALSO phrase is specified, the characters of the native character set specified by the value of *literal-1* and *literal-3* are assigned to the same ordinal position in the collating sequence being specified.

5. The character that has the highest ordinal position in the program collating sequence is associated with the figurative constant HIGH-VALUE, except when this figurative constant is specified as a literal in the SPECIAL-NAMES paragraph.

   If more than one character has the highest position in the program collating sequence, the last character specified is associated with the figurative constant HIGH-VALUE.

6. The character that has the lowest ordinal position in the program collating sequence is associated with the figurative constant LOW-VALUE, except when this figurative constant is specified as a literal in the SPECIAL-NAMES paragraph. If more than one character has the lowest position in the program collating sequence, the first character specified is associated with the figurative constant LOW-VALUE.

7. When specified as literals in the SPECIAL-NAMES paragraph, the figurative constants HIGH-VALUE and LOW-VALUE are associated with those characters having the highest and lowest positions, respectively, in the native collating sequence.

8. If the IN phrase is not specified, *symbolic-character-1* represents the character whose ordinal position in the native character set is specified by *integer-1.* If the IN phrase is specified, *integer-1* specifies the ordinal position of the character that is represented in the character set named by *alphabet-name-2.*

9. The internal representation of *symbolic-character-1* is the internal representation of the character that is represented in the native character set.

10. The CLASS clause provides a means for relating a name to the specified set of characters listed in that clause. Class name can be referenced only in a class condition. The characters specified by the values of the literals in this clause define the exclusive set of characters of which this class name consists.

    The value of each literal specifies:

    a. The ordinal number of a character within the native character set, if the literal is numeric. This value must not exceed the value that represents the number of characters in the native character set.

b. The actual character within the native character set, if the literal is nonnumeric. If the value of the nonnumeric literal contains multiple characters, each character in the literal is included in the set of characters identified by class name.

   If the THROUGH phrase is specified, the contiguous characters in the native character set beginning with the character specified by the value of literal-4 and ending with the character specified by the value of literal-5 are included in the set of characters identified by class name In addition, the contiguous characters specified by a given THROUGH phrase can specify characters of the native character set in either ascending or descending sequence.

11. *Literal-6*, which appears in the CURRENCY SIGN clause, is used in the PICTURE clause to represent the currency symbol. The literal must be nonnumeric and is limited to a single character. It can be any character from the computer's character set except the following:

    c. Digits 0 through 9

    d. Alphabetic characters consisting of the uppercase letters A, B, C, D, P, R, S, V, X, Z; the lowercase letters a through z; or the space

    e. Special characters * + - , . ; ( ) " = /

    If this clause is not present, only the currency sign defined in the COBOL character set can be used as the currency symbol in the PICTURE clause.

12. The clause DECIMAL-POINT IS COMMA means that the functions of comma and period are exchanged in the character string of the PICTURE clause and in numeric literals.

3.5. Data Division

This subsection describes the Data Division and its associated storage areas and clauses.

3.5.1. General Description

The Data Division describes the data that is to be processed by the object program. The Data Division is optional in a COBOL source program.

3.5.2. Working-Storage Section

The Working-Storage Section, located in the Data Division of a source program, describes records and subordinate data items that are not part of data files. The Working-Storage Section is composed of the section header, followed by record description entries and data description entries for noncontiguous data items.

The general format of the Working-Storage Section is as follows:

```
WORKING-STORAGE SECTION.
[ 77-level-description-entry
  record-description-entry ]
```

Noncontiguous Working Storage

Items and constants in working storage that bear no hierarchical relationship to one another need not be grouped into records, provided they do not need to be further subdivided. Instead, they are classified and defined as noncontiguous elementary items. Each of these items is defined in a separate data description entry that begins with the special level number 77.

The following data clauses are required in each data description entry:

- Level number 77
- Data name
- The PICTURE, USAGE IS INDEX, COMPUTATIONAL-1, or COMPUTATIONAL-2 clauses

Other data description clauses are optional and can be used to complete the description of the item if necessary.

Working Storage Records

Data elements in working storage that bear a definite hierarchical relationship to one another must be grouped into records according to the rules for formation of record descriptions. Data elements in the Working-Storage Section that bear no hierarchical relationship to any other data item can be described as records that are single elementary items. All clauses used in record descriptions in the File Section can be used in record descriptions in the Working-Storage Section.

3.5.3. Record Description Structure

A record description consists of a set of data description entries describing the characteristics of a particular record. Each data description entry consists of a level number followed by the data name or FILLER clause, if specified, followed by a series of independent clauses, as required. A record description can have a hierarchical structure and, therefore, the clauses used within an entry can vary considerably, depending on whether or not they are followed by subordinate entries. The structure of a record description and the elements allowed in a record description entry are explained in 2.4.15 and 3.3.5.

3.5.4. Initial Values

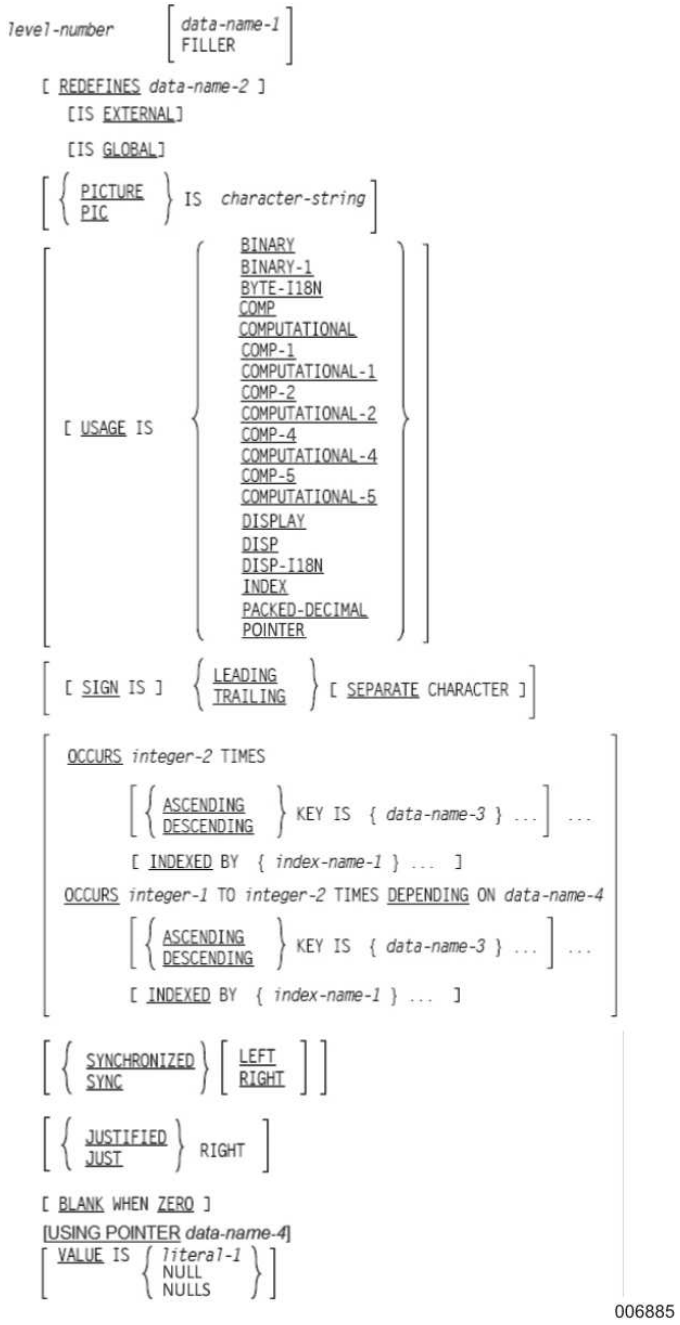
The initial value of any item in the Working-Storage Section except an index data item is specified by associating the VALUE clause with the data item. The initial value of any index data item or any data item not associated with a VALUE clause is undefined.

3.5.5. The Data Description Entry

Function

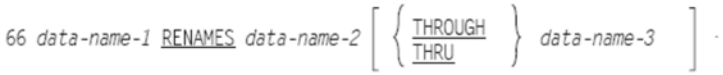
A data description entry specifies the characteristics of a particular item of data.

Format 1



006885

Format 2



Format 3



Syntax Rules

1. *Level-number* in format 1 can be any number from 01 through 49, or 77.
2. In format 1, the *data-name-1* or FILLER clause, if specified, must immediately follow *level-number*. The REDEFINES clause, if specified, must immediately follow the *data-name-1* clause or FILLER clause if either is specified; otherwise, it must immediately follow the *level-number*. The remaining clauses can be written in any order.
3. The PICTURE clause must be specified for every elementary item except an index data item or a floating-point data item (see Table 3–4) and the subject of the RENAMES clause. In these cases, the PICTURE clause is prohibited.
4. The words THRU and THROUGH are equivalent.

General Rules

1. The clauses SYNCHRONIZED, PICTURE, JUSTIFIED, and BLANK WHEN ZERO must not be specified except for an elementary data item.
2. Format 3 is used for each condition name. Each condition-name requires a separate entry with level number 88. Format 3 contains the name of the condition and the value, values, or range of values associated with the condition name. The condition name entries for a particular conditional variable must immediately follow the entry describing the item with which the name is associated. A condition name can be associated with any data description entry that contains a level number except the following:
   a. Another condition name
   b. A level-66 item
   c. A group containing items with descriptions including JUSTIFIED, SYNCHRONIZED, or USAGE (other than USAGE IS DISPLAY)
   d. An index data item
3. Multiple level-01 entries subordinate to any given level indicator other than the level indicator RD for a report description entry represent implicit redefinitions of the same area.

3.5.6. The BLANK WHEN ZERO Clause

The BLANK WHEN ZERO clause permits the blanking of an item when its value is zero.

General Format

```
BLANK WHEN ZERO
```

Syntax Rules

1. The BLANK WHEN ZERO clause can be specified only for an elementary item whose PICTURE is specified as numeric or numeric edited. (See Section 3.5.11)
2. The numeric or numeric edited data description entry to which the BLANK WHEN ZERO clause applies must be described, either implicitly or explicitly, as USAGE IS DISPLAY.

General Rules

1. When the BLANK WHEN ZERO clause is used, the item will contain nothing but spaces if the value of the item is zero.
2. When the BLANK WHEN ZERO clause is used for an item whose PICTURE is numeric, the category of the item is considered to be numeric edited.

The following example shows the contents of variable data items following a move of zeros to the field.

Picture	Blank When Zero Specified?	Edited Item
$ZZZ9	No	$ 0
$ZZZ9	Yes	
****	No	****
9(4)	No	0000
9(4)	Yes	

3.5.7. The Data Name or FILLER Clause

Function

A data name specifies the name of the data being described. The keyword FILLER specifies a data item that is not referred to explicitly.

General Format

```
[ data-name-1 ]
[ FILLER      ]
```

Syntax Rule

In the File, Working-Storage, Common-Storage, and Linkage Sections, *data-name-1* or the keyword FILLER, if either is specified, must be the first word following the level number in each data description entry.

General Rules

1. If this clause is omitted, the data item being described is treated as though FILLER had been specified.
2. The keyword FILLER can be used to name a data item. Under no circumstances can a FILLER item be referred to explicitly. However, the keyword FILLER can be used to name a conditional variable because such use does not require explicit reference to the data item itself, but only to the value contained therein. FILLER can be used in place of a data name to indicate a portion that is to contain no directly addressable data; for example, a space between two data items that are to be printed. As an example, if the following were written:

```
01 EMPLOYEE-HISTORY.
  02 EMPLOYEE-ID.
     03 LIFE-NO PICTURE IS 9(6).
     03 FILLER PICTURE IS X(6) VALUE IS SPACE.
     03 NAME PICTURE IS A(24).
```

A nonaddressable 6-character field would be inserted into the record between LIFE-NO and NAME and preset to contain spaces.

The contents of that 6-character field can then be modified only by changing the contents of the group item EMPLOYEE ID or the record EMPLOYEE-HISTORY.

3.5.8. The JUSTIFIED Clause

Function

The JUSTIFIED clause permits alternate positioning of data within a receiving data item.

General Format



Syntax Rules

1. The JUSTIFIED clause can be specified only at the elementary item level.
2. JUST is an abbreviation for JUSTIFIED.
3. The JUSTIFIED clause cannot be specified for any data item as numeric or for which editing is specified.
4. The JUSTIFIED clause must not be specified for an index data item or a USAGE POINTER data item.

General Rules

1. When a receiving data item is described with the JUSTIFIED clause and the sending data item is larger than the receiving data item, the leftmost characters are truncated. When the receiving data item is described with the JUSTIFIED clause and it is larger than the sending data item, the data is aligned at the rightmost character position in the data item with space-fill for the leftmost character positions.

   For example, a field described as X(5) JUSTIFIED RIGHT contains:

   ΔΔABC if the literal "ABC" were moved to it, or

   BCDEF if the literal "ABCDEF" were moved to it.

   A normally justified field would contain ABCΔΔ and ABCDE if the same literals were moved to it.

2. When the JUSTIFIED clause is omitted, the standard rules for aligning data within an elementary item apply. (See Section 2.4.19).

3.5.9. Level Number

Function

The level number indicates the position of a data item within the hierarchical structure of a logical record. In addition, it is used to identify entries for working storage items, linkage items, condition names, and the RENAMES clause.

General Format

level-number

Syntax Rules

1. A level number is required as the first element in each data description entry.
2. Data description entries subordinate to an FD or SD entry must have level numbers with the values 01 through 49, 66, or 88.
3. Data description entries in the Working-Storage Section, Common-Storage Section, and Linkage Section must have level numbers 01 through 49, 66, 77, or 88.

General Rules

1. The level number 01 identifies the first entry in each record description or a report group.
2. Special level numbers have been assigned to certain entries where there is no real concept of hierarchy:
   a. Level number 77 is assigned to identify noncontiguous data items and can be used only as described by format 1 of the data description entry. (See Section 3.5.5.)
   b. Level number 66 is assigned to identify RENAMES entries and can be used only as described by format 2 of the data description entry. (See Section 3.5.5.)

c. Level number 88 is assigned to entries that define condition names associated with a conditional variable and can be used only as described by format 3 of the data description entry. (See Section 3.5.5.)

3. Multiple level 01 entries subordinate to any given level indicator, other than the level indicator RD for a report description entry, represent implicit redefinitions of the same area.

3.5.10. The OCCURS Clause

Function

The OCCURS clause eliminates the need for separate entries of repeated data items and supplies information required for the application of subscripts.

General Format

Format 1

```
OCCURS integer-2 TIMES
  [ { ASCENDING  } KEY IS { data-name-2 } ... ]  ...
    { DESCENDING }
[ INDEXED BY { index-name-1 } ... ]
```

Format 2

```
OCCURS integer-1 TO integer-2 TIMES DEPENDING ON data-name-1
  [ { ASCENDING  } KEY IS { data-name-2 } ... ]  ...
    { DESCENDING }
[ INDEXED BY { index-name-1 } ... ]
```

Syntax Rules

1. The OCCURS clause must not be specified in a data description entry that has:
   a. A level number of 01, 66,77, or 88
   b. A variable occurrence data item subordinate to it
2. *Data-name-1* and *data-name-2* can be qualified.
3. The first specification of *data-name-2* must be the name of either the entry containing the OCCURS clause or an entry subordinate to the entry containing the OCCURS clause. Subsequent specification of *data-name-2* must be subordinate to the entry containing the OCCURS clause.
4. *Data-name-2* must be specified without the subscripting normally required.
5. Where both *integer-1* and *integer-2* are used, *integer-1* must be greater than or equal to zero and *integer-2* must be greater than *integer-1.*
6. *Data-name-1* must describe an integer.

7. In format 2, the data item defined by *data-name-1* must not occupy a character position within the range of the first character position defined by the data description entry containing the OCCURS clause and the last character position defined by the record description entry containing that OCCURS clause.
8. If the OCCURS clause is specified in a data description entry included in a record description entry containing the EXTERNAL clause, *data-name-1,* if specified, must refer to a data item possessing the external attribute described in the same Data Division.
9. If the OCCURS clause is specified in a data description entry subordinate to one containing the GLOBAL clause, *data-name-1,* if specified, must be a global name and must refer to a data item described in the same Data Division.
10. A data description entry that contains format 2 of the OCCURS clause can be followed, within that record description, only by data description entries subordinate to it.
11. The data item identified by *data-name-2* must not contain an OCCURS clause except when *data-name-2* is the subject of the entry.
12. There must not be any entry that contains an OCCURS clause between the descriptions of the data items identified by the data names in the KEY IS phrase and the subject of the entry.
13. An INDEXED BY phrase is required if the subject of this entry, or an entry subordinate to this entry, is to be referred to by indexing. The index name identified by this clause is not defined elsewhere since its allocation and format depend on the hardware and, not being data, cannot be associated with any data hierarchy. An index name is allocated as a one-word (36-bit) binary field.
14. *Index-name-1* must be a unique word within the program.

General Rules

1. Except for the OCCURS clause itself, all data description clauses associated with an item whose description includes an OCCURS clause apply to each occurrence of the item described.
2. The number of occurrences of the subject entry is defined as follows:
   a. In format 1, the value of *integer-2* represents the exact number of occurrences.
   b. In format 2, the current value of the data item referenced by *data-name-1* represents the number of occurrences.

   This format specifies that the subject of this entry has a variable number of occurrences. The value of *integer-2* represents the maximum number of occurrences and the value of *integer-1* represents the minimum number of occurrences. This does not imply that the length of the subject of the entry is variable, but that the number of occurrences is variable.

   When the subject of entry is referenced, or any data item subordinate or super ordinate to the subject of entry is referenced, the value of the data item referenced by *data-name-1* must fall within the range *integer-1* through *integer-2.* The contents of the data items whose occurrence numbers exceed the value of the data item referenced by *data-name-1* are undefined.

3. When a group item referred to has a subordinate entry specifying format 2 of the OCCURS clause, the part of the table area used in the operation is determined as follows:

   a. If the data item referred to by data-name-1 is outside the group, only that part of the table area that is specified by the value of the data item referred to by data-name-1 at the start of the operation is used.

   b. If the data item referred to by data-name-1 is included in the same group and the group data item is referred to as a sending item, only that part of the table area that is specified by the value of the data item referred to by data-name-1 at the start of the operation is used in the operation. If the group is a receiving item, the maximum length of the group is used.

4. When the KEY IS phrase is specified, the repeated data must be arranged in ascending or descending order according to the values contained in data-name-2. The ascending or descending order is determined according to the rules for comparison of operands (see Section a.). The data names are listed in their descending order of significance.

5. If format 2 is specified in a record description entry and the associated file description or sort-merge description entry contains the VARYING phrase of the RECORD clause, the records are of variable length. If the DEPENDING ON phrase of the RECORD clause is not specified, the content of the data item referred to by data-name-1 of the OCCURS clause must be set to the number of occurrences to be written before the execution of any RELEASE, REWRITE, or WRITE statement.

The statements affected by the OCCURS DEPENDING ON clause are

```
ACCEPT                  RELEASE
CALL                    RETURN . . . INTO
CANCEL                  REWRITE
DISPLAY                 SEARCH (condition only)
IF                      SOURCE
INSPECT                 STRING
MOVE                    UNSTRING
PERFORM                 WRITE
READ . . . INTO
```

The next example illustrates the effects of the OCCURS DEPENDING ON clause in a WRITE . . . FROM . . . statement.

```
IDENTIFICATION DIVISION.
PROGRAM-ID.  ODO-EXAMPLE.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER.  UNISYS-2200.
OBJECT-COMPUTER.  UNISYS-2200.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
    SELECT FILEA ASSIGN TO PRINTER PRINT-FILE.
DATA DIVISION.
FILE SECTION.
FD FILEA
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS PRINT-RECORD.
01 PRINT-RECORD.
   02 PRINT-SUB PIC 99.
   02 PRINT-ODO-GROUP.
      03 PRINT-ODO-ITEM OCCURS 1 TO 13
          TIMES DEPENDING ON PRINT-SUB.
          04 PRINT4  PIC X(4).
          04 PRINT6  PIC X(6).
WORKING-STORAGE SECTION.
01 ODO.
    02 ODO-SUB PIC 99.
    02 ODO-GROUP.
       03 ODO-ITEM OCCURS 1 TO 5 TIMES
           DEPENDING ON ODO-SUB.
           04 ODO6 PIC X(6).
           04 ODO4 PIC X(4).
PROCEDURE DIVISION.
P1.
    OPEN OUTPUT FILEA.
    MOVE 13 TO PRINT-SUB.
    MOVE ALL "A" TO PRINT-ODO-GROUP.
    WRITE PRINT-RECORD.
P2.
    MOVE 3 TO ODO-SUB.
    MOVE 2 TO PRINT-SUB.
    MOVE ALL "1234567890" TO ODO-GROUP.
    WRITE PRINT-RECORD FROM ODO.
    CLOSE FILEA.
P3.
    STOP RUN.
```

After execution of this program, the external file PRINT-FILE contains two records. The first record contains the characters “13” followed by 130 occurrences of the letter “A”. The second record contains the characters “03” followed by three occurrences of the string “1234567890”. Note that the value contained in PRINT-SUB before the execution of the statement WRITE PRINT-RECORD FROM ODO does not determine the number of occurrences of PRINT-ODO-ITEM written to FILEA.

3.5.11. The PICTURE Clause

Function

The PICTURE clause describes the general characteristics and editing requirements of an elementary item.

General Format



Syntax Rules

1. A PICTURE clause can be specified only at the elementary item level.
2. *Character-string* consists of certain allowable combinations of characters in the COBOL character set used as symbols. The allowable combinations determine the category of the elementary item.
3. The lowercase letters corresponding to the uppercase letters representing the PICTURE symbols A, B, P, S, V, X, Z, CR, and DB are equivalent to their uppercase representations in a PICTURE character string. No other lowercase letters are equivalent to their corresponding uppercase representation.
4. The maximum number of characters allowed in *character-string* is 30.
5. The PICTURE clause must be specified for every elementary item except an index data item, a floating-point data item, or the subject of the RENAMES clause. In these cases, the use of this clause is prohibited.
6. PIC is an abbreviation for PICTURE.
7. The asterisk, when used as the zero suppression symbol, and the clause BLANK WHEN ZERO cannot appear in the same entry.

General Rules

1. There are five categories of data that can be described with a PICTURE clause: alphabetic, numeric, alphanumeric, alphanumeric edited, and numeric edited.
2. To define an item as alphabetic
   a. Its PICTURE character string can contain only the symbol A.
   b. Its contents, when represented in standard data format, must be one or more alphabetic characters.
   c. An alphabetic item can have only DISPLAY usage associated with it. AAA is an example of a valid alphabetic PICTURE string.

d. Its PICTURE character string can contain only the symbols 9, P, S, and V. The number of digit positions that can be described by the PICTURE character string must range from 1 to 18 inclusive.

e. If unsigned, its contents when represented in standard data format must be one or more numeric characters; if signed, the item can also contain a "+", "-", or other representation of an operational sign. (See Section 3.5.14.)

SPP9(4) and 9V99 are valid numeric PICTURE strings.

3. To define an item as exact binary

a. When a PICTURE string contains at least one character 1 and no more than one operational sign S and no other symbols, it defines an exact binary data item. Exact binary data items must have an associated USAGE IS BINARY-1 clause.

b. The total allocated size of an exact binary field cannot exceed 72 bits if signed or 71 bits if unsigned. If the operational sign is present, the first bit is treated as a sign indicator with the remainder of the field assumed to be magnitude information. Therefore, a PICTURE string S1(4) has one sign bit and three magnitude bits.

S1(3) and 1111 are valid exact binary strings.

4. To define an item as alphanumeric

a. Its PICTURE character string is restricted to certain combinations of the symbols A, X, 9, and the item is treated as if the character string contained all Xs. A PICTURE character string that contains all As or all 9s does not define an alphanumeric item.

b. Its contents, when represented in standard data format, must be one or more characters in the computer's character set.9XX9, AAXX, and A9(3) are all valid alphanumeric picture strings. Furthermore, all three are equivalent and are treated as if X(4) had been written. An alphanumeric item has DISPLAY usage.

5. To define an item as alphanumeric edited

a. Its PICTURE character string is restricted to certain combinations of the following symbols: A, X, 9, B, 0, and / ; and must contain at least one A or X and at least one B, 0 (zero), or / (slant).

b. Its contents, when represented in standard data format, must be two or more characters in the computer's character set. XBBX and XB(15)X0(10) are valid alphanumeric edited PICTURE strings. An alphanumeric-edited field has DISPLAY usage.

6. To define an item as numeric edited

a. Its PICTURE character string is restricted to certain combinations of the symbols B, /, P, V, Z, 0, 9, ,, ., *, +, -, CR, DB, and the currency symbol. The allowable combinations are determined from the order of precedence of symbols and the editing rules:

- The number of digit positions that can be represented in the PICTURE character string must range from 1 to 18 inclusive.
- The character string must contain at least one 0, B, /, Z *, +, ,, ., -, CR, DB, or the currency symbol.

b. The content of each of the character positions must be consistent with the corresponding PICTURE symbol.

   A numeric-edited item must have DISPLAY usage associated with it.

   $$$, $$9.99CR, and -ZZZ are valid numeric-edited picture strings.

7. The size of an elementary item, where size means the number of character positions occupied by the elementary item in standard data format, is determined by the number of allowable symbols that represent character positions. An unsigned nonzero integer enclosed in parentheses following the symbols A, ,, X, 1, 9, P, Z, *, B, /, 0, +, -, or the currency symbol indicates the number of consecutive occurrences of the symbol. Note that the following symbols can appear only once in a given PICTURE: S, V, ., CR, and DB.

   The following examples illustrate the use of replication symbols:

   a. AAAAAA is equivalent to A(6)

   b. PPPP99999 is equivalent to P(4)9(5)

   c. S999V9999 is equivalent to S999V9(4)

8. The functions of the symbols used to describe an elementary item are:

   A

   Each A in the character string represents a character position that can contain only an alphabetic character and is counted in the size of the item.

   B

   Each B in the character string represents a character position into which the space character is inserted and is counted in the size of the item.

   P

   Each P in the character string indicates an assumed decimal scaling position and is used to specify the location of an assumed decimal point when the point is not within the number that appears in the data item. The scaling position character P is not counted in the size of the data item. Scaling position characters are counted in determining the maximum number of digit positions (18) in numeric edited items or numeric items. The scaling position character P can appear only to the left or right as a continuous string of Ps within a PICTURE description; since the scaling position character P implies an assumed decimal point (to the left of the Ps if the Ps are the leftmost PICTURE symbols and to the right if the Ps are the rightmost PICTURE symbols), the assumed decimal point symbol V is redundant as either the leftmost or rightmost character within such a PICTURE description. The symbol P and the insertion symbol . (period) cannot both occur in the same PICTURE character string.

In certain operations that refer to a data item whose PICTURE character string contains the symbol P, the algebraic value of the data item is used rather than the actual character representation of the data item. This algebraic value assumes the decimal point in the prescribed location and zero in place of the digit position specified by the symbol P. The size of the value is the number of digit positions represented by the PICTURE character string. These operations are any of the following:

- Any operation requiring a numeric sending operand
- A MOVE statement where the sending operand is numeric and its PICTURE character string contains the symbol P
- A MOVE statement where the sending operand is numeric edited and its PICTURE character string contains the symbol P and the receiving operand is numeric or numeric edited
- A comparison operation where both operands are numeric. In all other operations, the digit positions specified with the symbol P are ignored and are not counted in the size of the operand

The following example:

```
02 FLD-1 PIC SPPP99.
02 FLD-2 PIC P999.
02 FLD-3 PIC 999PP.
02 FLD-4 PIC 9(5)V9(5).
       .
       .
       .
MOVE .00123 TO FLD-1, FLD-2.
MOVE 12345 TO FLD-3.
ADD FLD-1, FLD-2, FLD-3 GIVING FLD-4.
```

results in the following:

```
FLD-1 would contain 23
FLD-2 would contain 012
FLD-3 would contain 123
FLD-4 would contain 12300.00143  (.00023 + .0012 + 12300)
```

S

The letter S is used in a character string to indicate the presence, but neither the representation nor, necessarily, the position, of an operational sign. It must be written as the leftmost character in the PICTURE. The S is not counted in determining the size (in terms of standard data format characters) of the elementary item unless the entry is subject to a SIGN clause that specifies the optional SEPARATE CHARACTER phrase. (See Section 3.5.14.)

The operational sign is used to indicate whether the value of a data item can assume negative, as well as positive, values. Unsigned fields are treated conceptually as positive.

The status of an operational sign can be printed by using editing sign control characters. The storage representation of the operational sign of an exact binary field is the high-order bit; 1's complement form is used for negative values. The default storage representation of the operational sign of a decimal character field is an "overpunch" on the low-order character. The actual octal value of a character appears in the low-order character representation of a signed positive, signed negative, and unsigned numeric field, as shown in Table 3–1. Also shown in the table is the character that is printed or displayed if a numeric field is not edited prior to output.

Table 3–1. Internal Representation of Overpunched Operational Signs

Digit	Unsigned		Signed Positive		Signed Negative	
	ASCII	Character	ASCII	Character	ASCII	Character
0	060	0	173,77	{ , ?	175,41	} , !
1	061	1	101	A	112	J
2	062	2	102	B	113	K
3	063	3	103	C	114	L
4	064	4	104	D	115	M
5	065	5	105	E	116	N
6	066	6	106	F	117	O
7	067	7	107	G	120	P
8	070	8	110	H	121	Q
9	071	9	111	I	122	R

Note: *There are two internal sign representations for ASCII: signed positive and signed negative. Both are valid and both are processed correctly.*

The ASCII graphic symbols ? and { represent overpunched positive 0.

The ASCII graphic symbols ! and } represent overpunched negative 0.

S

The presence or absence of an S in the PICTURE of a numeric item has the following effects:

1. If the S is written for a decimal field, an unsigned value is always handled correctly. The unsigned representation also can occur and is a valid representation of a positive value in a field described as signed. If the S is omitted from a decimal field, a signed value is not always handled correctly. An unsigned field is always handled correctly when it is modified by direct references to its elementary level. If, however, an unsigned item is contained in a group item that is read into, moved to, or redefined, it is possible that the sign position can be incorrect and thereby impact later calculations that reference the item.
2. The result of an arithmetic or MOVE statement is considered signed if any input operand is signed or if the arithmetic verb is SUBTRACT. When a signed result is stored in an unsigned item, the absolute value is computed and stored.
3. For an exact binary item, presence of the S determines whether the high-order bit is treated as a sign bit. Therefore, an exact binary PICTURE character string S1111 can contain a value ranging from -7 to +7 while the PICTURE character string 1111 can contain values ranging from 0 to 15.

V

The V is used in a character string to indicate the location of the assumed decimal point and can appear only once in a character string. The V does not represent a character position and therefore is not counted in the size of the elementary item. When the assumed decimal point is to the right of the rightmost symbol in the string representing a digit position or scaling position, the V is redundant.

X

Each X in the character string is used to represent a character position that contains any allowable character from the computer's character set and is counted in the size of the item.

Z

Each Z in a character string can be used only to represent the leftmost leading numeric character positions that are replaced by a space character when the content of that character position is a leading zero. Each Z is counted in the size of the item.

1

Each 1 in the character string represents a bit position in an exact binary data item and is counted in the size of the item. An exact binary PICTURE character string can contain only the symbols S and 1.

A maximum of 72 bit positions is allowed for signed strings. A maximum of 71 bit positions is allowed for unsigned strings, and they must specify USAGE IS BINARY-1.

9

Each 9 in the character string represents a digit position that contains a numeric character and is counted in the size of the item.

0

Each zero (0) in the character string represents a character position into which the character zero is inserted. The 0 is counted in the size of the item.

/

Each slant (/) in the character string represents a character position into which the slant character is inserted. The slant is counted in the size of the item.

,

Each comma (,) in the character string represents a character position into which the character, is inserted. This character position is counted in the size of the item.

.

When the period (.) appears in the character string, it is an editing symbol that represents the decimal point for alignment purposes and, in addition, represents a character position into which the character "." is inserted.

The character period (.) is counted in the size of the item. For a given program, the functions of the period and comma are exchanged if the clause DECIMAL-POINT IS COMMA is stated in the SPECIAL-NAMES paragraph. In this exchange, the rules for the period apply to the comma and the rules for the comma apply to the period wherever they appear in a PICTURE clause.

\+ - CR DB

These symbols are used as editing sign control symbols. When used, they represent the character position into which the editing sign control symbol is placed. The symbols are mutually exclusive in any one character string and each character used in the symbol is counted in determining the size of the data item.

*

Each asterisk (*) in the character string represents a leading numeric character position into which an asterisk will be placed when the content of that position is a leading zero. Each asterisk is counted in the size of the item.

cs

The currency symbol in the character string represents a character position into which a currency symbol is to be placed. The currency symbol in a character string is represented by either the currency sign or by the single character specified in the CURRENCY SIGN clause in the SPECIAL-NAMES paragraph. The currency symbol is counted in the size of the item.

The alternate currency sign can be any valid character except the digits 0 through 9, the alphabetic characters A, B, C, D, P, R, S, V, X, Z, or space, and the special characters = * + - , . ; ( ) " /.

The currency symbol ($) is used in the following discussion to indicate either the actual $ or the alternate currency sign specified in the CURRENCY SIGN clause.

Editing Rules

1. There are two general methods of performing editing in the PICTURE clause: insertion, or suppression and replacement. Four types of insertion editing are available:
   a. Simple insertion
   b. Special insertion
   c. Fixed insertion
   d. Floating insertion
2. Two types of suppression and replacement editing are available:
   a. Zero suppression and replacement with spaces
   b. Zero suppression and replacement with asterisks
3. The type of editing that can be performed on an item depends on the category to which the item belongs. The following table specifies which type of editing can be performed on a given category.

Category	Type of Editing
Alphabetic	None
Numeric	None
Alphanumeric	None
Alphanumeric edited	Simple insertion “0”, “B” and “/”
Numeric edited	All, subject to rules in rule 3

4. Floating insertion editing and editing by zero suppression and replacement are mutually exclusive in a PICTURE clause. Only one type of replacement can be used with zero suppression in a PICTURE clause.
5. Simple insertion editing

   The comma (,), space (B), zero (0), and slant (/) are used as the insertion characters. The insertion characters are counted in the size of the item and represent the position in the item into which the character is inserted. If the insertion character comma (,) is the last symbol in the PICTURE character string, the PICTURE clause must be the last clause of the data description entry and must be immediately followed by the separator period. This results in the combination of appearing in the data description entry, or, if the DECIMAL-POINT IS COMMA clause is used in two consecutive periods.

The only circumstance when simple insertion symbols cannot appear in a numeric edited field is when simple insertion characters are either embedded in or preceded on the left by suppression and replacement of floating insertion symbols. In this case, the simple insertion characters are suppressed and replaced if the digit position to their left is suppressed and replaced.

For example, a numeric-edited field defined as:

$*** , *** . 99

contains:

$**1,000.00

if the literal 1000 is moved to it or:

$****100.00

if the literal 100 is moved to it.

6. Special insertion editing

   The period (.) is used as the insertion character. In addition to being an insertion character, it also represents the decimal point for alignment purposes. The insertion character used for the actual decimal point is counted in the size of the item. The use of the assumed decimal point, represented by the symbol V, and the actual decimal point, represented by the insertion character, in the same PICTURE character string is disallowed. If the insertion character is the last symbol in the PICTURE character string, the PICTURE clause must be the last clause of that data description entry and must be immediately followed by the separator period. This results in two consecutive periods appearing in the data description entry, or in the combination of ,. if the DECIMAL-POINT IS COMMA clause is used. The result of special insertion editing is the appearance of the insertion character in the item in the same position as shown in the character string.

   The period (or the comma if DECIMAL-POINT IS COMMA is specified in the SPECIAL-NAMES paragraph) is the only special insertion symbol.

   The result of special insertion editing is the appearance of the insertion character in the item in the same position as shown in the character string. The only exception is if a PICTURE string uses only floating insertion symbols (+, -, or currency sign) or the suppression/replacement symbol Z to represent the capacity of an edited field and the value being edited to zero, the entire item, including the decimal point, contains spaces.

   For example, moving zero to a field defined by the picture string $$$, $$$. $$ results in 10 spaces. The effect of moving zero to a field defined by *** , *** . ** is the result *******.**.

7. Fixed insertion editing

   The currency symbol and the editing sign control symbols, +, -, CR and DB, are the insertion characters. Only one currency symbol and only one of the editing sign control symbols can be used in a given PICTURE character string. When the symbols CR or DB are used, they represent two character positions in determining the size of the item and they must represent the rightmost character positions that are counted in the size of the item. If these character positions contain the symbols CR or DB, the uppercase letters are the insertion characters.

The symbols + or - when used, must be either the leftmost or rightmost character position to be counted in the size of the item. The currency symbol must be the leftmost character position to be counted in the size of the item except that it can be preceded by either a + or a -symbol. Fixed insertion editing results in the insertion character occupying the same character position in the edited item as it is occupied in the PICTURE character string.

Editing sign control symbols produce the following results depending upon the value of the data item.

Editing Symbol in Picture Character-String	Result	
	Data Item Positive or Zero	Data Item Negative
+	+	-
-	Space	-
CR	two spaces	CR
DB	two spaces	DB

The multiple consecutive occurrence of the currency sign, the +, or the - implies floating insertion editing and must abide by the rules for that type of insertion editing. No PICTURE character string can specify two separate types of sign control editing.

8. Floating insertion editing

   The currency symbol and editing sign control symbols + or - are the floating insertion characters and as such are mutually exclusive in a given PICTURE character string.

   Floating insertion editing is indicated in a PICTURE character string by a string of at least two of the floating insertion characters. This string of floating insertion characters can contain any of the simple insertion characters or have simple insertion characters immediately to the right of this string. These simple insertion characters are part of the floating string. When the floating insertion character is the currency symbol, this string of floating insertion characters can have the fixed insertion characters CR and DB immediately to the right of this string.

   The leftmost character of the floating insertion string represents the leftmost limit of the floating symbols in the data item. The rightmost character of the floating string represents the rightmost limit of the floating symbols in the data item.

   The second floating character from the left represents the leftmost limit of the numeric data that can be stored in the data item. Nonzero numeric data can replace all the characters at or to the right of this limit.

   In a PICTURE character string, there are only two ways of representing floating insertion editing. One way is to represent any or all of the leading numeric character positions on the left of the decimal point by the insertion character. The other way is to represent all of the numeric character positions in the PICTURE character string by the insertion character.

In other words, a numeric edited picture string that represents any digit positions to the right of a decimal point indicator with the symbol 9 must represent all fractional digits with the symbol 9.

If the insertion character positions are only to the left of the decimal point in the PICTURE character string, the result is that a single floating insertion character is placed into the character position immediately preceding either the decimal point or the first nonzero digit in the data represented by the insertion symbol string, whichever is farther to the left in the PICTURE character string. The character positions preceding the insertion character are replaced with spaces.

For example, assume the following PICTURE character string:

```
  $$$,$$$.99
```

If 10000, 1000, 100 and .01, respectively, are moved to the edited field, it contains the following values:

```
$10,000.00
 $1,000.00
   $100.00
      $.01
```

If all numeric character positions in the PICTURE character string are represented by the insertion character, at least one of the insertion characters must be to the left of the decimal point.

When the floating insertion character is the editing control symbol + or - , the character inserted depends on the value of the data item.

Editing Symbol in Picture Character-String	Result	
	Data Item Positive or Zero	Data Item Negative
+	+	-
-	space	-

If all numeric character positions in the PICTURE character string are represented by the insertion character, the result depends on the value of the data. If the value is zero, the entire data item contains spaces. If the value is not zero, the result is the same as when the insertion character is only to the left of the decimal point.

For example, a field defined by the PICTURE character string

```
  ++,+++.++
```

contains the following values:

```
+1,000.00
  +100.00
    +1.00
     +.01
```

if 1000, 100, 1, 0, and .01, respectively, are moved to the edited field.

To avoid truncation, the minimum size of the PICTURE character string for the receiving data item must be the number of characters in the sending data item, plus the number of nonfloating insertion characters being edited into the receiving data item, plus one for the floating insertion character. If truncation does occur, the value of the data that is used for editing is the value after truncation (see Section 2.4.19).

In a given PICTURE character string, floating insertion and fixed insertion of the same symbol are disallowed. Also illegal is the specification of more than one type of floating insertion editing, or a mixture of floating insertion and suppression and replacement editing. In addition, a PICTURE character string can contain only one type of sign control editing or none.

9. Zero suppression editing

   The suppression of leading zeros in numeric character positions is indicated by the use of the alphabetic character Z or the asterisk (*) as suppression symbols in a PICTURE character string. These symbols are mutually exclusive in a given PICTURE character string. Each suppression symbol is counted in determining the size of the item. If Z is used, the replacement character is the space and if the asterisk is used, the replacement character is *.

   Zero suppression and replacement are indicated in a PICTURE character string by using a string of one or more of the allowable symbols to represent leading numeric character positions that are to be replaced when the associated character position in the data contains a leading zero. Any of the simple insertion characters embedded in the string of symbols or to the immediate right of this string is part of the string.

   In a PICTURE character string, there are only two ways of representing zero suppression. One way is to represent any or all of the leading numeric character positions to the left of the decimal point by suppression symbols. The other way is to represent all of the numeric character positions in the PICTURE character string by suppression symbols.

   If the suppression symbols appear only to the left of the decimal point, any leading zero in the data that corresponds to a symbol in the string is replaced by the replacement character. Suppression terminates at the first nonzero digit in the data represented by the suppression symbol string or at the decimal point, whichever is encountered first.

   If all numeric character positions in the PICTURE character string are represented by suppression symbols and the value of the data is not zero, the result is the same as if the suppression characters were only to the left of the decimal point. If the value is zero and the suppression symbol is Z, the entire data item, including any editing characters, is spaces. If the value is zero and the suppression symbol is *, the data item, including any insertion editing symbols except the actual decimal point, is *. In this case, the actual decimal point will appear in the data item.

10. The symbols +, - , *, Z, and the currency symbol, when used as floating replacement characters, are mutually exclusive within a given character string.

Precedence Rules

Figure 3–1 shows the order of precedence for symbols in a PICTURE character string. An X at an intersection indicates that the symbols at the top of the column can precede (but not necessarily immediately) the symbols at the left of the row in a given character string. Arguments appearing in braces ({ }) indicate that the symbols are mutually exclusive. The currency symbol is indicated by the symbol *cs.*

At least one of the symbols A, X, Z, 1, 9, or *, or at least two occurrences of one of the symbols +, - , or *cs* must be present in a PICTURE character string.

Nonfloating insertion symbols + and - , floating insertion symbols Z, *, +, - , and *cs,* and other symbol P appear twice in Figure 3–1. The leftmost column and uppermost row for each symbol represent its use to the left of the decimal point position. The second appearance of the symbol in the chart represents its use to the right of the decimal point position.

First Symbol / Second Symbol		Nonfloating Insertion Symbols									Floating Insertion Symbols						Other Symbols						
		B	0	/	,	.	{+ -}	{+}	{CR DB}	cs	{Z +-}	{Z -}	{+ -}	{+}	cs	cs	9	A X	S	V	P	P	1
Nonfloating Insertion Symbols	B	X	X	X	X	X	X			X	X	X	X	X	X	X	X	X		X		X	
	0	X	X	X	X	X	X			X	X	X	X	X	X	X	X	X		X		X	
	/	X	X	X	X	X	X			X	X	X	X	X	X	X	X	X		X		X	
	,	X	X	X	X	X	X			X	X	X	X	X	X	X	X			X		X	
	.	X	X	X	X		X			X	X		X		X		X						
	{+ -}																						
	{+}	X	X	X	X	X				X	X	X			X	X	X			X	X	X	
	{CR DB}	X	X	X	X	X				X	X	X			X	X	X			X	X	X	
	cs						X																
Floating Insertion Symbols	{Z +-}	X	X	X	X		X			X	X												
	{Z -}	X	X	X	X	X	X			X	X	X								X		X	
	{+ -}	X	X	X	X					X			X										
	{+}	X	X	X	X	X				X			X	X						X		X	
	cs	X	X	X	X		X								X								
	cs	X	X	X	X	X	X								X	X				X		X	
Other Symbols	9	X	X	X	X	X	X			X	X		X		X		X	X	X	X		X	
	A X	X	X	X													X	X					
	S																						
	V	X	X	X	X		X			X	X		X		X		X		X		X		
	P	X	X	X	X		X			X	X		X		X		X		X		X		
	P						X			X									X	X		X	
	1																		X				X

NOTE: PICTURE 1 is a Unisys extension

Figure 3–1. PICTURE Character Precedence Chart

Table 3–2 and Table 3–3 summarize the nonediting and editing picture strings.

Editing Symbol in Picture Character String	Result	
	Data Item Positive or Zero	Data Item Negative
+	+	-
-	Space	-
CR	two spaces	CR
DB	two spaces	DB

Table 3–2. Examples of Nonediting PICTURE Strings

Picture	Field in Main Storage	Edited Field	Class
99999	56789	56789	Numeric
9(3)V9(2)	56789	567.89	Numeric
S999V99	(-)56789	(-)567.89	Numeric
SV9(5)	(+)56789	(+).56789	Numeric
S111	binary 011	(+)3	Exact Binary Numeric
XXXXX	ABCDE	ABCDE	Alphanumeric
AAAAA	BCDEF	BCDEF	Alphabetic
X(5)	12345	12345	Alphanumeric
99X99	67.89	67.89	Alphanumeric
99AA9	23FG4	23FG4	Alphanumeric
A(2)X(3)	CD2M3	CD2M3	Alphanumeric
99P(3)	34	34000	Numeric
P(3)99	34	00034	Numeric
P(3)9(2)	34	00034	Numeric

Note: *The (+) and (-) indicate the status of the operational sign of the numeric value.*

Table 3–3. Examples of Editing PICTURE Strings

Source Area		Receiving Area	
Picture	**Field in Main Storage**	**Picture**	**Edited Field**
9(5)	45678	$ZZ,ZZ9.99	$45,678.00
9(3)V99	45678	$ZZ,ZZ9.99	$ 456.78
9(3)V99	00067	+ZZ,ZZZ.ZZ	+.67
9(3)V99	00004	+ZZ,ZZZ.ZZ	+.04
9(5)	00000	ZZ,ZZZ.ZZ	
9(5)	00000	**,***.**	******.**
S9(4)	(-)5678	$$,$$$.99BCR	$5,678.00 CR
S9PP	(-)3	$$,$$$.99DB	$300.00DB
SPP99	(+)23	+++.9999	+.0023
SPP99	(+)23	+++.++++	+.0023
S9(4)	(-)1234	- -900PP	-1200
S9(3)99	(+)56789	999B999B99-	000 567 89
999	567	9/9/9	5/6/7
AAA	XYZ	XBXBXB00	X Y Z 00
X(5)	12345	X0X0X0XX	10203045
X(5)	12345	005BB	123
9(5)	56789	$***.00	$789.00

Note: *The (+) and (-) indicate the status of the operational sign of the numeric value.*

3.5.12. The REDEFINES Clause

Function

The REDEFINES clause allows the same computer storage area to be described by different data description entries.

General Format

```
level-number  [ data-name-1 ]  REDEFINES data-name-2
              [ FILLER      ]
```

Note: *Level-number, data-name-1, and FILLER are shown in this format to improve clarity. Level-number, data-name-1, and FILLER are not part of the REDEFINES clause.*

Syntax Rules

1. The REDEFINES clause, when specified, must immediately follow the subject of the entry.
2. The level numbers of *data-name-2* and the subject of the entry must be identical, but must not be 66 or 88.
3. This clause must not be used in level 01 entries in the File Section.
4. The data description entry for *data-name-2* cannot contain an OCCURS clause. However, *data-name-2* can be subordinate to an item whose data description entry contains an OCCURS clause. In this case, the reference to *data-name-2* in the REDEFINES clause cannot be subscripted. Neither the original definition nor the redefinition can include a variable occurrence data item.
5. If the data item referred to by *data-name-2* is either declared to be an external data record or is specified with a level number other than 01, the number of character positions it contains must be greater than or equal to the number of character positions in the data item referred to by the subject of this entry. If the data name referred to by *data-name-2* is specified with a level number of 01 and is not declared to be an external data record, there is no such constraint.
6. *Data-name-2* must not be qualified even if it is not unique since no ambiguity of reference exists in this case because of the required placement of the REDEFINES clause within the source program.
7. Multiple redefinitions of the same character or bit positions are permitted. Multiple redefinitions of the same character or bit positions must all use the data name of the entry that originally defined the area.
8. The entries giving the new description of the character positions must not contain any VALUE clauses, except in condition name entries.
9. No entry having a level number numerically lower than the level number of *data-name-2* and the subject of the entry can occur between the data description entries of *data-name-2* and the subject of the entry.
10. The entries giving the new descriptions of the character or bit positions must follow the entries defining the area of *data-name-2,* without intervening entries that define new character or bit positions.
11. In level 1, *data-name-2* cannot be subordinate to any entry that contains a REDEFINES clause. In level 2, *data-name-2* can be subordinate to an entry that contains a REDEFINES clause.

General Rules

1. Storage allocation starts at *data-name-2* and continues over a storage area sufficient to contain the area of character or bit positions in the data item referred to by the *data-name-1* or FILLER clause.
2. When the same character or bit position is defined by more than one data description entry, the data name associated with any of those data description entries can be used to refer to that character or bit position.

3.5.13. The RENAMES Clause

Function

The RENAMES clause permits alternative, possibly overlapping, groupings of elementary items.

General Format

```
66 data-name-1 RENAMES data-name-2 [ { THROUGH } data-name-3 ] .
                                     { THRU    }
```

Note: *The level-number "66" and data-name-1 are shown in this format to improve clarity. The level number and data-name-1 are not part of the RENAMES clause.*

Syntax Rules

1. Any number of RENAMES entries can be written for a logical record.

   All RENAMES entries referring to data items within a given logical record must immediately follow the last data description entry of the associated record description entry.

2. *Data-name-1* cannot be used as a qualifier, and can only be qualified by the names of the associated level 01, FD, or SD entries. Neither *data-name-2* nor *data-name-3* can have an OCCURS clause in its data description entry nor can be subordinate to an item that has an OCCURS clause in its data description entry.

3. *Data-name-2* and *data-name-3* must be names of elementary items or groups of elementary items in the same logical record, and cannot be the same data name. A 66 level entry cannot rename another 66 level entry nor can it rename a 77, 88, or 01 level entry.

4. *Data-name-2* and *data-name-3* can be qualified.

5. None of the items within the range, including *data-name-2* and *data-name-3,* if specified, can be variable occurrence data items.

6. The words THROUGH and THRU are equivalent.

7. The beginning of the area described by *data-name-3* must not be to the left of the beginning of the area described by *data-name-2.* The end of the area described by *data-name-3* must be to the right of the end of the area described by *data-name-2. Data-name-3,* therefore, cannot be subordinate to *data-name-2.*

The RENAMES entry can, however, span an entire table or array. If it does, the size of the RENAMES item includes any unused main storage introduced by elementary or group item synchronization.

For example the following sequence is legal:

```
02 B
   03 C OCCURS 3 TIMES
   03 D OCCURS 3 TIMES
      04 E
      04 F
   03 G
02 H
66 J RENAMES B THRU H
```

However, the following sequence alone is not correct.

```
66 K RENAMES C THRU F
```

The following sequence results in K being the first six characters in either A or B, and L being the third through seventh character in either A or B.

```
02 A.
   03 D PIC X(2).
   03 C PIC X(2).
   03 G
      04 E PIC X(2).
      04 F PIC X(2).
02 B REDEFINES A.
   03 H PIC X(3).
   03 I PIC X(4).
   03 J PIC X.
 66 K RENAMES D THRU E.
 66 L RENAMES C THRU I.
```

General Rules

1. When *data-name-3* is specified, *data-name-1* is a group item that includes all elementary items starting with *data-name-2* (if *data-name-2* is an elementary item) or the first elementary item in *data-name-2* (if *data-name-2* is a group item), and concluding with *data-name-3* (if *data-name-3* is an elementary item) or the last elementary item in *data-name-3* (if *data-name-3* is a group item).
2. When *data-name-3* is not specified, all of the data attributes of *data-name-2* become the data attributes for *data-name-1.*
3. The usage, and therefore the initial alignment, of a RENAMES entry is determined by the usage of the *data-name-2* or the first elementary item in *data-name-2* (if *data-name-2* is a group item).

   An elementary data item, or an elementary data item subordinate to a group item whose declaration contains a USAGE specifying BINARY-1, must be declared with a PICTURE character string that describes an exact binary data item, that is, a PICTURE character string that contains only the symbols S and 1 (see Section 3.5.11).

3.5.14. The SIGN Clause

Function

The SIGN clause specifies the position and the mode of representation of the operational sign when it is necessary to describe these properties explicitly.

General Format



Syntax Rules

1. The SIGN clause can be specified only for a numeric data description entry whose PICTURE contains the character S, or a group item containing at least one such numeric data description entry.
2. The numeric data description entries to which the SIGN clause applies must be described, implicitly or explicitly, as USAGE IS DISPLAY.
3. If the CODE-SET clause is specified in a file description entry, any signed numeric data description entries associated with that file description entry must be described with the SIGN IS SEPARATE clause.

General Rules

1. The optional SIGN clause, if present, specifies the position and the mode of representation of the operational sign for the numeric data description entry to which it applies, or for each numeric data description entry subordinate to the group to which it applies. The SIGN clause applies only to numeric data description entries whose PICTURE contains the character S; the S indicates the presence, but neither the representation nor, necessarily, the position, of the operational sign.
2. If a SIGN clause is specified in a group item subordinate to a group item for which a SIGN clause is specified, the SIGN clause specified in the subordinate group item takes precedence for that subordinate group item.
3. If a SIGN clause is specified in an elementary numeric data description entry subordinate to a group item for which a SIGN clause is specified, the SIGN clause specified in the subordinate elementary numeric data description entry takes precedence for that elementary numeric data item.
4. A numeric data description entry whose PICTURE contains the character S, but to which no optional SIGN clause applies, has an operational sign, but neither the representation nor, necessarily, the position of the operational sign is specified by the character S. In this (default) case, the operational sign is represented as an overpunch character (see Table 3–1). General rules 5 through 7 do not apply to such signed numeric data items.

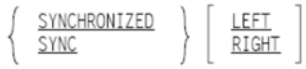
5. If the optional SEPARATE CHARACTER phrase is not present
   a. The operational sign is presumed to be associated with the leading (or, respectively, trailing) digit position of the elementary numeric data item.
   b. The letter S in a PICTURE character string is not counted in determining the size of the item (in terms of standard data format characters).
   c. The valid signs for data items are listed in Table 3–1.
6. If the optional SEPARATE CHARACTER phrase is present
   a. The operational sign is presumed to be the leading (or, respectively, trailing) character position of the elementary numeric data item. This character position is not a digit position.
   b. The letter S in a PICTURE character string is counted in determining the size of the item (in terms of standard data format characters).
   c. The operational signs for positive and negative are the standard data format characters + and -, respectively.
7. Every numeric data description entry whose PICTURE contains the character S is a signed numeric data description entry. If a SIGN clause applies to such an entry and conversion is necessary for purposes of computation or comparisons, conversion takes place automatically.

3.5.15. The SYNCHRONIZED Clause

Function

The SYNCHRONIZED clause specifies the alignment of an elementary item on the natural boundaries of the computer storage.

General Format



Syntax Rules

1. This clause can appear only with an elementary item.
2. SYNC is an abbreviation for SYNCHRONIZED.

General Rules

1. This clause specifies that the subject data item is to be aligned in the computer such that no other data item occupies any of the bit positions between the leftmost and rightmost natural boundaries delimiting this data item. If the number of bit positions required to store this data item is less than the number of bit positions between those natural boundaries, the unused bits must not be used for any other data item. Such unused bits, however, are included in:
   a. The size of any group items to which the elementary item belongs.

   b. The number of bits allocated when any such group item is the object of a REDEFINES clause. The unused bits are not included in the bits redefined when the elementary item is the object of a REDEFINES clause.

2. SYNCHRONIZED followed by neither RIGHT nor LEFT specifies that the elementary item is to be SYNCHRONIZED LEFT if alphanumeric and SYNCHRONIZED RIGHT if numeric.
3. SYNCHRONIZED LEFT specifies that the elementary item is to be positioned such that it begins at the left character position of the natural boundary in which the elementary item is placed.
4. SYNCHRONIZED RIGHT specifies that the elementary item is to be positioned such that it terminates on the right character position of the natural boundary in which the elementary item is placed.
5. Whenever a SYNCHRONIZED item is referred to in the source program, the original size of the item, as shown in the PICTURE clause, the USAGE clause, and the SIGN clause, is used in determining any action that depends on size, such as justification, truncation, or overflow.
6. If the data description of an item contains an operational sign and any form of the SYNCHRONIZED clause, the sign of the item appears in the sign position explicitly or implicitly specified by the SIGN clause.
7. When the SYNCHRONIZED clause is specified in a data description entry of a data item that also contains an OCCURS clause, or in a data description entry of a data item subordinate to a data description entry that contains an OCCURS clause

   a. Each occurrence of the data item is SYNCHRONIZED.

   b. Any implicit FILLER generated for other data items within that same table are generated for each occurrence of those data items. (See general rule 8b.)

8. This clause is hardware-dependent and, in addition to rules 1 through 7, the elementary items associated with this clause are handled as follows:

   a. The format on the external media of records or groups containing elementary items whose data description contains the SYNCHRONIZED clause is the same as the internal format.

   b. Any necessary generation of implicit FILLER, if the elementary item immediately preceding an item containing the SYNCHRONIZED clause does not terminate at a natural boundary, is not included as part of the elementary item. Such automatically generated FILLER positions are included in

      - The size of any group item to which the FILLER item belongs
      - The number of character positions allocated when the group item of which the FILLER item is a part appears as the object of a REDEFINES clause

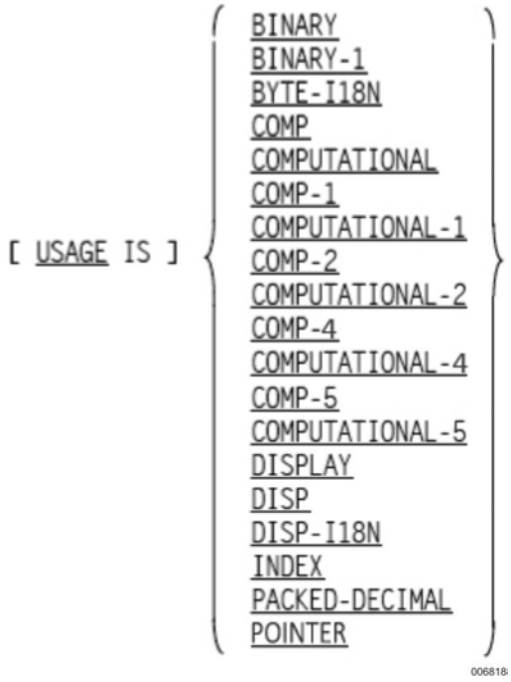
9. The presence of the SYNCHRONIZED clause has no effect on the alignment of data items.

3.5.16. The USAGE Clause

Function

The USAGE clause specifies the format of a data item in the computer storage.

General Format



Syntax Rules

1. The USAGE clause can be written in any data description entry with a level number other than 66 or 88.
2. If the USAGE clause is written in the data description entry for a group item, it can also be written in the data description entry for any subordinate elementary item or group item, but the same usage must be specified in both entries.
3. An elementary data item, or an elementary data item subordinate to a group item, whose declaration contains a USAGE clause specifying BINARY, COMPUTATIONAL, or PACKED-DECIMAL must be declared with a PICTURE character string that describes a numeric item, that is, a PICTURE character string that contains only the symbols P, S, V, and 9 (see Section 3.5.11). An elementary data item, or an elementary data item subordinate to a group item whose declaration contains a USAGE specifying BINARY-1, must be declared with a PICTURE character string that describes an exact binary data item, that is, a PICTURE character string that contains only the symbols S and 1 (see Section 3.5.11).
4. COMP, COMP-1, COMP-2, COMP-4 and DISP are abbreviations for COMPUTATIONAL, COMPUTATIONAL-1, COMPUTATIONAL-2, COMPUTATIONAL-4, and DISPLAY, respectively.

5. An index data item can be referred to explicitly only in a SEARCH or SET statement, a relation condition, the USING phrase of a Procedure Division header, or the USING phrase of a CALL statement.
6. The BLANK WHEN ZERO, JUSTIFIED, PICTURE, and VALUE clauses must not be specified for data items declared with the USAGE IS INDEX clause.
7. An elementary data item described with a USAGE IS INDEX clause must not be a conditional variable.
8. COMPUTATIONAL-1 and COMPUTATIONAL-2 usage specifications cannot appear in a data description that has a PICTURE clause, because the computer storage format of these items is fixed.
9. COMP-5 and COMPUTATIONAL-5 usage specification can appear only in data descriptions with a PICTURE clause of S9(4) or S9(9).
10. Any data item whose declaration contains a USAGE DISP-I18N clause, must be an elementary item declared with a PICTURE character string that describes an alphabetic or alphanumeric item; that is, a PICTURE character string that contains only the symbols A or X. No editing characters are allowed in a PICTURE declared with USAGE DISP-I18N.
11. Any data item whose declaration contains a USAGE BYTE-I18N must be an elementary data item declared with a PICTURE X(n).
12. In the PROCEDURE DIVISION, POINTER data items can be explicitly referred to only when
    - named as input parameters through the USING clause of the PROCEDURE DIVISION
    - used in a relation condition comparing for equal or not equal
    - moved to another POINTER item
    - used as the target or source in a format 5 SET statement
    - used as output parameters in the USING phrase of a CALL statement
    - displayed in a DISPLAY statement (the output format is octal).

General Rules

1. If the USAGE clause is written at a group level, it applies to each elementary item in the group. The USAGE clause has the following meaning for a group item characters:
    a. Any explicit usage for a group item applies to all the contained group or elementary items.
    b. The group is always considered to be an alphanumeric string of ASCII

2. The USAGE clause specifies the manner in which a data item is represented in the storage of a computer. It does not affect the use of the data item, although the specifications for some statements in the Procedure Division can restrict the USAGE clause of the operands referred to. The USAGE clause can affect the radix or type of character representation of the item. Table 3–4 describes the meaning and internal representation of data. Table 3–5 describes the alignment of data items by usage. Level 01 and level 77 data items are always aligned on word boundaries.
3. A COMPUTATIONAL, COMPUTATIONAL-1, or COMPUTATIONAL-2 item is capable of representing a value to be used in computations and must be numeric. If a group item is described as COMPUTATIONAL, COMPUTATIONAL-1, or COMPUTATIONAL-2, the elementary items in the group are COMPUTATIONAL, COMPUTATIONAL-1, or COMPUTATIONAL-2, respectively. The group item itself is not COMPUTATIONAL, COMPUTATIONAL-1, or COMPUTATIONAL-2. Such a group item cannot be used in computations.
4. A BINARY, BINARY-1, or COMPUTATIONAL-4 item is capable of representing a value used in computations and must be numeric. If a group item is described as BINARY, BINARY-1, or COMPUTATIONAL-4, the elementary items in the group are BINARY, BINARY-1, or COMPUTATIONAL-4, respectively. The group item itself is not BINARY, BINARY-1, or COMPUTATIONAL-4. Such a group item cannot be used in computations.
5. The USAGE IS BINARY, BINARY-1, or COMPUTATIONAL-4 clause specifies that the data item is represented in the storage of the computer with a radix of 2.
6. A PACKED-DECIMAL item is capable of representing a value used in computations and must be numeric. If a group item is described as PACKED-DECIMAL, the elementary items in the group are PACKED-DECIMAL. The group item itself is not PACKED-DECIMAL. Such a group item cannot be used in computations or in NUMERIC class condition tests.
7. The USAGE IS PACKED-DECIMAL clause specifies that the data item is represented in the storage of the computer with a radix of 10. PACKED-DECIMAL usage is equivalent to ASCII COBOL COMPUTATIONAL-3 usage (IBM OS/360 allocation).
8. The USAGE IS DISPLAY clause (whether specified explicitly or implicitly) specifies that a standard data format is used to represent a data item in the storage of the computer and that the data item is aligned on a character boundary.
9. If the USAGE clause is not specified for an elementary item or for any group to which the item belongs, the usage is implicitly DISPLAY.
10. The USAGE IS INDEX clause specifies that a data item is an index data item and contains a value that must correspond to an occurrence number of a table element. If a group item is described with the USAGE IS INDEX clause, the elementary items in the group are all index data items. The group itself is not an index data item and must not be used in the SEARCH or SET statement or in a relation condition, but can be referenced in the USING phrase either of a CALL statement or of the program's Procedure Division header.
11. When a MOVE statement or an input-output statement that references a group item that contains an index data item is executed, no conversion of the index data item takes place.

12. A COMPUTATIONAL-5 item depending on its picture declaration is capable of representing a half-word or full-word binary signed numeric value to be used in computations. Its main purpose is parameter passing between COBOL and C programs.
13. Whenever a data item described with USAGE DISP-I18N is explicitly used in a comparison (that is, IF, EVALUATE, or SEARCH), that comparison is done in a culturally sensitive manner (that is, the comparison is done based on the locale (LC_COLLATE) that is in force at the time the comparison is done). All other uses of data items declared as DISP-I18N are the same as if they had been declared as USAGE DISPLAY.
14. When a BYTE-I18N data item is used in a comparison, the data item is treated as a bit string and no conversion is done. For comparison of different lengths, the shorter item is padded with binary zeros on the right. A data item defined as BYTE-I18N can be the receiver in a MOVE statement only if the sender is another BYTE-I18N data item or one of the figurative constants LOW-VALUES or HIGH-VALUES. A BYTE-I18N data item can be the sender in a MOVE statement only if the receiver is also a BYTE-I18N data item. A data item described as BYTE-I18N is used to hold the results of a call to the string_transform I18NLIB service routine. Data items declared as BYTE-I18N cannot be used in any arithmetic or string manipulation statements.
15. If a group item is described with the USAGE IS POINTER clause, the elementary items within the group are pointer data items; the group itself is an alphanumeric group (not a pointer data item) and cannot be used in the syntax where a pointer data item is allowed.

    Pointer data items can be part of an alphanumeric group that is referred to in a MOVE statement or an input/output statement. However, if a pointer data item is part of a group, there is no conversion of values when the statement is executed.

    A pointer data item cannot be a conditional variable.

    A pointer data item does not belong to any class or category.

    The JUSTIFIED, PICTURE, and BLANK WHEN ZERO clauses cannot be used to describe group or elementary items defined with the USAGE IS POINTER clause.

    USAGE of POINTER is implied for the ADDRESS OF phrase.

Table 3–4. Meaning and Allocation of Data Types by USAGE

Usage	Meaning	Truncation	Allocation
DISPLAY	ASCII characters	Decimal	9 bits, quarter-word, per allocatable character represented in PICTURE clause.
DISP-I18N	9-bit characters	N/A	9 bits, quarter-word, per allocatable character represented in the PICTURE clause.

Table 3–4. Meaning and Allocation of Data Types by USAGE

Usage	Meaning	Truncation	Allocation
COMP	ASCII numeric digits	Decimal	A range of 1 to 18 digits is allowed. 9 bits, quarter-word per allocatable digit in PICTURE clause, sign is trailing in included.
BINARY	Fixed-point binary	Decimal	1-2 digits, 1/4 word, 9 bits 3-5 digits, 1/2 word, 18 bits 6-7 digits, 3/4 word, 27 bits 8-10 digits, 1 word, 36 bits 11-13 digits, 1-1/4 word, 45 bits 14-15 digits, 1-1/2 word, 54 bits 16-18 digits, 1-3/4 word, 63 bits
BINARY-1	Bits	Binary	A range of 1 to 72 bits is allowed. One bit for each bit in PICTURE clause.
PACKED-DECIMAL	Packed decimal. Each 9-bit byte consists of one slack bit and two 4-bit decimal digits, except for the right-most byte, which contains one slack bit, one 4-bit decimal digit and one 4-bit sign field. *	Decimal	1 digit, 1/4 word, 9 bits 2-3 digits, 1/2 word, 18 bits 4-5 digits, 3/4 word, 27 bits 6-7 digits, 1 word, 36 bits 8-9 digits, 1-1/4 word, 45 bits 10-11 digits, 1-1/2 word, 54 bits 12-13 digits, 1-3/4 word, 63 bits 14-15 digits, 2 words, 72 bits 16-17 digits, 2-1/4 words, 81 bits 18 digits, 2-1/2 words, 90 bits
INDEX	Index data item		36 bits, 1 word implicit synchronization
COMP-1	Single-precision signed floating point	Binary	36 bits
COMP-2	Double-precision signed floating point	Binary	72 bits
COMP-4	Fixed-point binary	Decimal	1-2 digits, 1/4 word, 9 bits 3-5 digits, 1/2 word, 18 bits 6-7 digits, 3/4 word, 27 bits 8-10 digits, 1 word, 36 bits 11-13 digits, 1-1/4 word, 45 bits 14-15 digits, 1-1/2 word, 54 bits 16-18 digits, 1-3/4 word, 63 bits
COMP-5	Fixed-point binary	Decimal	4 digits, 18 bits 9 digits, 36 bits

Table 3–4. Meaning and Allocation of Data Types by USAGE

Usage	Meaning	Truncation	Allocation
BYTE-I18N	Bits	N/A	9 bits, quarter-word, per allocatable character represented in the PICTURE clause
POINTER	UC pointer	N/A	72 bits

* For a PACKED-DECIMAL field, the sign field can contain a hexadecimal digit through F. The sign values A, C, E, and F is treated as plus (+) signs; and B and D is treated as minus (-) signs.

Table 3–5. Alignment of Data Types by USAGE

Usage	Alignment
DISPLAY	Quarter-word boundary
DISP-I18N	Quarter-word boundary
COMPUTATIONAL	Quarter-word boundary
COMPUTATIONAL-1	Quarter-word boundary
COMPUTATIONAL-2	Quarter-word boundary
COMPUTATIONAL-4	Quarter-word boundary
COMPUTATIONAL-5	Quarter-word boundary
PACKED-DECIMAL	Quarter-word boundary
BINARY	Quarter-word boundary
BINARY-1	See rules below.
INDEX	Quarter-word boundary
BYTE-I18N	Quarter-word boundary
POINTER	Quarter-word boundary

The alignment of an elementary exact binary data item is determined by the following rules:

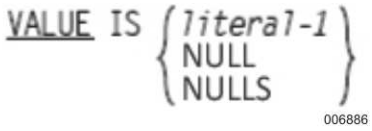
1. If the current data item is the first elementary data item in a group, the current data item is aligned on the same character boundary as the group item.
2. If the current data item is the first data item following the last data item of a group, the current data item is aligned on a character boundary.
3. If the preceding elementary data item is not an exact binary data item, the current data item is aligned on a character boundary.
4. If the preceding elementary data item is an exact binary data item and none of the above rules applies, the current data item is aligned on the bit following the preceding data item.

3.5.17. The VALUE Clause

Function

The VALUE clause defines the initial values of Common-Storage Section and Working-Storage Section data names, and the values associated with condition names.

Format 1



Format 2



Syntax Rules

1. The words THRU and THROUGH are equivalent.
2. A signed numeric literal must have associated with it a signed numeric PICTURE character string.
3. All numeric literals in a VALUE clause of an item must have a value within the range of values indicated by the PICTURE clause, and must not have a value that would require truncation of nonzero digits. Nonnumeric literals in a VALUE clause of an item must not exceed the size indicated by the PICTURE clause.
4. The VALUE clause must not be specified in any entry that is part of the description or redefinition of an external data record. The VALUE clause must be specified for condition name entries associated with such data description entries.

General Rules

The VALUE clause must not conflict with other clauses in the data description of the item or in the data description within the hierarchy of the item. The following rules apply:

1. If the category of the item is numeric, all literals in the VALUE clause must be numeric. If the literal defines the value of a working storage item, the literal is aligned in the data item according to the standard alignment rules. (See Section 2.4.19)
2. If the category of the item is alphabetic, alphanumeric, alphanumeric edited, or numeric edited, all literals in the VALUE clause must be nonnumeric literals. The literal is aligned in the data item as if the data item had been described as alphanumeric. (See Section 2.4.19.) Editing characters in the PICTURE clause are included in determining the size of the data item but have no effect on initialization of the data item. (See Section 3.5.11) Therefore, the VALUE for an edited item must be specified in an edited form.
3. Initialization is not affected by any BLANK WHEN ZERO or JUSTIFIED clause that can be specified.

Condition-Name Rules

1. In a condition-name entry, the VALUE clause is required. The VALUE clause and the condition name itself are the only two clauses permitted in the entry. The characteristics of a condition name are implicitly those of its conditional variable.
2. Format 2 can be used only in connection with condition names. (See Section 2.4.7) Wherever the THRU phrase is used, *literal-2* must be less than *literal-3.*

Data Description Entries Other Than Condition Names

1. Rules governing the use of the VALUE clause differ with the respective sections of the Data Division:
   a. In level 1, the VALUE clause cannot be used in the File Section. In the File Section, the VALUE clause can be used only in condition-name entries; therefore, the initial value of the data items in the File Section is undefined.
   b. In level 1, the VALUE clause cannot be used in the Linkage Section. In the Linkage Section, the VALUE clause can be used only in condition-name entries.
   c. In the Working-Storage Section and the Common-Storage Section, the VALUE clause must be used in condition-name entries. VALUE clauses in the Working-Storage and Common-Storage Sections of a program take effect only when the program is placed into its initial state. If the VALUE clause is used in the description of the data item, the data item is initialized to the defined value. If the VALUE clause is not associated with a data item, the initial value of that data item is undefined.
2. The VALUE clause must not be stated in a data description entry that contains a REDEFINES clause or in an entry that is subordinate to an entry containing a REDEFINES clause. This rule does not apply to condition-name entries.

3. If the VALUE clause is used in an entry at the group level, the literal must be a figurative constant or a nonnumeric literal, and the group area is initialized without consideration for the individual elementary or group items contained within this group. The VALUE clause cannot be stated at the subordinate levels within this group.
4. The VALUE clause must not be specified for a group item containing items subordinate to it with descriptions including JUSTIFIED, SYNCHRONIZED, or USAGE (other than USAGE IS DISPLAY).
5. If a VALUE clause is specified in a data description entry of a data item that is associated with a variable occurrence data item, the initialization of the data item behaves as if the value of the data item referenced by the DEPENDING ON phrase in the OCCURS clause specified for the variable occurrence data item is set to the maximum number of occurrences as specified by that OCCURS clause. A data item is associated with a variable occurrence data item in any of the following cases:

   a. It is a group data item that contains a variable occurrence data item.

   b. It is a variable occurrence data item.

   c. It is a data item that is subordinate to a variable occurrence data item.

   If a VALUE clause is associated with the data item referenced by a DEPENDING ON phrase, that value is considered to be placed in the data item after the variable occurrence data item is initialized. (See Section 3.5.10.)
6. A format 1 VALUE clause specified in a data description entry that contains an OCCURS clause or in an entry that is subordinate to an OCCURS clause causes every occurrence of the associated data item to be assigned the specified value.
7. A VALUE clause associated with a data item declared as USAGE BYTE-I18N can only refer to the figurative constants HIGH-VALUES or LOW-VALUES.
8. A VALUE clause associated with a data item declared as USAGE POINTER can only refer to the figurative constants NULL or NULLS.

3.5.18. The USING POINTER Clause

The USING POINTER clause permits the implicit basing of a data item. The data item is based upon the address that is pointed by the USING POINTER data item.

General Format

```
USING POINTER data-name-4
```

Syntax Rules

1. The USING POINTER clause can only be specified with an 01/77 level item declared in the Working-Storage Section.
2. *data-name-4* must be an elementary item defined as 1(36) BINARY-1.
3. *data-name-4* might be defined in Working-Storage, Common-Storage, or Linkage.
4. VALUE clauses must not be specified for the data item associated with the USING POINTER clause.

5. VALUE clauses must not be specified for any item subordinate to the data item associated with the USING POINTER clause.
6. The EXTERNAL clause must not be specified for the data item associated with the USING POINTER clause.
7. The USING POINTER *data-name-4* clause cannot be specified in the same declaration as a REDEFINES.
8. The data item that contains the USING POINTER *data-name-4* clause must not be specified in an APPLY EXREF/EXDEF clause.

General Rules

1. It is the user's responsibility to assign a value to *data-name-4* (virtual address) before referencing the data item associated with the USING POINTER clause or any item subordinate to it. This can be accomplished by using the LOC$, UCOBMALLOC$, UCOBCALLOC$ or UCOBREALLOC$ service routine or by passing the value into the program. See Volume 2 for further details. The contents of *data-name-4* must be a full Virtual Address (VA) pointing to a whole word (a word pointer).
2. Arithmetic operations can be performed on *data-name-4*, but the user must be careful to ensure that *data-name-4* contains a valid address. Note that adding a value of 1 to *data-name-4* results in pointing to the next word.
3. *data-name-4* can be associated with multiple 01 level items.

Example 1

```
working-storage section.

01  ptr1 pic 1(36) binary-1.

01  baseditem USING POINTER ptr1 pic xxxx.

01  buffer1 pic xxxx value 'ABCD'.

01  buffer2 pic xxxx value 'WXYZ'.

procedure division.

begin.

* Get the address of buffer1 and place it into ptr1, then display

* the value in the based item.

    call 'loc$' using buffer1 ptr1.

    display baseditem.

  * Change the value of buffer1 and redisplay the value in the
    based item
```

```
            move 'JKLM' to buffer1.
            display baseditem.
       * Get the address of buffer2 and place it into ptr1, then display
       * the value in the based item.
            call 'loc$' using buffer2 ptr1.
            display baseditem.
```

The result of executing the above code segment will be

```
ABCD
JKLM
WXYZ
```

Example 2

```
       WORKING-STORAGE SECTION.
       01 buf USING POINTER addrname1.
           02 bfgp OCCURS 1 TO 1000 DEPENDING ON bufsize.
             03 bfchar PIC 1(9) BINARY-1.
       01 bufsize PIC s9(10) BINARY VALUE 1000.
       01 retstat    PIC 1(36) BINARY-1 VALUE 0.
       01 addrname1 PIC 1(36) BINARY-1.
       01 i   PIC 9(10) BINARY VALUE 0.
       procedure division.
       pppp.
      *Allocate buf
            CALL 'UCOBMALLOC$' USING bufsize, addrname1, retstat.
            IF retstat NOT = 0 DISPLAY 'allocation failed' STOP RUN.
            PERFORM bufsize TIMES
               ADD 1 TO i
```

```
            MOVE i TO bfchar (i)
        END-PERFORM.
        IF bfchar(200) NOT EQUAL 200
            DISPLAY 'comparison failed'.
*Free buf
         CALL 'UCOBFREE$' USING addrname1, retstat.
         IF retstat NOT = 0 DISPLAY 'free failed'.
```

3.5.19. USAGE POINTER pointers

Pointer handling allows for UCS COBOL to be compatible with IBM COBOL pointers and based storage. UCOB has two separate mechanisms for defining based storage and two kinds of pointers. One approach involves the USING POINTER clause and the other instance is the USAGE POINTER clause.

The USING POINTER clause is used on 01 level declarations in the WORKING-STORAGE SECTION to define a based item and the pointer it is based on. The pointers hold a one-word VA address defined in Section 3.5.18 The USING POINTER clause.

The USAGE POINTER clause is used on a data item for a based structure in the Linkage Section. Use the SET statement to base a Linkage Section 01/77 data item. Precede the data name with the ADDRESS OF phrase and populate with either the ADDRESS OF another structure or variable (at any level) or to a pointer that was declared as USAGE POINTER. This pointer form has byte granularity and can point at byte boundaries. In order to access items on byte boundaries, the UCS C two-word pointer form is used for the pointers. (They are compatible with UCS C pointers.) Arithmetic on these C-style pointers is not allowed since it would produce invalid results (adding 1 to one of these pointers would NOT cause it to point at the next byte).

Loading an Address of a Data Item

There are three methods available to load a USAGE POINTER data item. The first method involves using a call to LOC$. The second argument of the call to LOC$ must be declared as USAGE POINTER in order to receive a pointer address.

For example:

```
CALL 'LOC$' USING data-item   usage-pointer-item.
```

data-item	Any elementary or group item declared at 01/77 level
usage-pointer-item	A USAGE POINTER data item

The second method involves using a SET statement to load a pointer data with the address of another data item. The SET statement requires use of the ADDRESS OF phrase to obtain a pointer.

For example:

```
SET pointer-name TO ADDRESS OF data-item
```

data-item	Any elementary or group item declared at any level
pointer-name	A USAGE POINTER data item

The third method involves calling UCOBMALLOC$2 to get new storage, defined in Volume 2, Section 10.25.5).

For example:

```
CALL 'UCOBMALLOC$2' USING number-of-bytes, address-name, return-status
```

number-of-bytes	is an integer variable. Define *number-of-bytes* as PIC 1(36) BINARY-1
address-name	is a USAGE POINTER variable
return-status	is an integer variable. Define return-status as PIC 1(36) BINARY-1

Basing Data Items

Data items declared in the Linkage Section can be based on other data items or pointers. The following must be true to base a Linkage Section data item:

- The Linkage Section structure or data item must be at the 01/77 level
- There must be at least one SET ADDRESS OF <data-item> statement in the PROCEDURE division to give it an address
- Note that a based Linkage Section data item may or may not also be a parameter to the PROCEDURE (specified in the USING clause)

An internal implicit pointer is created and the entire 01/77 data item can be based on the address of another structure. Any 01/77 Linkage Section item not in the PROCEDURE USING clause is a BASED item, whether its address is SET or not. Note that you must ensure that the storage you are basing a structure on is large enough to encompass the extent of the structure. The compiler cannot check this for you. If the structure exceeds the storage size, you will reference data meant for other variables and could overwrite them, giving unknown (but incorrect) results.

```
SET ADDRESS OF data-name TO usage-pointer-item
```

Note: *Runtime failures can occur if an invalid address is accessed.*

Setting the Address of Linkage Section Items

Generally, when one UCOB program calls another, operands are passed from the calling UCOB program to the subprogram in the following manner:

- The calling program uses the CALL USING statement to pass operands to the called program, and
- The called program specifies the USING phrase in the Procedure Division header.

For each operand that is listed on the CALL USING statement in the calling program, there must be a corresponding operand that is specified by the USING phrase in the Procedure Division of the called program.

When using the ADDRESS OF phrase, you no longer need to ensure a one-to-one mapping between the USING phrases of two programs. For those data items in the Linkage Section that are not specified in the USING phrase of the Procedure Division header, you can use a SET statement to specify the starting address of the data structure. Once the SET statement is executed, the data item can be freely referenced after the address of the data item is set. You can also SET the ADDRESS OF a Linkage Section data item that is a parameter to the routine (specified in the USING clause). This moves references to the parameter data item from the original caller's storage to the new location.

Initializing and Comparing Pointers using the NULL Constant

The NULL figurative constant represents a two-word value of binary zeroes. It can be used in place of a USAGE POINTER data item in executable statements to represent a null or void pointer value. For example:

```
       WORKING-STORAGE SECTION.
       77 APTR  USAGE POINTER VALUE NULL.
       LINKAGE SECTION.
       01 AGRP  PIC X(9).
       PROCEDURE DIVISION.
           SET ADDRESS OF AGRP TO NULL.
           IF APTR = NULL THEN
              DISPLAY 'APTR IS NULL'
           END-IF.
```

In the above example, pointer APTR is set to NULL in the Working-Storage section. The comparison in the procedure division will be TRUE and the DISPLAY statement is executed. Note that only the EQUAL and NOT EQUAL operators are allowed as relational operators with USAGE POINTER pointers. Also, pointers can only be compared to other pointers or to NULL.

Examples

The following are example programs containing pointers, using based storage, and executing SET statements:

Example 1

```
     IDENTIFICATION DIVISION.
     PROGRAM-ID. usagepointertest1.
     ENVIRONMENT DIVISION.
     CONFIGURATION SECTION.
     SPECIAL-NAMES.
         PRINTER IS PR.
     DATA DIVISION.
     WORKING-STORAGE SECTION.
    *This is the real working record:
     01  bfr                    pic x(20) value all '*'.
    *This pointer will be used to access the data
     01  p-bfr                  usage pointer.
    *Just for displaying a number
     01  disnum                 pic S9(9) sign leading separate.
    *
    *Even though this is a main program, it contains a
    *Linkage Section to define different overlays over bfr
     LINKAGE SECTION.
    *First overlay:
     01  layout-1.
         05  r1-a               pic x(4).
         05  r1-b               pic s9(9) comp-5.
         05  r1-c.
             10  r1-c-a         pic x(4).
             10  r1-c-b         pic x(4).
             10  r1-c-c         pic x(4).
    *Second overlay:
     01  layout-2.
         05  r2-a               pic x(4).
         05  r2-b               pic s9(9) comp-5.
         05  r2-c               pic x(12).
    *Third overlay:
     01  overlay-3.
         05  r3-a               pic x(2).
         05  r3-b               pic x(2).
    *Fourth overlay:
     77  only4char              pic x(4).
     PROCEDURE DIVISION.
```

```
     m-start.
          display "Pointer test:"
     *First show the whole record:
          display "bfr: " bfr
     *Get pointer to buffer.
          set p-bfr to address of bfr

*****************************************************************
     *Now we do a dereferencing (or casting) of our pointer to access
     *to access working storage
     *We can access the working storage via layout-1:
          set address of layout-1 to p-bfr
     *as well as via layout-2:
          set address of layout-2 to p-bfr
     *Move something in
          move 'AAAA' to r1-a
          move 0 to r1-b
          move 'aaaa' to r1-c-a
          move 'bbbb' to r1-c-b
          move 'cccc' to r1-c-c
     *Just "dump" the whole buffer after -
     * all three should be the same:
          display "bfr: " bfr
          display "layout-1: " layout-1
          display "layout-2: " layout-2
     *Add 1 to the integer and display it:
          add 1 to r2-b
          move r2-b to disnum
          display "r2-b: " disnum
     *Add 61 but display via different record layout:
          add 61 to r1-b
          move r2-b to disnum
          display "r2-b: " disnum
     *Different accesses:
          display "r1-c: " r1-c-a r1-c-b r1-c-c
          display "r2-c: " r2-c
     *Access only the first 4 bytes (by a 77 definition):
          set address of only4char to p-bfr
          display "only4char: " only4char
     *Last we access a sub-part of our working storage with a different
     *layout:
          set address of overlay-3 to address of r1-c-c
          display "r3-b: " r3-b
     *Last we modify it and dump the whole record via WS definition:
          move "XX" to r3-a
          display "bfr: " bfr

          display "End of test."
          GOBACK.
```

The result of the preceding code sample is:

```
Pointer test:
bfr: ********************
bfr: AAAAaaaabbbbcccc
layout-1: AAAAaaaabbbbcccc
layout-2: AAAAaaaabbbbcccc
r2-b: +000000001
r2-b: +000000062
r1-c: aaaabbbbcccc
r2-c: aaaabbbbcccc
only4char: AAAA
r3-b: cc
bfr: AAAA>aaaabbbbXXcc
End of test.
```

Example 2

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. USAGEPTRTEST2.
      *PURPOSE OF TEST:
      *    1)  DEMONSTRATE LINKAGE SECTION IN A MAIN PROGRAM
      *    2)  PLACING A RECORD LAYOUT (M-1) OVER A BUFFER
      *    3)  PLACING A 'SLIDING' RECORD LAYOUT OVER A BUFFER
       ENVIRONMENT DIVISION.
       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01  BUFFER.
           05 BFR                  PIC X(1) OCCURS 26.
       01  BFRIND                  PIC 9(4) BINARY.
       01  P-BFR-1                 POINTER.
       01  P-BFR-2                 POINTER.
       01  BUFFER-DATA             PIC X(26)
           VALUE "ABCDEFGHIJKLMNOPQRSTUVWXYZ".
       LINKAGE SECTION.
       01  M-1.
           05  FIRST4              PIC X(4).
           05  FILLER              PIC X(18).
           05  LAST4               PIC X(4).
       01  M-2.
           05  SLCHAR              PIC X(4).
       PROCEDURE DIVISION.
       0000-START.
           DISPLAY "START ..."
      * LOAD DATA INTO BUFFER
           MOVE BUFFER-DATA TO BUFFER
      * DISPLAY THIS BUFFER
           DISPLAY "BUFFER: " BUFFER
      * NOW TRY IT WITH A STRUCTURE OVERLAY
           DISPLAY "Parts of this buffer via overlay:"
           SET P-BFR-1 TO ADDRESS OF BUFFER
           SET ADDRESS OF M-1 TO P-BFR-1
```

```
           DISPLAY "FIRST 4: " FIRST4
           DISPLAY "LAST  4: " LAST4
      * DISPLAY PARTS OF THE BUFFER WITH A 'SLIDING' OVERLAY
      * USING 2 SET STATEMENTS
           DISPLAY "Parts of this buffer with 'sliding' overlay (1):"
           PERFORM VARYING BFRIND FROM 1 BY 1 UNTIL BFRIND > 8
               SET P-BFR-2 TO ADDRESS OF BFR(BFRIND)
               SET ADDRESS OF M-2 TO P-BFR-2
               DISPLAY "INDEX: " BFRIND " DATA: " SLCHAR
           END-PERFORM
      * SAME LIKE BEFORE BUT DIFFERENT SET STATEMENT
      * USING 1 SET STATEMENT.
           DISPLAY "Parts of this buffer with 'sliding' overlay (2):"
           PERFORM VARYING BFRIND FROM 1 BY 1 UNTIL BFRIND > 8
               SET ADDRESS OF M-2 TO ADDRESS OF BFR(BFRIND)
               DISPLAY "INDEX: " BFRIND " DATA: " SLCHAR
           END-PERFORM
           DISPLAY "END.".
           GOBACK.
```

Output is

```
START ...
BUFFER: ABCDEFGHIJKLMNOPQRSTUVWXYZ
Parts of this buffer via overlay:
FIRST 4: ABCD
LAST  4: WXYZ
Parts of this buffer with 'sliding' overlay (1):
INDEX: 0001 DATA: ABCD
INDEX: 0002 DATA: BCDE
INDEX: 0003 DATA: CDEF
INDEX: 0004 DATA: DEFG
INDEX: 0005 DATA: EFGH
INDEX: 0006 DATA: FGHI
INDEX: 0007 DATA: GHIJ
INDEX: 0008 DATA: HIJK
Parts of this buffer with 'sliding' overlay (2):
INDEX: 0001 DATA: ABCD
INDEX: 0002 DATA: BCDE
INDEX: 0003 DATA: CDEF
INDEX: 0004 DATA: DEFG
INDEX: 0005 DATA: EFGH
INDEX: 0006 DATA: FGHI
INDEX: 0007 DATA: GHIJ
INDEX: 0008 DATA: HIJK
END.
```

Example 3

```
 * Show how a single pointer input parameter can be used to handle very
 * diverse inputs.
 * A bit of common layout at the beginning so the function can be
 * determined as needed (unless the function is passed separately).
  IDENTIFICATION DIVISION.
  PROGRAM-ID.  usageptrtest3.
  ENVIRONMENT DIVISION.
  CONFIGURATION SECTION.
  SPECIAL-NAMES.
      PRINTER IS PR.
 * DISPLAYs go to the Operator's console without PRINTER IS PR
  DATA DIVISION.
  WORKING-STORAGE SECTION.
  77  p  usage pointer.
  77  i      pic s9(10) binary.
  77  n      pic s9(10) binary.
  77  multx  pic s9(13)v99.
  01 binaddstruc.
    03 fctnin      pic s9(10) binary value 1.
    03 numitems    pic s9(10) binary value 16.
    03 totsum      pic s9(10) binary value 0.
    03 avaluelist  pic s9(10) binary occurs 16 times.
  01 multstruc.
    03 fctnin     pic s9(10) binary value 2.
    03 numitems   pic s9(10) binary value 16.
    03 overflowcount pic s9(10) binary value 0.
    03 mult1      pic s9(13)v99 value 2.50.
    03 mvaluelist pic s9(13)v99 occurs 16 times.
  PROCEDURE DIVISION.
  ST.
      move 1000000.01 to multx.
      perform varying i from 1 by 1 until i is equal to 16
          move i to avaluelist(i)
      end-perform.
      perform varying i from 1 by 1 until i is equal to 16
          move multx to mvaluelist(i)
          add 100000.10 to multx
      end-perform.
* ****************************************************************
      move numitems of binaddstruc to n.
      display " " upon pr.
      display "Called WORKER passing BINADDSTRUC "
              "holding " n " integer values" upon pr.
      display "The values passed were:" upon pr.
      perform varying i from 1 by 1 until i is equal to n
         display "  Value# " i " : " avaluelist(i) upon pr
      end-perform.
      set p to address of binaddstruc.
      call "WORKER" using p.
      display "The SUM of the " n " values is " totsum upon pr.
```

```
* ****************************************************************
      move numitems of multstruc to n.
      display " " upon pr.
      display "Calling WORKER passing MULTSTRUC with a multiplier "
              "of " mult1 upon pr.
      display "The values passed were:" upon pr.
      perform varying i from 1 by 1 until i is equal to n
         display "  Value# " i " : " mvaluelist(i) upon pr
      end-perform.
      set p to address of multstruc.
      call "WORKER" using p .
      display "The " n
            " multiplied values returned by WORKER are:" upon pr.
      perform varying i from 1 by 1 until i is equal to n
         display "  Value# " i " : " mvaluelist(i) upon pr
      end-perform.
      if overflowcount is greater than 0 then
         display "There were " overflowcount " numeric "
                 "overflows" upon pr
      else
        display "There were no numeric overflows" upon pr
      end-if.
* ****************************************************************
      display " " upon pr.
      display "Calling WORKER passing an illegal function;  "
              "Expect an ERROR:" upon pr.
      move 99 to fctnin of binaddstruc.
      set p to address of binaddstruc.
      call "WORKER" using p.
      display "All done testing WORKER()" upon pr.
      goback.
  end program  usageptrtest3.

  IDENTIFICATION DIVISION.
  PROGRAM-ID. worker.
  ENVIRONMENT DIVISION.
  CONFIGURATION SECTION.
  SPECIAL-NAMES.
      PRINTER IS PR.
  DATA DIVISION.
  WORKING-STORAGE SECTION.
 * Used to determine which paragraph to select
  01  fctn   pic s9(10) binary.
  77  sumx   pic s9(10) binary.
  77  i      pic s9(10) binary.
  77  im1    pic s9(10) binary.
  77  iout   pic s9(10) binary.
  77  nmoves pic s9(10) binary.
  77  totmoves pic s9(10) binary.
  77  n      pic s9(10) binary.
  77  ov     pic s9(10) binary.
  LINKAGE SECTION.
```

```
 01 binaddstruc.
   03 fctnin     pic s9(10) binary.
   03 numitems   pic s9(10) binary.
   03 totsum     pic s9(10) binary.
   03 avaluelist  pic s9(10) binary occurs 1 to 16000000 times
                  depending on numitems of binaddstruc.
 01 multstruc.
   03 fctnin     pic s9(10) binary.
   03 numitems   pic s9(10) binary.
   03 overflowcount pic s9(10) binary.
   03 mult1      pic s9(13)v99.
   03 mvaluelist pic s9(13)v99 occurs 1 to 4400000 times
                 depending on numitems of multstruc.
 01 p   usage pointer.
 PROCEDURE DIVISION using p.
 ST.
* All input parm forms have the function in the same place, so
* base our BINADDSTRUC so we can access the function code:
     set address of binaddstruc to p.
     move fctnin of binaddstruc to fctn.
     evaluate fctn
       when  1            perform addwork
       when  2            perform multiplywork
       when other perform badinputcode
     end-evaluate.
     goback.
   addwork.
* Add each value and come up with a final sum TOTSUM.
     move 0 to sumx.
     move numitems of binaddstruc to n.
     perform varying i from 1 by 1 until i is equal to n
        add sumx to avaluelist of binaddstruc(i) giving sumx
     end-perform.
     move sumx to totsum of binaddstruc.
     goback.
  multiplywork.
* Multiply MULT1 times each array item, replacing the array item
* with the result. Return a count of the # of overflows also.
     set address of multstruc to p.
     move 0 to ov.
     move numitems of multstruc to n.
     perform varying i from 1 by 1 until i is equal to n
        multiply mult1 by mvaluelist(i) giving mvaluelist(i)
           on size error add 1 to ov
        end-multiply
     end-perform.
     move ov to overflowcount.
     goback.
  badinputcode.
     display "*ERROR: unknown function input to WORKER: '"
             fctn "'" upon pr.
```

```
       goback.
   end program worker.
```

Output is

```
Called WORKER passing BINADDSTRUC holding +0000000016 integer values
The values passed were:
  Value# +0000000001 : +0000000001
  Value# +0000000002 : +0000000002
  Value# +0000000003 : +0000000003
  Value# +0000000004 : +0000000004
  Value# +0000000005 : +0000000005
  Value# +0000000006 : +0000000006
  Value# +0000000007 : +0000000007
  Value# +0000000008 : +0000000008
  Value# +0000000009 : +0000000009
  Value# +0000000010 : +0000000010
  Value# +0000000011 : +0000000011
  Value# +0000000012 : +0000000012
  Value# +0000000013 : +0000000013
  Value# +0000000014 : +0000000014
  Value# +0000000015 : +0000000015
The SUM of the +0000000016 values is +0000000120

Calling WORKER passing MULTSTRUC with a multiplier of 00000000000025{
The values passed were:
  Value# +0000000001 : 00000010000000A
  Value# +0000000002 : 00000011000001A
  Value# +0000000003 : 00000012000002A
  Value# +0000000004 : 00000013000003A
  Value# +0000000005 : 00000014000004A
  Value# +0000000006 : 00000015000005A
  Value# +0000000007 : 00000016000006A
  Value# +0000000008 : 00000017000007A
  Value# +0000000009 : 00000018000008A
  Value# +0000000010 : 00000019000009A
  Value# +0000000011 : 00000020000010A
  Value# +0000000012 : 00000021000011A
  Value# +0000000013 : 00000022000012A
  Value# +0000000014 : 00000023000013A
  Value# +0000000015 : 00000024000014A
The +0000000016 multiplied values returned by WORKER are:
  Value# +0000000001 : 00000025000000B
  Value# +0000000002 : 00000027500002G
  Value# +0000000003 : 00000030000005B
  Value# +0000000004 : 00000032500007G
  Value# +0000000005 : 00000035000010B
  Value# +0000000006 : 00000037500012G
  Value# +0000000007 : 00000040000015B
  Value# +0000000008 : 00000042500017G
  Value# +0000000009 : 00000045000020B
  Value# +0000000010 : 00000047500022G
```

```
  Value# +0000000011 : 00000050000025B
  Value# +0000000012 : 00000052500027G
  Value# +0000000013 : 00000055000030B
  Value# +0000000014 : 00000057500032G
  Value# +0000000015 : 00000060000035B
There were no numeric overflows

Calling WORKER passing an illegal function;  Expect an ERROR:
*ERROR: unknown function input to WORKER: '+0000000099'
All done testing WORKER()
```

3.6. Procedure Division

This subsection describes the Procedure Division and its associated arithmetic expressions.

3.6.1. General Description

The Procedure Division contains procedures to be executed by the object program. The Procedure Division is optional in a COBOL source program.

The general formats of the Procedure Division in the Nucleus are shown below.

Format 1

```
PROCEDURE DIVISON.

{section-name SECTION.

[paragraph-name.

 [sentence]...]...}...
```

Format 2

```
PROCEDURE DIVISON.

[paragraph-name.

 [sentence]...]...}...
```

3.6.2. Arithmetic Expressions

This subsection defines an arithmetic expression and identifies available arithmetic operators.

Definition of an Arithmetic Expression

An arithmetic expression can be an identifier of a numeric elementary item; a numeric literal; the figurative constant ZERO (ZEROS, ZEROES), such identifiers, figurative constants, and literals separated by arithmetic operators; two arithmetic expressions separated by an arithmetic operator; or an arithmetic expression enclosed in parentheses. Any arithmetic expression can be preceded by a unary operator.

The permissible combinations of variables, numeric literals, arithmetic operators, and parentheses are given in Table 3–6, "Combination of Symbols in Arithmetic Expressions".

Those identifiers and literals appearing in an arithmetic expression must represent either numeric elementary items or numeric literals on which arithmetic can be performed.

Arithmetic Operators

There are five binary arithmetic operators and two unary arithmetic operators that can be used in arithmetic expressions. They are represented by specific characters that must be preceded by a space and followed by a space.

Binary Arithmetic Operators	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Exponentiation

Unary Arithmetic Operators	Meaning
+	The effect of multiplication by numeric literals +1
-	The effect of multiplication by numeric literals -1

Formation and Evaluation Rules

1. Parentheses can be used in arithmetic expressions to specify the order in which elements are to be evaluated. Expressions within parentheses are evaluated first, and, within nested parentheses, evaluation proceeds from the least inclusive set to the most inclusive set. When parentheses are not used, or parenthesized expressions are at the same level of inclusiveness, the following hierarchical order of execution is implied:

   1st = Unary plus and minus

   2nd = Exponentiation

   3rd = Multiplication and division

   4th = Addition and subtraction

2. Parentheses are used either to eliminate ambiguities in logic where consecutive operations of the same hierarchical level appear, or to modify the normal hierarchical sequence of execution in expressions where it is necessary to have some deviation from the normal precedence. When the sequence of execution is not specified by parentheses, the order of execution of consecutive operations of the same hierarchical level is from left to right.

3. The ways in which identifiers, literals, operators, and parentheses can be combined in an arithmetic expression are summarized in Table 3–6, where

   a. The letter P indicates a permissible pair of symbols.

   b. The character - indicates an invalid pair.

Table 3–6. Combination of Symbols in Arithmetic Expressions

First Symbol	Second Symbol				
	Identifier or Literal	**+ - * / ****	**Unary + or -**	**(**	**)**
Identifier or literal	-	P	-	-	P
+ - * / **	P	-	P	P	-
Unary + or -	P	-	-	P	-
(	P	-	P	P	-
)	-	P	-	-	P

4. An arithmetic expression can begin only with the symbol (, +, -, an identifier, or a literal and can end only with a), an identifier, or a literal. There must be a one-to-one correspondence between left and right parentheses of an arithmetic expression such that each left parenthesis is to the left of its corresponding right parenthesis. If the first operator in an arithmetic expression is a unary operator, it must be immediately preceded by a left parenthesis if that arithmetic expression immediately follows an identifier or another arithmetic expression.

5. The following rules apply to evaluation of exponentiation in an arithmetic expression:
   a. If the value of an expression to be raised to a power is zero, the exponent must have a value greater than zero. Otherwise, the size error condition exists. (See Section 3.6.5.)
   b. If the evaluation yields both a positive and a negative real number, the value returned as the result is the positive number.
   c. If no real number exists as the result of the evaluation, the size error condition exists.
6. Arithmetic expressions allow the user to combine arithmetic operations without the restrictions on composite of operands or receiving data items. See, for example, syntax rule 3, in Section 3.6.7.

The following examples illustrate natural and parentheses-enforced ordering of COBOL arithmetic expressions. In these examples, let A = 4, B = 6, C = 2, D = 3, and E = 12.

Expression	Evaluation
A + B * C ** D	52
A + B * C	16
E - A / C	10
(A + B * C) ** D	4096
(A + B) * C	20
(E - A) / C	4
((A + B) * C) ** D	8000

There are instances where the rules of precedence are insufficient to represent the precise meaning of an algebraic formula without using parentheses. For example:

$$\frac{A + B}{A - B}$$

cannot be expressed as a COBOL expression according to the rules of precedence established previously without using parentheses. To write it as:

```
A + B / A - B
```

is incorrect since this represents the formula:

$$A + \frac{B}{A} - B$$

In this case, parentheses must be used and the desired result is obtained by writing:

```
(A + B) / (A - B)
```

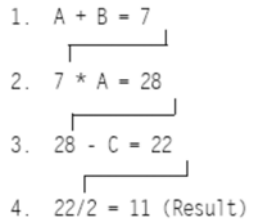
Parentheses have higher precedence than any operator. Operations within parentheses, therefore, have precedence over any other in a calculation. In the following example, let A = 4, B = 3, and C = 6:

```
(A + B) / (A - B)
```

This produces 13 as a result, whereas:

```
((A + B) * A - C) / 2
```

produces 11 with the sequence of evaluation proceeding from the innermost parenthesized operation to the outermost as follows:



Double exponentiation is sometimes used in an algebraic expression. For example:

$$A^{b^c}$$

cannot be written as:

```
A ** B ** C
```

but by writing:

```
A ** (B ** C)
```

In ensuing paragraphs, it shows how arithmetic expressions can be used in a conditional case to determine if the expression defines a specific value.

3.6.3. Conditional Expressions

Conditional expressions identify conditions that are tested to enable the object program to select between alternate paths of control, depending on the truth value of the condition. A conditional expression has a truth value of true or false. Conditional expressions are specified in the EVALUATE, IF, PERFORM, and SEARCH statements. Two categories of conditions are associated with conditional expressions: simple and complex. Each category can be enclosed within any number of paired parentheses, in which case its category is not changed.

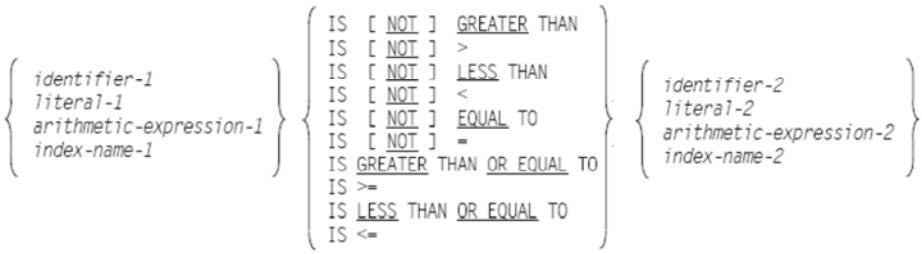
Simple Conditions

The simple conditions are the relation, class, condition name, switch-status, and sign conditions. A simple condition has a truth value of true or false. The inclusion in parentheses of simple conditions does not change the simple condition truth value.

Relation Condition

A relation condition causes a comparison of two operands, each of which can be the data item referenced by an identifier, a literal, the value resulting from an arithmetic expression, or an index name. A relation condition has a truth value of true if the relation exists between the operands. Comparison of two numeric operands is permitted regardless of the formats specified in their respective USAGE clauses. However, for all other comparisons, the operands must have the same usage. If either of the operands is a group item, the nonnumeric comparison rules apply.

The format of a relation condition is



The first operand (*identifier-1, literal-1, arithmetic-expression-1,* or *index-name-1*) is called the subject of the condition; the second operand (*identifier-2, literal-2, arithmetic-expression-2,* or *index-name-2*) is called the object of the condition. The relation condition must contain at least one reference to a variable.

The relational operators specify the type of comparison to be made in a relation condition. A space must precede and follow each reserved word comprising the relational operator. When used, NOT and the next keyword or relation character are one relational operator that defines the comparison to be executed for truth value. The following relational operators are equivalent: IS NOT GREATER THAN is equivalent to IS LESS THAN OR EQUAL TO; IS NOT LESS THAN is equivalent to IS GREATER THAN OR EQUAL TO.

The meanings of the relational operators are as follows:

Meaning	Relational Operator
Greater than or not greater than	IS [NOT] GREATER THAN IS [NOT] >
Less than or not less than	IS [NOT] LESS THAN IS [NOT] <
Equal to or not equal to	IS [NOT] EQUAL TO IS [NOT] =
Greater than or equal to	IS GREATER THAN OR EQUAL TO IS >=
Less than or equal to	IS LESS THAN OR EQUAL TO IS <=

Comparison of Numeric Operands

For operands whose class is numeric (see Section 2.4.17) a comparison is made with respect to the algebraic value of the operands. The length of the literal or arithmetic expression operands, in terms of the number of digits represented, is not significant. Zero is considered a unique value regardless of the sign.

Comparison of these operands is permitted regardless of the manner in which their usage is described. Unsigned numeric operands are considered positive for purposes of comparison.

Comparison of COMPUTATIONAL-1 and COMPUTATIONAL-2 operands with nonfloating-point operands might not always be valid. Since COMPUTATIONAL-1 and COMPUTATIONAL-2 operands are floating-point and binary-scaled, a loss of decimal precision might occur during arithmetic operations. This loss of decimal precision might result in invalid comparisons between COMPUTATIONAL-1 and COMPUTATIONAL-2 operands and decimal operands.

Comparison of Nonnumeric Operands

For nonnumeric DISPLAY operands, or one numeric DISPLAY and one nonnumeric DISPLAY operand, a comparison is made with respect to a specified collating sequence of characters. (See Section 3.4.4.) If one of the operands is specified as numeric, it must be an integer data item or an integer literal and:

1. If the nonnumeric operand is an elementary data item or nonnumeric literal, the numeric operand is treated as though it were moved to an elementary alphanumeric data item of the same size as the numeric data item (in terms of standard data format characters), and the contents of this alphanumeric data item were then compared to the nonnumeric operand. (See Section 3.6.19 and Section 3.5.11.)

2. If the nonnumeric operand is a group item, the numeric operand is treated as though it were moved to a group item of the same size as the numeric data item (in terms of standard data format characters), and the contents of this group item were then compared to the nonnumeric operand. (See Section 3.6.19 and Section 3.5.11.)
3. A noninteger numeric operand cannot be compared to a nonnumeric operand. The size of an operand is the total number of standard data format characters in the operand. Numeric and nonnumeric operands can be compared only when their usage is the same.

Comparison of Equal and Unequal Operand Sizes

There are two cases to consider: operands of equal size and operands of unequal size.

1. Operands of equal size

   If the operands are of equal size, comparison involves characters in corresponding character positions starting from the high order end and continuing until either a pair of unequal characters is encountered or the low order end of the operand is reached, whichever comes first. The operands are determined to be equal if all pairs of corresponding characters are equal.

   The first encountered pair of unequal characters is compared to determine their relative position in the collating sequence. The operand that contains the character that is positioned higher in the collating sequence is considered to be the greater operand.

2. Operands of unequal size

   If the operands are of unequal size, comparison proceeds as though the shorter operand were extended on the right by enough spaces to make the operands of equal size.

The following example shows the effect of a collating sequence on alphanumeric comparisons:

```
ENVIRONMENT DIVISION.
  .
  .
  .
OBJECT-COMPUTER.  UNISYS-2200 PROGRAM COLLATING SEQUENCE IS NEW-
ALPHABET.
SPECIAL-NAMES.

  ALPHABET NEW-ALPHABET IS "A" "B" "CDE" "F" ALSO "f" 72 THRU "U"
  "Z" THRU "V" "0" THRU 58.
  .
  .
  .
DATA DIVISION.
WORKING-STORAGE SECTION.
01 DATA-NAMES.
  02 DATA-NAME1 PIC XXX VALUE 'FFF'.
  02 DATA-NAME2 PIC XXX VALUE 'fff'.
```

```
  02 DATA-NAME3 PIC XXX VALUE '42X'.
  02 DATA-NAME4 PIC 99 VALUE 42.
   .
   .
   .
PROCEDURE DIVISION.
P1.
  IF DATA-NAME1 = DATA-NAME2 THEN MOVE LOW-VALUES TO DATA-NAME1.
  IF DATA-NAME2 LESS THAN DATA-NAME3 THEN ADD 1 TO COUNTER.
  IF DATA-NAME3 > DATA-NAME4 THEN GO TO P2.
   .
   .
   .
P2.
  STOP RUN.
```

A new program collating sequence is defined in Figure 3–2. Note that new low and high values are also defined such that LOW-VALUE = "A" and HIGH-VALUE = 0377.

Also note that a numeric literal represents the ordinal position of that character in the native collating sequence as defined in Table A-1 of the *UCS COBOL Programming Reference Manual Volume 2*. For example, 72 represent the 72nd character in the ASCII collating sequence, which is the character G (octal 107).

Source	Coll Seq Order	ASCII Char	ASCII Octal	Source	Coll Seq Order	ASCII Char	ASCII Octal	Source	Coll Seq Order	ASCII Char	ASCII Octal	Source	Coll Seq Order	ASCII Char	ASCII Octal
A	0	A	101	Unused	044	None	00	Unused	0110	S	44	Unused	0154	m	155
B	1	B	102	Characters	045	None	01	Characters	0111	%	45	Characters	0155	n	156
	2	C	103	in	046	None	02	in	0112	&	46	in	0156	o	157
CDE	3	D	104	Native	047	None	03	Native	0113	'	47	Native	0157	p	160
	4	E	105	Collating	050	None	04	Collating	0114	(	50	Collating	0160	q	161
F	5	Ff	106	Sequence	051	None	05	Sequence	0115	)	51	Sequence	0161	r	162
Also f			146												
72	6	G	107	Order	052	None	06	Order	0116	*	52	Order	0162	s	163
	7	H	110	•	053	None	07	•	0117	+	53	•	0163	t	164
	010	I	111	•	054	None	10	•	0120	,	54	•	0164	u	165
	011	J	112	•	055	None	11	•	0121	-	55	•	0165	v	166
	012	K	113		056	None	12		0122	.	56		0166	w	167
	013	L	114		057	None	13		0123	/	57		0167	x	170
thru	014	M	115		060	None	14		0124	:	72		0170	y	171
	015	N	116		061	None	15		0125	;	73		0171	z	172
	016	O	117		062	None	16		0126	<	74		0172	{	173
	017	P	120		063	None	17		0127	=	75		0173	|	174
	020	Q	121		064	None	20		0130	>	76		0174	}	175
	021	R	122		065	None	21		0131	?	77		0175	~	176
	022	S	123		066	None	22		0132	@	100		0176	Del	177
	023	T	124		067	None	23		0133	[	133		0177	None	200
U	024	U	125		070	None	24		0134	\	134		0200	None	201
Z	025	Z	132		071	None	25		0135	]	135		•	•	
	026	Y	131		072	None	26		0136	^	136		•	•	
thru	027	X	130		073	None	27		0137	_	137		•	•	
	030	W	127		074	None	30		0140	`	140				
V	031	V	126		075	None	31		0141	a	141				
0	032	0	60		076	None	32		0142	b	142				
	033	1	61		077	None	33		0143	c	143				
	034	2	62		0100	None	34		0144	d	144				
	035	3	63		0101	None	35		0145	e	145				
thru	036	4	64		0102	None	36		0146	g	147				
	037	5	65		0103	None	37		0147	h	150				
	040	6	66		0104	SP	40		0150	i	151				
	041	7	67		0105	!	41		0151	j	152				
	042	8	70		0106	"	42		0152	k	153		---	---	
58	043	9	71		0107	#	43		0153	l	154		0376	None	377

Figure 3–2. Collating Sequence Example

The comparison of DATA-NAME1 = DATA-NAME2 will be true since the relative collating sequence numbers of the characters F in DATA-NAME1 and f in DATA-NAME2 are equal. Therefore the value of DATA-NAME1 will be changed to AAA. See Figure 3–2.

The comparison of DATA-NAME2 < DATA-NAME3 will also be true since the first pair of characters encountered in the values of DATA-NAME2 and DATA-NAME3 have relative collating sequence numbers of 05_8 (f) and 036_8 (4).

The comparison of DATA-NAME3 DATA-NAME4 will be false since the relative collating sequence number of the character "X" in the value of DATA-NAME3 is less than the relative collating sequence number of the character "Δ"in the value of DATA-NAME4.

Comparisons Involving Index Names and Index Data Items

Relation tests can be made only between

1. Two index names. The result is the same as if the corresponding occurrence numbers were compared.
2. An index name and a data item (other than an index data item) or literal. The occurrence number that corresponds to the value of the index name is compared to the data item or literal.
3. An index data item and an index name or another index data item. The actual values are compared without conversion.

Comparisons Involving Culturally Sensitive Data Items

If either of the operands involved in the comparison are described as USAGE DISP-I18N, then the comparison is done based on the locale (LC_COLLATE) that is currently active. This comparison results in a call to the internationalization service routine string_collate.

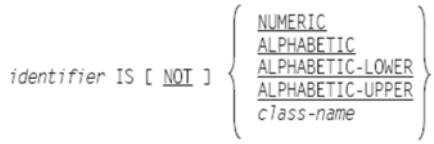
If both operands that are explicitly specified in a comparison are described as USAGE BYTE-I18N, they are compared (left aligned) as bit strings with no conversions taking place. If the operands are of unequal lengths, binary zero padding is done on the right of the shorter operand. No reference modification can be specified on BYTE-I18N data items being compared.

Class Condition

The class condition determines whether the operand is numeric, alphabetic, alphabetic-lower, alphabetic-upper, or contains only the characters in the set of characters specified by the CLASS clause as defined in the SPECIAL-NAMES paragraph of the Environment Division. The class of an operand is determined as follows:

1. An operand is numeric if it consists entirely of the characters 0, 1, 2, 3, ..., 9, with or without an operational sign.
2. An operand is alphabetic if it consists entirely of the uppercase letters A through Z, space, or the lowercase letters a through z, space, or any combination of the uppercase and lowercase letters and spaces.
3. An operand is alphabetic lower if it consists entirely of the lowercase letters a through z and space.
4. An operand is alphabetic upper if it consists entirely of the uppercase letters A through Z and space.
5. An operand is in conformance to class name if it consists entirely of the characters listed in the definition of class name in the SPECIAL-NAMES paragraph.

The general format of the class condition is



The identifier being tested must reference a data item whose usage is explicitly or implicitly DISPLAY or DISP-I18N. DISP-I18N items are treated the same as items declared with USAGE DISPLAY. If the identifier is a function identifier, it must reference an alphanumeric function. The NUMERIC test can also be used on an elementary item with usage PACKED-DECIMAL.

When used, NOT and the next keyword specify one class condition that defines the class test to be executed for truth value; for example, NOT NUMERIC is a truth test for determining that an operand is nonnumeric.

The NUMERIC test cannot be used with an item whose data description describes the item as alphabetic or as a group item composed of elementary items whose data description indicates the presence of operational signs. If the data description of the item being tested does not indicate the presence of an operational sign, the item being tested is determined to be numeric only if the contents are numeric and an operational sign is not present. If the data description of the item does indicate the presence of an operational sign, the item being tested is determined to be numeric only if the contents are numeric and a valid operational sign is present. Valid operational signs for data items described with the SIGN IS SEPARATE clause are the standard data format characters, plus (+) and minus (-). See Table 3–1 for what constitutes valid signs for data items not described with the SIGN IS SEPARATE clause. For signed or unsigned PACKED-DECIMAL fields, the valid operational signs are +, - , or “unsigned.”

The ALPHABETIC test cannot be used with an item whose data description describes the item as numeric. The result of the test is true if the contents of the data item referenced by *identifier* consist entirely of alphabetic characters.

The ALPHABETIC-LOWER test cannot be used with an item whose data description describes the item as numeric. The result of the test is true if the content of the data item referred to by *identifier* consists entirely of the lowercase alphabetic characters a through z and the space.

The ALPHABETIC-UPPER test cannot be used with an item whose data description describes the item as numeric. The result of the test is true if the content of the data item referred to by *identifier* consists entirely of the uppercase alphabetic characters A through Z and the space.

The class name test must not be used with an item whose data description describes the item as numeric.

Condition-Name Condition (Conditional Variable)

In a condition-name condition, a conditional variable is tested to determine whether or not its value is equal to one of the values associated with *condition-name-1*. The general format for the condition name condition is

```
condition-name-1
```

If *condition-name-1* is associated with a range or ranges of values, then the conditional variable is tested to determine whether or not its value falls in this range, including the end values.

The rules for comparing a conditional variable with a condition-name value are the same as those specified for relation conditions.

The result of the test is true if one of the values corresponding to *condition-name-1* equals the value of its associated conditional variable.

The following example represents the data description for conditional valiable MARITAL-STATUS:

```
02 MARITAL-STATUS PICTURE IS 9.
    88 SINGLE VALUE IS 1.
    88 MARRIED VALUE IS 2.
    88 DIVORCED VALUE IS 3.
    88 WIDOWED VALUE IS 4.
    88 EVER-MARRIED VALUE 2 THRU 4.
```

Using an IF statement, the test to determine marital status can appear as

```
IF MARRIED . . .
```

which is equivalent to

```
IF MARITAL-STATUS IS EQUAL TO 2 . . .
```

Again, as in the relational condition, NOT is used to specify the opposite of what the condition-name test would normally specify. So, for example

```
IF NOT SINGLE . . .
```

is equivalent to

```
IF MARITAL-STATUS IS NOT EQUAL TO 1 . . .
```

The same situation could be checked by writing

```
IF EVER-MARRIED . . .
```

which is equivalent to

```
IF MARITAL-STATUS NOT < 2 AND
  MARITAL-STATUS NOT > 4 . . .
```

When a single condition name is associated with a list of values or range of values, a programmer can express what is equivalent to rather complex compound relationals in a simple condition-name condition.

The following example defines the code names for various bolts in inventory:

```
03 PART-NUMBER PIC X(5).
    88 BOLT VALUE IS "AA000" THRU
      "AA999", "AX010" THRU "AZ999",
       "P1000", "P1010".
```

Using the IF statement to determine whether a part is a bolt, a programmer can write

```
IF BOLT GO TO IS-BOLT.
```

which is equivalent to writing

```
IF PART-NUMBER NOT < "AA000" AND
      PART-NUMBER NOT > "AA999" OR
      PART-NUMBER NOT < "AX010" AND
      PART-NUMBER NOT > "AZ999" OR
      PART-NUMBER = "P1000" OR "P1010"
      GO TO IS-BOLT .
```

Switch-Status Condition

A switch-status condition determines the on or off status of an implementor-defined switch. The implementor name and the on or off value associated with the condition must be named in the SPECIAL-NAMES paragraph of the Environment Division.

The general format for the switch-status condition is

```
condition-name.
```

The result of the test is true if the switch is set to the specified position corresponding to *condition-name.*

The implementor names SWITCH-13, SWITCH-14, . . . , SWITCH-24 correspond to the bits in the last third of the condition word set by the SET *mnemonic-name* statement. Therefore, *SWITCH*-13 is bit 11, SWITCH-14 is bit 10, . . . , SWITCH-24 is bit 0. The switch-status condition ON checks for a 1 in the appropriate bit; OFF checks for a 0.

The implementor names SWITCH-1, SWITCH-2, . . . , SWITCH-12 correspond to the bits in the middle or second third of the condition word set by the @SETC command statement. Therefore, SWITCH-1 is bit 23, SWITCH-2 is bit 22, . . . , SWITCH-12 is bit 12. The switch-status condition ON checks for a 1 in the appropriate bit, OFF checks for a 0.

The following example sets SWITCH-6, SWITCH-7, SWITCH-9, SWITCH-11, and SWITCH-12 on; other switches are off.

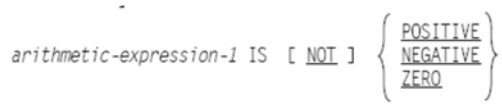
```
@SETC     153/T2
```

Therefore, a program can contain the following statements:

```
ENVIRONMENT DIVISION.
     .
     .
     .
SPECIAL-NAMES.
    SWITCH-6 IS DAILY ON IS DAILY-CYCLE
    SWITCH-7 IS WEEKLY ON IS WEEKLY-CYCLE
    SWITCH-9 IS BI-WEEKLY ON IS BI-WEEKLY-CYCLE
    SWITCH-11 IS MONTH ON IS MONTHLY-CYCLE
    SWITCH-12 IS ANNUAL ON IS ANNUAL-CYCLE.
PROCEDURE DIVISION.
     .
     .
     .
    IF ANNUAL-CYCLE GO TO FISCAL-CLOSEOUT.
    IF MONTHLY-CYCLE GO TO MONTHLY-BILLING.
    IF BI-WEEKLY-CYCLE GO TO TWO-WEEK-REPORT.
    IF WEEKLY-CYCLE GO TO WEEK-END-SUMMARY.
    IF DAILY-CYCLE GO TO ACCOUNTING-UPDATE.
```

Sign Condition

The sign condition determines whether or not the algebraic value of an arithmetic expression is less than, greater than, or equal to zero. The general format for a sign condition is



NOT, when used, and the next keyword specify one sign condition that defines the algebraic test to be executed for truth value; for example, NOT ZERO is a truth test for a nonzero (positive or negative) value. An operand is positive, if its value is greater than zero, negative if its value is less than zero, and zero if its value is equal to zero. The zero test will be true for any numeric item containing a zero even if the field contains a positive or negative operational sign. The arithmetic expression must contain at least one reference to a variable.

Any sign condition can also be expressed by a relational expression. For example

```
A + B + C IS POSITIVE
```

can also be expressed in a relational expression:

```
A + B + C IS GREATER THAN 0
```

Complex Conditions

A complex condition is formed by combining simple conditions, complex conditions with logical connectors (logical operators AND and OR), or both, or by negating these conditions with logical negation (the logical operator NOT). The truth value of a complex condition, whether parenthesized or not, is that truth value that results from the interaction of the stated logical operators on its constituent conditions.

The logical operators and their meanings are

Logical Operators	Meaning
AND	Logical conjunction; the truth value is "true" if both of the conjoined conditions are true, "false" if one or both of the conjoined conditions is false.
OR	Logical inclusive OR; the truth value is "true" if one or both of the included conditions is true, "false" if both included conditions are false.
NOT	Logical negation or reversal of truth value; the truth value is "true" if the condition is false, "false" if the condition is true.

The logical operators must be preceded by a space and followed by a space.

Negated Conditions

A condition is negated by the logical operator NOT, which reverses the truth value of the condition to which it is applied. Therefore, the truth value of a negated condition is true if and only if the truth value of the condition being negated is false; the truth value of a negated condition is false if, and only if, the truth value of the condition being negated is true. Including a negated condition in parentheses does not change its truth value.

The general format for a negated condition is

```
NOT condition
```

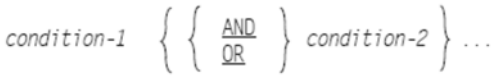
The NOT reverses the true/false value of the condition. Therefore the statement

```
NOT A = B
```

has the effect that if A = B is true, NOT A = B is false, and vice versa.

Combined Conditions

A combined condition results from connecting conditions with one of the logical operators AND or OR. The general format of a combined condition is



Precedence of Logical Operators and the Use of Parentheses

In the absence of the relevant parentheses in a complex condition, the precedence (that is, binding power) of the logical operators determines the conditions to which the specified logical operators apply and implies the equivalent parentheses. The order of precedence is NOT, AND, OR. Therefore, specifying *condition-1* OR NOT *condition-2* AND *condition-3* implies and is equivalent to specifying *condition-1* OR ((NOT *condition-2*) AND *condition-3).*

Where parentheses are used in a complex condition, they determine the binding of conditions to logical operator. Parentheses can, therefore, be used to depart from the normal precedence of logical operators as specified above. Therefore, the example complex condition above can be given a different meaning by specifying it as (condition-1 OR (NOT condition-2)) AND condition-3. (See Section 3.6.4.)

Table 3–7 indicates the ways in which conditions and logical operators can be combined and parenthesized. There must be a one-to-one correspondence between left and right parentheses such that each left parenthesis is to the left of its corresponding right parenthesis.

Table 3–7. Combinations of Conditions, Logical Operators, and Parentheses

Given the following element:	Location in conditional expression	In a left-to-right sequence of elements:		
	May element be first?	May element be last?	When not first, element can be immediately preceded only by:	When not last, element can be immediately followed only by:
simple-condition	Yes	Yes	OR, NOT, AND, (	OR, AND,)
OR or AND	No	No	simple-condition,)	simple-condition, NOT, (
NOT	Yes	No	OR, AND, (	simple-condition, (
(	Yes	No	OR, NOT, AND, (	simple-condition, NOT, (
)	No	Yes	simple-condition,)	OR, AND,)

Therefore, the element pair OR NOT is permissible while the pair NOT OR is not permissible; NOT (is permissible while NOT NOT is not permissible.

The logical operators express a logical relationship between the conditions and determine, according to rules for such operations, the truth or falsity of the entire operation. The logical determination of true or false is based upon the rules defined in the following table. Here A and B represent simple conditions.

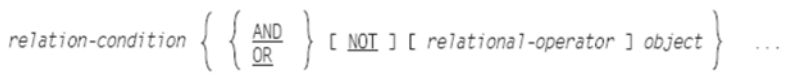
If A is:	and B is:	A AND B will be:	A OR B will be:	A AND NOT B will be:	A OR NOT B will be:
True	True	True	True	False	True
False	True	False	True	False	False
True	False	False	True	True	True
False	False	False	False	False	True

Abbreviated Combined Relation Condition

When simple or negated simple relation conditions are combined with logical connectives in a consecutive sequence such that a succeeding relation condition contains a subject or subject and relational operator that is common with the preceding relation condition, and no parentheses are used within such a consecutive sequence, any relation condition except the first can be abbreviated by

1. The omission of the subject of the relation condition, or
2. The omission of the subject and relational operator of the relation condition.

The format for an abbreviated combined relation condition is



Within a sequence of relation conditions, both of these forms of abbreviation can be used. The effect of using such abbreviations is as if the last preceding stated subject were inserted in place of the omitted subject, and the last stated relational operator were inserted in place of the omitted relational operator. The result of such implied insertion must comply with the rules of Table 3–7. This insertion of an omitted subject, or omitted subject and relational operator, terminates once a complete simple condition is encountered within a complex condition.

The following interpretation is applied to the use of the word NOT in an abbreviated combined relation condition:

1. If the word immediately following NOT is a relational operator (GREATER, >, LESS, <, EQUAL, or =), the NOT participates as part of the relational operator
2. If the word immediately following NOT is not a relational operator, the NOT is interpreted as a logical operator and, therefore, the implied insertion of subject or relational operator results in a negated relation condition.

Some examples of abbreviated combined and negated combined relation conditions and expanded equivalents follow.

Abbreviated Combined Relation Condition	Expanded Equivalent
a NOT *b* OR = *c*	(*a* NOT = *b*) OR (*a* = *c*)
a > *b* AND NOT < *c* OR *d*	((*a* > *b)* AND (a NOT < *c*)) OR (*a* NOT < *d*)
a NOT EQUAL *b* OR *c*	(*a* NOT EQUAL *b*) OR (*a* NOT EQUAL *c*)
NOT *a* = *b* OR *c*	(NOT (*a* = *b*)) OR (*a* = *c*)
NOT (*a* GREATER *b* OR < *c*)	NOT ((*a* GREATER *b*) OR (*a*<*c*))
NOT (*a* NOT > *b* AND *c* AND NOT *d*)	NOT (((*a* NOT > *b*) AND (*a* NOT > *c*))AND (NOT (*a* NOT > *d*)))
a > *b* AND NOT NOT > *c* OR *d*	((*a* > *b*) AND NOT (*a* NOT > *c*)) OR (*a* NOT > *d*)

3.6.4. Order of Evaluation of Condition

Parentheses, both explicit and implicit, denote a level of inclusiveness within a complex condition. Two or more conditions connected only by the logical operator AND or only by the logical operator OR at the same level of inclusiveness establish a hierarchical level within a complex condition. Therefore, an entire complex condition can be considered to be a nested structure of hierarchical levels with the entire complex condition itself being the most inclusive hierarchical level. Within this context, the evaluation of the conditions within an entire complex condition begins at the left of the entire complex condition and proceeds according to the following rule recursively applied where necessary:

The constituent connected conditions within a hierarchical level are evaluated in order from left to right, and evaluation of that hierarchical level terminates as soon as a truth value for it is determined, regardless of whether all the constituent connected conditions within that hierarchical level have been evaluated.

Values are established for arithmetic expressions and functions if and when the conditions containing them are evaluated. Similarly, negated conditions are evaluated if and when it is necessary to evaluate the complex condition that they represent. (See Section 3.6.2).

Application of these rules is shown in Figure 3–3 through Figure 3–6. These figures are not intended to dictate implementation.

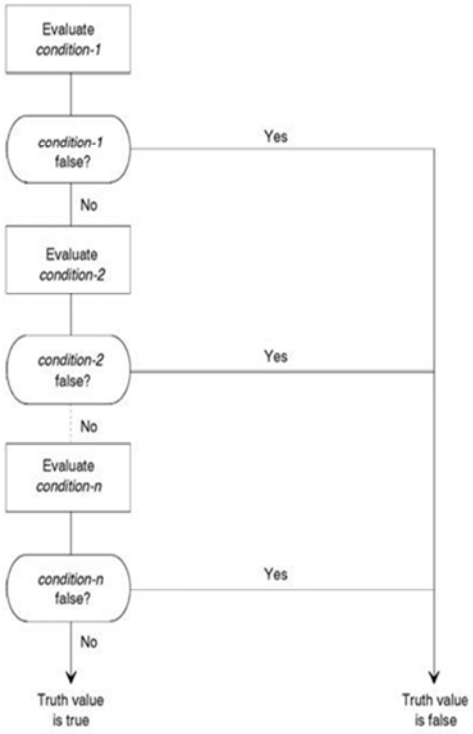


Figure 3–3. Evaluation of the Hierarchical Level condition-1 AND condition-2 AND ... condition-n

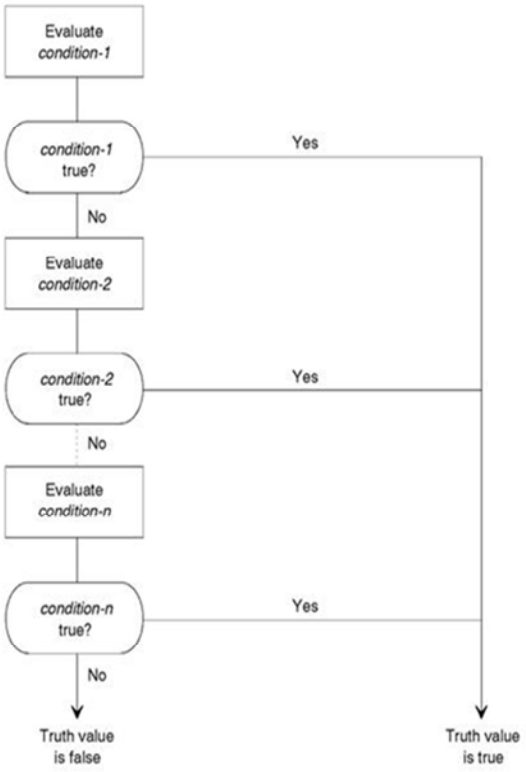


Figure 3–4. Evaluation of the Hierarchical Level condition-1 OR condition-2 OR ... condition-n

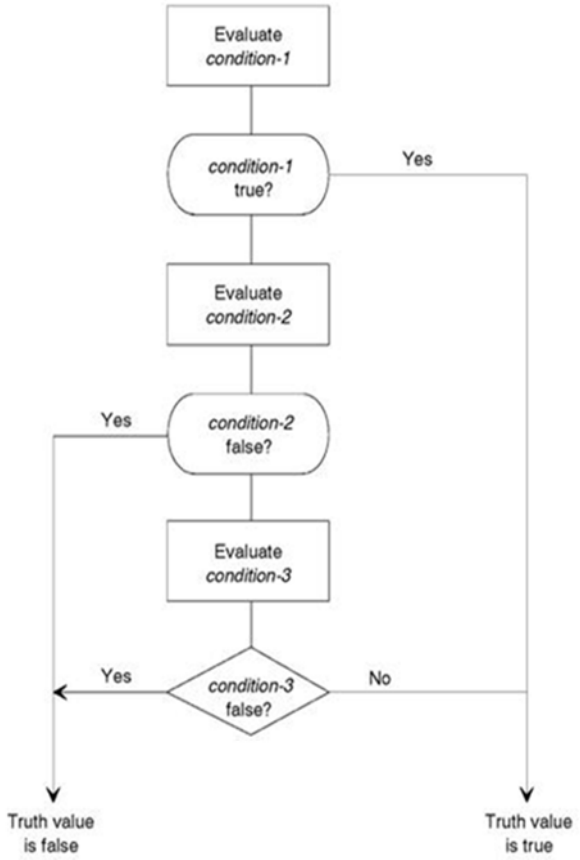


Figure 3–5. Evaluation of condition-1 OR condition-2 AND condition-3

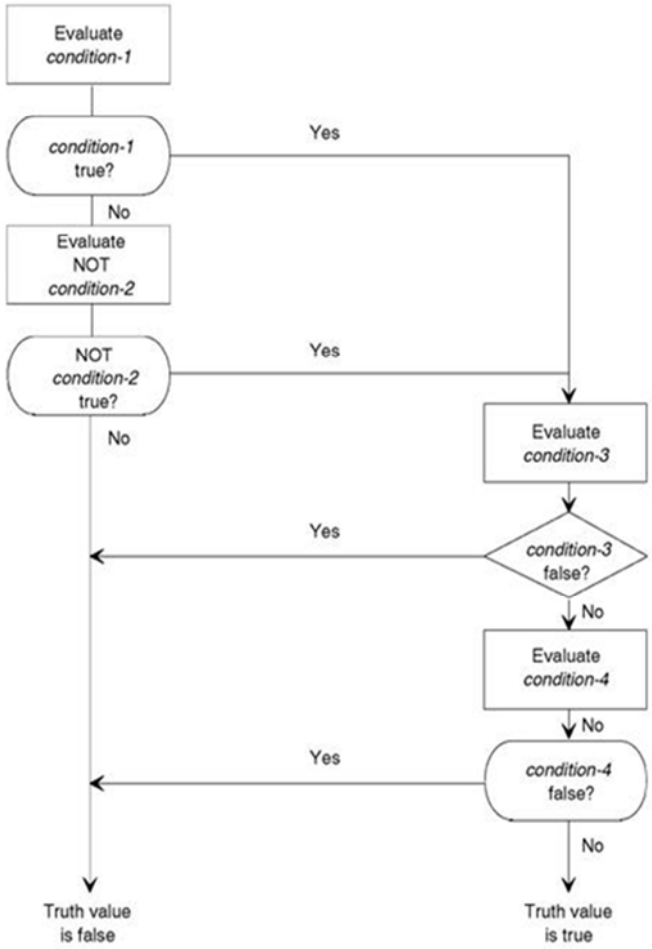


Figure 3–6. Evaluation of (condition-1 OR NOT condition-2) AND condition-3 AND condition-4

3.6.5. Common Options and Rules for Statements

Section 3.6.5 provides a description of the common options and conditions that pertain to or appear in several different statements.

The ROUNDED Phrase

If, after decimal point alignment, the number of places in the fractions of the result of an arithmetic operation is greater than the number of places provided for the fraction of the resultant identifier, truncation is relative to the size provided for the resultant identifier. When rounding is requested, the absolute value of the resultant identifier is increased by one in the low-order position whenever the most significant digit of the excess is greater than or equal to five.

When the low-order integer positions in a resultant identifier are represented by the character P in the picture for that resultant identifier, rounding or truncation occurs relative to the rightmost integer position for which storage is allocated.

For example, if addition operands are

```
02 A PIC 99V99 VALUE 10.51.
02 B PIC 99V9(3) VALUE 96.299.
02 C PIC 999.
02 D PIC 999V9(3).
02 E PIC 999V9.
02 F PIC 999P.
```

and the statements executed are

```
ADD A , B GIVING C ROUNDED.
ADD A , B GIVING C.
ADD A , B GIVING D ROUNDED.
ADD A , B GIVING D.
ADD A , B GIVING E ROUNDED.
ADD A , B GIVING F ROUNDED.
```

the results are C = 107, C = 106, D = 106.809, D = 106.809, E = 106.8, and F = 011.

The ON SIZE ERROR Phrase

The size error condition occurs under the following circumstances:

1. Violation of the rules for evaluation of exponentiation always terminates the arithmetic operation and always causes a size error condition. (See Section 3.6.2)
2. Division by zero always terminates the arithmetic operation and always causes a size error condition.

3. If, after radix point alignment or normalization, the absolute value of a result exceeds the largest value that can be contained in the associated resultant identifier, a size error condition exists.

   In the case where the USAGE IS BINARY clause is specified for the resultant identifier, the largest value that can be contained in the resultant identifier is the maximum value implied by the associated decimal PICTURE character string. If the ROUNDED phrase is specified, rounding takes place before checking for size error.

If the ON SIZE ERROR phrase is specified and a size error condition exists after the execution of the arithmetic operations specified by an arithmetic statement, the values of the affected resultant identifiers remain unchanged from the values they had before execution of the arithmetic statement. The values of resultant identifiers for which no size error condition exists are the same as they would have been if the size error condition had not resulted for any of the resultant identifiers. After completion of the arithmetic operations, control is transferred to the imperative-statement specified in the ON SIZE ERROR phrase and execution continues according to the rules for each statement specified in that imperative-statement. If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of the imperative-statement specified in the ON SIZE ERROR phrase, control is transferred to the end of the arithmetic statement and the NOT ON SIZE ERROR phrase, if specified, is ignored.

If the ON SIZE ERROR phrase is not specified and a size error condition exists after the execution of the arithmetic operations specified by an arithmetic statement, the values of the affected resultant identifiers are undefined. The values of resultant identifiers for which no size error condition exists are the same as they would have been if the size error condition had not resulted for any of the resultant identifiers. After completion of the arithmetic operations, control is transferred to the end of the arithmetic statement and the NOT ON SIZE ERROR phrase, if specified, is ignored.

If the size error condition does not exist after the execution of the arithmetic operations specified by an arithmetic statement, the ON SIZE ERROR phrase, if specified, is ignored and control is transferred to the end of the arithmetic statement or to the imperative statement specified in the NOT ON SIZE ERROR phrase if it is specified. In the latter case, execution continues according to the rules for each statement specified in that imperative statement. If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of the imperative statement specified in the NOT ON SIZE ERROR phrase, control is transferred to the end of the arithmetic statement.

For the ADD statement with the CORRESPONDING phrase and the SUBTRACT statement with the CORRESPONDING phrase: If any of the individual operations produces a size error condition, *imperative-statement-1* in the SIZE ERROR phrase is not executed until all of the individual additions or subtractions are completed.

For example, if data fields are defined as

```
02 A PIC 99V9.
02 B PIC 99V9.
02 C PIC 999V9.
```

the statement ADD A, B GIVING C does not produce a size error condition if A and B contain numeric values. If, however, a sum is being developed such as

```
ADD A TO B.
```

a size error condition can occur.

Note: *For an item with the USAGE BINARY clause, the test for SIZE ERROR condition uses the maximum decimal value according to the original PICTURE regardless of bit size. For example:* **A1 PIC 99 BINARY**. *A1 is allocated 9 bits. A value of 100 can fit in 9 bits but it would cause a size error condition.*

For floating-point arithmetic a size error condition occurs if a hardware floating-point interrupt occurs or if a floating-point intermediate result cannot be contained in the designated receiving field. Exponentiation is accomplished in floating-point arithmetic. If a floating-point interrupt occurs during an exponentiation, it is treated as a run time error condition rather than a size error condition and the execution is terminated. Termination occurs if

1. The result has a zero characteristic and a mantissa less than or equal to zero.
2. The result has a characteristic less than zero and a nonintegral mantissa.
3. The result is such that the mantissa times the LN of the characteristics is greater than 709.089.

The CORRESPONDING Phrase

For the purpose of this discussion, d1 and d2 must each be identifiers that refer to group items. A pair of data items, one from d1 and one from d2 corresponds if the following conditions exist:

1. A data item in d1 and a data item in d2 are not designated by the keyword FILLER and have the same data name and the same qualifiers up to, but not including, d1 and d2.
2. At least one of the data items is an elementary data item and the resulting move is legal according to the move rules in the case of a MOVE statement with the CORRESPONDING phrase; and both of the data items are elementary numeric data items in the case of the ADD statement with the CORRESPONDING phrase or the SUBTRACT statement with the CORRESPONDING phrase.
3. The description of d1 and d2 must not contain level numbers 66, 77, or 88 or the USAGE IS INDEX clause.

4. A data item that is subordinate to d1 or d2 and contains a REDEFINES, RENAMES, OCCURS, or USAGE IS INDEX clause is ignored, as well as those data items subordinate to the data item that contains the REDEFINES, OCCURS, or USAGE IS INDEX clause. Neither d1 nor d2 can be referenced modified.
5. The name of each data item that satisfies the above conditions must be unique after application of the implied qualifiers.

The following example shows the result of a CORRESPONDING addition and move. Assume that the data is defined:

```
02 GROUP1.
     03 A.
         04 A1 PIC S99.
         04 FILLER PIC X.
         04 A2 PIC S9(4)V99.
     03 B OCCURS 2 TIMES
         04 B1 PIC 9(3).
         04 B2 PIC SV999.
     03 C.
         04 C1.
             05 C11 PIC PP99.
             05 C12 PIC 9(6) COMP.
         04 C2 PIC 99.

02 GROUP2.
     05 C.
        07 C1.
           09 C11 PIC 9V9(4).
           09 C12 PIC 9(6).
        07 C2.
           09 C21 PIC 9.
           09 C22 PIC 9.
     05 B.
        07 B1 PIC 9(3).
        07 B2 PIC SV999.
     05 A.
        07 A2 COMP-1.
        07 A1 COMP-2.
        07 FILLER PIC X.
```

The statement

```
ADD CORRESPONDING GROUP1 TO GROUP2.
```

is equivalent to

```
ADD C11 OF GROUP1 TO C11 OF GROUP2.
ADD C12 OF GROUP1 TO C12 OF GROUP2.
ADD A1 OF GROUP1 TO A1 OF GROUP2.
ADD A2 OF GROUP1 TO A2 OF GROUP2
```

The statement

```
MOVE CORRESPONDING GROUP1 TO GROUP2
```

is equivalent to

```
MOVE C11 OF GROUP1 TO C11 OF GROUP2.
MOVE C12 OF GROUP1 TO C12 OF GROUP2.
MOVE C2 OF GROUP1 TO C2 OF GROUP2.
MOVE A1 OF GROUP1 TO A1 OF GROUP2.
MOVE A2 OF GROUP1 TO A2 OF GROUP2.
```

B, B1, and B2 are never involved because of the presence of an OCCURS clause on B in GROUP1. The field C2 is moved but not added because C2 in GROUP2 is not an elementary item.

The statement

```
MOVE CORRESPONDING B OF GROUP1(2) TO B OF GROUP2
```

is equivalent to

```
MOVE B1 OF GROUP15(2) TO B1 OF GROUP2.
MOVE B2 OF GROUP15(2) TO B2 OF GROUP2.
```

The Arithmetic Statements

The arithmetic statements are the ADD, COMPUTE, DIVIDE, MULTIPLY, and SUBTRACT statements. They have several common features.

1. The data descriptions of the operands need not be the same; any necessary conversion and decimal point alignment are supplied throughout the calculation.
2. The maximum size of each operand is 18 decimal digits. The composite of operands, which is a hypothetical data item resulting from the superimposition of specified operands in a statement aligned on their decimal points (see Section 3.6.7 for the ADD statement, Section 3.6.12 for the DIVIDE statement, 3.6.20 for the MULTIPLY statement, and Section 3.6.26 for the SUBTRACT statement), must not contain more than 18 decimal digits.

Overlapping Operands

When a sending and a receiving item in any statement share a part or all of their storage areas, yet are not defined by the same data description entry, the result of the execution of such a statement is undefined. In addition, the results are undefined for some statements in which sending and receiving items are defined by the same data description entry. These cases are addressed in the general rules associated with those statements.

Multiple Results in Arithmetic Statements

The ADD, COMPUTE, DIVIDE, MULTIPLY, and SUBTRACT statements can have multiple results. Such statements behave as though they had been written in the following way:

1. A statement whose execution accesses all data items that are part of the initial evaluation of the statement performs any necessary arithmetic or combining of these data items and stores the result of this operation in a temporary location. See the individual statements for the rules indicating which items are part of the initial evaluation.
2. A sequence of statements whose execution transfers or combines the value in this temporary location with each single resulting data item. These statements are considered to be written in the same left-to-right sequence that the multiple results are specified. The result of the statement

```
ADD a, b, c TO c, d (c), e
```

is equivalent to

```
ADD a, b, c GIVING temp
ADD temp TO c
ADD temp TO d (c)
ADD temp TO e
```

and the result of the statement

```
MULTIPLY a i BY i, a (i)
```

is equivalent to

```
MOVE a (i) TO temp
MULTIPLY temp BY i
MULTIPLY temp BY a (i)
```

In both cases, *temp* is an intermediate result item provided by the compiler.

Incompatible Data

Except for the class condition, when the content of a data item is referenced in the Procedure Division and the content of that data item is not compatible with the class specified for that data item by its PICTURE clause or function definition, then the result of such a reference is undefined (see Section a).

Arithmetic Intermediate Results

Whenever an arithmetic expression is resolved or an arithmetic statement is executed, there is an intermediate result that has characteristics determined by the characteristics of the operands and in some cases by the characteristics of the eventual resultant field.
These characteristics are mode (USAGE of the item), sign (the presence or absence of an operational sign), size (the number of digit positions that the item can contain), and scaling (the position of the assumed decimal point). The characteristics determine what values an item can assume and how the calculations involving the item are made.

When the characteristics and the values are not compatible, either a truncation or a size error condition exists. A truncation condition occurs when the value, after decimal alignment, contains excessive low order digits. A size error condition occurs when the value after decimal alignment contains excessive high order digits.

The statements ADD, SUBTRACT, MULTIPLY, and COMPUTE allow you to correct or compensate when either situation occurs by using the ROUNDED and SIZE ERROR options.

Arithmetic expressions in the PERFORM, IF, and SEARCH statements, however, do not provide alternate actions and can produce either left or right truncation of the resolved value of the expression.

The characteristics of intermediate results, regardless of the statement type, are described in the following paragraphs.

The compiler deals with the arithmetic operations separately for each pair of operands. Take for example the following statement:

```
COMPUTE A = B + C - D
```

B and C determine the characteristics of the first intermediate result of the addition. The sum and the field D determine the characteristics of the final intermediate result. The characteristics of A are used to determine the correct way to move the result to the final receiving fields. Therefore, the preceding example is equivalent to the following sequence:

```
COMPUTE temp1 = B + C
COMPUTE temp2 = temp1 - D
MOVE temp2 TO A
```

Each value is aligned and each result is computed based solely on the characteristics of the operands involved in the intermediate calculation. Only for division are the characteristics of scale and size of the receiving field used to determine the characteristics of the intermediate result.

Arithmetic results are calculated in one of three ways. The method is determined by the USAGE of the fields involved and the operator specified. The three modes of calculation are fixed DECIMAL, fixed BINARY, and FLOATING-POINT binary.

Since OS 2200 systems are intrinsically binary rather than decimal, binary fields can actually contain more digits of precision than are indicated by their PICTURE clause. For example, an S9(10) BINARY item is allocated as a 36-bit field. Those 36 bits can contain an absolute value up to 34,359,738,367.

If during intermediate calculations, such a field contains a value greater than 2^{35}, the full value is used in subsequent intermediate calculations.

Similarly, a full two-word binary number can contain 21 digits of significance. Therefore, intermediate results of binary calculations are held and used in subsequent intermediate calculations as maximum 21-digit capacity items. During calculations involving binary items, the compiler assures that intermediate results are sufficiently large to retain precision and significance.

Floating-point intermediate results are maintained as either single or double precision floating-point items, whichever is sufficiently large to hold the calculated results. The capacity of floating-point data items is sufficiently large for most applications. Single precision floating-point (USAGE COMP-1) can represent a binary characteristic (exponent) of 127 through -128 applied to a mantissa (base) of $\pm 2^{27} - 1$.

COMPUTATIONAL-2 fields and double precision floating-point intermediate results can represent a binary characteristic of 1023 to -1024 applied to a mantissa of $\pm 2^{60} - 1$. Floating-point calculations that result in invalid floating-point values are explained in 3.6.5.

If either or both operands of any calculation are floating-point (USAGE COMPUTATIONAL-1 or USAGE COMPUTATIONAL-2) or if exponentiation is required, or if the number of significant digits exceeds the maximum allowed for the fixed decimal or fixed binary operations, calculations are performed in floating-point, with any nonfloating-point items being converted to floating-point format.

The following defines the algorithms for calculating the size and assumed decimal point of nonfloating-point arithmetic intermediate results. In the following table, the symbols

N1 and N2

are the number of integer places in operand 1 and 2.

N3

is the number of integer places in the intermediate sum, difference, product, or quotient.

N4

is the number of integer places in the remainder of division operation.

D1, D2, D3, D4

are the number of scaling positions in the intermediate result.

NR and DR

are the number of integer and decimal digits in the ultimate receiving field. If there are multiple receiving fields, the largest NR and DR are used. In this case, the NR and DR do not necessarily derive from the same receiving field.

In order to provide 21 digits of decimal precision for COBOL programs, the internal type of float decimal is used to provide this capability. Float decimal maintains the most significant 24 digits in its mantissa. Operations involving float decimal are software simulated by library routines.

Table 3–8. Arithmetic Statements and Expressions

Operation	Case		Integer Places	Decimal Places
+ - ADD SUBTRACT			N3 = 21, or N3 = maximum of N1 + 1 and N2 + 1, whichever is smaller If (MAX (N1,N2) + MAX (D1,D2) > 21 then the operation is done in float decimal.	D3 = 21 - N3, or D3 = maximum of D1 and D2, whichever is smaller
* MULTIPLY			N3 = N1 + N2 If N3 + D3 > 21 then the operation is done in float decimal.	D3 = D1 + D2
/	Operand 1 is BINARY-1	N3 + D3 =< 20	N3 = N1 + D2 N4 = N3 + D3 - D4	D3=DR(see Note 2) D4 = D2 + DR
		N3 + D3 > 20	The operation is done in float decimal.	
			N4 = N1 + 1, or N2 + NR + 1, whichever is greater	D4 = D2 +DR
DIVIDE	Operand 1 and Operand-2 are BINARY -1	N3 + D3 =< 20	N3 = N1 or N2 whichever is greater N4 = N3 +D3 - D4	D3=DR(see Note 2) D4 = DR
		N3 + D3 > 20	The operation is done in float decimal.	
			N4 = N1 + 1, or N2 + NR + 1 whichever is greater	D4 = DR

Table 3–8. Arithmetic Statements and Expressions

Operation	Case		Integer Places	Decimal Places
	All other cases	N3 + D3 =< 21	N3 = N1 + D2 N4 = N3 + D3 - D4	D3 = DR or D1 whichever is greater D4 = D2 + DR, or D1, whichever is greater(see Note 2)
		N3 + D3 > 21	The operation is done in float decimal.	
			N4 = N1 + 1, or N2 + NR + 1 whichever is greater	D4 = D2 + DR , or D1 whichever is greater

N1 and N2 are the number of integer places in operand 1 and 2.

N3 is the number of integer places in the intermediate sum, difference, product, or quotient.

N4 is the number of integer places in the remainder of division operation.

D1, D2, D3, D4 are the number of scaling positions in the intermediate result.

NR and DR are the number of integer and decimal digits in the ultimate receiving field. If there are multiple receiving fields, the largest NR and DR are used. In this case, the NR and DR do not necessarily derive from the same receiving field.

Notes:

- *For the operators +, -, *, and /, if float decimal is used once in an arithmetic statement, all additional operations are done in float decimal.*
- *When ROUNDED is specified, DR = DR + 1*

Implicit Arithmetic Calculations

Several COBOL statements perform arithmetic calculations during the execution of the statement. The compiler generates implicit arithmetic calculations for statements such as INSPECT TALLYING, PERFORM VARYING, and UNSTRING TALLYING. The arithmetic calculations are required for the statements to be executed and for the various fields to be filled in properly. You must declare the user variables large enough so that they can hold the largest possible value that can be returned by the COBOL statements as a result of the implicit arithmetic calculations. If you do not declare the variables large enough, then a size error condition can occur when the implicit arithmetic calculation is performed. When the error occurs, an unexpected value might be returned into the user defined variable because an ON SIZE ERROR clause cannot be specified with these statements.

3.6.6. The ACCEPT Statement

Function

The ACCEPT statement causes low volume data to be made available to the specified data item.

Format 1

```
ACCEPT  identifier -1 [ FROM mnemonic-name-1 ]
```

Format 2

```
ACCEPT identifier-2 FROM { DATE [YYYYMMDD]
                         { DAY [YYYYDDD]
                         { DAY-OF-WEEK
                         { TIME
```

Syntax Rule

Mnemonic-name-1 in format 1 must also be specified in the SPECIAL-NAMES paragraph of the Environment Division and must be associated with the implementor names CARD-READER or CONSOLE.

General Rules

Format 1

1. The ACCEPT statement causes the transfer of data from the hardware device. This data replaces the contents of the data item referred to by *identifier-1.*

   When the device specified is CONSOLE, programs executing in batch mode accepts data from the operator's console. In batch mode, CARD-READER specifies the program card file device. For programs executing in the demand mode, CONSOLE and CARD-READER both specify the demand terminal.

   The execution of an ACCEPT statement specifying the operator's terminal or demand terminal does not permit execution of subsequent instructions until a response is made. It is, therefore, usually advisable to precede such ACCEPT statements with a DISPLAY to the console or terminal explaining what data is desired.

2. A signed numeric data item can be added to the program by means of an ACCEPT statement with either the overpunched sign format or the leading sign format. The compiler implements all necessary conversion to accept these two formats.

3. The amount of data that can be handled in a single transfer differs with the type of device.

Specified Device Implementor Name	Size of a Single Data Transfer
CARD-READER	80 characters for programs executing in batch mode
	72 characters for programs executing in demand mode
CONSOLE	60 characters for programs executing in batch mode
	72 characters for programs executing in demand mode

4. If a hardware device can transfer data of the same size as the receiving data item, the transferred data is stored in the receiving data item.
5. If a hardware device cannot transfer data of the same size as the receiving data item and the receiving item is alphanumeric, one of two conditions exists:
   a. If the size of the receiving data item (or of the portion of the receiving data item not yet currently occupied by transferred data) exceeds the size of the transferred data, the transferred data is stored aligned to the left in the receiving data item (or the portion of the receiving data item not yet occupied), and additional data is requested. In level 1, only one transfer of data is provided.
   b. If the size of the transferred data exceeds the size of the receiving data item (or the portion of the receiving data item not yet occupied by transferred data), only the leftmost characters of the transferred data are stored in the receiving data item (or in the portion remaining). The remaining characters of the transferred data that do not fit into the receiving data item are ignored.
6. For a numeric receiving field, data preceding a space in the field is assumed to be an integer value that is converted as necessary, aligned to the right and zero-filled, and treated as an integer for subsequent transfer to the receiving field.
7. The acceptance devices all use character data representation. Therefore, input is in ASCII characters. When the receiving field has a usage of other than ASCII, the data is converted and then placed in the receiving area.

   The actual value of the accepted data is not examined. Therefore, coding a value in the form of a floating-point constant will be treated simply as a character string that happens to have the character E embedded in it.
8. If the FROM phrase is not given, CONSOLE is assumed to be the input device.

Format 2

1. The ACCEPT statement causes the information requested to be transferred to the data item specified by *identifier-2* according to the rules for the MOVE statement. (See Section 3.6.19) DATE, DAY, DAY-OF-WEEK, and TIME are conceptual data items and, therefore, are not described in the COBOL program.

2. DATE, without the phrase YYYYMMDD, is composed of the data elements year of the current century, month of the year, and day of the month. DATE, without the phrase YYYYMMDD, when accessed by a COBOL program, behaves as if it had been described as an unsigned elementary integer data item of usage display six digits in length, the character positions of which, numbered from left to right, are:

   a. 1-2: The two rightmost numeric characters of the year in the Gregorian calendar.

   b. 3-4: Two numeric characters of the month of the year in the range 01 through 12.

   c. 5-6: Two numeric characters of the day of the month in the range 01 through 31.

   DATE, with the phrase YYYYMMDD, is composed of the data elements year in the Gregorian calendar, month of the year, and day of the month. DATE, with the phrase YYYYMMDD, when accessed by a COBOL program, behaves as if it had been described as an unsigned elementary integer data item of usage display eight digits in length, the character positions of which, numbered from left to right are:

   a. 1-4: Four numeric characters in the year of the Gregorian calendar.

   b. Two numeric characters of the month of the year in the range 01 through 12.

   c. Two numeric characters of the day of the month in the range 01 through 31.

3. DAY, without the phrase YYYYDDD, is composed of the data elements year of the current century and day of the year. DAY, without the phrase YYYYDDD, when accessed by a COBOL program, behaves as if it had been described as an unsigned elementary integer data item of usage display five digits in length, the character positions of which numbered from left to right are:

   a. 1-2: The two rightmost numeric characters of the year in the Gregorian calendar.

   b. 3-5: Three numeric characters of the day of the year in the range 01 through 366.

   DAY, with the phrase YYYYDDD, is composed of the data elements year in the Gregorian calendar and day of the year. DAY, with the phrase YYYYDDD, when accessed by a COBOL program, behaves as if it had been described as an unsigned elementary integer data item of usage display seven digits in length, the character positions of which, numbered from left to right, are:

   a. 1-4: Four numeric characters of the year in the Gregorian calendar.

   b. 5-7: Three numeric characters of the day of the year in the range 01 through 366.

4. TIME is composed of the data elements hours, minutes, seconds, and hundredths of a second. TIME is based on elapsed time after midnight on a 24-hour clock basis—therefore, 2:41 p.m. is expressed as 14410000. TIME, when accessed by a COBOL program, behaves as if it had been described in a COBOL program as an unsigned elementary numeric integer data item eight digits in length. The minimum value of TIME is 00000000; the maximum value of TIME is 23595999.

5. DAY-OF-WEEK is composed of a single data element whose content represents the day of the week. DAY-OF-WEEK, when accessed by a COBOL program, behaves as if it had been described in the COBOL program as an unsigned elementary numeric integer data item one digit in length. In DAY-OF-WEEK, the value 1 represents Monday, 2 represents Tuesday, ... , 7 represents Sunday.

3.6.7. The ADD Statement

Function

The ADD statement causes two or more numeric operands to be summed and the result to be stored.

Format 1

```
ADD { identifier-1 } ... TO { identifier-2 [ ROUNDED ] } ...
    { literal-1    }
  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-ADD ]
```

Format 2

```
ADD { identifier-1 } ... TO { identifier-2 } GIVING { identifier-3 [ ROUNDED ] } ...
    { literal-1    }       { literal-2    }
      [ ON SIZE ERROR imperative-statement-1 ]
      [ NOT ON SIZE ERROR imperative-statement-2 ]
      [ END-ADD ]
```

Format 3

```
ADD { CORRESPONDING } identifier-1 TO identifier-2 [ ROUNDED ]
    { CORR          }
  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-ADD ]
```

Syntax Rules

1. In formats 1 and 2, each identifier must refer to an elementary numeric item, except that in format 2 each identifier following the word GIVING must refer to either an elementary numeric item or an elementary numeric-edited item. In format 3, each identifier must refer to a group item.
2. Each literal must be a numeric literal.
3. The composite of operands must not contain more than 18 digits (see Section 3.6.5).
   a. In format 1, the composite of operands is determined by using all of the operands in a given statement.

b. In format 2, the composite of operands is determined by using all of the operands in a given statement, excluding the data items that follow the word GIVING.

c. In format 3, the composite of operands is determined separately for each pair of corresponding data items.

d. CORR is an abbreviation for CORRESPONDING.

General Rules

1. If format 1 is used, the values of the operands preceding the word TO are added together and the sum is stored in a temporary data item. The value in this temporary data item is added to the value of the data item referenced by *identifier-2,* storing the result into the data item referenced by *identifier-2* and repeating this process for each successive occurrence of *identifier-2* in the left-to-right order in which *identifier-2* is specified.
2. If format 2 is used, the values of the operands preceding the word GIVING are added together; then the sum is stored as the new content of each data item referenced by *identifier-3.*
3. If format 3 is used, data items in *identifier-1* are added to and stored in corresponding data items in *identifier-2.*
4. The compiler ensures that enough places are carried so as not to lose any significant digits during execution.
5. The individual addends are decimal-point aligned prior to addition. The size of such aligned items must not exceed 18 decimal digits.

   For example, a field defined as V9(18) cannot be added to a field defined as 9(18). However, two fields defined as 9(7)P(2) can be added.
6. As long as the aligned size of items does not exceed 18 digits, the compiler ensures that enough places are carried so as not to lose any significant digits during execution.

   For example, a field defined as S9V9(5) and 99V9(4) can be added. The intermediate result can be conceptually considered as S9(3)V9(5) and is therefore large enough to contain the result of any legal values contained in the addends.
7. Additional rules and explanations relative to this statement are given under the appropriate paragraphs. (See Section 2.4.21 for the Scope Terminators, Section 3.6.5 for the ROUNDED phrase, Section 3.6.5 for the SIZE ERROR phrase, Section 3.6.5 for arithmetic statements, Section 3.6.5 for overlapping operands, and Section 3.6.5 for multiple results in arithmetic statements.

3.6.8. The ALTER Statement

Function

The ALTER statement modifies a predetermined sequence of operations. The ALTER statement is an obsolete element in this revision of American National Standard COBOL and is to be deleted from the next revision of American National Standard COBOL.

General Format

```
ALTER  { procedure-name-1 TO  [ PROCEED TO ]  procedure-name-2 }  ...
```

Syntax Rules

1. *Procedure-name-1* is the name of a paragraph that contains a single sentence consisting of a GO TO statement without the DEPENDING phrase.
2. *Procedure-name-2* is the name of a paragraph or section in the Procedure Division.

General Rules

1. Execution of the ALTER statement modifies the GO TO statement in the paragraph named *procedure-name-1* so that subsequent executions of the modified GO TO statement cause transfer of control to *procedure-name-2.*
2. A GO TO statement in a section whose segment number is greater than or equal to 50 must not be referred to by an ALTER statement in a section with a different segment number. All other uses of the ALTER statement are valid and are performed even if *procedure-name-1* is in a fixed segment that can be overlayed.

3.6.9. The COMPUTE Statement

Function

The COMPUTE statement assigns the value of an arithmetic expression to one or more data items.

General Format

```
COMPUTE  { identifier-1  [ ROUNDED ] } ...  = arithmetic-expression-1
  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-COMPUTE ]
```

Syntax Rules

Identifier-1 must reference either an elementary numeric item or an elementary numeric-edited item.

General Rules

1. An arithmetic expression consisting of a single identifier or literal provides a method of setting the value of the data item referenced by *identifier-1* equal to the literal or the value of the data item referenced by the single identifier.
2. If more than one identifier is specified for the result of the operation (that is, preceding the equal sign), the value of the arithmetic expression is developed and then this value is stored as the new value of each of the data items referenced by *identifier-1.*
3. The COMPUTE statement allows the user to combine arithmetic operations without the restrictions on composite of operands or receiving data items imposed by the arithmetic statements ADD, SUBTRACT, MULTIPLY, and DIVIDE.
4. Additional rules and explanations relative to this statement are given under the appropriate paragraphs. See Section 3.6.5 for the ROUNDED phase, SIZE ERROR phase, the arithmetic statements, overlapping operands, and for multiple results.

3.6.10. The CONTINUE Statement

Function

The CONTINUE statement is a no-operation statement. It indicates that no executable statement is present.

General Format

```
CONTINUE
```

Syntax Rule

The CONTINUE statement can be used anywhere a conditional statement or an imperative statement can be used.

General Rule

The CONTINUE statement has no effect on the execution of the program.

3.6.11. The DISPLAY Statement

Function

The DISPLAY statement causes low-volume data to be transferred to an appropriate hardware device.

General Format



Syntax Rules

1. *Mnemonic-name* is associated with a hardware device (implementor name PRINTER, CONSOLE, or CARD-PUNCH) in the SPECIAL-NAMES paragraph in the Environment Division.
2. If *literal* is numeric, it must be an unsigned integer.

General Rules

1. The DISPLAY statement causes the content of each operand to be transferred to the hardware device in the order listed.
2. For programs executing in batch mode, CONSOLE specifies the operator's console and PRINTER specifies the program print file device. For programs executing in demand mode, CONSOLE and PRINTER both specify the demand terminal. CARD-PUNCH always specifies the program punch file device.
3. The maximum size of data that can be handled in a single transfer depends on the receiving device.

Device	Mode	Size of Single Data Transfer
PRINTER	Batch	132
	Demand	132
CONSOLE	Batch	60
	Demand	132
CARD-PUNCH	N/A	80

4. If a figurative constant is specified as one of the operands, only a single occurrence of the figurative constant is displayed.
5. If the hardware device is capable of receiving data of the same size as the data item being transferred, the data item is transferred.
6. If a hardware device is not capable of receiving data of the same size as the data item being transferred, one of the following applies:
   a. If the size of the data item being transferred exceeds the size of the data that the hardware device is capable of receiving in a single transfer, the data beginning with the leftmost character is stored aligned to the left in the receiving hardware device, and the remaining data is then transferred according to general rules 5 and 6 until all the data has been transferred. The last transfer is stored aligned to the left and space-filled. In level 1, only one transfer of data is provided.
   b. If the size of the data item that the hardware device is capable of receiving exceeds the size of the data being transferred, the transferred data is stored aligned to the left in the receiving hardware device and space-filled.

7. When a DISPLAY statement contains more than one operand, the size of the sending item is the sum of the sizes associated with the operands, and the values of the operands are transferred in the sequence in which the operands are encountered without modifying the positioning of the hardware device between the successive operands.
8. If the UPON phrase is not specified, CONSOLE is assumed.
9. Numeric items that have an assumed decimal point are converted as necessary and displayed without an indication of the decimal location. Therefore, a field defined as 9(5)V9(5) is displayed as a 10-character value.

   Binary identifiers are converted to the number of digits stated in the PICTURE clause before being output. Exact binary identifiers are converted to the number of decimal digits required by standard binary-decimal equivalence. For example, an item identified as 1(3) is converted and displayed as an item identified as 9, while an item identified as 1(36) is converted and displayed as an item identified as 9(11).

   Floating-point identifiers (USAGE, COMPUTATIONAL-1 and COMPUTATIONAL-2) are displayed in floating-point notation with the following format:

   ```
   [± mantissa E [±] exponent
   ```

   Each DISPLAY statement begins a new image and continues over as many images as are necessary to display specified items, up to a maximum of 512 characters. For example

```
DISPLAY "TOTAL AMOUNT" SPACE TOTAL"AMOUNT
```

   results in

```
TOTAL AMOUNT 04800
```

   if the data item TOTAL-AMOUNT contained the value 04800.

   The location of the assumed decimal point is not considered. Therefore, the image in the previous example would be the same if TOTAL-AMOUNT were defined as 9(5), 99V999, or V9(5) with usages DISPLAY or COMPUTATIONAL.
10. If the WITH NO ADVANCING phrase is specified, then the positioning of the hardware device is not reset to the next line or changed in any other way following the display of the last operand.
11. If the WITH NO ADVANCING phrase is not specified, then after the last operand has been transferred to the hardware device, the positioning of the hardware device is reset to the leftmost position of the next line of the device.
12. If vertical positioning is not applicable on the hardware device, the operating system ignores the positioning specified or implied.
13. The DISPLAY statement prints signed numeric fields with an overpunch sign for USAGE DISPLAY and COMP items. It prints signed numeric fields with a leading separate sign for USAGE COMP-1, COMP-2, PACKED-DECIMAL, BINARY, and BINARY-1 items. If the keyword option COMPAT is specified, signed numeric fields are displayed with a leading separate sign for USAGE COMP and COMP-3, as well.

3.6.12. The DIVIDE Statement

Function

The DIVIDE statement divides one numeric data item into others and sets the values of data items equal to the quotient and remainder.

Format 1

```
DIVIDE { identifier-1 } INTO { identifier-2 [ ROUNDED ] } ...
       { literal-1    }
  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-DIVIDE ]
```

Format 2

```
DIVIDE { identifier-1 } INTO { identifier-2 } GIVING { identifier-3 [ ROUNDED ] } ...
       { literal-1    }      { literal-2    }
  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-DIVIDE ]
```

Format 3

```
DIVIDE { identifier-1 } BY { identifier-2 } GIVING { identifier-3 [ ROUNDED ] } ...
       { literal-1    }    { literal-2    }
  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-DIVIDE ]
```

Format 4

```
DIVIDE { identifier-1 } INTO { identifier-2 } GIVING { identifier-3 [ ROUNDED ] }
       { literal-1    }      { literal-2    }
  REMAINDER identifier-4  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-DIVIDE ]
```

Format 5

```
DIVIDE { identifier-1 } BY { identifier-2 } GIVING { identifier-3 [ ROUNDED ] }
       { literal-1    }    { literal-2    }
  REMAINDER identifier-4  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-DIVIDE ]
```

Syntax Rules

1. Each identifier must refer to an elementary numeric item, except that any identifier associated with the GIVING or REMAINDER phrase must refer to either an elementary numeric item or an elementary numeric-edited item.
2. Each literal must be a numeric literal.
3. The composite of operands must not contain more than 18 digits. The composite is the hypothetical data item that results from superimposing all receiving data items (except the REMAINDER data item) of a given statement aligned on their decimal points.

General Rules

1. When format 1 is used, *literal-1* or the value of the data item referred to by *identifier-1* is stored in a temporary data item. The value in this temporary data item is then divided into the value of the data item referred to by *identifier-2.* The value of the dividend (the value of the data item referred to by *identifier-2*) is replaced by this quotient; similarly, the temporary data item is divided into each successive occurrence of *identifier-2* in the left-to-right order in which *identifier-2* is specified.
2. When format 2 is used, *literal-1* or the value of the data item referred to by *identifier-1* is divided into *literal-2* or the value of the data item referred to by *identifier-2* and the result is stored in each data item referred to by *identifier-3.*
3. When format 3 is used, *literal-1* or the value of the data item referred to by *identifier-1* is divided by *literal-2* or the value of the data item referred to by *identifier-2* and the result is stored in each data item referred to by *identifier-3.*
4. When format 4 is used, *literal-1* or the value of the data item referred to by *identifier-1* is divided into *literal-2* or the value of the data item referred to by *identifier-2* and the result is stored in the data item referred to by *identifier-3.*

   The remainder is then calculated and the result is stored in the data item referred to by *identifier-4.* If *identifier-4* is subscripted, the subscript is evaluated immediately before the remainder is stored in the data item referred to by *identifier-4*.
5. When format 5 is used, *literal-1* or the value of the data item referred to by *identifier-1* is divided by *literal-2* or the value of the data item referred to by *identifier-2* and the division operation continues as specified for format 4.
6. Formats 4 and 5 are used when a remainder from the division operation is desired, namely *identifier-4.* The remainder in COBOL is defined as the result of subtracting the product of the quotient (*identifier-3*) and the divisor from the dividend. If *identifier-3* is defined as a numeric-edited item, the quotient used to calculate the remainder is an intermediate field that contains the unedited quotient. If ROUNDED is specified, the quotient used to calculate the remainder is an intermediate field that contains the quotient of the DIVIDE statement, truncated rather than rounded. This intermediate field is defined as a numeric field that contains the same number of digits, the same decimal point location, and the same presence or absence of a sign as the quotient (*identifier-3*).

7. In formats 4 and 5, the accuracy of the REMAINDER data item (*identifier-4*) is defined by the calculation described previously. Appropriate decimal alignment and truncation (not rounding) is performed for the value of the data item referred to by *identifier-4,* as needed.
8. When the ON SIZE ERROR phrase is used in formats 4 and 5, the following rules pertain:
   a. If the size error occurs on the quotient, no remainder calculation is meaningful. Therefore, the contents of the data items referred to by both *identifier-3* and *identifier-4* remains unchanged.
   b. If the size error occurs in the remainder, the content of the data item referred to by *identifier-4* remains unchanged. However, as with other instances of multiple results of arithmetic statements, the user's has to do their own analysis to recognize which situation has actually occurred.
9. Additional rules and explanations relative to this statement are given under the appropriate paragraphs. (See Section 2.4.21 and Section 3.6.5. See also general rules 6 and 8 for a presentation of the ROUNDED phrase and the SIZE ERROR phrase as they pertain to formats 4 and 5.)

In the example

```
DIVIDE A BY B GIVING C REMAINDER D
```

if the fields are defined as

```
A: 99V9
B: 9V99
C: 9999V99
D: 9V99
```

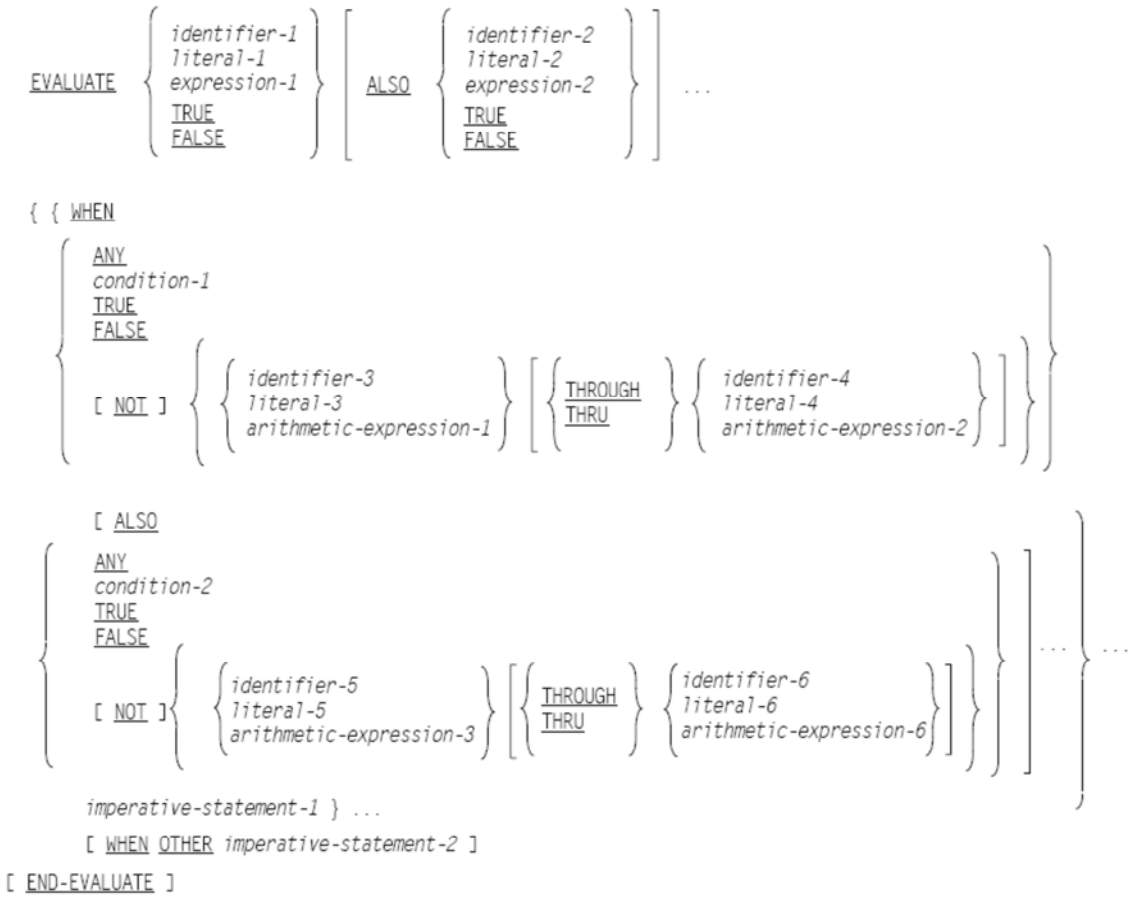
and contain only legal numeric values, no truncation or size error condition must arise, except for the zero divisor case.

3.6.13. The EVALUATE Statement

Function

The EVALUATE statement describes a multi-branch, multi-join structure. It can cause multiple conditions to be evaluated. The subsequent action of the object program depends on the results of these evaluations.

General Format



Syntax Rules

1. The operands or the words TRUE and FALSE that appear before the first WHEN phrase of the EVALUATE statement are referred to individually as selection subjects and collectively, for all those specified, as the set of selection subjects.
2. The operands or the words TRUE, FALSE, and ANY that appear in a WHEN phrase of an EVALUATE statement are referred to individually as selection objects and collectively, for all those specified in a single WHEN phrase, as the set of selection objects.
3. The words THROUGH and THRU are equivalent.
4. The operands connected by a THROUGH phrase must be of the same class. The two operands therefore connected constitute a single selection object.
5. The number of selection objects within each set of selection objects must be equal to the number of selection subjects.
6. Each selection object within a set of selection objects must correspond to the selection subject having the same ordinal position within the set of selection subjects according to the following rules:
   a. Identifier, literals, or arithmetic expressions appearing within a selection object must be valid operands for comparison to the corresponding operand in the set of selection subjects. (See Section a).
   b. *C ondition-1* or the words TRUE or FALSE appearing as a selection object must correspond to a conditional expression or the words TRUE or FALSE in the set of selection subjects.
   c. The word ANY can correspond to a selection subject of any type.

General Rules

1. The execution of the EVALUATE statement operates as if each selection subject and selection object were evaluated and assigned a numeric or nonnumeric value, a range of numeric or nonnumeric values, or a truth value. These values are determined as follows:
   a. Any selection subject specified by *identifier-1, identifier-2,* and any selection object specified by *identifier-3, identifier-5,* without either the NOT or the THROUGH phrases, are assigned the value and class of the data item referenced by the identifier.
   b. Any selection subject specified by *literal-1, literal-2,* and any selection object specified by *literal-3, literal-5,* without either the NOT or the THROUGH phrases, are assigned the value and class of the specified literal. If *literal-3, literal-5* is the figurative constant ZERO, it is assigned the class of the corresponding selection subject.
   c. Any selection subject in which *expression-1, expression-2* is specified as an arithmetic expression and any selection object, without either the NOT or the THROUGH phrases, in which *arithmetic-expression-1, arithmetic-expression-3* is specified are assigned a numeric value according to the rules for evaluating an arithmetic expression. (See Section 3.6.2.).

d. Any selection subject in which *expression-1, expression-2* is specified as a conditional expression and any selection object in which *condition-1, condition-2* is specified are assigned a truth value according to the rules for evaluating conditional expressions. (See Section a.).

e. Any selection subject or any selection object specified by the words TRUE or FALSE is assigned a truth value. The truth value "true" is assigned to those items specified with the word TRUE, and the truth value "false" is assigned to those items specified with the word FALSE.

f. Any selection object specified by the word ANY is not further evaluated.

g. If the THROUGH phrase is specified for a selection object without the NOT phrase, the range of values is all values that, when compared to the selection subject, are greater than or equal to the first operand and less than or equal to the second operand according to the rules for comparison. (See Section a) If the first operand is greater than the second, there are no values in the range.

h. If the NOT phrase is specified for a selection object, the values assigned to that item are all values not equal to the value, or range of values, that would have been assigned to the item had the NOT phrase not been specified.

2. The execution of the EVALUATE statement then proceeds as if the values assigned to the selection subjects and selection objects were compared to determine if any WHEN phrase satisfies the set of selection subjects. This comparison proceeds as follows:

   a. Each selection object within the set of selection objects for the first WHEN phrase is compared to the selection subject having the same ordinal position within the set of selection subjects. One of the following conditions must be satisfied if the comparison is to be satisfied:

      - If the items being compared are assigned numeric or nonnumeric values, or a range of numeric or nonnumeric values, the comparison is satisfied if the value, or one of the range of values, assigned to the selection object is equal to the value assigned to the selection subject according to the rules for comparison. (See Section a).
      - I f the items being compared are assigned truth values, the comparison is satisfied if the items are assigned the identical truth value.
      - If the selection object being compared is specified by the word ANY, the comparison is always satisfied regardless of the value of the selection subject.

   b. If the above comparison is satisfied for every selection object within the set of selection objects being compared, the WHEN phrase containing that set of selection objects is selected as the one satisfying the set of selection subjects.

   c. If the above comparison is not satisfied for one or more selection objects within the set of selection objects being compared, that set of selection objects does not satisfy the set of selection subjects.

   d. This procedure is repeated for subsequent sets of selection objects, in the order of their appearance in the source program, until either a WHEN phrase satisfying the set of selection subjects is selected or until all sets of selection objects are exhausted.

3. After the comparison operation is completed, execution of the EVALUATE statement proceeds as follows:
   a. If a WHEN phrase is selected, execution continues with the first *imperative-statement-1* following the selected WHEN phrase.
   b. If no WHEN phrase is selected and a WHEN OTHER phrase is specified, execution continues with *imperative-statement-2.*
   c. The scope of execution of the EVALUATE statement is terminated when execution reaches the end of the scope of the selected WHEN phrase or WHEN OTHER phrase or when no WHEN phrase is selected and no WHEN OTHER phrase is specified. (See Section 2.4.21).

Examples

```
01  DN-1  PIC  X(4).
01  DN-3  PIC  X(8).
01  DN-4  PIC  X(8)  VALUE 'XYZ'.
01  DN-5  PIC  999  VALUE 3.
01  DN-6  PIC  999  VALUE 2.
01  DN-7  PIC  X(8)  VALUE SPACES.
01  DN-8  PIC  999  VALUE 6.
01  DN-9  PIC  X(8)  VALUE 'ABCD'.

    EVALUATE  DN-1
       WHEN  'ABCD'  MOVE 'when 1' TO DN-3
       WHEN  OTHER  MOVE 'No Chg' TO DN-3.
```

If DN-1 contains the value 'ABCD', then 'when 1' is moved to DN-3; otherwise, 'No Chg' is moved to DN-3.

```
EVALUATE  DN-4   ALSO   'ABCD'   ALSO   DN-5 + DN-6
   WHEN   DN-7   ALSO    DN-4    ALSO   ZERO THRU 4    MOVE 'when 1' TO
                                                      DN-3
   WHEN   DN-9   ALSO    DN-7    ALS0   DN-8           MOVE 'when 2' TO
                                                      DN-3
   WHEN  'XYZ'   ALSO    DN-9    ALSO   NOT 6 THRU 10  MOVE 'when 3' TO
                                                      DN-3
```

In this example, 'when 3' is moved to DN-3 because DN-4 = 'XYZ' and 'ABCD' = DN-9 and DN-5 + DN-6 is not equal to 6 through 10. Since this was the first set of objects that matches the subjects, it resulted in DN-3 being filled in with 'when 3'.

```
EVALUATE  DN-5 = DN-6   ALSO   TRUE   ALSO          DN-6 = 7
   WHEN   FALSE         ALSO   ANY    ALSO          TRUE
                               MOVE 'when 1' TO DN-3
   WHEN   FALSE         ALSO   DN-5 * DN-6 = DN-8   ALSO   DN-8 < DN-5
                               MOVE 'when 2' TO DN-3
   WHEN   ANY           ALSO   ANY                  ALSO   ANY
                               MOVE 'when 3' TO DN-3
```

In this example, 'when 2' is moved to DN-3.

```
EVALUATE    DN-5          ALSO     DN-6
  WHEN      ANY           ALSO     NOT 2   MOVE 'when 1' TO DN-3
  WHEN      DN-8 - DN-5   ALSO     ANY     MOVE 'when 2' TO DN-3
  WHEN      9             ALSO     2       MOVE 'when 3' TO DN-3
```

In this example, 'when 2' is moved to DN-3.

3.6.14. The EXIT Statement

Function

The EXIT statement provides a common end point for a series of procedures.

General Format

```
EXIT
```

Syntax Rule

The EXIT statement must appear only in a sentence by itself and comprise the only sentence in the paragraph.

General Rule

An EXIT statement assigns a procedure name to a given point in a program. An EXIT statement has no other effect on the compilation or execution of the program.

The following example shows the case of the simple EXIT as a common transfer point:

```
.
    MOVE 1 TO A.
    MOVE 9 TO B.
    PERFORM PARA1 THRU PARA2.
   .
   .
   .
PARA1.
    IF A = 0 GO TO PARA2.
    ADD 1 TO A.
    IF A = B GO TO PARA2.
    GO TO PARA1.
PARA2.
    EXIT.
PARA3.
```

3.6.15. The GO TO Statement

Function

The GO TO statement transfers control from one part of the Procedure Division to another. The option of *procedure-name-1* in format 1 of the GO TO statement is an obsolete element in this revision of American National Standard COBOL and is to be deleted from the next revision of American National Standard COBOL.

Format 1

```
GO TO  procedure-name-1
```

Format 2

```
GO TO {procedure-name-1}. . . DEPENDING ON identifier-1
```

Syntax Rules

1. *Identifier-1* must refer to a numeric elementary data item that is an integer.
2. When a paragraph is referenced by an ALTER statement, that paragraph can consist only of a paragraph header followed by a format 1 GO TO statement.
3. A format 1 GO TO statement, without *procedure-name-1,* can appear only in a single statement paragraph.
4. If a GO TO statement represented by format 1 appears in a consecutive sequence of imperative statements within a sentence, it must appear as the last statement in that sequence.

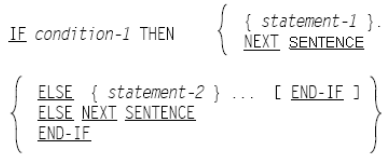
General Rules

1. When a GO TO statement represented by format 1 is executed, control is transferred to *procedure-name-1.*
2. If *procedure-name-1* is not specified in format 1, an ALTER statement, referring to this GO TO statement, must be executed prior to the execution of this GO TO statement.
3. When a GO TO statement represented by format 2 is executed, control is transferred to *procedure-name-1, procedure-name-2,* and so forth, depending on the value of *identifier-1* being 1, 2, 3, ..., *n.* If the value of the *identifier-1* is anything other than the positive or unsigned integers 1, 2, 3, . . ., *n,* then no transfer occurs and control passes to the next statement in the normal sequence for execution.

3.6.16. The IF Statement

Function

The IF statement causes a condition (see Section a) to be evaluated. The subsequent action of the object program depends on whether the value of the condition is true or false.



Syntax Rules

1. *Statement-1* and *statement-2* represent either an imperative statement or a conditional statement optionally preceded by an imperative statement. A further description of the rules governing *statement-1* and *statement-2* is given elsewhere. (See Section 2.4.21)
2. The ELSE NEXT SENTENCE phrase can be omitted if it immediately precedes the terminal period of the sentence.
3. If the END-IF phrase is specified, the NEXT SENTENCE phrase must not be specified.

General Rules

1. The scope of an IF statement can be terminated by any of the following:
   a. An END-IF phrase at the same level of nesting
   b. A separator period
   c. If nested, by an ELSE phrase associated with an IF statement at a higher level of nesting.
2. When an IF statement is executed, the following transfers of control occur:
   a. If the condition is true and *statement-1* is specified, control is transferred to the first statement of *statement-1* and execution continues according to the rules for each statement specified in *statement-1.* If a procedure branching or conditional statement is executed that causes an explicit transfer of control, control is explicitly transferred in accordance with the rules of that statement. Upon completion of the execution of *statement-1,* the ELSE phrase, if specified, is ignored and control passes to the end of the IF statement.
   b. If the condition is true and the NEXT SENTENCE phrase is specified instead of *statement-1,* the ELSE phrase, if specified, is ignored and control passes to the next executable sentence.
   c. If the condition is false and *statement-2* is specified, *statement-1* or its surrogate NEXT SENTENCE is ignored, control is transferred to the first statement of *statement-2* and execution continues according to the rules for each statement specified in *statement-2.*

If a procedure branching or conditional statement is executed that causes an explicit transfer of control, control is explicitly transferred in accordance with the rules of that statement. Upon completion of the execution of *statement-2,* control passes to the end of the IF statement.

d. If the condition is false and the ELSE phrase is not specified, *statement-1* is ignored and control passes to the end of the IF statement.

e. If the condition is false and the ELSE NEXT SENTENCE phrase is specified, *statement-1* is ignored and control passes to the next executable sentence.

3. *Statement-1, statement-2,* or both can contain an IF statement. In this case, the IF statement is said to be nested. More detailed rules on nesting are given in Section 2.4.21.

   IF combinations proceeding from left to right. Therefore, any ELSE or END-IF encountered is considered to apply to the immediately preceding IF that has not been already paired with an ELSE and END-IF.

Example

```
IF condition-1 statement-1
    IF condition-2
        IF condition-3 statement-2
        ELSE statement-3
    ELSE statement-4
        IF condition-4
            IF condition-5 statement-5
            ELSE statement-6
                IF condition-6 NEXT SENTENCE
                ELSE statement-7.
New-sentence.
```

The correspondence of IFs and ELSEs is shown by indentation, a frequently used means of making such complex sentences more readable.

This example cannot be executed, as shown in the flowchart in Figure 3–7, if it contains a GO TO statement.

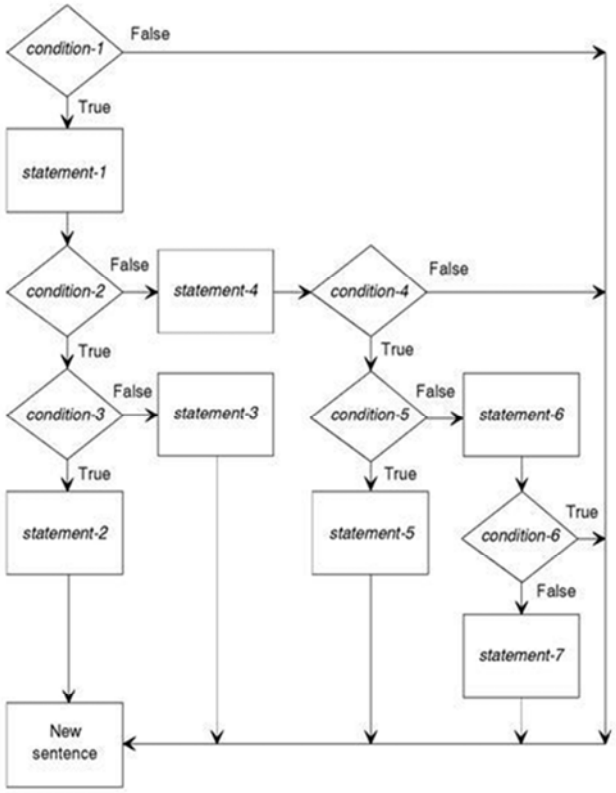


Figure 3–7. Execution Flowchart

3.6.17. The INITIALIZE Statement

Function

The INITIALIZE statement provides the ability to set selected types of data fields to predetermined values, for example, numeric data to zeros or alphanumeric data to spaces.

General Format

```
INITIALIZE { identifier-1 } ...
    [ REPLACING { { ALPHABETIC
                  ALPHANUMERIC
                  NUMERIC
                  ALPHANUMERIC-EDITED
                  NUMERIC-EDITED } DATA BY { identifier-2
                                             literal-1 } } ... ]
```

Syntax Rules

1. *Literal-1* and the data item referred to by *identifier-2* represent the sending area; the data item referred to by *identifier-1* represents the receiving area.
2. Each category stated in the REPLACING phrase must be a permissible category as a receiving operand in a MOVE statement where the corresponding data item referenced by *identifier-2* or *literal-1* is used as the sending operand (see Section 3.6.19).
3. The same category cannot be repeated in a REPLACING phrase.
4. The description of the data item referenced by *identifier-1* or any items subordinate to *identifier-1* cannot contain the DEPENDING phrase of the OCCURS clause.
5. An index data item cannot appear as an operand of an INITIALIZE statement.
6. The data description entry for the data item referenced by *identifier-1* must not contain a RENAMES clause.

General Rules

1. The keyword following the word REPLACING corresponds to a category of data as defined in Section 2.4.17, "Concepts of Classes of Data".
2. Whether *identifier-1* refers to an elementary item or a group item, all operations are performed as if a series of MOVE statements had been written, each of which has an elementary item as its receiving field, subject to the following rules. If the REPLACING phrase is specified:

   a. If *identifier-1* references a group item, any elementary item within the data item referenced by *identifier-1* is initialized only if it belongs to the category specified in the REPLACING phrase.

   b. If *identifier-1* references an elementary item, that item is initialized only if it belongs to the category specified in the REPLACING phrase.

   This initialization takes place as follows: The data item referenced by *identifier-2* or *literal-1* acts as the sending operand in an implicit MOVE statement to the identified item.

   All such elementary receiving fields, including all occurrences of table items within the group, are affected; the only exceptions are those fields specified in general rules 3 and 4.
3. Index data items and elementary FILLER data items are not affected by the execution of an INITIALIZE statement.
4. Any item that is subordinate to a receiving area identifier and that contains the REDEFINES clause or any item that is subordinate to such an item is excluded from this operation. However, a receiving area identifier can itself have a REDEFINES clause or be subordinate to a data item with a REDEFINES clause.

5. When the statement is written without the REPLACING phrase, data items of the categories alphabetic, alphanumeric, and alphanumeric edited are set to spaces; data items of the categories numeric and numeric edited are set to zeros. In this case, the operation is as if each affected data item is the receiving area in an elementary MOVE statement with the indicated source literal (that is spaces or zeros).
6. In all cases, the content of the data item referred to by *identifier-1* is set to the indicated value in the order (left to right) of the appearance of *identifier-1* in the INITIALIZE statement. Within this sequence, where *identifier-1* refers to a group item, affected elementary items are initialized in the sequence of their definition within the group.
7. If *identifier-1* occupies the same storage area as *identifier-2,* the result of the execution of this statement is undefined, even if they are defined by the same data description entry. (See Section 3.6.5)

3.6.18. The INSPECT Statement

Function

The INSPECT statement provides the ability to tally or replace occurrences of single characters or groups of characters in a data item.

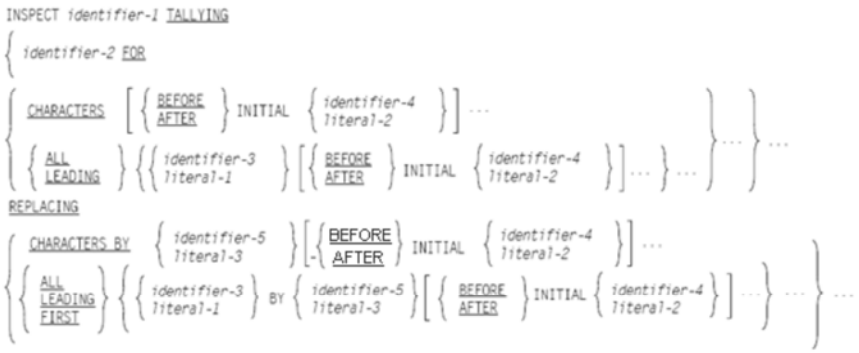
Format 1

```
INSPECT identifier-1 TALLYING
{ identifier-2 FOR
{ CHARACTERS [ { BEFORE } INITIAL { identifier-4 } ] ...                                    }
{              { AFTER  }         { literal-2   }                                          }
{ { ALL     } { { identifier-3 } [ { BEFORE } INITIAL { identifier-4 } ] ... } ...         } ...
{ { LEADING } { { literal-1    } [ { AFTER  }         { literal-2   } ] ... } ...         }
```

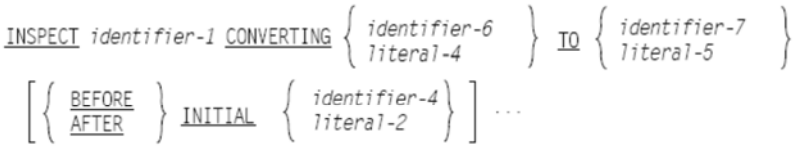
Format 2

```
INSPECT identifier-1 REPLACING
{ CHARACTERS BY { identifier-5 } [ { BEFORE } INITIAL { identifier-4 } ] ...                                                    }
{              { literal-3    } [ { AFTER  }         { literal-2   } ] ...                                                    }
{ { ALL     } { { identifier-3 } BY { identifier-5 } [ { BEFORE } INITIAL { identifier-4 } ] ... } ...                         } ...
{ { LEADING } { { literal-1    }    { literal-3   } [ { AFTER  }         { literal-2   } ] ... } ...                         }
{ { FIRST   }                                                                                                                  }
```

Format 3



Format 4



Syntax Rules

The following is a list of syntax rules.

All formats

The following syntax rules apply to all formats:

1. *Identifier-1* must refer to either a group item or any category of elementary item described, implicitly or explicitly, as USAGE IS DISPLAY or DISP-I18N.
2. *Identifier-3 . . . identifier-n* must refer to an elementary item described, implicitly or explicitly, as USAGE IS DISPLAY or DISP-I18N.
3. Each literal must be a nonnumeric literal and must not be a figurative constant that begins with the word ALL. If *literal-1, literal-2,* or *literal-4* is a figurative constant, it refers to an implicit 1-character data item.
4. No more than one BEFORE phrase and one AFTER phrase can be specified for any one ALL, LEADING, CHARACTERS, FIRST, or CONVERTING phrase.
5. In level 1, *literal-1, literal-2, literal-3,* and the data items referenced by *identifier-3, identifier-4,* and *identifier-5* must be one character in length. Except as specifically noted in syntax and general rules, this restriction on length does not apply to level 2.

Formats 1 and 3

The following syntax rules apply to formats 1 and 3:

6. *Identifier-2* must refer to an elementary numeric data item.

Formats 2 and 3

The following syntax rules apply to formats 2 and 3:

7. The size of *literal-3* or the data item referred to by *identifier-5* must be equal to the size of *literal-1* or the data item referred to by *identifier-3.* When a figurative constant is used as *literal-3,* the size of the figurative constant is equal to the size of *literal-1* or the size of the data item referred to by *identifier-3.*
8. When the CHARACTERS phrase is used, *literal-2, literal-3,* or the size of the data item referred to by *identifier-4, identifier-5* must be one character in length.

Format 4

The following syntax rules apply to format 4:

9. The size of *literal-5* or the data item referred to by *identifier-7* must be equal to the size of *literal-4* or the data item referred to by *identifier-6.* When a figurative constant is used as *literal-5,* the size of the figurative constant is equal to the size of *literal-4* or the size of the data item referred to by *identifier-6.*
10. The same character must not appear more than once either in *literal-4* or in the data item referred to by *identifier-6.*

General Rules

The following general rules apply to all formats:

1. Inspection (which includes the comparison cycle, the establishment of boundaries for the BEFORE or AFTER phrase, and the mechanism for tallying and replacing) begins at the leftmost character position of the data item referenced by *identifier-1,* regardless of its class, and proceeds from left to right to the rightmost character position as described in general rules 5 through 7.
2. For use in the INSPECT statement, the contents of the data item referred to by *identifier-1, identifier-3, identifier-4, identifier-5, identifier-6,* or *identifier-7* will be treated as follows:
    a. If any of *identifier-1, identifier-3, identifier-4, identifier-5, identifier-6,* or *identifier-7* refer to an alphabetic or alphanumeric data item, the INSPECT statement treats the contents of each such identifier as a character string.
    b. If any of *identifier-1, identifier-3, identifier-4, identifier-5, identifier-6,* or *identifier-7* refer to alphanumeric edited, numeric-edited, or unsigned numeric data items, the data item is inspected as though it had been redefined as alphanumeric (see general rule 2a) and the INSPECT statement had been written to refer to the redefined data item.
    c. If any of *identifier-1, identifier-3, identifier-4, identifier-5, identifier-6, or identifier-7* refers to a signed numeric data item, the data item is inspected as though it had been moved to an unsigned numeric data item of the same length and then the rules in general rule 2b had been applied. (See 3.6.19.) If *identifier-1* is a signed numeric item, the original value of the sign is retained upon completion of the INSPECT statement.
3. In general rules 5 through 17, all references to *literal-1, literal-2, literal-3, literal-4,* or *literal-5* apply equally to the contents of the data item referred to by *identifier-3, identifier-4, identifier-5, identifier-6,* or *identifier-7,* respectively.

4. If any identifier is subscripted, or is a function identifier, the subscript or function identifier is evaluated only once as the first operation in the execution of the INSPECT statement.

Formats 1 and 2

The following general rules apply to formats 1 and 2:

1. During inspection of the content of the data item referred to by *identifier-1,* each properly matched occurrence of *literal-1* is tallied (format 1) or replaced by *literal-3* (format 2).
2. The comparison operation to determine the occurrences of *literal-1* to be tallied or to be replaced, occurs as follows:
   a. The operands of the TALLYING or REPLACING phrase are considered in the order they are specified in the INSPECT statement from left to right. The first *literal-1* is compared to an equal number of contiguous characters, starting with the leftmost character position in the data item referred to by *identifier-1. Literal-1* matches that portion of the content of the data item referred to by *identifier-1* if they are equal, character for character, and if
      - Neither LEADING nor FIRST is specified
      - The LEADING adjective applies to *literal-1* and *literal-1* is a leading occurrence as defined in general rules 10 and 13
      - The FIRST adjective applies to *literal-1* and *literal-1* is the first occurrence as defined in general rules 10 and 13

   b. If no match occurs in the comparison of the first *literal-1,* the comparison is repeated with each successive *literal-1,* if any, until either a match is found or there is no next successive *literal-1.* When there is no next successive *literal-1,* the character position in the data item referred to by *identifier-1* immediately to the right of the leftmost character position considered in the last comparison cycle is considered as the leftmost character position, and the comparison cycle begins again with the first *literal-1.*

   c. Whenever a match occurs, tallying or replacing takes place as described in general rules 10 and 13. The character position in the data item referred to by *identifier-1* immediately to the right of the rightmost character position that participated in the match is now considered to be the leftmost character position of the data item referred to by *identifier-1,* and the comparison cycle starts again with the first *literal-1.*

   d. The comparison operation continues until the rightmost character position of the data item referred to by *identifier-1* has participated in a match or has been considered as the leftmost character position. When this occurs, inspection is terminated.

   e. If the CHARACTERS phrase is specified, an implied 1-character operand participates in the cycle described in paragraphs 6a through 6d as if it had been specified by *literal-1,* except that no comparison to the content of the data item referred to by *identifier-1* takes place. This implied character is considered always to match the leftmost character of the content of the data item referred to by *identifier-1* participating in the current comparison cycle.

3. The comparison operation defined in general rule 6 is restricted by the BEFORE and AFTER phrases as follows:

   a. If neither the BEFORE nor the AFTER phrase is specified, *literal-1* or the implied operand of the CHARACTERS phrase participates in the comparison operation as described in general rule 6. *Literal-1* or the implied operand of the CHARACTERS phrase is first eligible to participate in matching at the leftmost character position of *identifier-1.*

   b. If the BEFORE phrase is specified, the associated *literal-1* or the implied operand of the CHARACTERS phrase participates only in those comparison cycles that involve that portion of the content of the data item referred to by *identifier-1* from its leftmost character position up to, but not including, the first occurrence of *literal-2* within the content of the data item referred to by *identifier-1.* The position of this first occurrence is determined before the first cycle of the comparison operation described in general rule 6 is begun. If, on any comparison cycle, *literal-1* or the implied operand of the CHARACTERS phrase is not eligible to participate, it is considered not to match the content of the data item referred to by *identifier-1.* If there is no occurrence of *literal-2* within the content of the data item referred to by *identifier-1,* its associated *literal-1* or the implied operand of the CHARACTERS phrase participates in the comparison operation as though the BEFORE phrase had not been specified.

   c. If the AFTER phrase is specified, the associated *literal-1* or the implied operand of the CHARACTERS phrase can participate only in those comparison cycles that involve that portion of the content of the data item referred to by *identifier-1* from the character position immediately to the right of the rightmost character position of the first occurrence of *literal-2* within the content of the data item referred to by *identifier-1* to the rightmost character position of the data item referred to by *identifier-1. Literal-1* or the implied operand of the CHARACTERS phrase is first eligible to participate in matching at the leftmost character position of *identifier-1.* The position of this first occurrence is determined before the first cycle of the comparison operation described in general rule 6 is begun. If, on any comparison cycle, *literal-1* or the implied operand of the CHARACTERS phrase is not eligible to participate, it is considered not to match the content of the data item referred to by *identifier-1.* If there is no occurrence of *literal-2* within the content of the data item referred to by *identifier-1,* its associated *literal-1* or the implied operand of the CHARACTERS phrase is never eligible to participate in the comparison operation.

Format 1

The following general rules apply to format 1:

1. The required words ALL and LEADING are adjectives that apply to each succeeding *literal-1* until the next adjective appears.
2. The content of the data item referred to by *identifier-2* is not initialized by the execution of the INSPECT statement.

3. The rules for tallying are as follows:

   a. If the ALL phrase is specified, the content of the data item referred to by *identifier-2* is increased by one for each occurrence of *literal-1* matched within the content of the data item referred to by *identifier-1.*

   b. If the LEADING phrase is specified, the content of the data item referred to by *identifier-2* is increased by the first and each subsequent contiguous occurrence of *literal-1* matched within the content of the data item referred to by *identifier-1,* provided that the leftmost such occurrence is at the point where comparison began in the first comparison cycle in which *literal-1* was eligible to participate.

   c. If the CHARACTERS phrase is specified, the content of the data item referred to by *identifier-2* is increased by one for each character matched, in the sense of general rule 6e, within the content of the data item referred to by *identifier-1.*

4. If *identifier-1, identifier-3,* or *identifier-4* occupy the same storage area as *identifier-2,* the result of the execution of this statement is undefined, even if they are defined by the same data description entry. (See Section 3.6.5)

Format 2

The following general rules apply to format 2:

1. The required words ALL, LEADING, and FIRST are adjectives that apply to each succeeding BY phrase until the next adjective appears.
2. The rules for replacement are as follows:

   a. When the CHARACTERS phrase is specified, each character matched (in the sense of general rule 6e) in the content of the data item referred to by *identifier-1* is replaced by *literal-3.*

   b. When the adjective ALL is specified, each occurrence of *literal-1* matched in the content of the data item referred to by *identifier-1* is replaced by *literal-3.*

   c. When the adjective LEADING is specified, the first and each successive contiguous occurrence of *literal-1* matched in the content of the data item referred to by *identifier-1* is replaced by *literal-3,* provided that the leftmost occurrence is at the point where comparison began in the first comparison cycle in which *literal-1* was eligible to participate.

   d. When the adjective FIRST is specified, the leftmost occurrence of *literal-1* matched within the content of the data item referred to by *identifier-1* is replaced by *literal-3.* This rule applies to each successive specification of the FIRST phrase regardless of the value of *literal-1.*

3. If *identifier-3, identifier-4,* or *identifier-5* occupy the same storage area as *identifier-1,* the result of the execution of this statement is undefined, even if they are defined by the same data description entry. (See Section 3.6.5.)

Format 3

The following general rules apply to format 3:

A format 3 INSPECT statement is interpreted and executed as though two successive INSPECT statements specifying the same *identifier-1* had been written with one statement being a format 1 statement with TALLYING phrases identical to those specified in the format 3 statement, and the other statement being a format 2 statement with REPLACING phrases identical to those specified in the format 3 statement. The general rules given for matching and counting apply to the format 1 statement and the general rules given for matching and replacing apply to the format 2 statement. Subscripting associated with any identifier in the format 2 statement is evaluated only once before executing the format 1 statement.

Format 4

The following general rules apply to format 4:

1. A format 4 INSPECT statement is interpreted and executed as though a format 2 INSPECT statement specifying the same *identifier-1* had been written with a series of ALL phrases, one for each character of *literal-4.* The effect is as if each of these ALL phrases referenced, as *literal-1,* a single character of *literal-4* and referenced, as *literal-3,* the corresponding single character of *literal-5.* Correspondence between the characters of *literal-4* and the characters of *literal-5* is by ordinal position within the data item.
2. If *identifier-4, identifier-6,* or *identifier-7* occupy the same storage area as *identifier-1,* the result of the execution of this statement is undefined, even if they are defined by the same data description entry. (See Section 3.6.5)

Examples

In each of the following examples of the INSPECT statement, COUNT-n is assumed to be zero immediately before execution of the statement. The results shown for each example, except the last, are the result of executing the two successive INSPECT statements shown above them.

Example

```
INSPECT ITEM TALLYING

    COUNT-0 FOR ALL "AB", ALL "D"
    COUNT-1 FOR ALL "BC"
    COUNT-2 FOR LEADING "EF"
    COUNT-3 FOR LEADING "B"
    COUNT-4 FOR CHARACTERS;
INSPECT ITEM REPLACING

    ALL "AB" BY "XY", "D" BY "X"
    ALL "BC" BY "VW"
    LEADING "EF" BY "TU"
    LEADING "B" BY "S"
    FIRST "G" BY "R"
    FIRST "G" BY "P"
    CHARACTERS BY "Z"
```

Initial Value of ITEM	Count-0	Count-1	Count-2	Count-3
EFABDBCGABCFGG	3	1	1	0
BABABC	2	0	0	1
BBBC	0	1	0	2

Example

```
INSPECT ITEM TALLYING

    COUNT-0 FOR CHARACTERS
    COUNT-1 FOR ALL "A";

INSPECT ITEM REPLACING

    CHARACTERS BY "Z"
    ALL "Z" BY "X"
```

Initial Value of ITEM	COUNT-0	COUNT-1	Final Value of ITEM
BBB	3	0	ZZZ
AAA	3	0	ZZZ

Example

```
INSPECT ITEM TALLYING

   COUNT-0 FOR ALL "AB" BEFORE "BC"
   COUNT-1 FOR LEADING "B" AFTER "D"
   COUNT-2 FOR CHARACTERS AFTER "A" BEFORE "C";

INSPECT ITEM REPLACING
   ALL "AB" BY "XY" BEFORE "BC"
   LEADING "B" BY "W" AFTER "D"
   FIRST "E" BY "V" AFTER "D"
   CHARACTERS BY "Z" AFTER "A" BEFORE "C"
```

Initial Value of ITEM	COUNT-0	COUNT-1	COUNT-2	Final Value of ITEM
BBEABDABABBCABEE	3	0	2	BBEXYZXYXYZCABVE
ADDDDC	0	0	4	AZZZZC
ADDDDA	0	0	5	AZZZZZ
CDDDDC	0	0	0	CDDDDC
BDBBBDB	0	3	0	BDWWWDB

Example

```
INSPECT ITEM TALLYING

    COUNT-0 FOR ALL "AB" AFTER "BA" BEFORE "BC"

INSPECT ITEM REPLACING
    ALL "AB" BY "XY" AFTER "BA" BEFORE "BC"
```

Initial Value of ITEM	COUNT-0	Final Value of ITEM
ABABABABC	1	ABABXYABC

Example

```
INSPECT ITEM CONVERTING

    "ABCD" TO "XYZX" AFTER QUOTE BEFORE "@".
```

The above INSPECT is equivalent to the following INSPECT:

```
INSPECT item REPLACING

    ALL "A" BY "X" AFTER QUOTE BEFORE "@"
    ALL "B" BY "Y" AFTER QUOTE BEFORE "@"
    ALL "C" BY "Z" AFTER QUOTE BEFORE "@"
    ALL "D" BY "X" AFTER QUOTE BEFORE "@".
```

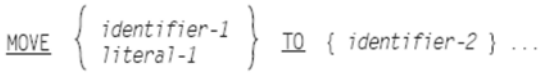
Initial Value of ITEM	Final Value of ITEM
AC"AEBDFBCD@AB"D	AC"XEYXFYZX@AB"D

3.6.19. The MOVE Statement

Function

The MOVE statement transfers data, in accordance with the rules of editing, to one or more data areas.

Format 1



Format 2



Syntax Rules

1. *Literal-1* or the data item referred to by *identifier-1* represents the sending area. The data item referred to by *identifier-2* represents the receiving area.
2. CORR is an abbreviation for CORRESPONDING.
3. When the CORRESPONDING phrase is used, all identifiers must be group items.
4. An index or a USAGE POINTER data item must not appear as an operand of a MOVE statement. (See 3.5.16.)

General Rules

1. If the CORRESPONDING phrase is used, selected items within *identifier-1* are moved to selected items within *identifier-2.* The results are the same as if the user had referred to each pair of identifiers in separate MOVE statements.

2. *Literal-1* or the content of the data item referred to by *identifier-1* is moved to the data item referred to by each *identifier-2* in the order in which it is specified. The rules referring to *identifier-2* also apply to the other receiving areas. Any length evaluation or subscripting associated with *identifier-2* is evaluated immediately before the data is moved to the respective data item. If *identifier-1* is reference modified, subscripted, or is a function identifier, the reference modifier, subscript, or function identifier is evaluated only once. This is done immediately before data is moved to the first of the receiving operands. The length of the data item referred to by *identifier-1* is evaluated only once, immediately before the data is moved to the first of the receiving operands.

   The evaluation of the length of *identifier-1* or *identifier-2* can be affected by the DEPENDING ON phrase of the OCCURS clause (see Section 3.5.10).

   The result of the statement:

```
MOVE A (B) TO B, C (B)
```

   is equivalent to:

```
MOVE A (B) TO temp
MOVE temp TO B
MOVE temp TO C (B)
```

   where temp is an intermediate result item provided by the compiler.

3. Any move in which the receiving operand is an elementary item and the sending operand is either a literal or an elementary item is an elementary move. Every elementary item belongs to one of the following categories: numeric, alphabetic, alphanumeric, numeric edited or alphanumeric edited. (See Section 3.5.11) Numeric literals belong to the category numeric, and nonnumeric literals belong to the category alphanumeric. The figurative constant ZERO (ZEROS, ZEROES), when moved to a numeric or numeric-edited item, belongs to the category numeric. In all other cases, it belongs to the category alphanumeric. The figurative constant SPACE (SPACES) belongs to the category alphabetic. All other figurative constants belong to the category alphanumeric.

   The following rules apply to an elementary move between these categories:

   a. The figurative constant SPACE, an alphanumeric-edited data item, or an alphabetic data item must not be moved to a numeric or numeric-edited data item.

   b. A numeric literal, the figurative constant ZERO, a numeric data item, or a numeric edited data item must not be moved to an alphabetic data item.

   c. A noninteger numeric literal or a noninteger numeric data item must not be moved to an alphanumeric or alphanumeric-edited data item.

   d. In level 1, a numeric-edited data item must not be moved to a numeric or numeric edited data item.

e. All other elementary moves are legal and are performed according to the rules given in general rule 4.

4. Any necessary conversion of data from one form of internal representation to another takes place during legal elementary moves, along with any editing specified for, or de-editing implied by, the receiving data item:

   a. When an alphanumeric-edited or alphanumeric item is a receiving item, alignment and any necessary space-filling takes place as defined in Section 2.4.19, "Standard Alignment Rules". If the sending operand is described as being signed numeric, the operational sign is not moved; if the operational sign occupies a separate character position, which character is not moved and the size of the sending operand is considered to be one less than its actual size in terms of standard data format characters. (See Section 3.5.14.) If the sending operand is numeric edited, no de-editing takes place. If the usage of the sending operand is different from that of the receiving operand, conversion of the sending operand to the internal representation of the receiving operand takes place. If the sending operand is numeric and contains the PICTURE symbol 'P', all digit positions specified by this symbol are considered to have the value zero and are counted in the size of the sending operand.

   b. When a numeric or numeric-edited item is the receiving item, alignment by decimal point and any necessary zero-filling takes place as defined in Section 2.4.19, "Standard Alignment Rules" except where zeros are replaced because of editing requirements. When the sending operand is numeric edited, the editing is implied to establish the operand's unedited numeric value, which can be signed; then the unedited numeric value is moved to the receiving field.

      - When a signed numeric item is the receiving item, the sign of the sending operand is placed in the receiving item. (See Section 3.5.14.) Conversion of the representation of the sign takes place as necessary. If the sending item is unsigned, a positive sign is generated for the receiving item.
      - When an unsigned numeric item is the receiving item, the absolute value of the sending operand is moved and no operational sign is generated for the receiving item.
      - When the sending operand is described as being alphanumeric, data is moved as if the sending operand were described as an unsigned numeric integer.
      - If a numeric-edited sender contains data that is not consistent with its PICTURE clause, the results of de-editing are undefined.

   c. When a receiving field is described as alphabetic, justification and any necessary space-filling takes place as defined in Section 2.4.19, "Standard Alignment Rules".

5. Any move that is not an elementary move is treated exactly as if it were an alphanumeric-to-alphanumeric elementary move, except that there is no conversion of data from one form of internal representation to another. In such a move, the receiving area will be filled without consideration for the individual elementary or group items contained within either the sending or receiving area, except as noted in general rule 3 of the OCCURS clause (see Section 3.5.10).

6. Data in the following table summarizes the legality of the various types of MOVE statements. The general rule reference indicates the rule that prohibits the move or that describes the behavior of a legal move.

Category of Sending Operand	**Category of Receiving Data Item**		
	Alphabetic	**Alphanumeric Edited Alphanumeric**	**Numeric Integer Numeric Noninteger Numeric Edited**
Alphabetic	Yes/4c	Yes/4a	No/3a
Alphanumeric	Yes/4c	Yes/4a	Yes/4b
Alphanumeric Edited	Yes/4c	Yes/4a	No/3a
Numeric Integer	No/3b	Yes/4a	Yes/4b
Numeric Noninteger	No/3b	No/3c	Yes/4b
Numeric Edited	No/3b	Yes/4a	Yes/4b

The following table shows the results of elementary moves.

Source Area		**Receiving Area**	
Picture	**Value**	**Picture**	**Value after Moving**
9V9	12	99V99	0120
9V999	8765	V99	76
9999V99	567891	9999V99	567891
9999V99	567891	9999V9	56789
9V9	78	999V99	00780
XXX	M8N	XXXXX	M8NΔΔ
99V99	6789	999.99	067.89
AAAAAA	WARREN	AAA	WAR
99V99	6789	$$$99.99	ΔΔ$67.89

7. If the last byte is partially filled, as in the case of exact binary items, the residual bits are undefined.
8. The size of a group item includes slack bits or bytes, if any. Therefore, if a group contains mixed usages, synchronized, justified, exact binary, floating-point (COMPUTATIONAL-1 or COMPUTATIONAL-2), or dimensioned items, it is advisable to construct moves that handles the slack areas properly.

3.6.20. The MULTIPLY Statement

Function

The MULTIPLY statement causes numeric data items to be multiplied and sets the values of data items equal to the results.

Format 1

```
MULTIPLY { identifier-1 } BY { identifier-2 [ ROUNDED ] } ...
         { literal-1    }
  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-MULTIPLY ]
```

Format 2

```
MULTIPLY { identifier-1 } BY { identifier-2 } GIVING { identifier-3 [ ROUNDED ] } ...
         { literal-1    }    { literal-2    }
         [ ON SIZE ERROR imperative-statement-1 ]
         [ NOT ON SIZE ERROR imperative-statement-2 ]
         [ END-MULTIPLY ]
```

Syntax Rules

1. Each identifier must refer to a numeric elementary item, except that in format 2 each identifier following the word GIVING must refer to either an elementary numeric item or an elementary numeric-edited item.
2. Each literal must be a numeric literal.
3. The composite of operands must not contain more than 18 digits. The composite of operands is the hypothetical data item that results from superimposing all receiving data items of a given statement aligned on their decimal points.

General Rules

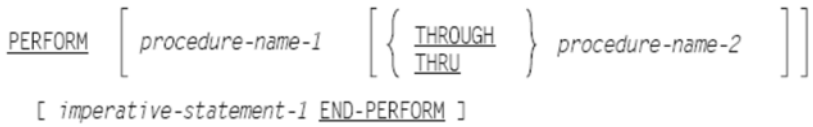
1. When format 1 is used, *literal-1* or the value of the data item referred to by *identifier-1* is stored in a temporary data item. The value in this temporary data item is then multiplied by the value of the data item referred to by *identifier-2.* The value of the multiplier (the value of the data item referred to by *identifier-2*) is replaced by this product; similarly, the temporary data item is multiplied by each successive occurrence of *identifier-2* in the left-to-right order in which *identifier-2* is specified.
2. When format 2 is used, *literal-1* or the value of the data item referred to by *identifier-1* is multiplied by *literal-2* or the value of the data item referred to by *identifier-2* and the result is stored in the data items referred to by each *identifier-3*
3. Additional rules and explanations relative to this statement are given under the appropriate paragraphs. (See Section 2.4.21 and Section 3.6.5).

3.6.21. The PERFORM Statement

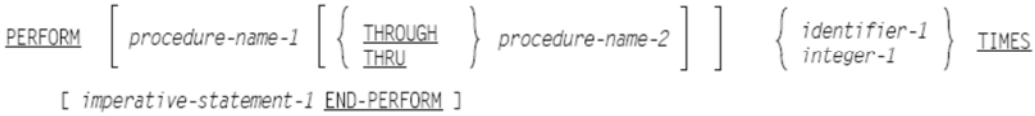
Function

The PERFORM statement is used to transfer control explicitly to one or more procedures and to return control implicitly when execution of the specified procedure is complete. The PERFORM statement is also used to control execution of one or more imperative statements that are within the scope of that PERFORM statement.

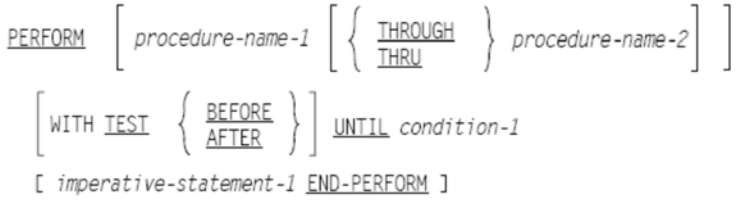
Format 1



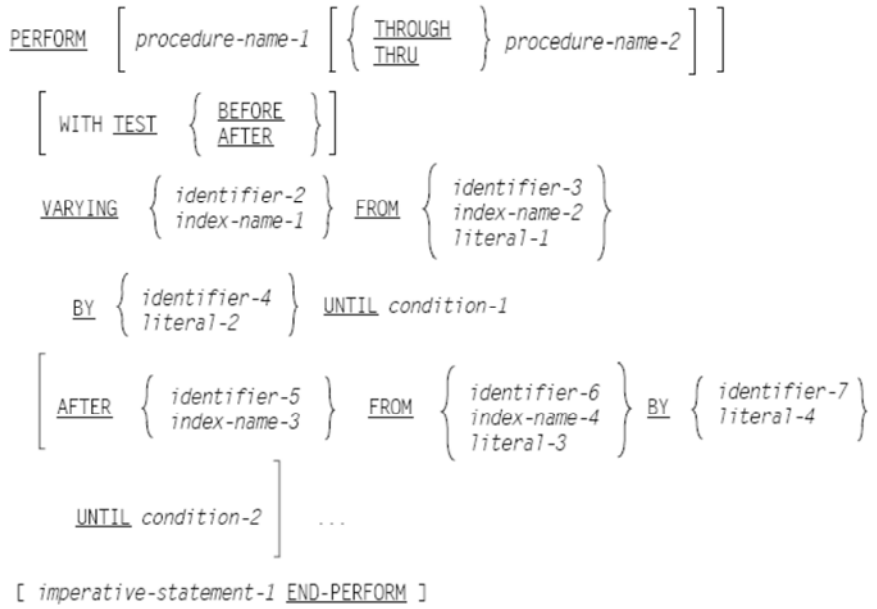
Format 2



Format 3



Format 4



Syntax Rules

1. If *procedure-name-1* is omitted, *imperative-statement-1* and the END-PERFORM phrase must be specified; if *procedure-name-1* is specified, *imperative-statement-1* and the END-PERFORM phrase must not be specified.
2. If neither the TEST BEFORE nor the TEST AFTER phrase is specified, the TEST BEFORE phrase is assumed.
3. Each identifier represents a numeric elementary item described in the Data Division. In format 2, *identifier-1* must be described as a numeric integer.
4. Each literal represents a numeric literal.
5. The words THROUGH and THRU are equivalent.
6. If an index name is specified in the VARYING or AFTER phrase, then:
   a. The identifier in the associated FROM and BY phrases must refer to an integer data item.
   b. The literal in the associated FROM phrase must be a positive integer.
   c. The literal in the associated BY phrase must be a nonzero integer.
7. If an index name is specified in the FROM phrase, then:
   a. The identifier in the associated VARYING or AFTER phrase must refer to an integer data item.
   b. The identifier in the associated BY phrase must refer to an integer data item.
   c. The literal in the associated BY phrase must be an integer.
8. *Literal* in the BY phrase must not be zero.

9. *Condition-1, condition-2, ...* can be any conditional expression as described in Section a.
10. Wh ere *procedure-name-1* and *procedure-name-2* are both specified and either is the name of a procedure in the declaratives portion of Procedure Division, both must be procedure names in the same declarative section.
11. At least six AFTER phrases are permitted in format 4 of the PERFORM statement.

General Rules

1. The data items referred to by *identifier-4* and *identifier-7* must not have zero values.
2. If an index name is specified in the VARYING or AFTER phrase, and an identifier is specified in the associated FROM phrase, the data item referred to by the identifier must have a positive value.
3. When *procedure-name-1* is specified, the PERFORM statement is referred to as an out-of-line PERFORM statement; when *procedure-name-1* is omitted, the PERFORM statement is referred to as an inline PERFORM statement.
4. The statements contained within the range of *procedure-name-1* (through *procedure-name-2* if specified) for an out-of-line PERFORM statement or contained within the PERFORM statement itself for an inline PERFORM statement are referred to as the specified set of statements.
5. The END-PERFORM phrase delimits the scope of the inline PERFORM statement. (See Section 2.4.21)
6. An inline PERFORM statement functions according to the following general rules for an otherwise identical out-of-line PERFORM statement, with the exception that the statements contained within the inline PERFORM statement are executed in place of the statements contained within the range of *procedure-name-1* (through *procedure-name-2* if specified). Unless specially qualified by the words inline or out-of-line, all the general rules that apply to the out-of-line PERFORM statement also apply to the inline PERFORM statement.
7. When the PERFORM statement is executed, control is transferred to the first statement of the specified set of statements (except as indicated in general rules 10b, 10c, and 10d). This transfer of control occurs only once for each execution of a PERFORM statement. For those cases where a transfer of control to the specified set of statements does take place, an implicit transfer of control to the end of the PERFORM statement is established as follows:
    a. If *procedure-name-1* is a paragraph name and *procedure-name-2* is not specified, the return is after the last statement of *procedure-name-1*.
    b. If *procedure-name-1* is a section name and *procedure-name-2* is not specified, the return is after the last statement of the last paragraph in *procedure-name-1*.
    c. If *procedure-name-2* is specified and it is a paragraph name, the return is after the last statement of the paragraph.
    d. If *procedure-name-2* is specified and it is a section name, the return is after the last statement of the last paragraph in the section.

e. If an inline PERFORM statement is specified, an execution of the PERFORM statement is completed after the last statement contained within it has been executed.

8. There is no necessary relationship between *procedure-name-1* and *procedure-name 2* except that a consecutive sequence of operations is to be executed beginning at the procedure named *procedure-name-1* and ending with the execution of the procedure named *procedure-name-2.* In particular, GO TO and PERFORM statements can occur between *procedure-name-1* and the end of *procedure-name-2.* If there are two or more logical paths to the return point, then *procedure-name-2* can be the name of a paragraph consisting of the EXIT statement, to which all of these paths must lead.

9. If control passes to the specified set of statements by means other than a PERFORM statement, control passes through the last statement of the set to the next executable statement as if no PERFORM statement referenced the set.

10. The PERFORM statements operate as follows:

    a. Format 1 is the basic PERFORM statement. The specified set of statements referred to by this type of PERFORM statement is executed once and then control passes to the end of the PERFORM statement.

    b. Format 2 is the PERFORM . . . TIMES. The specified set of statements is performed the number of times specified by *integer-1* or by the initial value of the data item referred to by *identifier-1* for that execution. If, at the time of execution of a PERFORM statement, the value of the data item referred to by *identifier-1* is equal to zero or is negative, control passes to the end of the PERFORM statement. Following the execution of the specified set of statements the specified number of times, control is transferred to the end of the PERFORM statement.

       During execution of the PERFORM statement, references to *identifier-1* cannot alter the number of times the specified set of statements is to be executed from that which was indicated by the initial value of the data item referenced by *identifier-1.*

    c. Format 3 is the PERFORM . . . UNTIL. The specified set of statements is performed until the condition specified by the UNTIL phrase is true. When the condition is true, control is transferred to the end of the PERFORM statement. If the condition is true when the PERFORM statement is entered and the TEST BEFORE phrase is specified or implied, no transfer to *procedure-name-1* takes place, and control is passed to the end of the PERFORM statement. If the TEST AFTER phrase is specified, the PERFORM statement functions as if the TEST BEFORE phrase were specified except that the condition is tested after the specified set of statements has been executed. Any subscripting or reference modification associated with the operands specified in condition-1 is evaluated each time the condition is tested.

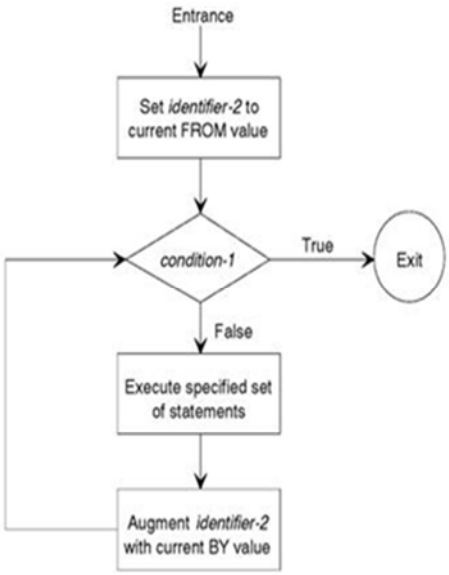
d. Format 4 is the PERFORM . . . VARYING. This variation of the PERFORM statement is used to augment the values referred to by one or more identifiers or index names in an orderly fashion during the execution of a PERFORM statement. In the following discussion, every reference to an identifier as the object of the VARYING, AFTER, and FROM (*current value*) phrases also refers to index names. If *index-name-1* or *index-name-3* is specified, value of the associated index at the beginning of the PERFORM statement must be set to an occurrence number of an element in the table. If *index-name-2* or *index-name-4* is specified, the value of the data item referred to by *identifier-2* or *identifier-5* at the beginning of the PERFORM statement must be equal to an occurrence number of an element in a table associated with *index-name-2* or *index-name-4.* Subsequent augmentation, as described below, of *index-name-1* or *index-name-3* must not result in the associated index being set to a value outside the range of the table associated with *index-name-1* or *index-name-3,* except that, at the completion of the PERFORM statement, the index associated with *index-name-1* can contain a value that is outside the range of the associated table by one increment or decrement value. If *identifier-2* or *identifier-5* is subscripted, the subscripts are evaluated each time the content of the data item referred to by the identifier is set or augmented. If *identifier-3, identifier-4, identifier-6,* or *identifier-7* are subscripted, the subscripts are evaluated each time the content of the data item referred to by the identifier is used in a setting augmenting operation. Any subscripting or reference modification associated with the operands specified in *condition-1* or *condition-2* is evaluated each time the condition is tested.

Representations of the actions of several types of format 4 PERFORM statements are given in the four flowcharts on the following pages.

1. If the TEST BEFORE phrase is specified or implied:

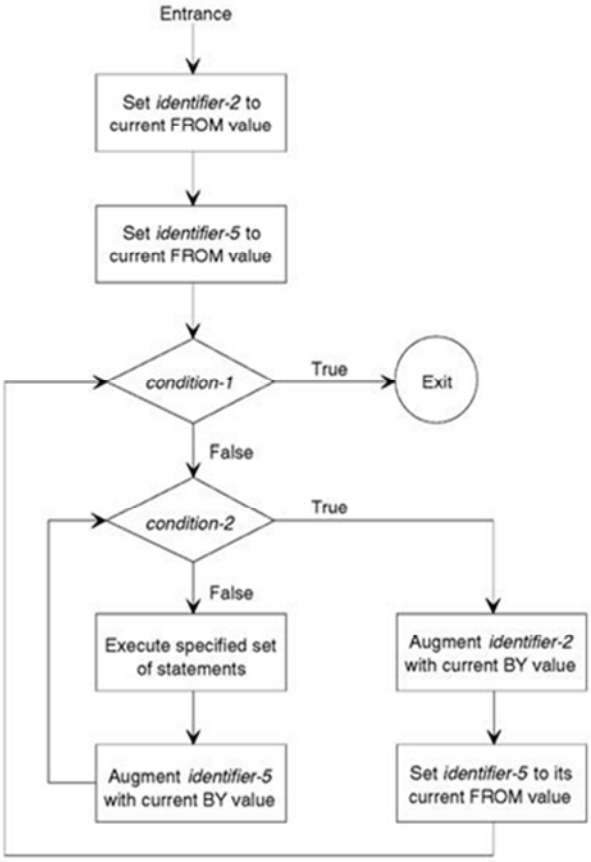
   When the data item referred to by one identifier is varied, the content of the data item referred to by *identifier-2* is set to *literal-1* or the current value of the data item referred to by *identifier-3* at the point of initial execution of the PERFORM statement; then, if the condition of the UNTIL phrase is false, the specified set of statements is executed once. The value of the data item referred to by *identifier-2* is augmented by the specified increment or decrement value (*literal-2* or the value of the data item referred to by *identifier-4*) and *condition-1* is evaluated again. The cycle continues until this condition is true, at which point control is transferred to the end of the PERFORM statement. If *condition-1* is true at the beginning of execution of the PERFORM statement, control is transferred to the end of the PERFORM statement.

The following flowchart is for the VARYING option of a PERFORM statement with the TEST BEFORE phrase having one condition.



When the data items referred to by two identifiers are varied, the content of the data item referred to by *identifier-2* is set to *literal-1* or the current value of the data item referred to by *identifier-3* and then the content of the data item referred to by *identifier-5* is set to *literal-3* or the current value of the data item referred to by *identifier-6.* After the contents of the data items referred to by identifiers have been set, *condition-1* is evaluated; if true, control is transferred to the end of the PERFORM statement; if false, *condition-2* is evaluated. If *condition-2* is false, the specified set of statements is executed once, then the content of the data item referred to by *identifier-5* is augmented by *literal-4* or the content of the data item referred to by *identifier-7* and *condition-2* is evaluated again. This cycle of evaluation and augmentation continues until this condition is true. When *condition-2* is true, the content of the data item referred to by *identifier-2* is augmented by *literal-2* or the content of the data item referred to by *identifier-4,* the content of the data item referred to by *identifier-5* is set to *literal-3* or the current value of the data item referred to by *identifier-6,* and *condition-1* is reevaluated. The PERFORM statement is completed if *condition-1* is true; if it is not true, the cycle continues until *condition-1* is true.

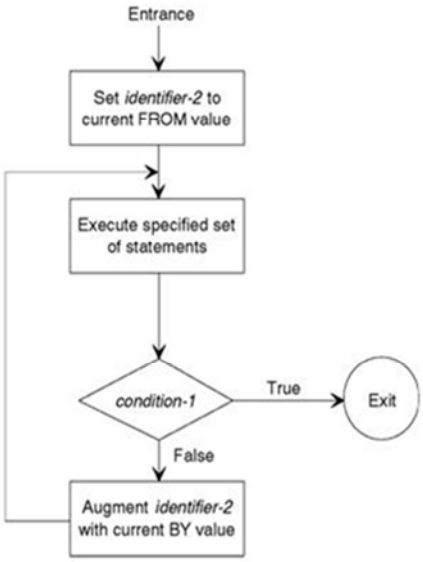
The following flowchart is for the VARYING option of a PERFORM statement with the TEST BEFORE phrase having two conditions.



At the termination of the PERFORM statement, the data item referred to by *identifier-5* contains *literal-3* or the current value of the data item referred to by *identifier-6*. The data item referred to by *identifier-2* contains a value that exceeds the last used setting by one increment or decrement value, unless *condition-1* was true when the PERFORM statement was entered, in which case the data item referred to by *identifier-2* contains *literal-1* or the current value of the data item referred to by *identifier-3*.

2. If the TEST AFTER phrase is specified: When the data item referred to by one identifier is varied, the content of the data item referred to by *identifier-2* is set to *literal-1* or the current value of the data item referred to by *identifier-3* at the point of execution of the PERFORM statement; then the specified set of statements is executed once and *condition-1* of the UNTIL phrase is tested. If the condition is false, the value of the data item referred to by *identifier-2* is augmented by the specified increment or decrement value (*literal-2* or the value of the data item referred to by *identifier-4*) and the specified set of statements is executed again. The cycle continues until *condition-1* is tested and found to be true, at which point control is transferred to the end of the PERFORM statement.

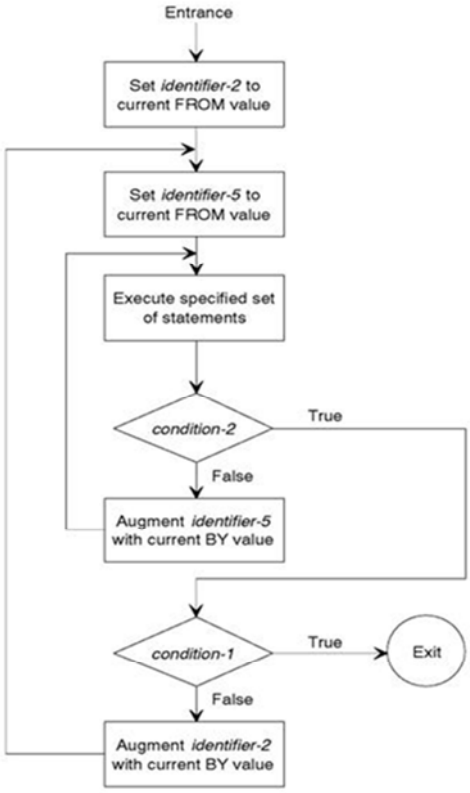
   The following flowchart is for the VARYING option of a PERFORM statement with the TEST AFTER phrase having one condition:



   When the data items referred to by two identifiers are varied, the content of the data item referred to by *identifier-2* is set to *literal-1* or the current value of the data item referred to by *identifier-3*; then the content of the data item referred to by *identifier-5* is set to *literal-3* or the current value of the data item referred to by *identifier-6*, and the specified set of statements is then executed. *Condition-2* is then evaluated; if false, the content of the data item referred to by *identifier-5* is augmented by *literal-4* or the content of the data item referred to by *identifier-7* and the specified set of statements is again executed. The cycle continues until *condition-2* is again evaluated and found to be true, at which time *condition-1* is evaluated; if false, the content of the data item referred to by *identifier-2* is augmented by *literal-2* or the content of the data item referred to by *identifier-4*, the content of the data item referred to by *identifier-5* is set to *literal-3* or the current value of the data item referred to by *identifier-6*, and the specified set of statements is again executed. This cycle continues until *condition-1* is again evaluated and found to be true, at which time control is transferred to the end of the PERFORM statement.

After completion of the PERFORM statement, each data item varied by an AFTER or VARYING phrase contains the same value it contained at the end of the most recent execution of the specified set of statements.

The following flowchart is for the VARYING option of a PERFORM statement with the TEST AFTER phrase with two conditions:



During the execution of the specified set of statements associated with the PERFORM statement, any change to the VARYING variable (the data item referred to by *identifier-2* and *index-name-1*), the BY variable (the data item referred to by *identifier-4*), the AFTER variable (the data item referred to by *identifier-5* and *index-name-3*), or the FROM variable (the data item referred to by *identifier-3* and *index-name-2*) is taken into consideration and affects the operation of the PERFORM statement.

When the data items referred to by two identifiers are varied, the data item referred to by *identifier-5* goes through a complete cycle (FROM, BY, UNTIL) each time the content of the data item referred to by *identifier-2* is varied. When the contents of three or more data items referred to by identifiers are varied, the mechanism is the same as for two identifiers except that the data item being varied by each AFTER phrase goes through a complete cycle each time the data item being varied by the preceding AFTER phrase is augmented.

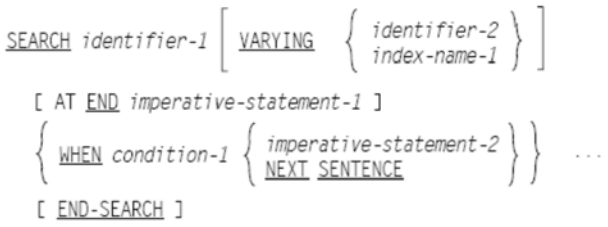
11. The range of a PERFORM statement consists logically of all those statements that are executed as a result of executing the PERFORM statement through execution of the implicit transfer of control to the end of the PERFORM statement. The range includes all statements that are executed as the result of a transfer of control by CALL, EXIT, GO TO, and PERFORM statements in the range of the PERFORM statement, as well as all statements in declarative procedures that are executed as a result of the execution of statements in the range of the PERFORM statement. The statements in the range of a PERFORM statement need not appear consecutively in the source program.
12. Statements executed as the result of a transfer of control caused by executing an EXIT PROGRAM statement are not considered to be part of the range of the PERFORM statement when
    a. The EXIT PROGRAM statement is specified in the same program in which the PERFORM statement is specified.
    b. The EXIT PROGRAM statement is within the range of the PERFORM statement.
13. *Procedure-name-1* and *procedure-name-2* must not name sections or paragraphs in any other program in the run unit, irrespective of whether or not the other program contains or is contained within the program that includes the PERFORM statement. Statements in other programs in the run unit can be obeyed only as a result of executing a PERFORM statement; if the range of that PERFORM statement includes CALL and EXIT PROGRAM statements. (See Section 7.2.3.)
14. If the range of a PERFORM statement includes another PERFORM statement, the sequence of procedures associated with the included PERFORM must itself either be totally included in, or totally excluded from, the logical sequence referred to by the first PERFORM. Therefore, an active PERFORM statement whose execution point begins within the range of another active PERFORM statement must not allow control to pass to the exit of the other active PERFORM statement; furthermore, two or more such active PERFORM statements cannot have a common exit.
15. A PERFORM statement that appears in a section that is not in an independent segment can have within its range, in addition to any declarative sections whose execution is caused within that range, only one of the following:
    a. Sections or paragraphs or both wholly contained in one or more nonindependent segments
    b. Sections or paragraphs or both wholly contained in a single independent segment
16. A PERFORM statement that appears in an independent segment can have within its range, in addition to any declarative sections whose execution is caused within that range, only one of the following:
    a. Sections or paragraphs or both wholly contained in one or more nonindependent segments.
    b. Sections or paragraphs or both wholly contained in the same independent segment as that PERFORM statement.
    c. Sections or paragraphs or both wholly contained in the same independent segment as that PERFORM statement.

3.6.22. The SEARCH Statement

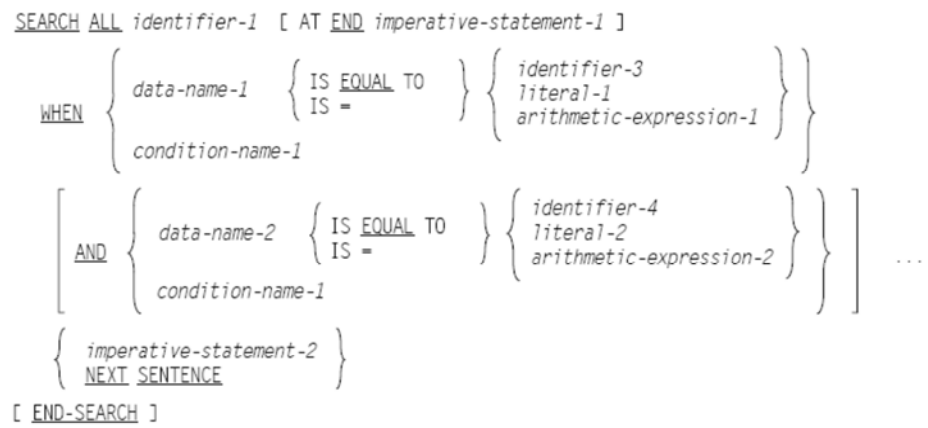
Function

The SEARCH statement is used to search a table for a table element that satisfies the specified condition, and to adjust the value of the associated index to indicate that table element.

Format 1



Format 2



Note: *The required relational character = is not underlined to avoid confusion with other symbols.*

Syntax Rules

1. In both formats 1 and 2, *identifier-1* must not be subscripted or reference-modified, but its description must contain an OCCURS clause including an INDEXED BY phrase. The description of *identifier-1* in format 2 must also contain the KEY IS phrase in its OCCURS clause
2. *Identifier-2* must refer to a data item described as USAGE IS INDEX or as a numeric elementary data item without any positions to the right of the assumed decimal point. *Identifier-2* cannot be subscripted by the first (or only) index name specified in the INDEXED BY phrase in the OCCURS clause associated with *identifier-1*.
3. In format 1, *condition-1* can be any conditional expression as described in Section a, “C onditional Expressions”.

4. In format 2, all referenced condition names must be defined as having only a single value. The data name associated with a condition name must appear in the KEY IS phrase in the OCCURS clause referred to by *identifier-1.* Each *data-name-1, data-name-2* can be qualified. Each *data-name-1, data-name-2* must be subscripted by the first index name associated with *identifier-1* along with other subscripts as required, and must be referred to in the KEY IS phrase in the OCCURS clause referred to by *identifier-1.*

   Identifier-3, identifier-4, or identifiers specified in *arithmetic-expression-1, arithmetic-expression-2* must not be referred to in the KEY IS phrase in the OCCURS clause referred to by *identifier-1* or be subscripted by the first index name associated with *identifier-1.*

   In format 2, when a data name in the KEY IS phrase in the OCCURS clause referred to by *identifier-1* is referred to or when a condition name associated with a data name in the KEY IS phrase in the OCCURS clause referred to by *identifier-1* is referenced, all preceding data names in the KEY IS phrase in the OCCURS clause referred to by *identifier-1* or their associated condition names must also be referred to.

5. If the END-SEARCH phrase is specified, the NEXT SENTENCE phrase must not be specified.

General Rules

1. The scope of a SEARCH statement can be terminated by any of the following:
   a. An END-SEARCH phrase at the same level of nesting
   b. A separator period
   c. An ELSE or END-IF phrase associated with a previous IF statement (see Section 2.4.21).
2. If format 1 of the SEARCH statement is used, a serial type of search operation takes place, starting with the current index setting.
   a. If, at the start of execution of the SEARCH statement, the index name associated with *identifier-1* contains a value that corresponds to an occurrence number that is greater than the highest permissible occurrence number for *identifier-1,* the SEARCH is terminated immediately. The number of occurrences of *identifier-1,* the last of which is the highest permissible, is discussed in Section 3.5.10, "The OCCURS Clause". Then, if the AT END phrase is specified, *imperative-statement-1* is executed. If the AT END phrase is not specified, control passes to the end of the SEARCH statement.

b. If, at the start of execution of the SEARCH statement, the index name associated with *identifier-1* contains a value that corresponds to an occurrence number that is not greater than the highest permissible occurrence number for *identifier-1* (the number of occurrences of *identifier-1,* the last of which is the highest permissible, is discussed in Section 3.5.10, "The OCCURS Clause"), the SEARCH statement operates by evaluating the conditions in the order that they are written, making use of the index settings, wherever specified, to determine the occurrence of those items to be tested. If none of the conditions is satisfied, the index name for *identifier-1* is incremented to obtain reference to the next occurrence. The process is then repeated using the new index name settings unless the new value of the index name settings for *identifier-1* corresponds to a table element outside the permissible range of occurrence values, in which case the search terminates as indicated in step 2a. If one of the conditions is satisfied upon its evaluation, the search terminates immediately, and control passes to the imperative statement associated with that condition, if present, or if the NEXT SENTENCE phrase is associated with that condition, to the next executable sentence; the index name remains set at the occurrence that caused the condition to be satisfied.

3. In a format 2 SEARCH, the results of the SEARCH ALL operation are predictable only when:

   a. The data in the table is ordered in the same manner as described in the KEY IS phrase of the OCCURS clause referred to by *identifier-1*.

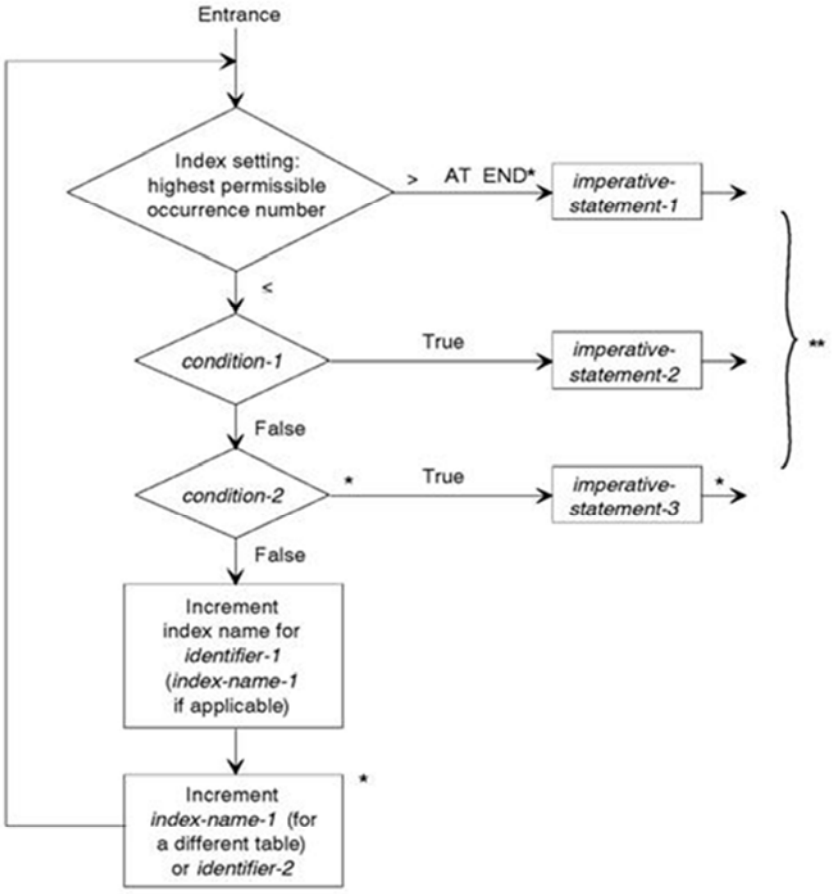
   b. The contents of the keys referred to in the WHEN phrase are sufficient to identify a unique table element.

4. If format 2 of the SEARCH statement is used, a nonserial type of search operation takes place; the initial setting of the index name for *identifier-1* is ignored and its setting is varied during the search operation according to a binary search algorithm, with the restriction that at no time is it set to a value that exceeds the value that corresponds to the last element of the table or that is less than the value that corresponds to the first element of the table. The length of the table is discussed in the OCCURS clause. (See Section 3.5.10.) If any of the conditions specified in the WHEN phrase cannot be satisfied for any setting of the index within the permitted range, control is passed to *imperative-statement-1* of the AT END phrase, when specified, or to the end of the SEARCH statement when this phrase is not specified; in either case, the final setting of the index is not predictable. If all the conditions can be satisfied, the index indicates an occurrence that allows the conditions to be satisfied, and control passes to *imperative-statement-2,* if specified or to the next executable sentence if the NEXT SENTENCE phrase is specified.

5. After execution of *imperative-statement-1* or *imperative-statement-2* that does not terminate with a GO TO statement, control passes to the end of the SEARCH statement.
6. In format 2, the index name that is used for the search operation is the first (or only) index name specified in the INDEXED BY phrase in the OCCURS clause associated with *identifier-1.* Any other index names for *identifier-1* remain unchanged.
7. In format 1, if the VARYING phrase is not used, the index name that is used for the search operation is the first (or only) index name specified in the INDEXED BY phrase in the OCCURS clause associated with *identifier-1.* Any other index names for *identifier-1* remain unchanged.
8. In format 1, if the VARYING *index-name-1* phrase is specified and if *index-name-1* appears in the INDEXED BY phrase in the OCCURS clause referred to by *identifier-1,* that index name is used for this search. If this is not the case or if the VARYING *identifier-2* phrase is specified, the first (or only) index name given in the INDEXED BY phrase in the OCCURS clause referred to by *identifier-1* is used for the search. In addition, the following operations occurs:
    a. If the VARYING *index-name-1* phrase is used and if *index-name-1* appears in the INDEXED BY phrase in the OCCURS clause referred to by another table entry, the occurrence number represented by *index-name-1* is increased by the same amount as, and at the same time as, the occurrence number represented by the index name associated with *identifier-1.*

    b. If the VARYING *identifier-2* phrase is specified and *identifier-2* is an index data item, then the data item referred to by *identifier-2* is increased by the same amount as, and at the same time as, the index associated with *identifier-1.* If *identifier-2* is not an index data item, the data item referred to by *identifier-2* is increased by the value one at the same time that the index referred to by the index name associated with *identifier-1.*
9. The END-SEARCH phrase delimits the scope of the SEARCH statement. (See Section 2.4.21)

The following flowchart shows the action of a format 1 SEARCH statement containing two WHEN phrases:



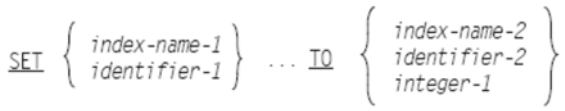
* These optional operations are included only when specified in the SEARCH statement.
** Each of these control transfers is to the end of the SEARCH statement unless the imperative statement ends with a GO TO statement.

3.6.23. The SET Statement

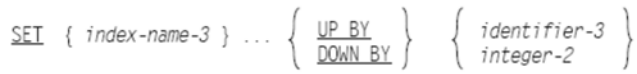
Function

1. The SET statement establishes reference points for a table-handling operation by setting indexes associated with table elements.
2. The SET statement is also used to alter the status of external switches.
3. The SET statement is also used to alter the value of conditional variables.
4. The SET statement is also used to set USAGE POINTER data items to a data address.

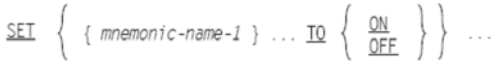
Format 1



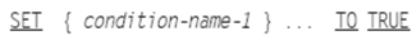
Format 2



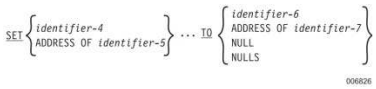
Format 3



Format 4



Format 5



Syntax Rules

1. All references to *index-name-1, identifier-1,* and *index-name-3* apply equally to all recursions thereof.
2. *Identifier-1* and *identifier-2* must each refer to an index data item or an elementary item described as an integer.
3. *Identifier-3* must refer to an elementary numeric integer.
4. *Integer-1* and *integer-2* can be signed. *Integer-1* must be positive
5. *Mnemonic-name-1* must be associated with an external switch, the status of which can be altered. SWITCH-13, SWITCH-14, . . ., SWITCH-24 can be referred to by the SET statement. ON sets a bit in the program condition word. OFF clears a bit in the program condition word.
6. *Condition-name-1* must be associated with a conditional variable.
7. Receiving field *identifier-5* must be level-01 or level-77 items defined in the Linkage Section. The addresses of these items are set to the value of the operand specified in the TO phrase.
8. *Identifier-7* must name an item of any level except 66 or 88 in the Linkage Section or the Working-Storage Section. ADDRESS OF *identifier-7* contains the two-word address of the identifier and not the content of the identifier.
9. NULL and NULLS set the receiving field to contain the value of two words of binary zero.

General Rules

The following is a list of general rules.

Formats 1 and 2

The following general rules apply to formats 1 and 2:

1. Index names are associated with a given table by being specified in the INDEXED BY phrase of the OCCURS clause for that table.
2. If index-name-1 is specified, the value of the index after the execution of the SET statement must correspond to an occurrence number of an element in the table associated with index-name-1.

   The value of the index associated with an index name after the execution of a PERFORM or SEARCH statement can be set to an occurrence number that is outside the range of its associated table. (See Section 3.6.21 and Section 3.6.22.)

   If *index-name-2* is specified, the value of the index before the execution of the SET statement must correspond to an occurrence number of an element in the table associated with *index-name-1.*

   If *index-name-3* is specified, the value of the index both before and after the execution of the SET statement must correspond to an occurrence number of an element in the table associated with *index-name-3.*
3. In format 1, the following action occurs:
   a. *Index-name-1* is set to a value causing it to refer to the table element that corresponds in occurrence number to the table element referred to by *index-name-2, identifier-2,* or *integer-1.* If *identifier-2* refers to an index data item, or if *index-name-2* is related to the same table as *index-name-1,* no conversion takes place.
   b. If *identifier-1* refers to an index data item, it can be set equal to either the content of *index-name-2* or *identifier-2,* where *identifier-2* also refers to an index data item; no conversion takes place in either case.
   c. If *identifier-1* does not refer to an index data item, it can be set only to an occurrence number that corresponds to the value of *index-name-2.* Neither *identifier-2* nor *integer-1* can be used in this case.
   d. The process is repeated for each recurrence of *index-name-1* or *identifier-1* if specified. Each time, the value of *index-name-2* or the data item referred to by *identifier-2* is used as it was at the beginning of the execution of the statement. Any subscripting associated with *identifier-1* is evaluated immediately before the value of the respective data item is changed.
4. In format 2, the contents of index-name-3 are incremented (UP BY) or decremented (DOWN BY) by a value that corresponds to the number of occurrences represented by the value of integer-2 or the data item referred to by identifier-3; thereafter, the process is repeated for each recurrence of index-name-3. For each repetition, the value of the data item referred to by identifier-3 is used as it was at the beginning of the execution of the statement.

5. Data in the following table represents the validity of various operand combinations in format 1 of the SET statement. The general rule reference indicates the applicable general rule.

Sending Item	Receiving Item		
	Integer Data Item	**Index**	**Index Data Item**
Integer Literal	No/3c	Valid/3a	No/3b
Integer Data Item	No/3c	Valid/3a	No/3b
Index	Valid/3c	Valid/3a	Valid/3b*
Index Data Item	No/3c	Valid/3a*	Valid/3b*

* No conversion takes place

Format 3

The following general rule applies to format 3:

6. The status of each external switch associated with the specified *mnemonic-name-1* is modified such that the truth value resultant from evaluation of a condition name associated with that switch reflects as on status if the ON phrase is specified or an off status if the OFF phrase is specified. (See Section a)

Format 4

The following general rules apply to format 4:

7. The literal in the VALUE clause associated with condition-name-1 is moved to the conditional variable as if a statement of the form "MOVE literal TO conditional-variable" were executed. If more than one literal is specified in the VALUE clause, the literal in the MOVE statement is the first literal in the VALUE clause.

8. If multiple condition names are specified, the results are the same as if a separate SET statement had been written for each condition name in the same order as specified in the SET statement.

Format 5

9. When this form of the SET statement is executed, the current value of the receiving field is replaced by the address value contained in the sending field.

The following table shows valid combination of sending and receiving fields in a format-5 SET statement:

Sending Field	Receiving Field		
	USAGE IS POINTER	ADDRESS OF	NULL/NULLS
USAGE IS POINTER	Valid	Valid	Invalid
ADDRESS OF	Valid	Valid	Invalid
NULL/NULLS	Valid	Valid	Invalid

3.6.24. The STOP Statement

Function

The STOP statement causes a permanent or temporary suspension of the execution of the run unit. The literal variation of the STOP statement is an obsolete element in this revision of American National Standard COBOL and is to be deleted from the next revision of American National Standard COBOL.

General Format



Syntax Rules

1. *Literal-1* must not be a figurative constant that begins with the word ALL.
2. If a STOP RUN statement appears in a consecutive sequence of imperative statements within a sentence, it must appear as the last statement in that sequence.
3. If the *literal* is numeric, it must be an unsigned integer.

General Rules

1. If the RUN phrase is specified, execution of the run unit ceases and control is transferred to the operating system.
2. During the execution of a STOP RUN statement, an implicit CLOSE statement without any optional phrases is executed for each file that is in the open mode in the run unit. Any USE procedures associated with any of these files are not executed.
3. If STOP *literal-1* is specified, the execution of the run unit is suspended and *literal-1* is communicated to the operator using the CONSOLE as defined in the DISPLAY statement.

Upon receipt of any key-in by the operator upon the CONSOLE as defined in the ACCEPT statement, one of the following events occurs:

a. If the operator has keyed in GO, continuation of the execution of the run unit begins with the next executable statement.

b. If the operator has keyed in NO, the object program terminates as if a STOP RUN had been encountered.

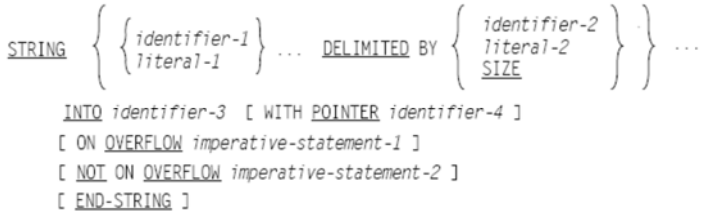
c. If the operator has keyed in an invalid response, a message informing the operator of the valid response is displayed and the STOP literal statement is executed again.

3.6.25. The STRING Statement

Function

The STRING statement provides juxtaposition of the partial or complete contents of two or more data items into a single data item.

General Format



Syntax Rules

1. *Literal-1* and *literal-2* must not be a figurative constant that begins with the word ALL.
2. All literals must be described as nonnumeric literals, and all identifiers, except *identifier-4,* must be described implicitly or explicitly as USAGE IS DISPLAY or DISP-I18N.
3. *Identifier-3* must not be reference-modified.
4. *Identifier-3* must not represent an edited data item and must not be described with the JUSTIFIED clause.
5. *Identifier-4* must be described as an elementary numeric integer data item of sufficient size to contain a value equal to 1 plus the size of the data item referred to by identifier-3. The symbol "P" cannot be used in the PICTURE character string of *identifier-4.*
6. Where *identifier-1* or *identifier-2* is an elementary numeric data item, it must be described as an integer without the symbol "P" in its PICTURE character string.

General Rules

1. *Identifier-1* or *literal-1* represent the sending item. *Identifier-3* represents the receiving item.
2. *Literal-2* or the content of the data item referred to by *identifier-2* indicate the characters delimiting the move. If the SIZE phrase is used, the content of the complete data item defined by *identifier-1* or *literal-1* is moved. When a figurative constant is used as the delimiter, it is a single-character nonnumeric literal.
3. When a figurative constant is specified as *literal-1* or *literal-2,* it refers to an implicit 1-character data item whose usage is DISPLAY.
4. When the STRING statement is executed, the transfer of data is governed by the following rules:
   a. The characters from *literal-1* or from the contents of the data item referred to by *identifier-1* are transferred to the data item referred to by *identifier-3* in accordance with the rules for alphanumeric-to-alphanumeric moves, except that space-filling will not be provided. (See general rule 4a in 3.6.19.)
   b. If the DELIMITED phrase is specified without the SIZE phrase, the content of the data item referred to by *identifier-1,* or the value of *literal-1,* is transferred to the receiving data item in the sequence specified in the STRING statement beginning with the leftmost character and continuing from left to right until the end of the sending data item is reached or the end of the receiving data item is reached, or until the characters specified by *literal-2* or by the content of the data item referred to by *identifier-2* are encountered. The characters specified by *literal-2* or by the data item referenced by *identifier-2* are not transferred.
   c. If the DELIMITED phrase is specified with the SIZE phrase, the entire content of *literal-1,* or the content of the data item referred to by *identifier-1,* is transferred in the sequence specified in the STRING statement to the data item referred to by *identifier-3* until all data has been transferred or the end of the data item referred to by *identifier-3* has been reached.

      This behavior is repeated until all occurrences of *literal-1* or data items referred to by *identifier-1* have been processed.
5. If the POINTER phrase is specified, the data item referred to by *identifier-4* must be set to an initial value greater than zero before the execution of the STRING statement.
6. If the POINTER phrase is not specified, the following general rules apply as if the user had specified *identifier-4* referring to a data item with an initial value of one.
7. When characters are transferred to the data item referred to by *identifier-3,* the moves behave as though the characters were moved one at a time from the source into the character position of the data item referred to by *identifier-3* designated by the value of the data item referred to by *identifier-4* (provided the value of the data item referred to by *identifier-4* does not exceed the length of the data item referred to by *identifier-3*), and then the data item referred to by *identifier-4* was increased by one before the move of the next character or before the end of execution of the STRING statement. The value of the data item referred to by *identifier-4* is changed during execution of the STRING statement only by the behavior specified in the preceding rules.

8. At the end of execution of the STRING statement, only the portion of the data item referred to by *identifier-3* that was referred to during the execution of the STRING statement is changed. All other portions of the data item referred to by *identifier-3* contains data that was present before this execution of the STRING statement.

9. Before each move of a character to the data item referred to by *identifier-3,* if the value associated with the data item referred to by *identifier-4* is either less than one or exceeds the number of character positions in the data item referred to by *identifier-3,* no (further) data is transferred to the data item referred to by *identifier-3,* and the NOT ON OVERFLOW phrase, if specified, is ignored and control is transferred to the end of the STRING statement or, if the ON OVERFLOW phrase is specified, to *imperative-statement-1.* If control is transferred to *imperative-statement-1,* execution continues according to the rules for each statement specified in *imperative-statement-1.* If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of *imperative-statement-1,* control is transferred to the end of the STRING statement.

10. If, at the time of execution of a STRING statement with the NOT ON OVERFLOW phrase, the conditions described in general rule 9 are not encountered, after completion of the transfer of data according to the other general rules, the ON OVERFLOW phrase, if specified, is ignored and control is transferred to the end of the STRING statement or, if the NOT ON OVERFLOW phrase is specified, to *imperative-statement-2.* If control is transferred to *imperative-statement-2,* execution continues according to the rules for each statement specified in *imperative-statement-2.* If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of *imperative-statement-2,* control is transferred to the end of the STRING statement.

11. The END-STRING phrase delimits the scope of the STRING statement (see Section 2.4.21)

12. If *identifier-1* or *identifier-2* occupy the same storage area as *identifier-3* or *identifier-4* or if *identifier-3* and *identifier-4* occupy the same storage area, the result of the execution of this statement is undefined, even if they are defined by the same data description entry. (See Section 3.6.5)

The following examples illustrate some of the ways STRING can be used. In each case assume that the receiving field is defined as an X(20) and contains spaces.

Example

```
STRING "DEAR" DELIMITED SIZE,
    TITLE DELIMITED SPACE,
    SPACE DELIMITED SIZE,
    LAST-NAME DELIMITED SPACE,
    ":" DELIMITED SIZE INTO LETTER-SALUTE.
```

would result in

```
DEAR MR JONES:
```

or

```
DEAR MISS MARTIN:
```

depending on the contents of TITLE and LAST-NAME.

Example

```
MOVE 1 TO CHAR-CT.
    STRING FLD1, "/", FLD2, "/", FLD3, "/", FLD4, "/"
        DELIMITED BY SPACE INTO
        COMPRESS-FLD POINTER CHAR-CT.
```

Assuming

```
FLD1 contains "AA A"
FLD2 contains "BB20202020"
FLD3 contains "CCC"
FLD4 contains "D2020"
```

COMPRESS-FLD contains

```
AA/BB/CCC/D/
```

CHAR-CT equals to 13 and could be used to begin a new stringing operation at location 13.

3.6.26. The SUBTRACT Statement

Function

The SUBTRACT statement is used to subtract one, or the sum of two or more, numeric data items from one or more items, and set the values of one or more items equal to the results.

Format 1

```
SUBTRACT { identifier-1 } ... FROM { identifier-2 [ ROUNDED ] } ...
         { literal-1    }
    [ ON SIZE ERROR imperative-statement-1 ]
    [ NOT ON SIZE ERROR imperative-statement-2 ]
    [ END-SUBTRACT ]
```

Format 2

```
SUBTRACT { identifier-1 } ... FROM { identifier-2 }
         { literal-1    }        { literal-2    }
    GIVING { identifier-2 [ ROUNDED ] } ...
  [ ON SIZE ERROR imperative-statement-1 ]
  [ NOT ON SIZE ERROR imperative-statement-2 ]
  [ END-SUBTRACT ]
```

Format 3

```
SUBTRACT { CORRESPONDING } identifier-1 FROM identifier-2 [ ROUNDED ]
         { CORR          }
    [ ON SIZE ERROR imperative-statement-1 ]
    [ NOT ON SIZE ERROR imperative-statement-2 ]
    [ END-SUBTRACT ]
```

Syntax Rules

1. Each identifier must refer to a numeric elementary item except that:
   a. In format 2, each identifier following the word GIVING must refer to either an elementary numeric item or an elementary numeric-edited item.
   b. In format 3, each identifier must refer to a group item.
2. Each literal must be a numeric literal.
3. The composite of operands must not contain more than 18 digits. (See Section 3.6.5)
   a. In format 1, the composite of operands is determined by using all of the operands in a given statement.
   b. In format 2, the composite of operands is determined by using all of the operands in a given statement excluding the data items that follow the word GIVING.
   c. In format 3, the composite of operands is determined separately for each pair of corresponding data items.

4. CORR is an abbreviation for CORRESPONDING.

General Rules

1. When format 1 is used, the values of the operands preceding the word FROM are added together and the sum is stored in a temporary data item. The value in this temporary data item is subtracted from the value of the data item referred to by *identifier-2,* storing the result into the data item referred to by *identifier-2,* and repeating this process for each successive occurrence of *identifier-2* in the left-to-right order in which *identifier-2* is specified.
2. In format 2, all literals and the values of the data items referred to by the identifiers preceding the word FROM are added together, the sum is subtracted from *literal-2* or the value of the data item referred to by *identifier-2,* and the result of the subtraction is stored as the new content of each data item referred to by *identifier-3.*
3. If format 3 is used, data items in *identifier-1* are subtracted from and stored into corresponding data items in *identifier-2.*
4. The compiler ensures that enough places are carried so as not to lose significant digits during execution.
5. Additional rules and explanations relative to the SUBTRACT statement are given under the appropriate paragraphs. See Section 3.6.5 for the ROUNDED phrase, Section 3.6.5 for the SIZE ERROR phrase, Section 2.4.21 for the Scope Terminators; Section 3.6.5 for arithmetic statements, Section 3.6.5 for overlapping operands, and Section 3.6.5 for multiple results in arithmetic statements.

For the examples of the SUBTRACT statement, assume the following definition and values for the operands:

```
A       PIC      99V9          VALUE  11.1
B       PIC      S9V9(3)       VALUE - 0.002
C       PIC      S99V9(5)      VALUE   1.11111
D       PIC      $$99.99.
```

Statement	Calculation	Result
SUBTRACT 16,A,B FROM C	1.11111 - (16 + 11.1 - .002)	25V9868R
SUBTRACT A, B FROM D	Illegal because D contains editing characters	-
SUBTRACT A, B FROM 126	Illegal because result cannot be stored in a literal	-
SUBTRACT A, B FROM 126 GIVING D	126 - (11.1 - 0.002)	$114.90

In the first example, V indicates the assumed decimal point and R indicates the operational sign (negative) and last digit (9) of the result.

3.6.27. The UNSTRING Statement

Function

The UNSTRING statement causes contiguous data in a sending field to be separated and placed into multiple receiving fields.

General Format

```
UNSTRING identifier-1
  [ DELIMITED BY [ ALL ] { identifier-2 } [ OR [ ALL ] { identifier-3 } ] ... ]
                         { literal-1    }             { literal-2    }
INTO { identifier-4 [ DELIMITER IN identifier-5 ] [ COUNT IN identifier-6 ] } ...
[ WITH POINTER identifier-7 ] [ TALLYING IN identifier-8 ]
[ ON OVERFLOW imperative-statement-1 ]
[ NOT ON OVERFLOW imperative-statement-2 ]
[ END-UNSTRING ]
```

Syntax Rules

1. Each literal must be a nonnumeric literal. In addition, literals can be any figurative constant, but they cannot begin with the word ALL.
2. *Identifier-1, identifier-2, identifier-3,* and *identifier-5* must refer to data items described, implicitly or explicitly, as category alphanumeric.
3. *Identifier-4* can be described as either the category alphabetic, alphanumeric, or numeric (except that the symbol P cannot be used in the PICTURE character string), and must be described, implicitly or explicitly, as USAGE IS DISPLAY or DISP-I18N.
4. *Identifier-6* and *identifier-8* must refer to integer data items (except that the symbol P cannot be used in the PICTURE character string).
5. *Identifier-7* must be described as an elementary numeric integer data item of sufficient size to contain a value equal to 1 plus the size of the data item referred to by *identifier-1.* The symbol P cannot be used in the PICTURE character string of *identifier-7.*
6. The DELIMITER IN phrase and the COUNT IN phrase can be specified only if the DELIMITED BY phrase is specified.
7. *Identifier-1* must not be reference modified.

General Rules

1. All references to *identifier-2* and *literal-1* apply equally to *identifier-3* and *literal-2*, respectively.
2. The data item referred to by *identifier-1* represents the sending area.
3. The data item referred to by *identifier-4* represents the data receiving area. The data item referred to by *identifier-5* represents the receiving area for delimiters.
4. *Literal-1* or the data item referred to by *identifier-2* specifies a delimiter.

5. The data item referred to by *identifier-6* represents the count of the number of characters within the data item referred to by *identifier-1* isolated by the delimiters for the move to the data item referred to by *identifier-4*. This value does not include a count of the delimiter characters.
6. The data item referred to by *identifier-7* contains a value that indicates a relative character position within the area referred to by *identifier-1.*
7. The data item referred to by *identifier-8* is a counter that is increased by one for each occurrence of the data item referred to by *identifier-4* accessed during the UNSTRING operation.
8. When a figurative constant is used as the delimiter, it stands for a single-character nonnumeric literal. When the ALL phrase is specified, one occurrence or two or more contiguous occurrences of *literal-1* (figurative constant or not) or the content of the data item referred to by *identifier-2* are treated as if they were only one occurrence, and one occurrence of *literal-1* or the data item referred to by *identifier-2* is moved to the receiving data item according to the rules in general rule 13d.
9. When any examination encounters two contiguous delimiters, the current receiving area is space-filled if it is described as alphabetic or alphanumeric, or zero-filled if it is described as numeric.
10. *Literal-1* or the content of the data item referred to by *identifier-2* can contain any character in the computer's character set.
11. Each *literal-1* or the data item referred to by *identifier-2* represents one delimiter. When a delimiter contains two or more characters, all of the characters must be present in contiguous positions of the sending item, and in the order given, to be recognized as a delimiter.
12. When two or more delimiters are specified in the DELIMITED BY phrase, an OR condition exists between them. Each delimiter is compared to the sending field. If a match occurs, the characters in the sending field are considered to be a single delimiter. No characters in the sending field can be considered a part of more than one delimiter.

    Each delimiter is applied to the sending field in the sequence specified in the UNSTRING statement.
13. When the UNSTRING statement is initiated, the current receiving area is the data item referred to by *identifier-4.*

    Data is transferred from the data item referred to by *identifier-1* to the data item referred to by *identifier-4* according to the following rules:

    a. If the POINTER phrase is specified, the string of characters referred to by *identifier-1* is examined beginning with the relative character position indicated by the content of the data item referred to by *identifier-7.* If the POINTER phrase is not specified, the string of characters is examined beginning with the leftmost character position.

b. If the DELIMITED BY phrase is specified, the examination proceeds left to right until either a delimiter specified by *literal-1* or the value of the data item referred to by *identifier-2* is encountered. (See general rule 11.) If the DELIMITED BY phrase is not specified, the number of characters examined is equal to the size of the current receiving area. However, if the sign of the receiving item is defined as occupying a separate character position, the number of characters examined is one less than the size of the current receiving area.

   If the end of the data item referred to by *identifier-1* is encountered before the delimiting condition is met, the examination terminates with the last character examined.

c. The characters therefore examined (excluding the delimiting characters, if any) are treated as an elementary alphanumeric data item and are moved into the current receiving area according to the rules for the MOVE statement. (See Section 3.6.19.)

d. If the DELIMITER IN phrase is specified, the delimiting characters are treated as an elementary alphanumeric data item and are moved into the data item referred to by *identifier-5* according to the rules for the MOVE statement. (See Section 3.6.19.) If the delimiting condition is the end of the data item referred to by *identifier-1*, the data item referred to by *identifier-5* is space-filled.

e. If the COUNT IN phrase is specified, a value equal to the number of characters therefore examined (excluding the delimiter characters, if any) is moved into the area referred to by *identifier-6* according to the rules for an elementary move.

f. If the DELIMITED BY phrase is specified, the string of characters is further examined beginning with the first character to the right of the delimiter. If the DELIMITED BY phrase is not specified, the string of characters is further examined beginning with the character to the right of the last character transferred.

g. After data is transferred to the data item referred to by *identifier-4,* the current receiving area is the data item referred to by the next recurrence of *identifier-4.* The behavior described in paragraphs 13b through 13f is repeated until either all the characters are exhausted in the data item referred to by *identifier-1* or until there are no more receiving areas.

14. The initialization of the contents of the data items associated with the POINTER phrase or the TALLYING phrase is the responsibility of the user.

15. The content of the data item referred to by *identifier-7* is incremented by one for each character examined in the data item referred to by *identifier-1*. When the execution of an UNSTRING statement with a POINTER phrase is completed, the content of the data item referred to by *identifier-7* contains a value equal to the initial value plus the number of characters examined in the data item referred to by *identifier-1.*

16. When the execution of an UNSTRING statement with a TALLYING phrase is completed, the content of the data item referred to by *identifier-8* contains a value equal to its value at the beginning of execution of the statement plus a value equal to the number of *identifier-4* receiving data items accessed during execution of the statement.
17. Either of the following situations causes an overflow condition:
    a. An UNSTRING is initiated, and the value in the data item referred to by *identifier-7* is less than one or greater than the size of the data item referred to by *identifier-1*.
    b. If, during execution of an UNSTRING statement, all receiving areas have been acted upon, and the data item referred to by *identifier-1* contains characters that have not been examined.
18. When an overflow condition exists, the UNSTRING operation is terminated, the NOT ON OVERFLOW phrase, if specified, is ignored, and control is transferred to the end of the UNSTRING statement or, if the ON OVERFLOW phrase is specified, to *imperative-statement-1*. If control is transferred to *imperative-statement-1,* execution continues according to the rules for each statement specified in *imperative-statement-1.* If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of *imperative-statement-1,* control is transferred to the end of the UNSTRING statement.
19. The END-UNSTRING phrase delimits the scope of the UNSTRING statement. (See Section 2.4.21)
20. If, at the time of execution of an UNSTRING statement, the conditions described in general rule 17 are not encountered, after completion of the transfer of data according to the other general rules, the ON OVERFLOW phrase, if specified, is ignored and control is transferred to the end of the UNSTRING statement or, if the NOT ON OVERFLOW phrase is specified, to *imperative-statement-2*. If control is transferred to *imperative-statement-2,* execution continues according to the rules for each statement specified in *imperative-statement-2.* If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of *imperative-statement-2,* control is transferred to the end of the UNSTRING statement.
21. If identifier-*1, identifier-2,* or *identifier-3* occupy the same storage area as *identifier-4, identifier-5, identifier-6, identifier-7,* or *identifier-8,* or if *identifier-4, identifier-5,* or *identifier-6,* occupy the same storage area as *identifier-7* or *identifier-8,* or if *identifier-7* and *identifier-8* occupy the same storage area, the result of the execution of this statement is undefined, even if they are defined by the same data description entry. (See Section 3.6.5.).

The following examples illustrate some of the ways the UNSTRING statement can be used.

Example

```
02 FLD1 PIC X(5)
02 FLD2 PIC X(5)
02 FLD3 PIC X(5)
02 FLD4 PIC X(5)
02 PACKED-FLD PIC X (14)
    .
    .
    .
UNSTRING PACKED-FLD
DELIMITED BY "/" INTO
FLD1, FLD2, FLD3, FLD4
```

Assume PACKED-FLD contains

```
AA/BB/CCCCCC/D
```

After execution, the contents of the other fields are

```
FLD1:  AA
FLD2:  BB
FLD3:  CCCCC
FLD4:  D
```

Example

```
02 PT-CT PIC 99.
02 UNPACK OCCURS 4 TIMES
    03 RECVR PIC XX.
    03 DELM PIC XX.
    03 CT PIC 99.
       .
       .
       .
MOVE 0 TO TALLY-REG
MOVE 5 TO PT-CT
UNSTRING PACK-FLD
   DELIMITED BY SPACE OR ALL "/" INTO
   RECVR (1) DELIMITER DELM (1) COUNT CT (1)
   RECVR (2) DELIMITER DELM (2) COUNT CT (2)
   RECVR (3) DELIMITER DELM (3) COUNT CT (3)
   RECVR (4) DELIMITER DELM (4) COUNT CT (4)
   POINTER PT-CT
   TALLYING IN TALLY-REG
   ON OVERFLOW STOP RUN.
```

Assume PACK-FLD is defined as X(17) and contains

```
//ΔABCDΔ/DE//FΔΔG
```

The execution of the preceding code results in the STOP RUN being executed and the fields contains

N	RECVR (N)	DELM (N)	CT(N)
1	BC	ΔΔ	3
2	ΔΔΔ	/Δ	0
3	DE	/Δ	2
4	FΔ	ΔΔ	1

```
PT-CT       = 16
TALLY-REG   =  4
```

Example

```
02 NUM1.
     03 THOUSANDS PIC 999.
     03 HUNDREDS PIC 999.
     03 FRCTN PIC X(4).

02 NUM2 REDEFINES NUM1 PIC 9(6)V9(4).
        .
        .
        .
UNSTRING EDIT-NUMBER
     DELIMITED BY SPACE OR "," OR "."
     INTO THOUSANDS, HUNDREDS, FRCTN.
INSPECT FRCTN REPLACING ALL SPACE BY ZERO
```

Assuming EDIT-NUMBER contains 1,105,345DDDD and is described as X(13), after the execution of the UNSTRING and the INSPECT, NUM2 contains the value 001105V3450.

3.7. Debugging

This subsection provides information on debugging within the Nucleus.

3.7.1. General Description

Debugging within the Nucleus module provides the user with debugging lines and a compile-time switch for debugging lines.

3.7.2. Compile-Time Switch

The WITH DEBUGGING MODE clause is written as part of the SOURCE-COMPUTER paragraph (see Section 3.4.3). It serves as a compile-time switch over the debugging lines written in the separately compiled program.

When the WITH DEBUGGING MODE clause is specified in a separately compiled program, all debugging lines are compiled as specified in this presentation of the Nucleus module. When the WITH DEBUGGING MODE clause is not specified, all debugging lines are compiled as if they were comment lines.

The presence or absence of the WITH DEBUGGING MODE clause is logically determined after all COPY and REPLACE statements are processed.

3.7.3. Debugging Lines

A debugging line is any line with a D in the indicator area of the line. Any debugging line that consists solely of spaces from margin A to margin R is considered the same as a blank line.

The contents of a debugging line must be such that a syntactically correct program is formed with or without the debugging lines being considered as comment lines.

After all COPY and REPLACE statements have been processed, a debugging line will be considered to have all the characteristics of a comment line if the WITH DEBUGGING MODE clause is not specified in the SOURCE-COMPUTER paragraph.

Successive debugging lines are allowed.

A debugging line is permitted only in the separately compiled program after the OBJECT-COMPUTER paragraph.

Section 4
Sequential I-O

The Sequential I-O module provides a capability to access records of a file in established sequence. The sequence is established as a result of writing the records to the file.

4.1. Level Characteristics

Sequential I-O level 1 provides limited capabilities for the file control entry, the file description entry, and entries in the I-O-CONTROL paragraph. Within the Procedure Division, the Sequential I-O module level 1 provides limited capabilities for the CLOSE, OPEN, READ, USE, and WRITE statements; full capabilities are provided for the REWRITE statement.

Sequential I-O level 2 provides full capabilities for the file control entry, the entries in the I-O-CONTROL paragraph, and the file description entry. Within the Procedure Division, the Sequential I-O module level 2 provides full capabilities for the CLOSE, OPEN, READ, REWRITE, USE, and WRITE statements.

4.2. Language Concepts

This subsection describes language concepts associated with the Sequential I-O module.

4.2.1. Organization

Sequential files are organized so that each record, except the last, has a unique successor record; each record, except the first, has a unique predecessor record. The successor relationships are established by the order of execution of WRITE statements when the file is created. Once established, the successor relationships do not change except in cases where records are added to the end of the file.

A sequentially organized mass storage file has the same logical structure as a file on any sequential medium; however, a sequential mass storage file can be updated in place. When this technique is used, new records cannot be added to the file, and each replaced record must be the same size as the original record.

4.2.2. Access Mode

For sequential organization, the order of sequential access is the order in which the records were originally written.

4.2.3. Current Volume Pointer

The current volume pointer is a conceptual entity used in this manual to specify the current physical volume of a sequential file. The status of the current volume pointer is affected by the CLOSE, OPEN, READ, and WRITE statements.

4.2.4. File Position Indicator

The file position indicator is a conceptual entity used in this manual to specify the next record to be accessed within a given file during certain sequences of input-output operations. The setting of the file position indicator is affected only by the CLOSE, OPEN, and READ statements. The concept of the file position indicator has no meaning for a file opened in the output or extend mode.

4.2.5. I-O Status

The I-O status is a 2-character conceptual entity whose value is set to indicate the status of an input-output operation during the execution of a CLOSE, OPEN, READ, REWRITE, or WRITE statement and before the execution of any imperative statement associated with that input-output statement or before the execution of any applicable USE AFTER STANDARD EXCEPTION procedure. The value of the I-O status is made available to the COBOL program through the use of the FILE STATUS clause in the file description entry or the file control entry for the file.

The I-O status also determines whether an applicable USE AFTER STANDARD EXCEPTION procedure is executed. If any condition other than those contained in a successful completion results, such a procedure can be executed according to rules stated for the USE statement. If one of the conditions listed under successful completion results, no such procedure is executed.

Certain classes of I-O status values indicate critical error conditions. These are any that begin with the digit 3 or 4, and any that begin with the digit 9 that the implementor defines as critical. If the value of the I-O status for an input-output operation indicates such an error condition, the action taken after the execution of any applicable USE AFTER STANDARD EXCEPTION procedure, or if none applies, after completion of the normal input-output control system error processing is defined in B.5.41.

I-O status expresses one of the following conditions upon completion of the input-output operation:

1. Successful completion. The input-output statement was successfully executed.
2. At end. A sequential READ statement was unsuccessfully executed as a result of an at-end condition.
3. Permanent error. The input-output statement was unsuccessfully executed as the result of an error that precluded further processing of the file. Any specified exception procedures are executed. The permanent error condition remains in effect for all subsequent input-output operations on the file.
4. Logic error. The input-output statement was unsuccessfully executed as a result of an improper sequence of input-output operations that were performed on the file.

5. Defined by Unisys. The input-output statement was unsuccessfully executed as a result of a condition not indicated by other defined values of I-O status.

The following list describes the values placed in the I-O status for the previously named conditions resulting from the completion of an input-output operation on a sequential file. If more than one value applies, the first status encountered is placed in the I-O status.

1. Successful completion:
    a. I-O Status = 00. The input-output statement is successfully executed and no further information is available concerning the input-output operation.
    b. I-O Status = 04. A READ statement is successfully executed but the length of the record being processed does not conform to the fixed file attributes for that file.
    c. I-O Status = 05. An OPEN statement is successfully executed, but the referenced optional file is not present at the time the OPEN statement is executed. If the open mode is I-O or extend, the file has been created.
    d. I-O Status = 07. The input-output statement is successfully executed. However, for a CLOSE statement with the NO REWIND, REEL/UNIT, or FOR REMOVAL phrase or for an OPEN statement with the NO REWIND phrase, the referenced file is on a nonreel/unit medium.
2. At-end condition with unsuccessful completion:

    I-O Status = 10. A sequential READ statement is attempted and no next logical record exists in the file because:

    a. The end of the file has been reached.
    b. A sequential READ statement is attempted for the first time on an optional input file that is not present.
3. Permanent error condition with unsuccessful completion:
    a. I-O Status = 30. A permanent error exists and no further information is available concerning the input-output operation.
    b. I-O Status = 34. A permanent error exists because of a boundary violation; an attempt is made to write beyond the externally defined boundaries of a sequential file, as defined on the @ASG statement for the file.
    c. I-O Status = 35. A permanent error exists because an OPEN statement with the INPUT, I-O, or EXTEND phrase is attempted on a nonoptional file that is not present.
    d. I-O Status = 37. A permanent error exists because an OPEN statement is attempted on a file and that file does not support the open mode specified in the OPEN statement. The possible violations are
        - The EXTEND or OUTPUT phrase is specified but the file does not support write operations.
        - The I-O phrase is specified but the file does not support the input and output operations that are permitted for a sequential file when opened in the I-O mode.

- The INPUT phrase is specified but the file does not support read operations.

e. I-O Status = 38. A permanent error exists because an OPEN statement is attempted on a file previously closed with lock.

f. I-O Status = 39. The OPEN statement is unsuccessful because a conflict has been detected between the fixed file attributes and the attributes specified for that file in the program.

4. Logic error condition with unsuccessful completion

a. I-O Status = 41. An OPEN statement is attempted for a file in the open mode.

b. I-O Status = 42. A CLOSE statement is attempted for a file not in the open mode.

c. I-O Status = 43. For a mass storage file in the sequential access mode, the last input-output statement executed for the associated file before the execution of a REWRITE statement was not a successfully executed READ statement.

d. I-O Status = 44. A boundary violation exists because

- An attempt is made to WRITE or REWRITE a record that is larger than the largest or smaller than the smallest record allowed by the RECORD IS VARYING clause of the associated file name.
- An attempt is made to REWRITE a record to a sequential file and the record is not the same size as the record being replaced.

e. I-O Status = 46. A sequential READ statement is attempted on a file open in the input or I-O mode, and no valid next record has been established because

- The preceding READ statement was unsuccessful but did not cause an at end condition.
- The preceding READ statement caused an at-end condition.

f. I-O Status = 47. The execution of a READ statement is attempted on a file not open in the input or I-O mode.

g. I-O Status = 48. The execution of a WRITE statement is attempted on a file not open in the output or extend mode.

h. I-O Status = 49. The execution of a REWRITE statement is attempted on a file not open in the I-O mode.

4.2.6. The At-End Condition

The at-end condition can occur as a result of the execution of a READ statement. (See Section B.5.27.)

4.2.7. The File Attribute Conflict Condition

The file attribute conflict condition can result from the execution of an OPEN, REWRITE, or WRITE statement. When the file attribute conflict condition occurs, execution of the input-output statement that recognized the condition is unsuccessful and the file is not affected. (See B.5.25, B.5.30, and B.5.43.)

When the file attribute conflict condition is recognized, these actions take place in the following order:

1. A value is placed in the I-O status associated with the file name to indicate the file attribute conflict condition.
2. Any USE AFTER EXCEPTION procedure associated with the file name is executed.

4.2.8. Special Register LINAGE-COUNTER

The reserved word LINAGE-COUNTER is a name for a line counter generated by the presence of a LINAGE clause in a file description entry. (See Section 4.4.5.) The implicit description is that of an unsigned integer whose size is equal to the size of *integer-1* or the data item referred to by *data-name-1* in the LINAGE clause. LINAGE-COUNTER can be referred to only in Procedure Division statements; however, only the input-output control system can change the value of LINAGE-COUNTER.

If LINAGE IS SYSTEM is specified, the implicit size of the LINAGE-COUNTER is large enough to contain the system standard page size.

4.3. Environment Division/Input-Output Section

The Input-Output Section is located in the Environment Division of a source program. The Input-Output Section deals with the information needed to control transmission and handling of data between external media and the object program. The Input-Output Section is optional in the Environment Division of a COBOL source program.

The general format of the Input-Output Section is shown below.

```
INPUT-OUTPUT SECTION.
FILE-CONTROL.  {file-control-entry} . . .
[I-O CONTROL.  [input-output-control-entry] ]
```

4.3.1. The FILE-CONTROL Paragraph

Function

The FILE-CONTROL paragraph allows specification of file related information.

General Format

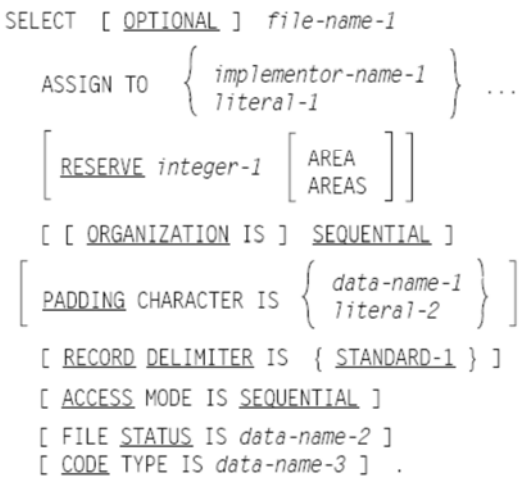
```
FILE-CONTROL. { file-control-entry } . . .
```

4.3.2. The File Control Entry

Function

The file control entry declares the relevant physical attributes of a sequential file.

General Format



Syntax Rules

1. The SELECT clause must be specified first in the file control entry.The clauses that follow the SELECT clause can appear in any order.
2. Each file name in the Data Division must be specified only once in the FILE-CONTROL paragraph. Each file name specified in the SELECT clause must have a file description entry in the Data Division of the same program. The name specified for *file-name-1* is the file connector name (see Section 7.2.3).
3. *Literal-1* must be a nonnumeric literal and must not be a figurative constant.
4. The first occurrence of *implementor-name-1* or *literal-1* is any of the following, depending on the external media:

   CARD-PUNCH
   CARD-READER
   DISC
   INTERCHANGE
   PRINTER
   TAPE
   ANS74-PRINTER

5. For files assigned to TAPE, DISC, or INTERCHANGE

   a. The second item in the list containing *implementor-name-1* or *literal-1* is used to link the file to the OS 2200 Exec file control. If *implementor-name-1* is given, it is a 1- to 12-character name and must appear on an @ASG or @USE statement in the runstream for executing the object program. See Section 4.5.3 for possible implicit @ASG actions automatically done in the absence of an @ASG statement.

If *literal-1* is given, the file name can include Exec file qualifier, file name, element name, and version name in the same format as would appear in the operating system control cards. File cycles, element cycles, and file read and write keys are not allowed. When this format of SELECT is used, an @USE name of *file-name-1* is attached to the qualifier and file name given in *literal-1.* Elements cannot be used for files assigned to TAPE.

Note: *The format of a specified element is a Processor Common Input/Output System (PCIOS) System Data Format (SDF) element, not a source-type element.*

b. When there is only one specification of implementor-name-1 or literal-1, the first 12 characters of the file name are used to link the file to the OS 2200 Executive File Control.

c. If the SELECT statement specifies a quoted name string for the implementor-name-1 field, it is processed as an external file name. In this case, the UCS Runtime System generates an internal name string of the form

```
NAME$$$$nnnn
```

where *nnnn* is a numeric value indicating a unique internal name ranging from 0001 to 9999.

d. This generated internal name is used by the UCS Runtime System for all file interfaces with the Operating System, including the default FILE ID value when performing file label processing.

6. For files assigned to CARD-PUNCH, CARD-READER, PRINTER, ANSI74-PRINTER

   a. The second item in the list containing *implementor-name-1* or *literal-1* specifies an alternate system input/output file. If *implementor-name-1* is given, it is a 1- to 12-character name. If *literal-1* is given, it can include an Exec file qualifier and file name. Elements cannot be used for an alternate input/output file.

   b. When there is only one specification of *implementor-name-1* or *literal-1, implementor-name-1* or *literal-1* is linked to the system input-output files associated with each run unit (see Section 7.2.3 and Section 7.2.3).

7. For files assigned to INTERCHANGE

   a. The file format is that described in American National Standard X3.27-1969 Magnetic Tape Labels for Information Interchange. (See Volume 2.)

   b. The file must be a sequential tape file.

8. The third and all subsequent reoccurrences of *implementor-name-1* or *literal-1* are for documentation only.

General Rules

1. If the file connector referred to by *file-name-1* is an external file connector (see Section 7.6.6), all file control entries in the run unit that refer to this file connector must have

   a. The same specification for the OPTIONAL phrase

b. A consistent specification for *implementor-name-1* or *literal-1* in the ASSIGN clause

c. The same value for *integer-1* in the RESERVE clause

d. The same organization

e. The same access mode

f. The same specification for the PADDING CHARACTER clause

2. The OPTIONAL phrase applies only to files opened in the input I-O, or extend mode. Its specification is required for files that are not necessarily present each time the object program is executed.
3. The ASSIGN clause specifies the association of the file referred to by *file-name-1* to a storage medium referred to by *implementor-name-1* or *literal-1*.
4. The ACCESS MODE clause, the CODE TYPE IS clause, the FILE STATUS clause, the ORGANIZATION IS SEQUENTIAL clause, the PADDING CHARACTER clause, the RECORD DELIMITER clause, and the RESERVE clause are presented in Section 4.3.2.

The ACCESS MODE Clause

Function

The ACCESS MODE clause specifies the order in which records are to be accessed in the file.

General Format

```
ACCESS MODE IS SEQUENTIAL
```

General Rules

1. If the ACCESS MODE clause is not specified, sequential access is assumed.
2. Records in the file are accessed in the sequence dictated by the file organization. For sequential files, this sequence is specified by predecessor-successor record relationships established by the execution of WRITE statements when the file is created or extended.
3. If the associated file connector is an external file connector, every file control entry in the run unit that is associated with that file connector must specify the same access mode.

The CODE TYPE Clause

Function

The CODE TYPE clause contains information pertaining to the Coded Character Set (CCS) of the data that is in the file.

General Format

```
CODE TYPE IS data-name-3
```

Syntax Rules

1. *Data-name-3* can be qualified.
2. *Data-name-3* must be defined in the Working-Storage Section as an unsigned numeric field described as PIC 99 USAGE BINARY. If *data-name-3* does not have a VALUE clause associated with it, it is given an initial value of 1 (which is the code type value for ASCII).
3. The CODE TYPE clause is allowable for files with the following implementor names specified in the SELECT clause.

CARD-PUNCH	PRINTER
CARD-READER	DISC
ANS74-PRINTER	A-SHARED-FILE

General Rules

1. *data-name-3* is used to contain the code type (CCS) of the data that is stored in the file that is being processed. A complete list of the defined code types can be found in the current version of the *I18NLIB Programming Reference Manual*. The value of the code type data name must be in the range of 0 to 63; however, the values 7, 29-32, and 48-63 are reserved for future use and might not be used at this time.
2. For nonsymbiont files (DISC, A-SHARED-FILE), *data-name-3* is used to contain the code type of the data that is stored in the file processed. For files that are opened for output in order to initially create the file, the value of *data-name-3* is stored in the code type field in the header block of the file. If the file is opened for input the value of the code type stored in the file is placed into *data-name-3*. The following table describes what happens for various types of OPEN statements for nonsymbiont files.

   Note: *A file is available if it is physically present and is recognized by the I-O control system.*

Phrase	File is Available	File is Unavailable
INPUT	Normal open, code type retrieved	Open is unsuccessful, no code type available
INPUT (optional file)	Normal open, code type retrieved	Normal open, first read causes at end condition, no code type available
I-O	Normal open, code type retrieved	Open is unsuccessful, no code type available
I-O (optional file)	Normal open, code type retrieved	Open causes the file to be created, code type set

Phrase	File is Available	File is Unavailable
OUTPUT	Normal open, file contains no records, code type set	Open causes the file to be created, code type set
EXTEND	Normal open, code type retrieved	Open unsuccessful, no code type available
EXTEND (optional file)	Normal open, code type retrieved	Open causes the file to be created, code type set

Note: *OPEN I-O and OPEN EXTEND of optional files is treated the same as OPEN OUTPUT (that is, data-name-3 must contain a valid code type value). If the file is not available, then the value in data-name-3 is stored in the code type field in the file header when the file is created. If the file is available, then data-name-3 is updated to contain the value of the code type field for the file that is being opened.*

For symbiont files (CARD-PUNCH, CARD-READER, PRINTER, ANS74-PRINTER), *data-name-3* is used to contain the code type of each record that is written to the file that is being processed. The code type information is stored with the record that is being written each time a WRITE statement is executed. When the symbiont file is opened for input, the code type information is placed into *data-name-3* each time a READ statement is executed indicating the code type of the record that is being read.

3. If the CODE TYPE clause is not present, the code type of the en_US.ASCII locale (01) is stored when a non-symbiont file is created (OPEN OUTPUT) or a symbiont file is written to (WRITE statement).

   If the CODE TYPE clause is not present, but the program is compiled with the I18N-IO keyword option, the code type of the currently active locale (value LC_ALL) is stored when a non-symbiont file is created (OPEN OUTPUT) or a symbiont file is written to (WRITE statement).

4. If the CODE TYPE clause is not present, the code type of the data in the file cannot be retrieved into the source program.

The FILE STATUS Clause

Function

The FILE STATUS clause specifies a data item that contains the status of an input-output operation.

General Format

```
FILE STATUS IS data-name-1
```

Syntax Rules

1. *Data-name-1* can be qualified.

2. *Data-name-1* must be defined in the Data Division as a 2-character data item of the category alphanumeric and must not be defined in the File Section or the Report Section.

General Rules

1. If the FILE STATUS clause is specified, the data item referenced by *data-name-1* is always updated to contain the value of the I-O status whenever the I-O status is updated. This value indicates the status of execution of the statement (see Section 4.2.5).
2. The data item referenced by *data-name-1* that is updated during the execution of an input-output statement is the one specified in the file control entry associated with that statement.

The ORGANIZATION IS SEQUENTIAL Clause

Function

The ORGANIZATION IS SEQUENTIAL clause specifies sequential organization as the logical structure of a file.

General Format

```
[ORGNIZATION IS] SEQUENTIAL
```

General Rules

1. The ORGANIZATION IS SEQUENTIAL clause specifies sequential organization as the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.
2. Sequential organization is a permanent logical file structure in which a record is identified by a predecessor-successor relationship established when the record is placed into the file.
3. When the ORGANIZATION clause is not specified, sequential organization is implied.

The PADDING CHARACTER Clause

Function

The PADDING CHARACTER clause specifies the character that is to be used for block padding on sequential files.

General Format



Syntax Rules

1. *Literal-1* must be a 1-character nonnumeric literal.
2. *Data-name-1* can be qualified.

3. *Data-name-1* must be defined in the Data Division as a 1-character data item of the category alphanumeric and must not be defined in the File Section or the Report Section.

General Rules

1. The PADDING CHARACTER clause specifies the character that is to be used for block padding on sequential files. During input operations, any portion of a block that exists beyond the last logical record and consists entirely of padding characters is bypassed.
2. During input operations, a logical record that consists solely of padding characters is ignored. During output operations, any portion of a block that exists beyond the last logical record is filled entirely with padding characters.
3. If the PADDING CHARACTER clause is not applicable to the device type to which the file is assigned, the creation or recognition of padding characters does not occur. The only applicable device type is INTERCHANGE.
4. *Literal-1* or the value of the data item referred to by *data-name-1,* at the time the OPEN statement that creates the file is executed, is used as the value of the padding character. The padding character is a fixed file attribute.
5. If the CODE-SET clause is specified for the file, conversion of the padding character specified by *literal-1* or the contents of *data-name-1* is established for the file when the file is opened.
6. If the PADDING CHARACTER clause is not specified, the value used for the padding character is the circumflex (^), as specified in American National Standard X3.27-1978, Magnetic Tape Labels and File Structure for Information Interchange.
7. If the associated file connector is an external file connector, all PADDING CHARACTER clauses in the run unit that are associated with that file connector must have the same specifications. If *data-name-1* is specified, it must reference an external data item.

The RECORD DELIMITER Clause

Function

The RECORD DELIMITER clause indicates the method of determining the length of a variable-length record on the external medium.

General Format

```
RECORD DELIMITER IS {STANDARD}
```

Syntax Rules

1. The RECORD DELIMITER clause can be specified only for variable-length records.
2. The STANDARD-1 phrase specifies that the external medium must be a magnetic tape file.

General Rules

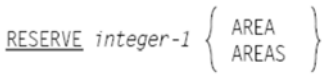
1. If the STANDARD-1 phrase is specified, the method used for determining the length of a variable-length record is that specified in American National Standard X3.27-1978, Magnetic Tape Labels and File Structures for Information Interchange, and International Standard 1001 1979, Magnetic Tape Labels and File Structure for Information Interchange.
2. If the RECORD DELIMITER clause is not specified, the compiler determines the method used to calculate the length of a variable-length record.
3. At the time of a successful execution of an OPEN statement, the record delimiter is the one specified in the RECORD DELIMITER clause in the file control entry associated with the file name specified in the OPEN statement.
4. If the associated file connector is an external file connector, all RECORD DELIMITER clauses in the run unit that are associated with that file connector must have the same specifications.

The RESERVE Clause

Function

The RESERVE clause allows the user to specify the number of input-output areas allocated.

General Format



General Rule

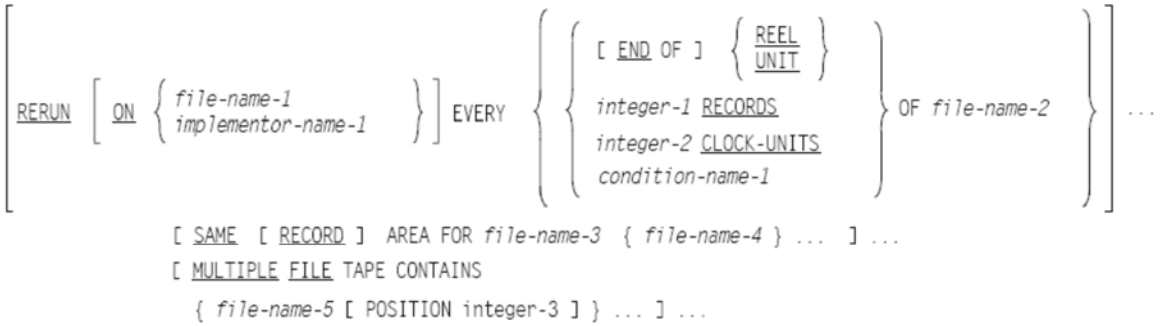
If the RESERVE clause is specified, it is for documentation only. One input-output area is always allocated for CARD-PUNCH, CARD-READER, PRINTER, and ANS74-PRINTER files. Two input-output areas are always allocated for DISC, TAPE, and RELATIVE DISC files.

4.3.3. The I-O-CONTROL Paragraph

Function

The I-O-CONTROL paragraph specifies the points at which rerun is to be established, the memory area that is to be shared by different files, and the location of files on a multiple file reel. The RERUN clause and the MULTIPLE FILE TAPE clause within the I-O-CONTROL paragraph are obsolete elements in this revision of American National Standard COBOL and they are to be deleted from the next revision of American National Standard COBOL.

General Format



Syntax Rule

The order of appearance of the clauses is immaterial.

General Rule

The MULTIPLE FILE TAPE clause, the RERUN clause, and the SAME AREA clause are presented in Section 4.3.3.

The MULTIPLE FILE TAPE Clause

Function

The MULTIPLE FILE TAPE clause specifies the location of files on a multiple file reel. The MULTIPLE FILE TAPE clause is an obsolete element in this revision of American National Standard COBOL and it is to be deleted in the next revision of American National Standard COBOL.

General Format

```
MULTIPLE FILE TAPE CONTAINS { file-name-1 [ POSITION integer-1] }...
```

Syntax Rules

1. Each file name specified in a MULTIPLE FILE TAPE clause must be specified in the FILE-CONTROL paragraph of the same program.
2. *File-name-1* cannot refer to an external file connector.
3. Each file name specified in a MULTIPLE FILE TAPE clause must have an associated file description entry.
4. The same file name must not be specified more than once in a MULTIPLE FILE TAPE clause.

General Rule

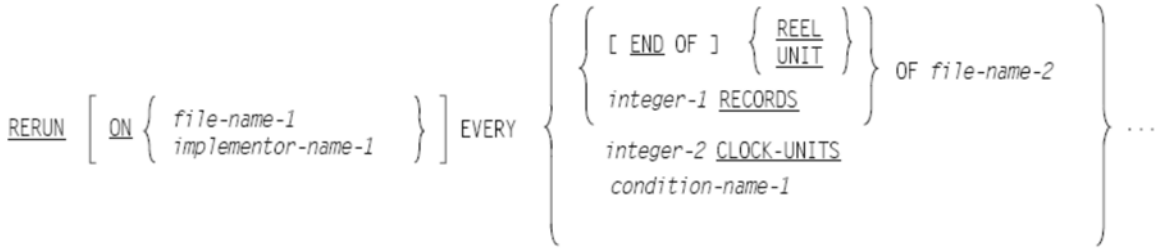
The MULTIPLE FILE TAPE clause is required when more than one file shares the same physical reel of tape. Regardless of the number of files on a single reel, only those files that are used in the object program need be specified. If all file names have been listed in consecutive order, the POSITION clause need not be given. If any file in the sequence is not listed, the position relative to the beginning of the tape must be given. Not more than one file on the same tape reel can be open at one time.

The RERUN Clause

Function

The RERUN clause specifies the points at which rerun is to be established. The RERUN clause is an obsolete element in this revision of American Nation Standard COBOL, and it is to be deleted from the next revision of American National Standard COBOL.

General Format



Syntax Rules

1. The allowable implementor names are TAPE and DISC.
2. When the *integer-1* RECORDS clause is specified, *implementor-name* must be given in the RERUN clause.
3. When multiple *integer-1* RECORDS clauses are specified, no two of them can specify the same *file-name-2*.
4. The ON *file-name-1* clause, the integer-2 CLOCK-UNITS clause, the END OF REEL/UNIT OF *file-name-2* clause, and the *condition-name* clause serve for documentation only.

General Rules

1. The RERUN clause specifies when and where the rerun information is recorded.
2. If *implementor-name* is specified, the rerun information is written as a separate file whose file name is RERUN$.
3. In UCS COBOL, rerun points can be established when the *integer-1* RECORDS clause is used. In this case, the rerun information is written on the file whose file name is RERUN$, which must be specified in the ON clause whenever *integer-1* records of *file-name-2* have been processed. *File-name-2* can be either an input or output file with any organization or access.

The SAME Clause

Function

The SAME clause specifies the storage area that is to be shared by different files.

General Format

```
SAME [RECORD] AREA FOR file-name-1 {file-name-2}...
```

Syntax Rules

1. Each file name must be specified in the FILE-CONTROL paragraph of the same program.
2. Each file name cannot refer to an external file connector.
3. More than one SAME clause can be included in the program, subject to the following restrictions:
   a. A file name must not appear in more than one SAME AREA clause.
   b. A file name must not appear in more than one SAME RECORD AREA clause.
   c. If one or more file names of a SAME AREA clause appear in a SAME RECORD AREA clause, all of the file names in that SAME AREA clause must appear in the SAME RECORD AREA clause. However, additional file names not appearing in that SAME AREA clause can also appear in that SAME RECORD AREA clause. The rule that only one of the files mentioned in a SAME AREA clause can be open at a time takes precedence over the rule that all files mentioned in a SAME RECORD AREA clause can be open at a time.
4. The files referred to in the SAME AREA or SAME RECORD AREA clause need not all have the same organization or access.

General Rules

1. The SAME AREA clause specifies that two or more files referred to by *file-name-1, file-name-2* that do not represent sort or merge files are to use the same storage area during processing. The area being shared includes all storage areas assigned to the files referred to by *file-name-1, file-name-2*; therefore, it is not valid to have more than one of these files in the open mode at the same time. (See syntax rule 3c.)
2. The SAME RECORD AREA clause specifies that two or more files referred to by *file-name-1, file-name-2* is to use the same storage area for processing of the current logical record. All of these files can be in the open mode at the same time. A logical record in the SAME RECORD AREA is considered as a logical record of each file open in the output mode and of the most recently read file open in the input mode when the file name appears in this SAME RECORD AREA clause. This is equivalent to an implicit redefinition of the area; that is, records are aligned on the leftmost character position.

4.4. Data Division/File Section

The File Section is located in the Data Division of a source program. The File Section defines the structure of data files. Each file is defined by a file description entry and one or more record description entries. Record description entries are written immediately following the file description entry.

4.4.1. General Format

```
FILE SECTION.
  [file-description-entry
  {record-description-entry} . . .  ] . . .
```

4.4.2. File Description Entry

In a COBOL program, the file description (FD) entry represents the highest level of organization in the File Section. The File Section header is followed by a file description entry consisting of a level indicator (FD), a file name, and a series of independent clauses. The clauses of a file description entry specify the size of the logical and physical records, the presence or absence of label records, the value of label items defined by Unisys, the names of the records that comprise the file, and finally the number of lines to be written on a logical printer page. The entry itself is terminated by a period.

4.4.3. Record Description Structure

A record description consists of a set of data description entries that describe the characteristics of a particular record. Each data description entry consists of a level number followed by the data name or FILLER clause, if specified, followed by a series of independent clauses, as required. A record description can have a hierarchical structure and, therefore, the clauses used with an entry can vary considerably, depending on whether or not it is followed by subordinate entries. The structure of a record description and the elements allowed in a record description entry are explained in 2.4.15 and 3.5. The availability of specific clauses in the data description entry depends on the level of Nucleus module supported.

4.4.4. Initial Values

The initial value of a data item in the File Section is not allowed.

4.4.5. The File Description - Complete Entry Skeleton

Function

The file description entry furnishes information concerning the physical structure, identification, and record names pertaining to a sequential file.

General Format



Syntax Rules

1. The level indicator FD identifies the beginning of a file description entry and must precede *file-name-1.*
2. The clauses that follow *file-name-1* can appear in any order.
3. One or more record description entries must follow the file description entry.

General Rules

1. A file description entry associates *file-name-1* with a file connector.
2. The BLOCK CONTAINS clause, the CODE-SET clause, the DENSITY clause, the DATA RECORDS clause, the LABEL RECORDS clause, the LINAGE clause, the RECORD clause, and the VALUE OF clause are presented in the following subsections.

The BLOCK CONTAINS Clause

Function

The BLOCK CONTAINS clause specifies the size of a physical record.

General Format

```
BLOCK CONTAINS [ integer-1 TO ] integer-2 { RECORDS }
                                          { CHARACTERS }
```

General Rules

1. This clause is required except when one or more of the following conditions exist:
   a. A physical record contains only one complete logical record.
   b. The hardware device assigned to the file has only one physical record size.
   c. The number of records contained in a block is specified in the operating environment.
2. The size of a physical record can be stated in terms of records unless one or more of the following situations exists, in which case the RECORDS phrase must not be used:
   a. In mass storage files, where logical records can extend across physical records.
   b. The physical record contains padding (area not contained in a logical record).
   c. Logical records are grouped in such a manner that an inaccurate physical record size would be implied.
3. If the CHARACTERS phrase is specified, the physical record size is specified in terms of the number of character positions required to store the physical record, regardless of the types of characters used to represent the items within the physical record.
4. If *integer-1* is not specified, *integer-2* represents the exact size of the physical record. If *integer-1* and *integer-2* are both specified, they refer to the minimum and maximum size of the physical record, respectively.
5. If the associated file connector is an external file connector, all BLOCK CONTAINS clauses in the run unit that are associated with that file connector must have the same value for *integer-1* and *integer-2.*

The CODE-SET Clause

Function

The CODE-SET clause specifies the character code convention used to represent data on the external media.

General Format

```
CODE-SET IS alphabet-name-1
```

Syntax Rules

1. If the CODE-SET clause is specified for a file, all data in that file must be described as USAGE IS DISPLAY and any signed numeric data must be described with the SIGN IS SEPARATE clause.
2. The *alphabet-name-1* clause referenced by the CODE-SET clause must not specify the literal phrase.

General Rules

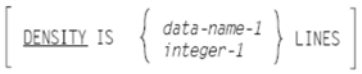
1. If the CODE-SET clause is specified:
   a. Upon successful execution of an OPEN statement, the character set used to represent the data on the external media is the one referenced by alphabet-name-1 in the file description entry associated with the file name specified in the OPEN statement (See Section 3.4.5).
   b. It specifies the algorithm for converting the character set on the external media from/to the native character set during the execution of an input or output operation.
2. If the CODE-SET clause is not specified, the native character set is assumed for data on the external media.
3. If the associated file connector is an external file connector, all CODE-SET clauses in the run unit that are associated with that file connector must have the same character set.

The DENSITY Clause

Function

The DENSITY clause specifies the number of lines per inch for printer output.

General Format



Syntax Rules

1. The DENSITY clause is meaningful only for printer files.
2. *Data-name-1* must refer to an elementary unsigned integer data item.
3. *Integer-1* or the data item referred to by *data-name-1* must be greater than zero.

General Rule

The number of lines per inch for printer output depends on the integer or data name specified in the DENSITY clause, subject to system constraints specified in Volume 2.

The DATA RECORDS Clause

Function

The DATA RECORDS clause serves only as documentation for the names of data records within their associated file. The DATA RECORDS clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted in the next revision of American National Standard COBOL.

General Format



Syntax Rule

Data-name-1 is the name of a data record and must have a 01 level number record description, with the same name, associated with it.

General Rules

1. More than one data name indicates that the file contains more than one type of data record. These records can be of differing sizes, different formats, and so forth. The order in which they are listed is not significant.
2. Conceptually, all data records within a file share the same area. This is in no way altered by the presence of more than one type of data record within the file.

The LABEL RECORDS Clause

Function

The LABEL RECORDS clause specifies whether labels are present. The LABEL RECORDS clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted from the next revision of American National Standard COBOL.

General Format



General Rules

1. OMITTED specifies that no explicit labels exist for the file or the device to which the file is assigned.
2. STANDARD specifies that labels exist for the file or the device to which the file is assigned and the labels conform to Unisys label specifications. (See Volume 2.)
3. If the LABEL RECORDS clause is not specified for a file, label records for that file must conform to the Executive System's label specifications. (See the ER Programming Reference Manual (UP-4144).)

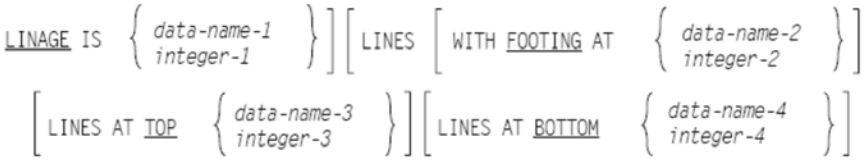
4. If the file connector associated with this file description entry is an external file connector (see Section 7.6.6), all LABEL RECORDS clauses in the run unit that are associated with this file connector must be the same.

The LINAGE Clause

Function

The LINAGE clause provides a means for specifying the depth of a logical page in terms of number of lines. It also provides for specifying the size of the top and bottom margins on the logical page and the line number, within the page body, at which the footing area begins.

Format 1



Format 2

```
LINAGE IS SYSTEM LINES
```

Syntax Rules

1. *Data-name-1, data-name-2, data-name-3,* and *data-name-4* must refer to elementary unsigned numeric integer data items.
2. *Data-name-1, data-name-2, data-name-3,* and *data-name-4* can be qualified.
3. *Integer-2* must not be greater than *integer-1.*
4. *Integer-3* and *integer-4* can be zero.

General Rules

1. The LINAGE clause provides a means for specifying the size of a logical page in terms of number of lines. The logical page size is the sum of the values referred to by each phrase except the FOOTING phrase. If the LINES AT TOP or LINES AT BOTTOM phrases are not specified, the values of these items are zero. If the FOOTING phrase is not specified, no end-of-page condition independent of the page overflow condition exists.

   Format 2 of the LINAGE clause specifies a logical page, which is equal to the system standard definition. Format 2 is allowed only with files assigned to PRINTER.

   There is no relationship between the size of the logical page and the size of a physical page.

2. *Integer-1* or the data item referred to by *data-name-1* specifies the number of lines that can be written and spaced on the logical page. The value must be greater than zero. That part of the logical page in which these lines can be written or spaced is called the page body.

3. *Integer-2* or the data item referred to by *data-name-2* specifies the line number within the page body at which the footing area begins. The value must be greater than zero and not greater than *integer-1* or the value of the data item referred to by *data-name-1*. The footing area comprises the area of the page body between the line represented by *integer-2* or the value of the data item referred to by *data-name-2* and the line represented by *integer-1* or the value of the data item referred to by *data-name-1*, inclusive.
4. *Integer-3* or the value of the data item referred to by *data-name-3* specifies the number of lines that comprise the top margin on the logical page. The value can be zero.
5. *Integer-4* or the value of the data item referred to by *data-name-4* specifies the number of lines that comprise the bottom margin on the logical page. The value can be zero.
6. *Integer-1*, *integer-3*, and *integer-4*, if specified, are used at the time the file is opened by the execution of an OPEN statement with the OUTPUT phrase to specify the number of lines that comprise each of the indicated sections of a logical page. *Integer-2*, if specified, is used at that time to define the footing area. These values are used for all logical pages written for that file during a given execution of the program.
7. The values of the data items referred to by *data-name-1, data-name-3,* and *data-name-4,* if specified, are used as follows:
    a. The values of the data items, at the time an OPEN statement with the OUTPUT phrase is executed for the file, are used to specify the number of lines that are to comprise each of the indicated sections for the first logical page.
    b. The values of the data items, at the time a WRITE statement with the ADVANCING PAGE phrase is executed or a page overflow condition occurs, are used to specify the number of lines that are to comprise each of the indicated sections for the next logical page. (See Section B.5.43.)
8. The value of the data item referred to by *data-name-2,* if specified, at the time an OPEN statement with the OUTPUT phrase is executed for the file, is used to define the footing area for the first logical page. At the time a WRITE statement with the ADVANCING PAGE phrase is executed or a page overflow condition occurs, it is used to define the footing area for the next logical page.
9. A LINAGE-COUNTER is generated by the presence of a LINAGE clause. The value in the LINAGE-COUNTER at any given time represents the line number at which the device is positioned within the current page body. The rules governing the LINAGE-COUNTER are:
    a. A separate LINAGE-COUNTER is supplied for each file described in the File Section whose file description entry contains a LINAGE clause.

b. LINAGE-COUNTER can be referred to only in Procedure Division statements; however, only the input-output control system can change the value of LINAGE-COUNTER. Since more than one LINAGE-COUNTER can exist in a program, the user must qualify LINAGE-COUNTER by file name when necessary.

c. LINAGE-COUNTER is automatically modified according to the following rules during the execution of a WRITE statement to an associated file:

- When the ADVANCING PAGE phrase of the WRITE statement is specified, LINAGE-COUNTER is automatically reset to one. During the resetting of LINAGE-COUNTER to the value one, the value of LINAGE-COUNTER is implicitly incremented to exceed the value specified by *integer-1* or the data item referred to by *data-name-1.*
- When the ADVANCING *identifier-2* or *integer-1* phrase of the WRITE statement is specified, LINAGE-COUNTER is incremented by *integer-1* or the value of the data item referred to by *identifier-2.*
- When the ADVANCING phrase of the WRITE statement is not specified, LINAGE-COUNTER is increased by the value one. (See Section B.5.43.)
- The value of LINAGE-COUNTER is automatically reset to one when the device is repositioned to the first line that can be written on for each of the succeeding logical pages. (See Section B.5.43.)
- The value of LINAGE-COUNTER is automatically reset to one at the time an OPEN statement with the OUTPUT phrase is executed for the associated file.

10. Each logical page is contiguous to the next with no additional spacing provided.
11. If the file connector associated with this file description entry is an external file connector, all file description entries in the run unit that are associated with this file connector must have
    a. A LINAGE clause, if any file description entry has a LINAGE clause.
    b. The same corresponding values for *integer-1, integer-2, integer-3*, and *integer-4,* if specified.
    c. The same corresponding external data items referred to by *data-name-1, data-name-2, data-name-3,* and *data-name-4.*

The RECORD Clause

Function

The RECORD clause specifies the number of character positions in a fixed-length record, or specifies the range of character positions in a variable-length record. If the number of character positions does vary, the clause specifies the minimum and maximum number of character positions.

Format 1

```
RECORD CONTAINS integer-1 CHARACTERS
```

Format 2

```
RECORD IS VARYING IN SIZE [ [FROM integer-2] [ TO integer-3] CHARACTERS ]
    [DEPENDING ON data-name-1]
```

Format 3

```
RECORD CONTAINS integer-4 TO integer-5 CHARACTERS
```

Syntax Rules

The following is a list of the syntax rules.

Format 1

The following syntax rule applies to format 1:

No record description entry for the file can specify a number of character positions greater than *integer-1.*

Format 2

The following syntax rule applies to format 2:

1. Record descriptions for the file must not describe records that contain a lesser number of character positions than that specified by *integer-2,* nor records that contain a greater number of character positions than that specified by *integer-3.*
2. *Integer-3* must be greater than *integer-2.*
3. *Data-name-1* must describe an elementary unsigned integer in the Working-Storage or Linkage Section.

General Rules

The following is a list of general rules.

All Formats

The following general rules apply to all formats:

1. If the RECORD clause is not specified, the size of each data record is completely defined in the record description entry.
2. If the associated file connector is an external file connector, all file description entries in the run unit that are associated with that file connector must specify the same values for *integer-1* or *integer-2* and *integer-3.* If the RECORD clause is not specified, all record description entries associated with this file connector must be the same length.

Format 1

The following general rule applies to format 1:

Format 1 is used to specify fixed-length records. *Integer-1* specifies the number of character positions contained in each record in the file.

Format 2

The following general rules apply to format 2:

1. Format 2 is used to specify variable-length records. *Integer-2* specifies the minimum number of character positions to be contained in any record of the file. *Integer-3* specifies the maximum number of character positions in any record of the file.
2. The number of character positions associated with a record description is determined by the sum of the number of character positions in all elementary data items excluding redefinitions and renaming, plus any implicit FILLER due to synchronization. If a table is specified
    a. The minimum number of table elements described in the record is used in the previous summation to determine the minimum number of character positions associated with the record description.
    b. The maximum number of table elements described in the record is used in the previous summation to determine the maximum number of character positions associated with the record description.
3. If *integer-2* is not specified, the minimum number of character positions to be contained in any record of the file is equal to the least number of character positions described for a record in that file.
4. If *integer-3* is not specified, the maximum number of character positions to be contained in any record of the file is equal to the greatest number of character positions described for a record in that file.
5. If *data-name-1* is specified, the number of character positions in the record must be placed into the data item referred to by *data-name-1* before any RELEASE, REWRITE, or WRITE statement is executed for the file.
6. If *data-name-1* is specified, the execution of a DELETE, RELEASE, REWRITE, START, or WRITE statement or the unsuccessful execution of a READ or RETURN statement does not alter the contents of the data item referred to by *data-name-1*.
7. During the execution of a RELEASE, REWRITE, or WRITE statement, the number of character positions in the record is determined by the following conditions:
    a. If *data-name-1* is specified, by the content of the data item referred to by *data-name-1*.
    b. If *data-name-1* is not specified and the record does not contain a variable-occurrence data item, by the number of character positions in the record.

c. If *data-name-1* is not specified and the record does contain a variable-occurrence data item, by the sum of the fixed portion and that portion of the table described by the number of occurrences at the time of execution of the output statement.

8. If *data-name-1* is specified, after the successful execution of a READ or RETURN statement for the file, the contents of the data item referred to by *data-name-1* indicate the number of character positions in the record just read.
9. If the INTO phrase is specified in the READ or RETURN statement, the number of character positions in the current record that participate as the sending data items in the implicit MOVE statement is determined by the following conditions:

   a. If *data-name-1* is specified, by the content of the data item referred to by *data-name-1.*

   b. If *data-name-1* is not specified, by the value that would have been moved into the data item referred to by *data-name-1* had *data-name-1* been specified.

Format 3

The following general rules apply to format 3:

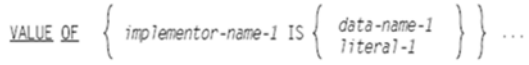
1. When format 3 of the RECORD clause is used, *integer-4* and *integer-5* refer to the minimum number of characters in the smallest size data record and the maximum number of characters in the largest size data record, respectively. However, in this case, the size of each data record is completely defined in the record description entry.
2. The size of each data record is specified in terms of the number of character positions required to store the logical record, regardless of the types of characters used to represent the items within the logical record. The size of a record is determined by the sum of the number of characters in all fixed-length elementary items plus the sum of the maximum number of characters in any variable-length item subordinate to the record. This sum can be different from the actual size of the record. (See Section 3.5.15 and Section 3.5.16.)

The VALUE OF Clause

Function

The VALUE OF clause defines the description of an item in the label records associated with a file. The VALUE OF clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted in the next revision of American National Standard COBOL.

General Format



Syntax Rules

1. The allowable names for *implementor-name-1* are:

For CARD-PUNCH Files	For PRINTER Files	For TAPE Files	For DISC Files
FORMS-REQUEST	CARTRIDGE-ID FORMS-REQUEST	FILE-ACCESS FILE-ID PURGE-DATE	FILE-ID

These implementor names have the following implied definitions:

Forms-Request	**Pic X(24).**
CARTRIDGE ID	PICXX.
FILE ID	PICX(17).
FILE-ACCESS	PICX.
PURGE-DATE	PIC 9(6).

2. *Data-name-1* must be qualified when necessary, but it can neither be subscripted nor be an item described with the USAGE IS INDEX clause.
3. *Data-name-1* must be in the Working-Storage Section.
4. *Data-name-1* must be described as USAGE IS DISPLAY.

General Rules

1. For an input file, the appropriate label routine checks to see if the value of *implementor-name-1* is equal to *literal-1* or the content of the data item referred to by *data-name-1,* whichever has been specified.

   For an output file, at the appropriate time the value of *implementor-name-1* is made equal to *literal-1* or the content of the data item referred to by *data-name-1,* whichever has been specified.
2. The contents of the literal or the data item referred to by the data name specified in the VALUE OF CARTRIDGE-ID clause are used to specify the cartridge to be used for this print file. These contents must contain the cartridge ID code that appears on the cartridge case.
3. The contents of the literal or the data item referred to by the data name specified in the VALUE OF FORMS-REQUEST clause are used to specify the special form message for this card punch or printer file. These contents must contain the form type, name, or number required by this file at execution time.

4.5. Procedure Division

This subsection describes the Procedure Division and its associated statements.

4.5.1. General Description

The Procedure Division contains declarative procedures when the USE statement from the Sequential I-O module is present in a COBOL source program. Shown below is the general format of the Procedure Division when the USE statement is present.

```
PROCEDURE DIVISION.

DECLARATIVES.

{section-name SECTION.

     USE statement.

[paragraph-name.

   [sentence]...]...}...

END DECLARATIVES.

{section-name SECTION.

[paragraph-name.

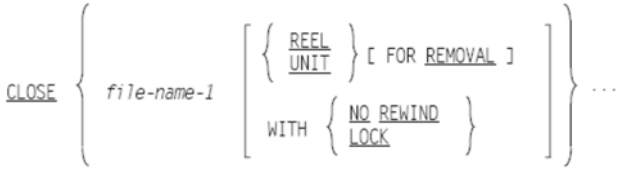
   [sentence]...]...}...
```

4.5.2. The CLOSE Statement

Function

The CLOSE statement terminates the processing of reels/units and files with optional rewind, and/or lock, or removal options where applicable.

General Format



Syntax Rule

The files referred to in the CLOSE statement need not all have the same organization or access.

General Rules

Except where otherwise stated in the following general rules, the terms "reel" and "unit" are synonymous and completely interchangeable in the CLOSE statement. Treatment of sequential mass storage files is logically equivalent to the treatment of a file on tape or analogous sequential media.

Treatment of a file contained in a multiple file tape environment is logically equivalent to the treatment of a sequential single-reel/unit file if the file is wholly contained on one reel, or to the treatment of a sequential multireel/unit file if the file is contained on more than one reel.

1. The REEL or UNIT phrase must be used only for files assigned to tape.
2. A CLOSE statement can be executed only for a file in an open mode.
3. To show the effect of various types of CLOSE statements as applied to various storage media, all files are divided into the following categories:
   a. Nonreel/unit: A file whose input or output medium is such that the concepts of rewind and reels/units have no meaning. This includes all symbiont and mass storage files.
   b. Sequential single-reel/unit: A sequential file that is entirely contained on one reel/unit. This includes only tape files.
   c. Sequential multireel/unit: A sequential file that is contained on more than one reel or unit. This includes only tape files.
4. The results of executing each type of CLOSE statement for each category of file are summarized in Table 4–1.

Table 4–1. Relationship of Categories of Files and the Formats of the CLOSE Statement

CLOSE statement Format	File Category		
	Nonreel/Unit	Sequential Single-Reel/Unit	Sequential Multi-Reel/Unit
CLOSE	C	C,G	A,C,G
CLOSE WITH LOCK	C,E	C,E,G	A,C,E,G
CLOSE WITH NO REWIND	C,H	B,C	A,B,C
CLOSE REEL/UNIT	F	F,G	F,G
CLOSE REEL/UNIT FOR REMOVAL	F	D,F,G	D,F,G n

The definitions of the symbols in the table follow. Where the definition depends on whether the file is an input, output, or input-output file, alternate definitions are given; otherwise, a definition applies to input, output, and input-output files.

A

Effect on previous reels/units

- Input files and input-output files

  All reels/units in the file before the current reel/unit are closed except those reels/units controlled by a previous CLOSE REEL/UNIT statement. If the current reel/unit is not the last in the file, the reels/units in the file following the current one are not processed.

- Output files

  All reels/units in the file before the current reel/unit are closed except those reels/units controlled by a previous CLOSE REEL/UNIT statement.

B

No rewind of current reel

The current reel/unit is left in its current position.

C

Close file

- Input files and input-output files

  If the file is positioned at its end and label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If the file is positioned at its end and label records are not specified for the file, label processing does not take place but other closing operations are executed. If the file is positioned other than at its end, the closing operations are executed, but there is no ending label processing.

- Output Files

  If label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If label records are not specified for the file, label processing does not take place but other closing operations are executed.

D

Reel/unit removal

The current reel or unit is rewound, when applicable, and the reel or unit is logically removed from the run unit; however, the reel or unit can be accessed again, in its proper order of reels or units within the file, if a CLOSE statement without the REEL or UNIT phrase is subsequently executed for this file followed by the execution of an OPEN statement for the file.

E

File lock

The file is locked and cannot be opened again during this execution of this run unit.

F

Close reel/unit

- Input files and input-output files
- If the current reel/unit is the last or only reel/unit for the file or the reel is on a nonreel/unit medium, there is no reel/unit swap and the current volume pointer remains unchanged.

  If another reel/unit exists for the file, a reel/unit swap occurs, the current volume pointer is updated to point to the next reel/unit existing in the file, and the standard beginning reel/unit label procedure is executed. If no data records exist for the current volume, another reel/unit swap occurs.
- Output files (reel/unit media)

  The following operations take place:

  – The standard ending reel/unit label procedure is executed.

  – A reel/unit swap. The current volume pointer is updated to point to the new reel/unit.

  – The standard beginning reel/unit label procedure is executed.

  – The next executed WRITE statement that refers to that file directs the next logical data record to the next reel/unit of the file.

  – Output files (nonreel/unit media)

  – Execution of this statement is considered successful. The file remains in the open mode, and no action takes place except as specified in general rule 4.

G

Rewind

The current reel or analogous device is positioned at its physical beginning.

H

Optional phrases ignored

The CLOSE statement is executed as if none of the optional phrases is present.

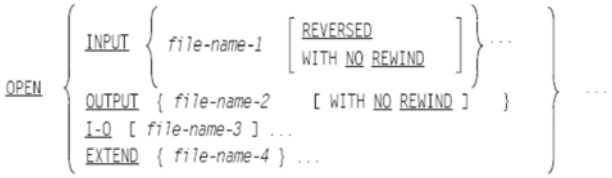
5. The execution of the CLOSE statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 4.2.5.)
6. If an optional input file is not present, no end-of-file or reel/unit processing is performed for the file and the file position indicator and the current volume pointer are unchanged.
7. Following the successful execution of a CLOSE statement without the REEL or UNIT phrase, the record area associated with *file-name-1* is no longer available. The unsuccessful execution of such a CLOSE statement leaves the availability of the record area undefined.
8. Following the successful execution of a CLOSE statement without the REEL or UNIT phrase, the file is removed from the open mode, and the file is no longer associated with the file connector.
9. If more than one *file-name-1* is specified in a CLOSE statement, the result of executing this CLOSE statement is the same as if a separate CLOSE statement had been written for each *file-name-1* in the same order as specified in the CLOSE statement.

4.5.3. The OPEN Statement

Function

The OPEN statement initiates the processing of files. The REVERSED phrase is an obsolete element in Standard COBOL because it is to be deleted from the next revision of Standard COBOL.

General Format



Syntax Rules

1. The REVERSED phrase can be used only with sequential files.
2. The EXTEND phrase must not be specified for a multiple file reel. (See Section 4.5.2.).
3. The EXTEND phrase must only be used for files for which the LINAGE clause has not been specified.
4. The files referred to in the OPEN statement need not all have the same organization or access.

General Rules

1. The successful execution of an OPEN statement determines the availability of the file, and results in the file being in an open mode. The successful execution of an OPEN statement associates the file with the file name through the file connector. A file is available if it is physically present and is recognized by the I-O control system. Table 4–2 shows the results of opening available and unavailable files.

Table 4–2. Availability of a File

Phrase	File is Available	File is Unavailable
INPUT	Normal open	Open is unsuccessful
INPUT (optional file)	Normal open	Normal open; the first read causes the at-end condition.
I-O	Normal open	Open is unsuccessful
I-O (optional file)	Normal open	Open causes the file to be created
OUTPUT	Normal open; the file contains no records	Open causes the file to be created
EXTEND	Normal open	Open is unsuccessful
EXTEND (optional file)	Normal open	Open causes the file to be created
OUTPUT	Normal open; the file contains no records	Open causes the file to be created

2. The successful execution of an OPEN statement makes the associated record area available to the program. If the file connector associated with file name is an external file connector, there is only one record area associated with the file connector for the run unit.
3. When a file is not in an open mode, no statement can be executed that refers to the file, either explicitly or implicitly, except for a MERGE statement with the USING or GIVING phrase, an OPEN statement, or a SORT statement with the USING or GIVING phrase.
4. An OPEN statement must be successfully executed before the execution of any of the permissible input-output statements. An X at an intersection indicates that the specified statement can be used with the open mode given at the top of the column.

Table 4–3. Permissible Statements

Statement	Open Mode			
	Input	Output	Input-Output	Extend
READ	X		X	
WRITE		X		X
REWRITE			X	

5. A file can be opened with the INPUT, OUTPUT, EXTEND, and I-O phrases in the same run unit. Following the initial execution of an OPEN statement for a file, each subsequent OPEN statement execution for that same file must be preceded by the execution of a CLOSE statement, without the REEL, UNIT, or LOCK phrase, for that file.
6. Execution of the OPEN statement does not obtain or release the first data record.
7. If label records are specified for the file, the beginning labels are processed as follows:
    a. When the INPUT phrase is specified, the execution of the OPEN statement causes the labels to be checked in accordance with the conventions for input label checking.
    b. When the OUTPUT phrase is specified, the execution of the OPEN statement causes the labels to be written in accordance with the conventions for output label writing.

    The behavior of the OPEN statement when label records are specified but not present, or when label records are not specified but are present, is undefined.
8. If, during the execution of an OPEN statement, a file attribute conflict condition occurs, the execution of the OPEN statement is unsuccessful. (See Section 4.3.3.)
9. The NO REWIND and REVERSED phrases must be used only with
    a. Sequential single-reel/unit files. (See Section 4.5.2.)
    b. Sequential files wholly contained within a single reel of tape within a multiple file tape environment. (See Section 4.3.3.)
10. The NO REWIND and REVERSED phrases is ignored if they do not apply to the storage media on which the file resides.
11. If the storage medium for the file permits rewinding, the following rules apply:
    a. When the REVERSED, the EXTEND, or the NO REWIND phrase are not specified, execution of the OPEN statement positions the file at its beginning.
    b. When the NO REWIND phrase is specified, execution of the OPEN statement does not reposition the file; the file must be already positioned at its beginning before execution of the OPEN statement.
    c. When the REVERSED phrase is specified, the file is positioned at its end by execution of the OPEN statement.
12. When the REVERSED phrase is specified, the subsequent READ statements for the file make the data records of the file available in reversed order; that is, starting with the last record.
13. If a file opened with the INPUT phrase is an optional file that is not present, the OPEN statement sets the file position indicator to indicate that an optional file is not present.
14. When files are opened with the INPUT or I-O phrase, the file position indicator is set to 1.
15. When the EXTEND phrase is specified, the OPEN statement positions the file immediately after the last logical record of that file. The last logical record of a sequential file is the last record written in the file.

16. When the EXTEND phrase is specified and the LABEL RECORDS clause indicates label records are present, the execution of the OPEN statement includes the following steps:
    a. The beginning file labels are processed only in the case of a single reel/unit file.
    b. The beginning reel/unit labels on the last existing reel/unit are processed as though the file were being opened with the INPUT phrase.
    c. The existing ending file labels are processed as though the file were being opened with the INPUT phrase. These labels are then deleted.
    d. Processing then proceeds as though the file had been opened with the OUTPUT phrase.
17. The OPEN statement with the I-O phrase must refer to a file that supports the input and output operations that are permitted for a sequential file when opened in the I-O mode. The execution of the OPEN statement with the I-O phrase places the referenced file in the open mode for both input and output operations.
18. When the I-O phrase is specified and the LABEL RECORDS clause indicates label records are present, the execution of the OPEN statement includes the following steps:
    a. The labels are checked in accordance with the conventions for input-output label checking.
    b. The new labels are written in accordance with the conventions for input-output label writing.
19. Treatment of a file contained in a multiple file tape environment is logically equivalent to the treatment of a sequential file contained in a single-file tape environment.
20. Whenever a set of files resides on a multiple file reel and one of this set of files is referred to in an OPEN statement, the following rules apply:
    a. Not more than one of the files can be in the open mode at one time.
    b. There is no constraint on the order in which files can be opened in the input mode.
    c. When one of the files referred to by a file name is the subject of an OPEN statement with the OUTPUT phrase, all files on the associated multiple file reel whose position numbers are less than the position number of that file must already exist on the reel at the time the OPEN statement is executed. Furthermore, no file on that multiple file reel whose position number is greater than the position number of that file can exist at the time on the reel.
    d. Each of the files must be a sequential file.
21. For an optional file that is unavailable, the successful execution of an OPEN statement with an EXTEND or I-O phrase creates the file. This creation takes place as if the following statements were executed in the order shown:

```
OPEN OUTPUT file-name.
CLOSE file-name.
```

These statements are followed by execution of the OPEN statement specified in the source program.

The successful execution of an OPEN statement with the OUTPUT phrase creates the file. After the successful creation of a file, that file contains no data records.

22. Upon successful execution of the OPEN statement, the current volume pointer is set
    a. To point to the first or only reel/unit for an available input or input-output file
    b. To point to the reel/unit containing the last logical record for an extend file
    c. To point to the new reel/unit for an unavailable output, I-O, or extend file
23. The execution of the OPEN statement updates the value of the I-O status associated with file name. (See Section 4.2.5)
24. If more than one file name is specified in an OPEN statement, the result of executing this OPEN statement is the same as if a separate OPEN statement had been written for each file name in the same order as specified in the OPEN statement.
25. The minimum and maximum record size for a file is established at the time the file is created and must not subsequently be changed.
26. When the DENSITY clause is present for other than a report file, the execution of the OPEN statement is set the density specified.
27. When the VALUE OF CARTRIDGE-ID clause is present for other than a report file, the execution of the OPEN statement is request the cartridge specified. (See Volume 2.)
28. When the VALUE OF FORMS-REQUEST clause is present for other than a report file, the execution of the OPEN statement requests the form specified. (See Volume 2.)
29. When the LINAGE and REPORT clauses are not present for a file assigned to PRINTER, execution of the OPEN statement causes the page margins to be set from the system standard margins (see the *ER Programming Reference Manual*).
30. When the LINAGE and REPORT clauses are not present for a file assigned to ANS74-PRINTER, execution of the OPEN statement causes the page to be defined as system standard lines with no margins (see the *ER Programming Reference Manual*).
31. For nonreport printer files, execution of the OPEN statement positions the printer to the first printable line on the first page.
32. For programs that are reading or writing program file elements, I-O and EXTEND mode are not available.
33. For programs that are writing program file elements, only one element can be open at a time for output within any given operating system external file.

4.5.4. The READ Statement

Function

The READ statement makes available the next logical record from a file.

General Format

```
READ file-name-1   [ NEXT ]   RECORD  [ INTO identifier-1 ]

[ AT END imperative-statement-1 ]
[ NOT AT END imperative-statement-2 ]
[ END-READ ]
```

Syntax Rules

1. The storage area associated with *identifier-1* and the record area associated with *file-name-1* must not be the same storage area.
2. The AT END phrase must be specified if no applicable USE AFTER STANDARD EXCEPTION procedure is specified for *file-name-1.*

General Rules

1. The file referred to by *file-name-1* must be open in the input or I-O mode at the time this statement is executed. (See Section 4.5.3.)
2. The NEXT phrase is optional and has no effect on the execution of the READ statement.
3. The execution of the READ statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 4.2.5)
4. The setting of the file position indicator at the start of the execution of a READ statement is used in determining the record to be made available according to the following rules. Comparisons for records in sequential files relate to the record number.
   a. If the file position indicator shows that no valid next record has been established, execution of the READ statement is unsuccessful.
   b. If the file position indicator shows that an optional file is not present, execution proceeds as specified in general rule 10.
   c. If the file position indicator was established by a previous OPEN statement, the first existing record in the file whose record number is greater than or equal to the file position indicator is selected.
   d. If the file position indicator was established by a previous READ statement, the first existing record in the file whose record number is greater than the file position indicator is selected.

If a record is found that satisfies the preceding rules, it is made available in the record area associated with *file-name-1*.

If no record is found that satisfies the preceding rules, the file position indicator is set to indicate that no next logical record exists and execution proceeds as specified in general rule 10.

If a record is made available, the file position indicator is set to the record number of the record made available.

5. Regardless of the method used to overlap access time with processing time, the concept of the READ statement is unchanged; a record is available to the object program before the execution of *imperative-statement-2,* if specified, or prior to the execution of any statement following the READ statement, if *imperative-statement-2* is not specified.
6. When the logical records of a file are described with more than one record description, these records automatically share the same record area in storage; this is equivalent to an implicit redefinition of the area. The contents of any data items that lie beyond the range of the current data record are undefined at the completion of the execution of the READ statement.
7. The INTO phrase can be specified in a READ statement if
   a. Only one record description is subordinate to the file description entry
   b. All record names associated with *file-name-1* and the data item referred to by *identifier-1* describe a group item or an elementary alphanumeric item.
8. The result of the execution of a READ statement with the INTO phrase is equivalent to the application of the following rules in the order specified:
   a. The execution of the same READ statement without the INTO phrase.
   b. The current record is moved from the record area to the area specified by *identifier-1* according to the rules for the MOVE statement without the CORRESPONDING phrase. The size of the current record is determined by rules specified in the RECORD clause. If the file description entry contains a RECORD VARYING clause, the implied move is a group move. The implied MOVE statement does not occur if the execution of the READ statement was unsuccessful. Any subscripting associated with *identifier-1* is evaluated after the record has been read and immediately before it is moved to the data item. The record is available in both the record area and the data item referred to by *identifier-1.*
9. If, during the execution of a READ statement, the end of a reel/unit is recognized or a reel/unit contains no logical records and the logical end of the file has not been reached, the following operations are executed:
   a. The standard ending reel/unit label procedure
   b. A reel/unit swap. The current volume pointer is updated to point to the next reel/unit existing for the file
   c. The standard beginning reel/unit label procedure

10. If the file position indicator indicates that no next logical record exists or that an optional input file is not present, the following occurs in the order specified:

    a. A value, derived from the setting of the file position indicator, is placed into the I-O status associated with *file-name-1* to indicate the at-end condition. (See Section 4.2.5.)

    b. If the AT END phrase is specified in the statement causing the condition, control is transferred to *imperative-statement-1* in the AT END phrase. Any USE AFTER STANDARD EXCEPTION procedure associated with *file-name-1* is not executed.

    c. If the AT END phrase is not specified, a USE AFTER STANDARD EXCEPTION procedure must be associated with this *file-name-1*, and that procedure is executed. Return from that procedure is to the next executable statement following the end of the READ statement.

    When the at-end condition occurs, execution of the READ statement is unsuccessful.

11. If an at-end condition does not occur during the execution of a READ statement, the AT END phrase is ignored, if specified, and the following actions occur:

    a. The file position indicator is set and the I-O status associated with *file-name-1* is updated.

    b. If an exception condition exists that is not an at-end condition, control is transferred according to rules of the USE statement following the execution of any USE AFTER EXCEPTION procedure applicable to *file-name-1*. (See Section 4.5.6.)

    c. If no exception condition exists, the record is made available in the record area and any implicit move resulting from the presence of an INTO phrase is executed. Control is transferred to the end of the READ statement or to *imperative-statement-2*, if specified. In the latter case, execution continues according to the rules for each statement specified in *imperative-statement-2*. If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of *imperative-statement-2,* control is transferred to the end of the READ statement.

12. Following the unsuccessful execution of a READ statement, the content of the associated record area is undefined and the file position indicator is set to indicate that no valid next record has been established.

13. If the number of character positions in the record that is read is less than the minimum size specified by the record description entries for *file-name-1,* the portion of the record area that is to the right of the last valid character read is undefined. If the number of character positions in the record that is read is greater than the maximum size specified by the record description entries for *file-name-1,* the record is truncated on the right to the maximum size. In either of these cases, the READ statement is successful, and an I-O status is set indicating a record length conflict has occurred. (See Section 4.2.5.)

14. The END-READ phrase delimits the scope of the READ statement. (See Section 2.4.21)

4.5.5. The REWRITE Statement

Function

The REWRITE statement logically replaces a record existing in a mass storage file.

General Format

```
REWRITE record-name-1  [ FROM identifier-1 ] [ END-REWRITE ]
```

Syntax Rules

1. If *identifier-1* is a function identifier, it must reference an alphanumeric function. When *identifier-1* is not a function identifier, *record-name-1* and *identifier-1* must not reference the same storage area.
2. *Record-name-1* is the name of a logical record in the File Section of the Data Division, and can be qualified.

General Rules

1. The file referred to by the file name associated with *record-name-1* must be a mass storage file and must be open in the I-O mode at the time of execution of this statement. (See Section 4.5.3.)
2. The last input-output statement executed for the associated file before the execution of the REWRITE statement must have been a successfully executed READ statement. The Mass Storage Control System (MSCS) logically replaces the record that was accessed by the READ statement.
3. The logical record released by a successful execution of the REWRITE statement is no longer available in the record area unless the file name associated with *record-name-1* is specified in a SAME RECORD AREA clause. The logical record is also available to the program as a record of other files referenced in the SAME RECORD AREA clause as the associated output file, as well as the file associated with *record-name-1.*
4. The result of the execution of a REWRITE statement with the FROM phrase is equivalent to the execution of the following statements in the order specified:
   a. The statement

      ```
      MOVE identifier-1 TO record-name-1
      ```

      according to the rules specified for the MOVE statement
   b. The same REWRITE statement without the FROM phrase
5. After the execution of the REWRITE statement is complete, the information in the area referred to by *identifier-1* is available, even though the information in the area referred to by *record-name-1* is not available except as specified by the SAME RECORD AREA clause.
6. The file position indicator is not affected by the execution of a REWRITE statement.
7. The execution of the REWRITE statement updates the value of the I-O status of the file name associated with *record-name-1.* (See Section 4.2.5)

8. The execution of the REWRITE statement releases a logical record to the operating system.
9. If the number of character positions specified in the record referred to by *record-name-1* is not equal to the number of character positions in the record being replaced, the execution of the REWRITE statement is unsuccessful, the updating operation does not take place, the content of the record area is unaffected, and the I-O status of the file associated with *record-name-1* is set to a value indicating the cause of the condition. (See Section 4.2.5.)
10. The END-REWRITE phrase delimits the scope of the REWRITE statement. (See Section 2.4.21)

4.5.6. The USE Statement

Function

The USE statement specifies procedures for input-output error handling that are in addition to the standard procedures provided by the input-output control system.

General Format



Syntax Rules

1. A USE statement, when present, must immediately follow a section header in the declaratives portion of the Procedure Division and must appear in a sentence by itself. The remainder of the section must consist of zero, one, or more procedural paragraphs that define the procedures to be used.
2. The USE statement is never executed; it merely defines the conditions that call for the execution of the USE procedures.
3. The appearance of *file-name-1* in a USE statement must not cause the simultaneous request for execution of more than one USE procedure.
4. The words ERROR and EXCEPTION are synonymous and can be used interchangeably.
5. The files implicitly or explicitly referred to in the USE statement need not all have the same organization or access.
6. The INPUT, OUTPUT, I-O, and EXTEND phrases can each be specified only once in the declaratives portion of a given Procedure Division.

General Rules

1. Declarative procedures can be included in any COBOL source program whether the program contains or is contained within another program. A declarative is invoked when any of the conditions described in the USE statement that prefaces the declarative occurs while the program is being executed.
2. Within a declarative procedure, there must be no reference to any nondeclarative procedures. Only a declarative within the separately compiled program that contains the statement that caused the qualifying condition is invoked when any of the conditions described in the USE statement that prefaces the declaratives occurs while that separately compiled program is being executed. If no clarifying declarative exists in the separately compiled program, no declarative is executed.
3. Procedure names associated with a USE statement can be referred to in a different declarative section or in a nondeclarative procedure only with a PERFORM statement.
4. When *file-name-1* is specified explicitly, no other USE statement applies to *file-name-1.*
5. The procedures associated with a USE statement are executed by the input-output control system after completion of the standard input-output exception routine upon the unsuccessful execution of an input-output operation unless an AT END phrase takes precedence. The rules concerning when the procedures are executed are as follows:
   a. If *file-name-1* is specified, the associated procedure is executed when the condition described in the USE statement occurs.
   b. If INPUT is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the input mode or in the process of being opened in the input mode, except those files referred to by *file-name-1* in another USE statement specifying the same condition.
   c. If OUTPUT is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the output mode or in the process of being opened in the output mode, except those files referred to by *file-name-1* in another USE statement specifying the same condition.
   d. If I-O is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the I-O mode or in the process of being opened in the I-O mode, except those files referred to by file-name-1 in another USE statement specifying the same condition.
   e. If EXTEND is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the extend mode or in the process of being opened in the extend mode, except those files referred to by *file-name-1* in another USE statement specifying the same condition.

6. After execution of the USE procedure, control is returned to the next executable statement following the input-output statement that caused the exception. If the I-O status indicates that a critical error has occurred, it is the user's responsibility to take whatever action is necessary to correct the problem. This might mean doing a STOP RUN or ROLLBACK (for UDS) so that the program terminates.

   If this is not done, execution continues and further errors can result in the program. When a USE procedure is defined in a program, it is the user's responsibility to handle any error recovery or program termination that is desired.

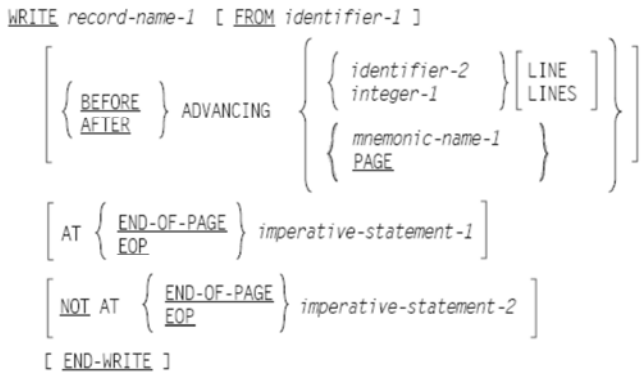
7. Within a USE procedure, no statement can be executed that would cause the execution of a USE procedure that had previously been invoked and had not yet returned control to the invoking routine.

4.5.7. The WRITE Statement

Function

The WRITE statement releases a logical record for an output file. It can also be used for vertical positioning of lines within a logical page.

General Format



Syntax Rules

1. If *identifier-1* is a function identifier, it must reference an alphanumeric function. When *identifier-1* is not a function identifier, *record-name-1* and *identifier-1* must not reference the same storage area.
2. *Record-name-1* is the name of a logical record in the File Section of the Data Division and can be qualified.
3. The ADVANCING *mnemonic-name-1* phrase cannot be specified when writing a record to a file that is associated with a file description entry containing a LINAGE clause.
4. *Identifier-2* must refer to an integer data item.
5. Integer-1 can be positive or zero, but must not be negative.
6. When mnemonic-name-1 is specified, the name must be associated with PAGE. Mnemonic-name-1 is defined in the SPECIAL-NAMES paragraph of the Environment Division.

7. The phrases ADVANCING PAGE and END-OF-PAGE must not both be specified in a single WRITE statement.
8. If the END-OF-PAGE or the NOT END-OF-PAGE phrase is specified, the LINAGE clause must be specified in the file description entry for the associated file.
9. The words END-OF-PAGE and EOP are equivalent.

General Rules

1. The file referred to by the file name associated with *record-name-1* must be open in the output or extend mode at the time of the execution of this statement. (See Section 4.5.3.)
2. The logical record released by the successful execution of the WRITE statement is no longer available in the record area unless the file name associated with *record-name-1* is specified in a SAME RECORD AREA clause. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as the file associated with *record-name-1.*
3. The result of the execution of the WRITE statement with the FROM phrase is equivalent to the execution of the following statements in the order specified:
   a. The statement

      ```
      MOVE identifier-1 TO record-name-1
      ```

      according to the rules specified for the MOVE statement
   b. The same WRITE statement without the FROM phrase
4. After execution of the WRITE statement, the information in the area referred to by *identifier-1* is available, even though the information in the area referred to by *record-name-1* is not available except as specified by the SAME RECORD AREA clause.
5. The file position indicator is not affected by the execution of a WRITE statement.
6. The execution of the WRITE statement updates the I-O status of the file name associated with *record-name*-1. (See Section 4.2.5.)
7. The execution of the WRITE statement releases a logical record to the operating system.
8. The number of character positions in the record referred to by *record-name-1* must not exceed the largest number of character positions in any record description entry specified for the file name associated with *record-name-1*, or be less than the smallest number of character positions allowed by the RECORD IS VARYING clause associated with the file name associated with *record-name-1*. In either of these cases, the execution of the WRITE statement is unsuccessful, the WRITE operation does not take place, the contents of the record area are unaffected, and the I-O status of the file associated with *record-name*-1 is set to a value indicating the cause of the condition. (See Section 4.2.5.)

9. If, during the execution of a WRITE statement with the NOT END-OF-PAGE phrase, the end-of-page condition does not occur, control is transferred to *imperative-statement*-2 at the appropriate time, as follows:
    a. If the execution of the WRITE statement is successful, after the record is written and after updating the I-O status of the file name associated with *record-name-1.*
    b. If the execution of the WRITE statement is unsuccessful, after updating the I-O status of the file name associated with *record-name*-1 and after executing the procedure, if any, specified by a USE AFTER STANDARD EXCEPTION PROCEDURE statement applicable to the file-name associated with *record-name-1.*
10. The END-WRITE phrase delimits the scope of the WRITE statement. (See Section 2.4.21)
11. The successor relationship of a sequential file is established by the order of execution of WRITE statements when the file is created. The relationship does not change unless records are added to the end of a file.
12. When a sequential file is open in the extend mode, the execution of the WRITE statement adds records to the end of the file as though the file were open in the output mode. If there are records in the file, the first record written after the execution of the OPEN statement with the EXTEND phrase is the successor of the last record in the file.
13. When an attempt is made to write beyond the externally defined boundaries of a sequential file, an exception condition exists and the contents of the record area are unaffected. The following actions occur:
    a. The value of the I-O status of the file associated with *record-name*-1 is set to a value indicating a boundary violation. (See Section 4.2.5)
    b. If a USE AFTER STANDARD EXCEPTION declarative is explicitly or implicitly specified for the file associated with *record-name*-1 that declarative procedure is then executed.
    c. If a USE AFTER STANDARD EXCEPTION is not explicitly or implicitly specified for the file associated with *record-name*-1, the result is undefined.
14. If the end of reel/unit is recognized and the externally defined boundaries of the file have not been exceeded, the following operations are executed:
    a. The standard ending reel/unit label procedure.
    b. A reel/unit swap. The current volume pointer is updated to point to the next reel/unit existing for the file.
    c. The standard beginning reel/unit label procedure.

15. Both the ADVANCING phrase and the END-OF-PAGE phrase allow control of the vertical positioning of each line on a representation of a printed page. If the ADVANCING phrase is not used, automatic advancing acts as if the user had specified AFTER ADVANCING 1 LINE. If the ADVANCING phrase is used, advancing is provided as follows:

    a. If *integer-1* or the value of the data item referred to by *identifier-2* is positive, the representation of the printed page is advanced the number of lines equal to that value.

    b. If the value of the data item referred to by *identifier-2* is negative, the results are undefined.

    c. If *integer-1* or the value of the data item referred to by *identifier-2* is zero, no repositioning of the representation of the printed page is performed.

    d. If the BEFORE phrase is used, the line is presented before the representation of the printed page is advanced according to rules a, b, c, and d.

    e. If the AFTER phrase is used, the line is presented after the representation of the printed page is advanced according to rules a, b, c, and d.

    f. If PAGE is specified and the LINAGE clause is specified in the associated file description entry, the record is presented on the logical page before or after (depending on the phrase used) the device is repositioned to the next logical page. The repositioning is to the first line that can be written on the next logical page as specified in the LINAGE clause.

    g. If PAGE or mnemonic-name-1 is specified and the LINAGE clause is not specified in the associated file description entry, the record is presented on the physical page before or after (depending on the phrase used) the device is repositioned to the next physical page. If the file is assigned to ANS74-PRINTER, the repositioning is to the first line on the next system standard page. If the file is assigned to PRINTER, the repositioning is to the first line specified by the system standard page definition. (See the *ER Programming Reference Manual*.) If physical page has no meaning in conjunction with a specific device, advancing occurs as if the user had specified BEFORE or AFTER (depending on the phrase used) ADVANCING 1 LINE.

16. If the logical end of the representation of the printed page is reached during the execution of a WRITE statement with the END-OF-PAGE phrase, *imperative-statement-1* specified in the END-OF-PAGE phrase is executed. The logical end is specified in the LINAGE clause associated with *record-name-1.*

17. An end-of-page condition occurs when the execution of a given WRITE statement with the END-OF-PAGE phrase causes printing or spacing within the footing area of a page body. This occurs when the execution of such a WRITE statement causes the linage counter to equal or exceed the value specified by *integer-2* or the data item referred to by *data-name-2* of the LINAGE clause. In this case, the WRITE statement is executed and then *imperative-statement-1* in the END-OF-PAGE phrase is executed.

    An automatic page overflow condition occurs when the execution of a given WRITE statement (with or without an END-OF-PAGE phrase) cannot be fully accommodated within the current page body.

This occurs when a WRITE statement, if executed, would cause the linage counter to exceed the value specified by *integer-1* or the data item referred to by *data-name-1* of the LINAGE clause. In this case, the record is presented on the logical page before or after (depending on the phrase used) the device is repositioned to the first line that can be written on the next logical page as specified in the LINAGE clause.

Imperative-statement-1 in the END-OF-PAGE phrase, if specified, is executed after the record is written and the device has been repositioned.

A page overflow condition occurs when the execution of a given WRITE statement would cause the LINAGE-COUNTER simultaneously to exceed the value of both *integer-2* or the data item referred to by *data-name-2* of the LINAGE clause and *integer-1* or the data item referred to by *data-name-1* of the LINAGE clause.

18. When a file description entry does not contain a LINAGE clause and the file is assigned to PRINTER, a page overflow condition is reached whenever execution of a given WRITE statement (with or without an END-OF-PAGE phrase) cannot be fully accommodated within the system standard page definition. (See Section 4.5.3.) In this case, the record is presented on the logical page before or after (depending on the phrase used) the device is repositioned to the first line that can be written on the next logical page, as specified by the system standard page definition.
19. No page overflow condition can occur when a file description entry does not contain a LINAGE clause and the file is assigned to ANS74-PRINTER. (See Section 4.5.3.)

4.6. Example of Sequential I-O

This subsection provides an example sequential I-O source program and its generated output.

4.6.1. Source Program

```
IDENTIFICATION DIVISION.
PROGRAM-ID.
   SEQUENTIAL-FILE-EXAMPLE.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
*
    SELECT REPORT-FILE
      ASSIGN TO PRINTER ALT-FILE
      ORGANIZATION IS SEQUENTIAL.
*
    SELECT BACKUP-FILE
      ASSIGN TO TAPE
      ORGANIZATION IS SEQUENTIAL.
```

```
*
    SELECT MASTER-FILE
      ASSIGN TO DISC IMP-NAME
      ORGANIZATION IS SEQUENTIAL.
*
    SELECT INPUT-FILE
      ASSIGN TO CARD-READER
      FILE STATUS IS INPUT-FILE-STATUS.
*
 I-O-CONTROL.
     SAME RECORD AREA FOR BACKUP-FILE, MASTER-FILE.
 DATA DIVISION.
 FILE SECTION.
*
 FD  INPUT-FILE
     LABEL RECORDS ARE OMITTED.
 01  INPUT-RECORD   PIC X(50).
*
 FD  MASTER-FILE
     LABEL RECORDS ARE STANDARD
     VALUE OF FILE-ID IS "MASTER-FILE"
     BLOCK CONTAINS 100 RECORDS.
 01  MASTER-RECORD   PIC X(80).
*
 FD  REPORT-FILE
     LABEL RECORDS ARE OMITTED
     VALUE OF FORMS-REQUEST IS "FORM-118"

     LINAGE IS 16 LINES
       WITH FOOTING AT FOOTING-VALUE
       LINES AT TOP 4
       LINES AT BOTTOM 4
     DENSITY IS 8 LINES.
 01  REPORT-RECORD   PIC X(80).
*
 FD  BACKUP-FILE
     LABEL RECORDS ARE STANDARD
     VALUE OF FILE-ID IS "MASTER-BACKUP"
     BLOCK CONTAINS 100 RECORDS.

 01  BACKUP-RECORD   PIC X(80).
 WORKING-STORAGE SECTION.
 01  INPUT-FILE-ERROR-RECORD.
     05 FILLER VALUE
                "INPUT FILE ERROR STATUS KEY EQUALS " PIC A(35).
     05 INPUT-FILE-STATUS  VALUE ZERO.
        10 STATUS-KEY-1   PIC X.
           88 INPUT-FILE-AT-END VALUE "1".
           88 INPUT-FILE-ERROR  VALUE "3" "4" "9".
        10 STATUS-KEY-2   PIC X.
```

```
 01  LINE-ADVANCE    PIC 99.
 01  FOOTING-VALUE   PIC 99 VALUE 12.
 01  REPORT-HEAD.
     05 FILLER VALUE "REPORT HEADING"   PIC A(15).
     05 FILLER PIC X(3) VALUE SPACES.
     05 DATE-VALUE.
        10 FILLER   PIC XX   VALUE SPACES.
        10 FILLER   VALUE "THE DATE IS  "
                    PIC X(15).
        10 REPORT-DATE-EDIT   PIC 99/99/99.
 01  REPORT-DATE PIC 9(6).
*
 01  REPORT-RECORD-FORMAT.
     05 REPORT-RECORD-ENTRY   PIC X(50).
     05 FILLER   VALUE
                "THIS IS PRINTED ON LINE "   PIC X(25).
     05 LINAGE-COUNTER-VALUE   PIC ZZ9.
 01  PAGE-FOOTING.
     05 FILLER   VALUE "PAGE NUMBER "   PIC A(13).
     05 PAGE-NUMBER-EDIT   PIC  ZZZ9.
 01  REPORT-FOOTING VALUE "END OF REPORT"   PICTURE A(13).
 01  PAGE-NUMBER   PIC 9(4)   VALUE 0.
 PROCEDURE DIVISION.
 DECLARATIVES.
 REPORT-FILE-AT-END SECTION.
*
     USE AFTER STANDARD EXCEPTION PROCEDURE ON INPUT-FILE.
*
 CHECK-INPUT-FILE-ERROR-BEGIN.
     IF INPUT-FILE-ERROR
        WRITE REPORT-RECORD FROM INPUT-FILE-ERROR-RECORD
             BEFORE ADVANCING 1
        CLOSE MASTER-FILE
              REPORT-FILE
        STOP RUN
     ELSE
        NEXT SENTENCE.
 CHECK-INPUT-FILE-ERROR-END.
     EXIT.
 END DECLARATIVES.
*
 CONTROLLER SECTION.
 MAIN-CONTROL.
     PERFORM OPEN-FILES.
     PERFORM MAIN-FUNCTION.
     PERFORM CLOSE-FILES.
     PERFORM CREATE-BACKUP.
     PERFORM FINAL-CLOSE.
     STOP RUN.
```

```
*
 OPEN-FILES SECTION.
 OPEN-BEGIN.
     OPEN INPUT INPUT-FILE
          OUTPUT REPORT-FILE
          EXTEND MASTER-FILE.
 OPEN-END.
     EXIT.

*
 MAIN-FUNCTION SECTION.
 MAIN-FUNCTION-BEGIN.
     PERFORM INITIATE-REPORT.
     PERFORM UPDATE-MASTER-GENERATE-REPORT.
     PERFORM TERMINATE-REPORT.
 MAIN-FUNCTION-END.
     EXIT.
*
 INITIATE-REPORT SECTION.
 INITIATE-BEGIN.
     ACCEPT REPORT-DATE FROM DATE.
     MOVE REPORT-DATE TO REPORT-DATE-EDIT.
     MOVE REPORT-HEAD TO REPORT-RECORD-ENTRY.
     PERFORM SET-LINE-VALUE.
     WRITE REPORT-RECORD FROM REPORT-RECORD-FORMAT
            BEFORE ADVANCING 5 LINES.
 INITIATE-END.
     EXIT.
*
 UPDATE-MASTER-GENERATE-REPORT SECTION.
 UPDATE-GENERATE-BEGIN.
     PERFORM READ-INPUT-FILE.
     IF INPUT-FILE-AT-END
        GO TO UPDATE-GENERATE-END
     ELSE
     PERFORM UPDATE-MASTER
     PERFORM GENERATE-REPORT
     GO TO UPDATE-GENERATE-BEGIN.
 UPDATE-GENERATE-END.
 EXIT.
*
 READ-INPUT-FILE SECTION.
 READ-INPUT-BEGIN.
     READ INPUT-FILE.
 READ-INPUT-END.
     EXIT.
```

```
*
 UPDATE-MASTER SECTION.
 UPDATE-MASTER-BEGIN.
     WRITE MASTER-RECORD FROM INPUT-RECORD.
 UPDATE-MASTER-END.
     EXIT.
*
 GENERATE-REPORT SECTION.
 GENERATE-REPORT-BEGIN.
     MOVE INPUT-RECORD TO REPORT-RECORD-ENTRY.
     PERFORM SET-LINE-VALUE.
     WRITE REPORT-RECORD FROM REPORT-RECORD-FORMAT
           BEFORE ADVANCING 2,
           AT END-OF-PAGE
            PERFORM REPORT-PAGE-FOOTING.
 GENERATE-REPORT-END.
     EXIT.
*
 REPORT-PAGE-FOOTING SECTION.
 PAGE-FOOTING-BEGIN.
     ADD 1 TO PAGE-NUMBER.
     MOVE PAGE-NUMBER TO PAGE-NUMBER-EDIT.
     MOVE PAGE-FOOTING TO REPORT-RECORD-ENTRY.
     PERFORM SET-LINE-VALUE.
     WRITE REPORT-RECORD FROM REPORT-RECORD-FORMAT
           BEFORE ADVANCING 1.
 PAGE-EJECT-CHECK.
     IF INPUT-FILE-AT-END
       GO TO PAGE-FOOTING-END
     ELSE
       MOVE SPACES TO REPORT-RECORD.
       WRITE REPORT-RECORD BEFORE ADVANCING PAGE.
 PAGE-FOOTING-END.
     EXIT.
*
 TERMINATE-REPORT SECTION.
 FORCE-PAGE-FOOTING.
     SUBTRACT LINAGE-COUNTER FROM FOOTING-VALUE
              GIVING LINE-ADVANCE.
     MOVE SPACES TO REPORT-RECORD.
     WRITE REPORT-RECORD
           BEFORE ADVANCING LINE-ADVANCE
           AT EOP
               PERFORM REPORT-PAGE-FOOTING.
 REPORT-FOOTING-CODE.
     MOVE REPORT-FOOTING TO REPORT-RECORD-ENTRY.
     PERFORM SET-LINE-VALUE.
     WRITE REPORT-RECORD FROM REPORT-RECORD-FORMAT
           BEFORE ADVANCING 1.
 TERMINATE-REPORT-END.
     EXIT.
```

```
*
 CLOSE-FILES SECTION.
 CLOSE-BEGIN.
     CLOSE INPUT-FILE WITH LOCK
           MASTER-FILE
           REPORT-FILE WITH LOCK.
 CLOSE-END.
     EXIT.
*
 CREATE-BACKUP SECTION.
 BACKUP-BEGIN.
     OPEN INPUT MASTER-FILE
          OUTPUT BACKUP-FILE.
 COPY-MASTER-TO-BACKUP.
     READ MASTER-FILE
          AT END GO TO BACKUP-END.
     WRITE BACKUP-RECORD.
     GO TO COPY-MASTER-TO-BACKUP.
 BACKUP-END.
     EXIT.
*

 FINAL-CLOSE SECTION.
 FINAL-BEGIN.
     CLOSE MASTER-FILE WITH LOCK
           BACKUP-FILE WITH LOCK.
 FINAL-END.
     EXIT.
*
 SET-LINE-VALUE SECTION.
 LINE-VALUE-BEGIN.
     MOVE LINAGE-COUNTER OF REPORT-FILE TO LINAGE-COUNTER-VALUE.
 LINE-VALUE-END.
     EXIT.
*
```

4.6.2. Generated Output

The following output is produced by the source program in 4.6.1.

```
REPORT  HEADING     THE DATE IS     76/12/22     THIS IS PRINTED ON
                                                 LINE   1


THIS IS RECORD NUMBER 1                          THIS IS PRINTED ON
                                                 LINE   6

THIS IS RECORD NUMBER 2                          THIS IS PRINTED ON
                                                 LINE   8

THIS IS RECORD NUMBER 3                          THIS IS PRINTED ON
                                                 LINE  10

PAGE NUMBER     1                                THIS IS PRINTED ON
                                                 LINE  12
THIS IS RECORD NUMBER 4                          THIS IS PRINTED ON
                                                 LINE   1

THIS IS RECORD NUMBER 5                          THIS IS PRINTED ON
                                                 LINE   3

THIS IS RECORD NUMBER 6                          THIS IS PRINTED ON
                                                 LINE   5

PAGE NUMBER     2                                THIS IS PRINTED ON
                                                 LINE  12

END OF REPORT                                    THIS IS PRINTED ON
                                                 LINE  13
```

Section 5
Relative I-O

The Relative I-O module provides a capability to access records of a mass storage file either randomly or sequentially. Each record in a relative file is uniquely identified by an integer value greater than zero that specifies the record's logical ordinal position in the file.

5.1. Level Characteristics

Relative I-O level 1 provides limited capabilities for the file control entry, the file description entry, and the entries in the I-O-CONTROL paragraph. Within the Procedure Division, the Relative I-O module level 1 provides limited capabilities for the CLOSE, OPEN, READ, REWRITE, USE, and WRITE statements; however, full capabilities for the DELETE statement.

Relative I-O level 2 provides full capabilities for the file control entry, the entries in the I-O-CONTROL paragraph, and the file description entry. Within the Procedure Division, the Relative I-O module level 2 provides full capabilities for the CLOSE, DELETE, OPEN, READ, REWRITE, START, USE, and WRITE statements.

5.2. Language Concepts

This subsection describes language concepts associated with the Relative I-O module.

5.2.1. Organization

A file with relative organization is a mass storage file from which any record can be stored or retrieved by providing the value of its relative record number.

Conceptually, a file with relative organization comprises a serial string of areas, each capable of holding a logical record and each numbered by a relative record number. Each logical record in a relative file is identified by the relative record number of its storage area. For example, the tenth record is the one addressed by relative record number 10 and is in the tenth record area, whether or not records have been written in any of the first through the ninth record areas.

In order to achieve more efficient access to records in a relative file, the number of character positions reserved on the medium to store a particular logical record can be different from the number of character positions in the description of that record in the program.

5.2.2. Access Modes

For relative organization, the order of sequential access is ascending based on the value of the relative record number. Only records that currently exist in the file are made available. The START statement can be used to establish a starting point for a series of subsequent sequential retrievals.

When a file is accessed in random mode, input-output statements are used to access the records in a programmer-specified order. For a file with relative organization, the programmer specifies the desired record by placing its relative record number in a relative key data item.

With dynamic access mode, the programmer can change from sequential accessing to random accessing, using appropriate forms of input-output statements.

5.2.3. File Position Indicator

The file position indicator is a conceptual entity used in this manual to facilitate exact specification of the next record to be accessed within a given file during certain sequences of input-output operations. The setting of the file position indicator is affected only by the CLOSE, OPEN, READ, and START statements. The concept of a file position indicator has no meaning for a file opened in the output or extend mode.

5.2.4. I-O Status

The I-O status is a 2-character conceptual entity whose value is set to indicate the status of an input-output operation during the execution of a CLOSE, DELETE, OPEN, READ, REWRITE, START, or WRITE statement and prior to the execution of any imperative statement associated with that input-output statement or prior to the execution of any applicable USE AFTER STANDARD EXCEPTION procedure. The value of the I-O status is made available to the COBOL program through the use of the FILE STATUS clause in the file description entry or the file control entry for the file.

The I-O status also determines whether an applicable USE AFTER STANDARD EXCEPTION procedure is executed. If any condition other than those contained in a successful completion results, such a procedure can be executed depending on rules stated for the USE statement. If one of the conditions listed under successful completion results, no such procedure is executed.

Certain classes of I-O values indicate critical error conditions. These are: any that begin with the digit 3 or 4, and any that begin with the digit 9 that the implementor defines as critical. If the value of the I-O status for an input-output operation indicates such an error condition, the implementor determines what action is taken after the execution of any applicable USE AFTER STANDARD EXCEPTION procedure, or if none applies, after completion of the normal input-output control system error processing.

I-O status expresses one of the following conditions upon completion of the input-output operation

1. Successful completion. The input-output statement was successfully executed.
2. At end. A sequential READ statement was unsuccessfully executed as a result of an at-end condition.
3. Invalid key. The input-output statement was unsuccessfully executed as a result of an invalid key condition.
4. Permanent error. The input-output statement was unsuccessfully executed as the result of an error that precluded further processing of the file. Any specified exception procedures are executed. The permanent error condition remains in effect for all subsequent input-output operations on the file.
5. Logic error. The input-output statement was unsuccessfully executed as a result of an improper sequence of input-output operations that were performed on the file or as a result of violating a limit defined by the user.
6. Defined by Unisys. The input-output statement was unsuccessfully executed as the result of a condition not indicated by other defined values of I-O status.

The following is a list of the values placed in the I-O status for the previously named conditions resulting from the completion of an input-output operation on a relative file. If more than one value applies, the implementor determines which of the applicable values to place in the I-O status.

1. Successful completion
    a. I-O Status = 00. The input-output statement is successfully executed and no further information is available concerning the input-output operation.
    b. I-O Status = 04. A READ statement is successfully executed but the length of the record being processed does not conform to the fixed file attributes for that file.
    c. I-O Status = 05. An OPEN statement is successfully executed but the referenced optional file is not present at the time the OPEN statement is executed. If the open mode is I-O or extend, the file has been created.
2. At-end condition with unsuccessful completion
    a. I-O Status = 10. A sequential READ statement is attempted and no next logical record exists in the file because
        - The end of the file has been reached
        - A sequential READ statement is attempted for the first time on an optional input file that is not present.
    b. I-O Status = 14. A sequential READ statement is attempted for a relative file and the number of significant digits in the relative record number is larger than the size of the relative key data item described for the file.

3. Invalid key condition with unsuccessful completion
    a. I-O Status = 22. An attempt is made to write a record that would create a duplicate key in a relative file.
    b. I-O Status = 23. This condition exists because
        - An attempt is made to randomly access a record that does not exist in the file.
        - A START or random READ statement is attempted on an optional input file that is not present.
    c. I-O Status = 24. An attempt is made to write beyond the externally defined boundaries of a relative file, as defined on the @ASG statement for the file. Or, a sequential WRITE statement is attempted for a relative file and the number of significant digits in the relative record number is larger than the size of the relative key data item described for the file.
4. Permanent error condition with unsuccessful completion
    a. I-O Status = 30. A permanent error exists and no further information is available concerning the input-output operation.
    b. I-O Status = 35. A permanent error exists because an OPEN statement with the INPUT, I-O, or EXTEND phrase is attempted on a nonoptional file that is not present.
    c. I-O Status = 37. A permanent error exists because an OPEN statement is attempted on a file and that file does not support the open mode specified in the OPEN statement. The possible violations are
        - The EXTEND or OUTPUT phrase is specified but the file does not support write operations.
        - The I-O phrase is specified but the file does not support the input and output operations that are permitted for a relative file when opened in the I-O mode.
        - The INPUT phrase is specified but the file does not support read operations.
    d. I-O Status = 38. A permanent error exists because an OPEN statement is attempted on a file previously closed with lock.
    e. I-O Status = 39. The OPEN statement is unsuccessful because a conflict has been detected between the fixed file attributes and the attributes specified for that file in the program.
5. Logic error condition with unsuccessful completion
    a. I-O Status = 41. An OPEN statement is attempted for a file in the open mode.
    b. I-O Status = 42. A CLOSE statement is attempted for a file not in the open mode.
    c. I-O Status = 43. In the sequential access mode, the last input-output statement executed for the file prior to the execution of a DELETE or REWRITE statement was not a successfully executed READ statement.

d. I-O Status = 44. A boundary violation exists because

- An attempt is made to WRITE or REWRITE a record that is larger than the largest or smaller than the smallest record allowed by the RECORD clause of the associated file name.
- In level 1 an attempt is made to REWRITE a record to a relative file and the record is not the same size as the record being replaced.

e. I-O Status = 46. A sequential READ statement is attempted on a file open in the input or input-output mode and no valid next record has been established because

- The preceding START statement was unsuccessful.
- The preceding READ statement was unsuccessful but did not cause an at-end condition.
- The preceding READ statement caused an at-end connection.

f. I-O Status = 47. The execution of a READ or START statement is attempted on a file not open in the input or I-O mode.

g. I-O Status = 48. The execution of a WRITE statement is attempted on a file not open in the I-O, output, [or extend] mode.

h. I-O Status = 49. The execution of a DELETE or REWRITE statement is attempted on a file not open in the I-O mode.

5.2.5. The Invalid Key Condition

The invalid key condition can occur as a result of the execution of a DELETE, READ, REWRITE, START, or WRITE statement. When the invalid key condition occurs, execution of the input-output statement that recognized the condition is unsuccessful and the file is not affected. For details of the causes of the condition, see 5.5.3 for the DELETE statement, 5.5.5 for the READ statement, Section 5.5.6 for the REWRITE statement, Section 5.5.7 for the START statement, and Section 5.5.9 for the WRITE statement.

If the invalid key condition exists after the execution of the input-output operation specified in an input-output statement, the following actions occur in the order shown:

1. The I-O status of the file connector associated with the statement is set to a value indicating the invalid key condition. (See Section 5.2.4.)
2. If the INVALID KEY phrase is specified in the input-output statement, any USE AFTER EXCEPTION procedure associated with the file connector is not executed and control is transferred to the imperative statement specified in the INVALID KEY phrase. Execution then continues according to the rules for each statement specified in that imperative statement. If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of the imperative statement specified in the INVALID KEY phrase, control is transferred to the end of the input-output statement and the NOT INVALID KEY phrase, if specified, is ignored.

3. If the INVALID KEY phrase is not specified in the input-output statement, a USE AFTER EXCEPTION procedure must be associated with the file connector and that procedure is executed. Upon completion of the execution of that procedure, control is transferred to the end of the input-output statement and the NOT INVALID KEY phrase, if specified, is ignored.

If the invalid key condition does not exist after the execution of the input-output operation specified by an input-output statement, the INVALID KEY phrase, if specified, is ignored and the following actions occur in the order shown

1. The I-O status of the file connector associated with the statement is updated and, if an unsuccessful exception condition that is not an invalid key condition exists, any USE AFTER EXCEPTION procedures associated with the file connector are executed. (See Section 5.2.4.)
2. Control is then transferred to the end of the input-output statement or to the imperative statement specified in the NOT INVALID KEY phrase if it is specified. In the latter case, execution continues according to the rules for each statement specified in that imperative statement. If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of the imperative statement specified in the NOT INVALID KEY phrase, control is transferred to the end of the input-output statement.

5.2.6. The At-End Condition

The at-end condition can occur as a result of the execution of a READ statement. (See Section 5.5.5.)

5.2.7. The File Attribute Conflict Condition

The file attribute conflict condition can result from the execution of an OPEN, REWRITE, or WRITE statement. When the file attribute conflict condition occurs, execution of the input-output statement that recognized the condition is unsuccessful and the file is not affected. (See Section 5.5.4, Section 5.5.6, and Section 5.5.9.)

When the file attribute conflict condition is recognized, these actions take place in the following order:

1. A value is placed in the I-O status associated with the file name to indicate the file attribute conflict condition. (See Section 5.2.4.)
2. Any USE AFTER EXCEPTION procedure associated with the file name is executed.

5.3. Environment Division/Input-Output Section

For information on the Input-Output Section, see Section 4.3.

5.3.1. The FILE-CONTROL Paragraph

For information on the FILE-CONTROL paragraph, see Section 4.3.1.

5.3.2. The File Control Entry

Function

The file control entry declares the relevant physical attributes of a relative file.

General Format

```
SELECT [ OPTIONAL ] file-name-1
    ASSIGN TO   { implementor-name-1 } ...
                { literal-1          }
  [ RESERVE integer-1 [ AREA  ] ]
              [ AREAS ]
  [ ORGANIZATION IS ]  RELATIVE
    [ ACCESS MODE IS  { SEQUENTIAL [RELATIVE KEY IS data-name-1]                  } ]
                      { { RANDOM  } RELATIVE KEY IS data-name-1                   }
                      { { DYNAMIC }                                               }
    [FILE STATUS IS data-name-2 ]
    [CODE TYPE IS data-name-3 ].
```

Syntax Rules

1. The SELECT clause must be specified first in the file control entry. The clauses that follow the SELECT clause can appear in any order.
2. Each file name in the Data Division must be specified only once in the FILE-CONTROL paragraph. Each file name specified in the SELECT clause must have a file description entry in the Data Division of the same program.
3. *Literal-1* must be a nonnumeric literal and not a figurative constant.
4. DISC or A-SHARED-FILE must be specified for *implementor-name-1* or *literal-1*.
5. For files assigned to DISC or A-SHARED-FILE:
   a. The second specification of *implementor-name-1* or *literal-1* is a 1- to 12-character name. It is used to link the file to the OS 2200 Executive File Control and must appear on an @ASG or @USE statement in the runstream for executing the object program.

b. When there is only one specification of *implementor-name-1* or *literal-1,* the first 12 characters of the file name are used to link the file to the OS 2200 Executive File Control.

6. The third and all subsequent recursions of *implementor-name-1* or *literal-1* are for documentation only.

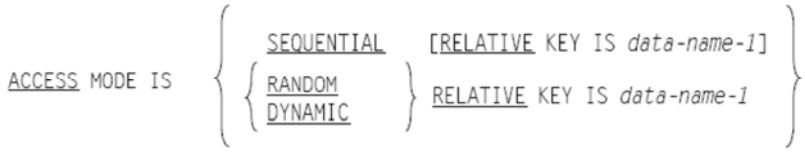
General Rules

1. If the file connector referenced by *file-name-1* is an external file connector (see Section 7.6.6), all file control entries in the run unit that reference this file connector must have
   a. The same specification for the OPTIONAL phrase.
   b. A consistent specification for *implementor-name-1* or *literal-1* in the ASSIGN clause.
   c. The same value for *integer-1* in the RESERVE clause.
   d. The same organization.
   e. The same access mode.
   f. The same external data item for data-name-1 in the RELATIVE KEY phrase.
2. The native character set is assumed for data on the external media.
3. The OPTIONAL phrase applies only to files opened in the input, I-O, or EXTEND mode. Its specification is required for input files that are not necessarily present each time the object program is executed.
4. The ASSIGN clause specifies the association of the file referenced by *file-name-1* to a storage medium referenced by *implementor-name-1* or *literal-1.*
5. The RESERVE clause for the Relative I-O module is the same as the RESERVE clause for the Sequential I-O module. Therefore, the specifications for the RESERVE clause are located in Section 4.3.2.
6. The FILE STATUS clause for the Relative I-O module is the same as the FILE STATUS clause for the Sequential I-O module. Therefore the specifications for the FILE STATUS clause are located in Section 4.3.2. The content of the data item associated with the FILE STATUS clause of a relative file is presented in Section 5.2.4.
7. The ACCESS MODE clause and the ORGANIZATION IS RELATIVE clause are presented in the following subsections.
8. The CODE TYPE clause for the Relative I-O Module is the same as the CODE TYPE clause for the Sequential I-O Module. Therefore, the specifications for the CODE TYPE clause are located in Section 4.3.2.

The ACCESS MODE Clause

Function

The ACCESS MODE clause specifies the order in which records are to be accessed in the file.



Syntax Rules

1. *Data-name-1* can be qualified.
2. *Data-name-1* must refer to an unsigned integer data item whose description does not contain the PICTURE symbol P.
3. *Data-name-1* must not be defined in a record description entry associated with that file name.
4. The ACCESS MODE IS RANDOM clause must not be specified for file names specified in the USING or GIVING phrase of a SORT or MERGE statement.
5. If a relative file is referred to by a START statement, the RELATIVE KEY phrase within the ACCESS MODE clause must be specified for that file.

General Rules

1. If the ACCESS MODE clause is not specified, sequential access is assumed.
2. If the access mode is sequential, records in the file are accessed in the sequence dictated by the file organization. For relative files, this sequence is the order of ascending relative record numbers of existing records in the file.
3. If the access mode is random, the value of the relative key data item for relative files indicates the record to be accessed.
4. If the access mode is dynamic, records in the file can be accessed sequentially and randomly.
5. All records stored in a relative file are uniquely identified by relative record numbers. The relative record number of a given record specifies the record's logical ordinal position in the file. The first logical record has a relative record number of 1, and subsequent logical records have relative record numbers of 2, 3, 4, ...
6. The relative key data item associated with the execution of an input-output statement is the data item referred to by *data-name-1* in the ACCESS MODE clause.
7. If the associated file connector is an external file connector, every file control entry in the run unit that is associated with that file connector must specify the same access mode. In addition, *data-name-1* must reference an external data item and the RELATIVE KEY phrase in each associated file control entry must

The ORGANIZATION IS RELATIVE Clause

Function

The ORGANIZATION IS RELATIVE clause specifies relative organization as the logical structure of a file.

General Format

```
[ ORGANIZATION IS ] RELATIVE
```

General Rules

1. The ORGANIZATION IS RELATIVE clause specifies relative organization as the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.
2. Relative organization is a permanent logical file structure in which each record is uniquely identified by an integer value greater than zero that specifies the record's logical ordinal position in the file.

5.3.3. The I-O-CONTROL Paragraph

Function

The I-O-CONTROL paragraph specifies the points at which rerun is to be established and the memory area that is to be shared by different files. The RERUN clause within the I-O-CONTROL paragraph is an obsolete element in this revision of American National Standard COBOL and it is to be deleted from the next revision of American National Standard COBOL.

General Format

```
I-O-CONTROL.
[ [ [ RERUN ON { file-name-1        } EVERY { integer-1 RECORDS OF file-name-2 } ] ]  ...
               { implementor-name-1 }       { integer-2 CLOCK-UNITS           }
                                            { condition-name-1                }
[ SAME   [ RECORD ]   AREA FOR file-name-3  { file-name-4 } ...  ] ...  . ]
```

General Rules

1. The RERUN clause for the Relative I-O module is a subset of the RERUN clause for the Sequential I-O module. Therefore, the specifications for the RERUN clause are located in 4.3.3.
2. The SAME AREA clause for the Relative I-O module is the same as the SAME AREA clause for the Sequential I-O module. (See Section 4.3.3.)
3. A file assigned with implementor name A-SHARED-FILE cannot be specified in the SAME AREA clause.

5.4. Data Division/File Section

The File Section is located in the Data Division of a source program. The File Section defines the structure of data files. Each file is defined by a file description entry and one or more record description entries. Record description entries are written immediately following the file description entry.

General Format

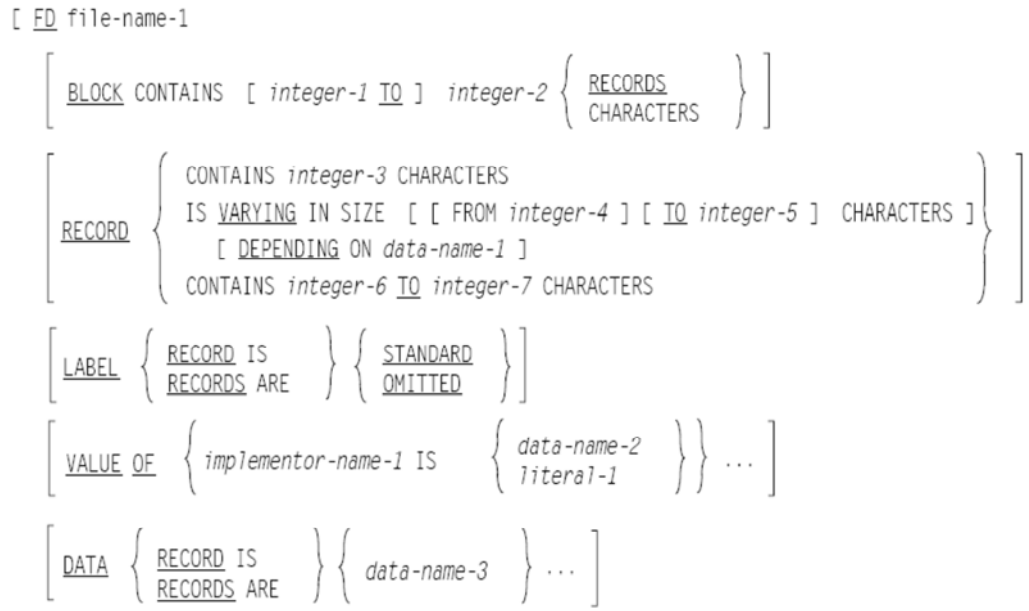
```
FILE SECTION.
[file-description-entry
{record-description-entry} . . . ] . . .
```

5.4.1. The File Description - Complete Entry Skeleton

Function

The file description entry furnishes information concerning the physical structure, identification, and record names pertaining to a given relative file.

General Format



Syntax Rules

1. The level indicator FD identifies the beginning of a file description entry and must precede *file-name-1.*
2. The clauses that follow *file-name-1* can appear in any order.
3. One or more record description entries must follow the file description entry.

General Rules

1. A file description associates *file-name-1* with a file connector.
2. The BLOCK CONTAINS clause for the Relative I-O module is the same as the BLOCK CONTAINS clause for the Sequential I-O module. Therefore, the specifications for the BLOCK CONTAINS clause are in Section 4.4.5.
3. The DATA RECORDS clause for the Relative I-O module is the same as the DATA RECORDS clause for the Sequential I-O module. Therefore, the specifications for the DATA RECORDS clause are in Section 4.4.5. The DATA RECORDS clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted from the next revision of American National Standard COBOL.
4. The LABEL RECORDS clause for the Relative I-O module is the same as the LABEL RECORDS clause for the Sequential I-O module. Therefore, the specifications for the LABEL RECORDS clause are in Section 4.4.5. The LABEL RECORDS clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted from the next revision of American National Standard COBOL.
5. The RECORD clause for the Relative I-O module is the same as the RECORD clause for the Sequential I-O module. Therefore, the specifications for the RECORD clause are in Section 4.4.5.
6. The VALUE OF clause for the Relative I-O module is the same as the VALUE OF clause for the Sequential I-O module. Therefore, the specifications for the VALUE OF clause are in Section 4.4.5. The VALUE OF clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted from the next revision of American National Standard COBOL.

5.5. Procedure Division

This subsection describes the Procedure Division and its associated statements.

5.5.1. General Description

The Procedure Division contains declarative procedures when the USE statement from the Relative I-O module is present in a COBOL source program. Shown below is the general format of the Procedure Division when the USE statement is present.

```
PROCEDURE DIVISION.

DECLARATIVES.

{section-name SECTION.

    USE statement.

[paragraph-name.

   [sentence]...]...}...

 END DECLARATIVES.
```

```
{section-name SECTION.

[paragraph-name.

   [sentence]...]...}...
```

5.5.2. The CLOSE Statement

Function

The CLOSE statement terminates the processing of files with optional lock.

General Format

```
CLOSE { file-name-1 [ WITH LOCK ] }...
```

Syntax Rule

The files referred to in the CLOSE statement need not all have the same organization or access.

General Rules

1. A CLOSE statement can be executed only for a file in an open mode.
2. Relative files are classified as belonging to the category of nonsequential single/multi-reel/unit. The results of executing each type of CLOSE statement for this category of file are summarized in the following table:

CLOSE Statement Format	File Category
	Nonsequential Single/Multireel/Unit
CLOSE	A
CLOSE WITH LOCK	A,B

The definitions of the symbols in the table follow. Where the definition depends on whether the file is an input, output, or input-output file, alternate definitions are given; otherwise, a definition applies to input, output, and input-output files.

A

Close File

- Input Files and Input-Output Files (Sequential Access Mode):

  If the file is positioned at its end and label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If the file is positioned at its end and label records are not specified for the file, label processing does not take place but other closing operations are executed. If the file is positioned other than at its end, the closing operations are executed, but there is no ending label processing.

- Input Files and Input-Output Files (Random or Dynamic Access Mode); Output Files (Random, Dynamic, or Sequential Access Mode):

  If label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If label records are not specified for the file, label processing does not take place but other closing operations are executed.

B

File Lock

The file is locked and cannot be opened again during the execution of the run unit.

3. The execution of the CLOSE statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 5.2.4.)
4. If an optional file is not present, no end-of-file processing is performed for the file, and the file position indicator is unchanged.
5. Following the successful execution of a CLOSE statement, the record area associated with *file-name-1* is no longer available. The unsuccessful execution of such a CLOSE statement leaves the availability of the record area undefined.
6. Following the successful execution of a CLOSE statement, the file is removed from the open mode and the file is no longer associated with the file connector.
7. If more than one *file-name-1* is specified in a CLOSE statement, the result of executing this CLOSE statement is the same as if a separate CLOSE statement had been written for each *file-name-1* in the same order as specified in the CLOSE statement.

5.5.3. The DELETE Statement

Function

The DELETE statement logically removes a record from a mass storage file.

General Format

```
DELETE file-name-1 RECORD
    [ INVALID KEY imperative-statement-1 ]
    [ NOT INVALID KEY imperative-statement-2 ]
    [ END-DELETE ]
```

Syntax Rules

1. The INVALID KEY phrase and NOT INVALID KEY phrase must not be specified for a DELETE statement that refers to a file that is in sequential access mode.
2. The INVALID KEY phrase must be specified for a DELETE statement that references a file which is not in sequential access mode and for which an applicable USE AFTER STANDARD EXCEPTION procedure is not specified.

General Rules

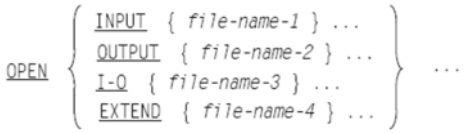
1. The file referred to by *file-name-1* must be a mass storage file and must be open in the I-O mode at the time of the execution of this statement. (See Section 5.5.4.)
2. For files in the sequential access mode, the last input-output statement executed for *file-name-1* prior to the execution of the DELETE statement must have been a successfully executed READ statement.
3. For a relative file in random or dynamic access mode, the record identified by the content of the relative key data item associated with *file-name-1* is logically removed. If the file does not contain the record specified by the key, the invalid key condition exists. (See Section 5.2.5.)
4. After the successful execution of a DELETE statement, the identified record has been logically removed from the file and can no longer be accessed.
5. The execution of a DELETE statement does not affect the content of the record area or the content of the data item referred to by the data name specified in the DEPENDING ON phrase of the RECORD clause associated with *file-name-1.*
6. The file position indicator is not affected by the execution of a DELETE statement.
7. The execution of the DELETE statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 5.2.4.)
8. Transfer of control following the successful or unsuccessful execution of the DELETE operation depends on the presence or absence of the optional INVALID KEY and NOT INVALID KEY phrases in the DELETE statement. (See Section 5.2.5.)
9. The END-DELETE phrase delimits the scope of the DELETE statement. (See Section 2.4.21.)

5.5.4. The OPEN Statement

Function

The OPEN statement initiates the processing of files.

General Format



Syntax Rules

1. The EXTEND phrase must be used only for files in the sequential access mode.
2. The files referred to in the OPEN statement need not all have the same organization or access.

General Rules

1. The successful execution of an OPEN statement determines the availability of the file and results in the file being in an open mode. The successful execution of the OPEN statement associates the file with the file name through the file connector.

   A file is available if it is physically present and is recognized by the I-O control system. Table 5–1 shows the results of opening available and unavailable files.

Table 5–1. Availability of a File

Phrase	File Is Available	File Is Unavailable
INPUT	Normal open	Open is unsuccessful
INPUT (optional file)	Normal open	Normal open; the first read causes the at-end condition or the invalid key condition
I-O	Normal open	Open is unsuccessful
I-O (optional file)	Normal open	Open causes the file to be created
OUTPUT	Normal open; the file contains no records	Open causes the file to be created
EXTEND	Normal open	Open is unsuccessful
EXTEND (optional file)	Normal open	Open causes the file to be created

2. The successful execution of an OPEN statement makes the associated record area available to the program. If the file connector associated with file name is an external file connector, only one record area is associated with the file connector for the run unit.

3. When a file is not in an open mode, no statement can be executed that refers to the file, either explicitly or implicitly, except for a MERGE or SORT statement with the USING or GIVING phrase, or an OPEN statement.
4. An OPEN statement must be successfully executed prior to the execution of any of the permissible input-output statements. In Table 5–2, an X at an intersection indicates that the specified statement, used in the access mode given for that row, can be used with the open mode given at the top of the column.

Table 5–2. Permissible Statements

File Access Mode	Statement	Open Mode			
		INPUT	OUTPUT	I-O	EXTEND
Sequential	READ	X		X	
	WRITE		X		X
	REWRITE			X	
	START	X		X	
	DELETE			X	
Random	READ	X		X	
	WRITE		X	X	
	REWRITE			X	
	START				
	DELETE			X	
Dynamic	READ	X		X	
	WRITE		X	X	
	REWRITE			X	
	START	X		X	
	DELETE			X	

5. A file can be opened with the INPUT, OUTPUT, EXTEND, and I-O phrases in the same run unit. Following the initial execution of an OPEN statement for a file, each subsequent OPEN statement execution for that same file must be preceded by the execution of a CLOSE statement, without the LOCK phrase, for that file.
6. Execution of the OPEN statement does not obtain or release the first data record.
7. If label records are specified for the file, the beginning labels are processed as follows:
    a. When the INPUT phrase is specified, the execution of the OPEN statement causes the labels to be checked in accordance with the conventions for input label checking.

b. When the OUTPUT phrase is specified, the execution of the OPEN statement causes the labels to be written in accordance with the conventions for output label writing.

The behavior of the OPEN statement when label records are specified but not present, or when label records are not specified but are present, is undefined.

8. If, during the execution of an OPEN statement, a file attribute conflict condition occurs, the execution of the OPEN statement is unsuccessful. (See Section 5.2.7.)
9. If a file opened with the INPUT phrase is an optional file that is not present, the OPEN statement sets the file position indicator to indicate that an optional file is not present.
10. When files are opened with the INPUT or I-O phrase, the file position indicator is set to 1.
11. When the EXTEND phrase is specified, the OPEN statement positions the file immediately after the last logical record for that file. The last logical record for a relative file is the currently existing record with the highest relative record number.
12. When the EXTEND phrase is specified and the LABEL RECORDS clause indicates label records are present, the execution of the OPEN statement includes the following steps:
    a. The beginning file labels are processed only in the case of a single reel/unit file.
    b. The beginning reel/unit labels on the last existing reel/unit are processed as though the file were being opened with the INPUT phrase.
    c. The existing ending file labels are processed as though the file were being opened with the INPUT phrase. These labels are then deleted.
    d. Processing then proceeds as though the file had been opened with the OUTPUT phrase.
13. The OPEN statement with the I-O phrase must reference a file that supports the input and output operations that are permitted for a relative file when opened in the I-O mode. The execution of the OPEN statement with the I-O phrase places the referenced file in the open mode for both input and output operations.
14. When the I-O phrase is specified and the LABEL RECORDS clause indicates label records are present, the execution of the OPEN statement includes the following steps:
    a. The labels are checked in accordance with the conventions for input-output label checking.
    b. The new labels are written in accordance with the conventions for input-output label writing.
15. For an optional file that is unavailable, the successful execution of an OPEN statement with an EXTEND or I-O phrase creates the file. This creation takes place as if the following statements were executed in the order shown:

```
OPEN OUTPUT file-name.
CLOSE file-name.
```

These statements are followed by execution of the OPEN statement specified in the source program.

The successful execution of an OPEN statement with the OUTPUT phrase creates the file. After the successful creation of a file, that file contains no data records.

16. The execution of the OPEN statement causes the value of the I-O status associated with file name to be updated. (See Section 5.2.4.)
17. If more than one file name is specified in an OPEN statement, the result of executing this OPEN statement is the same as if a separate OPEN statement had been written for each file name in the same order as specified in the OPEN statement.
18. The minimum and maximum record size for a file are established at the time the file is created and must not subsequently be changed.

5.5.5. The READ Statement

Function

For sequential access, the READ statement makes available the next logical record from a file. For random access, the READ statement makes available a specified record from a mass storage file.

Format 1

```
READ file-name-1 [ NEXT ]  RECORD  [ INTO identifier-1 ]
     [ AT END imperative-statement-1 ]
     [ NOT AT END imperative-statement-2 ]
     [ END-READ ]
```

Format 2

```
READ file-name-1 RECORD  [ INTO identifier-1 ]
    [ INVALID KEY imperative-statement-1 ]
    [ NOT INVALID KEY imperative-statement-2 ]
    [ END-READ ]
```

Syntax Rules

1. The storage area associated with *identifier-1* and the record area associated with *file-name-1* must not be the same storage area.
2. Use format 1 for all files in sequential access mode.
3. The NEXT phrase must be specified for files in dynamic access mode, when records are to be retrieved sequentially.
4. Format 2 is used for files in random access mode or for files in dynamic access mode when records are to be retrieved randomly.
5. The INVALID KEY phrase or the AT END phrase must be specified if no applicable USE AFTER STANDARD EXCEPTION procedure is specified for *file-name-1.*

General Rules

1. The file referred to by *file-name-1* must be open in the input or I-O mode at the time this statement is executed. (See Section 5.5.4.)
2. For files in sequential access mode, the NEXT phrase is optional and has no effect on the execution of the READ statement.
3. The execution of the READ statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 5.2.4.)
4. The setting of the file position indicator at the start of the execution of a format 1 READ statement is used in determining the record to be made available according to the following rules. Comparisons for records in relative files relate to the relative key number.
   a. If the file position indicator indicates that no valid next record has been established, execution of the READ statement is unsuccessful.
   b. If the file position indicator indicates that an optional file is not present, execution proceeds as specified in general rule 10.
   c. If the file position indicator was established by a previous OPEN or START statement, the first existing record in the file whose relative record number is greater than or equal to the file position indicator is selected.
   d. If the file position indicator was established by a previous READ statement, the first existing record in the file whose relative record number is greater than the file position indicator is selected.

   If a record is found that satisfies the preceding rules, it is made available in the record area associated with *file-name-1* unless the RELATIVE KEY phrase is specified for *file-name-1* and the number of significant digits in the relative record number of the selected record is larger than the size of the relative key data item, in which case, the file position indicator is set to indicate this condition and execution proceeds as specified in general rule 10.

   If no record is found that satisfies the preceding rules, the file position indicator is set to indicate that no next logical record exists and execution proceeds as specified in general rule 10.

   If a record is made available, the file position indicator is set to the relative record number of the record made available.
5. Regardless of the method used to overlap access time with processing time, the concept of the READ statement is unchanged; a record is available to the object program prior to the execution of *imperative-statement-2,* or *imperative-statement-4* if specified, or prior to the execution of any statement following the READ statement, if *imperative-statement-2* or *imperative-statement-4* are not specified.
6. When the logical records of a file are described with more than one record description, these records automatically share the same record area in storage; this is equivalent to an implicit redefinition of the area. The contents of any data items that lie beyond the range of the current data record are undefined at the completion of the execution of the READ statement.

7. The INTO phrase can be specified in a READ statement if
   a. Only one record description is subordinate to the file description entry.
   b. All record names associated with *file-name-1* and the data item referred to by *identifier-1* describe a group item or an elementary alphanumeric item.
8. The result of an execution of a READ statement with the INTO phrase is equivalent to the application of the following rules in the order specified:
   a. The execution of the same READ statement without the INTO phrase.
   b. The current record is moved from the record area to the area specified by *identifier-1* according to the rules for the MOVE statement without the CORRESPONDING phrase. The size of the current record is determined by rules specified for the RECORD clause. If the file description entry contains a RECORD VARYING clause, the implied move is a group move. The implied MOVE statement does not occur if the execution of the READ statement was unsuccessful. Any subscripting associated with *identifier-1* is evaluated after the record has been read and immediately before it is moved to the data item. The record is available in both the record area and the data item referred to by *identifier-1.*
9. If, at the time of the execution of a format 2 READ statement, the file position indicator indicates that an optional file is not present, the invalid key condition exists and execution of the READ statement is unsuccessful. (See Section 5.2.5.)
10. For a format 1 READ statement, if the file position indicator indicates that no next logical record exists or that the number of significant digits in the relative record number is larger than the size of the relative key data item or that an optional file is not present, the following occurs in the order specified:
    a. A value, derived from the setting of the file position indicator, is placed into the I-O status associated with *file-name-1* to indicate the at-end condition. (See Section 5.2.4.)
    b. If the AT END phrase is specified in the statement causing the condition, control is transferred to *imperative-statement-1* in the AT END phrase. Any USE AFTER STANDARD EXCEPTION procedure associated with *file-name-1* is not executed.
    c. If the AT END phrase is not specified, a USE AFTER STANDARD EXCEPTION procedure must be associated with *file-name-1*, and that procedure is executed. Return from that procedure is to the next executable statement following the end of the READ statement.

    When the at-end condition occurs, execution of the READ statement is unsuccessful.
11. If neither an at-end nor an invalid key condition occurs during the execution of a READ statement, the AT END phrase or the INVALID KEY phrase is ignored, if specified, and the following actions occur:
    a. The file position indicator is set and the I-O status associated with *file-name-1* is updated.

b. If an exception condition exists that is not an at-end condition or an invalid key condition, control is transferred according to rules of the USE statement following the execution of any USE AFTER EXCEPTION procedure applicable to *file-name-1*. (See Section 5.5.8.)

c. If no exception condition exists, the record is made available in the record area and any implicit move resulting from the presence of an INTO phrase is executed. Control is transferred to the end of the READ statement or to *imperative-statement-2*, if specified. In the latter case, execution continues according to the rules for each statement specified in *imperative-statement-2*. If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of *imperative-statement-2,* control is transferred to the end of the READ statement.

12. Following the unsuccessful execution of a READ statement, the content of the associated record area is undefined and the file position indicator is set to indicate that no valid next record has been established.

13. For a relative file for which dynamic access mode is specified, a format 1 READ statement with the NEXT phrase specified causes the next logical record to be retrieved from that file.

14. For a relative file, if the RELATIVE KEY phrase is specified for *file-name-1,* the execution of a format 1 READ statement moves the relative record number of the record made available to the relative key data item according to the rules for the MOVE statement. (See Section 3.6.19.)

15. For a relative file, the execution of a format 2 READ statement sets the file position indicator to the value contained in the data item referred to by the RELATIVE KEY phrase for the file, and the record whose relative record number equals the file position indicator is made available in the record area associated with *file-name-1*. If the file does not contain such a record, the invalid key condition exists and execution of the READ statement is unsuccessful. (See Section 5.2.5.)

16. If the number of character positions in the record that is read is less than the minimum size specified by the record description entries for *file-name-1,* the portion of the record area that is to the right of the last valid character read is undefined. If the number of character positions in the record that is read is greater than the maximum size specified by the record description entries for *file-name-1,* the record is truncated on the right to the maximum size. In either of these cases, the READ statement is successful and an I-O status is set indicating a record length conflict has occurred. (See Section 5.2.4.)

17. The END-READ phrase delimits the scope of the READ statement. (See Section 2.4.21.)

5.5.6. The REWRITE Statement

Function

The REWRITE statement logically replaces a record existing in a mass storage file.

General Format

```
REWRITE record-name-1 [ FROM identifier-1 ]

    [ INVALID KEY imperative-statement-1 ]
    [ NOT INVALID KEY imperative-statement-2 ]
    [ END-REWRITE ]
```

Syntax Rules

1. If *identifier-1* is a function identifier, it must reference an alphanumeric function. When *identifier-1* is not a function identifier, *record-name-1* and *identifier-1* must not reference the same storage area.
2. *Record-name-1* is the name of a logical record in the File Section of the Data Division and can be qualified.
3. The INVALID KEY and NOT INVALID KEY phrases must not be specified for a REWRITE statement that refers to a relative file in sequential access mode.
4. The INVALID KEY phrase must be specified in the REWRITE statement for relative files in the random or dynamic access mode, for which an appropriate USE AFTER STANDARD EXCEPTION procedure is not specified.

General Rules

1. The file referred to by the file name associated with *record-name-1* must be a mass storage file and must be open in the I-O mode at the time of execution of this statement. (See Section 5.5.4.)
2. For files in the sequential access mode, the last input-output statement executed for the associated file prior to the execution of the REWRITE statement must have been a successfully executed READ statement.
3. In level 1, the number of character positions in the record referred to by *record-name-1* must be equal to the number of character positions in the record being replaced. In level 2, the number of character positions in the record referred to by *record-name-1* can or cannot be equal to the number of character positions in the record being replaced.
4. The logical record released by a successful execution of the REWRITE statement is no longer available in the record area [unless the file name associated with *record-name-1* is specified in a SAME RECORD AREA clause. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as the file associated with *record-name-1*.

5. The result of the execution of a REWRITE statement with the FROM phrase is equivalent to the execution of the following statements in the order specified:

   a. The statement

   ```
   MOVE identifier-1 TO record-name-1
   ```

   according to the rules specified for the MOVE statement.

   b. The same REWRITE statement without the FROM phrase

6. After the execution of the REWRITE statement is complete, the information in the area referred to by *identifier-1* is available, even though the information in the area referred to by *record-name-1* is not available except as specified by the SAME RECORD AREA clause.

7. The file position indicator is not affected by the execution of a REWRITE statement.

8. The execution of the REWRITE statement causes the value of the I-O status of the file name associated with *record-name-1* to be updated. (See Section 5.2.4.)

9. The execution of the REWRITE statement releases a logical record to the operating system.

10. Transfer of control following the successful or unsuccessful execution of the REWRITE operation depends on the presence or absence of the optional INVALID KEY and NOT INVALID KEY phrases in the REWRITE statement. (See Section 5.2.5.)

11. The END-REWRITE phrase delimits the scope of the REWRITE statement. (See Section 2.4.21.)

12. The number of character positions in the record referred to by *record-name-1* must not be greater than the largest number of character positions nor less than the smallest number of character positions allowed by the RECORD IS VARYING clause associated with the file name associated with *record-name-1.* In either of these cases, the execution of the REWRITE statement is unsuccessful, the updating operation does not take place, the contents of the record area are unaffected, and the I-O status of the file associated with *record-name-1* is set to a value indicating the cause of the condition. (See Section 5.2.4.)

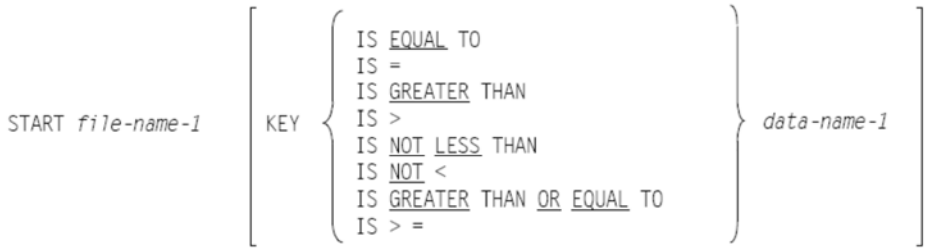
13. For a file accessed in either random or dynamic access mode, the MSCS logically replaces the record specified by the contents of the relative key data item of the file name associated with *record-name-1.* If the file does not contain the record specified by the key, the invalid key condition exists. (See Section 5.2.5.) When the invalid key condition is recognized, the execution of the REWRITE statement is successful, the updating operation does not take place, the contents of the record area are unaffected, and the I-O status of the file name associated with *record-name-1* is set to a value indicating the cause of the condition. (See Section 5.2.4 and Section 5.2.5.)

5.5.7. The START Statement

Function

The START statement provides a basis for logical positioning within a relative file, for subsequent sequential retrieval of records.

General Format




```
[ INVALID KEY imperative-statement-1 ]

[ NOT INVALID KEY imperative-statement-2 ]

[ END-START ]
```

Note: *The required relational symbols < and = are not underlined to avoid confusion with other symbols.*

Syntax Rules

1. *File-name-1* must be the name of a file with sequential or dynamic access.
2. *Data-name-1* can be qualified.
3. The INVALID KEY phrase must be specified if no applicable USE AFTER STANDARD EXCEPTION procedure is specified for *file-name-1.*
4. *Data-name-1,* if specified, must be the data item specified in the RELATIVE KEY phrase in the ACCESS MODE clause of the associated file description entry.

General Rules

1. The file referred to by *file-name-1* must be open in the input or I-O mode at the time that the START statement is executed. (See Section 5.5.4.)
2. If the KEY phrase is not specified, the relational operator IS EQUAL TO is implied.
3. The execution of the START statement does not alter either the contents of the record area or the contents of the data item referenced by the data name specified in the DEPENDING ON phrase of the RECORD clause associated with *file-name-1*.
4. The type of comparison specified by the relational operator in the KEY phrase occurs between a key associated with a record in the file referred to by *file-name-1* and a data item as specified in general rule 10. Numeric comparison rules apply. (See Section 3.6.3.)

a. The file position indicator is set to the relative record number of the first logical record in the file whose key satisfies the comparison.

b. If the comparison is not satisfied by any record in the file, the invalid key condition exists and the execution of the START statement is unsuccessful. (See Section 5.2.5.)

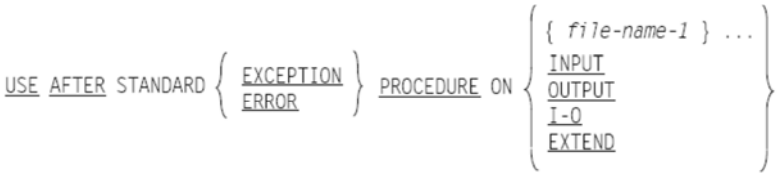
5. The execution of the START statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 5.2.4.)
6. If, at the time of the execution of the START statement, the file position indicator indicates that an optional file is not present, the invalid key condition exists and the execution of the START statement is unsuccessful. (See Section 5.2.5.)
7. Transfer of control following the successful or unsuccessful execution of the START operation depends on the presence or absence of the optional INVALID KEY and NOT INVALID KEY phrases in the START statement. (See Section 5.2.5.)
8. Following the unsuccessful execution of a START statement, the file position indicator is set to indicate that no valid next record has been established.
9. The END-START phrase delimits the scope of the START statement. (See Section 2.4.21)
10. The comparison described in general rule 4 uses the data item referred to by the RELATIVE KEY phrase of the ACCESS MODE clause associated with *file-name-1.*

5.5.8. The USE Statement

Function

The USE statement specifies procedures for input-output error handling that are in addition to the standard procedures provided by the input-output control system.

General Format



Syntax Rules

1. A USE statement, when present, must immediately follow a section header in the declaratives portion of the Procedure Division and must appear in a sentence by itself. The remainder of the section must consist of zero, one, or more procedural paragraphs that define the procedures to be used.
2. The USE statement is never executed; it merely defines the conditions calling for the execution of the USE procedures.
3. Appearance of *file-name-1* in a USE statement must not cause the simultaneous request for execution of more than one USE procedure.
4. The words ERROR and EXCEPTION are synonymous and can be used interchangeably.

5. The files implicitly or explicitly referred to in the USE statement need not all have the same organization or access.
6. The INPUT, OUTPUT, I-O, and EXTEND phrases can each be specified only once in the declaratives portions of a given Procedure Division.

General Rules

1. Declarative procedures can be included in any COBOL source program even if the program contains or is contained within another program. A declarative is invoked when any of the conditions described in the USE statement that prefaces the declarative occurs while the program is being executed. Only a declarative within the separately compiled program that contains the statement that caused the qualifying condition is invoked when any of the conditions described in the USE statement that prefaces the declarative occurs while that separately compiled program is being executed. If no declarative exists in the separately compiled program, no declarative is executed.
2. Within a declarative procedure, there must be no reference to any nondeclarative procedures.
3. Procedure names associated with a USE statement can be referenced in a different declarative section or in a nondeclarative procedure only with a PERFORM statement.
4. When *file-name-1* is specified explicitly, no other USE statement applies to *file-name-1.*
5. The procedures associated with a USE statement are executed by the input-output control system after completion of the standard input-output routine upon the unsuccessful execution of an input-output operation unless the AT END or INVALID KEY phrase takes precedence. The rules concerning when the procedures are executed are as follows:
    a. If *file-name-1* is specified, the associated procedure is executed when the condition described in the USE statement occurs.
    b. If INPUT is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the input mode or in the process of being opened in the input mode, except those files referred to by *file-name-1* in another USE statement specifying the same condition.
    c. If OUTPUT is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the output mode or in the process of being opened in the output mode, except those files referenced by *file-name-1* in another USE statement specifying the same condition.
    d. If I-O is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the I-O mode or in the process of being opened in the I-O mode, except those files referenced by *file-name-1* in another USE statement specifying the same condition.

e. If EXTEND is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the extend mode or in the process of being opened in the extend mode, except those files referred to by *file-name-1* in another USE statement specifying the same condition.

6. After execution of the USE procedure, control is returned to the next executable statement following the input-output statement that caused the exception. If the I-O status indicates that a critical error has occurred, it is the user's responsibility to take whatever action is necessary to correct the problem. This may mean doing a STOP RUN or ROLLBACK (for UDS) so that the program terminates. If this is not done, execution does continue and further errors may result in the program. When a USE procedure is defined in a program, it is the user's responsibility to handle any error recovery or program termination that is desired.
7. Within a USE procedure, there must not be the execution of any statement that would cause the execution of a USE procedure that had previously been invoked and had not yet returned control to the invoking routine.

5.5.9. The WRITE Statement

Function

The WRITE statement releases a logical record for an output or input-output file.

General Format

```
WRITE record-name-1  [ FROM identifier-1 ]
   [ INVALID KEY imperative-statement-1 ]
   [ NOT INVALID KEY imperative-statement-2 ]
   [ END-WRITE ]
```

Syntax Rules

1. If *identifier-1* is a function identifier, it must reference an alphanumeric function. When *identifier-1* is not a function identifier, *record-name-1* and *identifier-1* must not reference the same storage area.
2. *Record-name-1* is the name of a logical record in the File Section of the Data Division and can be qualified.
3. The INVALID KEY phrase must be specified if an applicable USE AFTER STANDARD EXCEPTION procedure is not specified for the associated file name.

General Rules

1. The file referred to by the file name associated with *record-name-1* must be open in the output, I-O, or extend mode at the time of the execution of this statement. (See Section 5.5.4.)
2. The logical record released by the successful execution of the WRITE statement is no longer available in the record area unless the file name associated with *record-name-1* is specified in a SAME RECORD AREA clause. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as the file associated with *record-name-1*.

3. The result of the execution of a WRITE statement with the FROM phrase is equivalent to the execution of the following statements in the order specified:

   a. The statement:

   ```
   MOVE identifier-1 TO record-name-1
   ```

      according to the rules specified for the MOVE statement

   b. The same WRITE statement without the FROM phrase.

4. After execution of the WRITE statement is complete, the information in the area referred to by *identifier-1* is available, even though the information in the area referred to by *record-name-1* is not available except as specified by the SAME RECORD AREA clause.

5. The file position indicator is not affected by the execution of a WRITE statement.

6. The execution of the WRITE statement causes the value of the I-O status of the file name associated with *record-name-1* to be updated. (See Section 5.2.4.)

7. The execution of the WRITE statement releases a logical record to the operating system.

8. The number of character positions in the record referred to by *record-name-1* must not be greater than the largest number of character positions or less than the smallest number of character positions allowed by the RECORD IS VARYING clause associated with the file name associated with *record-name-1*. In either of these cases, the execution of the WRITE statement is unsuccessful, the WRITE operation does not take place, the content of the record area is unaffected, and the I-O status of the file associated with *record-name-1* is set to a value indicating the cause of the condition. (See Section 5.2.4.)

9. Transfer of control following the successful or unsuccessful execution of the WRITE operation depends on the presence or absence of the INVALID KEY and NOT INVALID KEY phrases. (See Section 5.2.5.)

10. The END-WRITE phrase delimits the scope of the WRITE statement. (See Section 2.4.21.)

11. When a relative file is opened in the output mode, records can be placed into the file by one of the following:

    a. If the access mode is sequential, the WRITE statement causes a record to be released. The first record has a relative record number of one, and subsequent records released have relative record numbers of 2, 3, 4, and so forth. If the RELATIVE KEY phrase is specified for the file name associated with *record-name-1,* the relative record number of the record being released is moved into the relative key data item during execution of the WRITE statement according to the rules for the MOVE statement. (See Section 3.6.19.)

    b. If the access mode is random or dynamic, prior to the execution of the WRITE statement, the value of the relative key data item must be initialized by the program with the relative record number to be associated with the record in the record area. That record is then released by execution of the WRITE statement.

12. When a relative file is open in the extend mode, records are inserted into the file. The first record released has a relative record number one greater than the highest relative record number existing in the file. Subsequent records released have consecutively higher relative record numbers. If the RELATIVE KEY phrase is specified for the file name associated with *record-name-1,* the relative record number of the record being released is moved into the relative key data item during execution of the WRITE statement according to the rules for the MOVE statement. (See Section 3.6.19.)
13. When a relative file is opened in the I-O mode and the access mode is random or dynamic, records are to be inserted in the associated file. The value of the relative key data item must be initialized by the program with the relative record number to be associated with the record in the record area. Execution of the WRITE statement then causes the content of the record area to be released.
14. The invalid key condition exists under the following circumstances, when:
    a. The access mode is random or dynamic, and the relative key data item specifies a record that already exists in the file.
    b. An attempt is made to write beyond the externally defined boundaries of the file.
    c. The number of significant digits in the relative record number is larger than the size of the relative key data item described for the file.
15. When the invalid key condition is recognized, the execution of the WRITE statement is unsuccessful, the content of the record area is unaffected, and the I-O status of the file name associated with *record-name-1* is set to a value indicating the cause of the condition. Execution of the program proceeds according to the rules for an invalid key condition. (See Section 5.2.4 and Section 5.2.5.)

Section 6
Indexed I-O

The Indexed I-O module provides either random or sequential access to records of a mass storage file. Each record in an indexed file is uniquely identified by the value of one or more keys within that record.

6.1. Level Characteristics

Indexed I-O level 1 provides limited capabilities for the file control entry, the file description entry, and the entries in the I-O-CONTROL paragraph. Within the Procedure Division, the Indexed I-O module level 1 provides limited capabilities for the CLOSE, OPEN, READ, REWRITE, USE, and WRITE statements, and full capabilities for the DELETE statement.

Indexed I-O level 2 provides full capabilities for the file control entry, the entries in the I-O paragraph, and the file description entry. Within the Procedure Division, the Indexed I-O module level 2 provides full capabilities for the CLOSE, DELETE, OPEN, READ, REWRITE, START, USE, and WRITE statements.

6.2. Language Concepts

This subsection describes language concepts associated with the Indexed I-O module.

6.2.1. Organization

A file with indexed organization is a mass storage file from which any record can be accessed by giving the value of a specified key in that record. For each key data item defined for the records of a file, an index is maintained. Each such index represents the set of values from the corresponding key data item in each record. Each index, therefore, is a mechanism that can provide access to any record in the file.

Each indexed file has a primary index representing the prime record key of each record in the file. Each record is inserted in the file, changed, or deleted from the file based solely on the value of its prime record key. The prime record key of each record in the file must be unique, and it must not be changed when updating a record. The prime record key is declared in the RECORD KEY clause of the file control entry for the file.

Alternate record keys provide alternate means of retrieval for the records of a file. Such keys are named in the ALTERNATE RECORD KEY clause of the file control entry. The value of a particular alternate record key in each record need not be unique. When these values cannot be unique, the DUPLICATES phrase is specified in the ALTERNATE RECORD KEY clause.

6.2.2. Access Modes

For indexed organization, the order of sequential access is ascending, based on the value of the key of reference according to the collating sequence of the file. Any of the keys associated with the file can be established as the key of reference during the processing of a file. The order of retrieval from a set of records, which have duplicate key or reference values, is the original order of arrival of those records into that set. The START statement can be used to establish a starting point within an indexed file for a series of subsequent sequential retrievals.

When a file is accessed in random mode, input-output statements are used to access the records in a user-specified order. With the indexed organization, the programmer specifies the desired record by placing the value of one of its record keys in a record key or an alternate record key data item.

With dynamic access mode, the programmer can change from sequential accessing to random accessing, using appropriate forms of input-output statements.

6.2.3. File Position Indicator

The file position indicator is a conceptual entity used in this manual to help specify the next record to be accessed within a given file during certain sequences of input-output operations. The setting of the file position indicator is affected only by the CLOSE, OPEN, READ, and START statements. The concept of a file position indicator has no meaning for a file opened in the output or extend mode.

6.2.4. I-O Status

The I-O status is a 2-character conceptual entity whose value is set to indicate the status of an input-output operation during the execution of a CLOSE, DELETE, OPEN, READ, REWRITE, START, or WRITE statement, prior to the execution of any imperative statement associated with that input-output statement or prior to the execution of any applicable USE AFTER STANDARD EXCEPTION procedure. The value of the I-O status is made available to the COBOL program through the FILE STATUS clause in the file description entry or the file control entry for the file.

The I-O status also determines whether an applicable USE AFTER STANDARD EXCEPTION procedure is executed or not. If any condition other than those contained in a successful completion results, such a procedure can be executed according to rules stated for the USE statement. If one of the conditions listed under successful completion results, no such procedure is executed.

Certain classes of I-O status values indicate critical error conditions. These are: any that begin with the digit 3 or 4 and any that begin with the digit 9 that the implementor defines as critical. If the value of the I-O status for an input-output operation indicates such an error condition, the implementor determines what action is taken after the execution of any applicable USE AFTER STANDARD EXCEPTION procedure, or if none applies, after completion of the normal input-output control system error processing.

I-O status expresses one of the following conditions upon completion of the input-output operation:

1. Successful completion: The input-output statement has successfully executed.
2. At end: A sequential READ statement was unsuccessfully executed as a result of an at-end condition.
3. Invalid key: The input-output statement was unsuccessfully executed as a result of an invalid key condition.
4. Permanent error: The input-output statement was unsuccessfully executed as the result of an error that precluded further processing of the file. Any specified exception procedures are executed. The permanent error condition remains in effect for all subsequent input-output operations on the file.
5. Logic error: The input-output statement was unsuccessfully executed as a result of an improper sequence of input-output operations that were performed on the file or as a result of violating a user-defined limit.
6. Defined by Unisys: The input-output statement was unsuccessfully executed as the result of a condition not indicated by other defined values of I-O status.

The following is a list of the values placed in the I-O status for the previously named conditions resulting from the completion of an input-output operation on an indexed file. If more than one value applies, the first condition encountered determines which of the applicable values to place in the I-O status.

1. Successful completion
    a. I-O Status = 00: The input-output statement is successfully executed and no further information is available concerning the input-output operation.
    b. I-O Status = 02: The input-output statement is successfully executed but a duplicate key is detected.
        - For a READ statement, the key value for the current key of reference is equal to the value of the same key in the next record within the current key of reference.
        - For a REWRITE or WRITE statement, the record just written created a duplicate key value for at least one alternate record key for which duplicates are allowed.
    c. I-O Status = 04. A READ statement is successfully executed but the length of the record being processed does not conform to the fixed file attributes for that file.

d. I-O Status = 05. An OPEN statement is successfully executed but the referenced optional file is not present at the time the OPEN statement is executed. If the open mode is I-O or extend, the file has been created.

2. At-end condition with unsuccessful completion I-O Status = 10. A sequential READ statement is attempted and no next logical record exists in the file because:

   a. The end of the file has been reached.

   b. A sequential READ statement is attempted for the first time on an optional input file that is not present.

3. Invalid key condition with unsuccessful completion

   a. I-O Status = 21. A sequence error exists for a sequentially accessed indexed file. The prime record key value has been changed by the program between the successful execution of a READ statement and the execution of the next REWRITE statement for that file, or the ascending sequence requirements for successive record key values are violated. (See Section 6.5.9.)

   b. I-O Status = 22. An attempt is made to write (or rewrite) a record that would create a duplicate prime record key (or a duplicate alternate record key without the DUPLICATES phrase in an indexed file.)

   c. I-O Status = 23. This condition exists because

      - An attempt is made to randomly access a record that does not exist in the file.
      - A START or random READ statement is attempted on an optional input file that is not present.

   d. I-O Status = 24. An attempt is made to write beyond the externally defined boundaries of an indexed file.

4. Permanent error condition with unsuccessful completion

   a. I-O Status = 30. A permanent error exists and no further information is available concerning the input-output operation.

   b. I-O Status = 35. A permanent error exists because an OPEN statement with the INPUT, I-O, or EXTEND phrase is attempted on a nonoptional file that is not present.

   c. I-O Status = 37. A permanent error exists because an OPEN statement is attempted on a file and that file will not support the open mode specified in the OPEN statement. The possible violations are

      - The EXTEND or OUTPUT phrase is specified but the file will not support write operations.
      - The I-O phrase is specified but the file will not support the input and output operations that are permitted for an indexed file when opened in the I-O mode.
      - The INPUT phrase is specified but the file will not support read operations.

   d. I-O Status = 38. A permanent error exists because an OPEN statement is attempted on a file previously closed with lock.

e. I-O Status = 39. The OPEN statement is unsuccessful because a conflict has been detected between the fixed file attributes and the attributes specified for that file in the program.

5. Logic error condition with unsuccessful completion

   a. I-O Status = 41. An OPEN statement is attempted for a file in the open mode.

   b. I-O Status = 42. A CLOSE statement is attempted for a file not in the open mode.

   c. I-O Status = 43. In the sequential access mode, the last input-output statement executed for the file prior to the execution of a DELETE or REWRITE statement was not a successfully executed READ statement.

   d. I-O Status = 44. A boundary violation exists because

      - An attempt is made to WRITE or REWRITE a record that is larger than the largest or smaller than the smallest record allowed by the RECORD clause of the associated file name.
      - In level 1, an attempt is made to REWRITE a record to an indexed file and the record is not the same size as the record being replaced.

   e. I-O Status = 46. A sequential READ statement is attempted on a file open in the input or I-O mode and no valid next record has been established because

      - The preceding START statement was unsuccessful.
      - The preceding READ statement was unsuccessful but did not cause an at-end condition
      - The preceding READ statement caused an at-end condition.

   f. I-O Status = 47. The execution of a READ or START statement is attempted on a file not open in the input or I-O mode.

   g. I-O Status = 48. The execution of a WRITE statement is attempted on a file not open in the I-O, output, or extend mode.

   h. I-O Status = 49. The execution of a DELETE or REWRITE statement is attempted on a file not open in the I-O mode.

6. Defined by Unisys.

   I-O Status = 91. A locale has been returned that is not valid on the system that the program is executing.

6.2.5. The Invalid Key Condition

The invalid key condition can occur as a result of the execution of a DELETE, READ, REWRITE, START, or WRITE statement. When the invalid key condition occurs, execution of the input-output statement that recognized the condition is unsuccessful and the file is not affected. See Section 6.5.3 for the DELETE statement, Section 6.5.5 for the READ statement, Section 6.5.7 for the START statement, Section 6.5.9 for the WRITE statement and Section 6.5.6 for the REWRITE statement.

If the invalid key condition exists after the execution of the input-output operation specified in an input-output statement, the following actions occur in the order shown:

1. The I-O status of the file connector associated with the statement is set to a value indicating the invalid key condition. (See Section 6.2.4.)
2. If the INVALID KEY phrase is specified in the input-output statement, any USE AFTER EXCEPTION procedure associated with the file connector is not executed and control is transferred to the imperative statement specified in the INVALID KEY phrase. Execution then continues according to the rules for each statement specified in that imperative statement. If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of the imperative statement specified in the INVALID KEY phrase, control is transferred to the end of the input-output statement and the NOT INVALID KEY phrase, if specified, is ignored.
3. If the INVALID KEY phrase is not specified in the input-output statement, a USE AFTER EXCEPTION procedure must be associated with the file connector and that procedure is executed. Upon completion of the execution of that procedure, control is transferred to the end of the input-output statement and the NOT INVALID KEY phrase, if specified, is ignored.

If the invalid key condition does not exist after the execution of the input-output operation specified by an input-output statement, the INVALID KEY phrase, if specified, is ignored and the following actions occur in the order shown:

1. The I-O status of the file connector associated with the statement is updated and, if an unsuccessful exception condition that is not an invalid key condition exists, any USE AFTER EXCEPTION procedures associated with the file connector are executed. (See Section 6.2.4.)
2. Control is then transferred to the end of the input-output statement or to the imperative statement specified in the NOT INVALID KEY phrase if it is specified. In the latter case, execution continues according to the rules for each statement specified in that imperative statement. If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of the imperative statement specified in the NOT INVALID KEY phrase, control is transferred to the end of the input-output statement.

6.2.6. The At-End Condition

The at-end condition can occur as a result of the execution of a READ statement. (See Section 6.5.5.)

6.2.7. The File Attribute Conflict Condition

The file attribute conflict condition can result from the execution of an OPEN, REWRITE, or WRITE statement. When the file attribute conflict condition occurs, execution of the input-output statement that recognized the condition is unsuccessful and the file is not affected. (See Section 6.5.4, Section 6.5.6, and Section 6.5.9.)

When the file attribute conflict condition is recognized, these actions take place in the following order:

1. A value is placed in the I-O status associated with the file name to indicate the file attribute conflict condition. (See Section 6.2.4.)
2. A USE AFTER EXCEPTION procedure, if any, associated with the file name is executed.

6.2.8. Statistics

There is a function to provide the user with the statistical information for an indexed file. See format 2 of the ENTER statement. (See Section 7.7.4.)

6.3. Environment Division/Input-Output Section

Information concerning the Input-Output Section is located in Section 4.3.

6.3.1. The FILE-CONTROL Paragraph

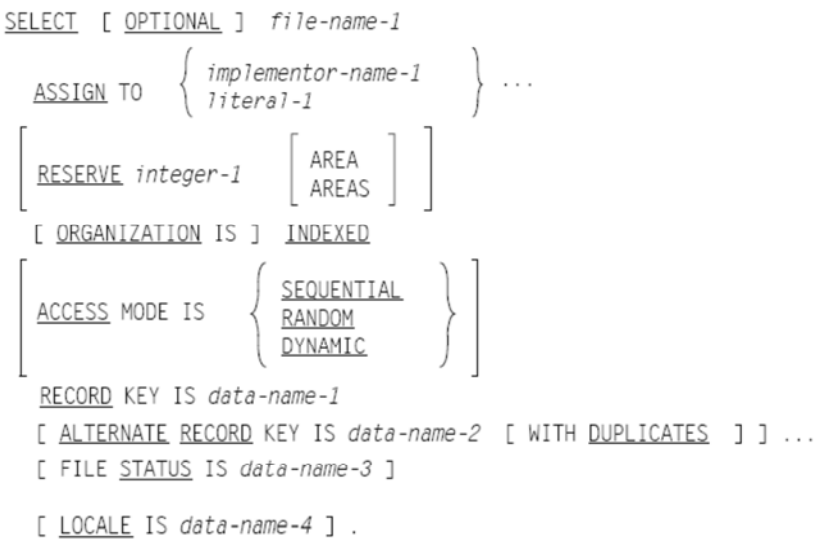
Information concerning the FILE-CONTROL paragraph is located in Section 4.3.1.

6.3.2. The File Control Entry

Function

The file control entry declares the relevant physical attributes of an indexed file.

General Format



Syntax Rules

1. Specify the SELECT clause first in the file control entry. The clauses that follow the SELECT clause can appear in any order.
2. Each file name in the Data Division must be specified only once in the FILE-CONTROL paragraph. Each file name specified in the SELECT clause must have a file description entry in the Data Division of the same program.
3. *Literal-1* must be a nonnumeric literal and not a figurative constant.
4. DISC or A-SHARED-FILE must be specified for *implementor-name-1* or *literal-1.*
5. For files assigned to DISC or A-SHARED-FILE:
   a. The second specification of *implementor-name-1* or *literal-1* is a 1- to 12-character name. It is used to link the file to the OS 2200 Executive File Control and must appear on an @ASG or @USE statement in the runstream for executing the object program.
   b. When there is only one specification of *implementor-name-1* or *literal-1*, the first 12 characters of the file name are used to link the file to the OS 2200 Executive File Control.
6. The third and all subsequent recursions of the *implementor-name-1* or *literal-1* are for documentation only.

General Rules

1. If the file connector referred to by *file-name-1* is an external file connector (see Section 7.6.6), all file control entries in the run unit that reference this file connector must have
   a. The same specification for the OPTIONAL phrase
   b. A consistent specification for *implementor-name-1* or *literal-1*

c. The same value for *integer-1* in the RESERVE clause

d. The same organization

e. The same access mode

f. The same data description entry for *data-name-1* with the same relative location within the associated record

g. The same data description entry for *data-name-2,* with the same relative location within the associated record, the same number of alternate record keys, and the same DUPLICATES phrase.

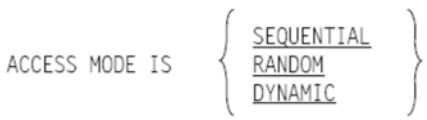
2. The native character set is assumed for data on the external media.
3. For an indexed file, the collating sequence associated with the native character set is assumed. This is the sequence of values of a given key of reference used to process the file sequentially.
4. The OPTIONAL phrase applies only to files opened in the input, I-O, or extend mode. Its specification is required for input files that are not necessarily present each time the object program is executed.
5. The ASSIGN clause specifies the association of the file referred to by *file-name-1* to a storage medium referenced by *implementor-name-1* or *literal-1*.
6. The RESERVE clause for the Indexed I-O module is the same as the RESERVE clause for the Sequential I-O module. (See Section 4.3.2.).
7. If the RESERVE clause is not specified, the number of input-output areas allocated is equal to the number of keys (prime record key and alternate record keys) specified for the file, plus four. If the RESERVE clause is specified, the number of input-output areas is increased by the value of (*integer-1* - 1). If zero is specified, the clause has no effect.
8. The FILE STATUS clause for the Indexed I-O module is the same as the FILE STATUS clause for the Sequential I-O module. (See Section 4.3.2 and Section 6.2.4.)
9. The ACCESS MODE clause, the ALTERNATE RECORD KEY clause, the LOCALE clause, the ORGANIZATION IS INDEXED clause, and the RECORD KEY clause are presented on the following pages.

The ACCESS MODE Clause

Function

The ACCESS MODE clause specifies the order in which records are to be accessed in the file.

General Format



Syntax Rule

The ACCESS MODE IS RANDOM clause must not be used for file names specified in the USING or GIVING phrase of a SORT or MERGE statement.

General Rules

1. If the ACCESS MODE clause is not specified, sequential access is assumed.
2. If the access mode is sequential, records in the file are accessed in the sequence dictated by the file organization. This sequence is ascending within a given key of reference according to the collating sequence of the file.
3. If the access mode is random, the value of a record key data item indicates the record to be accessed.
4. If the access mode is dynamic, records in the file can be accessed sequentially, randomly, or both.
5. If the associated file connector is an extend file connector, every file control entry in the run unit that is associated with that file connector must specify the same access mode.

The ALTERNATE RECORD KEY Clause

Function

The ALTERNATE RECORD KEY clause specifies an alternate record key access path to the records in an indexed file.

General Format

```
ALTERNATE RECORD KEY IS data-name-1 [ WITH DUPLICATES ]
```

Syntax Rules

1. *Data-name-1* can be qualified.
2. *Data-name-1* must be defined as a data item of the category alphanumeric within a record description entry associated with the file name to which the ALTERNATE RECORD KEY clause is subordinate.
3. *Data-name-1* must not refer to a group item that contains a variable occurrence data item.
4. *Data-name-1* must not refer to an item whose leftmost character position corresponds to the leftmost character position of the prime record key or of any other alternate record key associated with this file.
5. If the indexed file contains variable-length records, each alternate record key must be contained within the first *x* character positions of the record, where *x* equals the minimum record size specified for the file. (See Section 4.4.5.)

General Rules

1. An ALTERNATE RECORD KEY clause specifies an alternate record key for the file with which this clause is associated.
2. The data description of *data-name-1*, as well as its relative location within a record, must be the same as that used when the file was created. The number of alternate record keys for the file must also be the same as that used when the file was created.
3. The DUPLICATES phrase specifies that the value of the associated alternate record key can be duplicated within any of the records in the file. If the DUPLICATES phrase is not specified, the value of the associated alternate record key must not be duplicated among any of the records in the file.
4. If the file has more than one record description entry, *data-name-1* need be described in only one of these record description entries. The identical character positions referenced by *data-name-1* in any one record description entry are implicitly referenced as keys for all other record description entries of that file.
5. If the associated file connector is an external file connector, every file description entry in the run unit that is associated with that file connector must specify the same data description entry for *data-name-1*, the same relative location within the associated record, the same number of alternate record keys, and the same DUPLICATES phrase.

The ORGANIZATION IS INDEXED Clause

Function

The ORGANIZATION IS INDEXED clause specifies the indexed organization as the logical structure of a file.

General Format

```
[ORGANIZATION IS] INDEXED
```

General Rules

1. The ORGANIZATION IS INDEXED clause specifies indexed organization as the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.
2. Indexed organization is a permanent logical file structure in which each record is identified by the value of one or more keys within that record.

The RECORD KEY Clause

Function

The RECORD KEY clause specifies the prime record key access path to the records in an indexed file.

General Format

```
RECORD KEY IS data-name-1
```

Syntax Rules

1. *Data-name-1 can* be qualified.
2. *Data-name-1* must refer to a data item of the category alphanumeric within a record description entry associated with the file name to which the RECORD KEY clause is subordinate.
3. *Data-name-1* must not refer to a group item that contains a variable-occurrence data item.
4. If the indexed file contains variable-length records, the prime record key must be contained within the first *x* character positions of the record, where *x* equals the minimum record size specified for the file. (See Section 4.4.5.)

General Rules

1. The RECORD KEY clause specifies the prime record key for the file with which this clause is associated. The values of the prime record key must be unique among records of the file.
2. The data description of *data-name-1* as well as its relative location within a record must be the same as that used when the file was created.
3. If the file has more than one record description entry, *data-name-1* need be described in only one of these record description entries.

   The identical character positions referenced by *data-name-1* in any one record description entry are implicitly referenced as keys for all other record description entries of that file.
4. If the associated file connector is an external file connector, all file description entries in the run unit, which are associated with that file connector, must specify the same data description entry for *data-name-1* with the same relative location within the associated record.

The LOCALE Clause

Function

The LOCALE clause contains information pertaining to the locale of the data that is in the file when it was written.

General Format

```
LOCALE IS data-name-4
```

Syntax Rules

1. *Data-name-4* can be qualified.
2. *Data-name-4* must be defined in the Working-Storage Section as an elementary character data item of the alphanumeric category. The maximum length of *data-name-4* is 16. If *data-name-4* does not contain a VALUE clause, it is given an initial value of 'POSIX'.

General Rules

1. *Data-name-4* contains the locale name of the data that is stored in the file that is being processed. The locale, which is case sensitive, must be a valid locale on the system (see the *I18NLIB Programming Reference Manual* for additional information about locales). Trailing spaces in the locale name will be ignored.
2. If the LOCALE clause is specified and the file is opened for output, the numeric representation of the value of the locale in *data-item-4* is stored in the file to identify the locale under which the data was written. See the *I18NLIB Programming Reference Manual* for information about LOCALE names and their corresponding locale numbers.
3. If the LOCALE clause is specified, and the file is opened for input, the data item referenced by *data-item-4* contains the locale name that is equivalent to the locale's numeric representation that is in the file. If the locale that is returned is not valid and the FILE STATUS clause is specified, then the file status variable contains a value of 91. It the FILE STATUS was not specified and the locale returned is invalid, then a runtime error message is issued.
4. If the LOCALE phrase is not specified, the locale value of the en_US. ASCII locale (42) is stored in the file header when the file is initially opened in an output mode. If the LOCALE phrase is not specified, but the program is compiled with the I18N-IO keyword option, the numeric representation of the current locale is set in the file header to the value of the currently active locale (value of LC_ALL) when the file is initially opened in an output mode. If the file is opened for input and no LOCALE clause is present, then the locale information cannot be retrieved into the source program.
5. The locale information is set or retrieved as described in the following table.

   Note: *A file is available if it is physically present and is recognized by the I-O control system.*

Phrase	File is Available	File is Unavailable
INPUT	Normal open, locale retrieved	Open is unsuccessful, no locale available
INPUT (optional file)	Normal open, locale retrieved	Normal open, first read causes at end condition, no locale available
I-O	Normal open, locale retrieved	Open is unsuccessful, no locale available
I-O (optional file)	Normal open, locale retrieved	Open causes the file to be created. locale set
OUTPUT	Normal open, file contains no records, locale set	Open causes the file to be created. locale set
EXTEND	Normal open, locale retrieved	Open is unsuccessful, no locale available
EXTEND (optional file)	Normal open, locale retrieved	Open causes the file to be created, locale set

Note: *OPEN I-O and OPEN EXTEND of optional files are treated the same as OPEN OUTPUT (that is, data-name-4 must contain a valid locale value). If the file is not available, then the numeric representation of the locale specified in data-name-4 will be stored in the file header. If the file is available, then data-name-4 is updated to contain the value of the locale of the file that is being opened.*

6.3.3. The I-O-CONTROL Paragraph

Function

The I-O-CONTROL paragraph specifies the points at which rerun is to be established and the memory area that is to be shared by different files. The RERUN clause within the I-O-CONTROL paragraph is an obsolete element in this revision of American National Standard COBOL and it is to be deleted from the next revision of American National Standard COBOL.

General Format

```
[ [ RERUN ON { file-name-1          } EVERY ( { integer-1 RECORDS OF file-name-2 } ) ] ...
             { implementor-name-1   }       { integer-2 CLOCK-UNITS             }
                                            { condition-name-1                  }

[ SAME [RECORD] AREA FOR file-name-1 {file-name-2} ... ] ... . ]
```

General Rules

1. The RERUN clause for the Indexed I-O module is a subset of the RERUN clause for the Sequential I-O module. The specifications for the RERUN clause are located in Section 4.3.3.
2. The SAME AREA clause for the Indexed I-O module is the same as the SAME AREA clause for the Sequential I-O module. (See Section 4.3.3.)
3. A file assigned with implementor name A-SHARED-FILE cannot be specified in the SAME AREA clause.

6.4. Data Division/File Section

The File Section is located in the Data Division of a source program. The File Section defines the structure of data files. Each file is defined by a file description entry and one or more record description entries. Record description entries are written immediately following the file description entry.

General Format

```
FILE SECTION.

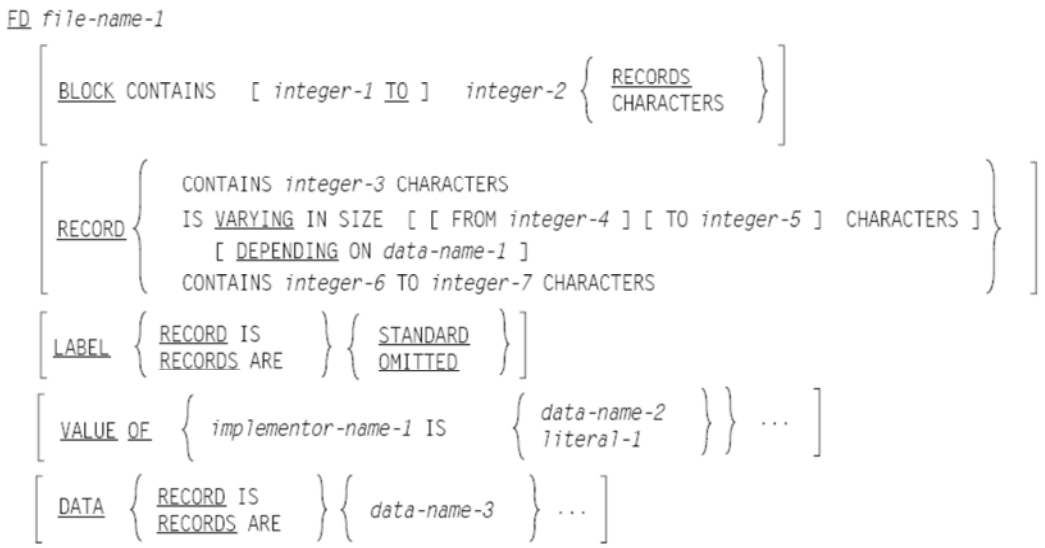
[file-description-entry
record-description-entry . . . ] . . .
```

6.4.1. The File Description - Complete Entry Skeleton

Function

The file description entry furnishes information concerning the physical structure, identification, and record names pertaining to an indexed file.

General Format



Syntax Rules

1. The level indicator FD identifies the beginning of a file description entry and must precede *file-name-1.*
2. The clauses that follow *file-name-1* can appear in any order.
3. One or more record description entries must follow the file description entry.

General Rules

1. A file description entry associates *file-name-1* with a file connector.
2. The BLOCK CONTAINS clause for the Indexed I-O module is the same as the BLOCK CONTAINS clause for the Sequential I-O module. Therefore, the specifications for the BLOCK CONTAINS clause are located in Section 4.4.5.
3. The DATA RECORDS clause for the Indexed I-O module is the same as the DATA RECORDS clause for the Sequential I-O module. Therefore, the specifications for the DATA RECORDS clause are located in Section 4.4.5. The DATA RECORDS clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted in the next revision of American National Standard COBOL.

4. The LABEL RECORDS clause for the Indexed I-O module is the same as the LABEL RECORDS clause for the Sequential I-O module. Therefore, the specifications for the LABEL RECORDS clause are located in Section 4.4.5. The LABEL RECORDS clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted in the next revision of American National Standard COBOL.
5. The RECORD clause for the Indexed I-O module is the same as the RECORD clause for the Sequential I-O module. Therefore, the specifications for the RECORD clause are located in Section 4.4.5.
6. The VALUE OF clause for the Indexed I-O module is the same as the VALUE OF clause for the Sequential I-O module. Therefore, the specifications for the VALUE OF clause are located in Section 4.4.5. The VALUE OF clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted in the next revision of American National Standard COBOL.

6.5. The Procedure Division

This subsection describes the Procedure Division and its associated statements.

6.5.1. General Description

The Procedure Division contains declarative procedures when the USE statement from the Indexed I-O module is present in a COBOL source program. Shown below is the general format of the Procedure Division when the USE statement is present.

```
PROCEDURE DIVISION.

DECLARATIVES.

{section-name SECTION.

   USE statement.

[paragraph-name.

   [sentence] ...] ... ...

END DECLARATIVES.

{section-name SECTION.

[paragraph-name.

[sentence]...]... ...
```

6.5.2. The CLOSE Statement

Function

The CLOSE statement terminates the processing of files with optional lock.

General Format

```
CLOSE  { file-name-1 [ WITH LOCK ] }. . .
```

Syntax Rule

The files referred to in the CLOSE statement need not all have the same organization or access.

General Rules

1. A CLOSE statement can be executed only for a file in an open mode.
2. Indexed files are classified as belonging to the category of nonsequential single/multireel/unit. The results of executing each type of CLOSE for this category of file are summarized in the following table.

CLOSE Statement Format	File Category
	Nonsequential Single/Multireel/Unit
CLOSE	A
CLOSE WITH LOCK	A,B

The definitions of the symbols in the table follow. Where the definition depends on whether the file is an input, output, or input-output file, alternate definitions are given; otherwise, a definition applies to input, output, and input-output files.

A

Close File

- Input files and input-output files (sequential access mode)

  If the file is positioned at its end and label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If the file is positioned at its end and label records are not specified for the file, label processing does not take place but other closing operations are executed. If the file is positioned other than at its end, the closing operations are executed, but there is no ending label processing.

- Input files and input-output files (random or dynamic access mode);

  Output files (random, dynamic, or sequential access mode)

  If label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If label records are not specified for the file, label processing does not take place but other closing operations are executed.

B

File Lock

The file is locked and cannot be opened again during the execution of the run unit.

3. The execution of the CLOSE statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 6.2.4.)
4. If an optional file is not present, no end-of-file processing is performed for the file and the file position indicator is unchanged.
5. Following the successful execution of a CLOSE statement, the record area associated with *file-name-1* is no longer available. The unsuccessful execution of such a CLOSE statement leaves the availability of the record area undefined.
6. Following the successful execution of a CLOSE statement, the file is removed from the open mode, and the file is no longer associated with the file connector.
7. If more than one *file-name-1* is specified in a CLOSE statement, the result of executing this CLOSE statement is the same as if a separate CLOSE statement had been written for each *file-name-1* in the same order as specified in the CLOSE statement.

6.5.3. The DELETE Statement

Function

The DELETE statement logically removes a record from a mass storage file.

General Format

```
DELETE file-name-1 RECORD

[ INVALID KEY imperative-statement-1 ]
[ NOT INVALID KEY imperative-statement-2 ]
[END-DELETE]
```

Syntax Rules

1. The INVALID KEY phrase and NOT INVALID KEY phrase must not be specified for a DELETE statement referring to a file in sequential access mode.
2. The INVALID KEY phrase must be specified for a DELETE statement referring to a file that is not in sequential access mode and for which an applicable USE AFTER STANDARD EXCEPTION procedure is not specified.

General Rules

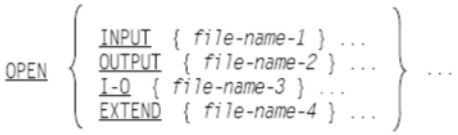
1. The file referred to by *file-name-1* must be a mass storage file and must be open in the I-O mode at the time of the execution of this statement. (See Section 6.5.4.)
2. For files in the sequential access mode, the last input-output statement executed for *file-name-1* prior to the execution of the DELETE statement must have been a successfully executed READ statement. The record that was accessed by that READ statement is logically removed.
3. For an indexed file in random or dynamic access mode, the record identified by the content of the prime record key data item associated with *file-name-1* is logically removed. If the file does not contain the record specified by the key, the invalid key condition exists. (See Section 6.2.5.)
4. After the successful execution of a DELETE statement, the identified record has been logically removed from the file and can no longer be accessed.
5. The execution of a DELETE statement does not affect the content of the record area or the content of the data item referred to by the data name specified in the DEPENDING ON phrase of the RECORD clause associated with *file-name-1.*
6. The file position indicator is not affected by the execution of a DELETE statement.
7. The execution of the DELETE statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 6.2.4.)
8. Transfer of control following the successful or unsuccessful execution of the DELETE operation depends on the presence or absence of the optional INVALID KEY and NOT INVALID KEY phrases in the DELETE statement. (See Section 6.2.5.)
9. The END-DELETE phrase delimits the scope of the DELETE statement (See Section 2.4.21.)

6.5.4. The OPEN Statement

Function

The OPEN statement initiates the processing of files.

General Format



Syntax Rules

1. The EXTEND phrase must be used only for files in the sequential access mode.
2. The files referred to in the OPEN statement need not all have the same organization or access.

General Rules

1. The successful execution of an OPEN statement determines the availability of the file and results in the file being in an open mode. The successful execution of an OPEN statement associates the file with the file name through the file connector.

   A file is available if it is physically present and is recognized by the I-O control system. Table 6–1 shows the results of opening available and unavailable files.

Table 6–1. Availability of a File

Phrase	File Is Available	File Is Unavailable
INPUT	Normal open	Open is unsuccessful
INPUT (optional file)	Normal open	Normal open; the first read causes the at end condition or the invalid key condition
I-O	Normal open	Open is unsuccessful
I-O (optional file)	Normal open	Open causes the file to be created
OUTPUT	Normal open; the file contains no records	Open causes the file to be created
EXTEND	Normal open	Open is unsuccessful
EXTEND (optional file)	Normal open	Open causes the file to be created

2. The successful execution of an OPEN statement makes the associated record area available to the program. If the file connector associated with the file name is an external file connector, there is only one record area associated with the file connector for the run unit.
3. When a file is not in an open mode, no statement can be executed that refers to the file, either explicitly or implicitly, except for a MERGE statement with the USING or GIVING phrase, an OPEN statement, or a SORT statement with the USING or GIVING phrases.
4. An OPEN statement must be successfully executed prior to the execution of any of the permissible input-output statements. In Table 6–2, an X at an intersection indicates that the specified statement, used in the access mode given for that row, can be used with the open mode given at the top of the column.

Table 6–2. Permissible Statements

File Access Mode	Statement	Open Mode			
		INPUT	OUTPUT	I-O	EXTEND
Sequential	READ	X		X	
	WRITE		X		X
	REWRITE			X	
	START	X		X	
	DELETE			X	
Random	READ	X		X	
	WRITE		X	X	
	REWRITE			X	
	START				
	DELETE			X	
Dynamic	READ	X		X	
	WRITE		X	X	
	REWRITE			X	
	START	X		X	
	DELETE			X	

5. A file can be opened with the INPUT, OUTPUT, EXTEND, and I-O phrases in the same run unit. Following the initial execution of an OPEN statement for a file, each subsequent OPEN statement execution for that same file must be preceded by the execution of a CLOSE statement, without the LOCK phrase for that file.
6. Execution of the OPEN statement does not obtain or release the first data record.
7. If label records are specified for the file, the beginning labels are processed as follows:
   a. When the INPUT phrase is specified, the execution of the OPEN statement causes the labels to be checked in accordance with the conventions for input label checking.
   b. When the OUTPUT phrase is specified, the execution of the OPEN statement causes the labels to be written in accordance with the conventions for output label writing.

   The behavior of the OPEN statement when label records are specified but not present, or when label records are not specified but are present, is undefined.
8. If, during the execution of an OPEN statement, a file attribute conflict condition occurs, the execution of the OPEN statement is unsuccessful. (See Section 6.2.7.)

9. If a file opened with the INPUT phrase is an optional file that is not present, the OPEN statement sets the file position indicator to indicate that an optional input file is not present.
10. When files are opened with the INPUT or I-O phrase, the file position indicator is set to the characters that have the lowest ordinal position in the collating sequence associated with the file, and the prime record key is established as the key of reference.
11. When the EXTEND phrase is specified, the OPEN statement positions the file immediately after the last logical record for that file. The last logical record is the currently existing record with the highest prime key value.
12. When the EXTEND phrase is specified and the LABEL RECORDS clause indicates label records are present, the execution of the OPEN statement indicates the following steps:
    a. The beginning file labels are processed only in the case of a single reel/unit file.
    b. The beginning reel/unit labels on the last existing reel/unit are processed as though the file were being opened with the INPUT phrase.
    c. The existing ending file labels are processed as though the file were being opened with the INPUT phrase. These labels are then deleted.
    d. Processing then proceeds as though the file had been opened with the OUTPUT phrase.
13. The OPEN statement with the I-O phrase must refer to a file that supports the input and output operations that are permitted for an indexed file when opened in the I-O mode. The execution of the OPEN statement with the I-O phrase places that file in the open mode for both input and output operations.
14. When the I-O phrase is specified and the LABEL RECORDS clause indicates label records are present, the execution of the OPEN statement includes the following steps:
    a. The labels are checked in accordance with the conventions for input-output label checking.
    b. The new labels are written in accordance with the conventions for input-output label writing.
15. For an optional file that is unavailable, the successful execution of an OPEN statement with an EXTEND or I-O phrase creates the file. This creation takes place as if the following statements were executed in the order shown:

    ```
    OPEN OUTPUT file-name.
    CLOSE file-name.
    ```

    These statements are followed by execution of the OPEN statement specified in the source program.

    The successful execution of an OPEN statement with the OUTPUT phrase creates the file. After the successful creation of a file, that file contains no data records.
16. The execution of the OPEN statement causes the value of the I-O status associated with the file name to be updated. (See Section 6.2.4.)

17. If more than one file name is specified in an OPEN statement, the result of executing this OPEN statement is the same as if a separate OPEN statement had been written for each file name in the same order as specified in the OPEN statement.
18. The minimum and maximum record size for a file is established at the time the file is created and must not subsequently be changed.

6.5.5. The READ Statement

Function

For sequential access, the READ statement makes available the next logical record from a file. For random access, the READ statement makes available a specified record from a mass storage file.

Format 1

```
READ file-name-1     [ NEXT ]     RECORD [ INTO identifier-1 ]
     [ AT END imperative-statement-1 ]
     [ NOT AT END imperative-statement-2 ]
     [ END-READ ]
```

Format 2

```
READ file-name-1  RECORD  [ INTO identifier-1 ]
     [ KEY IS data-name-1 ]
     [ INVALID KEY imperative-statement-3 ]
     [ NOT INVALID KEY imperative-statement-4 ]
     [ END-READ ]
```

Syntax Rules

1. The storage area associated with *identifier-1* and the record area associated with *file-name-1* must not be the same storage area.
2. *Data-name-1* must be the name of a data item specified as a record key associated with *file-name-1*.
3. *Data-name-1* can be qualified.
4. Format 1 must be used for all files in sequential access mode.
5. The NEXT phrase must be specified for files in dynamic access mode, when records are to be retrieved sequentially.
6. Use format 2 for files in random access mode or for files in dynamic access mode when records are to be retrieved randomly.
7. The INVALID KEY phrase or the AT END phrase must be specified if no applicable USE AFTER STANDARD EXCEPTION procedure is specified for *file-name-1*.

General Rules

1. The file referred to by *file-name-1* must be open in the input or I-O mode at the time this statement is executed. (See Section 6.5.4.)
2. For files in sequential access mode, the NEXT phrase is optional and has no effect on the execution of the READ statement.
3. The execution of the READ statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 6.2.4.)
4. The setting of the file position indicator at the start of the execution of a format 1 READ statement is used in determining the record to be made available according to the following rules. Comparisons for records in indexed files relate to the value of the current key of reference. For indexed files, the comparisons are made according to the collating sequence of the file.

   a. If the file position indicator indicates that no valid next record has been established, execution of the READ statement is unsuccessful.

   b. If the file position indicator indicates that an optional input file is not present, execution proceeds as specified in general rule 10.

   c. If the file position indicator was established by a previous OPEN or START statement, the first existing record in the file whose key value is greater than or equal to the file position indicator is selected.

   d. If the file position indicator was established by a previous READ statement and the current key of reference does not allow duplicates, the first existing record in the file whose key value is greater than the file position indicator is selected.

   e. If the file position indicator was established by a previous READ statement and the current key of reference does allow duplicates, the first record is selected in the file whose key value is either equal to the file position indicator and whose logical position within the set of duplicates is immediately after the record that was made available by that previous READ statement, or whose key value is greater than the file position indicator.

   If a record is found that satisfies the above rules, it is made available in the record area associated with *file-name-1*.

   If a record is not found that satisfies the above rules, the file position indicator is set to indicate that no next logical record exists and execution proceeds as specified in general rule 10.

   If a record is made available, the file position indicator is set to the value of the current key of reference of the record made available.
5. Regardless of the method used to overlap access time with processing time, the concept of the READ statement is unchanged; a record is available to the object program prior to the execution of *imperative-statement-2* or *imperative-statement-4* if specified, or prior to the execution of any statement following the READ statements, if *imperative-statement-2* or *imperative-statement-4* are not specified.

6. When the logical records of a file are described with more than one record description, these records automatically share the same record area in storage; this is equivalent to an implicit redefinition of the area. The contents of any data items that lie beyond the range of the current data record are undefined at the completion of the execution of the READ statement.
7. The INTO phrase can be specified in a READ statement if:
   a. Only one record description is subordinate to the file description entry.
   b. All record names associated with *file-name-1* and the data item referred to by *identifier-1* describe a group item or an elementary alphanumeric item.
8. The result of the execution of a READ statement with the INTO phrase is equivalent to the application of the following rules in the order specified:
   a. The execution of the same READ statement without the INTO phrase.
   b. The current record is moved from the record area to the area specified by *identifier-1* according to the rules for the MOVE statement without the CORRESPONDING phrase. The size of the current record is determined by rules specified for the RECORD clause. If the file description entry contains a RECORD VARYING clause, the implied move is a group move. The implied MOVE statement does not occur if the execution of the READ statement was unsuccessful. Any subscripting associated with *identifier-1* is evaluated after the record has been read and immediately before it is moved to the data item. The record is available in both the record area and the data item referred to by *identifier-1*.
9. If, at the time of the execution of a format 2 READ statement, the file position indicator indicates that an optional input file is not present, the invalid key condition exists and execution of the READ statement is unsuccessful. (See Section 6.2.5.)
10. For a format 1 READ statement, if the file position indicator indicates that no next logical record exists or that an optional input file is not present, the following occurs in the order specified:
    a. A value derived from the setting of the file position indicator is placed into the I-O status associated with *file-name-1* to indicate the at-end condition. (See Section 6.2.4.)
    b. If the AT END phrase is specified in the statement causing the condition, control is transferred to *imperative-statement-1* in the AT END phrase. Any USE AFTER STANDARD EXCEPTION procedure associated with *file-name-1* is not executed.
    c. If the AT END phrase is not specified, a USE AFTER STANDARD EXCEPTION procedure must be associated with *file-name-1,* and that procedure is executed. Return from that procedure is to the next executable statement following the end of the READ statement.

       When the at-end condition occurs, execution of the READ statement is unsuccessful.

11. If neither an at-end nor an invalid key condition occurs during the execution of a READ statement and if either a NOT AT END or a NOT INVALID KEY phrase is specified, control is transferred to *imperative-statement-2* or *imperative-statement-4* at the appropriate time as follows:

    a. After the record is made available, after setting the file position indicator, after updating the value of the I-O status associated with *file-name-1*, and after executing any implicit move resulting from the presence of an INTO phrase.

    b. If the record is not made available for a reason other than an at-end or an invalid key condition, after updating the value of the I-O status associated with *file-name-1*, and after executing the procedure, if any, specified by a USE AFTER STANDARD EXCEPTION PROCEDURE statement applicable to *file-name-1*.

12. Following the unsuccessful execution of a READ statement, the content of the associated record area is undefined, the key of reference is undefined, and the file position indicator is set to indicate that no valid next record has been established.

13. For an indexed file for which dynamic access mode is specified, a format 1 READ statement with the NEXT phrase specified causes the next logical record to be retrieved from that file.

14. For an indexed file being sequentially accessed, records having the same duplicate value in an alternate record key, which is the key of reference, are made available in the same order in which they are released by execution of WRITE statements or by execution of REWRITE statements that create such duplicate values.

15. If the KEY phrase is specified in a format 2 READ statement, *data-name-1* is established as the key of reference for this retrieval. If the dynamic access mode is specified, this key of reference is also used for retrievals by any subsequent executions of format 1 READ statements for the file until a different key of reference is established for the file.

16. If the KEY phrase is not specified in a format 2 READ statement, the prime record key is established as the key of reference for this statement. If the dynamic access mode is specified, this key of reference is also used for retrievals by any subsequent execution of format 1 READ statements for the file until a different key of reference is established for the file.

17. Execution of a format 2 READ statement sets the file position indicator to the value in the key of reference. This value is compared with the value contained in the corresponding data item of the stored records in the file until the first record having an equal value is found. In the case of an alternate key with duplicate values, the first record found is the first record of a sequence of duplicates that was released. The record so found is made available in the record area associated with *file-name-1*. If no record can be so identified, the invalid key condition exists and execution of the READ statement is unsuccessful. (See Section 6.2.5.)

18. If the number of character positions in the record that is read is less than the minimum size specified by the record description entries for *file-name-1*, the portion of the record area that is to the right of the last valid character read is undefined. If the number of character positions in the record that is read is greater than the maximum size specified by the record description entries for *file-name-1*, the record is truncated on the right to the maximum size. In either of these cases, the READ statement is successful and an I-O status is set indicating a record length conflict has occurred. (See Section 6.2.4.)
19. The END-READ phrase delimits the scope of the READ statement. (See 2.4.21)

6.5.6. The REWRITE Statement

Function

The REWRITE statement logically replaces a record existing in a mass storage file.

General Format

```
REWRITE record-name-1  [ FROM identifier-1 ]

    [ INVALID KEY imperative-statement-1 ]
    [ NOT INVALID KEY imperative-statement-2 ]
    [ END-REWRITE ]
```

Syntax Rules

1. If *identifier-1* is a function identifier, it must reference an alphanumeric function. When *identifier-1* is not a function identifier, *record-name-1* and *identifier-1* must not reference the same storage area.
2. *Record-name-1* is the name of a logical record in the File Section of the Data Division and can be qualified.
3. The INVALID KEY phrase must be specified if an applicable USE AFTER STANDARD EXCEPTION procedure is not specified for the associated file name.

General Rules

1. The file referred to by the file name associated with *record-name-1* must be a mass storage file and must be open in the I-O mode at the time of execution of this statement. (See Section 6.5.4.)
2. For files in the sequential access mode, the last input-output statement executed for the associated file prior to the execution of the REWRITE statement must have been a successfully executed READ statement. The record that was accessed by the READ statement is logically replaced.
3. In level 1, the number of character positions in the record referred to by *record-name-1* must be equal to the number of character positions in the record being replaced. In level 2, the number of character positions in the record referred to by *record-name-1* can or cannot be equal to the number of character positions in the record being replaced.

4. The logical record released by a successful execution of the REWRITE statement is no longer available in the record area unless the file name associated with *record-name-1* is specified in a SAME RECORD AREA clause. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as the file associated with *record-name-1*.
5. The result of the execution of a REWRITE statement with the FROM phrase is equivalent to the execution of the following statements in the order specified:
   a. The statement

```
MOVE identifier-1 TO record-name-1
```

      according to the rules specified for the MOVE statement

   b. The same REWRITE statement without the FROM phrase
6. After the execution of the REWRITE statement is complete, the information in the area referred to by *identifier-1* is available; the information in the area referred to by *record-name-1* is not available except as specified by the SAME RECORD AREA clause.
7. The file position indicator is not affected by the execution of a REWRITE statement.
8. The execution of the REWRITE statement causes the value of the I-O status of the file name associated with *record-name-1* to be updated. (See Section 6.2.4.)
9. The execution of the REWRITE statement releases a logical record to the operating system.
10. Transfer of control following the successful or unsuccessful execution of the REWRITE operation depends on the presence or absence of the optional INVALID KEY and NOT INVALID KEY phrases in the REWRITE statement. (See Section 6.2.5.)
11. The END-REWRITE phrase delimits the scope of the REWRITE statement. (See 2.4.21)
12. The number of character positions in the record referred to by *record-name-1* must not be greater than the largest number of character positions in any record description entry specified for the file name associated with *record-name-1*, nor less than the smallest number of character positions allowed by the RECORD clause associated with the file name associated with *record-name-1*. In either of these cases, the execution of the REWRITE statement is unsuccessful, the updating operation does not take place, the content of the record area is unaffected, and the I-O status of the file associated with *record-name-1* is set to a value indicating the cause of the condition. (See Section 6.2.4.)
13. For a file in the sequential access mode, the record to be replaced is specified by the value of the prime record key. When the REWRITE statement is executed, the value of the prime record key of the record to be replaced must be equal to the value of the prime record key of the last record read from this file.
14. For a file in the random or dynamic access mode, the record to be replaced is specified by the prime record key.
15. Execution of the REWRITE statement for a record that has an alternate record key occurs as follows:

a. When the value of a specific alternate record key is not changed, the order of retrieval when that key is the key of reference remains unchanged.

b. When the value of a specific alternate record key is changed, the subsequent order of retrieval of that record can be changed when that specific alternate record key is the key of reference. When duplicate key values are permitted, the record is logically positioned last within the set of duplicate records containing the same alternate record key value as the alternate record key value that was placed in the record.

16. The invalid key condition exists when

a. The file is open in the sequential access mode and the value of the prime record key of the record to be replaced is not equal to the value of the prime record key of the last record read from the file.

b. The file is open in the dynamic or random access mode, and the value of the prime record key of the record to be replaced is not equal to the value of the prime record key of any record existing in the file.

c. The value of an alternate record key of the record to be replaced, for which duplicates are not allowed, equals the value of the corresponding data item of a record already existing in the file.

17. When the invalid key condition is recognized, the execution of the REWRITE statement is unsuccessful, the updating operation does not take place, the content of the record area is unaffected, and the I-O status of the file name associated with *record-name-1* is set to a value indicating the cause of the condition. (See Section 6.2.4 and Section 6.2.5.)

6.5.7. The START Statement

Function

The START statement provides a basis for logical positioning within an indexed file for subsequent sequential retrieval of records.

General Format

```
START file-name-1  [ KEY { IS EQUAL TO
                         IS =
                         IS GREATER THAN
                         IS >
                         IS NOT LESS THAN
                         IS NOT <
                         IS GREATER THAN OR EQUAL TO
                         IS >= } data-name-1 ]
  [ INVALID KEY imperative-statement-1 ]
  [ NOT INVALID KEY imperative-statement-2 ]
  [ END-START ]
```

Note: *The required relational characters ">", "<", and "=" are not underlined to avoid confusion with other symbols.*

Syntax Rules

1. *File-name-1* must be the name of a file with sequential or dynamic access.
2. *Data-name-1* can be qualified.
3. The INVALID KEY phrase must be specified if no applicable USE AFTER STANDARD EXCEPTION procedure is specified for *file-name-1*.
4. If the KEY phrase is specified, *data-name-1* must refer to either:
   a. A data item specified as a record key associated with *file-name-1* (see Section 6.3.2 and Section 6.3.2).
   b. Any data item of category alphanumeric whose leftmost character position within a record of the file corresponds to the leftmost character position of a record key associated with *file-name-1* and whose length is not greater than the length of that record key.

General Rules

1. The file referred to by *file-name-1* must be open in the input or I-O mode at the time that the START statement is executed. (See Section 6.5.4.)
2. If the KEY phrase is not specified, the relational operator "IS EQUAL TO" is implied.
3. The execution of the START statement does not alter either the contents of the record area or the contents of the data item referenced by the data name specified in the DEPENDING ON phrase of the RECORD clause associated with *file-name-1*.
4. The type of comparison specified by the relational operator in the KEY phrase occurs between a key associated with a record in the file referred to by *file-name-1* and a data item as specified in general rules 10 and 11. The comparison is made on the ascending key of reference according to the collating sequence of the file. If the operands are of unequal size, comparison proceeds as though the longer one were truncated on the right such that its length is equal to that of the shorter. All other nonnumeric comparison rules apply. (See Section 3.6.3)
   a. The file position indicator is set to the value of the key of reference in the first logical record whose key satisfies the comparison.
   b. If the comparison is not satisfied by any record in the file, the invalid key condition exists and the execution of the START statement is unsuccessful. (See Section 6.2.5.)
5. The execution of the START statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 6.2.4.)
6. At the time of the execution of the START statement, the file position indicator indicates that an optional input file is not present, the invalid key condition exists and the execution of the START statement is unsuccessful. (See Section 6.2.4)
7. Transfer of control following the successful or unsuccessful execution of the START operation depends on the presence or absence of the optional INVALID KEY and NOT INVALID KEY phrases in the START statement. (See Section 6.2.5.)

8. Following the unsuccessful execution of a START statement, the file position indicator is set to indicate that no valid next record has been established. Also, the key of reference is undefined.
9. The END-START phrase delimits the scope of the START statement. (See Section 2.4.21.)
10. A key of reference is established as follows (see section 6.5.5):
    a. If the KEY phrase is not specified, the prime record key specified for *file-name-1* becomes the key of reference.
    b. If the KEY phrase is specified and *data-name-1* is specified as a record key for *file-name-1*, that record key becomes the key of reference.
    c. If the KEY phrase is specified and *data-name-1* is not specified as a record key for *file-name-1*, the record key whose leftmost character position corresponds to the leftmost character position of the data item specified by *data-name-1* becomes the key of reference.

    This key of reference is used to establish the ordering of records for the purpose of this START statement (see general rule 4), and, if the execution of the START statement is successful, the key of reference is also used for subsequent sequential READ statements. (See Section 6.5.5.)
11. If the KEY phrase is specified, the comparison described in general rule 4 uses the data item referred to by *data-name-1*.
12. If the KEY phrase is not specified, the comparison described in general rule 4 uses the data item referred to in the RECORD KEY clause associated with *file-name-1*.

6.5.8. The USE Statement

Function

The USE statement specifies procedures for input-output error handling that are in addition to the standard procedures provided by the input-output control system.

General Format



Syntax Rules

1. A USE statement must immediately follow a section header in the declaratives portion of the Procedure Division and must appear in a sentence by itself. The remainder of the section must consist of zero, one, or more procedural paragraphs that define the procedures to be used.
2. The USE statement itself is never executed; it merely defines the conditions calling for the execution of the USE procedures.

3. Appearance of *file-name-1* in a USE statement must not cause the simultaneous request for execution of more than one USE procedure.
4. The words ERROR and EXCEPTION are synonymous and can be used interchangeably.
5. The files implicitly or explicitly referred to in the USE statement need not all have the same organization or access.
6. The INPUT, OUTPUT, I-O, and EXTEND phrases can each be specified only once in the declaratives portion of a given Procedure Division.

General Rules

1. Declarative procedures can be included in any COBOL source program regardless of whether the program contains or is contained within another program. A declarative is invoked when any of the conditions described in the USE statement that prefaces the declarative occurs while the program is being executed. Only a declarative within the separately compiled program that contains the statement that caused the qualifying condition is invoked when any of the conditions described in the USE statement that prefaces the declarative occurs while that separately compiled program is being executed. If no qualifying declarative exists in the separately compiled program, no declarative is executed.
2. Within a declarative procedure, there must be no reference to any nondeclarative procedures.
3. Procedure names associated with a USE statement can be referred to in a different declarative section or in a nondeclarative procedure only with a PERFORM statement.
4. When *file-name-1* is specified explicitly, no other USE statement applies to *file-name-1.*
5. The procedures associated with a USE statement are executed by the input-output control system after completion of the standard input-output routine upon the unsuccessful execution of an input-output operation unless the AT END or INVALID KEY phrase takes precedence. The rules concerning when the procedures are executed are as follows:
    a. If *file-name-1* is specified, the associated procedure is executed when the condition described in the USE statement occurs.
    b. If INPUT is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the input mode or in the process of being opened in the input mode, except those files referred to by *file-name-1* in another USE statement specifying the same condition.
    c. If OUTPUT is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the output mode or in the process of being opened in the output mode, except those files referred to by *file-name-1* in another USE statement specifying the same condition.
    d. If I-O is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the I-O mode or in the process of being opened in the I-O mode, except those files referred to by *file-name-1* in another USE statement specifying the same condition.

e. If EXTEND is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the extend mode or in the process of being opened in the extend mode, except those files referred to by *file-name-1* in another USE statement specifying the same condition.

6. After execution of the USE procedure, control is returned to the next executable statement following the input-output statement that caused the exception. If the I-O status indicates that a critical error has occurred, it is the user's responsibility to take whatever action is necessary to correct the problem. This can mean doing a STOP RUN or ROLLBACK (for UDS) so that the program terminates. If this is not done, execution continues and further errors results in the program. When a USE procedure is defined in a program, it is the user's responsibility to handle any error recovery or program termination that is desired.

7. Within a USE procedure, there must not be the execution of any statement that would cause the execution of a USE procedure that had previously been invoked and had not yet returned control to the invoking routine.

6.5.9. The WRITE Statement

Function

The WRITE statement releases a logical record for an output or input-output file.

General Format

```
WRITE record-name-1  [ FROM identifier-1 ]

      [ INVALID KEY imperative-statement-1 ]
      [ NOT INVALID KEY imperative-statement-2 ]
      [ END-WRITE ]
```

Syntax Rules

1. If *identifier-1* is a function identifier, it must reference an alphanumeric function. When *identifier-1* is not a function identifier, *record-name-1* and *identifier-1* must not reference the same storage area.

2. *Record-name-1* is the name of a logical record in the File Section of the Data Division and can be qualified.

3. The INVALID KEY phrase must be specified if an applicable USE AFTER STANDARD EXCEPTION procedure is not specified for the associated file name.

General Rules

1. The file referred to by the file name associated with *record-name-1* must be open in the output, I-O, or extend mode at the time of the execution of this statement. (See Section 6.5.4.)

2. The logical record released by the successful execution of the WRITE statement is no longer available in the record area unless the file name associated with *record-name-1* is specified in a SAME RECORD AREA clause. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as the file associated with *record-name-1*.
3. The result of the execution of the WRITE statement with the FROM phrase is equivalent to the execution of the following statements in the order specified:
   a. The statement

      ```
      MOVE identifier-1 TO record-name-1
      ```

      according to the rules specified for the MOVE statement
   b. The same WRITE statement without the FROM phrase
4. After execution of the WRITE statement is complete, the information in the area referred to by *identifier-1* is available, even though the information in the area referred to by *record-name-1* is not available except as specified by the SAME RECORD AREA clause.
5. The file position indicator is unaffected by the execution of a WRITE statement.
6. The execution of the WRITE statement causes the value of the I-O status of the file name associated with *record-name-1* to be updated. (See Section 6.2.4.)
7. The execution of the WRITE statement releases a logical record to the operating system.
8. The number of character positions in the record referred to by *record-name-1* must not be greater than the largest number of character positions in any record description entry specified for the file name associated with *record-name-1* or less than the smallest number of character positions allowed by the RECORD IS VARYING clause associated with the file name associated with *record-name-1.* In either of these cases, the execution of the WRITE statement is unsuccessful, the WRITE operation does not take place, the content of the record area is unaffected, and the I-O status of the file associated with *record-name-1* is set to a value indicating the cause of the condition. (See Section 6.2.4.)
9. If, during the execution of a WRITE statement with the NOT INVALID KEY phrase, the invalid key condition does not occur, control is transferred to *imperative-statement-2* at the appropriate time as follows:
   a. If the execution of the WRITE statement is successful, after the record is written and after updating the I-O status of the *file-name* associated with *record-name-1*.
   b. If the execution of the WRITE statement is unsuccessful for a reason other than an invalid key condition, after updating the I-O status of the *file-name* associated with *record-name-1* and after executing the procedure, if any, specified by a USE AFTER STANDARD EXCEPTION PROCEDURE statement applicable to the file name associated with *record-name-1*.
10. The END-WRITE phrase delimits the scope of the WRITE statement. (See Section 2.4.21)

11. Execution of a WRITE statement causes the release of the content of the record area. The Mass Storage Control System (MSCS) utilizes the contents of the record keys in such a way that subsequent access of the record can be made based on any of these specified record keys.
12. The value of the prime record key must be unique within the records in the file.
13. The data item specified as the prime record key must be set by the program to the desired value prior to the execution of the WRITE statement.
14. If the file is open in the sequential access mode, records must be released in ascending order of prime record key values according to the collating sequence of the file. When the file is open in the extend mode, the first record released must have a prime record key whose value is greater than the highest prime record key value existing in the file.
15. If the file is open in the random or dynamic access mode, records can be released in any program-specified order.
16. When the ALTERNATE RECORD KEY clause is specified in the file control entry or file description entry for an indexed file, the value of the alternate record key cannot be unique only if the DUPLICATES phrase is specified for that data item. In this case, storage of records is such that when records are accessed sequentially, the order of retrieval of those records is the order in which they were released.
17. The invalid key condition exists when
    a. The file is open in the sequential access mode, and the file is also open in the output or extend mode, and the value of the prime record key is not greater than the value of the prime record key of the previous record.
    b. The file is open in the output or I-O mode, and the value of the prime record key equals the value of a prime record key of a record already existing in the file.
    c. The file is open in the output, extend, or I-O mode, and the value of an alternate record key for which duplicates are not allowed equals the value of the corresponding data item of a record already existing in the file.
    d. An attempt is made to write beyond the externally defined boundaries of the file.
18. When the invalid key condition is recognized, the execution of the WRITE statement is unsuccessful, the content of the record area is unaffected, and the I-O status of the file name associated with *record-name-1* is set to a value indicating the cause of the condition. Execution of the program proceeds according to the rules for an invalid key condition. (See Section 6.2.4 and Section 6.2.5.)

Section 7
Interprogram Communication

Complex data processing problems are frequently solved by developing a set of separately compilable, but logically coordinated, programs that at some time before execution can be compiled and assembled into a complete problem solution. The organization of COBOL programs and run units supports this approach of dividing large problem solutions into small, more manageable, portions that can be programmed and validated independently.

7.1. Level Characteristics

Interprogram Communication level 1 can transfer control to one or more programs whose names are known at compile time. Data can be shared among such programs.

Interprogram Communication level 2 can transfer control to one or more programs whose names are not known at compile time. It can determine the availability of object time storage for the program to which control is being passed. Interprogram Communication level 2 also provides external attributes, global names, and nesting of source programs.

7.2. Language Concepts

The Interprogram Communication module enables a program to communicate with one or more programs. The module provides this communication by transferring control from one program to another within a run unit and by passing parameters between programs to make certain data values available to a called program. The Interprogram Communication module also permits communication between two programs by the sharing of data and files.

7.2.1. Transfer of Control

There are four methods of transferring control within a program:

- A GO TO statement
- A PERFORM statement
- An input procedure associated with a SORT statement, or an output procedure associated with either a SORT or a MERGE statement
- A declarative procedure that is activated whenever certain conditions, including errors and exceptions, occur

An input-output procedure can be considered as an implicit PERFORM statement that is executed in conjunction with a SORT or MERGE statement; for this reason, the restrictions on the PERFORM statement apply equally to input-output procedures.

Stricter restrictions than those for the PERFORM statement apply to declarative procedures.

7.2.2. Shared Data

All the data declared in a program's Data Division can be referred to by the statements that constitute that program. Under certain conditions, a program can refer to data items whose declarations are not included in its Data Division. (See Section 7.2.3.)

7.2.3. Program and Run Unit Organization

There are two levels of computer programs in a UCS COBOL environment: the source level and the object level.

At the source level, the most inclusive unit of a computer program is a source program. A source program can contain other source programs. A source program is a syntactically correct set of COBOL statements as specified in this manual and consists of an Identification Division followed optionally by an Environment Division and/or a Data Division and/or a Procedure Division. A source program that itself is not contained within another source program can be converted by a compiler into an object program that either alone, or together with other object programs, is capable of being executed. In general, a source program that is contained within another program cannot itself be converted by a compiler into an object program since the contained program can reference data in the containing source program.

The Procedure Division of a source program is organized into a sequence of procedures of two types. Declarative procedures, normally termed declaratives, are procedures that are executed only when special conditions occur during the execution of a program. Nondeclarative procedures are procedures that are executed according to the normal flow of control within a program. Declaratives can contain nondeclarative procedures, but these are executed only during the execution of the declarative that contains them. Nondeclarative procedures can contain other nondeclarative procedures but must not contain a declarative. Neither declaratives nor nondeclarative procedures can contain programs. In other words, in COBOL the terms "procedure" and "program" are not synonyms.

At the object level, the most inclusive unit of organization of computer programs is the run unit. A run unit is a complete problem solution consisting of an object program or of several intercommunicating object programs. A run unit is an independent entity that can be executed without communicating with, or being coordinated with, any other run unit except that it can process data files and messages or set and test switches that were written or will be read by other run units.

When a program is called, parameters upon which it is to operate can be passed to it by the program that calls it. Because any separately compiled program can be the first program executed in a run unit, the first executed program of a run unit can receive parameters.

A run unit can also contain object code and data storage areas derived from the compilation of programs written in languages other than COBOL.

Nested Source Programs

A COBOL source program is a syntactically correct set of COBOL statements. A COBOL source program can contain other COBOL source programs and these contained programs can reference some of the resources of the program within which they are contained.

When a program, program B, is contained in another program, program A, it can be directly contained or indirectly contained. Program B is directly contained in program A if there is no program contained in program A that also contains program B. Program B is indirectly contained in program A if a program contained in program A also contains program B.

Following is an example of a directly contained program B and an indirectly contained program B.l



Accessing Data and Files

Some data items have associated with them a storage concept determining where data item values and other attributes of data items are represented with respect to the programs of a run unit. Likewise, file connectors have associated with them a storage concept determining where information concerning the positioning and status of a file and other attributes of file processing are represented with respect to the programs of a run unit.

Global Names and Local Names

A data name names a data item. A file name names a file connector. These names are classified as either global or local.

A global name can be used to refer to the object it is associated with either from within the program in which the global name is declared or from within any other program contained in the program that declares the global name.

A local name, however, can be used only to refer to the object it is associated with from within the program in which the local name is declared. Some names are always global; other names are always local; and some other names are either local or global, depending on specifications in the program in which the names are declared.

A record name is global if the GLOBAL clause is specified in the record description entry by which the record name is declared or, in the case of record description entries in the File Section, if the GLOBAL clause is specified in the file description entry for the file name associated with the record description entry. A data name is global if the GLOBAL clause is specified either in the data description entry by which the data name is declared or in another entry to which that data description entry is subordinate. A condition name declared in a data description entry is global if that entry is subordinate to another entry in which the GLOBAL clause is specified. However, specific rules sometimes prohibit specification of the GLOBAL clause for certain data description, file description, or record description entries.

A file name is global if the GLOBAL clause is specified in the file description entry for that file name.

If a data name, a file name, or a condition name declared in a data description entry is not global, the name is local.

Global names are transitive across programs contained within other programs.

Object Types

- Working-Storage records

  Working-storage records are allocations of sufficient storage to satisfy the record description entries in that section. Each record description entry in a program declares a different object. Renaming and redefining do not declare new objects; they provide alternate groupings or descriptions for objects that have already been declared.

- File connectors

  File connectors are storage areas that contain information about a file and are used as the linkage between a file name and a physical file and between a file name and its associated record area. The storage areas containing this information are allocated to the run unit when the run unit opens the file. If multiple compilation units open the same file, the original storage area is used and no additional storage is allocated to the run unit.

- Record areas for files

  No particular record description entry in the File Section is considered to declare the storage area for the record. Rather, the storage area is the maximum required to satisfy associated label record description entries. These entries can describe fixed or variable- length records. In this presentation, record description entries are said to be associated in two cases. First, when record description entries are subordinate to the same file description entry, they are always associated. Second, when record description entries are subordinate to different file description entries and these file description entries are referenced in the same SAME RECORD AREA clause, they are associated. All associated record description entries are redefinitions of the same storage area.

- Other objects

  Examples of other objects declared in NPE COBOL programs are: storage areas in the Linkage, Common-Storage, and Report Sections.

External Objects and Internal Objects

Accessible data items usually require that certain representations of data be stored. File connectors usually require that certain information concerning files be stored. The storage associated with a data item or a file connector can be external or internal to the program in which the object is declared.

A data item or file connector is external if the storage associated with that object is associated with the run unit rather than with any particular program within the run unit. An external object can be referred to by any program in the run unit that describes the object. References to an external object from different programs using separate descriptions of the object are always to the same object. In a run unit, there is only one representation of an external object.

An object is internal if the storage associated with that object is associated only with the program that describes the object.

Internal and external objects can have either global or local names.

A data record described in the Working Storage Section is given the external attribute by the presence of the EXTERNAL clause in its data description entry. Any data item described by a data description entry subordinate to an entry describing an external record also attains the external attribute. If a record or data item does not have the external attribute, it is part of the internal data of the program in which it is described.

A file connector is given the external attribute by the presence of the EXTERNAL clause in the associated file description entry. If the file connector does not have the external attribute, it is internal to the program in which the associated file name is described.

The data records described subordinate to a file description entry that does not contain the EXTERNAL clause or a sort-merge file description entry, as well as any data items described subordinate to the data description entries for such records, are always internal to the program describing the file name. If the EXTERNAL clause is included in the file description entry, the data records and the data items attain the external attribute. A data record described in the Common-Storage Section is accessible to all programs in the run unit.

Data records, subordinate data items, and various associated control information described in the Linkage and Report Sections of a program are always considered to be internal to the program describing that data. Special considerations apply to data described in the Linkage Section whereby an association is made between the data records described and other data items accessible to other programs.

Name Resolution

Certain conventions apply when programs contained within other programs assign the same names to data items, conditions, and file connectors. Consider the situation when program A contains program B, which itself contains program C; further, programs A and B, but not program C, contain Data Division entries for a condition name, data name, or a file name named DUPLICATE-NAME.

1. If either DUPLICATE-NAME references an internal object, two different though identically named objects exist. If both DUPLICATE-NAMEs reference an external object, only one object exists.
2. Program A's reference to DUPLICATE-NAME is always to the object that it declares. Program B's reference to DUPLICATE-NAME is always to the object that it declares.
3. If DUPLICATE-NAME is a local name in both programs A and B, program C cannot refer to that name.
4. If DUPLICATE-NAME in program B is a global name, program C can access the object referred to by the name in program B, regardless of whether or not DUPLICATE-NAME is a global name in program A.
5. If DUPLICATE-NAME in program A is a global name but in program B it is a local name, program C's reference to DUPLICATE-NAME is to the object referred to by the name declared in program A.

Common Programs and Initial Programs

All programs that form part of a run unit can possess none, one, or both of the following attributes: common and initial.

A common program is one that, despite being directly contained within another program, can be called by any program directly or indirectly contained in that other program. The common attribute is attained by specifying the COMMON clause in a program's Identification Division. The COMMON clause facilitates the writing of subprograms that are to be used by all the programs contained within a program.

An initial program is one whose program state is initialized when the program is called. Therefore, whenever an initial program is called, its program state is the same as when the program was first called in that run unit. During the process of initializing an initial program, that program's internal data is initialized; therefore, an item of the program's internal data whose description contains a VALUE clause is initialized to that defined value, but an item whose description does not contain a VALUE clause is initialized to an undefined value. Files with internal file connectors associated with the program are not in the open mode. The control mechanisms for all PERFORM statements contained in the program are set to their initial states. The initial attribute is attained by specifying the INITIAL clause in the program's Identification Division.

Interprogram Communications

When the complete solution to a data processing problem is subdivided into more than one program, the constituent programs must be able to communicate with each other. This communication can take four forms: the transfer of control, the passing of parameters, the reference to common data, and the reference to common files.

These four interprogram communication forms are provided both when the communicating programs are separately compiled and when one of the communicating programs is contained within the other program. The precise mechanisms provided in the last two cases differ from those in the first two cases; for example, a program contained within another program can refer to any data name or file name possessing a global name in the containing program.

Transfer of Control

The CALL statement provides the means whereby control can be transferred from one program to another within a run unit. A called program can itself contain CALL statements.

When control is transferred to a called program, execution proceeds from statement to statement beginning with the first nondeclarative statement. If control reaches a STOP RUN statement, this signals the logical end of the run unit. If control reaches an EXIT PROGRAM statement, this signals the logical end of the called program only, and control then reverts to the next executable statement following the CALL statement in the calling program. Therefore, the EXIT PROGRAM statement terminates only the execution of the program in which it occurs, while the STOP RUN statement terminates the execution of a run unit.

The CALL statement can be used to call a program that is not written in COBOL. A COBOL program can also be called from a program that is not written in COBOL.

Names of Programs

In order to call a program, a CALL statement identifies the program's name. The names assigned to programs that are contained directly or indirectly within another program must be unique.

The names assigned to each of the separately compiled programs that constitute a run unit must also be unique.

Scope of the CALL Statement

In the following, the calling program can or cannot possess any of the program attributes; it can either be separately compiled or not; and it can either be contained within programs or contain other programs:

- Any calling program can call any separately compiled program in the run unit.
- A calling program can call any program that is directly contained within the calling program.

- Any calling program can call any program possessing the common attribute that is directly contained within a program that itself directly or indirectly contains the calling program, unless the calling program is itself contained within the program possessing the common attribute.
- A calling program can call a program that neither possesses the common attribute nor is separately compiled only if that program is directly contained within the calling program.

Scope of Names of Programs

Within a separately compiled program, certain conventions apply when a name identical to that specified for another separately compiled program in the run unit is specified for a contained program.

Consider the situation when program A contains program B and program DUPLICATE-NAME, program B contains program BB, and program DUPLICATE-NAME contains program DD.

The name DUPLICATE-NAME has also been specified for a separately compiled program.

1. If program A, but not any of the programs it contains, calls program DUPLICATE-NAME, the program activated is the one contained within program A.
2. If either program B or program BB calls program DUPLICATE-NAME, one of two events take place:
   a. If the program DUPLICATE-NAME contained within program A possesses the common attribute, it is called.
   b. If the program DUPLICATE-NAME contained within program A does not possess the common attribute, the separately compiled program is called.
3. If either program DD or program DUPLICATE-NAME contained within program A calls program DUPLICATE-NAME, the program called is the separately compiled program.
4. If any other separately compiled program in the run unit or any other program contained within such a program calls the program DUPLICATE-NAME, the program called is the separately compiled program named DUPLICATE-NAME.

Passing Parameters to Programs

A program calls another program in order to have the called program perform, on behalf of the calling program, some defined part of the solution of a data processing problem. In many cases it is necessary for the calling program to define the called program. The precise part of the problem solution is to be executed by making certain data values required by the called program available for it. One method for ensuring the availability of these data values is by passing parameters to a program, as is described in this paragraph. Another method is to share the data (see Section 7.2.3). The data values passed as parameters also can identify some data to be shared; therefore, the two methods are not mutually exclusive.

- Identifying parameters

  A value to be passed as a parameter by a program calling another program must be accessible to the calling program and be declared as a data item in the Data Division of the called program. In the called program the parameters required are identified by listing references to the names assigned, in that program's data description entries, to the parameters in that program's Procedure Division header. In the calling program, the values of the parameters to be passed by the calling program are identified by listing references in the CALL statement used to call the called program.

  These lists establish, on a positional basis at object time, the correspondence between the values as they are known to each program; that is, the first parameter on one list corresponds to the first parameter on the other, the second to the second, and so forth. Therefore, a program that can be called by another program can include

```
PROGRAM-ID.  EXAMPLE.

PROCEDURE DIVISION USING NUM, PCODE, COST.
```

  and can be called by executing

```
CALL "EXAMPLE" USING NBR, PTYPE, PRICE.
```

  thereby establishing the following correspondence:

Called program (EXAMPLE)	Calling Program
NUM	NBR
PCODE	PTYPE
COST	PRICE

Only the positions of the data names are significant, not the names themselves.

- Values of parameters

  The calling program controls the methods by which a called program evaluates the values of the parameters passed to it and by which the called program returns results as modified parameter values.

  The individual parameters referred to in the CALL statement's USING phrase can be passed either by reference or by content. A called program is allowed to access and modify the value of the data item referred to in the calling program's CALL statement as a parameter passed by reference. This permission to access and modify a data item in the calling program is denied to the called program if the data item is specified in the CALL statement as a parameter passed by content. The value of the parameter is evaluated when the CALL statement is executed and is presented to the called program. This value can be changed by the called program during the course of its execution, but the value of the corresponding data item in the calling program is not modified. Therefore, a parameter passed by reference can be used by a called program to return a result to the calling program, whereas a parameter passed by content cannot be so used.

The parameters referred to in a called program's Procedure Division header must be described in the Linkage Section of that program's Data Division.

Sharing Data

Two programs in a run unit can reference common data in the following circumstances:

- The data content of an external data record can be referred to from any program provided that program has described that data record.
- If a program is contained within another program, both programs can refer to data possessing the global attribute either in the containing program or in any program that directly or indirectly contains the containing program.
- The mechanism whereby a parameter value is passed by reference from a calling program to a called program establishes a common data item; the called program, which can use a different identifier, can refer to a data item in the calling program.
- The data content of a common-storage data record can be referred to from any program, provided that program has described that data record in the Common-Storage Section.

Sharing Files

Two programs in a run unit can refer to common file connectors in the following circumstances:

- An external file connector can be referred to from any program that describes that file connector (see Section 7.2.3 and Section 7.2.3).
- If a program is contained within another program, both programs can refer to a common file connector by referring to an associated global file name in the containing program or in any program that directly or indirectly contains the containing program (see Section 7.2.3).
- If two or more programs in a run unit each refer to the system input-output file through the file control paragraph, the programs share the same system files. This file sharing occurs even though they have not been declared as either external or global.

  Note: *Unexpected results may occur when accessing the same file from different programs within the same run unit. For example, assume program A and program B each refer to the same system output file. Program A opens the file and calls program B. Program B opens the same file, for output, writes a record to the file and closes the file before returning to program A. Program A then attempts to write a record to the output file. If the length of the record to be written is larger than 132 characters, this write attempt will not work.*

Scope of Names

When programs are directly or indirectly contained within other programs, each program can use identical user-defined words to name objects independent of the use of these user-defined words by other programs. (See Section 2.4.7)

When identically named objects exist, a program's reference to such a name, even when it is a different type of user-defined word, is to the object that program describes, rather than to the object possessing the same name described in another program.

The following types of user-defined words can be referred to only by statements and entries in that program in which the user-defined word is declared:

- Paragraph-name
- Section-name

The following types of user-defined words can be referred to by any COBOL program, provided the entities referred to are known to the compiler:

- Library name
- Text name

The following types of names, when they are declared within a Configuration Section, can be referred to only by statements and entries either in that program containing a Configuration Section or in any program contained within that program:

- Alphabet name
- Class name
- Condition name
- Mnemonic name
- Symbolic character

Specific conventions for declarations and references apply to the following types of user-defined words when the conditions listed above do not apply:

- Condition name
- Data name
- File name
- Index name
- Program name
- Record name
- Report name

Conventions for Program Names

The program name of a program is declared in the PROGRAM-ID paragraph of the program's Identification Division. A program name can be referred to only by the CALL statement, the CANCEL statement, and the end program header. The program names allocated to programs constituting a run unit are not necessarily unique, but when two programs in a run unit are identically named, at least one of those two programs must be directly or indirectly contained within another separately compiled program that does not contain the other of those two programs.

The following rules regulate the scope of a program name:

1. If the program name is that of a program that does not possess the common attribute and that is directly contained within another program, that program name can be referred to only by statements included in that containing program.
2. If the program name is that of a program that possesses the common attribute and that is directly contained within another program, that program name can be referred to only by statements included in that containing program and any programs directly or indirectly contained within that containing program, except the program possessing the common attribute and any programs contained within it.
3. If the program name is that of a program that is separately compiled, that program name can be referred to by statements included in any other program in the run unit, except programs it directly or indirectly contains.

Conventions for Condition Names, Data Names, File Names, Record Names, and Report Names

When condition names, data names, file names, record names, and report names are declared in a source program, these names can be referred to only by that program, except when one or more of the names is global and the program contains other programs.

The requirements governing the uniqueness of the names allocated by a single program to be condition names, data names, file names, record names, and report names are explained in Section 2.4.7.

A program cannot reference any condition name, data name, file name, record name, or report name declared in any program it contains.

A global name can be referenced in the program in which it is declared or in any programs that are directly or indirectly contained within that program.

When a program, program B, is directly contained within another program, program A, both programs can define a condition name, a data name, a file name, a record name, or a report name using the same user-defined word. When such a duplicated name is referenced in program B, the following rules are used to determine the referenced object:

1. The set of names to be used for determination of a referenced object consists of all names that are defined in program B and all global names that are defined in program A and in any programs that directly or indirectly contain program A. Using this set of names, the normal rules for qualification and any other rules for uniqueness of reference are applied until one or more objects is identified.
2. If only one object is identified, it is the referenced object.
3. If more than one object is identified, no more than one of them can have a name local to program B. If zero or one of the objects has a name local to program B, the following rules apply:
    a. If the name is declared in program B, the object in program B is the referenced object.
    b. Otherwise, if program A is contained within another program, the referenced object is
        - The object in program A if the name is declared in program A.
        - The object in the containing program if the name is not declared in program A and is declared in the program containing program A. This rule is applied to further containing programs until a single valid name has been found.
4. The object in program A if the name is declared in program A.
5. The object in the containing program if the name is not declared in program A and is declared in the program containing program A. This rule is applied to further containing programs until a single valid name has been found.

Conventions for Index Names

If a data item possessing either or both the external or global attributes includes a table accessed with an index, that index also possesses the same external or global attributes depending on the data item attributes. Therefore, the scope of an index name is identical to that of the data name that names the table whose index is named by that index name and the scope of name rules for data names apply. Index names cannot be qualified.

7.3. Nested Source Programs

This subsection describes nested source programs and their organization.

7.3.1. General Description

A COBOL source program is a syntactically correct set of COBOL statements. A COBOL source program can contain other COBOL source programs and these contained programs can reference some of the resources of the program within which they are contained.

7.3.2. Organization

With the exception of COPY and REPLACE statements and the end program header, the statements, entries, paragraphs, and sections of a COBOL source program are grouped into four divisions that are sequenced in the following order:

1. The Identification division
2. The Environment division
3. The Data division
4. The Procedure division

The end of a COBOL source program is indicated either by the end program header, if specified, or by the absence of additional source program lines.

7.3.3. Structure

The following gives the general format and order of presentation of the entries and statements that constitute a COBOL source program. The generic terms identification division, environment division, data division, procedure division, source program, and end program header represent a COBOL Identification Division, a COBOL Environment Division, a COBOL Data Division, a COBOL Procedure Division, a COBOL source program, and a COBOL end program header, respectively.

General Format

```
identification-division
[procedure-division]
[source-program] . . .
[end-program-header]
```

Syntax Rules

The end program header must be present if:

1. The COBOL source program contains one or more additional COBOL source programs.
2. The COBOL source program is contained within another COBOL source program.

General Rules

1. The beginning of a division in a program is indicated by the appropriate division header. The end of a division is indicated by one of the following:
   a. The division header of a succeeding division in that program
   b. An Identification Division header that indicates the start of another source program
   c. The end program header
   d. That physical position after which no more source program lines occur

2. A COBOL source program that is directly or indirectly contained within another program is considered in these specifications as a separate program that can additionally refer to certain resources defined in the containing program.
3. The object code, resulting from compiling a source program contained within another program, is considered in these specifications to be inseparable from the object code resulting from compiling the containing program.

7.3.4. Initial State of a Program

The initial state of a program is the state of a program the first time it is called in a run unit.

Characteristics of a Program

1. The program's internal data contained in the Working-Storage Section are initialized. If a VALUE clause is used in the description of the data item, the data item is initialized to the defined value. If a VALUE clause is not associated with a data item, the initial value of the data item is undefined.
2. Files with internal file connectors associated with the program are not in the open mode.
3. The control mechanisms for all PERFORM statements contained in the program are set to their initial states.
4. A GO TO statement, referred to by an ALTER statement contained in the same program, is set to its initial state.

Programs in the Initial State

A program is in the initial state

- The first time the program is called in a run unit
- The first time the program is called after the execution of a CANCEL statement referencing the program or a CANCEL statement referencing a program that directly or indirectly contains the program
- Every time the program is called, if it possesses the initial attribute
- The first time the program is called after the execution of a CALL statement referencing a program that possesses the initial attribute and that directly or indirectly contains the program
- When a GO TO statement referred to by an ALTER statement contained in the same program is set to its initial state

7.3.5. End Program Header

Function

The end program header indicates the end of the named COBOL source program.

General Format

```
END PROGRAM program-name.
```

Syntax Rules

1. *Program-name* must conform to the rules for forming a user-defined word.
2. *Program-name* must be identical to a program name declared in a preceding PROGRAM-ID paragraph. (See Section 7.4)
3. If a PROGRAM-ID paragraph declaring a specific program name is stated between the PROGRAM-ID paragraph and the end program header declaring and referring to, respectively, another program name, the end program header referring to the former program name must precede that referring to the latter program name.

General Rules

1. The end program header must be present in every program that either contains or is contained within another program.
2. The end program header indicates the end of the specified COBOL source program.
3. If the program terminated by the end program header is contained within another program, the next statement must be either an Identification Division header or another end program header that terminates the containing program.
4. If the program terminated by the end program header is not contained within another program and if the next source statement is a COBOL statement, it must be the Identification Division header of a program to be compiled separately from that program terminated by the end program header.

7.4. Identification Division

This subsection describes the Identification Division and its associated PROGRAM-ID paragraph using nested source programs.

Function

The PROGRAM-ID paragraph specifies the name by which a program is identified and assigns selected program attributes to that program.

General Format



Syntax Rules

1. *Program-name* must conform to the rules for formation of a user-defined word.
2. A program contained within another program must not be assigned the same name as that of any other program contained within the separately compiled program that contains this program.

3. Use the optional COMMON clause only if the program is contained within another program.

General Rules

1. *Program-name* identifies the source program, the object program, and all listings pertaining to a particular program.
2. The COMMON clause specifies that the program is common. A common program is contained within another program but can be called from programs other than those containing it. (See Section 7.2.3.)
3. The INITIAL clause specifies that the program is initial. When an initial program is called, it and any programs contained within it are placed in their initial state.

7.5. Environment Division

This subsection describes the Environment Division and its associated I-O-Control paragraph using the APPLY clause.

Function

The APPLY clause specifies which files and working-storage records and data areas are to be shared during execution among two or more separately compiled programs.

General Format



Syntax Rules

1. *Identifier-1* . . . must specify a file name, or Working-Storage level 01 data-name and must be unique in the first 12 characters.
2. *Identifier-1* can be the object of only one APPLY EXDEF clause in a run unit.
3. *Identifier-1* can occur only once in an APPLY clause of a program.
4. *Identifier-1* can be the object of more than one APPLY EXREF clause in a run unit.
5. *Identifier-1* in an APPLY EXREF clause must specify the same type of item as was specified in its associated APPLY EXDEF clause.
6. *Identifier-1* in an APPLY clause must not be the subject of an EXTERNAL or a GLOBAL clause.
7. An APPLY clause cannot be specified in a nested program or a program that contains the INITIAL clause.
8. *Identifier-1* must not be a DMS data item.

General Rules

1. The tables and data areas associated with the identifier in the APPLY EXDEF clause are accessible by programs that specify the same identifier in an APPLY EXREF clause.
2. The file associated with each file name and SELECT clause in the APPLY EXDEF clause must be equivalent to the description associated with that same identifier when specified in an APPLY EXDEF clause.
3. All declarative procedures associated with a file must be specified in the same program that has the APPLY EXDEF for that file.
4. The APPLY EXDEF clause must be specified in the main program for any file that potentially invokes a declarative procedure. If a file name for a report file is referenced by an APPLY EXDEF clause, any reference to that same file name in an APPLY EXREF clause in another program must also be for a report file.
5. The identifier specified in the APPLY EXREF clause must not have a REDEFINES clause at the record level. It can, however, be the object of a REDEFINES clause and can contain subordinate items that are redefined.
6. The identifier specified in the APPLY EXREF clause must define the same number of character positions as the same identifier when specified in the APPLY EXDEF clause. It is important to note that the APPLY EXREF clause specifies the redefinition of a storage area, not of the data items occupying the area.
7. When APPLY EXREF is specified, the VALUE clause must not be present in that entry.
8. The APPLY EXDEF clause must be specified in the main program for any IBM compatible tape file that contains an OCCURS DEPENDING ON data item.
9. If a file specified in the APPLY EXDEF clause has a FILE STATUS associated with it, one of the following rules must be used to insure proper reference to that data item in any subprogram:
    a. Define the FILE STATUS data item in the COMMON-STORAGE SECTION of all programs that reference it.
    b. If the file is the object of an EXDEF clause in the main program, define the FILE STATUS in WORKING-STORAGE SECTION. In the subprograms that reference it, define it in the LINKAGE SECTION.

7.6. Data Division

This subsection describes the Data Division and its associated sections and entries.

7.6.1. Linkage Section

The Linkage Section is located in the Data Division of a source program. The Linkage Section appears in the called program and describes data items that are to be referred to by the calling program and the called program.

The Linkage Section in a program is meaningful only if the object program is to function under the control of a CALL statement and the CALL statement in the calling program contains a USING phrase.

The Linkage Section is used for describing data that is available through the calling program but is to be referred to in both the calling and the called program. See Section 7.7.1 and Section 7.7.2 for a description of the mechanism by which a correspondence is established between the data items described in the Linkage Section of a called program and data items described in the calling program. In the case of index names, no such correspondence is established and index names in the called and calling programs always refer to separate indexes.

The structure of the Linkage Section is the same as that previously described for the Working-Storage Section, beginning with a section header, followed by noncontiguous data items and record description entries.

The general format of the Linkage Section is as follows:

```
LINKAGE SECTION.
  [ 77-level-description-entry  ] ...
  [ record-description-entry    ]
```

If a data item in the Linkage Section is accessed in a program that is not a called program, the effect is undefined.

Noncontiguous Linkage Storage

Items in the Linkage Section that bear no hierarchical relationship to one another need not be grouped into records and are classified and defined as noncontiguous elementary items. Each of these items is defined in a separate data description entry that begins with the special level number 77.

The following data clauses are required in each data description entry:

- Level number 77
- Data name
- The PICTURE or the USAGE IS INDEX, COMPUTATIONAL-1, or COMPUTATIONAL-2 clause.

Other data description clauses are optional and can be used to complete the description of the item if necessary.

Linkage Records

Data elements in the Linkage Section that bear a definite hierarchical relationship to one another must be grouped into records according to the rules for formation of record descriptions. Data elements in the Linkage Section that bear no hierarchical relationship to any other data item can be described as records that are single elementary items.

Initial Values

The VALUE clause must not be specified in the Linkage Section except in condition name entries (level 88).

7.6.2. Common-Storage Section

The Common-Storage Section is a block of main storage that is shared by all COBOL programs in the run unit that define a Common-Storage Section.

The structure and permissible data definition statements for Common-Storage are identical to the Working-Storage Section.

If Common-Storage Sections are of different sizes, the total area allocated for an execution will be sufficient for the largest Common-Storage Section.

The main storage area shared by two or more COBOL Common-Storage Sections can also be shared by assembly language blank common, which is specified by the INFO directive.

The common areas can be used to conserve main storage by providing a "scratch pad" area that each main program and subprogram can use for itself, or it can be used to allow one program to address data that belongs to another program.

If Common-Storage is used for this purpose, the user must take the necessary actions to ensure that data field allocation and alignment are consistent among all programs that share the area. The use of the COPY statement is recommended for this function.

When using Common-Storage, you should define program and subprogram Common-Storage areas identically, including initial values. This avoids potential problems when using the Dynamic Linker. The Dynamic Linker loads the Common-Storage area (bank $BLANK$COMMON$) from the first OM encountered during execution that has COMMON-STORAGE. An execution problem can occur if the Common-Storage area size of that program is defined smaller than that of any subprograms that are called subsequently. The Static Linker, however, avoids the execution problem by utilizing the largest Common-Storage size defined by the input OMs included in a linked-up executable. This is done regardless of whether the largest size is that of a main program or a subprogram.

If a DMS application places records, data names or DMCA into COMMON using the INVOKE clause, then it is imperative that the Common-Storage Section be defined identically in the main program and in all subprograms. The same INVOKE statement except for the ERROR and ROLLBACK clauses should be used in all of them. If the DMCA is placed into COMMON using the INVOKE clause, the compiler also adds an area at the end of the Common-Storage Section to contain the DMS data structure D$WORK. This data structure is positioned after all user variables in the Common-Storage Section. If the Common-Storage Section of all programs in an executable is not identical, then the offsets for D$WORK are different. This can cause D$WORK to overlay user variables, and can result in execution failure. However, the D$WORK-WS keyword option can be used to force the D$WORK data structure to WORKING-STORAGE, thereby bypassing any potential problems that can occur if the layout of the COMMON-STORAGE Section is not the same in all programs in an executable. Therefore, it is recommended that DMS COBOL programs should always be compiled with the D$WORK-WS keyword option. Refer to *UCS COBOL Programming Reference Manual Volume 2* for the description of the D$WORK-WS

keyword option in subsection 2.2.7 “Compiler Call Keyword Options for Program Control”.

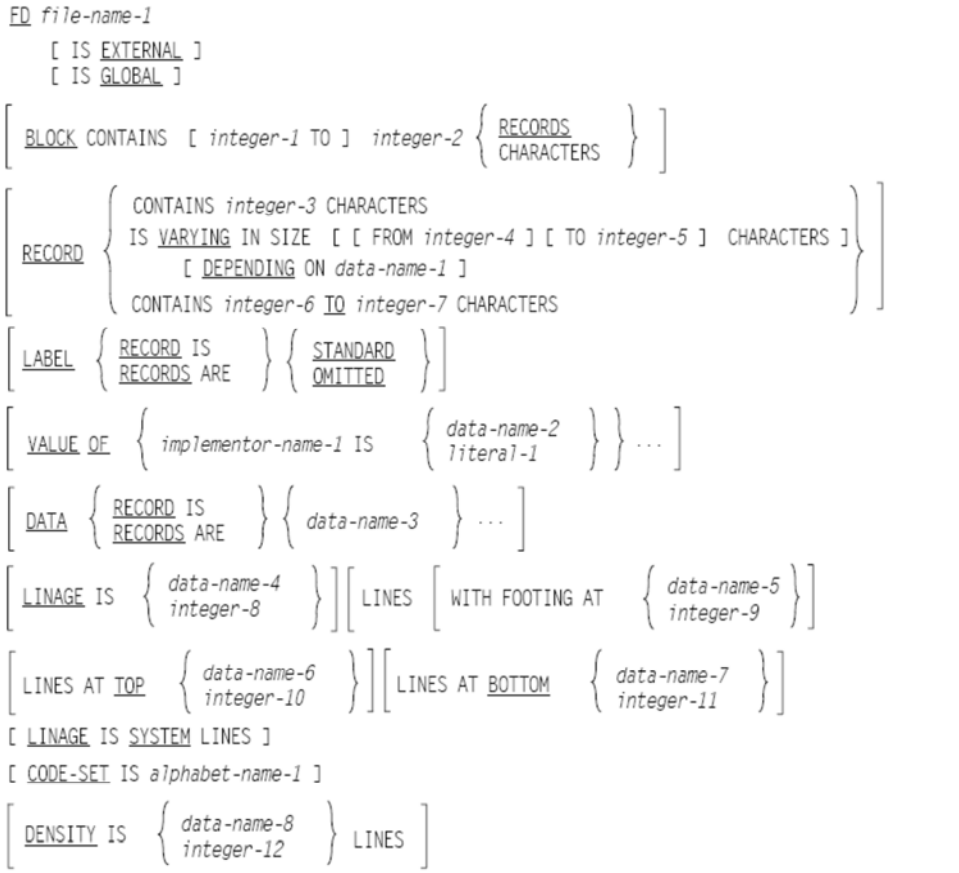
The UCS COBOL compiler allocates one extra word of storage for each common block declared in the program. This permits the compiler to generate more efficient coding sequences in some cases. The extra word difference can be noted in the Linker Information Listing and in the bank sizes lines, as the size of the common block banks is one larger than the common block declared in the program. See the *UCS COBOL Programming Reference Manual Volume 2* for more information.

In addition, when statically linking COBOL object modules and MASM object modules that both declare the same common block, the Linking System issues a warning diagnostic if the declared common block sizes do not match. To prevent the diagnostic, declare the common block size in the MASM routine one word larger than the declared size of the corresponding common block in the COBOL program.

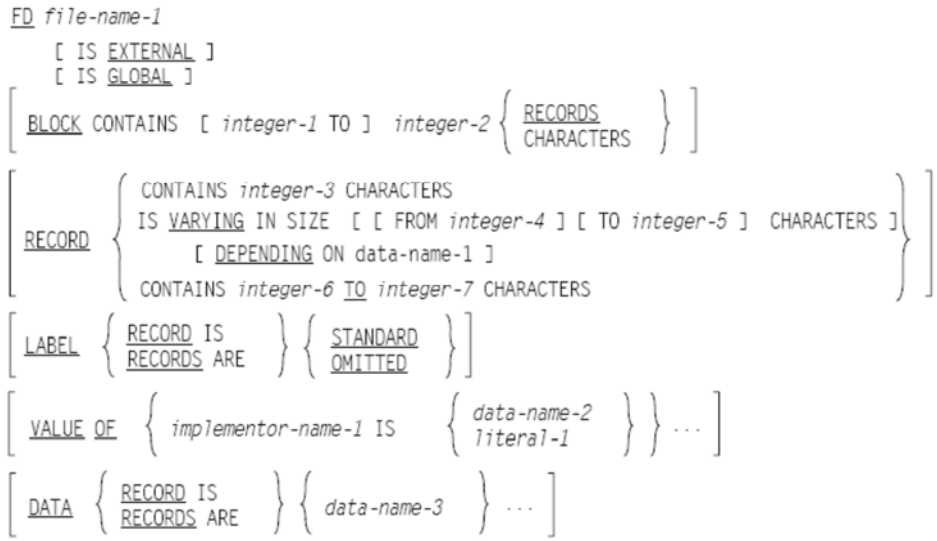
7.6.3. File Description Entry

Within the Interprogram Communication module, the file description entry in the File Section determines the internal or external attributes of a file connector, of the associated data records, and of the associated data items. The file description entry also determines whether a file name is a local name or a global name.

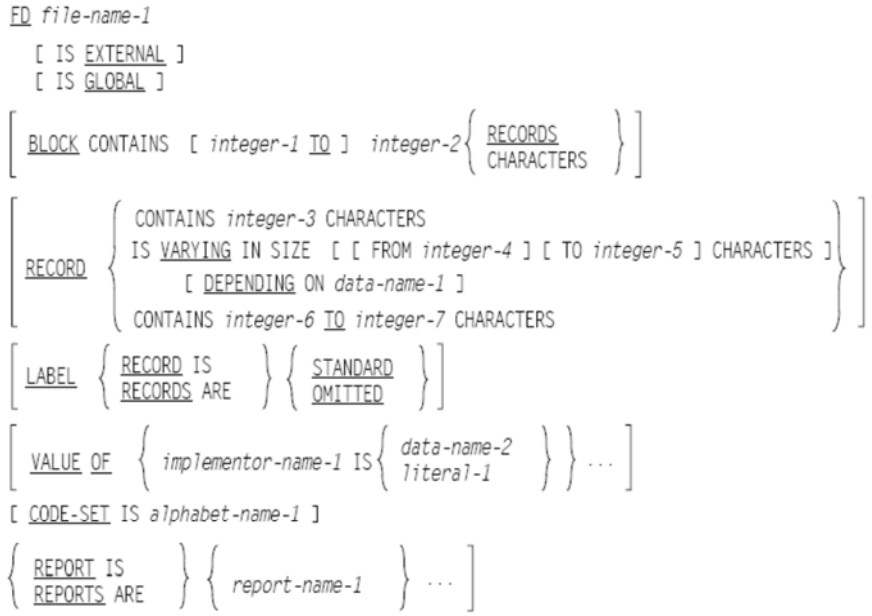
Format 1



Format 2



Format 3



Syntax Rules

1. Format 1 is the file description entry for a sequential file. (See Section 4.4.5.)
2. Format 2 is the file description entry for a relative file or an indexed file. (See Section 5.4.2 and Section 6.4.2.)
3. Format 3 is the file description entry for a report file. (See Section 10.3.1)

General Rules

1. If the file description entry for a sequential file contains the LINAGE clause and the EXTERNAL clause, the LINAGE-COUNTER data item is an external data item. If the file description entry for a sequential file contains the LINAGE clause and the GLOBAL clause, the special register LINAGE-COUNTER is a global name.
2. The EXTERNAL clause is presented in Section 2. Th e GLOBAL clause is presented in Section 2. Al l other clauses in the file description entry are presented in the appropriate module within these specifications.

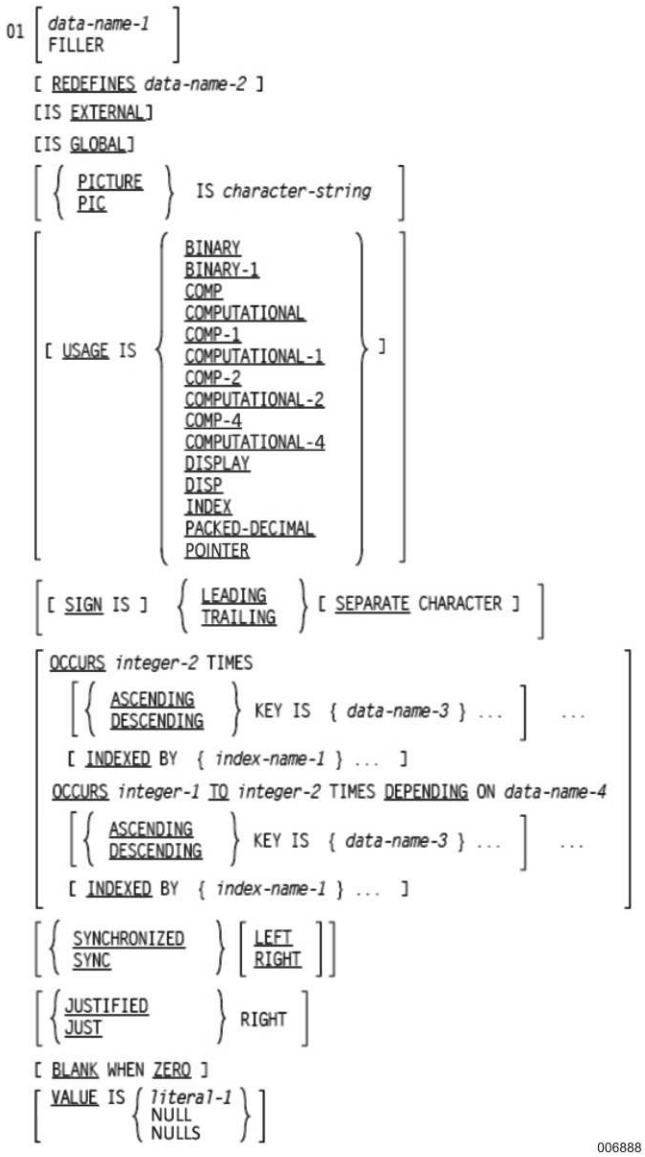
7.6.4. The Data Description Entry

Function

Within the Interprogram Communication module, a level 01 data description entry within the Working-Storage Section or File Section determines whether the data record and its subordinate data items have local names or global names.

Within the Interprogram Communication module, a level 01 data description entry in the Working-Storage Section determines the internal or external attribute of the data record and its subordinate data items.

General Format



Syntax Rules

1. The EXTERNAL clause can be specified only in data description entries in the Working-Storage Section whose level number is 01.
2. The EXTERNAL clause and the REDEFINES clause must not be specified in the same data description entry.
3. The GLOBAL clause can be specified only in data description entries whose level number is 01.

4. *Data-name-1* must be specified for any entry containing the GLOBAL or EXTERNAL clause or for record descriptions with a file description entry that contains the EXTERNAL or GLOBAL clause.
5. The syntax rules for all other clauses in the data description entry are presented in the Nucleus module. (See Section 3.5.)

General Rule

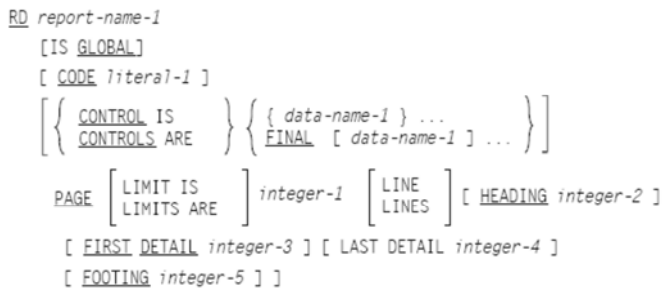
The EXTERNAL clause is presented in Section 2. Th e GLOBAL clause is presented in Section 2. Al l other clauses in the data description entry are presented in the Nucleus module within these specifications.

7.6.5. Report Description Entry

Function

Within the Interprogram Communication module, the report description entry in the Report Section determines whether a report name is a local name or a global name.

General Format



Syntax Rule

No specific syntax rule applies to the GLOBAL clause. The syntax rules for all other clauses in the report description entry are presented in the Report Writer module. (See Section 10.3.2)

General Rules

1. If the report description entry contains the GLOBAL clause, the special registers LINE-COUNTER and PAGE-COUNTER are global names.
2. The GLOBAL clause is presented in Section 2. Th e general rules for all other clauses in the report description entry are presented in the Report Writer module. (See Section 10.3.2)

7.6.6. The EXTERNAL Clause

Function

The EXTERNAL clause specifies that a data item or a file connector is external. The constituent data items and group items of an external data record are available to every program in the run unit that describes that record.

General Format

```
IS EXTERNAL
```

Syntax Rules

1. Specify the EXTERNAL clause only in file description entries (see Section 7.6.3) or in record description entries in the Working-Storage Section (see Section 7.6.4).
2. In the same program, the data name specified as the subject of the entry whose level number is 01 that includes the EXTERNAL clause must not be the same data name specified for any other data description entry that includes the EXTERNAL clause.
3. Do not use the VALUE clause in any data description entry that includes or is subordinate to any entry that includes the EXTERNAL clause. The VALUE clause can be specified for condition name entries associated with such data description entries.

General Rules

1. The data contained in the record named by the data name clause is external and can be accessed and processed by any program in the run unit that describes and, optionally, redefines it subject to the following general rules.
2. Within a run unit, if two or more programs describe the same external data record, each record name of the associated record description entries must be the same and the records must define the same number of standard data format characters. However, a program that describes an external record can contain a data description entry including the REDEFINES clause that redefines the complete external record, and this complete redefinition need not occur identically in other programs in the run unit. (See Section 3.5.12.)
3. Use of the EXTERNAL clause does not imply that the associated file name or data name is a global name. (See Section 2.)
4. T he file connector associated with this description entry is an external file connector.
5. Storage created by the EXTERNAL clause is common storage. For a description of common storage (See Section 7.6.2).

7.6.7. The GLOBAL Clause

Function

The GLOBAL clause specifies that a data name, a file name, or a report name is a global name. A global name is available to every program contained within the program that declares it.

General Format

```
IS GLOBAL
```

Syntax Rules

1. Specify the GLOBAL clause only in data description entries whose level number is 01 in the File Section or the Working-Storage Section, file description entries, or report description entries.
2. In the same Data Division, the data description entries for any two data items for which the same data name is specified must not include the GLOBAL clause.
3. If the SAME RECORD AREA clause is specified for several files, the record description entries or the file description entries for these files must not include the GLOBAL clause.

General Rules

1. A data name, file name, or report name described using a GLOBAL clause is a global name. All data names subordinate to a global name are global names. All condition names associated with a global name are global names.
2. A statement in a program contained directly or indirectly within a program that describes a global name can refer to that name without describing it again (see Section 7.2.3).
3. If you use a GLOBAL clause in a data description entry that contains either the REDEFINES clause or the RENAMES clause, it is only the subject of that REDEFINES or RENAMES clause, as appropriate that possesses the global attribute.

7.7. Procedure Division

This subsection describes the Procedure Division and its associated statements.

7.7.1. The Procedure Division Header

The Procedure Division is identified by, and must begin with, the following header:

```
PROCEDURE DIVISION
        [ USING { data-name-1 } . . . ]
        [ WITH ENTRY POINTS { procedure-name-1 } . . . ]    .
```

The USING phrase is necessary only if the object program is to be invoked by a CALL statement and that statement includes a USING phrase.

The USING phrase of the Procedure Division header identifies the names used by the program for any parameters passed to it by a calling program. The parameters passed to a called program are identified in the USING phrase of a calling program's CALL statement. The correspondence between the two lists of names is established on a positional basis.

Data-name-1 must be defined as a level 01 entry or a level 77 entry in the Linkage Section. A particular user-defined word cannot appear more than once as *data-name-1*. The data description entry for *data-name-1* must not contain a REDEFINES clause. *Data-name-1* can, however, be the object of a REDEFINES clause elsewhere in the Linkage Section.

The following additional rules apply:

1. If the reference to the corresponding data item in the CALL statement declares the parameter to be passed by content, and the parameter is identified as a data name, the value of this item is moved when the CALL statement is executed and placed into the system-defined storage item possessing the attributes declared in the Linkage Section for *data-name-1*. The data description of each parameter in the BY CONTENT phrase of the CALL statement must be the same (meaning no conversion, extension, or truncation) as the data description of the corresponding parameter in the USING phrase of the Procedure Division header. (See Section 7.7.2.)
2. If the reference to the corresponding data item in the CALL statement declares the parameter to be passed by reference, the object program operates as if the data item in the called program occupies the same storage area as the data item in the calling program. The description of the data item in the called program must describe the same number of character positions described by the corresponding data item in the calling program.
3. At all times in the called program, references to *data-name-1* are resolved in accordance with the description of the item given in the Linkage Section of the called program.

4. Data items defined in the Linkage Section of the called program can be referred to within the Procedure Division of that program only if they satisfy one of the following conditions:
   a. They are operands of the USING phrase of the Procedure Division header.
   b. They are subordinate to operands of the USING phrase of the Procedure Division header.
   c. They are defined with a REDEFINES or RENAMES clause, the object of which satisfies the above conditions.
   d. They are items subordinate to any item that satisfies the condition in rule 4c.
   e. They are condition names or index names associated with data items that satisfy any of the preceding four conditions.
5. A COBOL program that can be called by another program within the same run unit is normally executed starting with the first nondeclarative paragraph in the program. To have execution begin at other than the first nondeclarative statement, the ENTRY POINTS phrase is required.

The procedure names cannot be contained in the declaratives portion of a program. The procedure names must not be qualified and must be unique in the first 26 characters. The specified procedure names must not conflict with any program name or with any external data item. In addition, procedure names in the ENTRY POINTS phrase must not conflict with the procedure names in the ENTRY POINTS phrase, program names, or external data items for other programs within the run unit.

The execution of a CALL or ENTER statement that refers to a procedure name in the ENTRY POINTS phrase transfers control to the first executable statement following the specified name in the program.

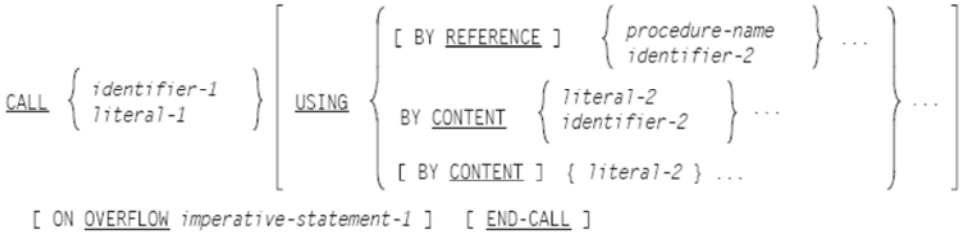
The use of entry point names does not permit recursive calling of a program.

7.7.2. The CALL Statement

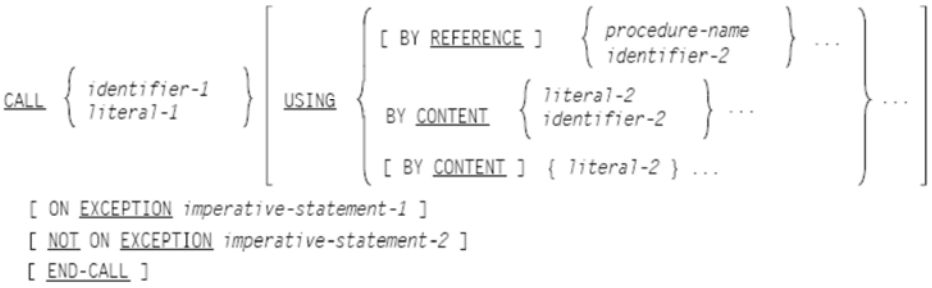
Function

The CALL statement transfers control from one object program to another within the run unit.

Format 1



Format 2



Syntax Rules

1. *Literal-1* must be a nonnumeric literal.
2. Define identifier-1 as an alphanumeric data item such that its value can be a program name or a procedure name specified on the ENTRY POINTS phrase.
3. Each of the operands in the USING phrase, other than literals or procedure names, must have been defined as a data item in the File Section, Working-Storage Section, or Linkage Section, and must be a level-01 data item, a level-77 data item, a group item subordinate to level-01 data item, or an elementary data item.
4. *Identifier-2* must not be a function identifier.

General Rules

1. *Literal-1,* or the contents of the data item referred to by *identifier-1,* is the program-ID of the called program or a procedure name specified in the ENTRY POINTS phrase of the called program's procedure division header. The program in which the CALL statement appears is the calling program.
2. If, when a CALL statement is executed, the program specified by the CALL statement is made available for execution, control is transferred to the called program. After control is returned from the called program, the ON OVERFLOW or ON EXCEPTION phrase, if specified, is ignored and control is transferred to the end of the CALL statement or, if the NOT ON EXCEPTION phrase is specified, to imperative-statement-2. If control is transferred to imperative-statement-2, execution continues according to the rules for each statement specified in imperative-statement-2. If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of imperative-statement-2, control is transferred to the end of the CALL statement.

3. If it is determined when a CALL statement is executed that the program specified by the CALL statement cannot be made available for execution at that time, one of the two actions listed below occurs:

   a. If the ON OVERFLOW or ON EXCEPTION phrase is specified in the CALL statement, control is transferred to *imperative-statement-1.* Execution then continues according to the rules for each statement specified in *imperative-statement-1.* If a procedure branching or conditional statement that causes explicit transfer of control is executed, control is transferred in accordance with the rules for that statement; otherwise, upon completion of the execution of *imperative-statement-1,* control is transferred to the end of the CALL statement and the NOT ON EXCEPTION phrase, if specified, is ignored.

   b. If the ON OVERFLOW or ON EXCEPTION phrase is not specified in the CALL statement, the NOT ON EXCEPTION phrase, if specified, is ignored. An error message will be issued from the Linking System and the program will abort.

4. Two or more programs in the run unit can have the same program name, and the reference in a CALL statement to such a program name is resolved by using the scope of names conventions for program names. (See Section 7.2.3)

   For example, when only two programs in the run unit have the same name as that specified in a CALL statement:

   a. One of those two programs must also be contained directly or indirectly either within the separately compiled program that includes that CALL statement or within the separately compiled program that itself directly or indirectly contains the program that includes that CALL statement.

   b. The other of those two programs must be a different, separately compiled program.

      The mechanism used in this example is as follows:

      - If one of the two programs having the same name as that specified in the CALL statement is directly contained within the program that includes that CALL statement, that program is called.
      - If one of the two programs having the same name as that specified in the CALL statement possesses the common attribute and is directly contained within another program that directly or indirectly contains the program that includes the CALL statement, that common program is called unless the calling program is contained within that common program.
      - Otherwise, the separately compiled program is called.

5. If the called program does not possess the initial attribute, it, and each program directly or indirectly contained within it, is in its initial state the first time it is called within a run unit and the first time it is called after a CANCEL to the called program.

   On all other entries into the called program, the state of the program and each program directly or indirectly contained within it remains unchanged from its state when last exited.

6. If the called program possesses the initial attribute, and each program is directly or indirectly contained within it then it is placed into its initial state every time the called program is called within a run unit.
7. Files associated with a called program's internal file connectors are not in the open mode when the program is in an initial state. (See Section 7.3.4.)

   On all other entries into the called program, the status and positioning of all such files is the same as when the called program was last exited.
8. The process of calling a program or exiting from a called program does not alter the status or positioning of a file associated with any external file connector.
9. If the program being called is a COBOL program, the USING phrase is included in the CALL statement only if there is a USING phrase in the Procedure Division header of the called program, in which case the number of operands in each USING phrase must be identical.
10. The sequence of appearance of data names in the USING phrase of the CALL statement and in the corresponding USING phrase in the called program's Procedure Division header determines the correspondence between the data names used by the calling and called programs. This correspondence is positional and not by name equivalence; the first data name in one USING phrase corresponds to the first data name in the other, the second to the second, and so forth.
11. The values of the parameters referred to in the USING phrase of the CALL statement are made available to the called program at the time the CALL statement is executed.
12. Both the BY CONTENT and BY REFERENCE phrases are transitive across the parameters that follow them until another BY CONTENT or BY REFERENCE phrase is encountered. If neither the BY CONTENT nor the BY REFERENCE phrase is specified before the first parameter, the BY REFERENCE phrase is assumed, for identifiers and procedure names, and the BY CONTENT is assumed for literals.
13. If the BY REFERENCE phrase is [either specified] or implied for a parameter, the object program operates as if the corresponding data item in the called program occupies the same storage area as the data item in the calling program. The description of the data item in the called program must describe the same number of character positions as described by the description of the corresponding data item in the calling program.
14. If the BY REFERENCE phrase is either specified or implied, and the parameter is a procedure name, the called procedure is treated as a function, and a parameter is returned that represents the procedure name to which the called procedure requires the calling program to go. See Volume 2 for an example of how this is used.
15. If the BY CONTENT phrase is specified or implied for a parameter, the called program cannot change the value of this parameter as referred to in the CALL statement's USING phrase, although the called program can change the value of the data item referred to by the corresponding data name in the called program's Procedure Division header.

The data description of each parameter in the BY CONTENT phrase of the CALL statement must be the same (meaning no conversion, extension, or truncation) as the data description of the corresponding parameter in the USING phrase of the Procedure Division header. (See Section 7.7.1.)

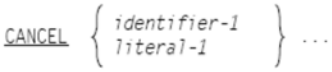
16. Called programs can contain CALL statements. However, a called program must not execute a CALL statement that directly or indirectly calls the calling program. If a CALL statement is executed within the range of a declarative, that CALL statement cannot directly or indirectly refer to any called program to which control has been transferred and has not completed execution.
17. The END-CALL phrase delimits the scope of the CALL statement.

7.7.3. The CANCEL Statement

Function

The CANCEL statement ensures that the next time the referenced program is called, it must be in its initial state.

General Format



Syntax Rules

1. *Literal-1* must be a nonnumeric literal.
2. *Identifier-1* must refer to an alphanumeric data item.

General Rules

1. *Literal-1* or the content of the data item referred to by *identifier-1* identifies the program to be canceled.
2. Subsequent to the execution of an explicit or implicit CANCEL statement, the program referred to therein ceases to have any logical relationship to the run unit in which the CANCEL statement appears. If the program referenced by a successfully executed explicit or implicit CANCEL statement in a run unit is subsequently called in that run unit, that program is in its initial state. (See Section 7.3.4.)
3. A program named in a CANCEL statement in another program must be callable by that other program. (See Section 7.2.3 and Section 7.7.2.)
4. All programs contained within a program named in a CANCEL statement are canceled when the CANCEL statement is executed.
5. A program named in the CANCEL statement must not refer directly or indirectly to any program that has been called and has not yet executed an EXIT PROGRAM statement.
6. A logical relationship to a canceled subprogram is established only by execution of a subsequent CALL statement naming that program.

7. A called program is canceled either by being referred to as the operand of a CANCEL statement, by the termination of the run unit of which the program is a member, or by execution of an EXIT PROGRAM statement in a called program that possesses the initial attribute.
8. No action is taken when an explicit or implicit CANCEL statement is executed naming a program that has not been called in this run unit or has been called and is at present canceled. Control is transferred to the next executable statement following the explicit CANCEL statement.
9. The contents of data items in external data records described by a program or in Common-Storage are not changed when that program is canceled.
10. During execution of a CANCEL statement, an implicit CLOSE statement without any optional phrases is executed for each file in the open mode that is associated with an internal file connector in the program named in the explicit CANCEL statement. Any USE procedures associated with any of these files are not executed.
11. The CANCEL statement can be used to cause the local data space of a previously called HVTIP bank subprogram to be freed and, when next called, to be reallocated and reinitialized. A "local" subprogram must not be explicitly cancelled, but is automatically cancelled upon the cancellation of its HVTIP bank entry-point subprogram. Common banked subprograms are handled as though they were locally contained in the HVTIP bank making the call.
12. CANCEL is not supported in the standard transaction processing environment.

7.7.4. The ENTER Statement

Function

The ENTER statement transfers control to a non-COBOL program within the same run unit.

Format 1

```
ENTER  { MASM } literal-1 [ USING { argument-1 } ... ]
```

Format 2

```
ENTER MASM "C$INFO" USING file-name, data-name.
```

Syntax Rules

Format 1

1. *Literal-1* must conform to the requirements for entry points for the languages specified.
2. Arguments for MASM can be literals, identifiers, and procedure names.
   a. Numeric literals are assumed to be fixed-point binary, right-justified in single or double precision as necessary. Floating-point literals are assumed to be double-precision floating point, and nonnumeric literals are assumed to be ASCII.

b. Identifiers can be any data items that are referred to in the calling program. Do not use level 88 items as arguments. Subscripts must be integer values.

c. Procedure names must contain at least one nonnumeric character to avoid conflict with numeric literal arguments.

Format 2

1. C$INFO is an entry-point name defined by Unisys.
2. The *file-name* parameter must refer to either an indexed file or a sequential file assigned to DISC.
3. The *data-name* parameter must be defined as a level 01 entry in the File Section or Working-Storage Section.

General Rules

Format 1

1. The program whose entry point is specified by the value of the literal is the called program; the program in which the ENTER statement appears is the calling program. The literal must not be "C$INFO".
2. Execution of an ENTER statement causes control to pass to the called program.
3. A called program must not contain a statement that directly or indirectly calls the calling program.
4. See Volume 2 for a description of preserving the UCS environment, calling sequences, and parameter format.

Format 2

1. If the name is an indexed file, the following rules apply:
   a. The file specified by *file-name* must be in the open mode at the time the statement is executed.
   b. The data name specified in the parameter list refers to an area into which a copy of the Multikeyed Indexed Access Method (MSAM) Information Statistics Area will be stored.
   c. If the parameter list is in error, control is returned to the calling program without changing the contents of *data-name.*
   d. If the file represented by *file-name* is not an MSAM file or the file is not in the open mode, the contents of *data-name* are cleared to zero and control is transferred to the calling program.
   e. The successful execution of the ENTER MASM 'C$INFO' statement causes the data item referenced by *data-name* to be initialized to the current contents of the MSAM information statistics table for the file referenced by *file-name*. The format of the data item referenced by the *data-name* is defined with the following COBOL record description:

```
01  group-data-name.
    10  data-name-0  PIC 9(10) BINARY.
```

The data item referenced by *data-name-0* contains the total number of records written when the file was initially created (between execution of an OPEN OUTPUT statement and execution of the first CLOSE statement for the file).

```
10  data-name-1  PIC 9(10) BINARY.
```

The data item referenced by *data-name-1* contains the total number of records in the file before the execution of the immediately preceding OPEN statement.

```
10  data-name-2  PIC 9(10) BINARY.
```

The data item referenced by data-name-2 contains the total number of records written before the execution of the immediately preceding OPEN statement.

```
10  data-name-3  PIC 9(10) BINARY.
```

The data item referenced by data-name-3 contains the total number of records read before the execution of the immediately preceding OPEN statement.

```
10  data-name-4  PIC 9(10) BINARY.
```

The data item referenced by data-name-4 contains the total number of records deleted before the execution of the immediately preceding OPEN statement.

```
10  data-name-5  PIC 9(10) BINARY.
```

The data item referenced by data-name-5 contains the total number of records modified before the execution of the immediately preceding OPEN statement.

```
10  data-name-6  PIC 9(10) BINARY.
```

The data item referenced by *data-name-6* contains the total number of start requests before the execution of the immediately preceding OPEN statement.

```
10  data-name-7  PIC 9(10) BINARY.
```

The data item referenced by *data-name-7* contains the total number of keys modified before the execution of the immediately preceding OPEN statement.

```
10  data-name-8  PIC 9(10) BINARY.
```

The data item referenced by *data-name-8* contains the total number of invalid key exits before the execution of the immediately preceding OPEN statement.

```
10  data-name-9  PIC 9(10) BINARY.
```

The data item referenced by *data-name-9* contains the total number of data blocks before the execution of the immediately preceding OPEN statement.

```
10  data-name-10  PIC 9(10) BINARY.
```

The data item referenced by *data-name-10* contains the total number of index blocks before the execution of the immediately preceding OPEN statement.

```
10  data-name-11  PIC 9(10) BINARY.
```

The data item referenced by *data-name-11* contains the total number of times that the file has been closed before the execution of the preceding OPEN statement.

```
10  data-name-12  PIC 9(10) BINARY.
```

The data item referenced by *data-name-12* contains the number of records written since the execution of the immediately preceding OPEN statement.

```
10  data-name-13  PIC 9(10) BINARY.
```

The data item referenced by *data-name-13* contains the number of records read since the execution of the immediately preceding OPEN statement.

```
10  data-name-14  PIC 9(10) BINARY.
```

The data item referenced by *data-name-14* contains the number of records deleted since the execution of the immediately preceding OPEN statement.

```
10  data-name-15  PIC 9(10) BINARY.
```

The data item referenced by *data-name-15* contains the number of records modified since the execution of the immediately preceding OPEN statement.

```
10  data-name-16  PIC 9(10) BINARY.
```

The data item referenced by *data-name-16* contains the number of start requests since the execution of the immediately preceding OPEN statement.

```
10  data-name-17  PIC 9(10) BINARY.
```

The data item referenced by *data-name-17* contains the number of keys modified since the execution of the immediately preceding OPEN statement.

```
10  data-name-18  PIC 9(10) BINARY.
```

The data item referenced by *data-name-18* contains the number of invalid key exits since the execution of the immediately preceding OPEN statement.

```
 10  data-name-19 PIC 9(10) BINARY.
```

The data item referenced by *data-name-19* contains the number of new data blocks created since the execution of the immediately preceding OPEN statement.

```
 10  data-name-20 PIC 9(10) BINARY.
```

The data item referenced by *data-name-20* contains the number of new index blocks created since the execution of the immediately preceding OPEN statement.

```
10  data-name-21 PIC 9(10) BINARY.
```

The data item referenced by *data-name-21* contains the number of blocks read since the execution of the immediately preceding OPEN statement.

```
10  data-name-22 PIC 9(10) BINARY.
```

The data item referenced by *data-name-22* contains the number of blocks written since the execution of the immediately preceding OPEN statement.

f. See the *PCIOS Administration and Programming Reference Manual* for a description of the Information Statistics Area.

2. If *file-name* is a sequential file, *data-name* must refer to a 01/77 PIC 9(10) USAGE BINARY elementary item. After execution of a format-2 ENTER statement, this data name contains the number of active (not DELETED) records in the file.

7.7.5. The EXIT PROGRAM Statement

Function

The EXIT PROGRAM statement marks the logical end of a called program.

General Format

```
EXIT PROGRAM
```

Syntax Rules

1. If an EXIT PROGRAM statement appears in a consecutive sequence of imperative statements within a sentence, it must appear as the last statement in that sequence.
2. The EXIT PROGRAM statement must not appear in a declarative procedure in which the GLOBAL phrase is specified.

General Rules

1. If the EXIT PROGRAM statement is executed in a program that is not under the control of a calling program, the EXIT PROGRAM statement causes execution of the program to continue with the next executable statement.
2. The execution of an EXIT PROGRAM statement in a called program that does not possess the initial attribute causes execution to continue with the next executable statement following the CALL statement in the calling program. The program state of the calling program is not altered and is identical to that which existed at the time it executed the CALL statement except that the contents of data items and the contents of data files shared between the calling and called program can have been changed. The program state of the called program is not altered except that the ends of the ranges of all PERFORM statements executed by that called program are considered to have been reached.
3. Besides the actions specified in general rule 2, the execution of an EXIT PROGRAM statement in a called program that possesses the initial attribute is also equivalent to executing a CANCEL statement referring to that program (see Section 7.7.3).

7.7.6. The GOBACK Statement

Function

The GOBACK statement provides a common end point for a series of procedures or a permanent suspension of the execution run unit. The GOBACK statement functions like the EXIT statement as part of a called program and like the STOP RUN statement when coded in a main program.

General Format

GOBACK

Syntax Rule

The GOBACK statement must be the only statement in a sentence. If used in a series of imperative statements, it must be the last statement in the series.

General Rules

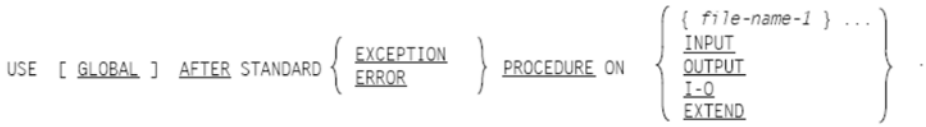
1. If a GOBACK statement is encountered in a called program, control returns to the statement in the calling program immediately following the CALL statement that initiated the called program. Thus, in a subprogram, the GOBACK statement acts in the same way as an EXIT statement.
2. If a GOBACK statement is used in a main program, it is equivalent to a STOP RUN statement. Thus, it indicates the logical end of the run unit, and control is passed to the operating system.

7.7.7. The USE Statement

Function

Within the Interprogram Communication module, the USE statement determines whether the associated declarative procedures are invoked during the execution of any program contained within the program that includes the USE statement.

General Format



Syntax Rule

Syntax rules for the USE statement are found in the Sequential I-O module, Relative I-O module, or Indexed I-O module. (See Section 4.5.6, Section 5.5.8, or Section 6.5.8.)

General Rules

1. Special precedence rules are followed when programs are contained within other programs. In applying these rules, only the first qualifying declarative will be selected for execution. The declarative selected for execution must satisfy the rules for execution of that declarative. The order of precedence for selecting a declarative is:

a. The declarative within the program that contains the statement causing the qualifying condition.

b. The declarative in which the GLOBAL phrase is specified and is within the program directly containing the program that was last examined for a qualifying declarative.

c. Any declarative selected by applying general rule 1b to each more inclusive containing program until general rule 1b is applied to the outermost program. If no qualifying declarative is found, none is executed.

2. If the USE statement references *file-name-1,* and *file-name-1* is the subject of an APPLY EXREF clause, the declarative will be invoked if the condition specified occurs in another separately compiled program that has *file-name-1* as the subject of an APPLY EXREF clause. A USE statement cannot reference a file name that is the subject of an APPLY EXREF clause in the same separate compilation.

7.7.8. The USE BEFORE REPORTING Statement

Function

Within the Interprogram Communication module, the USE BEFORE REPORTING statement determines whether the associated declarative procedures are invoked during the execution of any program contained within the program that includes the USE BEFORE REPORTING statement.

General Format

```
USE [GLOBAL] BEFORE REPORTING identifier-1.
```

Syntax Rule

Syntax rules for the USE BEFORE REPORTING statement are found in the Report Writer module. (See Section 10.4.9.)

General Rules

Special precedence rules are followed when programs are contained within other programs. In applying these rules, only the first qualifying declarative will be selected for execution. The declarative selected for execution must satisfy the rules for execution of that declarative. The order of precedence for selecting a declarative is:

1. The declarative within the program that contains the statement causing the qualifying condition.
2. The declarative in which the GLOBAL phrase is specified and is within the program directly containing the program that was last examined for a qualifying declarative.
3. Any declarative selected by applying general rule 1b to each more inclusive containing program until general rule 1b is applied to the outermost program. If no qualifying declarative is found, none is executed.

Section 8
Sort-Merge Module

8.1. General

The Sort-Merge module can order one or more files of records or can combine two or more identically ordered files of records, according to a set of user-specified keys contained within each record. Optionally, a user can apply some special processing to each of the individual records by input or output procedures. This special processing can be applied before and after the records are ordered by the SORT, or after the records have been combined by the MERGE.

Sort and Merge statements can be performed in a culturally sensitive manner. For details of how this can be accomplished, see Section 8.8.

8.2. Language Concepts

This subsection describes sort file and merge file language concepts.

8.2.1. Sort File

A sort file is a collection of records to be sorted by a SORT statement. The sort file has no label procedures that the programmer can control, and the rules for blocking and for allocation of internal storage are peculiar to the SORT statement. The RELEASE and RETURN statements imply nothing with respect to buffer areas, blocks, or reels. A sort file, then, can be considered as an internal file that is created (RELEASE statement) from the input file, processed (SORT statement), and then made available (RETURN statement) to the output file.

A sort file is named by a file control entry and is described by a sort-merge file description entry. A sort file is referred to by the RELEASE, RETURN, and SORT statements.

8.2.2. Merge File

A merge file is a collection of records to be merged by a MERGE statement. The merge file has no label procedures that the programmer can control, and the rules for blocking and for allocation of internal storage are peculiar to the MERGE statement. The RETURN statement implies nothing with respect to buffer areas, blocks, or reels. A merge file, then, can be considered as an internal file that is created from input files by combining them (MERGE statement) as the file is made available (RETURN statement) to the output file.

A merge file is named by a file control entry and is described by a sort-merge file description entry. A merge file is referred to by the RETURN and MERGE statements.

8.3. Environment Division/Input-Output Section

The Input-Output Section is located in the Environment Division of a source program. The Input-Output Section deals with the information needed to control transmission and handling of data between external media and the object program. The Input-Output Section is optional in the Environment Division of a COBOL source program. The general format of the Input-Output Section is shown below.

General Format

```
INPUT-OUTPUT SECTION.
FILE-CONTROL.  {file-control-entry} ...
[I-O-CONTROL.  [input-output-control-entry] ]
```

8.3.1. The FILE-CONTROL Paragraph

Function

The FILE-CONTROL paragraph allows specification of file-related information.

General Format

```
FILE-CONTROL. { file-control-entry}. . .
```

8.3.2. The File Control Entry

Function

The file control entry declares the relevant physical attributes of a sort or merge file.

General Format

```
SELECT  file-name-1  ASSIGN TO { implementor-name-1 } ...
                               { literal-1          }
```

Syntax Rules

1. Each sort or merge file described in the Data Division must be specified only once in the FILE-CONTROL paragraph. Each sort or merge file specified in the SELECT clause must have a sort-merge file description entry in the Data Division of the same program.
2. Since *file-name-1* represents a sort or merge file, only the ASSIGN clause is permitted to follow *file-name-1* in the FILE-CONTROL paragraph.
3. *Implementor-name-1* or *literal-1* must conform to the rules specified for sequential, relative, or indexed files. It is for documentation only.

General Rule

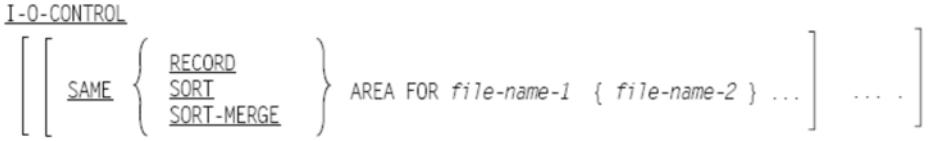
The ASSIGN clause specifies the association of the file referred to by *file-name-1* to a storage medium referred to by *implementor-name-1*. It is for documentation only.

8.3.3. The I-O-CONTROL Paragraph

Function

The I-O-CONTROL paragraph specifies the main storage area that is to be shared by different files, including sort or merge files.

General Format



Syntax Rule

The availability of the RECORD option of the SAME clause is described in 4.3.3.3.

General Rule

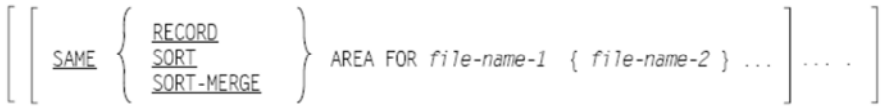
The SAME RECORD/SORT/SORT-MERGE AREA clause for the Sort-Merge module is presented in Section 8.3.4.

8.3.4. The SAME RECORD/SORT/SORT-MERGE AREA Clause

Function

The SAME RECORD/SORT/SORT-MERGE AREA clause specifies the storage area that is to be shared by different files, at least one of which is a sort or merge file.

General Format



Syntax Rules

1. Each file name specified in the SAME RECORD/SORT/SORT-MERGE AREA clause must be specified in the FILE-CONTROL paragraph of the same program.
2. *File-name-1* and *file-name-2* cannot refer to an external file connector.
3. SORT and SORT-MERGE are equivalent.
4. A file name that represents a sort or merge file must not appear in the SAME clause unless the SORT, SORT-MERGE, or RECORD phrase is used.
5. More than one SAME clause can be included in the program, subject to the following restrictions:
   a. A file name must not appear in more than one SAME RECORD AREA clause.
   b. A file name that represents a sort or merge file must not appear in more than one SAME SORT AREA or SAME SORT-MERGE AREA clause.
   c. If a file name that does not represent a sort or merge file appears in a SAME AREA clause (see Section 4.3.3) and one or more SAME SORT AREA or SAME SORT-MERGE AREA clauses, all of the files named in that SAME AREA clause must be named in the SAME SORT AREA or SAME SORT-MERGE AREA clauses.
6. The files referred to in the SAME RECORD/SORT/SORT-MERGE AREA clause need not all have the same organization or access.

General Rules

1. The SAME RECORD AREA clause specifies that two or more files referred to by *file-name-1, file-name-2* are to use the same storage area for processing of the current logical record. All of these files can be in the open mode at the same time. A logical record in the SAME RECORD AREA is considered as a logical record of each file open in the output mode whose file name appears in this SAME RECORD AREA clause and of the most recently read file open in the input mode whose file name appears in this SAME RECORD AREA clause. This is equivalent to an implicit redefinition of the area; that is, records are aligned on the leftmost character position.
2. If the SAME SORT AREA or SAME SORT-MERGE AREA clause is used, at least one of the file names must represent a sort or merge file. This clause specifies that storage is shared as follows:
   a. The SAME SORT AREA or SAME SORT-MERGE AREA clause specifies a storage area that is made available for use in sorting or merging each sort or merge file named. Therefore, any storage area allocated for the sorting or merging of a sort or merge file is available for reuse in sorting or merging any of the other sort or merge files.
   b. In addition, storage areas assigned to files that do not represent sort or merge files can be allocated as needed for sorting or merging the sort or merge files named in the SAME SORT AREA or SAME SORT-MERGE AREA clause.
   c. Files other than sort or merge files do not share the same storage area with each other. For these files to share the same storage area with each other, the program must contain a SAME AREA or SAME RECORD AREA clause specifying file names associated with these files.

d. During the execution of a SORT or MERGE statement that refers to a sort or merge file named in this clause, any nonsort or nonmerge files associated with file names named in this clause must not be in the open mode.

8.4. The Data Division/File Section

The File Section is located in the Data Division of a source program. The File Section defines the structure of sort files and merge files. Each sort file or merge file is defined by a sort-merge file description entry and one or more record description entries. Record description entries are written immediately following the sort-merge file description entry. Here is the general format of the File Section in the Sort-Merge module:

General Format

```
FILE SECTION.
[sort-merge-file-description-entry
{record-description-entry} . . . ] . . .
```

8.4.1. Sort-Merge File Description Entry

In a COBOL program, the sort-merge file description entry (SD entry) represents the highest level of organization in the File Section. The File Section header is followed by a sort-merge file description entry consisting of a level indicator (SD), a file name, and a series of independent clauses. The clauses of a sort-merge file description entry (SD entry) specify the size and the names of the data records associated with a sort file or a merge file. There are no label procedures that the user can control, and the rules for blocking and internal storage are peculiar to the SORT and MERGE statements. The sort-merge file description entry is terminated by a period.

8.4.2. Record Description Structure

A record description consists of a set of data description entries describing the characteristics of a particular record. Each data description entry consists of a level number followed by the data name or FILLER clause, if specified, followed by a series of independent clauses, as required. A record description can have a hierarchical structure; therefore, the clauses used with an entry can vary considerably, depending on whether or not it is followed by subordinate entries. The structure of a record description and the clauses allowed in a record description entry are explained in Section 2.4.15 and Section 3.5.

8.4.3. Initial Values

The initial values of data items in the File Section are undefined.

8.4.4. The Sort-Merge File Description Complete Entry Skeleton

Function

The sort-merge file description furnishes information concerning the physical structure, identification, and record names of the file to be sorted or merged.

General Format

```
SD file-name-1
[ RECORD { CONTAINS integer-1 CHARACTERS
         { IS VARYING IN SIZE [ [ FROM integer-2 ] [ TO integer-3 ] CHARACTERS ]
         {    [ DEPENDING ON data-name-1 ]
         { CONTAINS integer-4 TO integer-5 CHARACTERS                          } ]
[ DATA { RECORD IS  } { data-name-2 } ... ]
       { RECORDS ARE }
```

Syntax Rules

1. The level indicator SD identifies the beginning of the sort-merge file description entry and must precede *file-name-1.*
2. The clauses that follow *file-name-1* are optional and can appear in any order.
3. One or more record description entries must follow the sort-merge file description entry; however, no input-output statements can be executed for this sort or merge file.

General Rules

1. The DATA RECORDS clause for the Sort-Merge module is the same as the DATA RECORDS clause for the Sequential I-O module. (See Section 4.4.5). The DATA RECORDS clause is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted from the next revision of American National Standard COBOL.
2. The RECORD clause for the Sort-Merge module is the same as the RECORD clause for the Sequential I-O module. (See Section 4.4.5)

8.5. Procedure Division

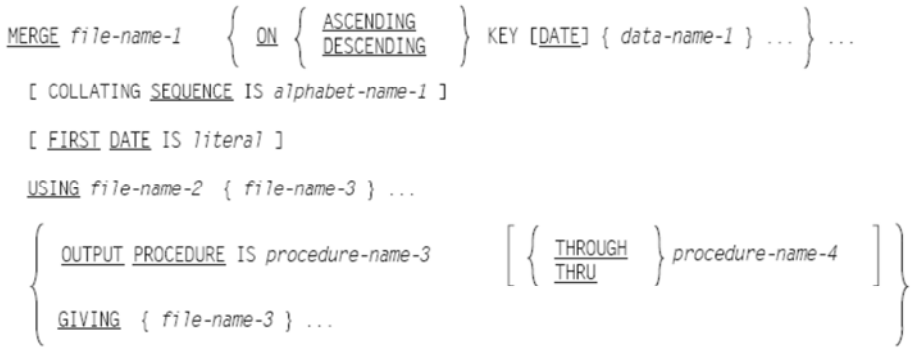
This subsection describes statements associated with the Procedure Division.

8.5.1. The MERGE Statement

Function

The MERGE statement combines two or more identically sequenced files on a set of specified keys, and during the process makes records available, in merged order, to an output procedure or to an output file.

General Format



Syntax Rules

1. A MERGE statement can appear anywhere in the Procedure Division except in the declaratives portion.
2. *File-name-1* must be described in a sort-merge file description entry in the Data Division.
3. If the file referred to by *file-name-1* contains variable-length records, the size of the records contained in the file referred to by *file-name-2* and *file-name-3* must neither be less than the smallest record nor greater than the largest record described for *file-name-1.* If the file referred to by *file-name-1* contains fixed-length records, the size of the records contained in the files referred to by *file-name-2* and *file-name-3* must not be greater than the largest record described for *file-name-1.*
4. *Data-name-1* is a key data name. Key data names are subject to the following rules:
   a. The data items identified by key data names must be described in records associated with *file-name-1*.
   b. Key data names can be qualified.
   c. The data items identified by key data names must not be group items that contain variable occurrence data items.
   d. If *file-name-1* has more than one record description, the data items identified by key data names need be described in only one of the record descriptions. The same character positions referred to by a key data name in one record description entry are taken as the key in all records of the file.
   e. None of the data items identified by key data names can be described by an entry that either contains an OCCURS clause or is subordinate to an entry that contains an OCCURS clause.
   f. If the file referred to by *file-name-1* contains variable-length records, all the data items identified by key data names must be contained within the first x character positions of the record, where x equals the minimum record size specified for the file referred to by *file-name-1*.
5. *File-name-2, file-name-3,* and *file-name-4* must be described in a file description entry, not in a sort-merge file description entry, in the Data Division.

6. No two files specified in any one MERGE statement can reside on the same multiple file reel.
7. File names must not be repeated within the MERGE statement.
8. No pair of file names in a MERGE statement can be specified in the same SAME AREA, SAME SORT AREA, or SAME SORT-MERGE AREA clause. The only file names in a MERGE statement that can be specified in the same SAME RECORD AREA clause are those associated with the GIVING phrase. (See Section 4.3.3 and Section 8.3.4.)
9. The words THRU and THROUGH are equivalent.
10. If file-name-4 references an indexed file, the first specification of data-name-1 must be associated with an ASCENDING phrase and the data item referenced by that data-name-1 must occupy the same character positions in its record as the data item associated with the prime record key for that file.
11. If the GIVING phrase is specified and the file referred to by file-name-4 contains variable-length records, the size of the records contained in the file referred to by file-name-1 must neither be less than the smallest record nor greater than the largest record described for file-name-4. If the file referred to by file-name-4 contains fixed-length records, the size of the records contained in the file referred to by file-name-1 must not be greater than the largest record described for file-name-4.
12. The literal must be a one- or two-digit integer in the range 0 through 99.
13. If a DATE reserved word is specified, the KEY data item that immediately follows is assumed to be a date key and the following restrictions and conditions exist:
    a. The data item must be described as a USAGE DISPLAY item that contains at least two characters.
    b. If the data item is numeric it must be unsigned.

General Rules

1. The MERGE statement merges all records contained on the files referred to by *file-name-2* and *file-name-3*.
2. If the file referred to by *file-name-1* contains only fixed-length records, any record in the file referred to by *file-name-2* or *file-name-3* containing fewer character positions than that fixed-length is space-filled on the right beginning with the first character position after the last character in the record when that record is released to the file referred to by *file-name-1*.
3. The data names following the word KEY are listed from left to right in the MERGE statement in order of decreasing significance without regard to how they are divided into KEY phrases. The leftmost data name is the major key, the next data name is the next most significant key, and so forth.
    a. When the ASCENDING phrase is specified, the merged sequence is from the lowest value of the contents of the data items identified by the key data names to the highest value, according to the rules for comparison of operands in a relation condition.

b. When the DESCENDING phrase is specified, the merged sequence is from the highest value of the contents of the data items identified by the key data names to the lowest value, according to the rules for comparison of operands in a relation condition.

4. When, according to the rules for the comparison of operands in a relation condition, the contents of all the key data items of one data record are equal to the contents of the corresponding key data items of one or more other data records, the order of return of these records:

   a. Follows the order of the associated input files as specified in the MERGE statement.

   b. Is such that all records associated with one input file are returned before the return of records from another input file.

5. The collating sequence that applies to the comparison of the nonnumeric key data items specified is determined at the beginning of the execution of the MERGE statement in the following order of precedence:

   a. First, the collating sequence established by the COLLATING SEQUENCE phrase, if specified, in that MERGE statement.

   b. Second, the collating sequence established as the program collating sequence.

   c. Third, the collating sequence established by the current locale if the key data item is declared as USAGE DISP-I18N.

6. The results of the merge operation are undefined unless the records in the files referred to by *file-name-2* and *file-name-3* are ordered as described in the ASCENDING or DESCENDING KEY clauses associated with the MERGE statement.

7. All the records in the files referred to by *file-name-2* and *file-name-3* are transferred to the file referred to by *file-name-1.* At the start of execution of the MERGE statement, the files referred to by *file-name-2* and *file-name-3* must not be in the open mode. For each of the files referred to by *file-name-2* and *file-name-3,* the execution of the MERGE statement causes the following actions to be taken:

   a. The processing of the file is initiated. The initiation is performed as if an OPEN statement with the INPUT phrase had been executed. If an output procedure is specified, this initiation is performed before control passes to the output procedure.

   b. The logical records are obtained and released to the merge operation. Each record is obtained as if a READ statement with the NEXT and the AT END phrases had been executed.

   c. The processing of the file is terminated. The termination is performed as if a CLOSE statement without optional phrases had been executed. If an output procedure is specified, this termination is not performed until after control passes the last statement in the output procedure.

      These implicit functions are performed such that any associated USE AFTER EXCEPTIONS/ERROR procedures are executed.

8. The output procedure can consist of any procedure needed to select, modify, or copy the records that are made available one at a time by the RETURN statement in merged order from the file referenced by *file-name-1.* The range includes all statements that are executed as the result of a transfer of control by CALL, EXIT, GO TO, and PERFORM statements in the range of the output procedure, as well as all statements in declarative procedures that are executed as a result of the execution of statements in the range of the output procedure. The range of the output procedure must not cause the execution of any MERGE, RELEASE, or SORT statement (see Section 2.4.21).
9. If an output procedure is specified, control passes to it during execution of the MERGE statement. The compiler inserts a return mechanism at the end of the last statement in the output procedure. When control passes the last statement in the output procedure, the return mechanism provides for termination of the merge and then passes control to the next executable statement after the MERGE statement. Before entering the output procedure, the merge procedure reaches a point at which it can select the next record in merged order when requested. The RETURN statements in the output procedure are the requests for the next record.
10. During the execution of the output procedure, no statement can be executed manipulating the file referenced by, or accessing the record area associated with, *file-name-2* or *file-name-3.* During the execution of any USE AFTER EXCEPTION procedure implicitly invoked while executing the MERGE statement, no statement can be executed manipulating the file referenced by, or accessing the record area associated with, *file-name-2, file-name-3,* or *file-name-4.*
11. If the GIVING phrase is specified, all the merged records are written on the file referred to by *file-name-4* as the implied output procedure for the MERGE statement. At the start of the execution of the MERGE statement, the file referred to by *file-name-4* must not be in the open mode. For each of the files referred to by *file-name-4,* the execution of the MERGE statement causes the following actions to be taken:
    a. The processing of the file is initiated. The initiation is performed as if an OPEN statement with the OUTPUT phrase had been executed.
    b. The merged logical records are returned and written onto the file. Each record is written as if a WRITE statement without any optional phrases had been executed.

       For a relative file, the relative key data item for the first record returned contains the value 1; for the second record returned, the value 2, and so forth. After execution of the MERGE statement, the content of the relative key data item indicates the last record returned to the file.

c. The processing of the file is terminated. The termination is performed as if a CLOSE statement without optional phrases had been executed.

These implicit functions are performed such that any associated USE AFTER EXCEPTION/ERROR procedures are executed; however, the execution of such a USE procedure must not cause the execution of any statement manipulating the file referenced by or accessing the record associated with *file-name-4.* in the first attempt to write beyond the externally defined boundaries of the file, any USE AFTER STANDARD EXCEPTION/ERROR procedure specified for the file is executed; if control is returned from that USE procedure or if no USE procedure is specified, the processing of the file is terminated as in general rule 11c.

12. If the file referred to by *file-name-4* contains only fixed-length records, any record in the file referred to by *file-name-1* containing fewer character positions than that fixed length is space-filled on the right beginning with the first character position after the last character in the record when that record is returned to the file referred to by *file-name-4*.

13. If DATE immediately precedes *data-name-1*, the data item is a date key and the two-digit year is assumed to be the leftmost two character positions in the data item.

    The remaining characters (bits) of the date key field are assumed to be a bit string (unsigned binary number).

14. The FIRST DATE phrase only affects the collation order for date keys and has no effect on the other keys or the MERGE statement if no date keys are specified.

15. Date keys are ordered according to the following conventions:

    a. If the date key is in an ASCENDING KEY phrase, the two-character year position is ordered as *literal, literal*+1*, literal*+2*,...,*'99', '00', '01', ... , *literal-1*. If *literal* is not specified in the FIRST DATE phrase of the MERGE statement, the Sort/Merge default is assumed for the collation order. When the year positions of the date key are the same, the remaining part of the key, if any, is ordered from all bits cleared to all bits set.

    b. If the date key is in a DESCENDING KEY phrase, the two-character year position is ordered as *literal-1*, *literal-2*, ... , '00', '99', '98', ... , *literal*. If *literal* is not specified in the FIRST DATE phrase of the MERGE statement, the Sort/Merge default is assumed for the collation order. When the year positions of the date key are the same, the remaining part of the key, if any, is ordered from all bits set to all bits cleared.

16. The COLLATING SEQUENCE in effect has no bearing over the DATE keys.

8.5.2. The RELEASE Statement

Function

The RELEASE statement transfers records to the initial phase of a sort operation.

General Format

```
RELEASE record-name-1 [ FROM identifier-1 ]
```

Syntax Rules

1. *Record-name-1* must be the name of a logical record in a sort-merge file description entry and can be qualified.
2. A RELEASE statement can be used only within the range of an input procedure associated with a SORT statement for the file name whose sort-merge file description entry contains *record-name-1*. (See Section 8.5.4.)
3. If *identifier-1* is a function identifier, it must reference an alphanumeric function. When *identifier-1* is not a function identifier, *record-name-1* and *identifier-1* must not reference the same storage area.

General Rules

1. The execution of a RELEASE statement causes the record named by *record-name-1* to be released to the initial phase of a sort operation.
2. The logical record released by the execution of the RELEASE statement is no longer available in the record area unless the sort-merge file name associated with *record-name-1* is specified in a SAME RECORD AREA clause. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as the file associated with *record-name-1*.
3. The result of the execution of a RELEASE statement with the FROM phrase is equivalent to the execution of the following statements in the order specified:
   a. The statement:

      ```
      MOVE identifier-1 TO record-name-1
      ```

      according to the rules specified for the MOVE statement
   b. The same RELEASE statement without the FROM phrase
4. After the execution of the RELEASE statement is complete, the information in the area referred to by *identifier-1* is available, even though the information in the area referred to by *record-name-1* is not available except as specified by the SAME RECORD AREA clause.

8.5.3. The RETURN Statement

Function

The RETURN statement obtains either sorted records from the final phase of a sort operation or merged records during a merge operation.

General Format

```
RETURN file-name-1 RECORD [ INTO identifier-1 ]
 AT END imperative-statement-1
 [ NOT AT END imperative-statement-2 ]
 [ END-RETURN ]
```

Syntax Rules

1. The storage area associated with *identifier-1* and the record area associated with *file-name-1* must not be the same storage area.
2. *File-name-1* must be described by a sort-merge file description entry in the Data Division.
3. A RETURN statement can only be used within the range of an output procedure associated with a SORT or MERGE statement for *file-name-1.*

General Rules

1. When the logical records in a file are described with more than one record description, these records automatically share the same storage area; this is equivalent to an implicit redefinition of the area. The contents of any data items that lie beyond the range of the current data record are undefined at the completion of the execution of the RETURN statement.
2. The execution of the RETURN statement causes the next existing record in the file referred to by *file-name-1* as determined by the keys listed in the SORT or MERGE statement to be made available in the record area associated with *file-name-1.* If no next logical record exists in the file referred to by *file-name-1,* the at-end condition exists and control is transferred to *imperative-statement-1* of the AT END phrase. When the at-end condition occurs, execution of the RETURN statement is unsuccessful and the contents of the record area associated with *file-name-1* are unchanged. After the execution of *imperative-statement-1* in the AT END phrase, no RETURN statement can be executed as part of the current output procedure.
3. If, during the execution of a RETURN with the NOT AT END phrase, an at-end condition does not occur, control is transferred to *imperative-statement-2* after the record is made available and after executing any implicit move resulting from the presence of an INTO phrase.
4. The END-RETURN phrase delimits the scope of the RETURN statement. (See Section 2.4.21)
5. The INTO phrase can be specified in a RETURN statement if
   a. Only one record description is subordinate to the sort-merge file description entry

b. All record names associated with *file-name-1* and the data item referred to by *identifier-1* describe a group item or an elementary alphanumeric item

6. The result of the execution of a RETURN statement with the INTO phrase is equivalent to the application of the following rules in the order specified:
   a. The execution of the same RETURN statement without the INTO phrase.
   b. The current record is moved from the record area to the area specified by *identifier-1* according to the rules for the MOVE statement without the CORRESPONDING phrase. The size of the current record is determined by rules specified for the RECORD clause. If the file description entry contains a RECORD VARYING clause, the implied move is a group move. The implied MOVE statement does not occur if the execution of the RETURN statement was unsuccessful. Any subscripting associated with *identifier-1* is evaluated after the record has been read and immediately before it is moved to the data item. The record is available in both the record area and the data item referred to by *identifier-1.*

8.5.4. The SORT Statement

Function

The SORT statement creates a sort file by executing an input procedure or by transferring records from another file, sorts the records in the sort file on a set of specified keys, and, in the final phase of the sort operation, makes available each record from the sort file, in sorted order, to some output procedures or to an output file.

General Format

```
SORT file-name-1      { ON { ASCENDING  } KEY [DATE] { data-name-1 } ... } ...
                           { DESCENDING }
  [ WITH DUPLICATES IN ORDER ]
  [ COLLATING SEQUENCE IS alphabet-name ]
  [ FIRST DATE IS literal ]
  { INPUT PROCEDURE IS procedure-name-1     [ { THROUGH } procedure-name-2 ] }
                                            [ { THRU    }                  ]
  { USING  { file-name-2 } ...                                             }
  { OUTPUT PROCEDURE IS procedure-name-3    [ { THROUGH } procedure-name-4 ] }
                                            [ { THRU    }                  ]
  { GIVING  { file-name-3 } ...                                            }
```

Syntax Rules

1. A SORT statement can appear anywhere in the Procedure Division except in the declaratives portion or in an input procedure.
2. *File-name-1* must be described in a sort-merge file description entry in the Data Division.

3. If the USING phrase is specified and the file referred to by *file-name-1* contains variable-length records, the size of the records contained in the file referred to by *file-name-2* must neither be less than the smallest record nor larger than the largest record described for *file-name-1*. If the file referred to by *file-name-1* contains fixed-length records, the size of the records contained in the file referred to by *file-name-2* must not be larger than the largest record described for the file referred to by *file-name-1*.
4. *Data-name-1* is a key data name. Key data names are subject to the following rules:
   a. The data items identified by key data names must be described in records associated with *file-name-1*.
   b. Key data names can be qualified.
   c. The data items identified by key data names must not be group items that contain variable occurrence data items.
   d. If *file-name-1* has more than one record description, then the data items identified by key data names need be described in only one of the record descriptions. The same character positions referred to by a key data name in one record description entry are taken as the key in all records of the file.
   e. None of the data items identified by key data names can be described by an entry that either contains an OCCURS clause or is subordinate to an entry containing an OCCURS clause.
   f. If the file referred to by *file-name-1* contains variable-length records, all the data items identified by key data names must be contained within the first *x* character positions of the record, where *x* equals the minimum record size specified for the file referred to by *file-name-1*.
5. The words THRU and THROUGH are equivalent.
6. *File-name-2* and *file-name-3* must be described in a file description entry, not in a sort-merge file description entry, in the Data Division.
7. The files referred to by *file-name-2* and *file-name-3* can reside on the same multiple file reel.
8. If *file-name-3* refers to an indexed file, the first specification of *data-name-1* must be associated with an ASCENDING phrase and the data item referenced by that *data-name-1* must occupy the same character positions in its record as the data item associated with the prime record key for that file.
9. No pair of file names in the same SORT statement can be specified in the same SAME SORT AREA or SAME SORT-MERGE AREA clause. File names associated with the GIVING phrase cannot be specified in the same SAME AREA clause. (See Section 4.3.3 and Section 8.3.4.)
10. If the GIVING phrase is specified and the file referred to by *file-name-3* contains variable-length records, the size of the records contained in the file referred to by *file-name-1* must neither be less than the smallest record nor larger than the largest record described for *file-name-3*. If the file referred to by *file-name-3* contains fixed-length records, the size of the records contained in the file referred to by *file-name-1* must not be larger than the largest record described for the file referred to by *file-name-3*.

11. *Literal* must be a one- or two-digit integer in the range 0 through 99.
12. If a DATE reserved word is specified, the KEY data item that immediately follows is assumed to be a date key and the following restrictions and conditions exist:
    a. The data item must be described as a USAGE DISPLAY item that contains at least two characters.
    b. If the data item is numeric it must be unsigned.

General Rules

1. If the file referred to by *file-name-1* contains only fixed-length records, any record in the file referred to by *file-name-2* containing fewer character positions than that fixed length is space-filled on the right beginning with the first character position after the last character in the record when that record is released to the file referred to by *file-name-1.*
2. The data names following the word KEY are listed from left to right in the SORT statement in order of decreasing significance without regard to how they are divided into KEY phrases. The leftmost data name is the major key, the next data name is the next most significant key, and so forth.
    a. When the ASCENDING phrase is specified, the sorted sequence is from the lowest value of the contents of the data items identified by the key data names to the highest value, according to the rules for comparison of operands in a relation condition.
    b. When the DESCENDING phrase is specified, the sorted sequence is from the highest value of the contents of the data items identified by the key data names to the lowest value, according to the rules for comparison of operands in a relation condition.
3. If the DUPLICATES phrase is specified and the contents of all the key data items associated with one data record are equal to the contents of the corresponding key data items associated with one or more other data records, then the order of return of these records is:
    a. The order of the associated input files as specified in the SORT statement. Within a given input file, the order is that in which the records are accessed from that file.
    b. The order in which these records are released by an input procedure when an input procedure is specified.
4. If the DUPLICATES phrase is not specified and the contents of all the key data items associated with one data record are equal to the contents of the corresponding key data items associated with one or more other data records, then the order of return of these records is undefined.
5. The collating sequence that applies to the comparison of the nonnumeric key data items specified is determined at the beginning of the execution of the SORT statement in the following order of precedence:
    a. First, the collating sequence established by the COLLATING SEQUENCE phrase, if specified, in the SORT statement
    b. Second, the collating sequence established as the program collating sequence

c. Third, the collating sequence established by the current locale if the key data item is declared as USAGE DISP-I18N.

6. The execution of the SORT statement consists of three distinct phases:

   a. Records are made available to the file referenced by *file-name-1.* This is achieved either by the execution of RELEASE statements in the input procedure or by the implicit execution of READ statements for *file-name-2.* When this phase commences, the file referenced by *file-name-2* must not be in the open mode. When this phase terminates, the file referenced by *file-name-2* is not in the open mode.

   b. The file referenced by *file-name-1* is sequenced. No processing of the files referenced by *file-name-2* and *file-name-3* takes place during this phase.

   c. The records of the file referenced by *file-name-1* are made available in sorted order. The sorted records are either written to the file referenced by *file-name-3* or, by the execution of a RETURN statement, are made available for processing by the output procedure. When this phase commences, the file referenced by *file-name-3* must not be in the open mode. When this phase terminates, the file referenced by *file-name-3* is not in the open mode.

7. The input procedure can consist of any procedure needed to select, modify, or copy the records that are made available one at a time by the RELEASE statement to the file referenced by *file-name-1.* The range includes all statements that are executed as the result of a transfer of control by CALL, EXIT, GO TO, and PERFORM statements in the range of the input procedure, as well as all statements in declarative procedures that are executed as a result of the execution of statements in the range of the input procedure. The range of the input procedure must not cause the execution of any MERGE, RETURN, or SORT statement (see Section 2.4.21).

8. If an input procedure is specified, control is passed to the input procedure before the file referred to by *file-name-1* is sequenced by the SORT statement. The compiler inserts a return mechanism at the end of the last section in the input procedure and, when control passes the last statement in the input procedure, the records that have been released to the file referred to by *file-name-1* are sorted.

9. If the USING phrase is specified, all the records in the files referred to by *file-name-2* are transferred to the file referred to by *file-name-1.* For each of the files referred to by *file-name-2,* the execution of the SORT statement causes the following actions to be taken:

   a. The processing of the file is initiated. The initiation is performed as if an OPEN statement with the INPUT phrase had been executed.

   b. The logical records are obtained and released to the sort operation. Each record is obtained as if a READ statement with the NEXT and the AT END phrases had been executed.

   c. For a relative file, the content of the relative key data item is undefined after the execution of a SORT statement if *file-name-2* is referenced in the GIVING phrase. The processing of the file is terminated. The termination is performed as if a CLOSE statement without optional phrases had been executed. This termination is performed before the file referred to by *file-name-1* is sequenced by the SORT statement.

These implicit functions are performed such that any associated USE AFTER EXCEPTION/ERROR procedures are executed; however, the execution of such a USE procedure must not cause the execution of any statement manipulating the file referenced by, or accessing the record area associated with, *file-name-2.*

10. The output procedure can consist of any procedure needed to select, modify, or copy the records that are made available one at a time by the RETURN statement in sorted order from the file referenced by *file-name-1.* The range includes all statements that are executed as the result of a transfer of control by CALL, EXIT, GO TO, and PERFORM statements in the range of the output procedure, as well as all statements in declarative procedures that are executed as a result of the execution of statements in the range of the output procedure. The range of the output procedure must not cause the execution of any MERGE, RELEASE, or SORT statement. (See Section 2.4.21.)
11. If an output procedure is specified, control passes to it after the file referred to by *file-name-1* has been sequenced by the SORT statement. The compiler inserts a return mechanism at the end of the last section in the output procedure, and, when control passes the last statement in the output procedure, the return mechanism provides for termination of the sort and then passes control to the next executable statement after the SORT statement. Before entering the output procedure, the sort procedure reaches a point at which it can select the next record in sorted order when requested. The RETURN statements in the output procedure are the requests for the next record.
12. If the GIVING phrase is specified, all the sorted records are written on the file referred to by *file-name-3* as the implied output procedure for the SORT statement. For each of the files referred to by *file-name-3,* the execution of the SORT statement causes the following actions to be taken:
    a. The processing of the file is initiated. The initiation is performed as if an OPEN statement with the OUTPUT phrase had been executed. This initiation is performed after the execution of any input procedure.
    b. The sorted logical records are returned and written onto the file. The records are written as if a WRITE statement without any optional phrases had been executed.

       For a relative file, the relative key data item for the first record returned contains the value 1; for the second record returned, the value 2, and so forth. After execution of the SORT statement, the content of the relative key data item indicates the last record returned to the file.
    c. The processing of the file is terminated. The termination is performed as if a CLOSE statement without optional phrases had been executed.

These implicit functions are performed such that any associated use procedures are executed; however, the execution of such a USE procedure must not cause the execution of any statement manipulating the file referenced by, or accessing the record area associated with, *file-name-3*. On the first attempt to write beyond the externally defined boundaries of the file, any USE AFTER STANDARD EXCEPTION procedure specified for the file is executed; if control is returned from the USE procedure or if no USE procedure is specified, the processing of the file is terminated as in general rule 12c.

13. If the file referred to by *file-name-3* contains only fixed-length records, any record in the file referred to by *file-name-1* containing fewer character positions than that fixed length is space-filled on the right beginning with the first character position after the last character in the record when that record is returned to the file referred to by *file-name-3*.
14. If DATE immediately precedes *data-name-1*, the data item is a date key and the two-digit year is assumed to be the leftmost two character positions in the data item. The remaining characters (bits) of the date key field are assumed to be a bit string (unsigned binary number).
15. The FIRST DATE phrase only affects the collation order for date keys and has no effect on the other keys or the SORT statement if no date keys are specified.
16. Date keys are ordered according to the following conventions:
    a. If the date key is in an ASCENDING KEY phrase, the two-character year position is ordered as *literal*, *literal+1*, *literal+2*,...,'99', '00', '01', ... , *literal-1*. If *literal* is not specified in the FIRST DATE phrase of the MERGE statement, the Sort/Merge default is assumed for the collation order. When the year positions of the date key are the same, the remaining part of the key, if any, is ordered from all bits cleared to all bits set.
    b. If the date key is in a DESCENDING KEY phrase, the two-character year position is ordered as *literal-1*, *literal-2*, ... , '00', '99', '98', ... , *literal*. If *literal* is not specified in the FIRST DATE phrase of the MERGE statement, the Sort/Merge default is assumed for the collation order. When the year positions of the date key are the same, the remaining part of the key, if any, is ordered from all bits set to all bits cleared.
17. The COLLATING SEQUENCE in effect has no bearing over the DATE keys.

8.6. Sort-Merge Considerations

The Sort-Merge interface provides the UCS COBOL user the facility to order one or more files of records or combine two or more identically ordered files of records, according to a set of user-specified keys.

8.6.1. Main Storage Size

At execution time, significant main storage is required to accomplish a sort or merge. The user can dynamically change the amount of main storage needed for a sort or merge by using the R$CORE feature. This feature requires mass storage assignment of a file name R$CORE whose maximum granularity on this assign is used to determine the amount of main storage that is required for the sort or merge. The Universal Runtime System multiplies the maximum granularity on the @ASG, T R$CORE statement by 1,000 to determine the actual amount of storage to dynamically assign. In the absence of an R$CORE statement, 22,500 words are used as the storage assignment. An example showing how to specify a 14,000-word Sort-Merge area is

```
@ASG,T R$CORE.,F///14
```

8.6.2. Sort-Merge Files

Special Sort-Merge files must be assigned prior to the execution of the program using the external names "XA" through "XZ" for the required mass-storage or tape. See the *Sort/Merge Programming Guide* for a more detailed discussion of these assignments.

8.6.3. Suppression of SORT Console and Log Messages

By default, the console and/or log messages are displayed when a program executes SORT/MERGE statements. UCS COBOL provides the user with a run-time facility to suppress or enable these messages. To suppress or enable the console and/or log messages, the following statements can be specified in a COBOL program:

Suppress console messages:

```
CALL 'SUPCON$'.
```

Suppress log messages:

```
CALL 'SUPLOG$'.
```

Suppress console and log messages:

```
CALL 'SUPCONLG$'.
```

Enable console messages:

```
CALL 'ENACON$'.
```

Enable log messages:

```
CALL 'ENALOG$'.
```

Enable console and log messages:

```
CALL 'ENACONLG$'.
```

When the CALL statement to suppress the console and/or log messages is executed, these messages are suppressed until a CALL statement to enable these messages is executed or the end of the program is reached.

8.7. Error Messages from a Sort or a Merge

Error messages produced during a sort or merge have the form:

```
COBOL SORT-MERGE ERROR CODE: NN [error test]
```

where *NN* is the Sort-Merge interface error number. An error of note is 85, which produces the error text:

```
YZ ERROR EXIT FROM SORT-MERGE SUBROUTINE
```

where *Y* is a letter and *Z* is a digit. These 2-character *(YZ)* error codes are described in the *Sort/Merge Programming Guide*.

8.8. Internationalization Considerations

For a more complete discussion of internationalization, see the *UCS COBOL Programming Reference Manual Volume 2*.

8.8.1. Culturally Sensitive Sort/Merge

Files can be sorted or merged in a culturally sensitive manner based on the value of the currently active locale. A culturally sensitive sort or merge is performed if one or more of the specified keys are declared with a data type of USAGE DISP-I18N and no additional COLLATING clauses have been specified. When a USAGE DISP-I18N data item is used as a key, UCOB passes this information onto the sort subroutines. This informs the sort subroutines to perform the sort or merge based on the value of the active locale rather than a normal binary sort.

Variables with a data type of USAGE BYTE-I18N can also be used as keys in a sort or merge. When this is done, the sort subroutine treats the key as an unsigned binary key and performs a normal binary sort or merge.

For additional information concerning internationalization and SORT/MERGE, see the *SORT/MERGE Programming Reference Manual.*

8.8.2. Code Type and Locale of Sort/Merge Files

When SORT or MERGE statements are processed, the code type of the data in the input files must be compatible with the code type of the currently active locale. This means that the code type of the input file must be the same as the code type of the active locale or the code type of the input file must be 1 (ASCII). When a SORT or MERGE that contains a USING clause is executed, checks are done to ensure that the code types of the USING files are compatible with the active locale. If they are not compatible, then a runtime error message is issued and the program execution halts. If an INPUT PROCEDURE is used, you must ensure that the code type of the input file is compatible with the active locale.

If the GIVING clause is specified in the SORT or MERGE statement, the code type of a sequential or relative output file is set to the code type of the currently active locale and the locale of an indexed file is set to the currently active locale. This action overrides any value specified by the CODE TYPE or LOCALE clauses associated with the GIVING files. If an OUTPUT PROCEDURE is used, you must properly set the code type or locale of the output file.

8.9. Examples

The following examples illustrate the two basic organizational approaches that the programmer can adopt in writing a sort program in COBOL. Note that a file XA must be assigned for use by the Sort routine (see the *Sort/Merge Programming Guide*).

Example 1

```
IDENTIFICATION DIVISION.
 PROGRAM-ID. SORTA.
*       THIS SORT PROGRAM EXAMPLE ILLUSTRATES
*       THE FOLLOWING SORT OPTIONS
*       INPUT PROCEDURE IS
*       OUTPUT PROCEDURE IS
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
 INPUT-OUTPUT SECTION.
 FILE-CONTROL.
     SELECT TO-BE-SORTED ASSIGN TO TAPE D.
     SELECT AFTER-THE-SORT ASSIGN TO TAPE E.
     SELECT WORK-ING ASSIGN TO DISC XA.
     SELECT FOR-THE-PRINTER ASSIGN TO PRINTER.
 DATA DIVISION.
 FILE SECTION.
 FD  TO-BE-SORTED
     LABEL RECORDS ARE OMITTED
     DATA RECORD IS A-FILE.
 01  A-FILE.
     02 1-ACCOUNT PICTURE IS X(4).
     02 1-TYPE PICTURE IS X.
     02 1-AREA PICTURE IS XXX.
```

```
    02 1-BODY PICTURE IS X(72).
FD  AFTER-THE-SORT
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS B-FILE.
01  B-FILE PICTURE IS X(80).
SD  WORK-ING
    DATA RECORD IS C-FILE.
01  C-FILE.
    02  3-ACCOUNT PICTURE IS X(4).
    02  3-TYPE PICTURE IS X.
    02  3-AREA PICTURE IS XXX.
    02  3-BODY PICTURE IS X(72).
FD  FOR-THE-PRINTER
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS D-FILE.
01  D-FILE PICTURE IS X(80).
PROCEDURE DIVISION.
FILE-OPENING SECTION.
P0. OPEN INPUT TO-BE-SORTED.
    OPEN OUTPUT AFTER-THE-SORT FOR-THE-PRINTER.
A-SORT SECTION.
P1. SORT WORK-ING ON ASCENDING KEY 3-ACCOUNT 3-TYPE 3-AREA
    INPUT PROCEDURE IS B-SORT THRU B1-SORT
    OUTPUT PROCEDURE IS C-SORT THRU C1-SORT
    GO TO OK-PRINT.
B-SORT SECTION.
P2. READ TO-BE-SORTED AT END GO TO B1-SORT.
    RELEASE C-FILE FROM A-FILE.
    GO TO B-SORT.
B1-SORT SECTION.
P3. CLOSE TO-BE-SORTED.
C-SORT SECTION.
P4. RETURN WORK-ING RECORD INTO B-FILE AT END GO TO C1-SORT.
    WRITE B-FILE.
    GO TO C-SORT.
C1-SORT SECTION.
P5. CLOSE AFTER-THE-SORT.
OK-PRINT SECTION.
P6. OPEN INPUT AFTER-THE-SORT.
A-LOOP SECTION.
P7. READ AFTER-THE-SORT AT END GO TO A-1-MOVE.
    PERFORM A-MOVE.
    WRITE D-FILE.
    GO TO A-LOOP.
A-MOVE SECTION.
P8. MOVE B-FILE TO D-FILE.
A-1-MOVE SECTION.
P9. EXIT.
ALL-DONE SECTION.
PA. CLOSE AFTER-THE-SORT FOR-THE-PRINTER.
    STOP RUN.
```

In Example 1, the files defined by FD entries are opened before the SORT statement is executed. The SORT statement sets the sorting mechanism in motion by transferring control to the INPUT PROCEDURE. The files can be opened with the INPUT PROCEDURE, but the input file must not be opened more than once or closed before the input operation is completed.

Sorting is conducted on an ASCENDING KEY. Note that the KEY identifiers are defined in the sort file WORK-ING, not in the input file TO-BE-SORTED.

The INPUT PROCEDURE is completed when the entire input file has been transferred to the sort file. The sort mechanism then transfers control to the OUTPUT PROCEDURE, and records are written on the file AFTER-THE-SORT in the order specified by the KEY parameters. The sorted records are then listed after the sort function has been accomplished.

Example 2

```
IDENTIFICATION DIVISION.
 PROGRAM-ID. SORTB.
*    THIS SORT PROGRAM EXAMPLE ILLUSTRATES
*    THE FOLLOWING SORT OPTIONS
*    USING
*    GIVING.
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
 INPUT-OUTPUT SECTION.
 FILE-CONTROL.
     SELECT TO-BE-SORTED ASSIGN TO TAPE D.
     SELECT AFTER-THE-SORT ASSIGN TO TAPE E.
     SELECT WORK-ING ASSIGN TO DISC XA.
     SELECT FOR-THE-PRINTER ASSIGN TO PRINTER.
 DATA DIVISION.
 FILE SECTION.
 FD  TO-BE-SORTED
     LABEL RECORDS ARE OMITTED
     DATA RECORD IS A-FILE.
 01  A-FILE.
     02 1-ACCT PICTURE IS X(4).
     02 1-DEPT PICTURE IS X(4).
     02 1-BODY PICTURE IS X(72).
 FD  AFTER-THE-SORT
     LABEL RECORDS ARE OMITTED
     DATA RECORD IS B-FILE.
 01  B-FILE.
     02 2-ACCT PICTURE IS X(4).
     02 2-DEPT PICTURE IS X(4).
     02 2-BODY PICTURE IS X(72).
 SD  WORK-ING
     DATA RECORD IS C-FILE.
 01  C-FILE.
```

```
    02  3-ACCT PICTURE IS X(4).
    02  3-DEPT PICTURE IS X(4).
    02  3-BODY PICTURE IS X(72).
FD  FOR-THE-PRINTER
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS D-FILE.
01  D-FILE.
    02 4-ACCT PICTURE IS X(4).
    02 4-DEPT PICTURE IS X(4).
    02 4-BODY PICTURE IS X(72).
PROCEDURE DIVISION.
FILE-OPENING SECTION.
P0. OPEN OUTPUT FOR-THE-PRINTER.
A-SORT SECTION.
P1. SORT WORK-ING ON ASCENDING KEY 3-ACCT 3-DEPT
    USING TO-BE-SORTED GIVING AFTER-THE-SORT.
OK-PRINT SECTION.
P2. OPEN INPUT AFTER-THE-SORT.
A-LOOP SECTION.
P3. READ AFTER-THE-SORT AT END GO TO A-1-MOVE.
    PERFORM A-MOVE.
    WRITE D-FILE.
    GO TO A-LOOP.
A-MOVE SECTION.
P4. MOVE B-FILE TO D-FILE.
A-1-MOVE SECTION.
P5. EXIT.
ALL-DONE SECTION.
P6. CLOSE AFTER-THE-SORT FOR-THE-PRINTER.
    STOP RUN.
```

In Example 2, the files that are directly involved in the sort must not be open at the time the SORT statement is encountered. This is because the USING and GIVING options perform the functions of the INPUT PROCEDURE and OUTPUT PROCEDURE (that is, OPEN, READ, RELEASE, RETURN, USE, WRITE, and CLOSE) automatically. Although the use of the GIVING and USING options requires a minimum amount of coding, it is not possible to add, delete, or modify individual records of the sort file in any way (that is, no special processing procedures can be incorporated into a sort function when this technique is adopted).

Example 3

```
IDENTIFICATION DIVISION.
 PROGRAM-ID. MERGEA.
*    THIS MERGE PROGRAM EXAMPLE ILLUSTRATES
*    THE FOLLOWING MERGE OPTIONS
*    USING
*    GIVING.
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
```

```
INPUT-OUTPUT SECTION.
FILE-CONTROL.
    SELECT TO-BE-MERGED1 ASSIGN TO DISC D.
    SELECT TO-BE-MERGED2 ASSIGN TO DISC F.
    SELECT AFTER-THE-MERGE ASSIGN TO DISC E.
    SELECT WORK-ING ASSIGN TO DISC XA.
    SELECT FOR-THE-PRINTER ASSIGN TO PRINTER.
DATA DIVISION.
FILE SECTION.
FD  TO-BE-MERGED1
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS A-FILE1.
01  A-FILE1.
    02 1-ACCT PICTURE IS X(4).
    02 1-DEPT PICTURE IS X(4).
    02 1-BODY PICTURE IS X(72).
FD  TO-BE-MERGED2
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS A-FILE2.
01  A-FILE2.
    02 1-ACCT PICTURE IS X(4).
    02 1-DEPT PICTURE IS X(4).
    02 1-BODY PICTURE IS X(72).
FD  AFTER-THE-MERGE
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS B-FILE.
01  B-FILE.
    02 2-ACCT PICTURE IS X(4).
    02 2-DEPT PICTURE IS X(4).
    02 2-BODY PICTURE IS X(72).
SD  WORK-ING
    DATA RECORD IS C-FILE.
01  C-FILE.
    02  3-ACCT PICTURE IS X(4).
    02  3-DEPT PICTURE IS X(4).
    02  3-BODY PICTURE IS X(72).
FD  FOR-THE-PRINTER
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS D-FILE.
01  D-FILE.
    02 4-ACCT PICTURE IS X(4).
    02 4-DEPT PICTURE IS X(4).
    02 4-BODY PICTURE IS X(72).
PROCEDURE DIVISION.
FILE-OPENING SECTION.
P0. OPEN OUTPUT FOR-THE-PRINTER.
A-MERGE SECTION.
P1. MERGE WORK-ING ON ASCENDING KEY 3-ACCT 3-DEPT
    USING TO-BE-MERGED1 TO-BE-MERGED2
    GIVING AFTER-THE-MERGE.
OK-PRINT SECTION.
P2. OPEN INPUT AFTER-THE-MERGE.
```

```
A-LOOP SECTION.
P3. READ AFTER-THE-MERGE AT END GO TO A-1-MOVE.
    PERFORM A-MOVE.
    WRITE D-FILE.
    GO TO A-LOOP.
A-MOVE SECTION.
P4. MOVE B-FILE TO D-FILE.
A-1-MOVE SECTION.
P5. EXIT.
ALL-DONE SECTION.
P6. CLOSE AFTER-THE-MERGE FOR-THE-PRINTER.
    STOP RUN.
```

In Example 3, the files that are directly involved in the merge must not be open at the time the MERGE statement is encountered. This is because the USING and GIVING options perform the functions of OPEN, READ, RETURN, WRITE, and CLOSE automatically. Although the use of the USING and GIVING options requires a minimum amount of coding, it is not possible to add, delete, or modify records of the sort-merge file in any way (that is, no special processing procedures can be incorporated into a merge function when this technique is used).

This example merges the files TO-BE-MERGED1 and TO-BE-MERGED2 into the file AFTER-THE-MERGE. This merged file is then listed after the merge function has been accomplished. The merged files were already in sorted order according to the keys defined in the MERGE statement.

Example 4

```
IDENTIFICATION DIVISION.
 PROGRAM-ID. MERGEB.
*    THIS MERGE PROGRAM EXAMPLE ILLUSTRATES
*    THE FOLLOWING MERGE OPTIONS
*    USING
*    OUTPUT PROCEDURE.
 ENVIRONMENT DIVISION.
 CONFIGURATION SECTION.
 SOURCE-COMPUTER. UNISYS-2200.
 OBJECT-COMPUTER. UNISYS-2200.
 INPUT-OUTPUT SECTION.
 FILE-CONTROL.
     SELECT TO-BE-SORTED ASSIGN TO DISC A.
     SELECT BBB ASSIGN TO DISC D.
     SELECT CCC ASSIGN TO DISC F.
     SELECT ABC ASSIGN TO DISC C.
     SELECT AFTER-THE-SORT ASSIGN TO DISC B.
     SELECT WORK-ING ASSIGN TO DISC XA.
     SELECT FOR-THE-PRINTER ASSIGN TO PRINTER.
 DATA DIVISION.
 FILE SECTION.
 FD  TO-BE-SORTED
     LABEL RECORDS ARE OMITTED
     DATA RECORD IS A-FILE.
```

```
01  A-FILE.
    02 1-ACCOUNT PIC 9999.
    02 1-TYPE PIC X.
    02 1-AREA PIC XXX.
    02 1-BODY PIC X(72).
FD  ABC
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS ABC-FILE.
01  ABC-FILE.
    02 A-ACCOUNT PIC X(80).
FD  BBB
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS BBB-FILE.
01  BBB-FILE.
    02 B-ACCOUNT PIC X(80).
FD  CCC
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS CCC-FILE.
01  CCC-FILE.
    02 C-ACCOUNT PIC X(80).
FD  AFTER-THE-SORT
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS B-FILE.
01  B-FILE PIC X(80).
SD  WORK-ING
    DATA RECORD IS C-FILE.
01  C-FILE.
    02  3-ACCT PIC XXXX.
    02  3-TYPE PIC X.
    02  3-DEPT PIC XXX.
    02  3-BODY PIC X(72).
FD  FOR-THE-PRINTER
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS D-FILE.
01  D-FILE PIC X(80).
WORKING-STORAGE SECTION.
01  KNT PIC 9999 VALUE 0.
PROCEDURE DIVISION.
FILE-OPENING SECTION.
P0. OPEN OUTPUT FOR-THE-PRINTER.
A-MERGE SECTION.
P1. MERGE WORK-ING ON ASCENDING KEY 3-ACCT
    USING TO-BE-SORTED ABC BBB CCC
    OUTPUT PROCEDURE IS OK-PRINT.
    GO TO WWW.
OK-PRINT SECTION.
P3.
    RETURN WORK-ING AT END GO TO RRR.
    DISPLAY 3-ACCT.
    GO TO P3.
RRR.
    EXIT.
```

```
WWW SECTION.
P4.
    CLOSE FOR-THE-PRINTER.
    STOP RUN.
```

In Example 4, the files that are directly involved in the merge must not be open at the time the MERGE statement is being executed. This is because the USING option in the merge and the RETURN statement in the OUTPUT PROCEDURE perform the functions of OPEN, READ, and CLOSE automatically.

In this program the files TO-BE-SORTED, ABC, BBB, and CCC are merged and are returned to the sort-merge record one at a time, at which time the key field of the record is displayed. The merged files were already in sorted order according to the keys defined in the MERGE statement.

Section 9
Source Text Manipulation

9.1. General

The Source Text Manipulation module contains the COPY statement and the REPLACE statement. Each of these statements can function, either independently of, or in conjunction with, the other, to provide an extensive capability to insert and replace source program text as part of the compilation of the source program.

9.1.1. Libraries and Source Programs

COBOL libraries contain texts that are available to the compiler at compile time. The effect of the interpretation of the COPY statement is to generate, from a library text, text that the compiler treats as part of the source program.

Similarly, COBOL source programs can be written in a programmer-defined notation that, at compile time, can be expanded into syntactically correct phrases, clauses, and statements. The effect of the interpretation of the REPLACE statement is to substitute new text for text appearing in the source program and have the substituted text treated by the compiler as part of the source program.

9.1.2. Level Characteristics

Source Text Manipulation level 1 provides for copying text from a single library into the source program. Text is copied from the library without change.

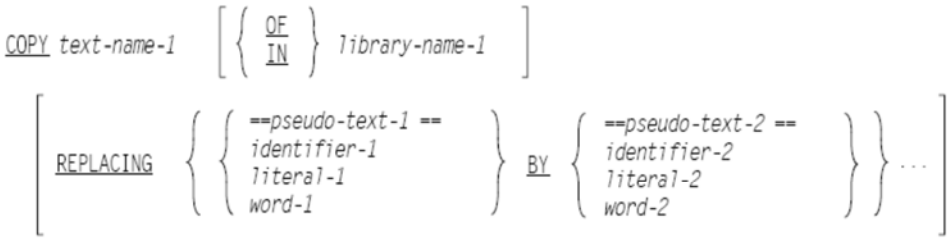
Source Text Manipulation level 2 provides the additional capability of replacing all occurrences of a given literal, identifier, word, or group of words in the library text, with alternate text, during the copying process. Level 2 also provides for the availability of more than one COBOL library at compile time and the substitution of new text for text appearing in the source program.

9.2. The COPY Statement

Function

The COPY statement incorporates text into a COBOL source program.

General Format



Syntax Rules

1. If more than one COBOL library is available during compilation, *text-name-1* must be qualified by *library-name-1* identifying the COBOL library in which the text associated with *text-name-1* resides.

   Within one COBOL library, each text name must be unique.

2. The COPY statement must be preceded by a space and terminated by the separator period.
3. *Pseudo-text-1* must contain one or more text words.
4. *Pseudo-text-2* can contain zero, one, or more text words.
5. Character strings within *pseudo-text-1* and *pseudo-text-2* can be continued.
6. *Word-1* or *word-2* can be any single COBOL word except COPY.
7. A COPY statement can be specified in the source program anywhere a character string or a separator, other than the closing quotation mark, can occur.
8. The compiler allows a length of 1 through 322 characters for a text word within pseudo-text and within library text.
9. *Pseudo-text-1* must not consist entirely of a separator comma or a separator semicolon.
10. If the word COPY appears in a comment entry or in the place where a comment entry can appear, it is considered part of the comment entry.

General Rules

1. The compilation of a source program containing COPY statements is logically equivalent to processing all COPY statements before processing the resultant source program.
2. The effect of processing a COPY statement is that the library text associated with *text-name-1* is copied into the source program, logically replacing the entire COPY statement, beginning with the reserved word COPY and ending with the punctuation character period, inclusive.

3. If the REPLACING phrase is not specified, the library text is copied unchanged. If the REPLACING phrase is specified, the library text is copied and each properly matched occurrence of *pseudo-text-1, identifier-1, word-1,* and *literal-1* in the library text is replaced by the corresponding *pseudo-text-2, identifier-2, word-2,* or *literal-2.*
4. For matching, *identifier-1, word-1,* and *literal-1* are treated as pseudo-text containing only *identifier-1, word-1,* or *literal-1,* respectively.
5. The comparison operation to determine text replacement occurs in the following manner:
    a. The leftmost library text word that is not a separator comma or separator semicolon is the first text word used for comparison. Any text word or space preceding this text word is copied into the source program. Starting with the first text word for comparison and the first *pseudo-text-1, identifier-1, word-1,* or *literal-1* that was specified in the REPLACING phrase, the entire REPLACING phrase operand that precedes the reserved word BY is compared to an equivalent number of contiguous library text words.
    b. *Pseudo-text-1, identifier-1, word-1,* or *literal-1* match the library text if the ordered sequence of text words that forms *pseudo-text-1, identifier-1, word-1,* or *literal-1* is equal, character for character, to the ordered sequence of library text words. For matching, each occurrence of a separator comma, semicolon, or space in *pseudo-text-1* or in the library text is considered to be a single space. Each sequence of one or more space separators is considered to be a single space.
    c. If no match occurs, the comparison is repeated with each next successive *pseudo-text-1, identifier-1, word-1,* or *literal-1,* if any, in the REPLACING phrase until either a match is found or there is no next successive REPLACING operand.
    d. When all the REPLACING phrase operands have been compared and no match has occurred, the leftmost library text word is copied into the source program. The next successive library text word is then considered as the leftmost library text word, and the comparison cycle starts again with the first *pseudo-text-1, identifier-1, word-1,* or *literal-1* specified in the REPLACING phrase.
    e. Whenever a match occurs between *pseudo-text-1, identifier-1, word-1,* or *literal-1* and the library text, the corresponding *pseudo-text-2, identifier-2, word-2,* or *literal-2* are placed into the source program. The library text word immediately following the rightmost text word that participated in the match is then considered as the leftmost text word. The comparison cycle starts again with the first *pseudo-text-1, identifier-1, word-1,* or *literal-1* specified in the REPLACING phrase.
    f. The comparison operation continues until the rightmost text word in the library text has either participated in a match or been considered as a leftmost library text word and participated in a complete comparison cycle.

6. Comment lines or blank lines occurring in the library text and in *pseudo-text-1* are ignored for matching, and the sequence of text words in the library text, if any, and in *pseudo-text-1* is determined by the rules for reference format. (See Section 2.6.1.) Comment lines or blank lines appearing in *pseudo-text-2* are copied into the resultant program unchanged whenever *pseudo-text-2* is placed into the source program as a result of text replacement. Comment lines or blank lines appearing in library text are copied into the resultant source program unchanged with the following exception: a comment line or blank line in library text is not copied if that comment line or blank line appears within the sequence of text words that match *pseudo-text-1.*
7. Debugging lines are permitted within library text and pseudo-text. Text words within a debugging line participate in the matching rules as if the "D" did not appear in the indicator area. A debugging line is specified within pseudo-text if the debugging line begins in the source program after the opening pseudo-text delimiter but before the matching closing pseudo-text delimiter.
8. The syntactic correctness of the library text cannot be independently determined. Except for COPY and REPLACE statements, the syntactic correctness of the entire COBOL source program cannot be determined until all COPY and REPLACE statements have been completely processed.
9. Each text word copied from the library, but not replaced, is copied so as to begin at the same point of the line in the resultant program as it begins in the line within the library. However, if a text word copied from the library begins in area A but follows another text word, which also begins in area A of the same line, and if replacement of a preceding text word in the line by replacement text of greater length occurs, the following text word begins in area B if it cannot begin in area A. Each text word in *pseudo-text-2* that is to be placed into the resultant program begins in the same area of the resultant program as it appears in *pseudo-text-2.* Each *identifier-2, literal-2,* and *word-2* that is to be placed into the resultant program begins in the same area of the resultant program as the leftmost library text word that participated in the match would appear if it had not been replaced.

   Library text must conform to the rules for COBOL reference format.

   If additional lines are introduced into the source program as a result of a COPY statement, each text word introduced appears on a debugging line if the COPY statement begins on a debugging line or if the text word being introduced appears on a debugging line in library text. When a text word specified in the BY phrase is introduced, it appears on a debugging line if the first library text word being replaced is specified on a debugging line. Except in the preceding cases, only those text words specified on debugging lines where the debugging line is *pseudo-text-2* appear on debugging lines in the resultant program. If any literal specified as *literal-2* or within *pseudo-text-2* or library text is of too great a length to be accommodated on a single line without continuation to another line in the resultant program and the literal is not being placed on a debugging line, additional continuation lines are introduced that contain the remainder of the literal. If replacement requires that the continued literal be continued on a debugging line, the program is in error.

10. For purposes of compilation, text words after replacement are placed in the source program according to the rules for reference format. (See Section 2.6.1.). When copying text words of *pseudo-text-2* into the source program, additional spaces can be introduced only between text words where there already exists a space (including the assumed space between source lines).
11. If additional lines are introduced into the source program as a result of the processing of COPY statements, the indicator area of the introduced line contains the same character as the line on which the text being replaced begins, unless that line contains a hyphen, in which case the introduced line contains a space. In the case where a literal is continued to an introduced line that is not a debugging line, a hyphen is placed in the indicator area.
12. If you do not specify the library name, the compiler searches for the copied text in the files determined by the compiler keyword options for source control (SEARCH-PF-SI and MULTI-PF), described in Volume 2 of the *UCS COBOL Programming Reference Manual*.
13. Nested COPY statements can occur. A nested COPY statement cannot contain the REPLACING phrase, and a COPY statement with the REPLACING phrase cannot contain nested COPY statements. COPY statements can cause recursion, however COPY recursion will result in a fatal error once the recursion level reaches 50.

Examples

The ensuing paragraphs provide examples of libraries and the manner in which calls are made on the libraries.

Contents of Library A

```
Column
 1 2      8    12

 ID PROC
              TEST-PROG.
  END
 COMPUTER PROC
          SOURCE-COMPUTER. UNISYS-2200.
          OBJECT-COMPUTER. UNISYS-2200.
   END
 NAMES PROC
               PRINTER IS PRINT-FILE.
   END
 FILECONT PROC
               SELECT FILEA ASSIGN TO PRINTER PRT.
          I-O-CONTROL.  SAME RECORD AREA FOR FILEA, FILEB.
   END
```

Contents of Library B

```
Column
1 2      8    12

LIBFD PROC
              FILE1 --- etc.
```

```
         01 RECORD1 --- etc.
  END
LIBSD PROC
              DATA RECORD SORT-REC --- etc.
  END
LIBRECORD PROC
              02 ELEM-ITEM --- etc.
              02 XYZ --- etc.
  END
```

Contents of Library C

```
Column
1 2      8    12

P1 PROC
         SECTION-0 SECTION.
         PARA-NAME-1.        (any COBOL statements)
           .
           .
           .
  END
P2 PROC
              SECTION.
         PARA-2.
  END
P3 PROC
              SORT-REC FROM RECORD1.
  END
P4 PROC
              ADD A  TO B.
  END
```

Library Calls in a COBOL Program Contained in File A

```
Column
 1 2      8    12

          IDENTIFICATION DIVISION.
          PROGRAM-ID. COPY-ID.
          ENVIRONMENT DIVISION.
          CONFIGURATION SECTION.
          COPY COMPUTER.
          SPECIAL-NAMES.  COPY NAMES.
          .
          .
          .
          FILE-CONTROL.  COPY FILECONT.
          .
          .
          .
          DATA DIVISION.
```

```
           FILE SECTION.
           FD COPY LIBFD IN B.
           SD FILEB COPY LIBSD OF B.
           01 SORT-REC. COPY LIBRECORD IN B REPLACING ELEM-ITEM BY GRP,
XYZ BY
                  ABC.
           .
           .
           .
           PROCEDURE DIVISION.
           COPY P1 IN C.
           .
           .
           .
           SECTION-1 COPY P2 IN C.
            .
            .
            .
           PARA-3.
                  RELEASE COPY P3 OF C.
            .
            .
            .
           PARA-4.
                    COPY P4 OF C REPLACING ==ADD A TO== BY ==
                    SUBTRACT C FROM==.
```

9.3. The REPLACE Statement

Function

Use the REPLACE statement to replace source program text.

Format 1



Format 2



Syntax Rules

1. A REPLACE statement can occur anywhere in the source program where a character string can occur. A separator period must precede it unless it is the first statement in a separately compiled program.
2. A separator period must terminate a REPLACE statement.
3. *Pseudo-text-1* must contain one or more text words.
4. *Pseudo-text-2* can contain zero, one, or more text words.
5. Character strings within *pseudo-text-1* and *pseudo-text-2* can be continued.

6. The compiler allows a length of 1 through 322 characters for a text word within pseudo-text.
7. *Pseudo-text-1* must not consist entirely of a separator comma or a separator semicolon.
8. If the word REPLACE appears in a comment entry or in the place where a comment entry can appear, it is considered part of the comment entry.

General Rules

1. The format 1 REPLACE statement specifies the text of the source program that the corresponding text will replace. Each matched occurrence of *pseudo-text-1* in the source program is replaced by the corresponding *pseudo-text-2.*
2. The format 2 REPLACE statement specifies that any text replacement currently in effect is discontinued.
3. A given occurrence of the REPLACE statement is in effect from the point at which it is specified until the next occurrence of the statement or the end of the separately compiled program, respectively.
4. Any REPLACE statements contained in a source program are processed after the processing of any COPY statements contained in a source program.
5. The text produced as a result of the processing of a REPLACE statement must not contain a REPLACE statement.
6. The comparison operation to determine text replacement occurs in the following manner:
    a. Starting with the leftmost source program text word and the first *pseudo-test-1, pseudo-text-1* is compared to an equivalent number of contiguous source program text words.
    b. *Pseudo-text-1* matches the source program text only if the ordered sequence of text words that forms *pseudo-text-1* is equal, character for character, to the ordered sequence of source program text words. For matching, each occurrence of a separator comma, semicolon, or space in *pseudo-text-1* or in the source program text is considered to be a single space. Each sequence of one or more separator spaces is considered to be a single space.
    c. If no match occurs, the comparison is repeated with each next successive occurrence of *pseudo-text-1* until either a match is found or there is no next successive occurrence of *pseudo-text-1*.
    d. When all occurrences of *pseudo-text-1* have been compared and no match has occurred, the next successive source program text word is then considered as the leftmost source program text word and the comparison cycle starts again with the first occurrence of *pseudo-text-1*.
    e. Whenever a match occurs between *pseudo-text-1* and the source program text, the corresponding *pseudo-text-2* replaces the matched text in the source program. The source program text word immediately following the rightmost text word that participated in the match is then considered as the leftmost source program text word. The comparison cycle starts again with the first occurrence of *pseudo-text-1*.

f. The comparison operation continues until the rightmost text word in the source program text that is within the scope of the REPLACE statement has either participated in a match or has been considered as a leftmost source program text word and participated in a complete comparison cycle.

7. Comment lines or blank lines occurring in the source program text and in *pseudo-text-1* are ignored for matching, and the sequence of text words in the source program text and in *pseudo-text-1* is determined by the rules for reference format. Comment lines or blank lines in *pseudo-text-2* are placed into the resultant program unchanged whenever *pseudo-text-2* is placed into the source program as a result of text replacement. A comment line or blank line in the source program text is not replaced if that comment line or blank line appears within the sequence of text words that match *pseudo-text-1.*

8. Debugging lines are permitted in pseudo-text. Text words within a debugging line participate in the matching rules as if the "D" did not appear in the indicator area.

9. Except for COPY and REPLACE statements, the syntactic correctness of the source program text cannot be determined until after all COPY and REPLACE statements have been completely processed.

10. Text words inserted into the source program as a result of processing a REPLACE statement are placed in the source program according to the rules for reference format (see Section 2.4.20). When inserting text words of *pseudo-text-2* into the source program, additional spaces can be introduced only between text words where there already exists a space (including the assumed space between source lines).

11. If additional lines are introduced into the source program as a result of the processing of REPLACE statements, the indicator area of the introduced lines contains the same character as the line on which the text being replaced begins, unless that line contains a hyphen, in which case the introduced line contains a space.

    If any literal within *pseudo-text-2* is of a length too great to be accommodated on a single line without continuation to another line in the resultant program and the literal is not being placed on a debugging line, additional continuation lines are introduced that contain the remainder of the literal. If replacement requires the continued literal to be continued on a debugging line, the program is in error.

Section 10
Report Writer

10.1. General

The report writer feature places its emphasis on the organization, format, and contents of an output report. Although a report can be produced using the standard COBOL language, the report writer language features provide a more concise facility for report structuring and report production. Much of the Procedure Division programming that would normally be supplied by the programmer is instead provided automatically by the compiler. Therefore, the programmer is relieved of writing procedures for moving data, constructing print lines, counting lines on a page, numbering pages, producing heading and footing lines, recognizing the end of logical data subdivisions, updating sum counters, and so forth. All these operations are accomplished as a consequence of source language statements that appear primarily in the Report Section of the Data Division of the source program.

10.1.1. Function

The Report Writer module provides a facility for producing reports by specifying the physical appearance of a report rather than requiring specification of the detailed procedures necessary to produce that report.

A hierarchy of levels is used in defining the logical organization of a report. Each report is divided into report groups, which in turn are divided into sequences of items. Such a hierarchical structure permits explicit reference to a report group with implicit reference to other levels in the hierarchy. A report group contains one or more items to be presented on zero, one, or more lines.

10.1.2. Report File

A report file is an output file having sequential organization. A report file has a file description entry containing a REPORT clause. The content of a report file consists of records that are implicitly written to the output file.

A report file is named by a file control entry and is described by a file description entry containing a REPORT clause. A report file is referred to and accessed by the OPEN, GENERATE, INITIATE, SUPPRESS, TERMINATE, USE AFTER STANDARD EXCEPTION PROCEDURE, USE BEFORE REPORTING, and CLOSE statements.

10.1.3. Report Section

The Report Section of a COBOL Data Division contains one or more Report Description entries (RD entries), each of which forms the complete description of a report.

The report named in the RD entry is not assigned directly to an output file; instead, it is associated with a file name in the File Section and that file name is associated with a file when an OPEN statement specifying the file name is executed. More than one report can be associated with the same file name and the CODE clause is used to differentiate among the reports. For an external file connector referenced by a file name, separately compiled programs can specify different reports for the same file name. The file description entry of a file name to which a report is assigned cannot contain record description entries that describe data records. This file description entry must specify the name of a report description entry for each report associated with that file name in this program.

The report description entry contains a set of clauses that names the report and supplies specific information about the format of the printed page and the organization of the subdivisions of the report. A user can give an identification code in the report description entry so that each report can be identified separately in an intermediate output file.

Following each report description entry are one or more 01 level number entries, each followed by a hierarchical structure similar to COBOL record descriptions. Each 01 level number entry and its subordinate entries describe a report group. Each report group consists of zero, one, or more print lines that are regarded as a unit. A report group that is to be printed is printed entirely on one logical page; it is never split across pages.

10.1.4. Report Structure

Major consideration must be given to vertical and horizontal spacing requirements, manipulation of data, and the physical and logical subdivisions of a report.

Vertical Spacing

The report writer feature allows the user to describe report groups containing multiple lines. The vertical positioning of the lines on a page is specified by the LINE NUMBER clause that is associated with each line. The NEXT GROUP clause indicates how many lines to space after presenting the last line of the group. The first LINE NUMBER clause of the next group indicates additional spacing information to be used in positioning of that group.

Horizontal Spacing

The report writer allows the user to position the fields of data on a report line by means of the COLUMN NUMBER clause. Undefined areas in the report line are space-filled by COBOL.

Data Manipulation

When the report writer feature is used, data movement to a report group is directed by Report Section clauses rather than Procedure Division statements. The Report Section clauses that affect the manipulation of data are the SOURCE, SUM, and VALUE clauses.

The SOURCE clause specifies the sending data item of an implicit MOVE statement. The receiving printable item is defined by the description of the report group item in which the SOURCE clause appears.

The SUM clause automatically causes the establishment of a sum counter. The object of the SUM clause names the data items that are added to the sum counter when a GENERATE statement is executed. The move of the sum counter contents to the receiving printable item, defined by the description of the report group item in which the SUM clause appears, is accomplished automatically when that report group is presented.

The VALUE clause defines a literal that appears in the printable item of a report group each time that report group is to be presented.

In summary, a data item in a report group is presented only if it has a COLUMN NUMBER clause specifying where it is to be presented. The value that is placed in a printable item is determined by the SOURCE, SUM, or VALUE clause stated in the report group description. Under no circumstances can a report group printable item receive a value directly through a Procedure Division statement.

Report Subdivisions

The physical and logical organization of a report interacts to determine what is presented on a page.

Physical Subdivision of a Report

The PAGE clause specifies the length of the page, the size of the heading and footing areas, and the size of the area in which the detail lines appears. The LINE NUMBER and NEXT GROUP clauses are used to position these report groups and, when necessary, to advance to a new page with automatic production of PAGE HEADING and PAGE FOOTING report groups.

Logical Subdivisions of a Report

DETAIL report groups can be structured into a nested set of control groups. Each control group can begin with a CONTROL HEADING report and end with a CONTROL FOOTING report group.

When nested control groups are defined, the recognition of a change in value of a control data item in a control hierarchy is called a control break and the heading and footing lines associated with the control data name are called CONTROL HEADING and CONTROL FOOTING report groups.

During the execution of a GENERATE statement, the control hierarchy is used to check automatically for control breaks. If a control break has occurred, all controls that are minor to it are considered to have changed, even though they cannot in fact have changed. The occurrence of a control break causes the following sequence of events to take place:

1. All CONTROL FOOTING report groups are presented up to and including the one at the level at which the control break occurred.
2. All CONTROL HEADING report groups are presented from the control break level down to the most minor control.
3. The DETAIL report group named in the GENERATE statement is presented.

10.1.5. Report Procedure

The report writer statements that appear in the Procedure Division are: INITIATE, GENERATE, TERMINATE, SUPPRESS, and USE BEFORE REPORTING.

The INITIATE statement automatically performs a number of initialization functions. A report must be initiated before any detail processing can take place.

The GENERATE statement that specifies a data name causes the named DETAIL report group to be formatted and written to the output device. In addition, it triggers the many implicit actions described in the preceding section.

The GENERATE statement that specifies a report name provides a means of summary reporting. A report produced by this type of statement has all detail print lines suppressed automatically and consists of only the summary totals accumulated during the processing of the DETAIL report group. A GENERATE report name statement is identical to that which occurs for a GENERATE data name statement, except that the former results in the suppression of detail print lines.

The TERMINATE statement causes the report writer control system to perform all of the automatic functions associated with the termination of a report. The TERMINATE statement must be executed before the file containing the report is closed.

The SUPPRESS statement provides the object time facility to suppress the printing of an entire report group.

The BEFORE REPORTING phrase of the USE statement provides a mechanism whereby Procedure Division statements can be executed at specific instances within the automatically performed procedures. The statements in the USE BEFORE REPORTING procedures can alter the contents of the data items that are referred to by SOURCE clauses. Therefore, control is possible over the contents of data items referred to within report groups that are produced automatically.

10.1.6. PAGE-COUNTER

The reserved word PAGE-COUNTER is a name for a page counter that is generated for each report description entry in the Report Section of the Data Division. The implicit definition is that of an unsigned integer that must be capable of representing a range of values from 1 through 999999. The value in PAGE-COUNTER is maintained and used by the program to number the pages of a report. PAGE-COUNTER can be referred to only in the SOURCE clause of the Report Section and in Procedure Division statements. (See Section 10.3.2)

10.1.7. LINE-COUNTER

The reserved word LINE-COUNTER is a name for a line counter that is generated for each report description entry in the Report Section of the Data Division. The implicit definition is that of an unsigned integer that must be capable of representing a range of values from 0 through 999999. The value in LINE-COUNTER is maintained and is used to determine the vertical positioning of a report. LINE-COUNTER can be referred to only in the SOURCE clause of the Report Section and in Procedure Division statements; however, the value of LINE-COUNTER must not be explicitly changed.

10.1.8. Subscripting

In the Report Section, neither a sum counter nor the special registers LINE-COUNTER and PAGE-COUNTER can be used as subscripts.

This subsection describes the Environment Division and its associated organization.

10.2. Environment Division

This subsection describes the Environment Division and its associated organization.

10.2.1. Input-Output Section

The Input-Output Section is located in the Environment Division of a source program. The Input-Output Section deals with the information needed to control transmission and handling of data between external media and the object program. The Input-Output Section is optional in the Environment Division of a COBOL source program.

General Format

```
INPUT-OUTPUT SECTION.
FILE-CONTROL.   {file-control-entry} ...
[I-O-CONTROL.   [input-output-control-entry] ]
```

10.2.2. The FILE-CONTROL Paragraph

Function

The FILE-CONTROL paragraph allows specification of file-related information.

General Format

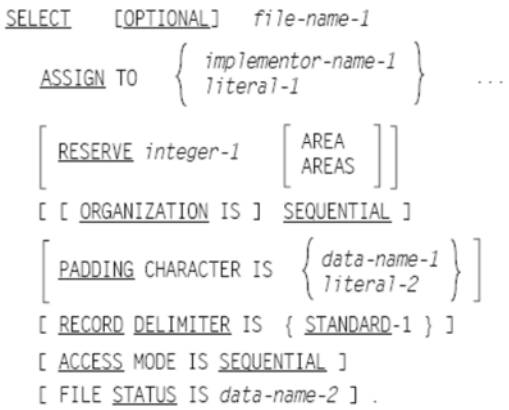
```
FILE-CONTROL.  {file-control-entry}. . .
```

10.2.3. The File Control Entry

Function

The File control entry declares the relevant physical attributes of a report file.

General Format



Syntax Rules

1. The SELECT clause must be specified first in the file control entry. The clauses following the SELECT clause can appear in any order.
2. Each report file described in the Data Division must be specified only once in the FILE-CONTROL paragraph. Each report file specified in the SELECT clause must have a file description entry containing a REPORT clause in the Data Division of the same program.
3. Literal-1 must be a nonnumeric literal and must not be a figurative constant.

General Rules

1. If the file connector referred to by *file-name-1* is an external file connector (see Section 7.6.6), all file control entries in the run unit that refer to this file connector must have:
   a. The same specification for the OPTIONAL phrase
   b. A consistent specification for *implementor-name-1* or *literal-1* in the ASSIGN clause
   c. The same value of *integer-1* in the RESERVE clause

d. The same organization

e. The same access mode

f. The same specification for the PADDING CHARACTER clause

2. The OPTIONAL phrase applies only to a report file opened in the extend mode. Its specification is required for a report file that is not necessarily present each time the object program is executed.
3. The ASSIGN clause specifies the association of the report file referred to by *file-name-1* to a storage medium referenced by *implementor-name-1.*
4. A report file has sequential organization. Therefore, all clauses within the file control entry for a report file shown in the general format in this subsection are presented within the Sequential I-O module (See Section 4.3.1).

10.2.4. The I-O-CONTROL Paragraph

Function

The I-O-CONTROL paragraph specifies the memory area that is to be shared by different files and the location of files on a multiple file reel.

General Format

```
I-O-CONTROL .
 [ [ SAME AREA FOR file-name-1 { file-name-2 } . . . ] . . .
 [ MULTIPLE FILE TAPE CONTAINS
  { file-name-3 [ POSITION integer-1 ] } . . . ] . . .  . ]
```

Syntax Rules

1. The order of appearance of the clauses is immaterial.
2. A file name that represents a report file can appear in a MULTIPLE FILE TAPE clause or in a SAME clause for which the RECORD phrase is not specified.
3. The availability of specific clauses in the I-O-CONTROL paragraph for a report file is described in Section 4.3.3.

General Rules

1. The MULTIPLE FILE TAPE clause is presented in Section 4.3.3.
2. The SAME clause is presented in Section 4.3.3.

10.3. Data Division

This subsection describes the Data Division and its associated organization.

10.3.1. File Section

The File Section is located in the Data Division of a source program. The File Section defines the structure of report files. Each report file is defined by a file description entry having a REPORT clause. A file description entry for a report file is not followed by record description entries.

The general format of the File Section in the Report Writer module is

```
FILE SECTION.

[file-description-entry]. . .
```

In a COBOL program, the File Description entry (FD entry) represents the highest level of organization in the File Section. The File Section header is followed by an FD entry consisting of a level indicator (FD), a file name, and a series of independent clauses. For a report file, the FD entry must contain the REPORT clause specifying the names of the reports to be written onto the report file.

The File Description--Complete Entry Skeleton

Function

The FD entry furnishes information concerning the physical structure, identification, and report names pertaining to a given report file.

General Format

```
FD file-name-1
  [ BLOCK CONTAINS   [ integer-1 TO ]   integer-2 { RECORDS / CHARACTERS } ]
  [ RECORD { CONTAINS integer-3 CHARACTERS
           / CONTAINS integer-4 CHARACTERS TO integer-5 CHARACTERS } ]
  [ LABEL { RECORD IS / RECORDS ARE } { STANDARD / OMITTED } ]
  [ VALUE OF { implementor-name-1 IS { data-name-1 / literal-1 } } ... ]
  [ CODE-SET IS alphabet-name-1 ]
  { REPORT IS / REPORTS ARE } { report-name-1 } . . .
  [ DENSITY IS { data-name-2 / integer-6 } LINES ]
```

Syntax Rules

1. The level indicator FD identifies the beginning of the FD entry for a report file and must precede the file name of the report file.
2. The clauses that follow *file-name-1* can appear in any order.

3. File-name-1 can refer only to a sequential file.
4. The subject of a file description entry that specifies a REPORT clause can be referred to in the Procedure Division only by the USE statement, the CLOSE statement, or the OPEN statement with the OUTPUT or EXTEND phrase.
5. With the exception of the REPORT clause, the syntax rules for all clauses within the file description entry for a report file are presented within the Sequential I-O module. (See Section 4.4.5.)

General Rules

1. A file description entry associates *file-name-1* with a file connector.
2. With the exception of the REPORT clause, the general rules for all clauses within the file description entry for a report file are presented within the Sequential I-O module. (See Section 4.4.2.)
3. The REPORT clause is presented in Section 10.3.1.

The REPORT Clause

Function

The REPORT clause specifies the names of reports that comprise a report file.

General Format

```
{ REPORT IS   }
{ REPORTS ARE } { report-name-1 } . . .
```

Syntax Rules

1. Each report name specified in a REPORT clause must be the subject of an RD entry in the Report Section of the same program. The order of appearance of the report names is not significant.
2. A report name must appear in at least one REPORT clause.
3. The subject of an FD entry that specifies a REPORT clause can be referred to in the Procedure Division only by the USE statement, the CLOSE statement, or the OPEN statement with the OUTPUT or EXTEND phrase.

General Rules

1. The presence of more than one report name in a REPORT clause indicates that the file contains more than one report.
2. After execution of an INITIATE statement and before the execution of a TERMINATE statement for the same report file, no input-output statement can be executed that refers to that report file.
3. If the associated file connector is an external file connector, every file description entry in the run unit that is associated with that file connector must describe it as a report file.

10.3.2. Report Section

The Report Section is located in the Data Division of a source program. The Report Section describes the reports to be written onto report files. The description of each report must begin with a Report Description entry (RD entry) and must be followed by one or more report group description entries.

```
REPORT SECTION.
[report-description-entry {report-group-description-entry} ... ] ...
```

Report Description Entry

In addition to naming the report, the RD entry defines the format of each page of the report by specifying the vertical boundaries of the region within which each type of report group can be printed. The RD entry also specifies the control data items. When the report is produced, changes in the values of the control data items cause the detail information of the report to be processed in groups called control groups.

Each report named in the REPORT clause of a file description entry in the File Section must be the subject of an RD entry in the Report Section.

Report Group Description Entry

The report group that composes the report describes the following RD entry. The description of each report group begins with a report group description entry, that is, an entry that has a 01 level number and a TYPE clause. Subordinate to the report group description entry, there can appear group and elementary entries that further describe the characteristics of the report group.

The Report Description--Complete Entry Skeleton

Function

The report description entry names a report, specifies any identifying characters to be appended to each print line in the report, and describes the physical structure and organization of that report.

General Format

```
RD report-name-1
  [ CODE literal-1 ]
  [ { CONTROL IS  } { { data-name-1 } ...                  } ]
  [ { CONTROLS ARE} { FINAL [ data-name-1 ] ...            } ]
  [ PAGE [ LIMIT IS ] integer-1 [ LINE ] [ HEADING integer-2 ] ]
  [ LIMITS ARE ]    [ LINES ]
  [ FIRST DETAIL integer-3 ] [ LAST DETAIL integer-4 ]
  [ FOOTING integer-5 ] .
```

Syntax Rules

1. *Report-name-1* must appear in at least one REPORT clause.
2. The order of appearance of the clauses following *report-name-1* is immaterial.
3. *Report-name-1* is the highest permissible qualifier that can be specified for LINE-COUNTER, PAGE-COUNTER, and all data names defined within the Report Section.

General Rule

The CODE clause, the CONTROL clause, and the PAGE clause are presented in Section 10.3.2.

PAGE-COUNTER Rules

1. PAGE-COUNTER is the reserved word used to refer to a special register that is automatically created for each report specified in the Report Section. (See Section 2.4.7 and Section 10.1.6.)
2. In the Report Section, a reference to PAGE-COUNTER can appear only in a SOURCE clause. In the Procedure Division, PAGE-COUNTER can be used in any context in which a data item with an integer value can appear.
3. If more than one PAGE-COUNTER exists in a program, PAGE-COUNTER must be qualified by a report name whenever it is referred to in the Procedure Division.

   In the Report Section, an unqualified reference to PAGE-COUNTER is qualified implicitly by the name of the report in whose report description entry the reference is made. Whenever the PAGE-COUNTER of a different report is referred to, PAGE-COUNTER must be explicitly qualified by the report name associated with the different report.
4. Execution of the INITIATE statement causes the PAGE-COUNTER of the referenced report to be set to one.
5. PAGE-COUNTER is automatically incremented by one each time a page advance is executed.
6. PAGE-COUNTER can be altered by Procedure Division statements.

LINE-COUNTER Rules

1. LINE-COUNTER is the reserved word used to refer to a special register that is automatically created for each report specified in the Report Section. (See Section 2.4.7 and Section 10.1.7.)
2. In the Report Section, a reference to LINE-COUNTER can appear only in a SOURCE clause. In the Procedure Division, LINE-COUNTER can be used in any context in which a data item with an integral value can appear. However, it must not be explicitly changed by the program.

3. If more than one LINE-COUNTER exists in a program, LINE-COUNTER must be qualified by a report name whenever it is referred to in the Procedure Division.

   In the Report Section, an unqualified reference to LINE-COUNTER is qualified implicitly by the name of the report in whose RD entry the reference is made. Whenever the LINE-COUNTER of a different report is referred to, LINE-COUNTER must be explicitly qualified by the report name associated with the different report.

4. Execution of an INITIATE statement causes the LINE-COUNTER of the referenced report to be set to zero. LINE-COUNTER is also reset to zero each time it executes a page advance.

5. The value of LINE-COUNTER is not affected by the processing of nonprintable report groups nor by the processing of a printable report group whose printing is suppressed by means of the SUPPRESS statement.

6. At the time each print line is presented, the value of LINE-COUNTER represents the line number on which the print line is presented. The value of LINE-COUNTER after the presentation of a report group is governed by the presentation rules for each report group. (See Section 10.3.2.)

The CODE Clause

Function

The CODE clause specifies a 2-character literal that identifies each print line as belonging to a specific report.

General Format

```
CODE literal-1
```

Syntax Rules

1. *Literal-1* must be a 2-character nonnumeric literal.
2. If the CODE clause is specified for any report in a file, it must be specified for all reports in that file.

General Rules

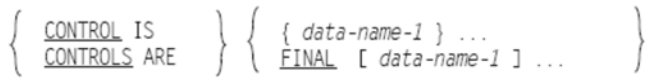
1. When the CODE clause is specified, *literal-1* is automatically placed in the first two character positions of each report writer logical record.
2. The positions occupied by *literal-1* are not included in the description of the print line, but are included in the logical record size.
3. The CODE clause is ignored if the report file is assigned to a printer file.

The CONTROL Clause

Function

The CONTROL clause establishes the levels of the control hierarchy for the report.

General Format



Syntax Rules

1. *Data-name-1* must not be defined in the Report Section. *Data-name-1* can be qualified.
2. Each recurrence of *data-name-1* must identify a different data item.
3. *Data-name-1* must not have subordinate to it a variable occurrence data item.

General Rules

1. *Data-name-1* and the word FINAL specify the levels of the control hierarchy. FINAL is the highest control, *data-name-1* is the major control, the next recurrence of *data-name-1* is an intermediate control, and so forth. The last recurrence of *data-name-1* is the minor control.
2. The execution of the chronologically first GENERATE statement for a given report causes the values of all control data items associated with that report to be saved. On subsequent executions of all GENERATE statements for that report, control data items are implicitly tested for a change of value. A change of value in any control data item causes a control break to occur. This control break is associated with the highest level for which a change of value is noted. (See Section 10.4.3.)
3. The control break is tested for by comparing the content of each control data item with the prior content of each control data item that was saved when the previous GENERATE statement for the same report was executed. The inequality relation test is applied as follows:
   a. If the control data item is a numeric data item, the relation test is for the comparison of two numeric operands.
   b. If the control data item is an index data item, the relation test is for the comparison of two index data items.
   c. If the control data item is a data item other than as described in 3a and 3b, the relation test is for the comparison of two nonnumeric operands.
   d. The inequality relation test is further explained in the appropriate paragraph. (See Section 3.6.3)
4. FINAL is used when the most inclusive control group in the report is not associated with a control data name.

The PAGE Clause

Function

The PAGE clause defines the length of a page and the vertical subdivisions within which report groups are presented.

General Format

```
PAGE [ LIMIT IS  ] integer-1 [ LINE  ]
     [ LIMITS ARE ]          [ LINES ]
[ HEADING integer-2 ]  [ FIRST DETAIL integer-3 ]
[ LAST DETAIL integer-4 ] [ FOOTING integer-5 ]
```

Syntax Rules

1. The HEADING, FIRST DETAIL, LAST DETAIL, and FOOTING phrases can be written in any order.
2. *Integer-1* must not be greater than 511.
3. *Integer-2* must be greater than or equal to one.
4. *Integer-3* must be greater than or equal to *integer-2.*
5. *Integer-4* must be greater than or equal to *integer-3.*
6. *Integer-5* must be greater than or equal to *integer-4.*
7. *Integer-1* must be greater than or equal to *integer-5.*
8. The following rules indicate the vertical subdivision of the page in which each type of report group can appear when the PAGE clause is specified. (See Section 10.3.2.)

    a. A report heading report group that is to be presented on a page by itself, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-2* to the line number specified by *integer-1,* inclusive.

    A report heading report group that is not to be presented on a page by itself, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-2* to the line number specified by *integer-3* minus 1, inclusive.

    b. A page heading report group, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-2* to the line number specified by *integer-3* minus 1, inclusive.

    c. A control heading or detail report group, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-3* to the line number specified by *integer-4,* inclusive.

    d. A control footing report group, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-3* to the line number specified by *integer-5,* inclusive.

    e. A page footing report group, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-5* plus 1 to the line number specified by *integer-1,* inclusive.

f. A report footing report group that is to be presented on a page by itself, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line specified by *integer-2* to the line number specified by *integer-1,* inclusive.

   A report footing report group that is not to be presented on a page by itself, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number by *integer-5* plus 1 to the line number specified by *integer-1,* inclusive.

9. All report groups must be described such that they can be presented on one page. A multiline report group is never split across page boundaries.

General Rules

1. The vertical format of a report page is established using the integer values specified in the PAGE clause.
   a. *Integer-1* defines the size of a report page by specifying the number of lines available on each page.
   b. HEADING *integer-2* defines the first line number on which a report heading or page heading report group can be presented.
   c. FIRST DETAIL *integer-3* defines the first line number on which a body group can be presented. Report heading (without NEXT GROUP NEXT PAGE) and page heading report groups cannot be presented on or beyond the line specified by *integer-3.*
   d. LAST DETAIL *integer-4* defines the last line number on which a control heading or detail report can be presented.
   e. FOOTING *integer-5* defines the last line number on which a control footing report group can be presented. Report footing (without LINE *integer-1* NEXT PAGE) and page footing report groups must follow the line number specified by *integer-5.*
2. If the PAGE clause is specified, the following implicit values are assumed for any omitted phrases:
   a. If the HEADING phrase is omitted, a value of one is assumed for *integer-2.*
   b. If the FIRST DETAIL phrase is omitted, a value equal to *integer-2* (HEADING) is given to *integer-3.*
   c. If the LAST DETAIL and the FOOTING phrases are both omitted, the value of *integer-1* (PAGE LIMIT) is given to both *integer-4* and *integer-5.*
   d. If the FOOTING phrase is specified and the LAST DETAIL phrase is omitted, the value of *integer-5* (FOOTING) is given to *integer-4.*
   e. If the LAST DETAIL phrase is specified and the FOOTING phrase is omitted, the value of *integer-4* (LAST DETAIL) is given to *integer-5.*
3. If the PAGE clause is omitted, the report consists of a single page of indefinite length.
4. The presentation rules for each type of report group are specified in the appropriate paragraph. (See Section 10.3.2)

PAGE Regions

The regions that are established by the PAGE clause are shown in the following table.

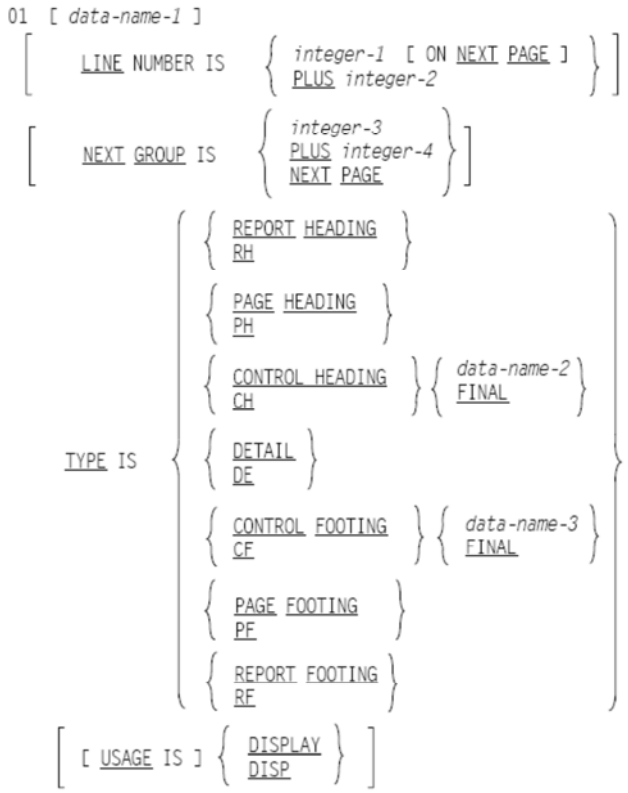
Report Groups That May Be Presented in the Region	First Line Number of the Region	Last Line Number of the Region	Relative Line Positioning for the First Report Group within a Region
Report heading described with 'NEXT GROUP NEXT PAGE	*integer-2* (HEADING)	*integer-1* (PAGE LIMIT)	LINE NUMBER plus *integer-2* (HEADING) minus 1
Report footing described with LINE *integer-1* NEXT PAGE	*integer-2* (HEADING)	*integer-1* (PAGE LIMIT)	
Report heading not described with NEXT GROUP NEXT PAGE	*integer-2* (HEADING)	*integer-3* (FIRST DETAIL) minus 1	LINE NUMBER plus *integer-2* (HEADING) minus 1
Page heading	*integer-2* (HEADING)	integer-3 (FIRST DETAIL) minus 1	LINE NUMBER plus *integer-2* (HEADING) minus 1
Control heading	*integer-3* (FIRST DETAIL)	*integer-4* (LAST DETAIL)	IF LINE-COUNTER ≥ *integer-3* (FIRST DETAIL), position on LINE-COUNTER plus 1. IF LINE-COUNTER < *integer-3* (FIRST DETAIL), position on *integer-3* (FIRST DETAIL).
Detail	*integer-3* (FIRST DETAIL)	*integer-4* (LAST DETAIL)	IF LINE-COUNTER ≥ *integer-3* (FIRST DETAIL), position on LINE-COUNTER plus 1. IF LINE-COUNTER < integer-3 (FIRST DETAIL), position on *integer-3* (FIRST DETAIL).
Control footing	*integer-3* (FIRST DETAIL)	*integer-5* (FOOTING)	Same as above.
Page footing	*integer 5* (FOOTING) plus 1	*integer-1* (PAGE LIMIT)	
Report footing not described with LINE *integer-1* NEXT PAGE	*integer 5* (FOOTING) plus 1	*integer-1* (PAGE LIMIT)	LINE NUMBER PLUS *integer-5* (FOOTING)

The Report Group Description-Complete Skeleton

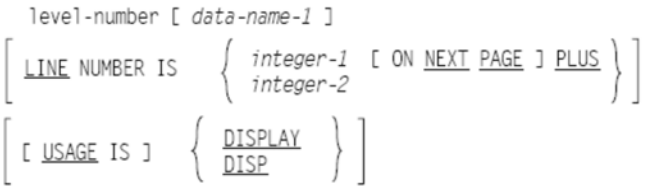
Function

The report group description entry specifies the characteristics of a report group and of the individual items within a report group.

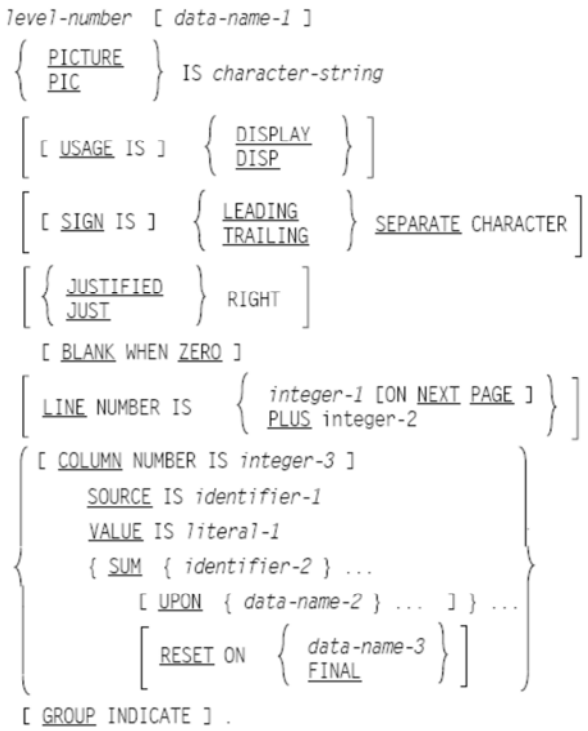
Format 1



Format 2



Format 3



Syntax Rules

1. The report group description entry can appear only in the Report Section.
2. Except for the data name clause, which (when present) must immediately follow the level number, the clauses can be written in any sequence.
3. In format 2, the level number can be any integer from 02 to 48, inclusive. In format 3, the level number can be any integer from 02 to 49, inclusive.
4. A description of a report group can consist of one, two, or three hierarchic levels:
   a. The first entry that describes a report group must be a format 1 entry.
   b. Both format 2 and format 3 entries can be immediately subordinate to a format 1 entry.
   c. At least one format 3 entry must be immediately subordinate to a format 2 entry.
   d. Format 3 entries must define elementary data items.
5. In a format 1 entry, *data-name-1* is required only when:
   a. A detail report group is referred to by a GENERATE statement.
   b. A detail report group is referred to by the UPON phrase of a SUM clause.
   c. A report group is referred to in a USE BEFORE REPORTING sentence.

d. The name of a control footing report group is used to qualify a reference to a sum counter.

   If specified, *data-name-1* can be referred to only by a GENERATE statement, the UPON phrase of a SUM clause, a USE BEFORE REPORTING sentence, or as a sum counter qualifier.

6. A format 2 entry must contain at least one optional clause.
7. In a format 2 entry, *data-name-1* is optional. If present, it can be used only to qualify a sum counter reference.
8. In the Report Section, the USAGE clause is used only to declare the usage of printable items.
   a. If the USAGE clause appears in a format 3 entry, that entry must define a printable item.
   b. If the USAGE clause appears in a format 1 or format 2 entry, at least one subordinate entry must define a printable item.
9. An entry that contains a LINE NUMBER clause must not have a subordinate entry that also contains a LINE NUMBER clause.
10. In format 3
    a. A GROUP INDICATE clause can appear only in a type detail report group.
    b. A SUM clause can appear only in a type control footing report group.
    c. An entry that contains a COLUMN NUMBER clause but no LINE NUMBER clause must be subordinate to an entry that contains a LINE NUMBER clause.
    d. *Data-name-1* is optional but can be specified in any entry. *Data-name-1* can be referred to only if the entry defines a sum counter.
    e. An entry that contains a VALUE clause must also have a COLUMN NUMBER clause.
11. Table 10–1 shows all permissible clause combinations for a format 3 entry. The table is read from left to right along the selected row. An M indicates that the presence of the clause is mandatory. A P indicates that the presence of the clause is permitted, but not required. A blank indicates that the clause is not permitted.

Table 10–1. Permissible Clause Combinations in Format 3 Entries

PIC	Column	Source	Sum	Value	Just	Blank When Zero	Group Indicate	Usage	Sign	Line
M			M						P	P
M	M		M			P		P	P	P
M	M	M			P		P	P	P	P
M	M	M				P	P	P	P	P
M	M			M	P		P	P	P	P

Legend

M The clause is mandatory.

P The clause is permitted but not required.

Blank The clause is permitted but not required.

General Rules

1. Format 1 is the report group entry. The report group is defined by the contents of this entry and all of its subordinate entries.
2. The BLANK WHEN ZERO clause, the JUSTIFIED clause, and the PICTURE clause for the Report Writer module are the same as the BLANK WHEN ZERO clause, the JUSTIFIED clause, and the PICTURE clause in the Nucleus module. (See Section 3.5.6, Section 3.5.8, and Section 3.5.11, respectively.) The other clauses of the report group description entry are presented in alphabetical order beginning at Section 10.3.2.

Presentation Rules Tables

This subsection describes the organization of the presentation rules tables.

Description

Figure 10–1 through Figure 10–3, Table 10–2 through Table 10–3, and their accompanying rules specify

- The permissible combinations of LINE NUMBER and NEXT GROUP clauses for each type of report group
- The requirements that are placed on the use of these clauses
- The compiler's interpretation of these clauses

Organization

There is an individual presentation rules table for each of the following types of report groups: report heading, page heading page footing and report footing (see Section 10.3.2). In addition, detail report groups, control heading report groups, or control footing report groups are treated jointly in the body group presentation rules table (see *Body Group Presentation Rules.)*

Columns 1 and 2 of a presentation rules table list all of the permissible combinations of LINE NUMBER and NEXT GROUP clauses for the designated report group type. Consequently, for the purpose of identifying the set of presentation rules that apply to a particular combination of LINE NUMBER and NEXT GROUP clauses, a presentation rules table is read from left to right, along the selected row.

The applicable rules columns of a presentation rules table are partitioned into two parts. The first part specifies the rules that apply if the report description contains a PAGE clause, and the second part specifies the rules that apply if the PAGE clause is omitted. The purpose of the rules named in the applicable rules columns is discussed below:

1. Upper-limit rules and lower-limit rules

   These rules specify the vertical subdivisions of the page within which the specified report group can be represented.

   In the absence of a PAGE clause, the printed report is not considered to be partitioned into vertical subdivisions. Consequently, within the tables no upper-limit rule or lower-limit rule is specified for a report description in which the PAGE clause is omitted.

2. Fit test rules

   The fit test rules are applicable only to body groups; therefore, fit test rules are specified only within the body group presentation rules table. At object time, the fit test rules are applied to determine whether the designated body group can be presented on the page to which the report is currently positioned.

   However, even for body groups there are no fit test rules when the PAGE clause is omitted from the report description entry.

3. First print line position rules

   The first print line position rules specify where on the report medium the first print line of the given report group is presented.

   The presentation rules tables do not specify where on the report medium the second and subsequent print lines (if any) of a report group are presented. Certain general rules determine where the second and subsequent print lines of a report group are presented. Refer to the LINE NUMBER clause general rules for this information. (See Section 10.3.2)

4. Next group rules

   The next group rules relate to the proper use of the NEXT GROUP clause.

5. Final LINE-COUNTER setting rules

   The terminal values placed in LINE-COUNTER after presenting report groups are specified by the final LINE-COUNTER setting rules.

LINE NUMBER Clause Notation

Column 1 of the presentation rules table uses a shorthand notation to describe the sequence of LINE NUMBER clauses that can appear in the description of a report group. The meaning of the abbreviations used in column 1 is as follows:

1. The letter A represents one or more absolute LINE NUMBER clauses, none of which has the NEXT PAGE phrase, that appear in consecutive order within the sequence of LINE NUMBER clauses in the report group description entry.
2. The letter R represents one or more relative LINE NUMBER clauses that appear in consecutive order within the sequence of LINE NUMBER clauses in the report group description entry.
3. The letters NP represent one or more absolute LINE NUMBER clauses that appear in consecutive order within the sequence of LINE NUMBER clauses in the report group description entry with the phrase NEXT PAGE appearing only in the first LINE NUMBER clause.

When two abbreviations appear together, they refer to a sequence of LINE NUMBER clauses that consists of the two specified consecutive sequences. For example A R refers to a report group description entry within which the A sequence (defined in rule 1) is immediately followed by the R sequence (defined in rule 2).

LINE NUMBER Clause Sequence Substitutions

Where A R is shown to be a permissible sequence in the presentation rules table, A is also permissible and the same presentation rules are applicable.

Where NP R is shown to be a permissible sequence in the presentation rules table, NP is also permissible and the same presentation rules are applicable.

Saved Next Group Integer Description

The saved next group integer is a data item that is not available to the COBOL program. When an absolute NEXT GROUP clause specifies a vertical positioning value that cannot be accommodated on the current page, the value is stored in the saved next group integer. After page advance processing, the next body group is positioned using the value stored in the saved next group integer.

Report Heading Group Presentation Rules

Figure 10–1 points to the appropriate presentation rules for all permissible combination of LINE NUMBER and NEXT GROUP clauses in a report heading report group. The report heading group presentation rules are as follows:

1. Upper-limit rule: The first line number on which the report heading report group can be presented is the line number specified by the HEADING phrase of the PAGE clause.

2. Lower-limit rules
    a. The last line number on which the report heading report group can be presented is the line number that is obtained by subtracting 1 from the value of *integer-3* of the FIRST DETAIL phrase of the PAGE clause.
    b. The last line number on which the report heading report group can be presented is the line number specified by *integer-1* of the PAGE clause.
3. First print line position rules
    a. The first print line of the report heading report group is presented on the line number specified by the integer of its LINE NUMBER clause.
    b. The first print line of the report heading report group is presented on the line number obtained by adding the integer of the first LINE NUMBER clause and the value obtained by subtracting 1 from the value of *integer-2* of the HEADING phrase of the PAGE clause.
    c. The report heading report group is not presented.
    d. The first print line of the report heading report group is presented on the line number obtained by adding the content of its LINE-COUNTER (in this case, zero) to the integer of the first LINE NUMBER clause.
4. Next group rules
    a. The NEXT GROUP integer must be greater than the line number on which the final print line of the report heading report group is presented. In addition, the NEXT GROUP integer must be less than the line number specified by the value of *integer-3* of the FIRST DETAIL phrase of the PAGE clause.
    b. The sum of the NEXT GROUP integer and the line number on which the final print line of the report heading report group is presented must be less than the value of *integer-3* of the FIRST DETAIL phrase of the PAGE clause.
    c. NEXT GROUP NEXT PAGE signifies that the report heading report group is to be presented entirely by itself on the first page of the report. No other report group is processed while positioned to the first page of the report.
5. Final LINE-COUNTER setting rules
    a. After the report heading report group is presented, the NEXT GROUP integer is placed into LINE-COUNTER as the final LINE-COUNTER setting.
    b. After the report heading report group is presented, the sum of the NEXT GROUP integer and the line number on which the final print line of the report heading report group was presented is placed into LINE-COUNTER as the final LINE-COUNTER setting.
    c. After the report heading report group is presented, zero is placed into LINE-COUNTER as the final LINE-COUNTER setting.
    d. After the report heading report group is presented, the final LINE-COUNTER setting is the line number on which the final print line of the report heading report group was presented.
    e. LINE-COUNTER is unaffected by the processing of a nonprintable report group.

**		Applicable Rules***						
		If the PAGE clause is specified					If the PAGE clause is omitted	
Sequence of LINE NUMBER clauses*	NEXT GROUP clause	Upper Limit	Lower Limit	First Print Line Position	Next Group	Final LINE-COUNTER Setting	First Print Line Position	Final LINE-COUNTER Setting
A R	Absolute	1	2a	3a	4a	5a		Illegal Combination+
A R	Relative	1	2a	3a	4b	5b		Illegal Combination+
A R	NEXT PAGE	1	2b	3a	4c	5c		Illegal Combination+
A R		1	2a	3a		5d		Illegal Combination+
R	Absolute	1	2a	3b	4a	5a		Illegal Combination++
R	Relative	1	2a	3b	4b	5b	3d	5b
R	NEXT PAGE	1	2b	3b	4c	5c		Illegal Combination++
R		1	2a	3b		5d	3d	5d
				3c		5e	3c	5e

* See 10.3.2. for a description of the abbreviations used in column 1.

** A blank entry in column 1 or column 2 indicates that the named clause is totally absent from the report group description entry.

*** A blank entry in an applicable rules column indicates the absence of the named rule for the given combination of LINE NUMBER and NEXT GROUF clauses.

\+ See 10.3.2.

++ See 10.3.2.

Figure 10–1. Report Heading Group Presentation Rules Table

Page Heading Group Presentation Rules

Table 10–2 points to the appropriate presentation rules for all permissible combinations of LINE NUMBER and NEXT GROUP clauses in a page heading report group.

Table 10–2. Page Heading Group Presentation Rules

Clause**		Applicable Rules # (if the PAGE clause is specified ##)				
Sequence of LINE NUMBER Clauses*	**NEXT GROUP Clause**	**Upper Limit**	**Lower Limit**	**First Print Line Position**	**Next Group**	**Final LINE-COUNTER Setting**
A R		1	2	3a		4a
R		1	2	3b		4a
				3c		4bn

* See Section 10.3.2. for a description of the abbreviations used in column 1.

** A blank entry in column 1 or column 2 indicates that the named clause is totally absent from the report group description entry.

A blank entry in an applicable rules column indicates the absence of the named rule for the given combination of LINE NUMBER and NEXT GROUP clauses.

If the PAGE clause is omitted from the report description entry, then a page heading report group cannot be defined. (See Section 10.3.2.)

The page heading group presentation rules are as follows:

1. Upper-limit rule

   If a report heading report group has been presented on the page on which the page heading report group is to be presented, then the first line number on which the page heading report group can be presented is one greater than the final LINE-COUNTER setting established by the REPORT HEADING.

   Otherwise, the first line number on which the page heading report group can be presented is the line number specified by the HEADING phrase of the PAGE clause.

2. Lower-limit rule

   The last line number on which the page heading report group can be presented is the line number that is obtained by subtracting one from the value of *integer-3* of the FIRST DETAIL phrase of the PAGE clause.

3. First print line position rules

   a. The first print line of the page heading report group is presented on the line number specified by the integer of its LINE NUMBER clause.

   b. If a report heading report group has been presented on the page on which the page heading report group is to be presented, then the sum of the final LINE-COUNTER setting established by the report heading report group and the integer of the first LINE NUMBER clause of the page heading report group defines the line number on which the first print line of the page heading report group is presented.

      Otherwise, the sum of the integer of the first LINE NUMBER clause of the page heading report group and the value obtained by subtracting one from the value of *integer-2* of the HEADING phrase of the PAGE clause defines the line number on which the first print line of the page heading report group is presented.

   c. The page heading report group is not presented.

4. Final LINE-COUNTER setting rules

   a. The final LINE-COUNTER setting is the line number on which the final print line of the page heading report group was presented.

   b. LINE-COUNTER is unaffected by the processing of a nonprintable report group.

Body Group Presentation Rules

Figure 10–2 points to the appropriate presentation rules for all permissible combinations of LINE NUMBER and NEXT GROUP clauses in control heading, detail, and control footing report groups. The body group presentation rules are as follows:

1. Upper-limit rule

   The first line number on which a body group can be presented is the line number specified by the FIRST DETAIL phrase of the PAGE clause.

2. Lower-limit rules

   a. The last line number on which a control heading report group or detail report group can be presented is the line number specified by the LAST DETAIL phrase of the PAGE clause.

   b. The last line number on which a control footing report group can be presented is the line number specified by the FOOTING phrase of the PAGE clause.

**		Applicable Rules***							
		If the PAGE clause is specified						If the PAGE clause is omitted	
Sequence of LINE NUMBER clauses*	NEXT GROUP clause	Upper Limit	Lower Limit	Fit Test	First Print Line Position	Next Group	Final LINE-COUNTER Setting	First Print Line Position	Final LINE-COUNTER Setting
A R	Absolute	1	2	3a	4a	5	6a	Illegal Combination+	
A R	Relative	1	2	3a	4a		6b	Illegal Combination+	
A R	NEXT PAGE	1	2	3a	4a		6c	Illegal Combination+	
A R		1	2	3a	4a		6d	Illegal Combination+	
R	Absolute	1	2	3b	4b	5	6a	Illegal Combination++	
R	Relative	1	2	3b	4b		6b	4d	6f
R	NEXT PAGE	1	2	3b	4b		6c	Illegal Combination++	
R		1	2	3b	4b		6d	4d	6d
NP R	Absolute	1	2	3c	4a	5	6a	Illegal Combination+	
NP R	Relative	1	2	3c	4a		6b	Illegal Combination+	
NP R	NEXT PAGE	1	2	3c	4a		6c	Illegal Combination+	
NP R		1	2	3c	4a		6d	Illegal Combination+	
					4c		6e	4c	6e

* See 10.3.2 for a description of the abbreviations used in column 1.

** A blank entry in column 1 or column 2 indicates that the named clause is totally absent from the report group description entry.

*** A blank entry in an applicable rules column indicates the absence of the named rule for the given combination of LINE NUMBER and NEXT GROUP clauses.

\+ See 10.3.2

\++ See 10.3.2

Figure 10–2. Body Group Presentation Rules

3. Fit test rules:

   a. If the value in LINE-COUNTER is less than the integer of the first absolute LINE NUMBER clause, then the body group is presented on the page to which the report is currently positioned. Otherwise, page advance processing takes place.

      After the page heading report group (if defined) has been processed, it is determined whether the saved next group integer location was set when the final body group was presented on the preceding page. (See final LINE-COUNTER setting rule 6a.) If the saved next group integer was not set, the body group is presented on the page to which the report is currently positioned. If the saved next group integer was set, the saved next group integer is moved into LINE-COUNTER, the saved next group integer is reset to zero, and fit test rule 3a is reapplied.

   b. If a body group has been presented on the page to which the report is currently positioned, a trial sum is computed in a work location. The trial sum is computed by adding the content of LINE-COUNTER to the integers of all LINE NUMBER clauses of the report group. If the trial sum is not greater than the body group's lower-limit integer, then the report group is presented on the current page. If the trial sum exceeds the body group's lower-limit integer, then page advance processing takes place. After the page heading report group (if defined) has been processed, fit test rule 3b is reapplied.

      If no body group has yet been presented on the page to which the report is currently positioned, it is determined whether the saved next group integer location was set when the final body group was presented on the preceding page. (See final LINE-COUNTER setting rule 6a.)

      If the saved next group integer was not set, the body group shall be presented on the page to which the report is currently positioned.

      If the saved next group integer was set, the saved next group integer is moved into LINE-COUNTER, the saved next group integer is reset to zero, and a trial sum is computed in a work location.

      The trial sum is computed by adding the content of LINE-COUNTER to the integer one and the integers of all but the first LINE NUMBER clause of the body group. If the trial sum is not greater than the body group's lower-limit integer, then the body group is presented on the current page. If the trial sum exceeds the body group's lower-limit integer, then page advance processing is executed. After the page heading report group (if defined) has been processed, the body group is presented on that page.

   c. If a body group has been presented on the page to which the report is currently positioned, page advance processing is executed. After the page heading report group (if defined) has been processed, fit test rule 3c is reapplied.

If no body group has yet been presented on the page to which the report is currently positioned, it is determined whether the saved next group integer location was set when the final body group was presented on the preceding page. (See final LINE-COUNTER setting rule 6a.) If the saved next group integer was not set, the body shall be presented on the page to which the report is currently positioned. If the saved next group integer was set, the saved next group integer is moved into LINE-COUNTER and the saved next group integer is reset to zero.

If the value in LINE-COUNTER is less than the integer of the first absolute LINE NUMBER clause, the body group shall be presented on the page to which the report is currently positioned. Otherwise, page advance processing is executed. After the page heading report group (if defined) has been processed, the body group is presented on that page.

4. First print line position rules

   a. The first print line of the body group is presented on the line number specified by the integer of its LINE NUMBER clause.

   b. If the value in LINE-COUNTER is equal to or greater than the line number specified by the FIRST DETAIL phrase of the PAGE clause and if no body group has previously been presented on the page to which the report is currently positioned, then the first print line of the current body group is presented on the line immediately following the line indicated by the value contained in LINE-COUNTER.

      If the value in LINE-COUNTER is equal to or greater than the line number specified by the FIRST DETAIL phrase of the PAGE clause and if a body group has previously been presented on the page to which the report is currently positioned, then the first print line of the current body group is presented on the line that is obtained by adding the content of LINE-COUNTER and the integer of the first LINE NUMBER clause of the current body group.

      If the value in LINE-COUNTER is less than the line number specified by the FIRST DETAIL phrase of the PAGE clause, then the first print line of the body group is presented on the line specified by the FIRST DETAIL phrase.

   c. The body group is not presented.

   d. The sum of the content of LINE-COUNTER and the integer of the first LINE NUMBER clause defines the line number on which the first print line is presented.

5. Next group rule

   The integer of the absolute NEXT GROUP clause must specify a line number that is not less than that specified in the FIRST DETAIL phrase of the PAGE clause and that is not greater than that specified in the FOOTING phrase of the PAGE clause.

6. Final LINE-COUNTER setting rules

   a. If the body group that has just been presented is a control footing report group and if the control footing report group is not associated with the highest level at which a control break was detected, then the final LINE-COUNTER setting is the line number on which the final print line of the control footing report group was presented.

For all other cases, a comparison of the line number on which the final print line of the body group was presented and the integer of the NEXT GROUP clause is made. If the former is less than the latter, the NEXT GROUP integer is placed into LINE-COUNTER as the final LINE-COUNTER setting. If the former is equal to or greater than the latter, then the line number specified by the FOOTING phrase of the PAGE clause is placed into LINE-COUNTER as the final LINE-COUNTER setting; in addition, the NEXT GROUP integer is placed into the saved next group integer location.

b. If the body group that has just been presented is a control footing report group and if the control footing report group is not associated with the highest level at which a control break was detected, then the final LINE-COUNTER setting is the line number on which the final print line of the control footing report group was presented.

   For all other cases, a trial sum is computed in a work location. The trial sum is computed by adding the integer of the NEXT GROUP clause to the line number on which the final print line of the body group was presented. If the sum is less than the line number specified by the FOOTING phrase of the PAGE clause, then that sum is placed into LINE-COUNTER as the final LINE-COUNTER setting. If the sum is equal to or greater than the line number specified by the FOOTING phrase of the PAGE clause, then the line number specified by the FOOTING phrase of the PAGE clause is placed into LINE-COUNTER setting.

c. If the body group that has just been presented is a control footing report group and if the control footing report group is not associated with the highest level at which a control break was detected, then the final LINE-COUNTER setting is the line number on which the final print line of the control footing report group was presented.

   For all other cases, the line number specified by the FOOTING phrase of the PAGE clause is placed into LINE-COUNTER as the final LINE-COUNTER setting.

d. The final LINE-COUNTER setting is the line number on which the final print line of the body group was presented.

e. LINE-COUNTER is unaffected by the processing of a nonprintable body group.

f. If the body group that has just been presented is a control footing report group and if the control footing report group is not associated with the highest level at which a control break was detected, then the final LINE-COUNTER setting is the line number on which the final print line of the control footing report group was presented.

   For all other cases, the sum of the line number on which the final print line was presented and the NEXT GROUP integer is placed into LINE-COUNTER as the final LINE-COUNTER setting.

Page Footing Presentation Rules

Table 10–3 points to the appropriate presentation rules for all permissible combinations of LINE NUMBER and NEXT GROUP clauses in a page footing report group.

Table 10–3. Page Footing Presentation Rules

Clause**		Applicable Rules # (If the PAGE clause is specified ##)				
Sequence of LINE NUMBER Clauses*	**NEXT GROUP Clause**	**Upper Limit**	**Lower Limit**	**First Print Line Position**	**Next Group**	**Final LINE-COUNTER Setting**
A R	Absolute	1	2	3a	4a	5a
A R	Relative	1	2	3a	4b	5b
A R		1	2	3a		5c
				3b		5d

* See 10.3.2. for a description of the abbreviations used in column 1.

** A blank entry in column 1 or column 2 indicates that the named clause is totally absent from the report group description entry.

\# A blank entry in an applicable rules column indicates the absence of the named rule for the given combination of LINE NUMBER and NEXT GROUP clauses.

If the PAGE clause is omitted from the report description entry, then a page heading report group cannot be defined. (See Section 10.3.2)

The page footing presentation rules are as follows:

1. Upper-limit rule

   The first line number on which the page footing report group can be presented is the line number obtained by adding one to the value of *integer-5* of the FOOTING phrase of the PAGE clause.

2. Lower-limit rule

   The last line number on which the page footing report group can be presented is the line number specified by *integer-1* of the PAGE clause.

3. First print line position rules

   The first print line of the page footing report group is presented on the line specified by the integer of its LINE NUMBER clause.

   The page footing report group is not presented.

4. Next group rules

   a. The NEXT GROUP integer must be greater than the line number on which the final print line of the page footing report group is presented. In addition, the NEXT GROUP integer must not be greater than the line number specified by *integer-D1* of the PAGE clause.

b. The sum of the NEXT GROUP integer and the line number on which the final print line of the page footing report group is presented must not be greater than the line number specified by *integer-1* of the PAGE clause.

5. Final LINE-COUNTER setting rules

   a. After the page footing report group is presented, the NEXT GROUP integer is placed into LINE-COUNTER as the final LINE-COUNTER setting.

   b. After the page footing report group is presented, the sum of the NEXT GROUP integer and the line number on which the final print line of the PAGE FOOTING report group is placed into LINE-COUNTER as the final LINE-COUNTER setting.

   c. After the page footing report group is presented the final LINE-COUNTER setting is the line number on which the final print line of the page footing report group was presented.

   d. LINE-COUNTER is unaffected by the processing of a nonprintable report group.

Report Footing Presentation Rules

Figure 10–3 points to the appropriate presentation rules for all permissible combinations of LINE NUMBER and NEXT GROUP clauses in a report footing report group. The report footing presentation rules are as follows:

1. Upper-limit rules

   If a page footing report group has been presented on the page to which the report is currently positioned, then the first line number on which the report footing report group can be presented is one greater than the final LINE-COUNTER setting established by the page footing report group.

   Otherwise, the first line number on which the report footing report group can be presented is the line number obtained by adding one and the value of *integer-5* of the PAGE clause. The first line number on which the report footing report group can be presented is the line number specified by the HEADING phrase of the PAGE clause.

2. Lower-limit rule

   The last line number on which the report footing report group can be presented is the line number specified by *integer-1* of the PAGE clause.

3. First print line position rules

   a. The first print line of the report footing report group is presented on the line specified by the integer of its LINE NUMBER clause.

   b. If a page footing report group has been presented on the page to which the report is currently positioned, then the sum of the final LINE-COUNTER setting established by the page footing report group and the integer of the first LINE NUMBER clause of the report footing report group defines the line number on which the first print line of the report footing report group is presented. Otherwise, the sum of the integer of the first LINE NUMBER clause of the report footing report group and the line number specified by the value of *integer-5* of the FOOTING phrase of the PAGE clause defines the line number on which the first print line of the report footing report group is presented.

c. The NEXT PAGE phrase in the first absolute LINE NUMBER clause directs that the report footing group is presented on a page on which no other report group has been presented. The first print line of the report footing report group is presented on the line number specified by the integer of its LINE NUMBER clause.

d. The sum of the content of LINE-COUNTER and the integer of the first LINE NUMBER clause defines the line number on which the first print line is presented.

e. The report footing report group is not presented.

4. Final LINE-COUNTER setting rules

a. The final LINE-COUNTER setting is the line number on which the final print line of the report footing report group is presented.

b. LINE-COUNTER is unaffected by the processing of a nonprintable report group.

**		Applicable Rules***							
		If the PAGE clause is specified.					If the PAGE clause is omitted.		
Sequence of LINE NUMBER clauses*	NEXT GROUP clause	Upper Limit	Lower Limit	First Print Line Position	Next Group	Final LINE-COUNTER Setting	First Print Line Position	Final LINE-COUNTER Setting
A R		1a	2	3a		4a	Illegal Combination+	
R		1a	2	3b		4a	3d	4a
NP R		1b	2	3c		4a	Illegal Combination+	
				3e		4b	3e	4b

* See 10.3.2. for a description of the abbreviations used in column 1.

** A blank entry in column 1 or column 2 indicates that the named clause is totally absent from the report group description entry.

*** A blank entry in an applicable rules column indicates the absence of the named rule for the given combination of LINE NUMBER and NEXT GROUP clauses.

\+ See 10.3.2.

Figure 10–3. Report Footing Presentation Rules

The COLUMN NUMBER Clause

Function

The COLUMN NUMBER clause identifies a printable item and specifies the position of the item on a print line.

General Format

```
COLUMN NUMBER IS integer-1
```

Syntax Rules

1. The COLUMN NUMBER clause can be specified only at the elementary level within a report group. The COLUMN NUMBER clause, if present, must appear in or be subordinate to an entry that contains a LINE NUMBER clause.
2. Within a given print line, the printable items must be defined in ascending column number order such that each printable item defined occupies a unique sequence of contiguous character positions.

General Rules

1. The COLUMN NUMBER clause indicates that the object of a SOURCE clause or the object of a VALUE clause or the sum counter defined by a SUM clause is to be presented on the print line. The absence of a COLUMN NUMBER clause indicates that the entry is not to be presented on a print line.
2. *Integer-1* specifies the column number of the leftmost character position of the printable item.
3. Space characters are implicitly inserted for all positions of a print line that are not occupied by printable items.
4. The leftmost position of the print line is considered to be column 1.

The Data Name Clause

Function

A data name specifies the name of the data item being described.

General Format

```
data-name-1
```

Syntax Rule

In the Report Section, *data-name-1* need not appear in a data description entry.

General Rule

In the Report Section, *data-name-1* must be given in the following cases:

when reference is to be made to the sum counter in the Procedure Division or Report Section; when a DETAIL report group is referred to in the UPON phrase of the SUM clause; and when *data-name-1* is required to provide sum counter qualification.

The GROUP INDICATE Clause

Function

The GROUP INDICATE clause specifies that the associated printable item is presented only on the first occurrence of its report group after a control break or page advance.

General Format

```
GROUP INDICATE
```

Syntax Rule

The GROUP INDICATE clause must be specified only in a DETAIL report group entry that defines a printable item.

General Rules

1. If a GROUP INDICATE clause is specified, it causes the SOURCE or VALUE clause to be ignored and spaces to be supplied, except in any of the following cases: on the first presentation of the DETAIL report group in the report on the first presentation of the DETAIL report group after every page advance on the first presentation of the DETAIL report group after every control break
2. If the report description entry specifies neither a PAGE clause nor a CONTROL clause, then a GROUP INDICATE printable item is presented the first time its DETAIL is presented after the INITIATE statement is executed. Thereafter, spaces are supplied for indicated items with SOURCE or VALUE clauses.

Level Number

Function

The level number indicates the position of a data item within the hierarchical structure of a logical record or report group.

General Format

```
level-number
```

Syntax Rules

1. A level number is required as the first element in each data description entry.
2. Data description entries subordinate to an RD entry must have level numbers 01 through 49 only.

General Rules

1. The level number 01 identifies the first entry in a report group.
2. Multiple level 01 entries subordinate to a report description entry having the level indicator RD do not represent implicit redefinitions of the same area.

The LINE NUMBER Clause

Function

The LINE NUMBER clause specifies vertical positioning information for its report group.

General Format

```
LINE NUMBER IS  { integer-1 [ON NEXT PAGE ] }
                { PLUS integer-2            }
```

Syntax Rules

1. *Integer-1* and *integer-2* must not exceed three significant digits in length.

   Neither *integer-1* nor *integer-2* can be specified in such a way as to cause any line of a report group to be presented outside the vertical subdivision of the page designated for that report group type, as defined by the PAGE clause. (See Section 10.3.2)

   Integer-2 can be zero.

2. Within a given report group description entry, an entry that contains a LINE NUMBER clause must not contain a subordinate entry that also contains a LINE NUMBER clause.
3. Within a given report group description entry, all absolute LINE NUMBER clauses must precede all relative LINE NUMBER clauses.
4. Within a given report group description entry, successive absolute LINE NUMBER clauses must specify integers that are in ascending order. The integers need not be consecutive.
5. If the PAGE clause is omitted from a given report group description entry, only relative LINE NUMBER clauses can be specified in any report group description entry within that report.
6. Within a given report group description entry, a NEXT PAGE phrase can appear only once and, if present, must be in the first LINE NUMBER clause in that report group description entry.
7. A LINE NUMBER clause with the NEXT PAGE phrase can appear only in the description of body groups and in a REPORT FOOTING report group.
8. Every entry that defines a printable item (see Section 10.3.2) must either contain a LINE NUMBER clause or be subordinate to an entry that contains a LINE NUMBER clause.
9. The first LINE NUMBER clause specified within a PAGE FOOTING report group must be an absolute LINE NUMBER clause.

General Rules

1. A LINE NUMBER clause must be specified to establish each print line of a report group.
2. The vertical positioning specified by a LINE NUMBER clause is set before presenting the print line established by that LINE NUMBER clause.

3. *Integer-1* specifies an absolute line number. An absolute line number specifies the line number on which the print line is presented.
4. *Integer-2* specifies a relative line number. If a relative LINE NUMBER clause is not the first LINE NUMBER in the report group description entry, then the line number on which its print line is presented is determined by calculating the sum of the line number on which the previous print line of the report group was presented and *integer-2* of the relative LINE NUMBER clause. If *integer-2* is zero, the line is printed on the same line as the previous print line.

   If a relative LINE NUMBER clause is the first LINE NUMBER clause in the report group description entry, then the line number on which its print line is presented is determined as follows:

   a. REPORT HEADING is presented on the line number obtained by adding the integer of the first LINE NUMBER clause and the value obtained by subtracting one from the value of *integer-2* of the HEADING phrase of the PAGE clause.

   b. PAGE HEADING

      - If a REPORT HEADING report group has been presented on the page on which the PAGE HEADING report group is to be presented, then the PAGE HEADING is presented relative to the final LINE-COUNTER setting of the REPORT HEADING.
      - If no REPORT HEADING has been presented on the page, the sum of the integer of the first LINE NUMBER clause of the PAGE HEADING report group and the value obtained by subtracting one from the value of *integer-2* of the HEADING phrase of the PAGE clause defines the line number on which the first print line of the PAGE HEADING report group is presented.

   c. Body Group (DETAIL, CONTROL HEADING, or CONTROL FOOTING)

      - If the value in LINE-COUNTER is less than the line number specified by the FIRST DETAIL phrase of the PAGE clause, then the first print line of the body group is presented on the line specified by the FIRST DETAIL phrase.
      - If the value in LINE-COUNTER is equal to or greater than the line number specified by the FIRST DETAIL phrase of the PAGE clause and if no body group has previously been presented on the page to which the report is currently positioned, then the first print line of the current body group is presented on the line immediately following the line indicated by the value contained in LINE-COUNTER.
      - If the value in LINE-COUNTER is equal to or greater than the line number specified by the FIRST DETAIL phrase of the PAGE clause and if a body group has previously been presented on the page to which the report is currently positioned, then the first print line of the current body group is presented on the line that is obtained by adding the contents of LINE-COUNTER and the integer of the first LINE NUMBER clause of the current body group.

d. REPORT FOOTING

- If a PAGE FOOTING report group has been presented on the page to which the report is currently positioned, then the sum of the final LINE-COUNTER setting established by the PAGE FOOTING report group and the integer of the first LINE NUMBER clause of the REPORT FOOTING report group defines the line number on which the first print line of the REPORT FOOTING report group is presented.
- If no PAGE FOOTING group has been presented, the sum of the integer of the first LINE NUMBER clause of the REPORT FOOTING report group, and the line number specified by the value of *integer-5* of the FOOTING phrase of the PAGE clause defines the line number on which the first print line of the REPORT FOOTING report group is presented.

5. The NEXT PAGE phrase specifies that the report group is to be presented beginning on the indicated line number on a new page. The use of this phrase in the first LINE NUMBER clause of a REPORT FOOTING report group indicates the REPORT FOOTING report group is to be presented on a page on which no other report group has been presented.

The NEXT GROUP Clause

Function

The NEXT GROUP clause specifies information for vertical positioning of a page following the presentation of the last line of a report group.

General Format



Syntax Rules

1. A report group entry must not contain a NEXT GROUP clause unless the description of that report group contains at least one LINE NUMBER clause.
2. *Integer-1* and *integer-2* must not exceed three significant digits in length.
3. If the PAGE clause is omitted from the report description entry only a relative NEXT GROUP clause can be specified in any report group description entry within that report.
4. The NEXT PAGE phrase of the NEXT GROUP clause must not be specified in a PAGE FOOTING report group.
5. The NEXT GROUP clause must not be specified in a REPORT FOOTING report group or in a PAGE HEADING report group.

General Rules

1. Any positioning of the page specified by the NEXT GROUP clause takes place after the presentation of the report group in which the clause appears. (See 10.3.2, general rule 4.)
2. A new value for LINE-COUNTER is determined from NEXT GROUP clause positioning information, along with information from the TYPE and PAGE clauses, and the value in LINE-COUNTER. (See Section 10.3.2, general rule 4.)
3. The NEXT GROUP clause is ignored when it is specified on a CONTROL FOOTING report group that is at a level other than the highest level at which a control break is detected.
4. The NEXT GROUP clause of a body group refers to the next body group to be presented, and therefore can affect the location at which the next body group is presented. The NEXT GROUP clause of a REPORT HEADING report group can affect the location at which the PAGE HEADING report group is presented. The NEXT GROUP clause of a PAGE FOOTING report group can affect the location at which the REPORT FOOTING report group is presented. (See Section 10.3.2, general rule 4.)

The SIGN Clause

Function

The SIGN clause specifies the position and the mode of representation of the operational sign when it is necessary to describe these properties explicitly.

General Format



Syntax Rules

1. The SIGN clause can be specified only for a numeric data description entry whose PICTURE contains the character S.
2. The numeric data description entries to which the SIGN clause applies must be described, implicitly or explicitly, as USAGE IS DISPLAY.
3. When the SIGN clause is included in a report group description entry, the SEPARATE CHARACTER phrase must be specified.

General Rules

1. The optional SIGN clause, if present, specifies the position and the mode of representation of the operational sign for the numeric data description entry to which it applies. The SIGN clause applies only to numeric data description entries whose PICTURE contains the character S; the S indicates the presence of, but neither the representation nor, necessarily, the position of, the operational sign.

2. A numeric data description entry whose PICTURE contains the character S, but to which no optional SIGN clause applies, has an operational sign, but neither the representation nor, necessarily, the position of the operational sign, is specified by the character S. In this (default) case, the operational sign is represented as an overpunch character (see Table 10–1). General rule 3 does not apply to such signed numeric data items.
3. Since a SIGN clause in a report group description entry must specify the SEPARATE CHARACTER phrase, the following applies:
   a. The operational sign is presumed to be the leading (or, respectively, trailing) character position of the elementary numeric data item; this character position is not a digit position.
   b. The letter S in a PICTURE character string is counted in determining the size of the item (in terms of standard data format characters).
   c. The operational signs for positive and negative are the standard data format characters + and -, respectively.
4. Every numeric data description entry whose PICTURE contains the character S is a signed numeric data description entry. If a sign clause applies to such an entry and conversion is necessary for purposes of computation or comparison, conversion takes place automatically.

The SOURCE Clause

Function

The SOURCE clause identifies the sending data item that is moved to an associated printable item defined within a report group description entry.

General Format

```
SOURCE IS identifier-1
```

Syntax Rules

1. *Identifier-1* can be defined in any section of the Data Division. If *identifier-1* is a Report Section item, it can only be
   a. PAGE-COUNTER
   b. LINE-COUNTER
   c. A sum counter of the report within which the SOURCE clause appears
2. *Identifier-1* specifies the sending data item of the implicit MOVE statement that moves the content of the data item referred to by *identifier-1* to the printable item. *Identifier-1* must be defined such that it conforms to the rules for sending items in the MOVE statement. (See Section 3.6.19.)

General Rule

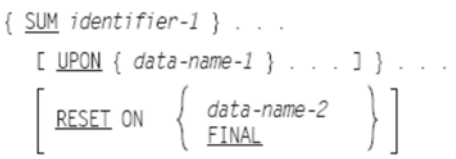
The print lines of a report group are formatted just before presenting the report group. (See Section 10.3.2.)Then the implicit MOVE statements specified by SOURCE clauses are executed.

The SUM Clause

Function

The SUM clause establishes a sum counter and names the data items to be summed.

General Format



Syntax Rules

1. The data item that is the subject of the report group description entry in which the SUM clause appears must not be defined as alphabetic. *Identifier-1* must reference a numeric data item. If *identifier-1* is defined in the REPORT Section, *identifier-1* must reference a sum counter.

   If the UPON phrase is omitted, any identifiers in the associated SUM clause that are themselves sum counters must be defined either in the same report group that contains this SUM clause or in a report group that is at a lower level in the control hierarchy of this report.

   If the UPON phrase is specified, any identifiers in the associated SUM clause must not be sum counters.

2. *Data-name-1* must be the name of a DETAIL report group described in the same report as the CONTROL FOOTING report group in which the SUM clause appears. *Data-name-1* can be qualified by a report name.

3. A SUM clause can appear only in the description of a CONTROL FOOTING report group.

4. *Data-name-2* must be one of the data names specified in the CONTROL clause for this report. *Data-name-2* must not be a lower level control than the associated control for the report group in which the RESET phrase appears.

   FINAL, if specified in the RESET phrase, must also appear in the CONTROL clause for this report.

5. The highest permissible qualifier of a sum counter is the report name.

General Rules

1. The SUM clause establishes a sum counter. The sum counter is a compiler-generated numeric data item with an operational sign. The size and decimal point location of the sum counter depend on the category of the data item specified by the report group description entry in which the SUM clause is specified. They are determined as follows:

   a. If the associated data item is numeric, the size and decimal point location of the sum counter are the same as those of that data item.

   b. If the associated data item is numeric edited, the size of the sum counter is the number of digit positions of that data item and the decimal point location is the same as that of the associated data item.

   c. If the associated data item is alphanumeric or alphanumeric edited, the size of the sum counter is the size of that data item, excluding any editing characters, or 18 characters, whichever is smaller, and the sum counter is an integer.

2. At object time, the value in each data item referred to by *identifier-1* is added to the SUM counter. This addition is consistent with the rules for arithmetic statements. (See Section 3.6.5 and Section 3.6.5.)

3. The printable size of the sum counter is equal to the number of receiving character positions specified by the PICTURE clause that accompanies the SUM clause in the description of the elementary item. Editing of a sum counter occurs only upon the presentation of that sum counter. At all other times the sum counter is treated as a numeric data item, with USAGE DISPLAY and size and scale corresponding to its picture clause description.

4. Only one sum counter exists for an elementary report entry regardless of the number of SUM clauses specified in the elementary report entry.

5. If the elementary report entry for a printable item contains a SUM clause, the sum counter serves as a source data item. The data contained in the sum counter is moved according to the rules of the MOVE statement to the printable item for presentation.

6. If a data name appears as the subject of an elementary report entry that contains a SUM clause, the data name is the name of the sum counter; the data name is not the name of the printable item that the entry can also define.

7. It is permissible for Procedure Division statements to alter the contents of sum counters.

8. Addition of the values of the data items referred to by identifiers into sum counters is performed during the execution of GENERATE and TERMINATE statements. There are three categories of sum counter accumulation: subtotaling, cross-footing, and rolling forward. Subtotaling is accomplished only during execution of GENERATE statements and after any control break processing, but before processing of the DETAIL report group. (See Section 10.4.3.) Cross-footing and rolling forward are accomplished during the processing of CONTROL FOOTING report groups. (See Section 10.3.2.)

9. The UPON phrase provides the capability to accomplish selective subtotaling for the DETAIL report groups named in the phrase.

10. Each individual addend is added into the sum counter at a time that depends on the characteristics of the addend.

    a. When the addend is a sum counter defined in the same CONTROL FOOTING report group, then the accumulation of that addend into the sum counter is termed cross-footing.

       Cross-footing occurs when a control break takes place and at the time the CONTROL FOOTING report group is processed.

       Cross-footing is performed according to the sequence in which sum counters are defined within the CONTROL FOOTING report group. That is, all cross-footing into the first sum counter defined in the CONTROL FOOTING report group is completed, and then all cross-footing into the second sum counter defined in the CONTROL FOOTING report group is completed. This procedure is repeated until all cross-footing operations are completed.

       When one of the addends is the sum counter defined by the data description entry in which that SUM clause appears, the initial value of that sum counter at the time of summation is used in the summing operation.

    b. When the addend is a sum counter defined in a lower level CONTROL FOOTING report group, the accumulation of that addend into the sum counter is termed rolling forward. A sum counter in a lower level CONTROL FOOTING report group is rolled forward when a break occurs and at the time that the lower level CONTROL FOOTING report group is processed.

    c. When the addend is not a sum counter, the accumulation into a sum counter of such an addend is called subtotaling. If the SUM clause contains the UPON phrase, the addends are subtotaled when a GENERATE statement for the designated DETAIL report group is executed.

       If the SUM clause does not contain the UPON phrase, the addends that are not sum counters are subtotaled when any GENERATE data name statement is executed for the report in which the SUM clause appears.

11. If two or more of the identifiers specify the same addend, the addend is added into the sum counter as many times as the addend is referred to in the SUM clause. It is permissible for two or more of the data names to specify the same DETAIL report group. When a GENERATE *data-name-1* statement for such a DETAIL report group is given, the incrementing occurs repeatedly, as many times as *data-name-1* appears in the UPON phrase.

12. For the subtotaling that occurs when a GENERATE *report-name-1* statement is executed, see Section 10.4.3.

13. In the absence of an explicit RESET phrase, a sum counter is set to zero at the time the CONTROL FOOTING report group within which the sum counter is defined is processed. If an explicit RESET phrase is specified, the sum counter is set to zero at the time of processing the designated level of the control hierarchy. (See Section 10.3.2.)

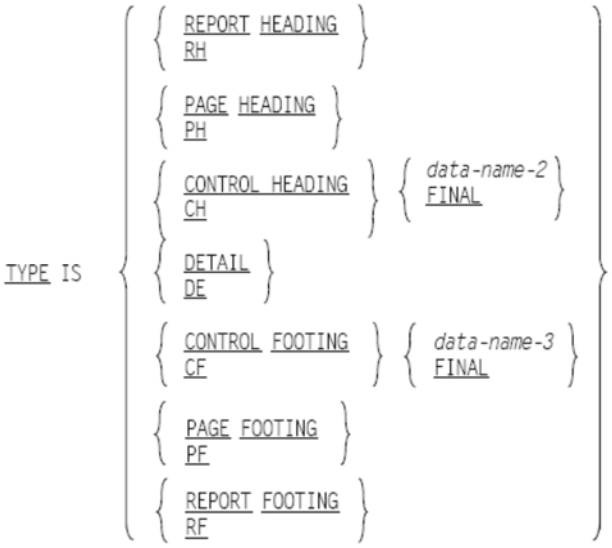
    Sum counters are initially set to zero during the execution of the INITIATE statement for the report containing the sum counter.

The TYPE Clause

Function

The TYPE clause specifies the particular type of report group that is described by this entry and indicates the time at which the report group is to be processed by the report writer control system.

General Format



Syntax Rules

1. The report group types have the following abbreviations:

RH	REPORT HEADING
PH	PAGE HEADING
CH	CONTROL HEADING
DE	DETAIL
CF	CONTROL FOOTING
PF	PAGE FOOTING
RF	REPORT FOOTING

2. REPORT HEADING, PAGE HEADING, CONTROL HEADING FINAL, CONTROL FOOTING FINAL, PAGE FOOTING, and REPORT FOOTING report groups can each appear no more than once in the description of a report.
3. PAGE HEADING and PAGE FOOTING report groups can be specified only if a PAGE clause is specified in the corresponding report description entry.

4. *Data-name-1, data-name-2,* and FINAL, if present, must be specified in the CONTROL clause of the corresponding report description entry. At most, one CONTROL HEADING report group and one CONTROL FOOTING report group can be specified for each data name or FINAL in the CONTROL clause of the report description entry. However, neither a CONTROL HEADING report group nor a CONTROL FOOTING report group is required for a data name or FINAL specified in the CONTROL clause of the report description entry.
5. In CONTROL FOOTING, PAGE HEADING, PAGE FOOTING, and REPORT FOOTING report groups, SOURCE clauses and associated USE statements must not refer to any of the following:
   a. Group data items containing a control data item
   b. Data items subordinate to a control data item
   c. A redefinition or renaming of any part of a control data item

   In PAGE HEADING and PAGE FOOTING report groups, SOURCE clauses and USE statements must not refer to control data names.
6. When a GENERATE *report-name-1* statement is specified in the Procedure Division, the corresponding report description entry must include no more than one DETAIL report group. If no GENERATE *data-name-1* statements are specified for such a report, a DETAIL report group is not required.
7. The description of a report must include at least one body group.

General Rules

1. DETAIL report groups are processed as a direct result of GENERATE statements. If a report group is other than TYPE DETAIL, its processing is an automatic function.
2. The REPORT HEADING phrase specifies a report group that is processed only once, per report, as the first report group of that report. The REPORT HEADING report group is processed during the execution of the chronologically first GENERATE statement for that report.
3. The PAGE HEADING phrase specifies a report group that is processed as the first report group on each page of that report except under the following conditions:
   a. A PAGE HEADING report group is not processed on a page that is to contain only a REPORT HEADING report group or only a REPORT FOOTING report group.
   b. A PAGE HEADING report group is processed as the second report group on a page when it is preceded by a REPORT HEADING report group that is not to be presented on a page by itself. (See Section 10.3.2, general rule 4.)
4. The CONTROL HEADING phrase specifies a report group that is processed at the beginning of a control group for a designated control data name or, in the case of FINAL, which is processed during the execution of the chronologically first GENERATE statement for that report. During the execution of any GENERATE statement at which a control break is detected, any CONTROL HEADING report groups associated with the highest control level of the break and lower levels are processed.
5. The DETAIL phrase specifies a report group that is processed when a corresponding GENERATE statement is executed.

6. The CONTROL FOOTING phrase specifies a report group that is processed at the end of a control group for a designated control data name.

   In the case of FINAL, the CONTROL FOOTING report group is processed only once per report as the last body group of that report. During the execution of any GENERATE statement in which a control break is detected, any CONTROL FOOTING report group associated with the highest level of the control break or more minor levels is presented. All CONTROL FOOTING report groups are presented during the execution of the TERMINATE statement if there has been at least one GENERATE statement executed for the report. (See Section 10.4.7.)

7. The PAGE FOOTING phrase specifies a report group that is processed as the last report group on each page except under the following conditions:

   a. A PAGE FOOTING report group is not processed on a page that is to contain only a REPORT HEADING report group or only a REPORT FOOTING report group.

   b. A PAGE FOOTING report group is processed as the second-to-last report group on a page when it is followed by a REPORT FOOTING report group that is not to be processed on a page by itself. (See Section 10.3.2, general rule 4.)

8. The REPORT FOOTING phrase specifies a report group that is processed only once per report and as the last report group of that report. The REPORT FOOTING report group is processed during the execution of a corresponding TERMINATE statement, if there has been at least one GENERATE statement executed for the report. (See Section 10.4.7.)

9. The sequence of steps executed during processing of a REPORT HEADING, PAGE HEADING, CONTROL HEADING, PAGE FOOTING, or REPORT FOOTING report group is described below.

   a. If there is a USE BEFORE REPORTING procedure that refers to the data name of the report group, the USE procedure is executed.

   b. If a SUPPRESS statement has been executed or if the report group is not printable, there is no further processing to be done for the report group.

   c. If a SUPPRESS statement has not been executed and the report group is printable, the print lines are formatted and the report group presented according to the presentation rules for that type of report group. (See Section 10.3.2, general rule 4.)

10. The sequence of steps executed when processing a CONTROL FOOTING report group follows.

    The GENERATE rules specify that when a control break occurs, the CONTROL FOOTING report groups are processed beginning at the minor level and proceeding upwards through the level at which the highest control break was sensed. Even though no CONTROL FOOTING report group has been defined for a given control data name, the step described in paragraph 10f is executed if a RESET phrase within the report description specifies that control data name.

    a. Sum counters are cross-footed; that is, all sum counters defined in this report group that are operands of SUM clauses in the same report group are added to their sum counters. (See Section 10.3.2)

b. Sum counters are rolled forward; that is, all sum counters defined in the report group that are operands of SUM clauses in higher level CONTROL FOOTING report groups are added to the higher level sum counters. (See Section 10.3.2)

c. If there is a USE BEFORE REPORTING procedure that refers to the data name of the report group, the USE procedure is executed.

d. If a SUPPRESS statement has been executed or if the report group is not printable, the step described in paragraph 10f is executed.

e. If the SUPPRESS statement has not been executed and the report group is printable, the print lines are formatted and the report group presented according to the presentation rules for CONTROL FOOTING report groups.

f. Sum counters are reset when processing this level in the control hierarchy. (See Section 10.3.2)

11. The DETAIL report group processing executed in response to a GENERATE *data-name-1* statement is described in paragraphs 11a through 11e.

    When the description of a report includes exactly one DETAIL report group, the detail-related processing executed in response to a GENERATE *report-name-1* statement is described in paragraphs 11a through 11e. These steps are performed as though a GENERATE *data-name-1* statement were being executed.

    When the description of a report includes no DETAIL report groups, the detail-related processing executed in response to a GENERATE *report-name-1* statement is described in paragraph 11a. This step is performed as though the description of the report included exactly one DETAIL report group and a GENERATE *data-name-1* statement were being executed.

    a. Any subtotaling that has been designated for the DETAIL report group is performed. (See Section 10.3.2)

    b. If there is a USE BEFORE REPORTING procedure that refers to the data name of the report group, the USE procedure is executed.

    c. If a SUPPRESS statement has been executed or if the report group is not printable, no further processing is done for the report group.

    d. If the DETAIL report group is being processed as a consequence of a GENERATE *report-name-1* statement, no further processing is done for the report group.

    e. If neither 11c nor 11d applies, the print lines are formatted and the report group presented according to the presentation rules for DETAIL report groups. (See 10.3.2, general rule 4.)

12. When processing a CONTROL HEADING, CONTROL FOOTING, or DETAIL report group, as described in general rules 9, 10, and 11, the processing of that body group can be interrupted after determining that the body group is to be presented and a page advance is executed (and process PAGE FOOTING and PAGE HEADING report groups) before actually presenting the body group.

13. During control break processing, the values of control data items used to detect a given control break are referred to as prior values.
    a. During control break processing of a CONTROL FOOTING report group, any references to control data items in a USE procedure or SOURCE clause associated with that CONTROL FOOTING report group are supplied with prior values.
    b. When a TERMINATE statement is executed, the prior control data item values are made available to SOURCE clause or USE procedure references in CONTROL FOOTING and REPORT FOOTING report groups as though a control break had been detected in the highest control data name.
    c. All other data item references within report groups and their USE procedures access the current values that are contained within the data items at the time the report group is processed.

The USAGE Clause

Function

The USAGE clause specifies the format of a data item in the computer storage.

General Format

```
[USAGE IS] DISPLAY
```

Syntax Rules

1. The USAGE clause can be written in any data description entry.
2. If the USAGE clause is written in the data description entry for a group item, it can also be written in the data description entry for a subordinate elementary item or group item.
3. The USAGE clause for a report group item can specify only USAGE IS DISPLAY.

General Rules

1. If the USAGE clause is written at a group level, it applies to each elementary item in the group.
2. The USAGE clause specifies the manner in which a data item is represented in the storage of a computer. It does not affect the use of the data item, although the specifications for some statements in the Procedure Division can restrict the USAGE clause of the operands referred to. The USAGE clause can affect the radix or type of character representation of the item.
3. The USAGE IS DISPLAY clause indicates that the format of the data is a standard data format.
4. If the USAGE clause is not specified for an elementary item or for any group to which the item belongs, the usage is implicitly DISPLAY.

The VALUE Clause

Function

The VALUE clause defines the value of Report Section printable items.

General Format

```
VALUE IS literal-1
```

Syntax Rules

1. A signed numeric literal must have associated with it a signed numeric PICTURE character string.
2. A numeric literal in a VALUE clause of an item must have a value that is within the range of values indicated by the PICTURE clause and must not have a value that would require truncation of nonzero digits. A nonnumeric literal in a VALUE clause of an item must not exceed the size indicated by the PICTURE clause.

General Rules

1. The VALUE clause must not conflict with other clauses in the data description of the item or in the data description within the hierarchy of the item. The following rules apply:
   a. If the category of the item is numeric, *literal-1* in the VALUE clause must be numeric.
   b. If the category of the item is alphabetic, alphanumeric, alphanumeric edited, or numeric edited, *literal-1* in the VALUE clause must be a nonnumeric literal. The literal is aligned in the data item as if the data item had been described as alphanumeric (see Section 2.4.19). Editing characters in the PICTURE clause are included in determining the size of the data item but have no effect on initialization of the data item (see Section 3.5.11). Therefore, the value for an edited item must be specified in an edited form.
   c. Initialization is not affected by any BLANK WHEN ZERO or JUSTIFIED clause that can be specified.
2. In the Report Section, if the elementary report entry containing the VALUE clause does not contain a GROUP INDICATE clause, then the printable item assumes the specified value each time its report group is printed. However, when the GROUP INDICATE clause is also present, the specified value is presented only when certain object time conditions exist (see Section 10.3.2).

10.4. Procedure Division in the Report Writer Module

This subsection describes the Procedure Division and its associated statements.

10.4.1. General Description

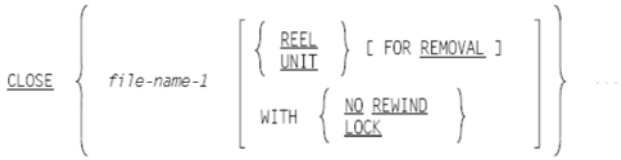
The Procedure Division contains declarative procedures when the USE BEFORE REPORTING statement from the Report Writer module is present in a COBOL source program. Shown below is the general format of the Procedure Division when the appropriate statement is present. (See Section 10.4.8 and Section 10.4.9.)

10.4.2. The CLOSE Statement

Function

The CLOSE statement terminates the processing of reel/units and files with optional rewind and/or lock or removal where applicable.

General Format



Syntax Rule

The files referred to in the CLOSE statement need not all have the same organization or access.

General Rules

Except where otherwise stated in the general rules below, the terms “reel” and “unit” are synonymous and completely interchangeable in the CLOSE statement. Treatment of sequential mass storage files is logically equivalent to the treatment of a file on tape or analogous sequential media. Treatment of a file contained in a multiple file tape environment is logically equivalent to the treatment of a sequential single-reel/unit file if the file is wholly contained on one reel.

1. The REEL or UNIT phrase must be used only for files assigned to tape.
2. A CLOSE statement can be executed only for a file in an open mode.
3. For the purpose of showing the effect of various types of CLOSE statements as applied to various storage media, all report files are divided into the following categories:

a. Nonreel/unit: A file whose output medium is such that the concepts of rewind and reels/units have no meaning. This includes all symbiont and mass storage files

b. Sequential single-reel/unit: sequential file that is entirely contained on one reel/unit. This includes tape files only.

c. Sequential multireel/unit: A sequential file that is contained on more than one reel/unit. This includes tape files only.

4. The results of executing each type of CLOSE for each category of file are summarized in Table 10–4.

Table 10–4. Relationship of Categories of Files and the Formats of the CLOSE Statements

CLOSE Statement Format	File Category		
	Nonreel/Unit	Sequential Single-Reel Unit	Sequential Multi-Reel Unit
CLOSE	C	C,G	A,C,G
CLOSE WITH LOCK	C,E	C,E,G	A,C,E,G
CLOSE WITH NO REWIND	C,H	B,C	A,B,C
CLOSE REEL/UNIT	F	F,G	F,G
CLOSE REEL/UNIT FOR REMOVAL	F	D,F,G	D,F,G

The definitions of the symbols in the table are as follows:

A

Effect on previous reels/units for an output report file

All reels/units in the report file prior to the current reel/unit are closed except those reels/units controlled by a prior CLOSE REEL/UNIT statement.

B

No rewind of current reel

The current reel/unit is left in its current position.

C

Close output report file

If label records are specified for the report file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. In addition, closing operations are executed. If label records are not specified for the report file, label processing does not take place but other closing operations are executed.

D

Reel/unit removal

The current reel or unit is rewound, when applicable, and the reel or unit is logically removed from the run unit; however, the reel or unit can be accessed again, in its proper order of reels or units within the report file, if a CLOSE statement without the REEL or UNIT phrase is subsequently executed for this report file followed by the execution of an OPEN statement for the report file.

E

File lock

The report file is locked and cannot be opened again during this execution of this run unit.

F

Close reel/unit

The following operations take place:

Reel/unit media

- The standard ending reel/unit label procedure is executed.
- A reel/unit swap. The current volume pointer is updated to point to the new reel/unit.
- The standard beginning reel/unit label procedure is executed.
- The next executed write operation that refers to that file directs the next logical data record to the next reel/unit of the file.

Nonreel/unit media

Execution of this statement is considered successful. The file remains in the open mode, and no action takes place except as specified in general rule 5.

G

Rewind

The current reel or analogous device is positioned at its physical beginning.

H

Optional phrases ignored

The CLOSE statement is executed as if none of the optional phrases is present.

5. The execution of the CLOSE statement causes the value of the I-O status associated with *file-name-1* to be updated. (See Section 4.2.5.)
6. All reports assigned to a report file by an INITIATE statement must be ended with the execution of a TERMINATE statement before a CLOSE statement is executed for that report file.

7. Following the successful execution of a CLOSE statement without the REEL or UNIT phrase, the report file is removed from the open mode, and the report file is no longer associated with the file connector.
8. If more than one *file-name-1* is specified in a CLOSE statement, the result of executing this CLOSE statement is the same as if a separate CLOSE statement had been written for each *file-name-1* in the same order as specified in the CLOSE statement.

10.4.3. The GENERATE Statement

Function

The GENERATE statement directs production of a report in accordance with the report description specified in the Report Section of the Data Division.

General Format



Syntax Rules

1. *Data-name-1* must name a TYPE DETAIL report group and can be qualified by a report name.
2. *Report-name-1* can be used only if the referenced report description contains
   a. A CONTROL clause
   b. Not more than one DETAIL report group
   c. At least one body group

General Rules

1. In response to a GENERATE *report-name-1* statement, summary processing is performed. If all of the GENERATE statements that are executed for a report are of the form GENERATE *report-name-1,* then the report that is produced is called a summary report. A summary report is one in which no DETAIL report group is presented.
2. In response to a GENERATE *data-name-1* statement, detail processing is performed that includes certain processing that is specific for the DETAIL report group designated by the GENERATE statement. Normally, the execution of a GENERATE *data-name-1* statement causes presentation of the designated DETAIL report group.

3. During the execution of the chronologically first GENERATE statement for a given report, the values within the control data items are saved. During the execution of the second and subsequent GENERATE statements for the same report and until a control break is detected, this set of control values is used to determine whether a control break has occurred. When a control break occurs, the new set of control values is saved and is thereafter used to sense for a control break until another control break occurs.
4. During report presentation, PAGE HEADING and PAGE FOOTING report groups, if defined, are processed when advancing the report to a new page for the purpose of presenting a body group. (See Section 10.3.2, general rule 4.)
5. When the chronologically first GENERATE statement for a given report is executed, the report groups that are named in the following list are processed in order, provided that such report groups are defined within the report description. PAGE HEADING and PAGE FOOTING report groups as described in general rule 4 are also processed. The actions taken when processing each type of report group are explained under the appropriate paragraph. (See Section 10.3.2)
    a. The REPORT HEADING report group is processed.
    b. The PAGE HEADING report group is processed.
    c. All CONTROL HEADING report groups are processed from major to minor.
    d. If a GENERATE *data-name-1* statement is being executed, the processing for the designated DETAIL report group is performed. If a GENERATE *report-name-1* statement is being executed, certain of the steps that are involved in the processing of a DETAIL report group are performed. (See Section 10.3.2)
6. When a GENERATE statement other than the chronologically first is executed for a given report, the steps enumerated below are processed as applicable. PAGE HEADING and PAGE FOOTING report groups as described in general rule 4 are also processed. The actions taken when processing each type of report group are explained under the appropriate paragraph. (See Section 10.3.2)
    a. Sense for control break. The rules for determining the equality of control data items are the same as those specified for relation conditions. If a control break has occurred, take the following steps:
        - Enable the CONTROL FOOTING USE procedures and CONTROL FOOTING SOURCE clauses to access the control data item values that were used to detect a given control break. (See Section 10.3.2)
        - Process the CONTROL FOOTING report groups in the order minor to major. The only CONTROL FOOTING report groups processed are ones that are not more major than the highest level at which a control break occurred.
        - Process the CONTROL HEADING report groups in major to minor order. The only CONTROL HEADING report groups processed are ones that are not more major than the highest level at which a control break occurred.
    b. If a GENERATE *data-name-1* statement is being executed, the processing for the designated DETAIL report group is performed. If a GENERATE *report-name-1* statement is being executed, certain steps that are involved in the processing of a DETAIL report group are performed. (See Section 10.3.2)

7. GENERATE statements for a report can be executed only after an INITIATE statement for the report has been executed and before a TERMINATE statement for the report has been executed.

10.4.4. The INITIATE Statement

Function

The INITIATE statement causes the processing of a report to begin.

General Format

```
INITIATE {report-name-1}. . .
```

Syntax Rule

Report-name-1 must be defined by a report description entry in the Report Section of the Data Division.

General Rules

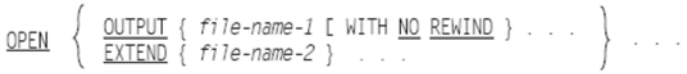
1. The INITIATE statement performs the following initialization functions for each named report:
   a. All sum counters are set to zero.
   b. LINE-COUNTER is set to zero.
   c. PAGE-COUNTER is set to one.
2. The INITIATE statement does not place the file associated with the report in the open mode; therefore, an OPEN statement with either the OUTPUT phrase or the EXTEND phrase for the file must be executed prior to the execution of the INITIATE statement.
3. A subsequent INITIATE statement for *report-name-1* must not be executed unless an intervening TERMINATE statement has been executed for that *report-name-1.*
4. If more than one report name is specified in an INITIATE statement, the result of executing this INITIATE statement is the same as if a separate INITIATE statement had been written for each report name in the same order as specified in the INITIATE statement.
5. When the DENSITY clause is present, the execution of the INITIATE statement sets the density specified.
6. When the VALUE OF CARTRIDGE-ID clause is present, the execution of the INITIATE statement requests the cartridge specified. (See Volume 2.)
7. When the VALUE OF FORMS-REQUEST clause is present for a report file, the execution of the INITIATE statement requests the form specified. (See Volume 2.)
8. For printer files, execution of the INITIATE statement causes the page margins to be set from the PAGE LIMIT clause if specified. If the PAGE LIMIT clause has not been specified, execution of the INITIATE statement causes the report to be defined as a single page of indefinite length. In both cases, the printer is positioned to the first printable line on the first (or only) page.

10.4.5. The OPEN Statement

Function

The OPEN statement initiates the processing of report files.

General Format



Syntax Rule

The OPEN statement for a report file must contain only the OUTPUT phrase or the EXTEND phrase. Additional syntax rules for the OPEN statement are located in the Sequential I-O module. (See Section 4.5.3.)

General Rules

1. The successful execution of an OPEN statement determines the availability of the file and results in the file being in an open mode. The successful execution of an OPEN statement associates the file with the file name through the file connector.

   A file is available if it is physically present and is recognized by the I-O control system. Table 10–5 shows the results of opening available and unavailable files.

Table 10–5. Availability of a File

Phrase	File is Available	File is Unavailable
OUTPUT	Normal open; the file contains no records	Open causes the file to be created
EXTEND	Normal open	Open is unsuccessful
EXTEND (optional file)	Normal open	Open causes the file to be created

2. When a file is not in an open mode, no statement can be executed that refers to the file, either explicitly or implicitly, except for an OPEN statement.
3. The OPEN statement for a report file must be executed prior to the execution of an INITIATE statement for any reports contained in the file.
4. A report file can be opened with the OUTPUT and EXTEND phrases in the same run unit. Following the initial execution of an OPEN statement for a report file, each subsequent OPEN statement execution for that same report file must be preceded by the execution of a CLOSE statement, without the REEL, UNIT, or LOCK phrase, for that file.
5. If label records are specified for the file and the OUTPUT phrase is specified, the execution of the OPEN statement causes the labels to be written in accordance with the conventions for output label writing. The behavior of the OPEN statement, when label records are specified but not present, or when label records are not specified but are present, is undefined.

6. If, during the execution of an OPEN statement, a file attribute conflict condition occurs, the execution of the OPEN statement is unsuccessful. (See Section 4.2.7.)
7. The NO REWIND phrase must be used only with
    a. Sequential single reel/unit files. (See Section 10.4.2.)
    b. Sequential files wholly contained within a single reel of tape within a multiple file tape environment. (See Section 4.3.3)
8. The NO REWIND phrase is ignored if it does not apply to the storage medium on which the file resides.
9. If the storage medium for the file permits rewinding, the following rules apply:
    a. When neither the EXTEND nor the NO REWIND phrase is specified, execution of the OPEN statement causes the file to be positioned at its beginning.
    b. When the NO REWIND phrase is specified, execution of the OPEN statement does not cause the file to be repositioned; the file must be already positioned at its beginning prior to execution of the OPEN statement.
10. When the EXTEND phrase is specified, the OPEN statement positions the file immediately after the last logical record for that file. The last logical record for a sequential file is the last record written in the file.
11. When the EXTEND phrase is specified and the LABEL RECORDS clause indicates label records are present, the execution of the OPEN statement includes the following steps:
    a. The beginning file labels are processed only in the case of a single reel/unit file.
    b. The beginning reel/unit labels on the last existing reel/unit are processed as though the file were being opened with the INPUT phrase.
    c. The existing ending file labels are processed as though the file were being opened with the INPUT phrase. These labels are then deleted.
    d. Processing then proceeds as though the file had been opened with the OUTPUT phrase.
12. Treatment of a file contained in a multiple file tape environment is logically equivalent to the treatment of a sequential file contained in a single file tape environment.
13. Whenever a set of files resides on a multiple file reel and one of this set of files is referenced in an OPEN statement, the following rules apply:
    a. Not more than one of the files can be in the open mode at one time.
    b. When one of the files referenced by a file name is the subject of an OPEN statement with the OUTPUT phrase, all files on the associated multiple file reel whose position numbers are less than the position number of that file must already exist on the reel at the time the OPEN statement is executed. Further, no file on that multiple file reel whose position number is greater than the position number of that file can exist at that time on the reel.
    c. Each of the files must be a sequential file.

14. For an optional file that is unavailable, the successful execution of an OPEN statement with an EXTEND phrase creates the file. This creation takes place as if the following statements were executed in the order shown:

```
OPEN OUTPUT file-name.
CLOSE file-name.
```

    These statements are followed by execution of the OPEN statement specified in the source program.

    The successful execution of an OPEN statement with the OUTPUT phrase creates the file. After the successful creation of a file, that file contains no data records.

15. Upon successful execution of the OPEN statement, the current volume pointer is set to point to
    a. The reel/unit containing the last logical record for an extend file
    b. The new reel/unit for an unavailable output or extend file
16. The execution of the OPEN statement causes the value of the I-O status associated with file name to be updated. (See Section 4.2.5.)
17. If more than one file name is specified in an OPEN statement, the result of executing this OPEN statement is the same as if a separate OPEN statement had been written for each file name in the same order as specified in the OPEN statement.
18. The minimum and maximum record size for a file is established at the time the file is created and must not subsequently be changed.

10.4.6. The SUPPRESS Statement

Function

The SUPPRESS statement causes the presentation of a report group to be inhibited.

General Format

```
SUPRESS PRINTING
```

Syntax Rule

The SUPPRESS statement can appear only in a USE BEFORE REPORTING procedure.

General Rules

1. The SUPPRESS statement inhibits presentation only for the report group named in the USE procedure within which the SUPPRESS statement appears.
2. The SUPPRESS statement must be executed each time the presentation of the report group is to be inhibited.
3. When the SUPPRESS statement is executed, processing of the following report group functions is inhibited:
    a. The presentation of the print lines of the report group
    b. The processing of all LINE clauses in the report group

c. The processing of the NEXT GROUP clause in the report group

d. The adjustment of LINE-COUNTER

10.4.7. The TERMINATE Statement

Function

The TERMINATE statement causes the processing of the specified reports to be completed.

General Format

```
TERMINATE {report-name-1}. . .
```

Syntax Rule

Report-name-1 must be defined by a report description entry in the Report Section of the Data Division.

General Rules

1. The TERMINATE statement causes the production of all the CONTROL FOOTING report groups beginning with the minor CONTROL FOOTING report group. Then the REPORT FOOTING report group is produced. The prior set of control data item values is made available to the CONTROL FOOTING and REPORT FOOTING SOURCE clauses and USE procedures, as though a control break has been sensed in the most major control data name.
2. If no GENERATE statements have been executed for a report during the interval between the execution of an INITIATE statement and a TERMINATE statement for that report, the TERMINATE statement does not cause production of any report groups or performance of any of the related processing.
3. During report presentation, PAGE HEADING and PAGE FOOTING report group are processed, if defined, when advancing the report to a new page in order to present a body group. (See Section 10.3.2, general rule 4.)
4. The TERMINATE statement cannot be executed for a report unless the TERMINATE statement was chronologically preceded by an INITIATE statement for that report and no TERMINATE statement has yet been executed for the report.
5. If more than one report name is specified in a TERMINATE statement, the result of executing this TERMINATE statement is the same as if a separate TERMINATE statement had been written for each report name in the same order as specified in the TERMINATE statement.
6. The TERMINATE statement does not close the file with which the report is associated; a CLOSE statement for the file must be executed. Every report that is in an initiated condition must be terminated before a CLOSE statement is executed for the associated file.

10.4.8. The USE AFTER STANDARD EXCEPTION PROCEDURE Statement

Function

The USE AFTER STANDARD PROCEDURE statement specifies procedures for input-output error handling that are in addition to the standard procedures provided by the input-output control system.

General Format



Syntax Rules

1. A USE statement, when present, must immediately follow a section header in the declaratives portion of the Procedure Division and must appear in a sentence by itself. The remainder of the section must consist of zero, one, or more procedural paragraphs that define the procedures to be used.
2. The USE statement is never executed; it merely defines the conditions calling for the execution of the USE procedures.
3. Appearance of *file-name-1* in a USE statement must not cause the simultaneous request for execution of more than one USE procedure.
4. The words ERROR and EXCEPTION are synonymous and can be used interchangeably.
5. The files implicitly or explicitly referenced in the USE statement need not all have the same organization or access.
6. The OUTPUT and EXTEND phrases can each be specified only once in the declaratives portion of a given Procedure Division.

General Rules

1. Declarative procedures can be included in any COBOL source program irrespective of whether the program contains or is contained within another program. A declarative is invoked when any of the conditions described in the USE statement that prefaces the declarative occurs while the program is being executed. Only a declarative within the separately compiled program that contains the statement that caused the qualifying condition is invoked when any of the conditions described in the USE statement that prefaces the declarative occurs while that separately compiled program is being executed. If no qualifying declarative exists in the separately compiled program, no declarative is executed.
2. Within a declarative procedure, there must be no reference to any nondeclarative procedures.
3. Procedure names associated with a USE statement can be referenced in a different declarative section or in a nondeclarative procedure only with a PERFORM statement.

4. When *file-name-1* is specified explicitly, no other USE statement applies to *file-name-1.*
5. The procedure associated with a USE statement is executed by the input-output control system after completion of the standard input-output exception routine upon the unsuccessful execution of an input-output operation unless an AT END phrase takes precedence. The rules concerning when the procedures are executed are as follows:
   a. If *file-name-1* is specified, the associated procedure is executed when the condition described in the USE statement occurs.
   b. If OUTPUT is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the output mode or in the process of being opened in the output mode, except those files referenced by *file-name-1* in another USE statement specifying the same condition.
   c. If EXTEND is specified, the associated procedure is executed when the condition described in the USE statement occurs for any file open in the extend mode or in the process of being opened in the extend mode, except those files referenced by *file-name-1* in another USE statement specifying the same condition.
6. After execution of the USE procedure, control is transferred to the invoking routine in the input-output control system. If the I-O status value does not indicate a critical input-output error, the input-output control system returns control to the next executable statement following the input-output statement whose execution caused the exception.
7. Within a USE procedure, there must not be the execution of any statement that would cause the execution of a USE procedure that had previously been invoked and had not yet returned control to the invoking routine.

10.4.9. The USE BEFORE REPORTING Statement

Function

The USE BEFORE REPORTING statement specifies Procedure Division statements that are executed just before a report group named in the Report Section of the Data Division is presented.

General Format

```
USE BEFORE REPORTING identifier-1.
```

Syntax Rules

1. A USE BEFORE REPORTING statement, when present, must immediately follow a section header in the declaratives portion of the Procedure Division and must appear in a sentence by itself. The remainder of the section must consist of zero, one, or more procedural paragraphs that define the procedures to be used.
2. *Identifier-1* must refer to a report group. *Identifier-1* must not appear in more than one USE BEFORE REPORTING statement.

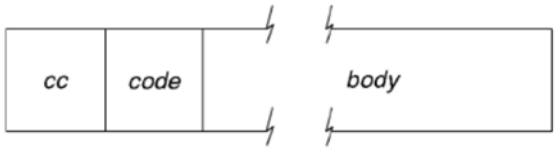
3. The GENERATE, INITIATE, or TERMINATE statements must not appear in a paragraph within a USE BEFORE REPORTING procedure. A PERFORM statement in a USE BEFORE REPORTING procedure must not have GENERATE, INITIATE, or TERMINATE statements in its range.
4. A USE BEFORE REPORTING procedure must not alter the value of any control data item.
5. The USE BEFORE REPORTING statement itself is never executed; it merely defines the conditions calling for the execution of the USE procedures.

General Rules

1. Declarative procedures can be included in any COBOL source program even if the program contains or is contained within another program. A declarative is invoked just before the named report group is produced during the execution of the program. The report group is named by *identifier-1* in the USE BEFORE REPORTING statement that prefaces the declarative.
2. Within a declarative procedure, there must be no reference to any nondeclarative procedures.
3. Procedure names associated with a USE BEFORE REPORTING statement can be referred to in a different declarative section or in a nondeclarative procedure only with a PERFORM statement.
4. In the USE BEFORE REPORTING statement, the designated procedures are automatically executed just before the named report group is produced. (See Section 10.3.2)
5. Within a USE procedure, no execution of any statement can cause the execution of a USE procedure that had previously been invoked and had not yet returned control to the invoking routine.

10.5. Report Format on Files Other Than Printer Files

If the file designated for a report is assigned to other than the printer, a record on the file will be in the following format:



where

cc

is the carriage control character specified as a signed binary item:

negative (-) means advance to the specified line on the next page; positive (+) means skip the specified number of lines. For example, -5 means skip to line 5 on the next page; +5 means advance 5 lines from the current position.

code

is the 2-character code optionally specified by literal-1 in the CODE clause.

body

is the report line.

The file and record description needed to process this file would be as follows:

```
FD  report-file
    (normal FD clauses)
    DATA RECORD IS report-image.
01  report-image.
    02 carriage-control     PIC  S1(18) BINARY-1.
    02 code                 PIC  X(2).
    02 body                 PIC  X(size-of-report-line).
```

10.6. Examples

This subsection includes two sample programs.

10.6.1. Program 1

The input data, source program, and output report for this sample program are described in the following paragraphs.

Input Data

The input data fields are arranged in the following format, using the first record as an example.

```
Month     Day      Dept #    # of Purchases    Item     Cost

JANUARY   05       B10       02                A        00300
```

The decimal point in the cost field is assumed to be two places from the right.

The following records are used in this report:

```
JANUARY  05 B10 02 A 00300
JANUARY  05 B12 01 A 00400
JANUARY  05 B12 03 C 01500
JANUARY  12 B11 02 B 01200
JANUARY  12 B14 10 A 02000
JANUARY  12 B14 10 C 06000
JANUARY  19 B11 02 B 01200
JANUARY  26 B10 04 B 01200
JANUARY  26 B10 01 C 00500
JANUARY  31 B11 16 A 03200
JANUARY  31 B11 04 B 02400
JANUARY  31 B11 20 D 04000
FEBRUARY 09 B14 05 A 01000
```

```
FEBRUARY 09 B14 05 B 02000
FEBRUARY 16 B10 02 A 00300
FEBRUARY 16 B11 02 B 01200
FEBRUARY 16 B12 01 A 00400
FEBRUARY 16 B13 01 A 03000
FEBRUARY 23 B14 10 A 02000
FEBRUARY 23 B15 20 A 02000
FEBRUARY 23 B16 30 A 01500
FEBRUARY 23 B17 30 D 03000
FEBRUARY 28 B10 02 A 00300
FEBRUARY 28 B12 01 A 00400
```

Source Program

```
000005 IDENTIFICATION DIVISION.
000010 PROGRAM-ID. EXPENDITURES.
000015 ENVIRONMENT DIVISION.
000020 CONFIGURATION SECTION.
000025 SOURCE-COMPUTER. UNISYS-2200.
000030 OBJECT-COMPUTER. UNISYS-2200.
000035 INPUT-OUTPUT SECTION.
000040 FILE-CONTROL.
000045     SELECT INFILE ASSIGN TO CARD-READER.
000050     SELECT REPORT-FILE ASSIGN TO PRINTER.
000055 DATA DIVISION.
000060 FILE SECTION.
000065 FD  INFILE
000070      LABEL RECORDS ARE OMITTED DATA RECORD IS IN-REC.
000075 01  IN-REC.
000080     02 MONTH PIC A(9).
000085     02 DAY-1 PIC XX.
000090     02 FILLER PIC X.
000095     02 DEPT PIC XXX.
000100     02 FILLER PIC X.
000105     02 PURCHS PIC 99.
000110     02 FILLER PIC X.
000115     02 ITEM PIC X.
000120     02 FILLER PIC X.
000125     02 COST PIC 999V99.
000130     02 FILLER PIC X(54).
000135 FD REPORT-FILE
000140      LABEL RECORDS ARE OMITTED REPORT IS EXPENDITURES.
000145 REPORT SECTION.
000150 RD  EXPENDITURES
000155       CONTROLS ARE FINAL, MONTH, DAY-1
000160       PAGE LIMIT IS 66 LINES, FIRST DETAIL 14,
000165       LAST DETAIL 54, FOOTING 59.
000170 01  TYPE RH.
000175     02 LINE NUMBER IS 4.
000180       03 COLUMN NUMBER 28 PICTURE X(26) VALUE IS
000185       "USER MANUFACTURING COMPANY".
```

```
000190     02 LINE NUMBER IS 6.
000195        03 COLUMN NUMBER IS 27 PICTURE X(28) VALUE IS
000200             "EXPENDITURES REPORT FOR 1989".
000205 01  TYPE PH.
000210     02 LINE NUMBER IS 10.
000215        03 COLUMN 5 PICTURE X(5) VALUE IS "MONTH".
000220        03 COLUMN 16 PICTURE X(36) VALUE IS
000225          "W/E DEPT NO ITEM COST".
000230        03 COLUMN 63 PICTURE X(10) VALUE IS "CUMULATIVE".
000235     02 LINE NUMBER IS 11.
000240        03 COLUMN 28 PICTURE X(16) VALUE IS
000245          "PURCHASES CODE".
000250        03 COLUMN 66 PICTURE X(4) VALUE IS "COST".
000255 01  TYPE CH MONTH.
000260     02 LINE PLUS 2.
000265        03 COLUMN 1 PICTURE X(9) SOURCE IS MONTH.
000270        03 COLUMN 11 PICTURE X(12) VALUE IS "EXPENDITURES".
000275 01  COST-REPORT TYPE DE.
000280     02 LINE PLUS 2.
000285        03 COLUMN  5 PICTURE X(9) SOURCE MONTH GROUP INDICATE.
000290        03 COLUMN 16 PICTURE XX   SOURCE DAY-1 GROUP INDICATE.
000295        03 COLUMN 24 PICTURE XXX  SOURCE DEPT.
000300        03 COLUMN 30 PICTURE Z9   SOURCE PURCHS.
000305        03 COLUMN 41 PICTURE X    SOURCE ITEM.
000310        03 PRICE COLUMN 47 PICTURE ZZ9.99 SOURCE COST.
000315 01  TYPE CF DAY-1.
000320     02 LINE PLUS 2.
000325        03 COLUMN 1 PICTURE IS X(19) VALUE IS
000330          "TOTAL COSTS FOR W/E".
000335        03 COLUMN 21 PICTURE IS X(9) SOURCE IS MONTH.
000340        03 COLUMN 31 PICTURE IS X(2) SOURCE IS DAY-1.
000345        03 WEEK-PRICE COLUMN 47 PICTURE ZZ9.99 SUM COST.
000350        03 CUME-PRICE COLUMN 64 PICTURE ZZ9.99 SUM WEEK-PRICE
000355          RESET ON FINAL.
000360 01  TYPE CF MONTH.
000365     02 LINE 55.
000370        03 COLUMN 4 PICTURE X(15) VALUE IS
000375          "TOTAL COSTS FOR".
000380        03 COLUMN 20 PICTURE X(9) SOURCE MONTH.
000385        03 COLUMN 31 PICTURE ZZ9.99 SUM COST.
000390 01  TYPE PF.
000395     02 LINE 60.
000400        03 COLUMN 55 PICTURE X(4) VALUE "PAGE".
000405        03 COLUMN 60 PICTURE 99 SOURCE PAGE-COUNTER.
000410 01  TYPE CF FINAL.
000415     02 LINE 58.
000420        04 COLUMN 1 PICTURE X(39) VALUE
000425             "TOTAL EXPENDITURES FOR YEAR-TO-DATE WAS".
000430        04 COLUMN 42 PICTURE 999.99 SOURCE CUME-PRICE.
000435 01  TYPE RF.
000440     02 LINE PLUS 2.
000445        03 COLUMN 32 PICTURE X(10) VALUE IS "END REPORT".
```

```
000450 PROCEDURE DIVISION.
000455 P1. OPEN INPUT INFILE OUTPUT REPORT-FILE.
000460     INITIATE EXPENDITURES.
000465 P2. READ INFILE RECORD AT END GO TO P3.
000470     GENERATE COST-REPORT.
000475     GO TO P2.
000480 P3. TERMINATE EXPENDITURES.
000485     CLOSE INFILE.
000490     CLOSE REPORT-FILE.
000495     STOP RUN.
```

Output Report

The following report is produced by the source program in 10.6.1. The numbers in parentheses are used in the explanation in Section 10.6.1.

```
          (1)  USER MANUFACTURING COMPANY

           (1)  EXPENDITURES REPORT FOR 1989

(2) MONTH     W/E    DEPT    NO       ITEM    COST      CUMULATIVE
(2)                        PURCHASES   CODE                 COST

JANUARY   EXPENDITURES  (3)
(4) JANUARY    05     B10    2          A      3.00
    (5)      (5)      B12   1           A      4.00
                      B12     3         C      15.00

TOTAL COSTS FOR W/E JANUARY  05                22.00       22.00   (6)
(7)  JANUARY    12     B11    2          B      12.00
                       B14    10         A      20.00
                       B14    10         C      60.00

TOTAL COSTS FOR W/E JANUARY  12                92.00     114.00
    JANUARY    19     B11     2          B      12.00

TOTAL COSTS FOR W/E JANUARY  19                12.00     126.00
    JANUARY    26     B10     4          B      12.00
                      B10     1          C      5.00

TOTAL COSTS FOR W/E JANUARY  26                17.00     143.00
    JANUARY 31        B11    16          A      32.00
                      B11     4          B      24.00
                      B11    20          D      40.00

TOTAL COSTS FOR W/E JANUARY  31                96.00     239.00
(8) TOTAL COSTS FOR JANUARY 239.00
                     (9)  PAGE 01
(10) MONTH     W/E    DEPT    NO       ITEM    COST      CUMULATIVE
                           PURCHASES   CODE                 COST
```

```
FEBRUARY   EXPENDITURES  (11)
(4) FEBRUARY   09    B14     5          A       10.00
    (5)      (5)     B14     5          B       20.00

TOTAL COSTS FOR W/E FEBRUARY  09               30.00     269.00  (6)
(7) FEBRUARY  16      B10      2          A       3.00
                      B11      2          B      12.00
                      B12      1          A       4.00
                      B13      1          A      30.00

TOTAL COSTS FOR W/E FEBRUARY  16              49.00        318.00
    FEBRUARY   23    B14     10          A       20.00
                      B15     20          A       20.00
                      B16     30          A       15.00
                      B17     30          D       30.00

TOTAL COSTS FOR W/E FEBRUARY 23                85.00       403.00
    FEBRUARY   28    B10      2          A        3.00
                      B12      1          A        4.00

TOTAL COSTS FOR W/E FEBRUARY 28                 7.00       410.00
  TOTAL COSTS FOR FEBRUARY   171.00
TOTAL EXPENDITURES FOR YEAR-TO-DATE WAS 410.00  (12)
                                          (13)  PAGE 02
                        (14)  END REPORT
```

Relationship between Output Report and Source Program

In the following explanation, the numbers relate to the numbers in parentheses in the output report in Section 10.6.1.

The 6-digit numbers refer to the sequence numbers of the source listing in Section 10.6.1.

1. Is the REPORT HEADING produced from source lines 000170 through 000200. The output lines are printed on lines 4 and 6, as specified in the report group description. This output is the result of the first GENERATE statement being executed in the object program.
2. Is the PAGE HEADING produced from source lines 000205 through 000250. The output lines are printed on lines 10 and 11, as specified in the report group description. This output is also the result of the first GENERATE statement being executed in the object program.
3. Is the CONTROL HEADING produced from source lines 000255 through 000270. The output line is printed on FIRST DETAIL *integer-3* (line 14) since this is the first body group with relative line spacing to be printed on the output page. Likewise, this output is the result of the first GENERATE statement being executed in the object program.

4. Is the DETAIL produced from source lines 000275 through 000310. Since this is not the first body group to be presented on the output page, it is printed two lines from the last line used by the previous report group. Note that each time the DETAIL is presented, the source item “COST” is added to the sum counter’s “WEEK-PRICE” and to the unnamed sum counter in the CONTROL FOOTING “MONTH” report group.
5. Note that the data items January and 05 appearing on the second input image do not appear as print output. This is a result of the GROUP INDICATE clauses in source lines 000285 and 000290. The GROUP INDICATE clause causes the printing of the item appearing in the source line to be suppressed at all times except for the first print line appearing after a control break and after a page break.
6. Is the CONTROL FOOTING “DAY-1” produced from source lines 000315 through 000355. This report group presentation results from the break in the value of the data item “DAY-1” compared to its previous value. Since this is not the first body group to be presented on the output page, it is printed two lines from the last line used by the previous report group. The RESET ON FINAL clause prevents the clearing of the sum counter “CUME-PRICE” until the “FINAL” control break occurs. The sum counter “WEEK-PRICE” is reset to zero.
7. The data items “MONTH” and “DAY-1” appear as printed output because this is the first print line following a control break.
8. Is the CONTROL FOOTING “MONTH” produced from source lines 000360 through 000385. This report group presentation results from the break in the value of the data item “MONTH” compared to its previous value. The output line is printed on line 55 as specified in source line 000365. The control break is considered to occur for the most major identifier at which a change has been recognized. Therefore, in this example, the control identifier “MONTH” is recognized as changing (not the control identifier “DAY-1”), and appropriate footings starting from the most minor control (“DAY-1”) through the control break (“MONTH”) are presented.
9. Is the PAGE FOOTING produced from source lines 000390 through 000405. The output line is printed on line 60 as specified in source line 000395. This report group presentation results from the following considerations:
    a. Following the CONTROL FOOTING "MONTH" report group print line, the next group to be printed is the CONTROL HEADING "MONTH" report group.
    b. However, the last allowable print line for a CONTROL HEADING is LAST DETAIL *integer-4* (line 54). This is recognized and a PAGE FOOTING is produced as part of the page break procedure.
10. Is the PAGE HEADING produced as part of the page break procedure.
11. Is the CONTROL HEADING “MONTH” still being presented as part of the control break recognized above. Appropriate CONTROL HEADING report groups are presented from the control break level down through the minor CONTROL HEADING report group. In our example, only CONTROL HEADING "MONTH" is presented since this is the only CONTROL HEADING specified.
12. Is the CONTROL FOOTING “FINAL” produced from source lines 000410 through 000430. These report groups, as well as the two CONTROL FOOTING report groups preceding it as print output, are results of the verb TERMINATE. The output line is printed on line 58, as specified in source line 000415.

13. Is the PAGE FOOTING produced as a result of executing the TERMINATE verb in the object program.
14. Is the REPORT FOOTING produced from source lines 000435 through 000445 as a result of executing the TERMINATE verb in the object program. Since a PAGE FOOTING was presented on the page, the REPORT FOOTING is presented on the last line used by the PAGE FOOTING plus 2 (line 62). Note that this falls within the print area defined by the PAGE LIMIT clause; that is, the last allowable print line for the REPORT FOOTING is less than or equal to PAGE LIMIT *integer-1.*

    Source line 000460 directs all sum counters to be reset to zero, to initialize the LINE-COUNTER to zero, and to initialize the PAGE-COUNTER to one. Source line 000470 generates the desired report. Source line 00480 in turn terminates the desired report and presents any CONTROL FOOTING, PAGE FOOTING, or REPORT FOOTING report groups necessary to accomplish this termination.

    Note: *Column spacing and line spacing in 10.6.1 are relative. For exact placement refer to the source code in 10.6.1.*

10.6.2. Program 2

The input data, source program, and output report for this sample program are described in the following paragraphs.

Input Data

The input data fields are arranged in the following format, using the first record as an example.

```
Month     Day     Dept #     # of Purchases    Item      Cost

JANUARY   05      B10        02                A         00300
```

The decimal point in the cost field is assumed to be two places from the right.

The following records are used in this report:

```
JANUARY  05 B10 02 A 00300
JANUARY  05 B12 01  A 00400
JANUARY  05 B12 03C 01500
JANUARY  12 B11 02 B 01200
JANUARY  12 B14 10 A 02000
JANUARY  12 B14 10 C 06000
FEBRUARY 09 B14 05 A 01000
FEBRUARY 09 B14 05 B 02000
```

Source Program

```
000005 IDENTIFICATION DIVISION.
000010 PROGRAM-ID. EXPENDITURES.
000015 ENVIRONMENT DIVISION.
000020 CONFIGURATION SECTION.
000025 SOURCE-COMPUTER. UNISYS-2200.
```

```
000030 OBJECT-COMPUTER. UNISYS-2200.
000035 INPUT-OUTPUT SECTION.
000040 FILE-CONTROL.
000045     SELECT INFILE ASSIGN TO CARD-READER.
000050     SELECT REPORT-FILE ASSIGN TO PRINTER.
000055 DATA DIVISION.
000060 FILE SECTION.
000065 FD  INFILE
000070      LABEL RECORDS ARE OMITTED DATA RECORD IS IN-REC.
000075 01  IN-REC.
000080     02 MONTH PIC A(9).
000085     02 DAY-1 PIC XX.
000090     02 FILLER PIC X.
000095     02 DEPT PIC XXX.
000100     02 FILLER PIC X.
000105     02 PURCHS PIC 99.
000110     02 FILLER PIC X.
000115     02 ITEM PIC X.
000120     02 FILLER PIC X.
000125     02 COST PIC 999V99.
000130     02 FILLER PIC X(54).
000135 FD  REPORT-FILE
000140      LABEL RECORDS ARE OMITTED REPORT IS EXPENDITURES.
000145 REPORT SECTION.
000150 RD   EXPENDITURES
000155       CONTROLS ARE FINAL, MONTH, DAY-1
000160       PAGE LIMIT IS 66 LINES, FIRST DETAIL 14,
000165       LAST DETAIL 54, FOOTING 59.
000170 01  TYPE RH NEXT GROUP NEXT PAGE.
000175     02 LINE NUMBER 3.
000180        03 COLUMN NUMBER 28 PIC X(26) VALUE IS
000185              "USER MANUFACTURING COMPANY".
000190     02 LINE NUMBER IS PLUS 13.
000195        03 COLUMN NUMBER 27 PIC X(28) VALUE IS
000200              "EXPENDITURES REPORT FOR 1989".
000205 01  TYPE PH.
000210     02 LINE NUMBER IS 4.
000215        03 COLUMN 5 PIC X(5) VALUE IS "MONTH".
000220        03 COLUMN 16 PIC X(36) VALUE IS
000225               "W/E DEPT NO     ITEM COST".
000230        03 COLUMN 63 PIC X(10) VALUE IS
000235               "CUMULATIVE".
000240     02 LINE NUMBER PLUS 2.
000245        03 COLUMN 28 PIC X(16) VALUE IS
000250             "PURCHASES   CODE".
000255        03 COLUMN 66 PIC X(4) VALUE IS "COST".
000260 01  TYPE CH MONTH.
000265     02 LINE NUMBER PLUS 2.
000270        03 COLUMN 1 PIC X(9) SOURCE IS MONTH.
000275        03 COLUMN 11 PIC X(12) VALUE IS "EXPENDITURES".
000280 01  COST-REPORT TYPE DE NEXT GROUP PLUS 1.
000285     02 LINE PLUS 1.
```

```
000290        03 COLUMN 5 PIC X(9) SOURCE MONTH GROUP INDICATE.
000295        03 COLUMN 16 PIC XX SOURCE DAY-1 GROUP INDICATE.
000300        03 COLUMN 24 PIC XXX SOURCE DEPT.
000305        03 COLUMN 30 PIC Z9 SOURCE PURCHS.
000310        03 COLUMN 41 PIC X SOURCE ITEM.
000315        03 PRICE COLUMN 47 PIC ZZ9.99 SOURCE COST.
000320 01  TYPE CF DAY-1 NEXT GROUP PLUS 5.
000325     02 LINE PLUS 1.
000330        03 COLUMN 1 PIC X(19) VALUE IS
000335              "TOTAL COSTS FOR W/E".
000340        03 COLUMN 21 PIC IS X(9) SOURCE IS MONTH.
000345        03 COLUMN 31 PIC IS X(2) SOURCE IS DAY-1.
000350        03 WEEK-PRICE COLUMN 47 PIC ZZ9.99 SUM COST.
000355        03 CUME-PRICE COLUMN 64 PIC ZZ9.99 SUM
000360              WEEK-PRICE RESET ON FINAL.
000365 01  TYPE CF MONTH NEXT GROUP NEXT PAGE.
000370     02 LINE PLUS 2.
000375        03 COLUMN 4 PIC X(15) VALUE IS
000380              "TOTAL COSTS FOR".
000385        03 COLUMN 20 PIC X(9) SOURCE MONTH.
000390        03 COLUMN 31 PIC ZZ9.99 SUM COST.
000395 01  TYPE PF.
000400     02 LINE 60.
000405        03 COLUMN 55 PIC X(4) VALUE "PAGE".
000410        03 COLUMN 60 PIC 99 SOURCE PAGE-COUNTER.
000415 01  TYPE CF FINAL.
000420     02 LINE PLUS 2.
000425           04 COLUMN 1 PIC X(39) VALUE
000430               "TOTAL EXPENDITURES FOR YEAR-TO-DATE WAS".
000435           04 COLUMN 42 PIC 999.99 SOURCE CUME-PRICE.
000440 01  TYPE RF.
000445     02 LINE NUMBER IS 4 NEXT PAGE.
000450        03 COLUMN 32 PIC X(10) VALUE IS öEND REPORTö.
000455 PROCEDURE DIVISION.
000460 P1. OPEN INPUT INFILE OUTPUT REPORT-FILE.
000465     INITIATE EXPENDITURES.
000470 P2. READ INFILE RECORD AT END GO TO P3.
000475     GENERATE COST-REPORT.
000480     GO TO P2.
000485 P3. TERMINATE EXPENDITURES.
000490     CLOSE INFILE.
000495     CLOSE REPORT-FILE.
000500     STOP RUN.
```

Output Report

The following is the report produced by the source program in Section 10.6.2. The numbers in parentheses are references used in the explanation in Section 10.6.2.

```
     (1) USER MANUFACTURING COMPANY

     (2) EXPENDITURES REPORT FOR 1989
(3)
 MONTH      W/E    DEPT    NO        ITEM    COST       CUMULATIVE
                          PURCHASES   CODE                  COST
JANUARY   EXPENDITURES  (4)
    JANUARY    05    B10    2         A       3.00 (5)
                     B12    1         A      4.00  (6)
                     B12     3        C      15.00

TOTAL COSTS FOR W/E JANUARY  05              22.00       22.00   (7)
    JANUARY    12    B11    2         B       12.00  (8)
                     B14    10        A       20.00
                     B14    10        C       60.00

TOTAL COSTS FOR W/E JANUARY  12              92.00     114.00   (9)
   TOTAL COSTS FOR JANUARY   114.00    (10)
                                          PAGE 02

   MONTH      W/E    DEPT    NO        ITEM    COST       CUMULATIVE
PURCHASES   CODE                 COST

FEBRUARY   EXPENDITURES  (4)
     FEBRUARY    09    B14    5         A      10.00  (5)

                     B14     5         B      20.00   (6)

TOTAL COSTS FOR W/E FEBRUARY 09              30.00       144.00 (11)

   TOTAL COSTS FOR FEBRUARY   30.00         (11)

  TOTAL EXPENDITURES FOR YEAR-TO-DATE WAS  144.00  (11)

                                                    PAGE 03

                                  END REPORT  (12)
```

Relation Between Output Report and Source Program

In the explanation that follows, the numbers relate to the numbers in parentheses in the output report in Section 10.6.2.

The 6-digit numbers refer to the sequence numbers included in the source listing of Section 10.6.2.

1. Is the first line of REPORT HEADING produced from source lines 000170 through 000185. The first printable line of the report group is presented on line number
2. Is the second line of the REPORT HEADING produced from source lines 000190 through 000200. The line is presented 13 lines from the last line printed, or line number 16. Note that the use of the NEXT GROUP NEXT PAGE clause has allowed the last allowable print line of the report group to assume the value of PAGE LIMIT *integer-1,* or 66. Failure to include the NEXT GROUP NEXT PAGE clause would have resulted in a diagnostic error condition.
3. Is the approximate area in which PAGE FOOTING would appear. However, the use of the NEXT GROUP NEXT PAGE clause in a REPORT HEADING results in the suppression of the PAGE FOOTING.
4. Is the CONTROL HEADING "MONTH" produced from source lines 000260 through 000275. Since this is the first body group to appear on the page and since relative line spacing is specified, the print line appears on line 14 (FIRST DETAIL)
5. Is the DETAIL report group produced from source lines 000280 through 000315. Note that it appears one line from the print line used by the last report group.
6. Is also a DETAIL report group. However, in contrast to (5), it appears two lines from the print line used by (5). This is because (5) included a relative NEXT GROUP value of one. Therefore, the NEXT GROUP value (1) plus the line value from 000285 (1) places the group two lines down on the print page.
7. Is the CONTROL FOOTING "DAY-1" produced from source lines 000320 through 000360. It appears because of a break in the control value "DAY-1". Since the break In addition, the set of contiguous characters specified by a given THROUGH phrase can specify characters of the native character set in either ascending or descending sequence is for "DAY-1", the NEXT GROUP clause on line 000320 is not ignored. Therefore, the calculation of the first print line of the next report group will be biased by the value of this NEXT GROUP PLUS 5 value.
8. Is the DETAIL report group, the printing of which has been biased by the NEXT GROUP value of (7). It is printed (5 plus 1) or six lines from the last print line used.
9. Note that the control break is recognized as occurring at the highest changed identifier value, or "MONTH" in this case. For this reason, the NEXT GROUP value is ignored.
10. Since the break is for "MONTH", the NEXT GROUP clause on source line 000365 is not ignored. This NEXT GROUP IS NEXT PAGE clause refers to the next body group. Therefore, a page advance is performed.
11. Is the result of the verb TERMINATE being executed in the object program.
12. Is the REPORT FOOTING produced from source lines 000440 through 000450. It is printed on a page by itself because of the LINE NUMBER IS 4 NEXT PAGE clause on source line 000445. This method of producing a REPORT FOOTING on a separate page is similar to using the NEXT GROUP NEXT PAGE clause for REPORT HEADING report groups in order to print it on a separate page.

    Note: *The column spacing and line spacing in the output report of Section 10.6.2 are relative. For exact placement refer to the source code of Section 10.6.2.*

Section 11
Segmentation Module

11.1. Introduction

The Segmentation module provides a means by which the user can communicate with the compiler to specify object program overlay requirements.

The Segmentation module is an obsolete element in this revision of American National Standard COBOL, and it is to be deleted from the next revision of American National Standard COBOL.

11.1.1. Level Characteristics

Segmentation level 1 provides a facility for specifying permanent and independent segments (see Section 11.1.3.). All sections with the same segment number must be contiguous in the source program. All segments specified as permanent segments must be contiguous in the source program.

Segmentation level 2 provides the facility for intermixing section with different segment numbers and allows the fixed portion of the source program to contain segments that can be overlaid. (see Section 11.1.3)

11.1.2. Scope

COBOL segmentation deals only with segmentation of procedures. As such, only the Procedure Division and the Environment Division are considered in determining segmentation requirements for an object program.

11.1.3. Organization

The Segmentation module allows you to organize your source program into program segments, a fixed portion, and independent segments.

Program Segments

Although it is not mandatory, the Procedure Division for a source program is usually written as a consecutive group of sections, each of which is composed of a series of closely related operations designed to collectively perform a particular function. However, when segmentation is used, the entire Procedure Division must be in sections. In addition, each section must be classified as belonging either to the fixed portion or to one of the independent segments of the object program. Segmentation in no way affects the need for qualification of procedure names to ensure uniqueness.

Fixed Portion

The fixed portion is defined as the part of the object program that is logically treated as if it were always in memory. This portion of the program is composed of two types of segments: fixed permanent segments and fixed overlayable segments.

A fixed permanent segment is a segment in the fixed portion that cannot be overlaid by any other part of the program. A fixed overlayable segment is a segment in the fixed portion that, although logically treated as if it were always in memory, can be overlaid by another segment to optimize memory utilization. Variation of the number of fixed permanent segments in the fixed portion can be accomplished by using a special facility called the SEGMENT-LIMIT clause (see Section 3). Su ch a segment, if called for by the program, is always made available in its last used state.

Independent Segments

An independent segment is defined as part of the object program that can overlay, and can be overlaid by, either a fixed overlayable segment or another independent segment. An independent segment is in its initial state whenever control is transferred (either implicitly or explicitly) to that segment for the first time during the execution of a program. On subsequent transfers of control to the segment, an independent segment is also in its initial state when:

1. Control is transferred to that segment as a result of the implicit transfer of control between consecutive statements from a segment with a different segment number.
2. Control is transferred to that segment as a result of the implicit transfer of control between a SORT or a MERGE statement, in a segment with a different segment number, and an associated input or output procedure in that independent segment.
3. Control is transferred explicitly to that segment from a segment with a different segment number (with the exception noted in item 2, following).

On a subsequent transfer of control to the segment, an independent segment is in its last-used state when:

1. Control is transferred implicitly to that segment from a segment with a different segment number (except as noted in the preceding items 1 and 2).
2. Control is transferred explicitly to that segment as the result of the execution of an EXIT PROGRAM statement.

11.1.4. Segmentation Classification

Sections that are to be segmented are classified, using a system of segment numbers (see Section 11.3.2) and the following criteria:

1. Logic requirements Sections that must be available for reference at all times or are referred to frequently are normally classified as belonging to one of the permanent segments; segments that are used less frequently are normally classified as belonging either to one of the overlayable segments or to one of the independent segments, depending on logic requirements.
2. Frequency of use generally, the more frequently a section is referred to, the lower its segment number; the less frequently it is referred to, the higher its segment number.
3. Relationship to other sections that frequently communicate with one another must be given the same segment numbers.

11.1.5. Segmentation Control

The logical sequence of the program is the same as the physical sequence except for specific transfers of control. If any reordering of the object program is required to handle the flow from segment to segment, according to rules in Section 11.3.2, the implementor must provide control transfers to maintain the logical flow specified in the source program. The implementor must also provide all controls necessary for a segment to operate whenever the segment is used. Control can be transferred within a source program to any paragraph in a section; that is, it is not mandatory to transfer control to the beginning of a section.

11.2. Environment Division in the Segmentation Module

This subsection describes the Environment Division in the Segmentation module.

11.2.1. Configuration Section

See Section 3.4.2 for information concerning the Configuration Section.

11.2.2. The OBJECT-COMPUTER Paragraph

Function

The OBJECT-COMPUTER paragraph provides a means of describing the computer on which the program is to be executed.

General Format

```
OBJECT-COMPUTER .  [ computer-name ]
[ MEMORY SIZE integer-1    { WORDS      } ]
                           { CHARACTERS }
                           { MODULES    }
[ PROGRAM COLLATING SEQUENCE IS alphabet-name-1 ]
[ SEGMENT-LIMIT IS segment-number ]
```

Syntax Rules

```
[Computer-name is a system name.]
```

General Rules

1. All clauses of the OBJECT-COMPUTER paragraph apply to the program in which they are explicitly or implicitly specified and to any program contained within that program.
2. General rules concerning *computer-name,* the MEMORY SIZE clauses, and the PROGRAM COLLATING SEQUENCE clause are presented in Section 3.4.4.
3. The SEGMENT-LIMIT clause is presented in Section 3.

11.2.3. The SEGMENT-LIMIT Clause

Function

Ideally, all program segments having segment numbers ranging from 0 through 49 must be specified as permanent segments. However, when insufficient memory is available to contain all permanent segments plus the largest overlayable segment, it becomes necessary to decrease the number of permanent segments. The SEGMENT-LIMIT feature provides the means by which a user can reduce the number of permanent segments in a program, while still retaining the logical properties of fixed portion segments (segment numbers 0 through 49).

General Format

```
SEGMENT-LIMIT IS segment-number
```

Syntax Rules

Segment number must be an integer ranging in value from 1 through 49.

General Rules

1. When the SEGMENT-LIMIT clause is specified, only those segments having segment numbers from 0 up to, but not including, the segment number designated as the segment limit are considered as permanent segments of the object program.

2. Those segments having segment numbers from the segment limit through 49 are considered as overlayable segments.
3. When the SEGMENT-LIMIT clause is omitted, all segments having segment numbers 0 through 49 are considered as permanent segments of the object program.

11.3. Procedure Division in the Segmentation Module

This subsection describes the Procedure Division in the Segmentation module.

11.3.1. General Description

The Procedure Division contains sections with segment numbers when the Segmentation module is used in a COBOL source program. Following is the general format of the Procedure Division when sections and segment numbers are present.

```
PROCEDURE DIVISION
[DECLARATIVES.
{section-name SECTION [segment-number].
     USE statement.
[paragraph-name.
     [sentence] ... ] ... } ...
END DECLARATIVES.]
{section-name SECTION [segment-number].
[paragraph-name.
     [sentence] ... ] ... } ...
```

11.3.2. Segment Numbers

Function

Section classification is accomplished by means of a system of segment numbers. The segment number is included in the section header within the Procedure Division.

General Format

```
section-name SECTION [segment-number].
```

Syntax Rules

1. The *segment-number* must be an integer ranging in value from 0 through 99.
2. If the *segment-number* is omitted from the section header, the *segment-number* is assumed to be 0.
3. Sections in the declaratives must contain *segment-numbers* less than 50.

General Rules

1. All sections that have the same segment number constitute a program segment. In level 1, all sections that have the same segment number must be together in the source program. In level 2, sections with the same segment numbers need not be physically contiguous in the source program.
2. Segments with segment number 0 through 49 belong to the fixed portion of the object program. In level 1, all sections with segment number 0 through 49 must be together in the source program.
3. Segments with segment numbers 50 through 99 are independent segments.

11.3.3. Restrictions on Program Flow

When segmentation is used, the following restrictions are placed on the ALTER, PERFORM, MERGE, and SORT statements.

The ALTER Statement

A GO TO statement in a section whose segment number is greater than or equal to 50 must not be referred to by an ALTER statement in a section with a different segment number.

All other uses of the ALTER statement are valid and are performed even if the GO TO to which the ALTER refers is in a fixed overlayable segment.

The PERFORM Statement

A PERFORM statement appearing in a section that is not in an independent segment can have within its range, in addition to any declarative sections whose execution is caused within that range, only one of the following:

- Sections and paragraphs wholly contained in one or more nonindependent segments.
- Sections and paragraphs wholly contained in a single independent segment.

A PERFORM statement appearing in an independent segment can have within its range, in addition to any declarative sections whose execution is caused with that range, only one of the following:

- Sections and paragraphs wholly contained in one or more nonindependent segments.
- Sections and paragraphs wholly contained in the same independent segment as that PERFORM statement.

The MERGE Statement

If the MERGE statement appears in a section that is not in an independent segment, then any output procedure referenced by that MERGE statement must appear:

- Totally within nonindependent segments
- Wholly contained in a single independent segment

If a MERGE statement appears in an independent segment, then any output procedure referenced by that MERGE statement must be contained:

- Totally within nonindependent segments
- Wholly within the same independent segment as that MERGE statement

The SORT Statement

If a SORT statement appears in a section that is not an independent segment, then any input procedures referenced by that SORT statement must appear:

- Totally within nonindependent segments
- Wholly contained in a single independent segment

If a SORT statement appears in an independent segment, then any input procedures or output procedures referenced by that SORT statement must be contained:

- Totally within nonindependent segments
- Wholly within the same independent segment as that SORT statement

Section 12
Intrinsic Functions

Intrinsic functions are procedures that are part of the UCS COBOL compiler. The compiler provides the functions and you provide the values for the functions to process. You call an intrinsic function from the Procedure Division of a COBOL source program. Each intrinsic function has constraints for the values upon which it can operate. When the statement containing the intrinsic functions executes, the system processes the function using the value that you provide and returns the resulting data value. The following program illustrates use of some of the intrinsic functions.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. FUNCTIONS.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-2200.
OBJECT-COMPUTER. UNISYS-2200.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 AA  PIC 99  VALUE 81.
01 BB  PIC 99  VALUE 36.
01 CC  PIC 999 VALUE 120.
01 DD  PIC 999 VALUE 300.
01 EE  PIC X    VALUE 'Z'.
01 FF  PIC X(5) VALUE 'ABCDE' .
01 GG  PIC X(5) VALUE 'fghjk' .
01 HH.
   05 JJ    OCCURS 10.
      10 KK        PIC 99      OCCURS 10.
01 FACT  PIC 99  VALUE 12.
01 TEMP  PIC 999.
01 RESULT PIC 9(9).
01 CONV-FACTOR  PIC 9V9(9).
PROCEDURE DIVISION.
BEGIN.
    MOVE FUNCTION ORD(EE) TO TEMP.
    DISPLAY 'ORD(Z) MUST BE 91IS = ' TEMP.
    ADD FUNCTION SQRT(AA) TO FUNCTION SUM(BB, CC) GIVING RESULT.
    DISPLAY 'SQRT(81)+SUM(36+120)   IS = ' RESULT.
    MOVE FUNCTION FACTORIAL(FACT) TO RESULT.
    DISPLAY 'FACTORIAL OF 12 MUST BE 479001600IS = ' RESULT.
    DISPLAY
      'CHAR(120) MUST BE w  IS = ' FUNCTION CHAR(CC).
    IF FUNCTION MAX(aa, dd, cc, bb) EQUAL 300
```

```
    DISPLAY 'CORRECT MAX MUST BE 300IS = ' dd
  ELSE
    DISPLAY 'INCORRECT MAX.'.
  IF FUNCTION MIN(dd, cc, bb, aa) EQUAL 36
    DISPLAY 'CORRECT MIN MUST BE 36 IS = ' bb
  ELSE
    DISPLAY 'INCORRECT MIN.'.
  INITIALIZE HH REPLACING NUMERIC BY ZEROS.
  MOVE 5 TO KK(5, 5). MOVE FUNCTION ORD-MAX(kk) TO TEMP.
  DISPLAY 'ORD-MAX MUST BE 45 .             IS = ' TEMP.
  INITIALIZE HH REPLACING NUMERIC BY 9.
  MOVE 2 TO KK(6, 7). MOVE FUNCTION ORD-MIN(KK) TO TEMP.
  DISPLAY 'ORD-MIN MUST BE 57 IS = ' TEMP.
  DISPLAY 'LOWER-CASE OF ABCDE IS = '
    FUNCTION LOWER-CASE(FF).
  DISPLAY 'UPPER-CASE OF fghjk IS = '
    FUNCTION UPPER-CASE(GG).
  COMPUTE CONV-FACTOR ROUNDED = 3.14159265358979323 / 180.
  COMPUTE FACT ROUNDED = FUNCTION COS(0 * CONV-FACTOR).
  IF FACT = 1
      DISPLAY 'COS OF 0 MUST BE 1  IS = ' FACT
    ELSE DISPLAY 'INCORRECT COS.' .
  COMPUTE FACT ROUNDED = FUNCTION COS(90 * CONV-FACTOR).
  IF FACT = 0
      DISPLAY 'COS OF 90 MUST BE 0  IS = ' FACT
    ELSE DISPLAY 'INCORRECT COS.' .
  STOP RUN.
```

The previous program produces the following results:

```
ORD(Z) MUST BE 91                                IS = 091
SQRT(81)+SUM(36+120)                             IS = 000000165
FACTORIAL OF 12 MUST BE 479001600                IS = 479001600
CHAR(120) MUST BE w                              IS = w
CORRECT MAX MUST BE 300                          IS = 300
CORRECT MIN MUST BE 36                           IS = 36
ORD-MAX MUST BE 45                               IS = 045
ORD-MIN MUST BE 57                               IS = 057
LOWER-CASE OF ABCDE                              IS = abcde
UPPER-CASE OF fghjk                              IS = FGHJK
COS OF 0 MUST BE 1                               IS = 01
COS OF 90 MUST BE 0                              IS = 00
```

See Section 12.5.1 through Section 12.5.42 for descriptions of all of the intrinsic functions that are available with UCS COBOL.

12.1. Language Concepts

The following subsections describe the language considerations that apply to using intrinsic functions.

12.1.1. Function Name

In the Intrinsic Function module, a function is a data item whose value is determined by invoking a mechanism provided by the compiler at the time the function is referenced by an executing statement. A function is uniquely identified by a function name.

A function name is a UCS COBOL word that names a mechanism provided by the compiler to determine the value of a function, and that is one of a specified list of UCS COBOL words that can be used in UCS COBOL source programs. The same UCS COBOL word, in a different context, can appear in a program as a user-defined word or a system name.

12.1.2. Value Returned by a Function

The value returned by a function is considered a data value. A mechanism is provided at object time to assign a data value to a function when it is referenced. In order to determine the function's value, the evaluation mechanism can require access to data values provided by the referencing program. Specifying parameters, known as arguments, provides these data values when referencing the function. Specific functions can place constraints on these arguments (for example, range). If, at the time a function is referenced, the arguments specified for that reference do not have values that comply with the specified constraints; the returned value for the function is undefined.

12.1.3. Function Identifier

A function identifier is used to reference a function within the Procedure Division of a UCS COBOL source program.

12.2. Function Definition and Returned Value

The definition of a function identifies:

- The size of the returned value for alphanumeric functions
- The size, point location, and sign of the returned value, including whether or not the function is an integer (for numeric and integer functions)
- The value returned for some other cases

12.3. Arguments

Arguments specify values used for evaluation of a function. These arguments can be specified as

- Identifiers
- Arithmetic expressions
- Literals

The definition of a function specifies the number of arguments required, which can be zero, one, or more. For some functions, the number of arguments that can be specified varies. The order in which arguments are specified in a function identifier determines the interpretation given to each value in arriving at the function value.

Arguments can be required to have a certain class or a subset of a certain class. The types of argument are

Numeric

An arithmetic expression must be specified. The value of the arithmetic expression, including operational sign, is used in determining the value of the function.

Alphabetic

An elementary data item of the class alphabetic must be specified. The size associated with the argument can be used in determining the value of the function.

Alphanumeric

A data item of the class alphabetic or alphanumeric must be specified. The size associated with the argument can be used in determining the value of the function.

Integer

An arithmetic expression that always results in an integer value must be specified. The value of the arithmetic expression, including operational sign, is used in determining the value of the function.

Arguments can have only certain values in order to permit meaningful determination of the function's value. If, at the time a function is referenced, the arguments specified for that reference do not have values within the permissible range; the returned value for the function is undefined.

When the definition of a function permits an argument to be repeated a variable number of times, a table can be referenced with the following form of subscripting:

- The data name and any qualifiers that identify the table are followed immediately by subscripting. The word ALL can be used as one or more of the subscripts.

- When ALL is specified as a subscript, the effect is as if each table element associated with that subscript position were specified. The order of implicit specification of each occurrence is from left to right, with
  - The first (or leftmost) specification being the identifier with each subscript specified by the word all replaced by one
  - The next specification being the same identifier with the rightmost subscript specified by the word ALL incremented by one.

  This process continues with the rightmost ALL subscript being incremented by one for each implicit specification until the rightmost ALL subscript has been incremented through its range of values. If there are any additional ALL subscripts, the following occurs:
  - The ALL subscript immediately to the left of the rightmost ALL subscript is incremented by one, the rightmost ALL subscript is reset to one, and the process of varying the rightmost ALL subscript is repeated.
  - The ALL subscript to the left of the rightmost ALL subscript is incremented by one through its range of values. For each additional ALL subscript, this process is repeated in turn until the leftmost ALL subscript has been incremented by one through its range of values.

  If the ALL subscript is associated with an OCCURS DEPENDING ON clause, the range of values is determined by the object of the OCCURS DEPENDING ON clause. The evaluation of an ALL subscript must result in at least one argument; otherwise, the returned value is undefined.

12.4. Types of Functions

Data item functions are elementary data items and return alphanumeric, numeric, or integer values. Data item functions are treated as elementary data items and cannot receive operands. Types of data item functions are as follows:

- Alphanumeric functions

  These are of the class and category alphanumeric. The number of character positions in this data item is specified in the function definition. Alphanumeric functions have an implicit usage of DISPLAY.
- Numeric functions

  These are of the class and category numeric. A numeric function is always considered to have an operational sign.

  A numeric function can be used only in an arithmetic expression.

  A function that is not defined as an integer cannot be referenced where an integer operand is required, even though a particular reference can yield an integer value.

- Integer function

  These are of the class and category numeric. An integer function is always considered to have an operational sign.

  An integer function can be used only in an arithmetic expression.

  An integer function can be referenced where an integer operand is required and where a signed operand is allowed.

12.5. UCS COBOL Intrinsic Functions

Table 12–1 summarizes the intrinsic functions that are available for UCS COBOL.

Table 12–1. Table of Functions

Function-Name	Arguments	Type	Value Returned	Section
ACOS	N1	N	Arc cosine of N1	12.5.1
ANNUITY	N1, I2	N	Ratio of annuity paid for I2 periods at interest of N1 to initial investment of one	12.5.2
ASIN	N1	N	Arc sine of N1	12.5.3
ATAN	N1	N	Arc tangent of N1	12.5.4
CHAR	I1	X	Character in position I1 of program collating sequence	12.5.5
COS	N1	N	Cosine of N1	12.5.6
CURRENT-DATE	None	X	Current date and time	12.5.7
DATE-OF-INTEGER	I1	I	Standard date equivalent (YYYYMMDD) of integer date	12.5.8
DAY-OF-INTEGER	I1	I	Julian date equivalent (YYYYDDD) of integer date	12.5.9
FACTORIAL	I1	I	Factorial of I1	12.5.10
INTEGER	N1	I	The greatest integer not greater than N1	12.5.11

Table 12–1. Table of Functions

Function-Name	Arguments	Type	Value Returned	Section
INTEGER-OF-DATE	I1	I	Integer date equivalent of standard date (YYYYMMDD)	12.5.12
INTEGER-OF-DAY	I1	I	Integer date equivalent of Julian date (YYYYDDD)	12.5.13
INTEGER-PART	N1	I	Integer part of N1	12.5.14
LENGTH	A1 or N1 or X1	I	Length of argument	12.5.15
LOG	N1	N	Natural logarithm of N1	12.5.16
LOG10	N1	N	Logarithm to base 10 of N1	12.5.17
LOWER-CASE	X1 or A1	X	All letters in X1 or A1 set lowercase	12.5.18
MAX	I1... or N1...	Depends upon arguments*	Value of maximum argument	12.5.19
MEAN	N1...	N	Arithmetic mean of arguments	12.5.20
MEDIAN	N1...	N	Median of arguments	12.5.21
MIDRANGE	N1...	N	Mean of minimum and maximum arguments	12.5.22
MIN	I1... or N1...	Depends upon arguments*	Value of minimum argument	12.5.23
MOD	I1, I2	I	I1 modulo I2	12.5.24
NUMVAL	X1	N	Numeric value of simple numeric string	12.5.25
NUMVAL-C	X1, X2	N	Numeric value of numeric string with optional commas currency sign	12.5.26

Table 12–1. Table of Functions

Function-Name	Arguments	Type	Value Returned	Section
ORD	X1 or A1	I	Ordinal of X1 or A1	12.5.27
ORD-MAX	A1... or X1... or N1...	I	Ordinal of maximum argument	12.5.28
ORD-MIN	A1... or X1... or N1...	I	Ordinal of minimum argument	12.5.29
PRESENT-VALUE	N1... N2...	N	Present value of a series of future period-end amounts (N2) at a discount rate of N1	12.5.30
RANDOM	I1	N	Random number	12.5.31
RANGE	I1... or N1...	Depends upon argument	Value of maximum argument minus value of minimum argument	12.5.32
REM	N1, N2	N	Remainder of N1/N2	12.5.33
REVERSE	A1 or X1	X	Reverse order of the characters of the argument	12.5.34
SIN	N1	N	Sine of N1	12.5.35
SQRT	N1	N	Square root of N1	12.5.36
STANDARD-DEVIATION	N1...	N	Standard deviation of arguments	12.5.37
SUM	I1... or N1...	Depends upon arguments	Sum of arguments	12.5.38
TAN	N1	N	Tangent of N1	12.5.39
UPPER-CASE	X1 or A1	X	All letters in X1 or A1 set uppercase	12.5.40
VARIANCE	N1...	N	Variance of argument	12.5.41

Table 12–1. Table of Functions

Function-Name	Arguments	Type	Value Returned	Section
WHEN-COMPILED	None	X	Date and time program was compiled	12.5.42

Legend

* A function that has only alphabetic arguments is type alphanumeric.

A alphabetic

I integer

N numeric

X alphanumeric

12.5.1. The ACOS Function

Function

The ACOS function returns a numeric value in radians that approximates the arc cosine of *argument-1.* The type of this function is numeric.

Format

```
FUNCTION ACOS (argument -1)
```

Arguments

1. *Argument-1* must be class numeric.
2. The value of *argument-1* must be greater than or equal to -1 and less than or equal to +1.

Returned Value

The returned value is the approximation of the arc cosine of *argument-1,* and is greater than or equal to zero and less than or equal to π (pi).

12.5.2. The ANNUITY Function

Function

The ANNUITY function (annuity immediate) returns a numeric value that approximates the ratio of an annuity paid at the end of each period for the number of periods specified by *argument-2* to an initial investment of one. Interest is earned at the rate specified by *argument-1* and is applied at the end of the period, before the payment. The type of this function is numeric.

Format

```
FUNCTION ANNUITY (argument-1 argument-2)
```

Arguments

1. *Argument-1* must be class numeric.
2. The value of *argument-1* must be greater than or equal to zero.
3. *Argument-2* must be a positive integer.

Returned Values

1. When the value of *argument-1* is zero, the value of the function is the approximation of

   ```
   1 / argument-2
   ```

2. When the value of *argument-1* is not zero, the value of the function is the approximation of

   ```
   argument-1 / (1 - (1 + argument-1) ** (- argument-2))
   ```

Example

```
01   RESULT1   PIC   9(5)V9(5).
01   RESULT2   PIC   9(5)V9(5).

COMPUTE RESULT1 = FUNCTION ANNUITY (0, 4).
COMPUTE RESULT2 = FUNCTION ANNUITY (.029, 4).

RESULT1 = 0.25000
RESULT2 = 2.26838
```

12.5.3. The ASIN Function

Function

The ASIN function returns a numeric value in radians that approximates the arc sine of *argument-1.* The type of this function is numeric.

Format

```
FUNCTION ASIN (argument-1)
```

Arguments

1. *Argument-1* must be class numeric.
2. The value of *argument-1* must be greater than or equal to -1, and less than or equal to +1.

Returned Values

The returned value is the approximation of the arc sine of *argument-1* and is greater than or equal to $-\pi/2$ and less than or equal to $+\pi/2$.

12.5.4. The ATAN Function

Function

The ATAN function returns a numeric value in radians that approximates the arc tangent of *argument-1.* The type of this function is numeric.

Format

```
FUNCTION ATAN (argument-1)
```

Arguments

Argument-1 must be class numeric.

Returned Values

The returned value is the approximation of the arc tangent of *argument-1*, and is greater than $-\pi/2$ and less than $+\pi/2$.

12.5.5. The CHAR Function

Function

The CHAR function returns a one-character alphanumeric value that is a character in the program collating sequence that has the ordinal position equal to the value of *argument-1.* The type of this function is alphanumeric.

Format

```
FUNCTION CHAR(argument-1)
```

Arguments

1. *Argument-1* must be an integer.
2. The value of *argument-1* must be greater than zero and less than or equal to the number of positions in the collating sequence.

Returned Values

1. If more than one character has the same position in the collating sequence, the character returned as the function value is that of the first literal specified for that character position in the ALPHABET clause.
2. If the current program collating sequence was not specified by an ALPHABET clause, the compiler determines the value.

12.5.6. The COS Function

Function

The COS function returns a numeric value that approximates the cosine of an angle or arc, expressed in radians, that is specified by *argument-1.* The type of this function is numeric.

Format

```
FUNCTION COS(argument-1)
```

Arguments

Argument-1 must be class numeric.

Returned Values

The returned value is the approximation of the cosine of *argument-1*, and is greater than or equal to -1 and less than or equal to +1.

12.5.7. The CURRENT-DATE Function

Function

The CURRENT-DATE function returns a 21-character alphanumeric value that represents the calendar date and time of day. This type of function is alphanumeric.

Format

```
FUNCTION CURRENT-DATE
```

Arguments

None

Returned Values

The character positions returned, numbered left to right, are

Positions	Contents
1-4	Four numeric digits of the year in the Gregorian calendar
5-6	Two numeric digits of the month of the year, in the range 01 through 12
7-8	Two numeric digits of the day of the month, in the range 01 through 31
9-10	Two numeric digits of the hour past midnight, in the range 00 through 23
11-12	Two numeric digits of the minutes past the hour, in the range 00 through 59
13-14	Two numeric digits of the seconds past the minute, in the range 00 through 59
15-16	Two numeric digits of the hundredths of a second past the second, in the range 00 through 99

Positions	Contents
17	Either the character '-', '+', or '0' is returned. '-' is returned if the local time indicated in the previous character positions is behind Coordinated Universal Time. '+' is returned if the local time indicated is the same as or ahead of Coordinated Universal Time. '0' is returned if the system on which this function is evaluated does not have the facility to provide the local time differential factor. This feature is only available on ClearPath Plus systems that have Exec that support UTC.
18-19	If character position 17 is '-', two numeric digits are returned in the range 00 through 12 indicating the number of hours that the reported time is behind Coordinated Universal Time. If character position 17 is '+', two numeric digits are returned in the range 00 through 13 indicating the number of hours that the reported time is ahead of Coordinated Universal Time. If character position 17 is '0', the value 00 is returned.
20-21	Two numeric digits are returned in the range 00 through 59 indicating the number of additional minutes that the reported time is ahead of or behind Coordinated Universal Time, depending on whether character position 17 is '+' or '-' respectively. If character position 17 is '0', the value 00 is returned.

Example

```
DISPLAY FUNCTION CURRENT-DATE.
```

If the program is running on an OS 2200 system that does not support Coordinated Universal Time (UTC), a value similar to the following is displayed:

```
2004030812232224000000
```

If the program is running on an OS 2200 system that does support Coordinated Universal Time (UTC), a value similar to the following is displayed, assuming that the program has run on a system on United States Central Standard Time (CST):

```
2004030812232224-0600
```

Note: *If you use the UTC time functions in HVTIP, you must use the INITIAL clause on the PROGRAM-ID.*

12.5.8. The DATE-OF-INTEGER Function

Function

The DATE-OF-INTEGER function converts a date in the Gregorian calendar from integer date form to standard date form (YYYYMMDD). The type of this function is integer.

Format

```
FUNCTION DATE-OF-INTEGER (argument-1)
```

Arguments

Argument-1 is a positive integer that represents a number of days succeeding December 31, 1600, in the Gregorian calendar.

Returned Values

1. The returned value represents the ISO Standard date equivalent of the integer specified in *argument-1*.
2. The returned value is in the form (YYYYMMDD) where YYYY represents a year in the Gregorian calendar, MM represents the month of that year, and DD represents the day of that month.

Example

```
01   RESULT1   PIC 9(8)
01   RESULT2   PIC 9(8)

COMPUTE RESULT1 = FUNCTION DATE-OF-INTEGER (1).
COMPUTE RESULT2 = FUNCTION DATE-OF-INTEGER (365).

RESULT1 = 16010101
RESULT2 = 16011231
```

12.5.9. The DAY-OF-INTEGER Function

Function

The DAY-OF-INTEGER function converts a date in the Gregorian calendar from integer date form to Julian date form (YYYYDDD). The type of this function is integer.

Format

```
FUNCTION DAY-OF-INTEGER(argument-1)
```

Arguments

Argument-1 is a positive integer that represents a number of days succeeding December 31, 1600, in the Gregorian calendar.

Returned Values

1. The returned value represents the Julian equivalent of the integer specified in argument-*1*.
2. The returned value is an integer form (YYYYDDD) where YYYY represents a year in the Gregorian calendar and DDD represents the day of that year.

Example

```
01   RESULT1   PIC 9(7)
01   RESULT2   PIC 9(7)
COMPUTE RESULT1 = FUNCTION DAY-OF-INTEGER (1).
COMPUTE RESULT2 = FUNCTION DAY-OF-INTEGER (400).

RESULT1 = 1601001
RESULT2 = 1602035
```

12.5.10. The FACTORIAL Function

Function

The FACTORIAL function returns an integer that is the factorial of *argument-1.* The type of this function is integer.

Format

```
FUNCTION FACTORIAL(argument-1)
```

Arguments

Argument-1 must be an integer greater than or equal to zero.

Returned Values

1. If the value of *argument-1* is zero, the value 1 is returned.
2. If the value of *argument-1* is positive, the factorial is returned.

12.5.11. The INTEGER Function

Function

The INTEGER function returns the greatest integer value that is less than or equal to the argument. The type of this function is integer.

Format

```
FUNCTION INTEGER (argument-1)
```

Arguments

Argument-1 must be class numeric.

Returned Values

The returned value is the greatest integer less than or equal to the value of *argument-1*. For example, if the value of *argument-1* is -1.5, -2 is returned. If the value of *argument-1* is +1.5, +1 is returned.

Example

```
DISPLAY FUNCTION INTEGER (4.578).
```

The value 4 is displayed.

```
DISPLAY FUNCTION INTEGER (-9.763).
```

The value -10 is displayed.

12.5.12. The INTEGER-OF-DATE Function

Function

The INTEGER-OF-DATE function converts a date in the Gregorian calendar form standard date form (YYYYMMDD) to integer date form. This type of function is integer.

Format

```
FUNCTION INTEGER-OF-DATE( argument-1)
```

Arguments

Argument-1 must be an integer of the form YYYYMMDD, whose value is obtained from the calculation (YYYY * 10,000) + (MM * 100) + DD.

1. YYYY represents the year in the Gregorian calendar. It must be an integer greater than 1600.
2. MM represents a month and must be a positive integer less than 13.
3. DD represents a day and must be a positive integer less than 32, provided that it is valid for the specified month and year combination.

Returned Values

The returned value is an integer that is the number of days the date represented by *argument-1* succeeds December 31, 1600, in the Gregorian calendar.

Example

```
01    RESULT1   PIC  9(7).

01    RESULT2   PIC  9(7).

COMPUTE RESULT1 = FUNCTION INTEGER-OF-DATE (16010101).

COMPUTE RESULT2 = FUNCTION INTEGER-OF-DATE (16020204).



RESULT1 = 1

RESULT2 = 400
```

12.5.13. The INTEGER-OF-DAY Function

Function

The INTEGER-OF-DATE function converts a date in the Gregorian calendar from Julian date form (YYYYDDD) to integer date form. This type of function is integer.

Format

```
FUCNTION INTEGER-OF-DAY (argument-1)
```

Arguments

Argument-1 must be an integer of the form YYYYDDD, whose value is obtained from the calculation (YYYY * 1,000) + DDD.

1. YYYY represents the year in the Gregorian calendar. It must be an integer greater than 1600.
2. DDD represents the day of the year. It must be a positive integer less than 367, provided that it is valid for the year specified.

Returned Values

The returned value is an integer that is the number of days the date represented by *argument-1* succeeds December 31, 1600, in the Gregorian calendar.

Example

```
01   RESULT1  PIC  9(7).

01   RESULT2  PIC  9(7).

COMPUTE RESULT1 = FUNCTION INTEGER-OF-DAY (1601001).

COMPUTE RESULT2 = FUNCTION INTEGER-OF-DAY (1602035).

RESULT1 = 1

RESULT2 = 400
```

12.5.14. The INTEGER-PART Function

Function

The INTEGER-PART function returns an integer that is the integer portion of *argument-1.* The type of this function is integer.

Format

```
FUNCTION INTEGER-PART (argument-1)
```

Arguments

Argument-1 must be class numeric.

Returned Values

1. If the value of *argument-1* is zero, the returned value is zero.
2. If the value of *argument-1* is positive, the returned value is the greatest integer less than or equal to the value of *argument-1.* For example, if the value of *argument-1* is +1.5, +1 is returned.

3. If the value of *argument-1* is negative, the returned value is the least integer greater than or equal to the value of *argument-1.* For example, if the value of *argument-1* is -1.5, -1 is returned.

12.5.15. The LENGTH Function

Function

The LENGTH function returns an integer equal to the length of the argument in character positions. The type of function is integer.

Format

```
FUNCTION LENGTH (argument-1)
```

Arguments

1. *Argument-1* can be a nonnumeric literal or a data item of any class or category.
2. If *argument-1* or any data item subordinate to *argument-1* is described with the DEPENDING phrase of the OCCURS clause, the contents of the data item referenced by the data-name specified in the DEPENDING phrase are used at the time the LENGTH function is evaluated.

Returned Values

1. If *argument-1* is a nonnumeric literal or an elementary data item, or *argument-1* is a group data item that does not contain a variable occurrence data item, the value returned is an integer equal to the length of *argument-1* in character positions.
2. If *argument-1* is a group data item containing a variable occurrence data item, the returned value is an integer determined by evaluation of the data item specified in the DEPENDING phrase of the OCCURS clause for that variable occurrence data item.

   This evaluation is accomplished according to the rules in the OCCURS clause dealing with the data item as a sending data item. (See Section 3.5.10 and Section 3.5.16.)
3. The returned value includes implicit FILLER characters, if any.

Example

```
01   K   PIC X(3)   VALUE   ALL "A"

DISPLAY FUNCTION LENGTH ("A")
```

This results in a value of 1 being displayed.

```
DISPLAY FUNCTION LENGTH (K)
```

This results in a value of 3 being displayed.

12.5.16.The LOG Function

Function

The LOG function returns a numeric value that approximates the logarithm to the base *e* (natural log) of *argument-1.* The type of this function is numeric.

Format

```
FUNCTION LOG (argument-1)
```

Arguments

1. *Argument-1* must be class numeric.
2. The value of *argument-1* must be greater than zero.

Returned Values

The returned value is the approximation of the logarithm to the base *e* of *argument-1.*

12.5.17.The LOG10 Function

Function

The LOG10 function returns a numeric value that approximates the logarithm to the base 10 of *argument-1.* The type of this function is numeric.

Format

```
FUNCTION LOG10 (argument-1)
```

Arguments

1. *Argument-1* must be class numeric
2. The value of *argument-1* must be a value greater than zero.

Returned Values

The returned value is the approximation of the logarithm to the base 10 of *argument-1.*

12.5.18. The LOWER-CASE Function

Function

The LOWER-CASE function returns a character string that is the same length as *argument-1* with all uppercase characters replaced by the equivalent lowercase characters. The type of this function is alphanumeric.

Format

```
FUNCTION LOWER-CASE (argument-1)
```

Arguments

Argument-1 must be an alphabetic or alphanumeric data item.

Returned Values

1. The same character string as *argument-1* is returned, except that each uppercase alphabetic character is replaced by the equivalent lowercase character.
2. If the computer character set does not include lowercase characters, no changes take place in the character string.

12.5.19. The MAX Function

Functions

The MAX function returns the content of the *argument-1* that contains the maximum value. The type of this function depends upon the argument types as follows.

Argument Types	Function Types
All arguments integer	Integer
Numeric (some arguments can be integer)	Numeric

Format

```
FUNCTION MAX({argument-1}...)
```

Arguments

If more than one *argument-1* is specified, all arguments must be of the same class.

Returned Values

1. The value returned is the content of the *argument-1* having the greatest value. The comparisons used to determine the greatest value are made according to the rules for simple conditions.
2. If more than one *argument-1* has the same greatest value, the content of the *argument-1* returned is the leftmost *argument-1* having that value.
3. If the type of the function is numeric, the characteristics of the returned value are defined by the compiler.

12.5.20. The MEAN Function

Function

The MEAN function returns a numeric value that is the arithmetic mean (average) of its arguments. The type of function is numeric.

Format

```
FUNCTION MEAN({argument-1}. . .)
```

Arguments

Argument-1 must be class numeric.

Returned Values

1. The returned value is the arithmetic mean of the *argument-1* series.
2. The returned value is defined as the sum of the *argument-1* series divided by the number of occurrences referenced by *argument-1.*

Example

```
01   RESULT1   PIC   S9(3)V99.
COMPUTE RESULT1 = FUNCTION MEAN (5, -2, -14, 0).
RESULT1 = -2.75
```

12.5.21.The MEDIAN Function

Function

The MEDIAN function returns the content of the argument whose value is the middle value in the list formed by arranging the arguments in sorted order. The type of function is numeric.

Format

```
FUNCTION MEDIAN ({argument-1}. . .)
```

Arguments

Argument-1 must be class numeric.

Returned Values

1. The returned value is the content of the *argument-1* having the middle value in the list formed by arranging all the *argument-1* values in sorted order.
2. If the number of occurrences referenced by *argument-1* is odd, the returned value is such that at least half (of the occurrences referenced by *argument-1*) are greater than or equal to the returned value, and at least half are less than or equal to the returned value. If the number of occurrences referenced by *argument-1* is even, the returned value is the arithmetic mean of the values referenced by the two middle occurrences.
3. The comparisons used to arrange the *argument-1* values in sorted order are made according to the rules for simple conditions. (See Section 3.6.3)

Example

```
01   RESULT1   PIC   S9(3)V99.
COMPUTE RESULT1 = FUNCTION MEDIAN (5, -2, -14, 0).
RESULT1 = -1.00
```

12.5.22. The MIDRANGE Function

Function

The MIDRANGE (middle range) function returns a numeric value that is the arithmetic mean (average) of the values of the minimum argument and the maximum argument. The type of this function is numeric.

Format

```
FUNCTION MIDRANGE ({argument-1}. . .)
```

Arguments

Argument-1 must be class numeric.

Returned Values

The returned value is the arithmetic mean of the greatest *argument-1* value and the least *argument-1* value. The comparisons used to determine the greatest and least values are made according to the rules for simple conditions. (See Section 3.6.3.)

Example

```
01   RESULT1   PIC   S9(3)V99.
COMPUTE RESULT1 = FUNCTION MIDRANGE (5, -2, -14, 0).
RESULT1 = -4.50
```

12.5.23. The MIN Function

Function

The MIN function returns the content of the *argument-1* that contains the minimum value. The type of this function depends upon the argument types as follows.

Argument Types	Function Types
All arguments integer	Integer
Numeric (some arguments can be integer)	Numeric

Format

```
FUNCTION MIN( {argument-1}. . .)
```

Arguments

If more than one *argument-1* is specified, all arguments must be of the same class.

Returned Values

1. The value returned is the content of the *argument-1* having the least value. The comparisons used to determine the least value are made according to the rules for simple conditions.
2. If more than one *argument-1* has the same least value, the content of the *argument-1* returned is the leftmost *argument-1* having that value.
3. If the type of the function is numeric, the characteristics of the returned value are defined by the compiler.

12.5.24. The MOD Function

Function

The MOD function returns an integer value that is *argument-1* modulo *argument-2.* The type of this function is integer.

Format

```
FUNCTION MOD (argument-1 argument-2)
```

Arguments

1. *Argument-1* and *argument-2* must be integers.
2. The value of *argument-2* must not be zero.

Returned Values

1. The returned value is *argument-1* modulo *argument-2*. The returned value is defined as :

```
argument-1 - (argument-2 * FUNCTION INTEGER (argument-1 / argument-2))
```

2. The following illustrates the expected results for some values of *argument-1* and *argument-2*.

Argument-1	Argument-2	Return
11	5	1
-11	5	4
11	-5	-4
-11	-5	-1

12.5.25. The NUMVAL Function

Function

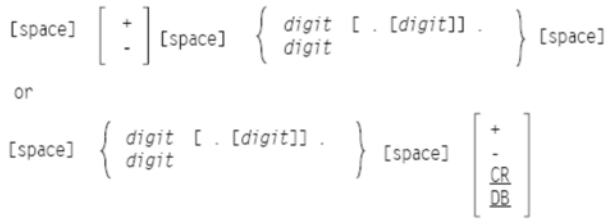
The NUMVAL function returns the numeric value represented by the character string specified by *argument-1*. Leading and trailing spaces are ignored. The type of this function is numeric.

Format

```
FUNCTION NUMVAL (argument-1)
```

Arguments

1. *Argument-1* must be a nonnumeric literal or alphanumeric data item whose content has one of the following two formats:



2. The total number of digits in *argument-1* must not exceed 18.
3. If the DECIMAL-POINT IS COMMA clause is specified in the SPECIAL-NAMES paragraph, a comma must be used in *argument-1* rather than a decimal point.

Returned Values

1. The returned value is the numeric value represented by *argument-1.*
2. The number of digits returned is 18.

Example

```
DISPLAY FUNCTION NUMVAL ("9").
```

This results in a 9 being displayed.

```
DISPLAY FUNCTION NUMVAL (" -  4929.0323).
```

This results in a -4929.0323 being displayed.

12.5.26. The NUMVAL-C Function

Function

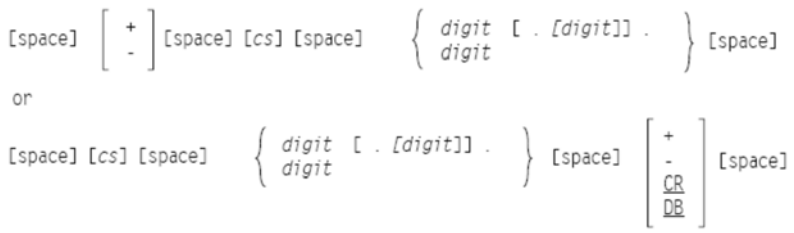
The NUMVAL-C function returns the numeric value represented by the character string specified by *argument-1*. Any optional currency symbol (*cs*) specified by *argument-2* and any optional commas preceding the decimal point are ignored. The type of this function is numeric.

Format

```
FUNCTION NUMVAL-C (argument -1 [argument-2])
```

Arguments

1. *Argument-1* must be a nonnumeric literal or alphanumeric data item whose content has one of the following two formats:



2. If the DECIMAL-POINT IS COMMA clause is specified in the SPECIAL-NAMES paragraph, the functions of the comma and decimal point in *argument-1* are reversed.
3. The total number of digits in *argument-1* must not exceed 18.
4. *Argument-2*, if specified, must be a nonnumeric literal or alphanumeric data item.
5. If *argument-2* is not specified, the character used for *cs* is the currency sign specified for the program.

Returned Values

1. The returned value is the numeric value represented by *argument-1.*
2. The number of digits returned is 18.

Example

```
DISPLAY FUNCTION NUMVAL-C (" - $ 890.21", "$").
```

This results in -890.21 being displayed.

12.5.27. The ORD Function

Function

The ORD function returns an integer value that is the ordinal position of *argument-1* in the collating sequence for the program. The lowest ordinal position is 1. The type of this function is integer.

Format

```
FUNCTION ORD (argument-1)
```

Arguments

Argument-1 must be a one-character alphabetic or alphanumeric elementary data item.

Returned Values

The returned value is the ordinal position of *argument-1* in the collating sequence for the program.

12.5.28. The ORD-MAX Function

Function

The ORD-MAX function returns a value that is the ordinal number of the *argument-1* that contains the maximum value. The type of this function is integer.

Format

```
FUNCTION ORD-MAX ( {argument-1}. . .)
```

Arguments

If more than one *argument-1* is specified, all arguments must be of the same class.

Returned Values

1. The value returned is the ordinal number that corresponds to the position of the *argument-1* having the greatest value in the *argument-1* series.
2. The comparisons used to determine the greatest-valued argument are made according to the rules for simple conditions.
3. If more than one *argument-1* has the same greatest value, the number returned corresponds to the position of the leftmost *argument-1* having that value.

12.5.29. The ORD-MIN Function

Function

The ORD-MIN function returns a value that is the ordinal number of the argument that contains the minimum value. The type of this function is integer.

Format

```
FUNCTION ORD-MIN ({argument-1}. . .)
```

Arguments

If more than one *argument-1* is specified, all arguments must be of the same class.

Returned Values

1. The value returned is the ordinal number that corresponds to the position of the *argument-1* having the least value in the *argument-1* series.
2. The comparisons used to determine the least-valued argument are made according to the rules for simple conditions.
3. If more than one *argument-1* has the same least value, the number returned corresponds to the position of the leftmost *argument-1* having that value.

12.5.30. The PRESENT-VALUE Function

Function

The PRESENT-VALUE function returns a value that approximates the present value of a series of future period-end amounts specified by *argument-2* at a discount rate specified by *argument-1*.

Format

```
FUNCTION PRESENT-VALUE ( argument-1{argument-2}. . .)
```

Arguments

1. *Argument-1* and *argument-2* must be class numeric.
2. The value of *argument-1* must be greater than -1.

Returned Values

The returned value is an approximation of the summation of a series of calculations with each term in the following form:

```
argument-2 / (1 + argument-1) ** n
```

There is one term for each occurrence of argument-2. The exponent *n* is incremented from one by one for each term in the series.

Example

```
01   RESULT1   PIC  9(3)V9(4).
01   RESULT2   PIC  9(3)V9(4).

COMPUTE RESULT1 = FUNCTION PRESENT VALUE (0, 23, 12, 9).
COMPUTE RESULT2 = FUNCTION PRESENT VALUE (1, 10, 20, 10, 5).

RESULT1 = 44.0000
RESULT2 = 11.5625
```

12.5.31.The RANDOM Function

Function

The RANDOM function returns a numeric value that is a pseudo-random number from a rectangular distribution. The type of this function is numeric.

Format

```
FUNCTION RANDOM [( argument-1)]
```

Arguments

1. If *argument-1* is specified, it must be zero or a positive integer. It is used as the seed value to generate a sequence of pseudo-random numbers.
2. If a subsequent reference specifies *argument-1*, a new sequence of pseudo-random numbers is started.
3. If the first reference to this function in the run unit does not specify *argument-1*, the seed is supplied by the compiler.
4. In each case, subsequent references without specifying *argument-1* return the next number in the current sequence.

Returned Values

1. The returned value is greater than or equal to zero and less than one.
2. For a given seed value, the sequence of pseudo-random numbers is always the same.

Example

```
01   RESULT1   PIC   V9(8).
01   RESULT2   PIC   V9(8).

COMPUTE RESULT1 = FUNCTION RANDOM.
COMPUTE RESULT2 = FUNCTION RANDOM (3).

RESULT1 is a value greater or equal to zero and less than one.
RESULT2 is a value greater or equal to zero and less than one.
```

12.5.32. The RANGE Function

Function

The RANGE function returns a value that is equal to the value of the maximum argument minus the value of the minimum argument. The type of this function depends upon the argument types as follows.

Argument Type	Function Type
All arguments integer	Integer
Numeric (some arguments can be integer)	Numeric

Format

```
FUNCTION RANGE({argument-1}. . .)
```

Arguments

Argument-1 must be class numeric.

Returned Values

The returned value is equal to the greatest value of *argument-1* minus the least value of argument-1. The comparisons used to determine the greatest and least values are made according to the rules for simple conditions. (See Section 3.6.3)

Example

```
01   RESULT1   PIC   999.
01   RESULT2   PIC   999V9.

COMPUTE RESULT1 = FUNCTION RANGE (5, -2, -14, 0).
COMPUTE RESULT2 = FUNCTION RANGE (3.9, -0.3, 8.7, 100, 2).

RESULT1 = 19
RESULT2 = 100.3
```

12.5.33. The REM Function

Function

The REM function returns a numeric value that is the remainder of *argument-1* divided by *argument-2*. The type of this function is numeric.

Format

```
FUNCTION REM (argument-1 argument-2)
```

Arguments

The value of *argument-2* must not be zero.

Returned Values

1. The returned value is the remainder of *argument-1* / *argument-2*. It is defined as the expression:

   ```
   argument-1 - (argument-2 * FUNCTION INTEGER-PART (argument-1 /
   argument-2))
   ```

2. The characteristics of the returned value are defined by the compiler.

12.5.34. The REVERSE Function

Function

The REVERSE function returns a character string of exactly the same length as *argument-1* and whose characters are exactly the same as those of *argument-1*, except that they are in reverse order. The type of this function is alphanumeric.

Format

```
FUNCTION REVERSE (argument-1)
```

Arguments

Argument-1 must be class alphabetic or numeric and must be at least one character in length.

Returned Values

If *argument-1* is a character string of length *n*, the returned value is a character string of length such that for 1 *j n*, the character in position *j* of the returned value is the character from position *n-j*+1 of *argument-1*.

Example

```
01   RESULT1   PIC   X(6).
01   RESULT2   PIC   X(4).

MOVE FUNCTION REVERSE ("figure") TO RESULT1.
MOVE FUNCTION REVERSE ("9562") TO RESULT2.

RESULT1 = erugif
RESULT2 = 2659
```

12.5.35. The SIN Function

Function

The SIN function returns a numeric value that approximates the sine of an angle or arc, expressed in radians, that is specified by *argument-1*. The type of this function is numeric.

Format

```
FUNCTION SIN (argument-1)
```

Arguments

Argument-1 must be class numeric.

Returned Values

The returned value is the approximation of the sine of *argument-1* and is greater than or equal to -1 and less than or equal to +1.

12.5.36. The SQRT Function

Function

The SQRT function returns a numeric value that approximates the square root of *argument-1*. The type of this function is numeric.

Format

```
FUNCTION SQRT (argument-1)
```

Arguments

The value of *argument-1* must be zero or positive.

Returned Values

The returned value is the absolute value of the approximation of the square root of *argument-1.*

12.5.37. The STANDARD-DEVIATION Function

Function

The STANDARD-DEVIATION function returns a numeric value that approximates the standard of its arguments. The type of function is numeric.

Format

```
FUNCTION STANDARD-DEVIATION ({argument-1}. . . )
```

Arguments

Argument-1 must be class numeric.

Returned Values

1. The returned value is the approximation of the standard deviation of the *argument-1* series.
2. The return value is calculated as follows:

   The difference between each *argument-1* value and the arithmetic mean of the *argument-1* series is calculated and squared. The values obtained are then added together. This quantity is divided by the number of values in the *argument-1* series. The square root of the quotient obtained is then calculated. The returned value is the absolute value of this square root.
3. If the *argument-1* series consists of only one value, or if the *argument-1* series consists of all variable occurrence data items and the total number for all of them is one, the returned value is zero.

Example

```
01   RESULT1   PIC   99V9(5)
01   RESULT2   PIC   99V9(5)

COMPUTE RESULT1 = FUNCTION STANDARD-DEVIATION (5, -2, -14, 0)
COMPUTE RESULT2 = FUNCTION STANDARD-DEVIATION (3.9, -0.3, 8.7, 100.2)

RESULT1 = 6.97764
RESULT2 = 41.7341
```

12.5.38. The SUM Function

Function

The SUM function returns a value that is the sum of the arguments. If all of the arguments are integers, the value returned is an integer. If any of the arguments is not an integer, a numeric value is returned. The type of this function depends upon the argument types as follows.

Argument Types	Function Types
All arguments integer	Integer
Numeric (some arguments can be integer)	Numeric

Format

```
FUNCTION SUM ({argument-1} . . . )
```

Arguments

Argument-1 must be class numeric.

Returned Values

1. The returned value is the sum of the arguments.
2. If the *argument-1* series are all integers, the value returned is an integer.
3. If the *argument-1* series are not all integers, a numeric value is returned. The characteristics of the returned value are defined by the compiler.

12.5.39. The TAN Function

Function

The TAN function returns a numeric value that approximates the tangent of an angle or arc, expressed in radians, that is specified by *argument-1.* The type of this function is numeric.

Format

```
FUNCTION TAN (argument-1)
```

Arguments

Argument-1 must be class numeric.

Returned Values

The returned value is the approximation of the tangent of *argument-1.*

12.5.40. The UPPER-CASE Function

Function

The UPPER-CASE function returns a character string that is the same length as *argument-1* with all lowercase characters replaced by the equivalent uppercase characters. The type of this function is alphanumeric.

Format

```
FUNCTION UPPER-CASE (argument-1)
```

Arguments

Argument-1 must be an alphabetic or alphanumeric data item.

Returned Values

1. The same character string as *argument-1* is returned, except that each lowercase alphabetic character is replaced by the equivalent uppercase character.
2. The character string returned has the same length as *argument-1.*

12.5.41. The VARIANCE Function

Function

The VARIANCE function returns a numeric value that approximates the variance of its arguments. The type of this function is numeric.

Format

```
FUNCTION VARIANCE ({argument-1}. . .)
```

Arguments

Argument-1 must be class numeric.

Returned Values

1. The returned value is the approximation of the variance of the *argument-1* series.
2. The returned value is defined as the square of the standard deviation of the *argument-1* series. (See Rule 2 under Returned Values for the STANDARD DEVIATION function in Section 12.5.37)
3. If the *argument-1* series consists of only one value, or if the *argument-1* series consists of all variable occurrence data items and the total number for all of them is one, the returned value is zero.

Example

```
01   RESULT1   PIC   99V9(4)
01   RESULT2   PIC   9(4)V99

COMPUTE RESULT1 = FUNCTION VARIANCE (5, -2, -14, 0)
COMPUTE RESULT2 = FUNCTION VARIANCE (3.9, -0.3, 8.7, 100.2)

RESULT1 = 48.6875
RESULT2 = 1741.74
```

12.5.42. The WHEN-COMPILED Function

Function

The WHEN-COMPILED function returns the date and time the program was compiled. The type of this function is alphanumeric.

Format

```
FUNCTION WHEN-COMPILED
```

Returned Values

1. The character positions, numbered from left to right, are:

Positions	Contents
1-4	Four numeric digits of the year in the Gregorian calendar
5-6	Two numeric digits of the month of the year, in the range 01 through 12
7-8	Two numeric digits of the day of the month, in the range 01 through 31
9-10	Two numeric digits of the hour past midnight, in the range 00 through 23
11-12	Two numeric digits of the minutes past the hour, in the range 00 through 59
13-14	Two numeric digits of the seconds past the minute, in the range 00 through 59
15-21	Contains a value of 0000000

2. The returned value is the date and time of compilation of the source program that contains this function. If the program is a contained program, the returned value is the compilation date and time associated with the separately compiled program in which it is contained.
3. The returned value must be consistent with the compilation date and time, if provided in the listing of the source program and in the generated object code for the source program, although their representation can differ.

Appendix A
Reserved Words

The following words are an inherent part of Universal Compiling System (UCS) COBOL and cannot be used for data or procedure names.

ACCEPT
ACCESS
ACQUIRE
ADD
ADVANCING
AFTER
ALL
ALAPHABET
ALAPHABETIC
ALAPHABETIC-LOWER
ALAPHABETIC-UPPER
ALPHANUMERIC
ALPHANUMERIC-EDITED
ALSO
ALTER
ALTERNATE
AND
ANY
APPLY
ARE
AREA
AREAS
ASCENDING
ASSIGN
AT
AUTHOR
BEFORE
BINARY
BINARY-1
BIT
BLANK
BLOCK
BOTTOM
BY
BYTE-I18N
CALL
CANCEL
CD
CF
CH
CHARACTER
CHARACTERS
CLASS
CLASS-NAME
CLOCK-UNITS
CLOSE
COBOL
CODE
CODE-SET
COLLATING
COLUMN
COMMA

COMMON
COMMON-STORAGE
COMMUNICATION
COMP
COMP-1
COMP-2
COMPUTATIONAL
COMPUTATIONAL-1
COMPUTATIONAL-2
COMPUTE
CONFIGURATION
CONTAINS
CONTENT
CONTINUE
CONTROL
CONTROLS
CONVERTING
COPY
CORR
CORRESPONDING
COUNT
CURRENCY
DATA
DATE
DATE-COMPILED
DATE-WRITTEN
DAY
DAY-OF-WEEK
DE
DEBUG-CONTENTS
DEBUG-ITEM
DEBUG-LINE
DEBUG-NAME
DEBUG-SUB-1
DEBUG-SUB-2
DEBUG-SUB-3
DEBUGGING
DECIMAL-POINT
DECLARATIVES
DELETE
DELIMITED
DELIMITER
DENSITY
DEPART
DEPENDING
DESCENDING
DESTINATION
DETAIL
DISABLE
DISP
DISP-2
DISP-I18N
DISPLAY
DISPLAY-2
DIVIDE
DIVISION
DOWN
DUPLICATE
DUPLICATES
DYNAMIC
EGI
ELSE
EMI
EMPTY
ENABLE
END
END-ADD
END-CALL
END-COMPUTE
END-DELETE
END-DIVIDE
END-EVALUATE

END-IF
END-MULTIPLY
END-OF-PAGE
END-PERFORM
END-READ
END-RECEIVE
END-RETURN
END-REWRITE
END-SEARCH
END-START
END-STRING
END-SUBTRACT
END-UNSTRING
END-WRITE
ENTER
ENTRY
ENVIRONMENT
EOP
EQUAL
ERROR
ESI
EVALUATE
EVERY
EXCEPTION
EXDEF
EXEC
EXIT
EXREF
EXTEND
EXTERNAL
FALSE
FD
FETCH
FILE
FILE-CONTROL
FILLER
FINAL
FIND
FIRST
FOOTING
FOR
FREE
FROM
FUNCTION
GENERATE
GET
GIVING
GLOBAL
GO
GREATER
GROUP
HEADING
HIGH-VALUE
HIGH-VALUES
I-O
I-O-CONTROL
IDENTIFICATION
IF
IMPART
IN
INDEX
INDEXED
INDICATE
INITIAL
INITIALIZE
INITIATE
INPUT
INPUT-OUTPUT
INSERT
INSPECT
INSTALLATION
INTO

INVALID
INVOKE
IS
JUST
JUSTIFIED
KEEP
KEY
LABEL
LAST
LEADING
LEFT
LENGTH
LESS
LIMIT
LIMITS
LINAGE
LINAGE-COUNTER
LINE
LINE-COUNTER
LINES
LINKAGE
LOCALE
LOCK
LOG
LOW-VALUE
LOW-VALUES
MEMORY
MERGE
MESSAGE
MODE
MODE-1
MODE-2
MODE-3
MODE-4
MODIFY
MODULES
MOVE
MULTIPLE
MULTIPLY
NATIVE
NEGATIVE
NEXT
NO
NOT
NUMBER
NUMERIC
NUMERIC-EDITED
OBJECT-COMPUTER
OCCURS
OF
OFF
OMITTED
ON
OPEN
OPTIONAL
OR
ORDER
ORGANIZATION
OTHER
OUTPUT
OVERFLOW
OWNER
PACKED-DECIMAL
PADDING
PAGE
PAGE-COUNTER
PERFORM
PF
PH
PIC
PICTURE
PLUS

POINTER
POINTS
POSITION
POSITIVE
PRINTING
PRIOR
PROCEDURE
PROCEDURES
PROCEED
PROGRAM
PROGRAM-ID
PURGE
QUEUE
QUOTE
QUOTES
RANDOM
RANK
RD
READ
RECEIVE
RECORD
RECORDING
RECORDS
REDEFINES
REEL
REFERENCE
REFERENCES
RELATIVE
RELEASE
REMAINDER
REMOVAL
REMOVE
RENAMES
REPLACE
REPLACING
REPORT
REPORTING
REPORTS
RERUN
RESERVE
RESET
RETURN
REVERSED
REWIND
REWRITE
RF
RH
RIGHT
ROUNDED
RUN
SAME
SD
SEARCH
SECTION
SECURITY
SEGMENT
SEGMENT-LIMIT
SELECT
SEND
SENTENCE
SEPARATE
SEQUENCE
SEQUENTIAL
SET
SIGN
SIZE
SORT
SORT-MERGE
SOURCE
SOURCE-COMPUTER
SPACE
SPACES

SPECIAL-NAMES
STANDARD
STANDARD-1
STANDARD-2
START
STATUS
STOP
STORE
STRING
SUB-QUEUE-1
SUB-QUEUE-2
SUB-QUEUE-3
SUBTRACT
SUM
SUPPRESS
SYMBOLIC
SYNC
SYNCHRONIZED
TABLE
TALLYING
TAPE
TERMINAL
TERMINATE
TEST
TEXT
THAN
THEN
THROUGH
THRU
TIME
TIMES
TO
TOP
TRAILING
TRUE
TYPE
UNIT
UNSTRING
UNTIL
UP
UPON
USAGE
USE
USING
VALUE
VALUES
VARYING
WHEN
WITH
WORDS
WORKING-STORAGE
WRITE
ZERO
ZEROES
ZEROS
+
-
*
/
**
>
<
=
>=
<=

Appendix B
Syntax Skeleton

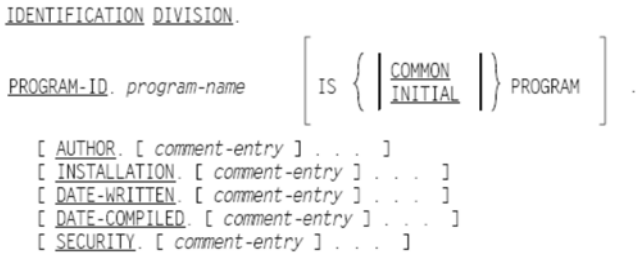
B.1. General Description

This appendix contains the composite language skeleton of UCS COBOL. It displays the complete and correct syntax.

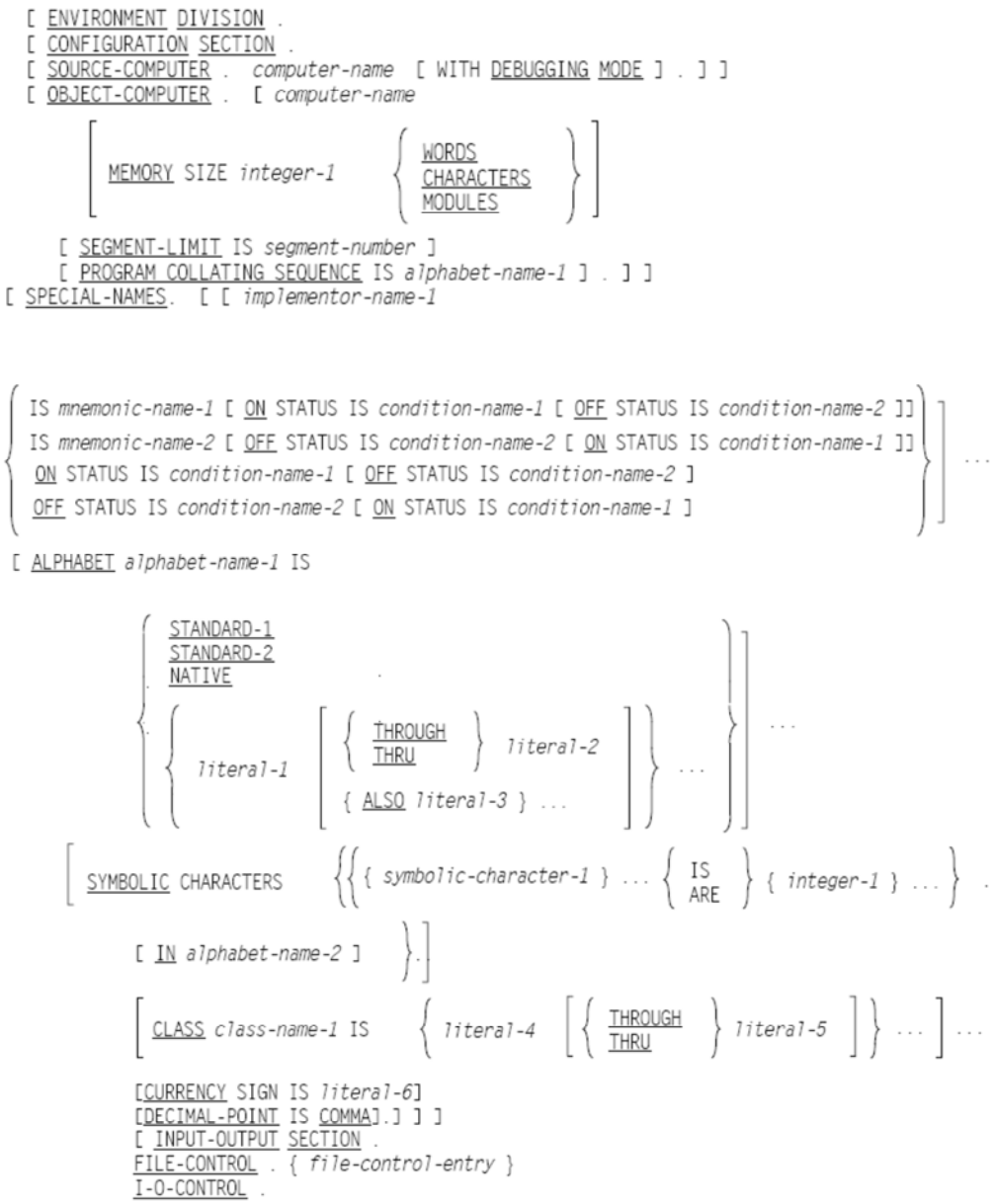
In B.2, the leftmost margin is equivalent to margin A in a COBOL source program. The first indentation after the leftmost margin is equivalent to margin B.

In B.5.1 through B.5.43, the leftmost margin indicates the beginning of the format for a new COBOL verb. The first indentation after the leftmost margin indicates continuation of the format of the COBOL verb.

B.2. Identification Division



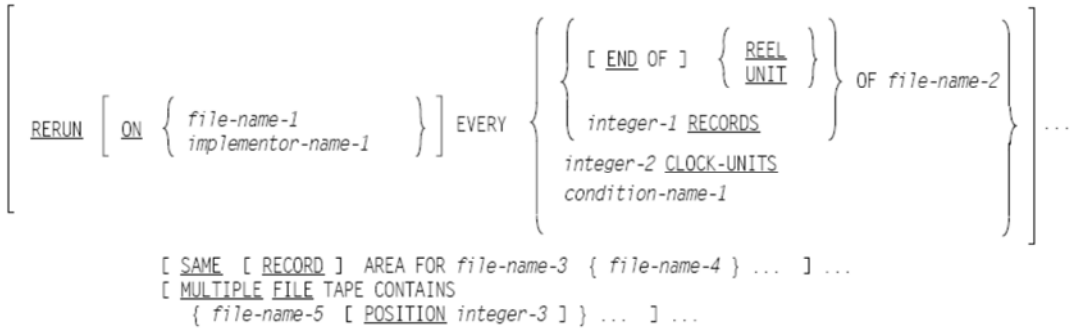
B.3. Environment Division



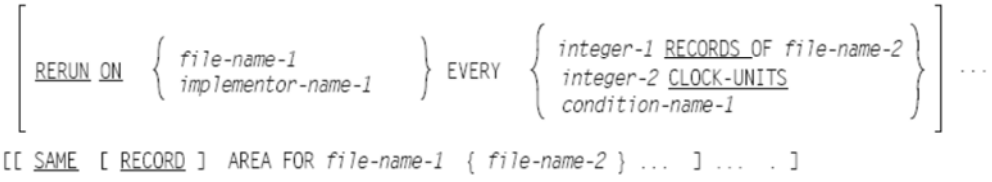
Interprogram Communication (IPC)



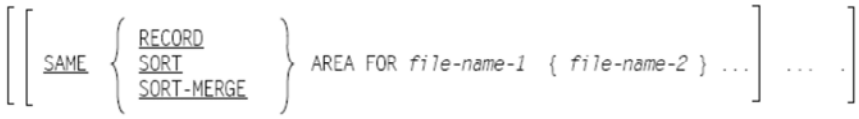
Sequential I–O Format



Relative and Indexed I–O Format

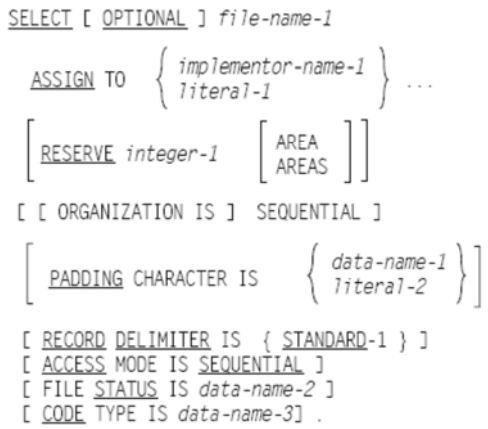


Sort–Merge Format

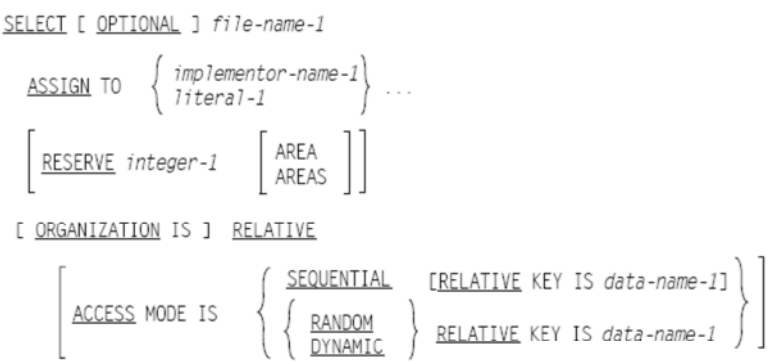


File–Control Entry

Sequential I–O Format



Relative I–O Format



Indexed I–O Format

```
SELECT [ OPTIONAL ] file-name-1
  ASSIGN TO { implementor-name-1 } ...
            { literal-1          }
  [ RESERVE integer-1 [ AREA  ] ]
                      [ AREAS ]
  [ ORGANIZATION IS ] INDEXED
    [ ACCESS MODE IS { SEQUENTIAL } ]
                     { RANDOM     }
                     { DYNAMIC    }
RECORD KEY IS data-name-1
  [ ALTERNATE RECORD KEY IS data-name-2 [ WITH DUPLICATES ] ] ...
  [ FILE STATUS IS data-name-3 ]
  [ LOCALE IS data-name-4 ] .
```

Sort–Merge Format

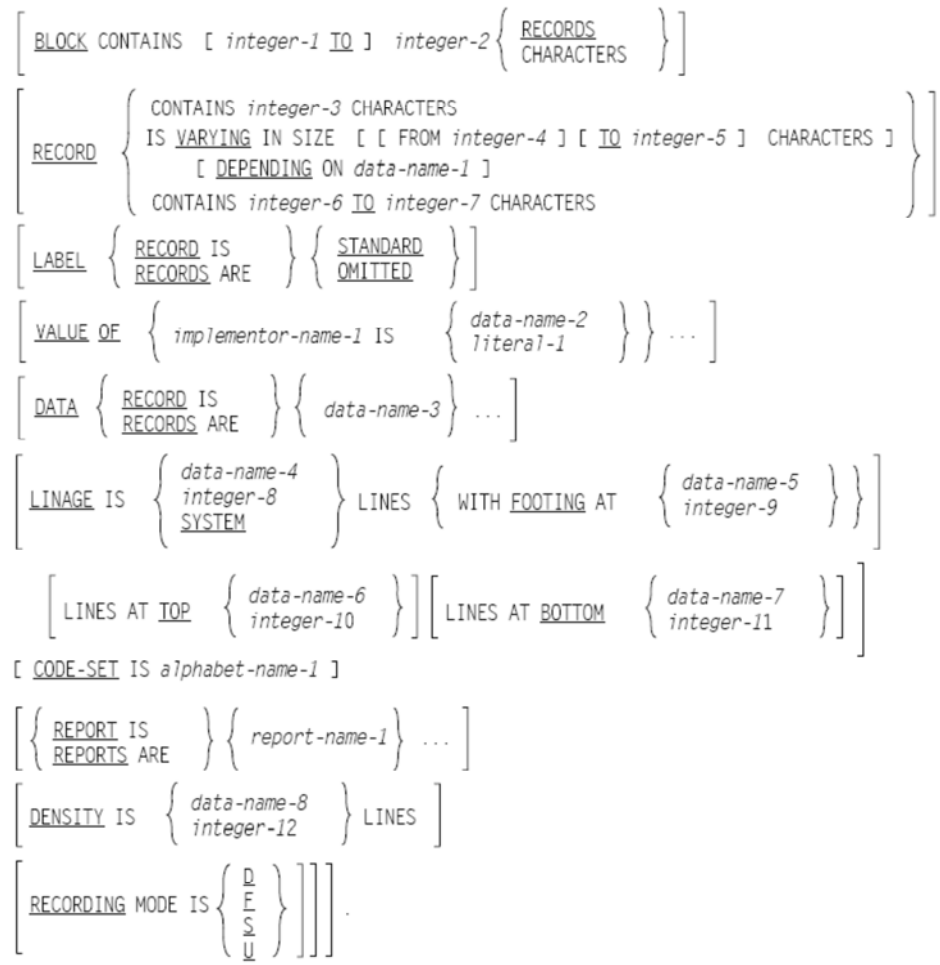
```
SELECT file-name-1 ASSIGN TO { implementor-name-1 } ...
```

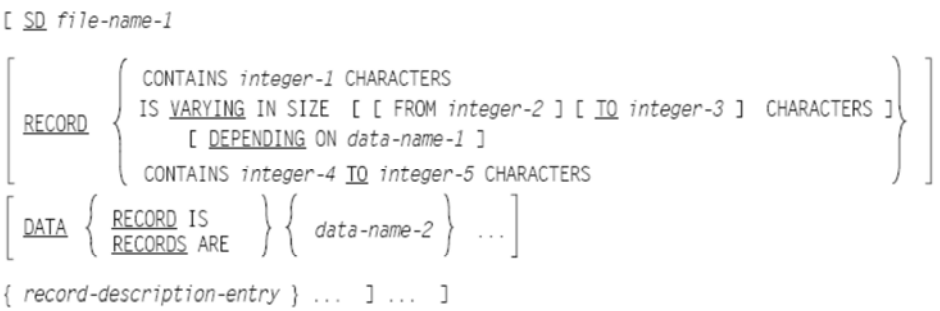
Information Interchange Tape Processing Format

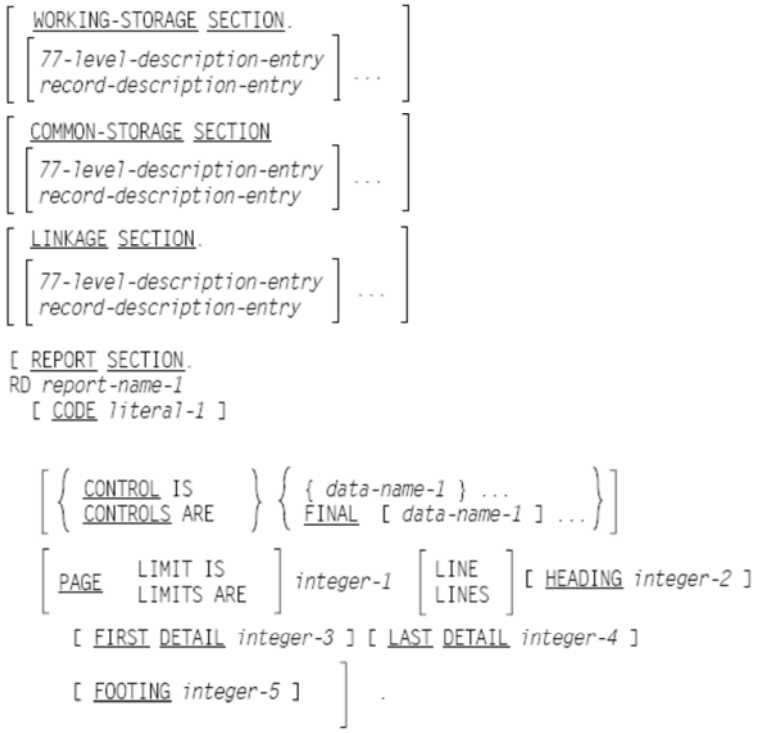
```
SELECT [ OPTIONAL ] file-name-1
  ASSIGN TO implementor-name-1 [ , implementor-name-2 ] ...
[ RESERVE integer-1 [ AREA  ] ]
                    [ AREAS ]
[ ORGANIZATION IS SEQUENTIAL ]
[ PADDING CHARACTER IS { data-name-1 } ]
                       { literal-1   }
[ ACCESS MODE IS SEQUENTIAL ]
[ FILE STATUS IS data-name-1 ] .
```

B.4. Data Division

```
DATA DIVISION .
[ FILE SECTION .
[ FD file-name-1
  [ IS EXTERNAL ]
  [ IS GLOBAL ]
```

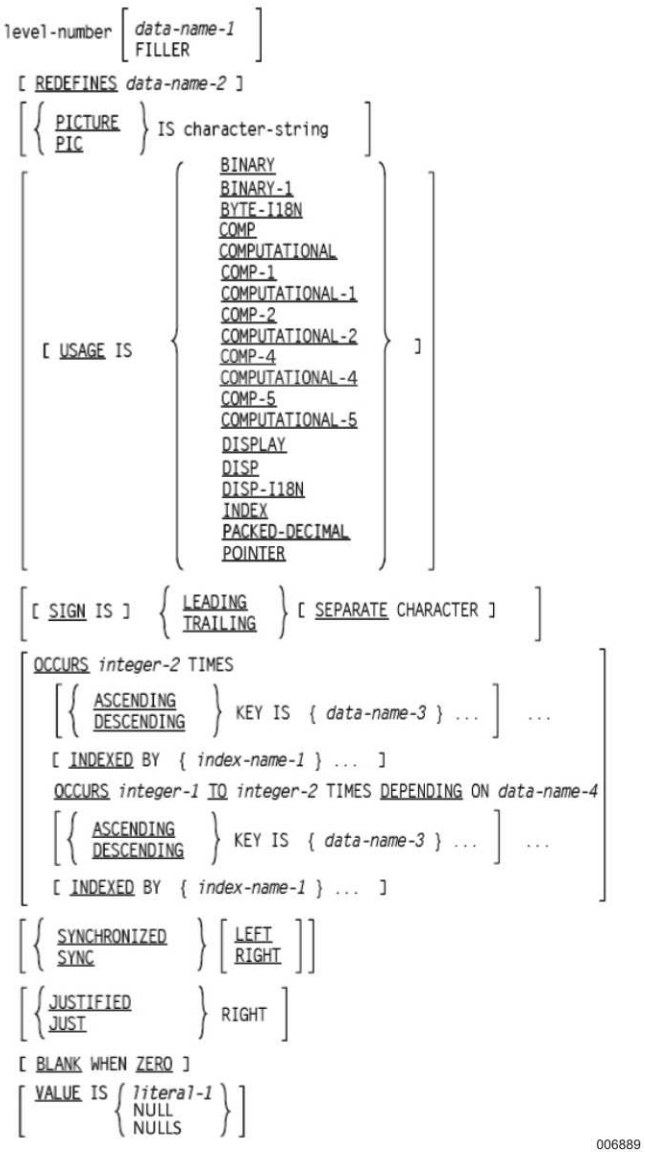

[BLOCK CONTAINS [integer-1 TO] integer-2 { RECORDS / CHARACTERS }]
[RECORD { CONTAINS integer-3 CHARACTERS / IS VARYING IN SIZE [[FROM integer-4] [TO integer-5] CHARACTERS] [DEPENDING ON data-name-1] / CONTAINS integer-6 TO integer-7 CHARACTERS }]
[LABEL { RECORD IS / RECORDS ARE } { STANDARD / OMITTED }]
[VALUE OF { implementor-name-1 IS { data-name-2 / literal-1 } } ...]
[DATA { RECORD IS / RECORDS ARE } { data-name-3 } ...]
[LINAGE IS { data-name-4 / integer-8 / SYSTEM } LINES { WITH FOOTING AT { data-name-5 / integer-9 } } [LINES AT TOP { data-name-6 / integer-10 }] [LINES AT BOTTOM { data-name-7 / integer-11 }]]
[CODE-SET IS alphabet-name-1]
[{ REPORT IS / REPORTS ARE } { report-name-1 } ...]
[DENSITY IS { data-name-8 / integer-12 } LINES]
[RECORDING MODE IS { D / F / S / U }]]]] .


[SD file-name-1
[RECORD { CONTAINS integer-1 CHARACTERS / IS VARYING IN SIZE [[FROM integer-2] [TO integer-3] CHARACTERS] [DEPENDING ON data-name-1] / CONTAINS integer-4 TO integer-5 CHARACTERS }]
[DATA { RECORD IS / RECORDS ARE } { data-name-2 } ...]
{ record-description-entry } ...] ...]

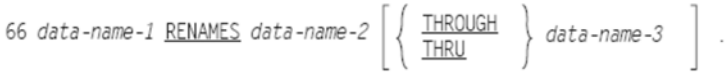

WORKING-STORAGE SECTION.
77-level-description-entry
record-description-entry
COMMON-STORAGE SECTION
77-level-description-entry
record-description-entry
LINKAGE SECTION.
77-level-description-entry
record-description-entry
[REPORT SECTION.
RD report-name-1
[CODE literal-1]
CONTROL IS
CONTROLS ARE
{ data-name-1 } ...
FINAL [data-name-1] ...
PAGE
LIMIT IS
LIMITS ARE
integer-1
LINE
LINES
[HEADING integer-2]
[FIRST DETAIL integer-3] [LAST DETAIL integer-4]
[FOOTING integer-5]

B.4.1. Data Description Entry

Format 1



Format 2

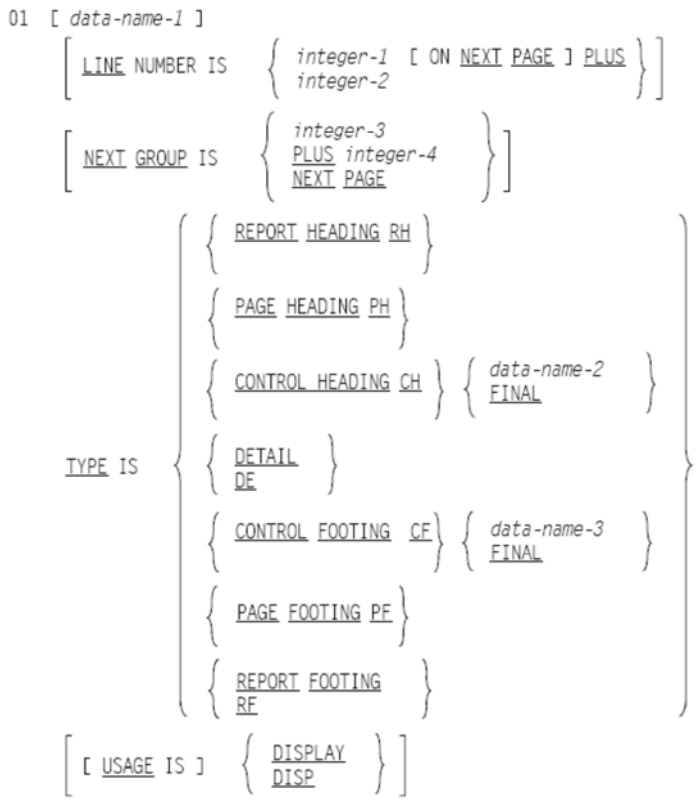


Format 3

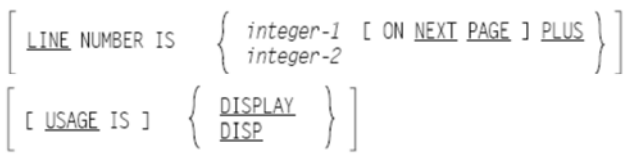


B.4.2. Report Group Description Entry

Format 1



Format 2



Format 3

```
level-number  [ data-name-1 ]
{ PICTURE | PIC } IS character-string
[ [ USAGE IS ] { DISPLAY | DISP } ]
[ [ SIGN IS ] { LEADING | TRAILING } SEPARATE CHARACTER ]
[ { JUSTIFIED | JUST } RIGHT ]
  [ BLANK WHEN ZERO ]
[ LINE NUMBER IS { integer-1 [ON NEXT PAGE ] | PLUS integer-2 } ]
[ COLUMN NUMBER IS integer-3 ]
{ SOURCE IS identifier-1
  VALUE IS literal-1
  { SUM  { identifier-2 } ...
      [ UPON  { data-name-2 } ...  ] } ...
      [ RESET ON { data-name-3 | FINAL } ] }
[ GROUP INDICATE ] .
```

B.5. Procedure Division

Format 1

```
PROCEDURE DIVISION  [ USING  { data-name-1 } ...  ]
    [ WITH ENTRY POINTS  { procedure-name-1 } ...  ]  .
[ DECLARATIVES.
{ section-name SECTION  [ segment-number ] .

   USE  [ GLOBAL ]  { AFTER STANDARD { EXCEPTION | ERROR } PROCEDURE ON
                                     { { file-name-1 }...
                                       INPUT
                                       OUTPUT
                                       I-O
                                       EXTEND }
                    | BEFORE REPORTING identifier }
         [ paragraph-name.
             [ sentence ] ...  ] ...  } ...
         END DECLARATIVES.
         { section-name SECTION  [ segment-number ].
         [ paragraph-name.
             [ sentence ] ...  ] ...  } ...
```

Format 2

```
PROCEDURE DIVISION  [ USING  { data-name-1 } ...  ]
    [ WITH ENTRY POINTS  { procedure-name-1 } ...  ] .
{ paragraph-name .
    [ sentence ] ...  } ...
```

B.5.1. The ACCEPT Statement

Format 1

```
ACCEPT identifier-2  [ FROM mnemonic-name-1 ]
```

Format 2

```
ACCEPT identifier-2 FROM { DATE [YYYYMMDD]
                         { DAY [YYYYDDD]
                         { DAY-OF-WEEK
                         { TIME
```

B.5.2. The ADD Statement

Format 1

```
ADD { identifier-1 } ...  TO  { identifier-2  [ ROUNDED ] } ...
    { literal-1    }

[ ON SIZE ERROR imperative-statement-1 ]
[ NOT ON SIZE ERROR imperative-statement-2 ]
[ END-ADD ]
```

Format 2

```
ADD { identifier-1 } ...  TO  { identifier-2  [ ROUNDED ] } ...
    { literal-1    }

[ ON SIZE ERROR imperative-statement-1 ]
[ NOT ON SIZE ERROR imperative-statement-2 ]
[ END-ADD ]
```

Format 3

```
ADD { CORRESPONDING } identifier-1 TO identifier-2  [ ROUNDED ]
    { CORR          }

[ ON SIZE ERROR imperative-statement-1 ]
[ NOT ON SIZE ERROR imperative-statement-2 ]
[ END-ADD ]
```

B.5.3. The ALTER Statement

```
ALTER  { procedure-name-1 TO  [ PROCEED TO ]  procedure-name-2 } ...
```

B.5.4. The CALL Statement

Format 1

```
CALL { identifier-1 } [ USING { [ BY REFERENCE ] { procedure-name } ...                    } ... ]
     { literal-1    }          { identifier-2    }
                               { BY CONTENT { literal-2 } ...                              }
                               { identifier-2 }
                               { [ BY CONTENT ] { literal-2 } ...                          }
  [ ON OVERFLOW imperative-statement-1 ]   [ END-CALL ]
```

Format 2

```
CALL { identifier-1 } [ USING { [ BY REFERENCE ] { procedure-name } ...                    } ... ]
     { literal-1    }          { identifier-2    }
                               { BY CONTENT { literal-2 } ...                              }
                               { identifier-2 }
                               { [ BY CONTENT ] { literal-2 } ...                          }
  [ ON EXCEPTION imperative-statement-1 ]
  [ NOT ON EXCEPTION imperative-statement-2 ]
  [ END-CALL ]
```

B.5.5. The CANCEL Statement

```
CANCEL { identifier-1 } ...
       { literal-1    }
```

B.5.6. The CLOSE Statement

Sequential I–O Format

```
CLOSE { file-name-1 [ { REEL   } [ FOR REMOVAL ]                  ] } ...
                    { UNIT   }                                      }
                    { WITH { NO REWIND }                            }
                           { LOCK      }
```

Relative and Indexed I–O Format

```
CLOSE  { file-name-1  [ WITH LOCK ] } ...
```

B.5.7. The COMPUTE Statement

```
COMPUTE  { identifier-1  [ ROUNDED ] } ...  = arithmetic-expression-1
   [ ON SIZE ERROR imperative-statement-1 ]
   [ NOT ON SIZE ERROR imperative-statement-2 ]
   [ END-COMPUTE ]
```

B.5.8. The CONTINUE Statement

```
CONTINUE
```

B.5.9. The DELETE Statement

```
DELETE file-name-1 RECORD
[ INVALID KEY imperative-statement-1 ]
[ NOT INVALID KEY imperative-statement-2 ]
[ END-DELETE ]
```

B.5.10. The DISPLAY Statement

```
DISPLAY { identifier } ... [ UPON mnemonic-name ] [ WITH NO ADVANCING ]
        { literal    }
```

B.5.11. The DIVIDE Statement

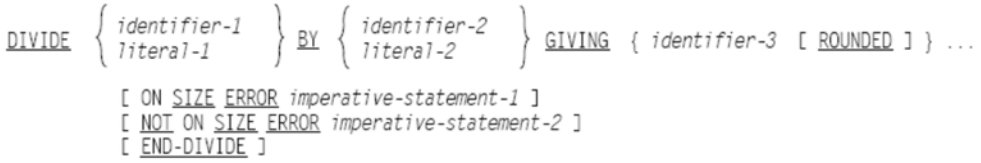
Format 1

```
DIVIDE { identifier-1 } INTO { identifier-2  [ ROUNDED ] } ...
       { literal-1    }
[ ON SIZE ERROR imperative-statement-1 ]
[ NOT ON SIZE ERROR imperative-statement-2 ]
[ END-DIVIDE ]
```

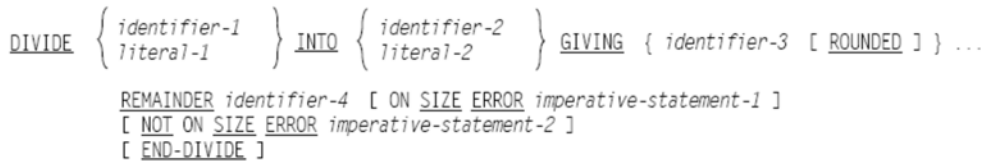
Format 2

```
DIVIDE { identifier-1 } INTO { identifier-2 } GIVING { identifier-3  [ ROUNDED ] } ...
       { literal-1    }      { literal-2    }
          [ ON SIZE ERROR imperative-statement-1 ]
          [ NOT ON SIZE ERROR imperative-statement-2 ]
          [ END-DIVIDE ]
```

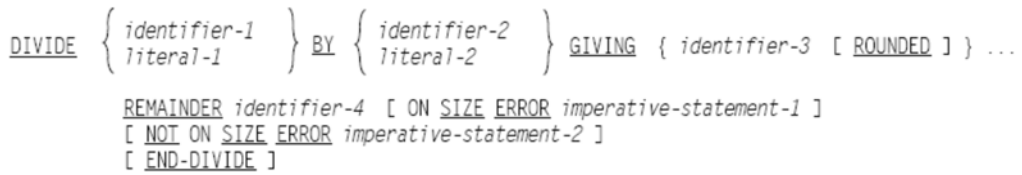
Format 3



Format 4

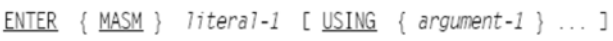


Format 5



B.5.12. The ENTER Statement

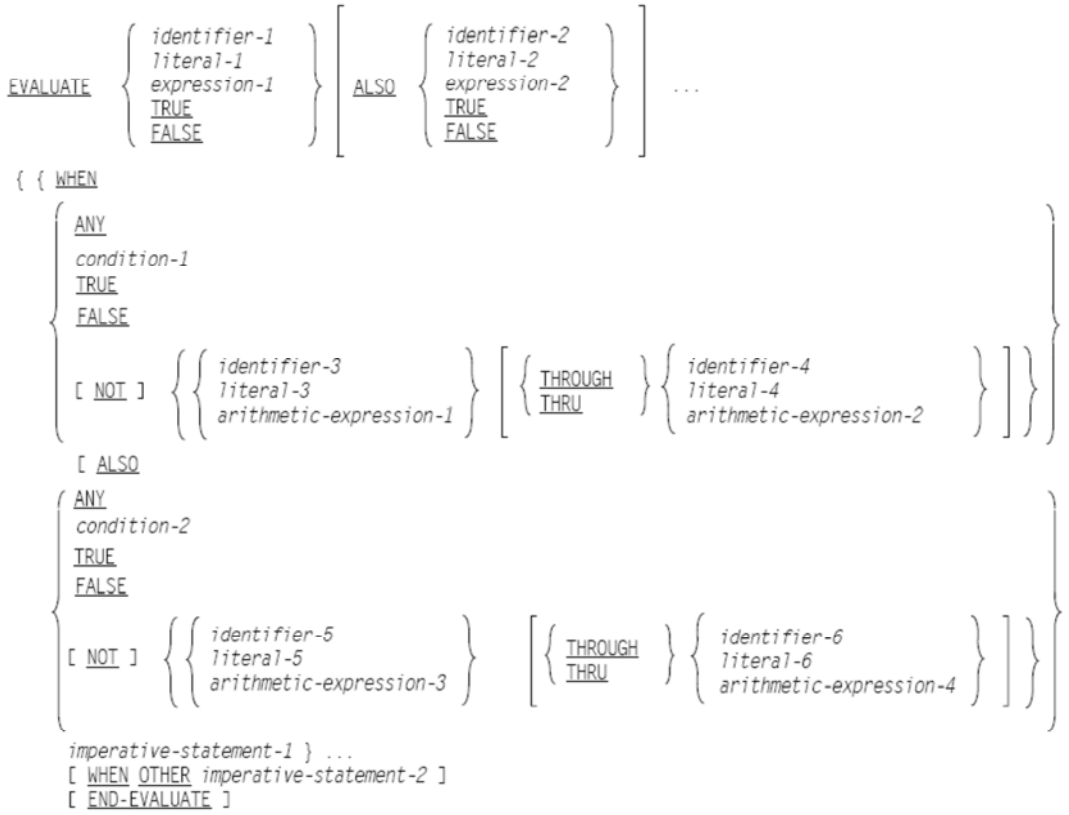
Format 1



Format 2



B.5.13. The EVALUATE Statement



B.5.14. The EXIT Statement

```
EXIT
```

B.5.15. The EXIT PROGRAM Statement

```
EXIT PROGRAM
```

B.5.16. The GENERATE Statement



B.5.17. The GOBACK Statement

```
GOBACK
```

B.5.18. The GO TO Statement

Format 1

```
GO TO [ procedure-name-1 ]
```

Format 2

```
GO TO  { procedure-name-1 } ...  DEPENDING ON identifier-1
```

B.5.19. The IF Statement

```
IF condition-1 THEN    { { statement-1 } ... }
                       { NEXT SENTENCE       }

{ ELSE { statement-2 } ...  [ END-IF ] ELSE }
{ NEXT SENTENCE                             }
{ END-IF                                    }
```

B.5.20. The INITIALIZE Statement

```
INITIALIZE { identifier-1 } ...
[ REPLACING { { ALPHABETIC          } DATA BY { identifier-2 } } ... ]
            { { ALPHANUMERIC        }         { literal-1    } }
            { { NUMERIC             }
            { { ALPHANUMERIC-EDITED }
            { { NUMERIC-EDITED      }
```

B.5.21. The INITIATE Statement

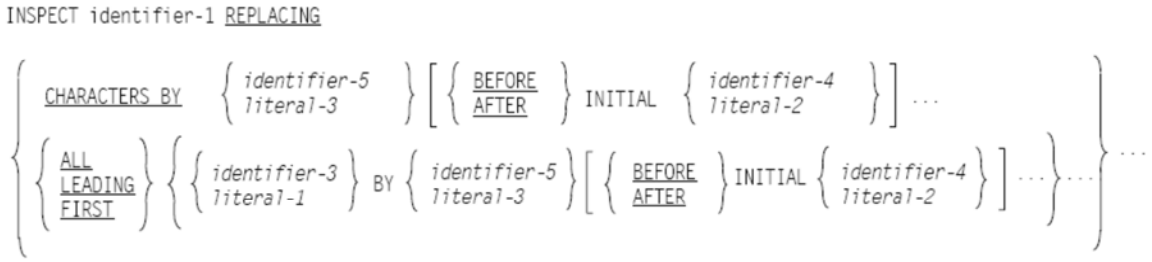
```
INITIATE {report-name-1}. . .
```

B.5.22. The INSPECT Statement

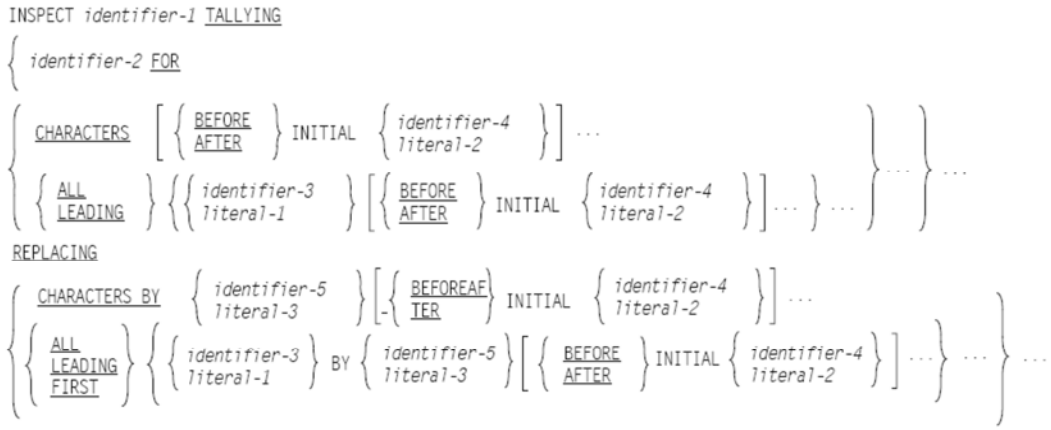
Format 1

```
INSPECT identifier-1 TALLYING
{ identifier-2 FOR
{ CHARACTERS [ { BEFORE } INITIAL { identifier-4 } ] ...                                                                  } ... } ...
{            { AFTER  }           { literal-2    }                                                                      }
{ { ALL     } { { identifier-3 } [ { BEFORE } INITIAL { identifier-4 } ] ... } ... }
{ { LEADING }   { { literal-1    }   { AFTER  }         { literal-2    }           }
```

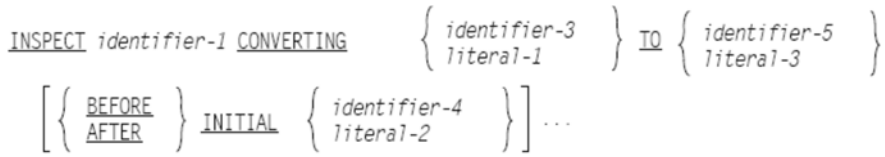
Format 2



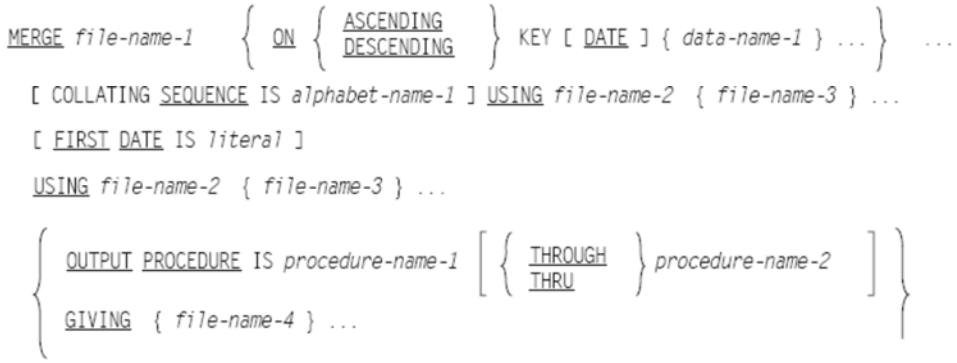
Format 3



Format 4

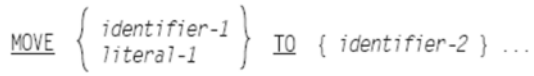


B.5.23. The MERGE Statement



B.5.24. The MOVE Statement

Format 1

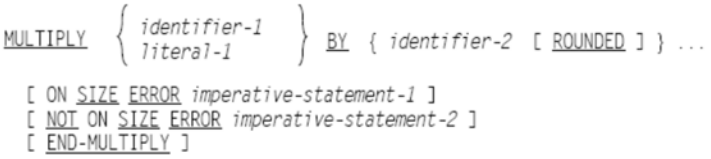


Format 2

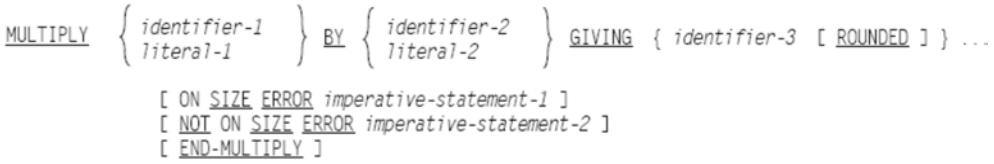


B.5.25. The MULTIPLY Statement

Format 1

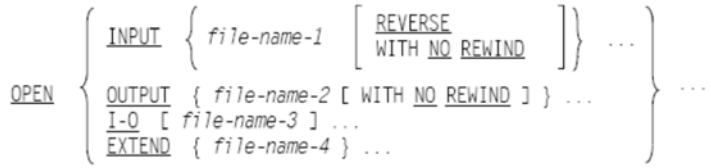


Format 2

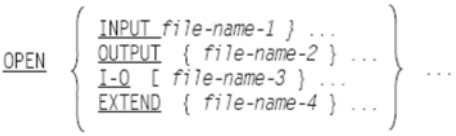


B.5.26. The OPEN Statement

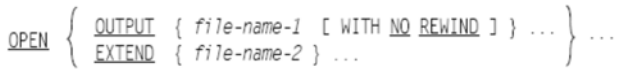
Sequential I–O Format



Relative and Indexed I–O Format



Report Writer Format

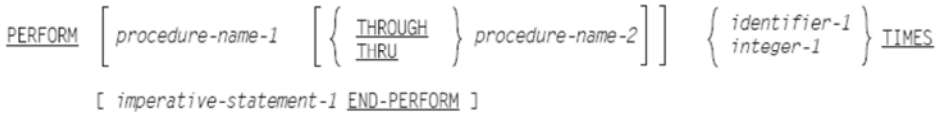


B.5.27. The PERFORM Statement

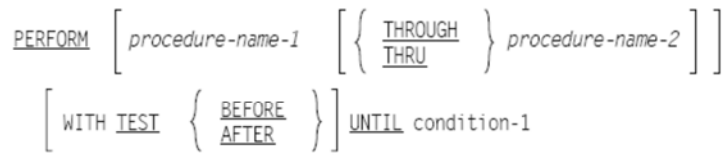
Format 1



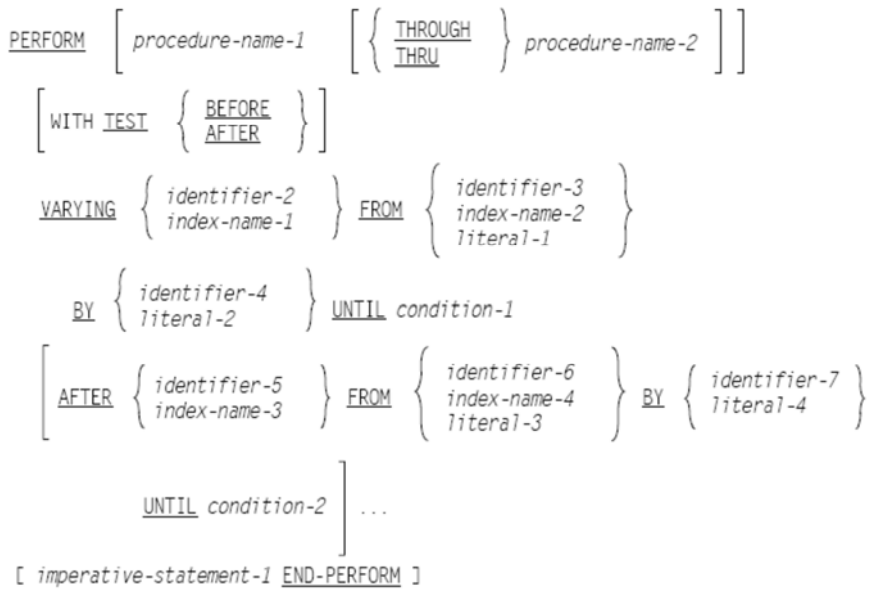
Format 2



Format 3



Format 4



B.5.28. The READ Statement

Sequential I–O Format

```
READ file-name-1  [ NEXT ]  RECORD  [ INTO identifier-1 ]
  [ AT END imperative-statement-1 ]
  [ NOT AT END imperative-statement-2 ]
  [ END-READ ]
```

Relative I–O Format

Format 1

```
READ file-name-1  [ NEXT ]  RECORD  [ INTO identifier-1 ]
  [ AT END imperative-statement-1 ]
  [ NOT AT END imperative-statement-2 ]
  [ END-READ ]
```

Format 2

```
READ file-name-1 RECORD  [ INTO identifier-1 ]
  [ INVALID KEY imperative-statement-1 ]
  [ NOT INVALID KEY imperative-statement-2 ]
  [ END-READ ]
```

Indexed I–O Format

Format 1

```
READ file-name-1  [ NEXT ]  RECORD  [ INTO identifier-1 ]
  [ AT END imperative-statement-1 ]
  [ NOT AT END imperative-statement-2 ]
  [ END-READ ]
```

Format 2

```
READ file-name-1 RECORD  [ INTO identifier-1 ]
  [ KEY IS data-name-1 ]
  [ INVALID KEY imperative-statement-3 ]
  [ NOT INVALID KEY imperative-statement-4 ]
  [ END-READ ]
```

B.5.29. The RELEASE Statement

```
RELEASE record-name-1  [ FROM identifier-1 ]
```

B.5.30. The RETURN Statement

```
RETURN file-name-1 RECORD  [ INTO identifier-1 ]
  AT END imperative-statement-1
  [ NOT AT END imperative-statement-2 ]
  [ END-RETURN ]
```

B.5.31. The REWRITE Statement

Sequential I–O Format

```
REWRITE record-name-1  [ FROM identifier-1 ] [ END-REWRITE ]
```

Relative and Indexed I–O Format

```
REWRITE record-name-1  [ FROM identifier-1 ]
 [ INVALID KEY imperative-statement-1 ]
 [ NOT INVALID KEY imperative-statement-2 ]
 [ END-REWRITE ]
```

B.5.32. The SEARCH Statement

Format 1

```
SEARCH identifier-1    [ VARYING { identifier-2 | index-name-1 } ]
   [ AT END imperative-statement-1 ]
   { WHEN condition-1   { imperative-statement-2 | NEXT SENTENCE } } ...
   [ END-SEARCH ]
```

Format 2

```
SEARCH ALL identifier-1  [ AT END imperative-statement-1 ]
     WHEN { data-name-1   { IS EQUAL TO IS | = } { identifier-3 | literal-1 | arithmetic-expression-1 } |
            condition-name-1 }
       [ AND { data-name-2   { IS EQUAL TO IS | = } { identifier-4 | literal-2 | arithmetic-expression-2 } |
               condition-name-1 } ] ...
       { imperative-statement-2 NEXT SENTENCE }
  [ END-SEARCH ]
```

B.5.33. The SET Statement

Format 1

```
SET { index-name-1 | identifier-1 } ... TO { index-name-2 | identifier-2 | integer-1 }
```

Format 2

```
SET { index-name-3 } ... { UP BY   } { identifier-3 }
                         { DOWN BY } { integer-2    }
```

Format 3

```
SET { { mnemonic-name-1 } ... TO { ON  } } ...
                                 { OFF }
```

Format 4

```
SET { condition-name-1 } ... TO TRUE
```

Format 5

```
SET { identifier-4          } ... TO { identifier-6          }
    { ADDRESS OF identifier-5 }      { ADDRESS OF identifier-7 }
                                     { NULL                    }
                                     { NULLS                   }
```

006826

B.5.34.The SORT Statement

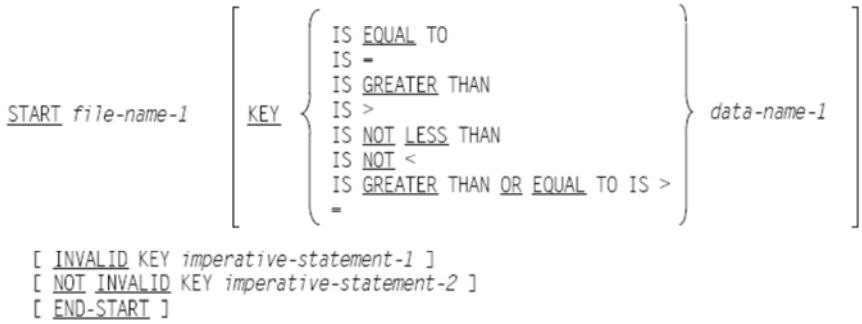
```
SORT file-name-1 { ON { ASCENDING  } KEY [DATE] { data-name-1 } ... } ...
                     { DESCENDING }

  [ WITH DUPLICATES IN ORDER ]
  [ COLLATING SEQUENCE IS alphabet-name ]
  [ FIRST DATE IS literal ]

  { INPUT PROCEDURE IS procedure-name-1 [ { THROUGH } procedure-name-2 ] }
                                        { THRU    }
  { USING { file-name-2 } ...                                           }

  { OUTPUT PROCEDURE IS procedure-name-3 [ { THROUGH } procedure-name-4 ] }
                                         { THRU    }
  { GIVING { file-name-3 } ...                                          }
```

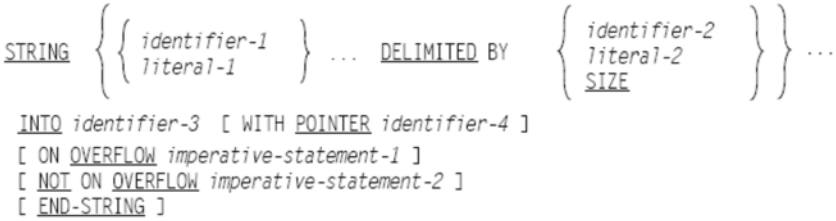
B.5.35. The START Statement



B.5.36. The STOP Statement

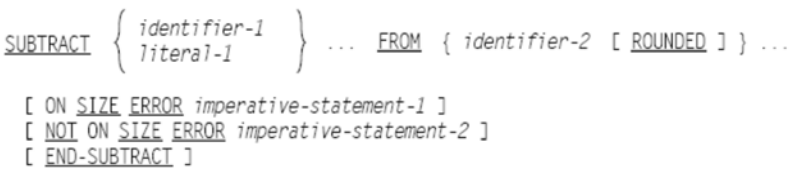


B.5.37. The STRING Statement

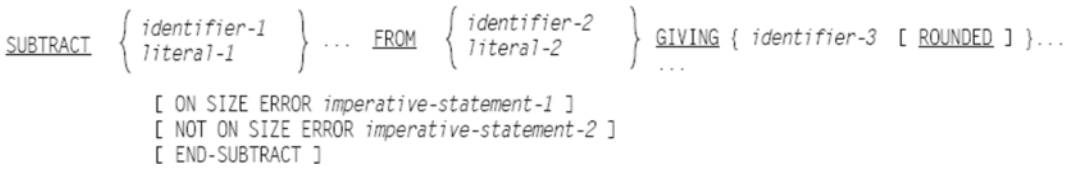


B.5.38. The SUBTRACT Statement

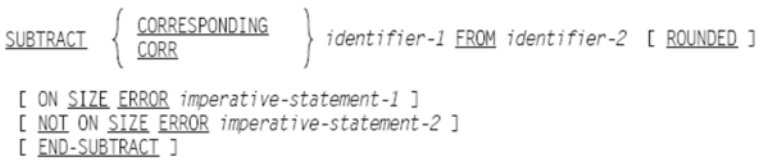
Format 1



Format 2



Format 3



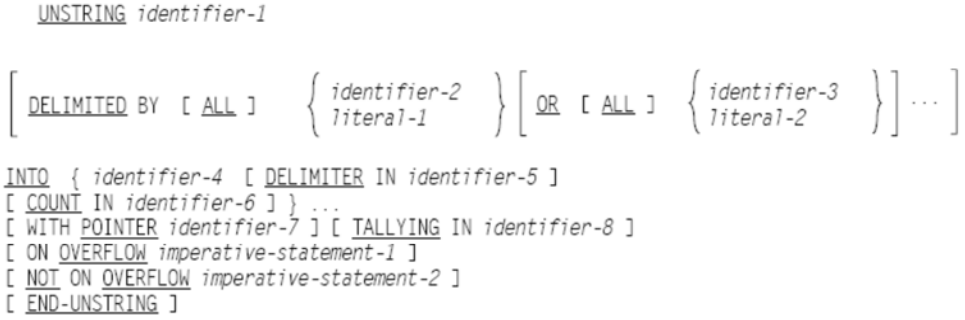
B.5.39. The SUPPRESS Statement

```
SUPPRESS PRINTING
```

B.5.40. The TERMINATE Statement

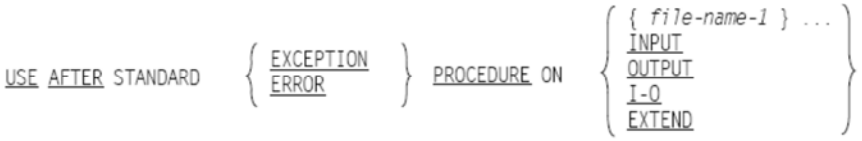
```
TERMINATE {report-name-1}...
```

B.5.41. The UNSTRING Statement

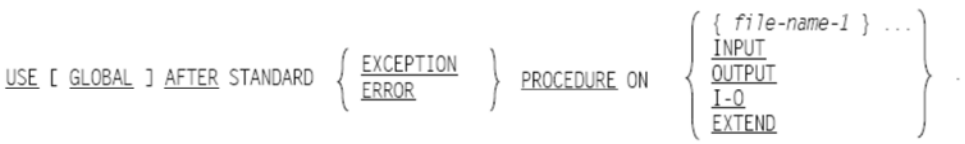


B.5.42. The USE Statement

Sequential, Relative, and Indexed I–O Format



Interprogram Communication Format



Report Writer Format



B.5.43. The USE BEFORE REPORTING Statement

Report Writer Format

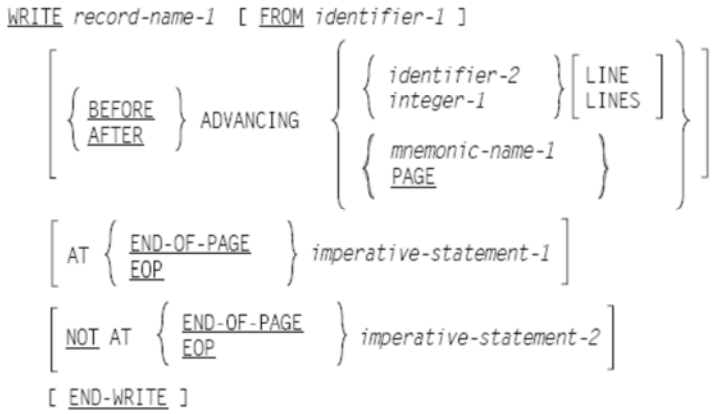
```
USE BEFORE REPORTING identifier-1.
```

Interprogram Communication Format

```
USE  [ GLOBAL ] BEFORE REPORTING identifier-1.
```

B.5.44. The WRITE Statement

Sequential I–O Format



Relative and Indexed I–O Format

```
WRITE record-name-1  [ FROM identifier-1 ]
  [ INVALID KEY imperative-statement-1 ]
  [ NOT INVALID KEY imperative-statement-2 ]
  [ END-WRITE ]
```

B.6. General Format for Nested Source Programs

```
  IDENTIFICATION DIVISION .
  PROGRAM-ID .  program-name-1  [ IS INITIAL PROGRAM ] .
[ ENVIRONMENT DIVISION .  environment-division-content ]
[ DATA DIVISION .  data-division-content ]
[ PROCEDURE DIVISION .  procedure-division-content ]
[ [ nested-source-program ] ...
  END PROGRAM program-name-1 . ]
```

B.7. General Format for Nested–Source–Program

```
IDENTIFICATION DIVISION .

PROGRAM-ID . program-name-2    [ IS { COMMON | INITIAL } PROGRAM ] .

[ ENVIRONMENT DIVISION . environment-division-content ]
[ DATA DIVISION . data-division-content ]
[ PROCEDURE DIVISION . procedure-division-content ]
  [ nested-source-program ] ...
  END PROGRAM program-name-2 .
```

B.8. General Format for a Sequence of Source Programs

```
{ IDENTIFICATION DIVISION .
  PROGRAM-ID . program-name-3 [ IS INITIAL PROGRAM ] .
[ ENVIRONMENT DIVISION . environment-division-content ]
[ DATA DIVISION . data-division-content ]
[ PROCEDURE DIVISION . procedure-division-content ]
[ nested-source-program ] ...
  END PROGRAM program-name-3 . } ...
  IDENTIFICATION DIVISION .
  PROGRAM-ID . program-name-4 [ IS INITIAL PROGRAM ] .
[ ENVIRONMENT DIVISION . environment-division-content ]
[ DATA DIVISION . data-division-content ]
[ PROCEDURE DIVISION . procedure-division-content ]
[ [ nested-source-program ] ...
  END PROGRAM program-name-4 . ]
```

B.9. Conditions

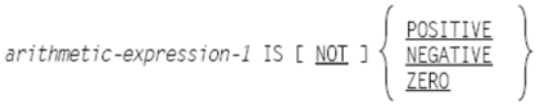
B.9.1. Relation Condition

```
{ identifier-1                  }   { IS [ NOT ] GREATER THAN                }   { identifier-2                  }
{ literal-1                     }   { IS [ NOT ] >                           }   { literal-2                     }
{ arithmetic-expression-1       }   { IS [ NOT ] LESS THAN >                 }   { arithmetic-expression-2       }
{ index-name-1                  }   { IS [ NOT ] <                           }   { index-name-2                  }
                                    { IS [ NOT ] EQUAL TO                    }
                                    { IS [ NOT ] =                           }
                                    { IS GREATER THAN OR EQUAL TO IS >=      }
                                    { IS LESS THAN OR EQUAL TO               }
                                    { IS <=                                  }
```

B.9.2. Class Condition

```
identifier IS [ NOT ]   { NUMERIC ALPHABETIC   }
                        { ALPHABETIC-LOWER     }
                        { ALPHABETIC-UPPER     }
                        { class-name           }
```

B.9.3. Sign Condition



B.9.4. *condition–name* Condition

```
condition-name-1
```

B.9.5. Switch–Status Condition

```
condition-name
```

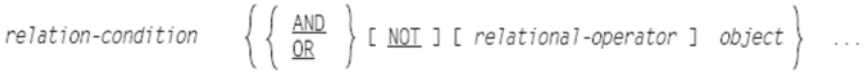
B.9.6. Negated Condition



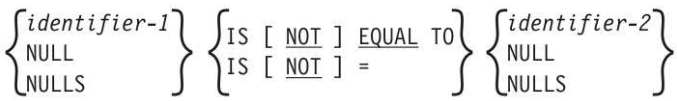
B.9.7. Combined Condition



B.9.8. Abbreviated Combined Relation Condition



B.9.9. Data Pointer Relation Condition

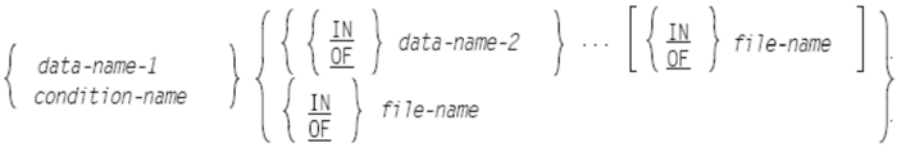


006887

B.10. Miscellaneous Formats

B.10.1. Qualification

Format 1



Format 2

```
paragraph-name  { IN | OF }  section-name
```

Format 3

```
text-name  { IN | OF }  library-name
```

Format 4

```
LINAGE-COUNTER  { IN | OF }  file-name
```

Format 5

```
{ PAGE-COUNTER | LINE=COUNTER }  { IN | OF }  report-name
```

Format 6

```
{ data-name-3 }  { { { IN | OF } data-name-4 } ... [ { IN | OF } report-name ]. }
                 { { IN | OF } report-name                                      }
```

B.10.2. Subscripting

```
{ condition-name-1 | data-name-1 }  ( { ALL                                  } ... )
                                     { integer-1                             }
                                     { data-name-2 [ { + } integer-2 ]       }
                                     { index-name-1 [ { + } integer-3 ]      }
                                     { arithmetic-expression                 }
```

B.10.3. Reference Modification

```
{ data-name-1                                    }
{ FUNCTION function-name-1 [ ( { argument-1 ... } ) ] }

( leftmost-character-position : [ length ] )
```

B.10.4. Identifier

Format 1

```
function-identifier-1
```

Format 2

```
data-name-1  [ { IN } data-name-2 ] ... [ { IN } { file-name-1   } ]
               { OF }                    { OF } { report-name-1 }

[ ( { subscript } ... ) ] [ leftmost-character-position : [ length ] ) ]
```

B.10.5. Function Identifier

```
FUNCTION function-name-1 [ ( { argument-1 } ... ) ]
[ reference-modifier ]
```

B.11. The COPY Statement

```
COPY text-name-1  [ { OF } library-name-1 ]
                  { IN }

[ REPLACING { ==pseudo-text-1 ==  } BY { ==pseudo-text-2 ==  } } ] ...
            { identifier-1        }     { identifier-2       }
            { literal-1           }     { literal-2          }
            { word-1              }     { word-2             }
```

B.12. The REPLACE Statement

Format 1

```
REPLACE { ==pseudo-text-1== BY ==pseudo-text-2== } ...
```

Format 2

```
REPLACE OFF
```

B.13. Function Format

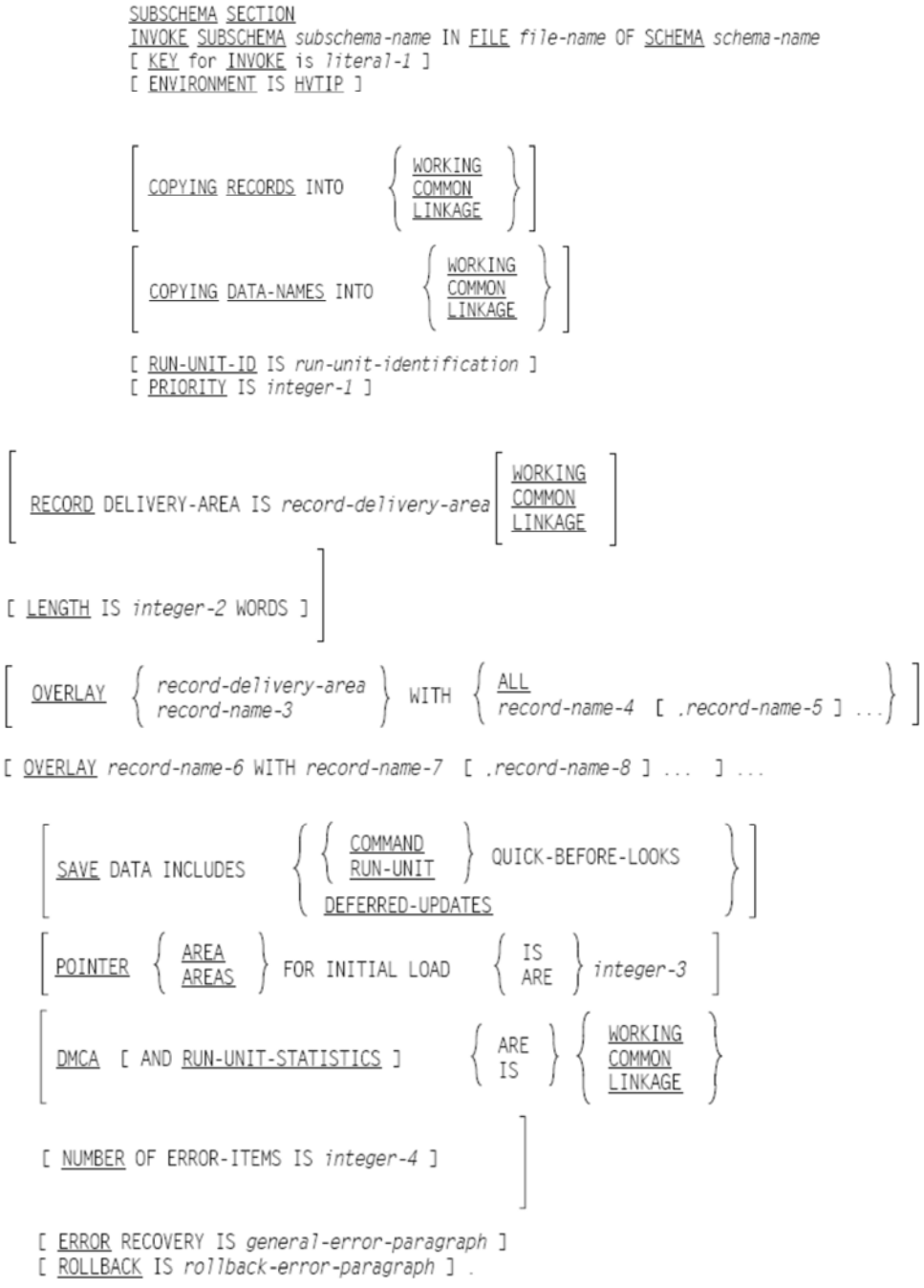
```
FUNCTION ACOS  ( argument-1 )
FUNCTION ASIN  ( argument-1 )
FUNCTION ANNUITY  ( argument-1 argument-2 )
FUNCTION ATAN  ( argument-1 )
FUNCTION CHAR  ( argument-1 )
FUNCTION COS  ( argument-1 )
FUNCTION CURRENT-DATE
FUNCTION DATE-OF-INTEGER  ( argument-1 )
FUNCTION DAY-OF-INTEGER  ( argument-1 )
FUNCTION FACTORIAL  ( argument-1 )
FUNCTION INTEGER ( argument-1 )
FUNCTION INTEGER-OF-DATE  ( argument-1 )
FUNCTION INTEGER-OF-DAY  ( argument-1 )
FUNCTION INTEGER-PART  ( argument-1 )
FUNCTION LENGTH  ( argument-1 )
FUNCTION LOG  ( argument-1 )
FUNCTION LOG10  ( argument-1 )
FUNCTION LOWER-CASE  ( argument-1 )
FUNCTION MAX  ( { argument-1 } ...  )
FUNCTION MEAN  ( { argument-1 } ...  )
FUNCTION MEDIAN  ( { argument-1 } ...  )
FUNCTION MIDRANGE  ( { argument-1 } ...  )
FUNCTION MIN  ( { argument-1 } ...  )
FUNCTION MOD  ( argument-1 argument-2 )
FUNCTION NUMVAL  ( argument-1 )
FUNCTION NUMVAL-C  ( argument-1  [ argument-2 ] )
FUNCTION ORD  ( argument-1 )
FUNCTION ORD-MAX  ( { argument-1 } ...  )
FUNCTION ORD-MIN  ( { argument-1 } ...  )
FUNCTION PRESENT-VALUE  ( argument-1  { argument-2 } ...  )
FUNCTION RANDOM  [ ( argument-1 ) ]
FUNCTION RANGE  ( { argument-1 } ...  )
FUNCTION REM  ( argument-1 argument-2 )
FUNCTION REVERSE  ( argument-1 )
FUNCTION SIN  ( argument-1 )
FUNCTION SQRT  ( argument-1 )
FUNCTION STANDARD-DEVIATION  ( { argument-1 } ...  )
FUNCTION SUM  ( { argument-1 } ...  )
FUNCTION TAN  ( argument-1 )
FUNCTION UPPER-CASE  ( argument-1 )
FUNCTION VARIANCE  ( { argument-1 } ...  )
FUNCTION WHEN-COMPILED
```

B.14. DMS 2200 Skeletons

The following subsections show the DMS 2200 statement entry skeletons for data and procedure divisions.

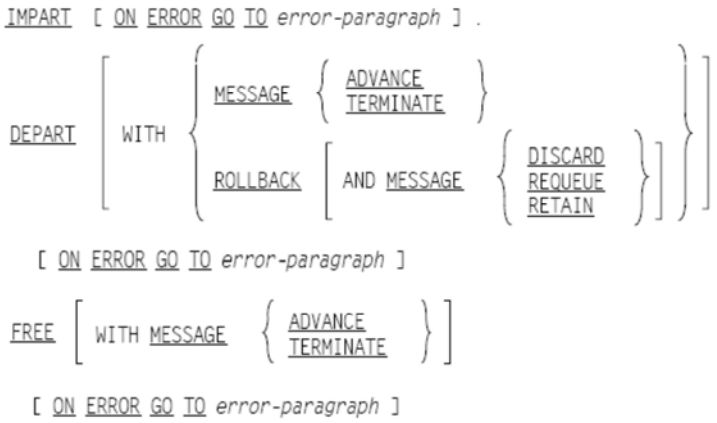
B.14.1. Complete Entry Skeleton for Statements in the Data-Division

General Format

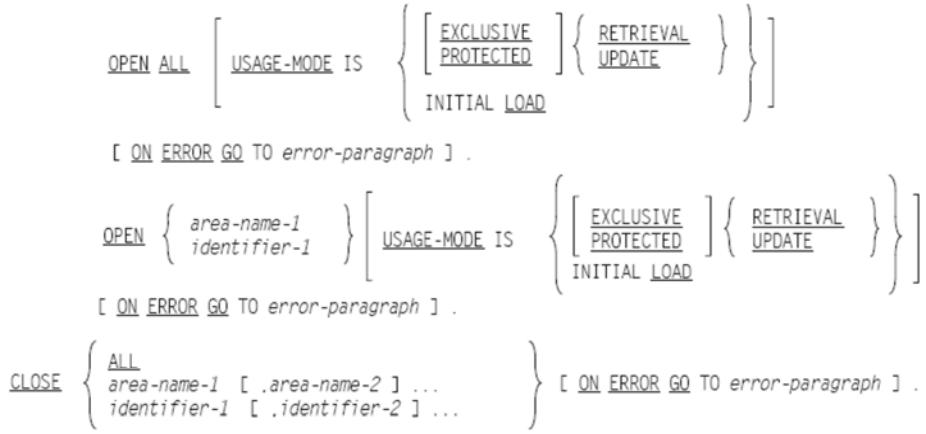


B.14.2. Complete Entry Skeleton for Statements in the Procedure Division

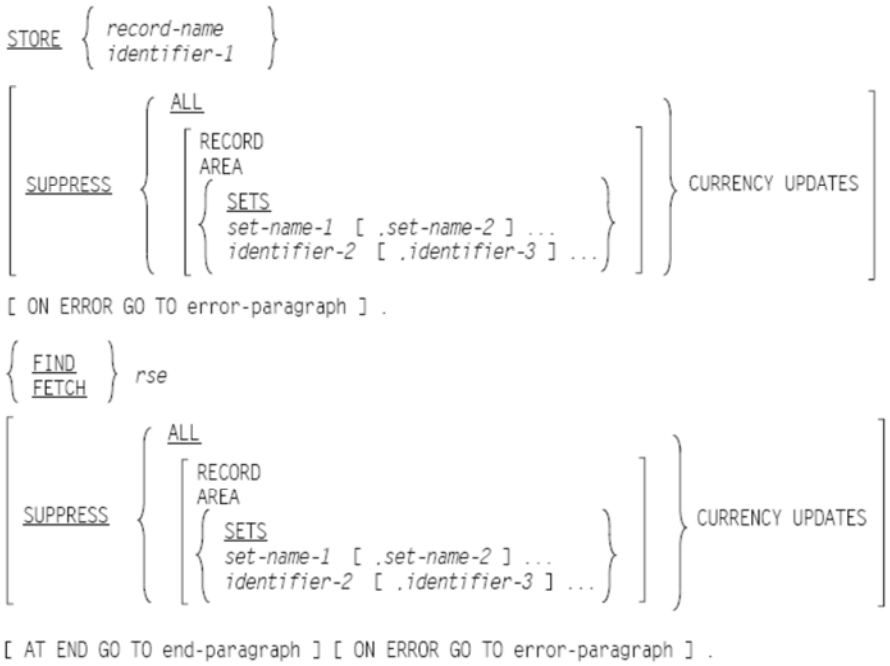
Run–Unit Oriented Commands



Area–Oriented Commands

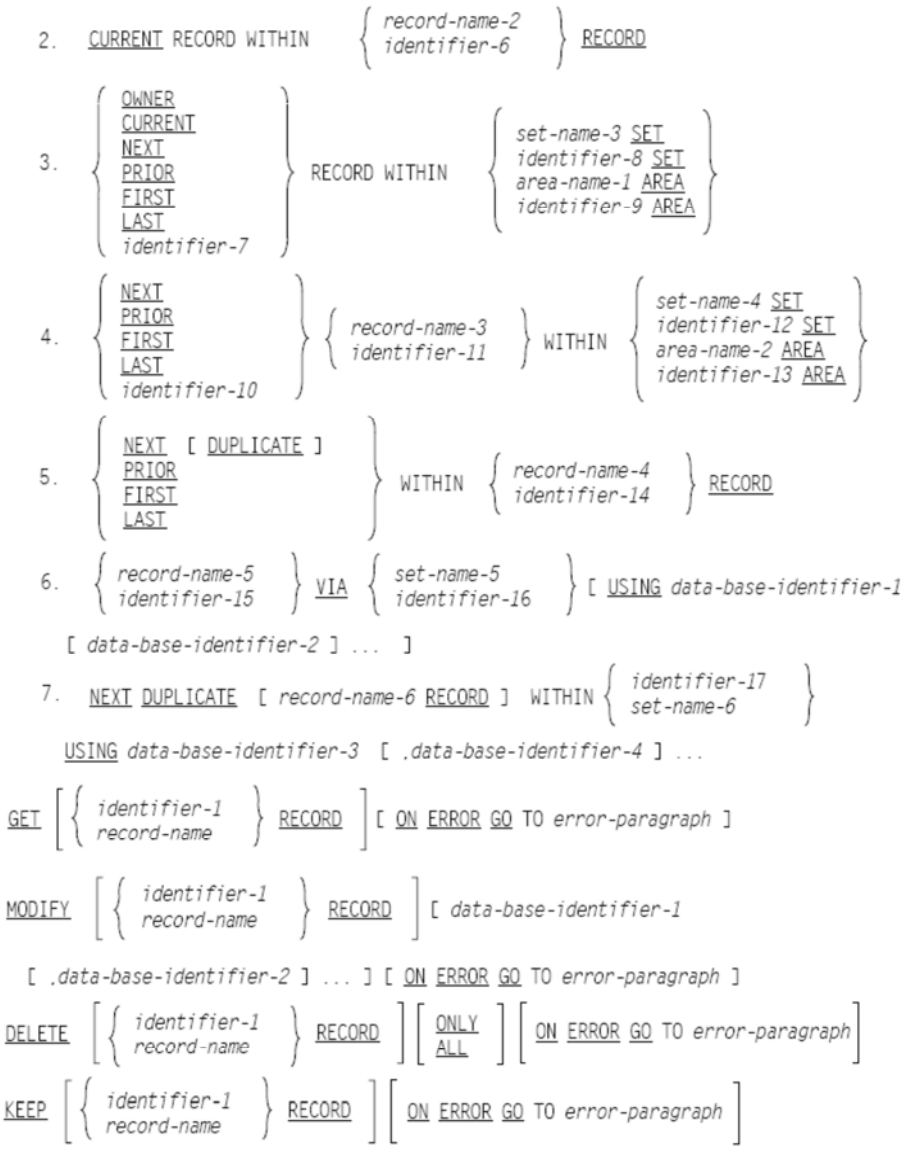


Record–Oriented Commands

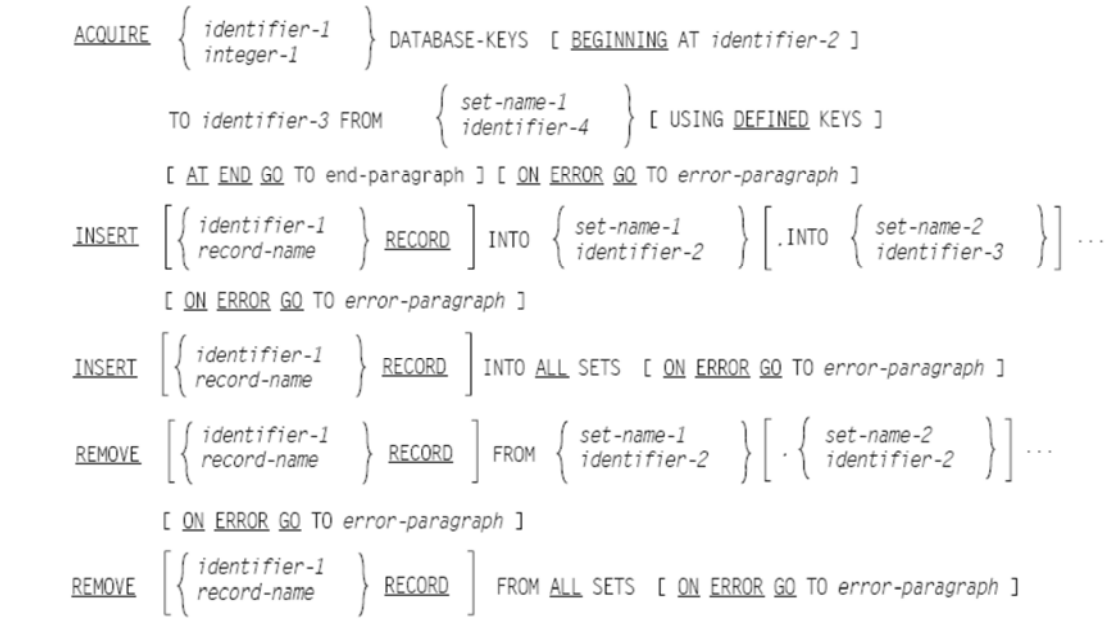


where the record selection expression (rse) is one of the following:

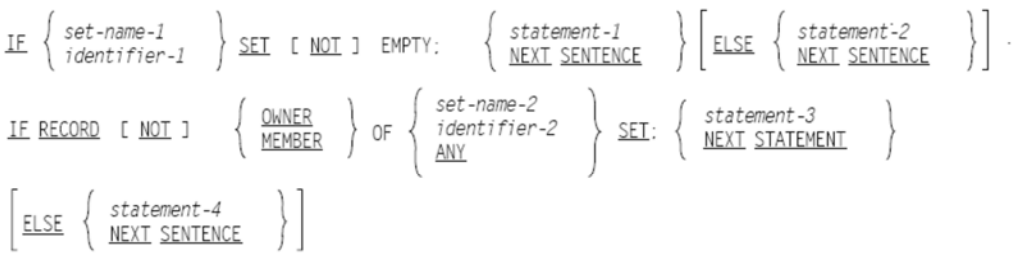



2. CURRENT RECORD WITHIN record-name-2 identifier-6 RECORD
3. OWNER CURRENT NEXT PRIOR FIRST LAST identifier-7 RECORD WITHIN set-name-3 SET identifier-8 SET area-name-1 AREA identifier-9 AREA
4. NEXT PRIOR FIRST LAST identifier-10 record-name-3 identifier-11 WITHIN set-name-4 SET identifier-12 SET area-name-2 AREA identifier-13 AREA
5. NEXT [DUPLICATE] PRIOR FIRST LAST WITHIN record-name-4 identifier-14 RECORD
6. record-name-5 identifier-15 VIA set-name-5 identifier-16 [USING data-base-identifier-1 [data-base-identifier-2] ...]
7. NEXT DUPLICATE [record-name-6 RECORD] WITHIN identifier-17 set-name-6 USING data-base-identifier-3 [,data-base-identifier-4] ...
GET [identifier-1 record-name RECORD] [ON ERROR GO TO error-paragraph]
MODIFY [identifier-1 record-name RECORD] [data-base-identifier-1 [,data-base-identifier-2] ...] [ON ERROR GO TO error-paragraph]
DELETE [identifier-1 record-name RECORD] [ONLY ALL] [ON ERROR GO TO error-paragraph]
KEEP [identifier-1 record-name RECORD] [ON ERROR GO TO error-paragraph]

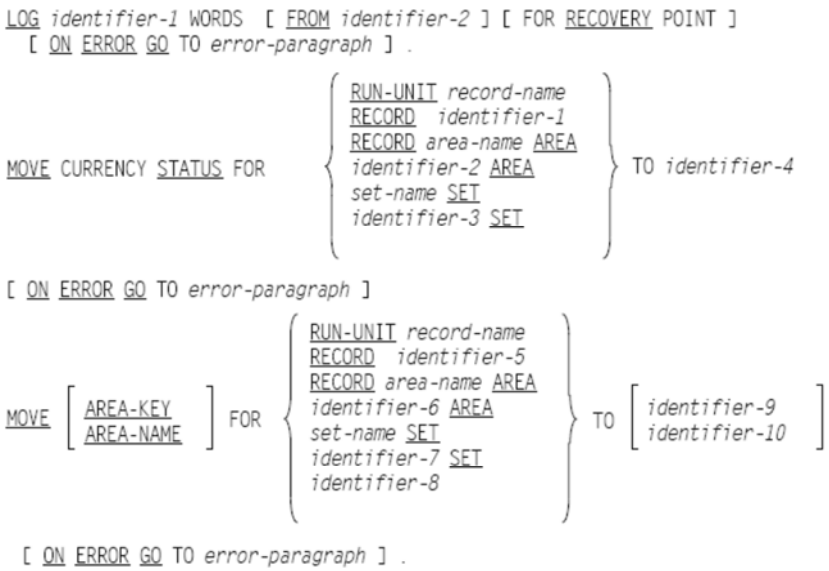
Set–Oriented Commands



Conditional Commands



Supporting Commands



B.15. RDMS 2200 Skeletons

The following subsections show the embedded SQL skeletons and the ENTER MASM skeleton.

B.15.1. Embedded SQL Skeletons

Data Division

```
    EXEC SQL BEGIN DECLARE SECTION END-EXEC.
01 sql-variable data-description.
    EXEC SQL END DECLARE SECTION ENDûEXEC.
```

Procedure Division

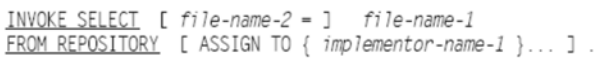
```
EXEC SQL sql-command
[ :sql-variable-1
      .
      .
      .
  :sql-variable-n ]
END-EXEC.
```

B.15.2. ENTER MASM Skeleton

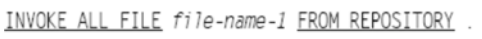
```
ENTER MASM 'ACOB$RDMR'
  USING sql-command
    error-status
    auxiliary-information
  [ :sql-variable-1
    :sql-variable-2
                .
                .
                .
    :sql-variable-n ]
```

B.16. UREP Skeletons

B.16.1. Inserting SELECT Statement Information with INVOKE

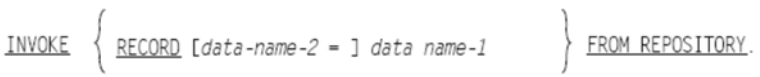


B.16.2. Inserting File Descriptions with INVOKE

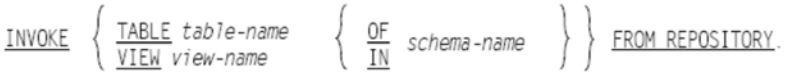


B.16.3. Inserting Data Descriptions with INVOKE

Inserting COBOL Data Item Definitions



Inserting Relational Data Definitions



Inserting Screen Definitions



Index

A

abbreviated combined relation conditions, 3-87
absolute line number, 10-37
ACCEPT statement, 3-103
ACCESS MODE clause, 6-9
  DYNAMIC, 6-9
  RANDOM, 6-9
  SEQUENTIAL, 4-5, 4-8, 6-9
access modes, 4-1, 5-2, 6-2
ACOS (arc cosine) function, 12-9
ADD statement, 3-106
  composite of operands, 3-97
  COMPUTE statement, 3-108
  data conversion, 3-97
  decimal alignment, 3-97
  maximum operand size, 3-97
  multiple results, 3-98
ADVANCING phrase, 4-45
AFTER phrase
  INSPECT statement, 3-126
  PERFORM statement, 3-139
  WRITE statement, 4-44
algebraic sign, 2-25
alignment of data, 2-26
  MOVE statement, 3-134
alignment rules, 2-26
ALL
  INSPECT statement, 3-126
  SEARCH statement, 3-149
ALL literal
  DISPLAY statement, 3-109
  INSPECT statement, 3-126
  STOP statement, 3-157
  STRING statement, 3-158
  UNSTRING statement, 3-164
ALPHABET clause, 3-9
  affecting CHAR function, 12-11
alphabet name, 2-14
  MERGE statement, 8-6
alphanumeric edited category, 3-26, 3-135
alphanumeric functions, 12-5
ALSO phrase, 3-9
ALTER statement
  GO TO statement, 3-120
  segmentation, 11-6
ALTERNATE RECORD KEY clause, 6-8, 6-10
  Indexed I<#106>O module, 6-10
AND, 3-85
  abbreviated combined relation, 3-87
  combined condition, 3-86
  hierarchy, 3-88
  SEARCH statement, 3-149
annuity (ANNUITY function), 12-9
ANNUITY (annuity immediate) function, 12-9
ANS74<#106>PRINTER
  ASSIGN clause, 4-6
arc cosine (ACOS function), 12-9
arc sine (ASIN function), 12-10
arc tangent (ATAN function), 12-11
arguments
  using with intrinsic functions, 12-3
arithmetic expression, 3-70
  COMPUTE statement, 3-108
  relation condition, 3-75
  sign condition, 3-84
arithmetic operator, 3-71
arithmetic statements, 3-97
ASCENDING KEY phrase
  MERGE statement, 8-8
  OCCURS clause, 3-21
  SEARCH statement, 3-151
  SORT statement, 8-14
ASIN (arc sine) function, 12-10
ASSIGN clause, 8-2
  ANS74<#106>PRINTER, 4-6
  CARD<#106>PUNCH, 4-6
  CARD<#106>READER, 4-6
  DISC, 4-6, 6-8
  Indexed I<#106>O module, 6-8
  PRINTER, 4-6
  Relative I<#106>O module, 5-7
  Sequential I<#106>O module, 4-5
  TAPE, 4-6
assumed decimal point, 3-27
AT END condition, 4-4
  READ statement, 4-39

RETURN statement, 8-13
USE statement, 4-42, 5-27, 6-32
AT END phrase
READ statement, 5-19, 6-23
RETURN statement, 8-13
SEARCH statement, 3-149
USE statement, 4-42, 5-27, 6-32
ATAN (arc tangent) function, 12-11
attributes
explicit, 2-35
implicit, 2-35

B

BEFORE phrase
INSPECT statement, 3-126
WRITE statement, 4-44
BINARY, 3-47
binary arithmetic operators, 3-71
BINARY<#106>1, 3-47
BINARY<#106>2, 3-47
blank line, 2-46
BLANK WHEN ZERO clause, 3-18
PICTURE clause, 3-25
VALUE clause, 3-54
braces, 2-10
brackets, 2-10
BY
DIVIDE statement, 3-112
INSPECT statement, 3-125
MULTIPLY statement, 3-138
PERFORM statement, 3-139

C

CALL statement, 7-7, 7-30
CANCEL statement, 7-34
Linkage Section, 7-18
MERGE statement, 8-10
Procedure Division header, 7-29
CANCEL statement, 7-34
CALL statement, 7-31
CARD<#106>PUNCH
ASSIGN clause, 4-6
CARD<#106>READER
ASSIGN clause, 4-6
category alphabetic, 3-25, 3-41, 3-135
category numeric, 3-135
category of data
editing, 3-31
MOVE statement, 3-135
PICTURE clause, 3-25, 3-41
VALUE clause, 3-54
CF (CONTROL FOOTING), 10-45
CH (CONTROL HEADING), 10-45
CHAR function, 12-11
character set, 2-12
character strings
description, 2-13
PICTURE, 2-22
CHARACTERS
RECORD CONTAINS clause, 4-25
class alphabetic, 3-80
class condition, 3-80
class numeric, 3-80
classes of data, 2-25
concept, 2-25
CLOSE statement
I<#106>O status, 4-2, 5-2, 6-2
Indexed I<#106>O module, 6-17
OPEN statement, 5-17, 6-21
READ statement, 6-25
Relative I<#106>O module, 5-13
Report Writer module, 10-51
Sequential I<#106>O module, 4-29
TERMINATE Statement, 10-59
COBOL 85, 1-3
COBOL source program, 2-37, 3-2
COBOL word, 2-13
COBOL X3.23-1985, 1-1
CODE clause, 10-12
collating sequence
affecting CHAR function, 12-11
specified by ALPHABET clause, 12-11
COLLATING SEQUENCE clause, 3-8
COLLATING SEQUENCE phrase
MERGE statement, 8-9
SORT statement, 8-14
COLUMN NUMBER clause, 10-35
combined condition, 3-86
comma
DECIMAL<#106>POINT IS COMMA clause, 3-9
library text word, 9-3
PICTURE symbol, 3-31
comment entry, 2-22
DATE COMPILED paragraph, 3-6
comment lines, 2-46
library [text], 9-3
Common, 3-20, 7-4
COMMON, 3-5
COMMON clause, 7-6
Common<#106>Storage Section, 7-20

Communication, Interprogram, 7-7
COMP, 3-47
COMP<#106>1, 3-47
COMP<#106>2, 3-47
comparison of nonnumeric operands, 3-76
comparison of numeric operands, 3-76
compiler directing statement, 2-42
complex condition, 3-85
COMPUTATIONAL (COMP), 3-47
COMPUTATIONAL<#106>1
(COMP<#106>1), 3-47
COMPUTATIONAL<#106>2
(COMP<#106>2), 3-47
COMPUTE statement, 3-108
data conversion, 3-97
decimal alignment, 3-97
maximum operand size, 3-97
multiple results, 3-98
computer name, 3-7, 3-8
condition, 3-74
abbreviated combined relation condition, 3-87
class condition, 3-80
combined condition, 3-86
complex condition, 3-85
condition<#106>name condition, 3-82
IF statement, 3-121
negated combined condition, 3-86
negated simple condition, 3-85
PERFORM UNTIL statement, 3-139
relation condition, 3-75
SEARCH statement, 3-149
sign condition, 3-84
simple condition, 3-75
SIZE ERROR condition, 3-93
switch<#106>status condition, 3-83
condition name, 2-33
data description entry, 3-15, 3-54
level number, 3-20
SPECIAL<#106>NAMES paragraph, 3-9
VALUE clause, 3-53
condition<#106>name condition, 3-82
conditional
expression, 3-74
statement, 2-42
variable, 3-82
Configuration section, 3-6
connectives, 2-17
logical, 3-85
continuation of lines, 2-46
comment entries, 3-6
library pseudo<#106>[text], 9-2
CONTINUE statement, 3-109
control
transfers, 2-34
control break
CONTROL clause, 10-12
GENERATE statement, 10-54
GROUP INDICATE clause, 10-36
TYPE clause, 10-45
CONTROL clause, 10-12
CONTROL FOOTING (CF), 10-45
CONTROL HEADING (CH), 10-17, 10-45
COPY statement, 9-1
CORR, 3-162
CORRESPONDING (CORR) phrase
MOVE statement, 3-134
SUBTRACT statement, 3-162
COS (cosine) function, 12-11
cross<#106>footing, 10-43
CURRENCY
PICTURE symbol, 3-31
currency sign, 3-31
CURRENCY SIGN clause, 3-9, 3-31
currency symbol, 3-31
current record pointer
Indexed I<#106>O module, 6-2
Relative I<#106>O module, 5-2
REWRITE statement, 4-41, 6-27
Sequential I<#106>O module, 4-1
START statement, 5-25
current volume pointer, 4-2
CURRENT<#106>DATE function, 12-12

D

data description, 3-15
Interprogram Communication module, 7-24
data description entry, 3-15
Linkage Section, 7-18
Working<#106>Storage Section, 3-14
Data Division, 2-39
entries, 2-47
Indexed I<#106>O module, 6-14
Interprogram Communication module, 7-18
Nucleus, 3-14
Relative I<#106>O module, 5-10
Report Writer module, 10-7
Sequential I<#106>O module, 4-16
Sort<#106>Merge module, 8-5
structure, 2-40
data name, 2-14, 3-18

clause, 10-35
DATA RECORDS clause
Sequential I<#106>O module, 4-21
date and time compiled
(WHEN<#106>COMPILED function), 12-34
date, current (CURRENT<#106>DATE function), 12-12
date, of integer
(DATE<#106>OF<#106>INTEGER function), 12-13
DATE<#106>COMPILED paragraph, 3-6
DATE<#106>OF<#106>INTEGER function, 12-13
DAY, 3-103
day, of integer
(DAY<#106>OF<#106>INTEGER function), 12-14
DAY<#106>OF<#106>INTEGER function, 12-14
DB PICTURE symbol, 3-31
DE (DETAIL), 10-45
debugging
compile time switch, 3-170
in the Nucleus, 3-169
lines, 3-170
decimal point
actual, 3-31
assumed, 3-30
DECIMAL<#106>POINT IS COMMA clause, 3-9
declaratives, 2-40
USE BEFORE REPORTING statement, 10-62
USE statement, 4-42, 5-26, 6-31
DECLARATIVES keyword, 2-48
DELETE statement
Indexed I<#106>O module, 6-18
OPEN mode, 5-16, 6-20
Relative I<#106>O module, 5-14
DEPENDING phrase
GO TO statement, 3-120
OCCURS clause, 3-21
DESCENDING KEY phrase
MERGE statement, 8-9
OCCURS clause, 3-21
SORT statement, 8-14
DETAIL (DE), 10-45
detail report, 10-54
DISC
ASSIGN clause, 4-6, 6-8
DISPLAY statement, 3-109
DIVIDE statement, 3-112
composite of operands, 3-97
COMPUTE statement, 3-108
data conversion, 3-97
decimal alignment, 3-97
maximum operand size, 3-97
multiple results, 3-98
SIZE ERROR phrase, 3-93
division header, 2-46
DOWN BY, 3-153
DUPLICATES phrase, 6-8

E

editing rules, 3-31
elementary item
noncontiguous, 3-15
elements, 2-10
ellipsis, 2-11
end, 2-48
END DECLARATIVES keywords, 2-48
end program header, 3-4
Interprogram Communication module, 7-15
END<#106>OF<#106>PAGE phrase, 4-45
ENTER statement, 7-35
ENTRY POINTS, 7-29
Environment Division, 2-38
Indexed I<#106>O module, 6-7
Nucleus, 3-6
Relative I<#106>O module, 5-6
Report Writer module, 10-5
Sequential I<#106>O module, 4-5
Sort<#106>Merge module, 8-2
structure, 2-39
EOP phrase, 4-44
EQUAL TO relation, 3-75
error messages
from a SORT or MERGE, 8-21
EVALUATE statement, 3-115
exact binary, 3-26
EXIT PROGRAM statement, 7-39
CANCEL statement, 7-34
EXIT statement, 3-119
MERGE statement, 8-10
explicit attributes, 2-35
exponentiation, 3-71
expressions
conditional, 3-74
EXTEND phrase
OPEN statement, 6-19
external clause, 7-27

external switch, 2-36

F

FACTORIAL function, 12-15
FD level indicator
  Indexed I<#106>O module, 6-15
  Relative I<#106>O module, 5-11
  Report Writer module, 10-8
  Sequential I<#106>O module, 4-18
figurative constant, 2-17, 2-20
  DISPLAY statement, 3-110
  INSPECT statement, 3-127
  STOP statement, 3-157
  STRING statement, 3-158, 3-159
  UNSTRING statement, 3-164
  VALUE clause, 3-54
file
  conceptual characteristics, 2-23
file attributes, 2-1
file control entry
  Indexed I<#106>O module, 6-7
  Relative I<#106>O module, 5-7
  Report Writer module, 10-6
  Sequential I<#106>O module, 4-5
  Sort<#106>Merge module, 8-2
file description entry
  Indexed I<#106>O module, 6-15
  Interprogram Communication module, 7-21
  Relative I<#106>O module, 5-11
  Report Writer module, 10-8
  Sequential, 4-17
  Sort<#106>Merge module, 8-5
file name, 2-14
file position indicator, 4-2
  DELETE statement, 5-15
  OPEN statement, 5-17
  REWRITE statement, 5-24
  WRITE statement, 6-34
File Section
  Indexed I<#106>O module, 6-14
  Relative I<#106>O module, 5-10
  Sequential I<#106>O module, 4-16
  Sort<#106>Merge module, 8-5
FILE STATUS clause
  Indexed I<#106>O module, 6-8
  Relative I<#106>O module, 5-7
FILE STATUS data item
  indexed I<#106>O module, 6-2
  Relative I<#106>O module, 5-2
FILE<#106>CONTROL paragraph
  Indexed I<#106>O module, 6-7
  Relative I<#106>O module, 5-6
  Report Writer module, 10-6
  Sequential I<#106>O module, 4-5
  Sort<#106>Merge module, 8-2
files, 2-1
FILLER clause, 3-18
FINAL
  CONTROL clause, 10-12
  SUM clause, 10-42
  TYPE clause, 10-45
FIPS PUB 21-2, 1-1
FIRST DETAIL, 10-13
fixed insertion editing, 3-33
floating insertion editing, 3-32
FOOTING, 10-13, 10-45
FOR, 3-126
format punctuation, 2-11
FROM phrase
  ACCEPT statement, 3-103
  PERFORM varying statement, 3-139
  RELEASE statement, 8-12
  REWRITE statement, 4-41, 5-23, 6-27
  SUBTRACT statement, 3-162
  WRITE statement, 4-45, 5-28, 6-33
function identifier, 2-30
function name, 2-17
functions, intrinsic, 12-1
  ALL subscript with, 12-4
  alphanumeric, 12-5
  arguments
    defined, 12-3
    types of, 12-4
  function definitions, 12-9
    ACOS function, 12-9
    ANNUITY function, 12-9
    ASIN function, 12-10
    ATAN function, 12-11
    CHAR function, 12-11
    COS function, 12-11
    CURRENT<#106>DATE function, 12-12
    DATE<#106>OF<#106>INTEGER function, 12-13
    DAY<#106>OF<#106>INTEGER function, 12-14
    FACTORIAL function, 12-15
    INTEGER function, 12-15
    INTEGER<#106>OF<#106>DATE function, 12-16
    INTEGER<#106>OF<#106>DAY function, 12-16
    INTEGER<#106>PART function, 12-17

LENGTH function, 12-18
LOG 10 function, 12-19
LOG function, 12-19
LOWER<#106>CASE function, 12-19
MAX function, 12-20
MEAN function, 12-20
MEDIAN function, 12-21
MIDRANGE function, 12-21
MIN function, 12-22
MOD function, 12-23
NUMVAL function, 12-23
NUMVAL<#106>C function, 12-24
ORD function, 12-25
ORD<#106>MAX function, 12-26
ORD<#106>MIN function, 12-26
PRESENT<#106>VALUE function, 12-27
RANDOM function, 12-27
RANGE function, 12-28
REM function, 12-29
REVERSE function, 12-29
SIN function, 12-30
SQRT function, 12-30
STANDARD<#106>DEVIATION function, 12-31
SUM function, 12-32
TAN function, 12-32
UPPER<#106>CASE function, 12-33
VARIANCE function, 12-33
WHEN<#106>COMPILED function, 12-34
function identifiers, defined, 12-3
function names, defined, 12-3
integer, 12-6
language concepts, overview, 12-3
numeric, 12-5
returned values, defined, 12-3
supported by UCS COBOL, 12-6
types of, 12-5
undefined values for, 12-3, 12-4, 12-5
using tables with, 12-4

G

general rules, 2-12
GENERATE, 10-4
GENERATE statement, 10-54
CONTROL clause, 10-13
SUM clause, 10-43
TERMINATE statement, 10-59
GIVING phrase
DIVIDE statement, 3-112
MERGE statement, 8-8
MULTIPLY statement, 3-138
SORT statement, 8-14
SUBTRACT statement, 3-162
GLOBAL clause, 1-6, 7-28
global names
Interprogram Communication module, 7-3
GO TO statement, 3-120
MERGE statement, 8-10
PERFORM statement, 3-142
segmentation, 11-6
GREATER THAN relation, 3-75
GROUP INDICATE clause, 10-36

H

header
division, 2-46
paragraph, 2-47
section, 2-47
HEADING, 10-13, 10-45
HIGH<#106>VALUE/HIGH<#106>VALUES
restriction, 3-2

I

I<#106>O status
Indexed I<#106>O module, 6-2
Relative I<#106>O module, 5-2
Sequential I<#106>O module, 4-2
I<#106>O<#106>CONTROL, 6-14
I<#106>O<#106>CONTROL paragraph
Indexed I<#106>O module, 6-14
Relative I<#106>O module, 5-10
Sequential I<#106>O module, 4-13
Sort<#106>Merge module, 8-3
Identification Division, 2-37, 3-4
Interprogram Communication module, 7-16
organization, 3-4
structure, 2-38
identifier, 2-33
IF statement, 3-121
imperative sentence, 2-43
imperative statement, 2-42
implementor name
SPECIAL<#106>NAMES paragraph, 3-9
implicit attributes, 2-35
implied relational operator, 3-87
INDEX, 3-47

index data item
CONTROL clause, 10-13
initial value, 3-15
MOVE statement, 3-134
index name, 2-14
OCCURS clause, 3-21
PERFORM statement, 3-139
SEARCH statement, 3-149
SET statement, 3-153
indexed file, 6-1
indexed I<#106>O
statistics, 6-7
Indexed I<#106>O module, 6-1
INDEXED phrase, 3-21
indexing
CONTROL clause, 10-12
MOVE statement, 3-134
RETURN statement, 8-13
initial values, 3-15, 7-19
INITIALIZE statement, 3-123
INITIATE, 10-4
INITIATE statement, 10-55
SUM clause, 10-44
USE BEFORE REPORTING statement, 10-62
Input<#106>Output Section
Environment Division, 10-5
Indexed I<#106>O module, 6-7
Relative I<#106>O module, 5-6
Report Writer module, 10-5
Sequential I<#106>O module, 4-5
Sort<#106>Merge module, 8-2
INSPECT statement, 3-125
INTEGER function, 12-15
integer functions, 12-6
INTEGER<#106>OF<#106>DATE function, 12-16
INTEGER<#106>OF<#106>DAY function, 12-16
INTEGER<#106>PART function, 12-17
Interprogram Communication description, 7-7
Interprogram Communication module, 7-1
INTO
DIVIDE statement, 3-112
READ statement, 4-38, 5-19, 6-23
RETURN statement, 8-13
STRING statement, 3-158
invalid key condition, 5-5, 6-6
DELETE statement, 5-14, 6-18
READ statement, 5-19, 6-23
REWRITE statement, 5-23, 6-27
START statement, 5-25, 6-29
WRITE statement, 5-28, 6-33
italics in this manual, 2-10

J

JUSTIFIED (JUST) clause, 3-19
VALUE clause, 3-55

K

KEY data names
SORT statement, 8-14
KEY phrase, 6-23
indexed I<#106>O alternate record key, 6-10
MERGE statement, 8-8
OCCURS clause, 3-21
READ statement, 6-23
Relative I<#106>O relative key, 5-7
SEARCH statement, 3-150
START statement, 6-30

L

LABEL RECORDS clause
Sequential I<#106>O module, 4-21
LAST DETAIL, 10-13
LEADING
INSPECT statement, 3-126
SIGN clause, 3-44
LEFT, 3-45
LENGTH function, 12-18
LESS THAN relation, 3-75
level characteristics, 3-1
level number, 2-10, 2-14, 2-24, 3-17, 3-20
level number clause, 10-36
levels
concept, 2-24
library name, 2-14, 9-2
LINAGE clause
Sequential I<#106>O module, 4-22
LINAGE<#106>COUNTER, 4-5, 4-23
line number
absolute, 10-37
relative, 10-37
LINE NUMBER clause, 10-37
notation, 10-22
sequence substitutions, 10-22
LINE<#106>COUNTER, 10-5, 10-11

Linkage Section, 7-18
  VALUE clause, 3-54
literal, 2-18
  figurative constant, 2-20
  nonnumeric, 2-18
  numeric, 2-19
  STOP statement, 3-157
local names
  Interprogram Communication module, 7-3
LOG (natural log) function, 12-19
LOG10 (base 10 log) function, 12-19
logical operator, 3-85
logical record, 2-2, 2-22
LOW<#106>VALUE/LOW<#106>VALUES
  restriction, 3-2
LOWER<#106>CASE function, 12-19

M

MAX (maximum) function, 12-20
maximum, ordinal number of
    (ORD<#106>MAX function), 12-26
MEAN function, 12-20
MEDIAN function, 12-21
merge file, 8-2
MERGE statement, 8-6
  segmentation, 11-7
MIDRANGE function, 12-21
MIN (minimum) function, 12-22
minimum, ordinal number of
    (ORD<#106>MIN function), 12-26
mnemonic name, 2-15
  ACCEPT statement, 3-103
  DISPLAY statement, 3-109
  SPECIAL<#106>NAMES paragraph, 3-9
  WRITE statement, 4-44
MOD (modulo) function, 12-23
MOVE statement, 3-134
  overlapping operands, 3-98
MULTIPLE FILE TAPE clause
  Sequential I<#106>O module, 4-14
multiple results in arithmetics, 3-98
MULTIPLY statement, 3-138
  composite of operands, 3-97
  compute statement, 3-108
  data conversion, 3-97
  maximum operand size, 3-97
  multiple results, 3-98
  SIZE ERROR phrase, 3-93

N

names
  system, 2-16
names of programs
  scope, 7-8
NATIVE phrase, 3-9
negated conditions, 3-85
negated simple condition, 3-86
NEXT GROUP clause, 10-39
NEXT PAGE phrase
  GROUP clause, 10-39
  LINE NUMBER clause, 10-37
NEXT phrase
  Indexed I<#106>O module, 6-23
  Relative I<#106>O module, 5-19
NEXT SENTENCE phrase
  SEARCH statement, 3-149
noncontiguous elementary item, 3-15
noncontiguous working storage, 3-15
nonnumeric comparison, 3-76
nonnumeric literal, 2-18
NOT
  logical operator, 3-85
Nucleus, 3-1
Nucleus module, 3-1
numeric
  functions, 12-5
numeric comparison, 3-76
numeric edited category, 3-26, 3-135
numeric literal, 2-19
NUMVAL function, 12-23
NUMVAL<#106>C function, 12-24

O

object types, 7-4
OBJECT<#106>COMPUTER paragraph, 3-8
OBJECT-COMPUTER paragraph, 2-38
OCCURS clause, 2-5, 3-21
  REDEFINES clause, 3-40
  SEARCH statement, 3-149
  SYNCHRONIZED Clause, 3-46
  VALUE clause, 3-53
OFF STATUS phrase, 3-9
ON SIZE ERROR phrase, 3-93
ON STATUS phrase, 3-9
OPEN statement, 4-33
  CLOSE statement, 6-17
  I<#106>O status, 4-1, 5-2, 6-2
  Indexed I<#106>O module, 6-19

INITIATE statement, 10-55
LINAGE clause, 4-23
READ statement, 5-19, 6-23
Relative I<#106>O module, 5-15
Report Writer module, 10-56
REWRITE statement, 5-23, 6-27
Sequential I<#106>O module, 4-33
START statement, 5-25, 6-29
WRITE statement, 5-28, 6-33
operands, 3-97
operator
logical, 3-85
relational, 3-75
OPTIONAL phrase, 4-5
optional words, 2-17
OR logical connective, 3-85
abbreviated combined relation, 3-87
hierarchy, 3-88
OR phrase, 3-164
ORD function, 12-25
ORD<#106>MAX function, 12-26
ORD<#106>MIN function, 12-26
ordinal number of maximum (ORD<#106>MAX function), 12-26
ordinal number of minimum (ORD<#106>MIN function), 12-26
ordinal position in collating sequence (ORD function), 12-25
ORGANIZATION IS INDEXED clause, 6-11
Indexed I<#106>O module, 6-11
ORGANIZATION IS RELATIVE clause, 5-7
Relative I<#106>O module, 5-9
ORGANIZATION IS SEQUENTIAL clause, 4-5, 4-11
OUTPUT PROCEDURE phrase
MERGE statement, 8-10
SORT statement, 8-14
OVERFLOW phrase
CALL statement, 7-30
STRING statement, 3-158
UNSTRING statement, 3-164
overlapping operands
Nucleus, 3-98

P

PACKED<#106>DECIMAL, 3-47
PADDING CHARACTER clause
Sequential I<#106>O module, 4-11
PAGE
WRITE statement, 4-45
PAGE clause
LAST DETAIL, 10-13
NEXT GROUP clause, 10-39
PAGE FOOTING (PF), 10-45
PAGE HEADING (PH), 10-45
PAGE regions, 10-16
PAGE<#106>COUNTER, 10-5, 10-11
paragraph, 2-47
header, 2-47
name, 2-15, 2-47
parameter passing, 7-8
parentheses, 3-70
condition, 3-86
PICTURE clause, 3-26
passing parameters to programs, 7-8
PERFORM statement, 3-139
MERGE statement, 8-10
segmentation, 11-6
USE statement, 4-43, 6-32
period (.) PICTURE symbol, 3-31
PF (PAGE FOOTING), 10-45
PH (PAGE HEADING), 10-45
PIC clause, 3-25
PICTURE
symbols, 3-25
PICTURE (PIC) clause, 3-25
BLANK WHEN ZERO clause, 3-18
character string, 2-22
editing rules, 3-32
Linkage Section, 7-19
precedence rules, 3-37
Working<#106>Storage Section, 3-15
plus (+) PICTURE symbol, 3-31
POINTER phrase
STRING statement, 3-158
PRESENT<#106>VALUE function, 12-27
presentation rules, 10-16
PRINTER
ASSIGN clause, 4-6
Procedure Division, 2-40
declaratives, 2-40
explicit references, 2-34
header, 7-29
implicit references, 2-34
implicit transfers of control, 2-34
Indexed I<#106>O module, 6-16
Interprogram Communication module, 7-29
Nucleus, 3-70
procedures, 2-40
Relative I<#106>O module, 5-12
Report Writer module, 10-51
Sequential I<#106>O module, 4-29

Sort<#106>Merge module, 8-6
structure, 2-41
procedures, 2-40
PROGRAM COLLATING SEQUENCE, 3-9
program name, 2-14
CANCEL statement, 7-34
program segments, 11-1
PROGRAM<#106>ID paragraph, 3-5
pseudo<#106>[text], 9-2

Q

qualification, 2-27
CONTROL clause, 10-12
Linkage Section, 7-18
MERGE statement, 8-7
OCCURS clause, 3-21
READ statement, 6-23
RELEASE statement, 8-12
RENAMES clause, 3-42
restriction, 3-2
REWRITE statement, 4-41, 5-23, 6-27
SORT statement, 8-14
START statement, 5-25, 6-30
VALUE OF clause, 4-28
Working<#106>Storage Section, 3-14
WRITE statement, 4-44, 5-28, 6-33
QUOTE/QUOTES, 3-2

R

RANDOM function, 12-27
RANGE function, 12-28
RD entry, 10-10
RD level indicator, 10-11
READ statement
DELETE statement, 6-19
indexed I<#106>O module, 6-23
OPEN statement, 5-16, 6-20
Relative I<#106>O module, 5-19
REWRITE statement, 5-23, 6-27
Sequential I<#106>O module, 4-38
record concepts, 2-23
RECORD CONTAINS clause
Sequential I<#106>O module, 4-24
RECORD DELIMITER clause
Sequential I<#106>O module, 4-12
record description entry
Sequential I<#106>O module, 4-17
record description structure, 3-15
RECORD KEY clause, 6-1, 6-11
record name, 2-14
Record Operations, 2-2
REDEFINES clause, 3-40
reference format
general description, 2-44
restriction, 3-2
text words, 9-4
reference modification, 2-31
relation condition, 3-75
abbreviated combined, 3-87
MERGE statement, 8-8
nonnumeric operands, 3-76
numeric operands, 3-76
Relative I<#106>O module, 5-1
RELATIVE KEY phrase, 5-7
relative line number, 10-37
relative record number, 5-1
RELEASE statement, 8-12
REM (remainder) function, 12-29
REMAINDER phrase, 3-112
RENAMES clause, 3-42
level number, 3-20
REPLACE statement, 9-7
REPLACING phrase, 3-126, 9-2
REPORT clause, 10-9
report description entry, 10-10
REPORT FOOTING (RF), 10-45
report format, 10-63
report group description
Report Writer module, 10-17
REPORT HEADING (RH), 10-45
report name, 2-14
report section, 10-5, 10-10
description, 10-2
Report Writer module, 10-1
Rerun, 2-4
RESERVE AREA/AREAS clause
Indexed I<#106>O module, 6-8
Relative I<#106>O module, 5-7
RESERVE clause
Sequential I<#106>O module, 4-13
reserved words, 2-16, A-1
RESET phrase, 10-42
RETURN statement, 8-13
REVERSE function, 12-29
REWRITE statement
Indexed I<#106>O module, 6-27
OPEN statement, 5-16, 6-20
Relative I<#106>O module, 5-23
Sequential I<#106>O module, 4-41
REWRITE statements, 2-3
RF (REPORT FOOTING), 10-45

RH (REPORT HEADING), 10-45
RIGHT, 3-45
rolling forward, 10-44
ROUNDED phrase, 3-93
RUN, 3-157

S

SAME AREA clause
  Relative I<#106>O module, 5-10
  Sequential I<#106>O module, 4-15
SAME clause, 4-15
SAME RECORD AREA clause
  Relative I<#106>O module, 5-10
  Sequential I<#106>O module, 4-15
  Sort<#106>Merge module, 8-3
SAME SORT AREA clause
  Sort<#106>Merge module, 8-3
SAME SORT<#106>MERGE AREA clause
  Sort<#106>Merge module, 8-3
scope
  names of programs, 7-8
scope of names, 7-11
scope terminators
  explicit, 2-36
  implicit, 2-36
SEARCH statement, 3-149
section header, 2-47
section name, 2-15
segment number, 2-14, 11-5
SEGMENT<#106>LIMIT clause, 11-4
segmentation
  ALTER statement, 11-6
  MERGE statement, 11-7
  PERFORM statement, 11-6
  SORT statement, 8-2, 8-18, 11-7
  TO GO statement, 11-6
Segmentation module, 11-1
SELECT clause
  Indexed I<#106>O module, 6-8
  Relative I<#106>O module, 5-7
  Sequential I<#106>O module, 4-6
  Sort<#106>Merge module, 8-3
separators, 2-12
sequence numbers, 2-45
sequential file, 4-1
Sequential I<#106>O module, 4-1
SET statement, 3-153
  overlapping operands, 3-98
  SEARCH statement, 3-152
SIGN clause, 3-44, 10-40
  class condition, 3-81
  MOVE statement, 3-136
  PICTURE symbol, 3-28
sign condition, 3-84
simple condition, 3-75
simple insertion editing, 3-32
SIN (sine) function, 12-30
SIZE ERROR phrase, 3-93
sort file, 8-1
SORT statement, 8-14
  OPEN statement, 4-34
  segmentation, 11-7
Sort<#106>Merge
  file description entry, 8-6
  files, 8-20
  main storage size, 8-20
  module, 8-1
SOURCE clause, 10-41
source program, 2-37, 3-2
  COPY statement, 9-1
source text manipulation module, 9-1
SOURCE<#106>COMPUTER paragraph, 3-7
SOURCE-COMPUTER paragraph, 2-38
space
  library text word, 9-3
SPACE/SPACES
  restriction, 3-2
special insertion editing, 3-33
special registers
  LINAGE<#106>COUNTER, 4-5
SPECIAL<#106>NAMES paragraph, 3-9
  ACCEPT statement, 3-103
  DISPLAY statement, 3-109
  switch<#106>status condition, 3-83
SPECIAL-NAMES paragraph, 2-38
specifications
  explicit, 2-33
  implicit, 2-33
SQRT (square root) function, 12-30
standard alignment rules, 2-26
STANDARD<#106>1 phrase, 3-9
STANDARD<#106>DEVIATION
    function, 12-31
START statement
  Indexed I<#106>O module, 6-29
  OPEN statement, 5-16, 6-20
  Relative I<#106>O module, 5-25
statistics, 6-7
STOP statement, 3-157
STRING statement, 3-158
  overlapping operands, 3-98
structure
  Environment Division, 2-39

Identification Division, 2-38
Interprogram Communication module, 7-14
Procedure Division, 2-41
style conventions used in this manual, 2-9
subscripting, 2-7
CONTROL clause, 10-12
subscripts, 2-29
substituting, 10-44
subtotaling, 10-44
SUBTRACT statement, 3-162
composite of operands, 3-97
COMPUTE statement, 3-108
data conversion, 3-97
decimal alignment, 3-97
maximum operand size, 3-97
multiple results, 3-98
SUM clause, 10-42
sum counter, 10-42
INITIATE statement, 10-55
SUM function, 12-32
summary report, 10-54
summing, 10-44
SUPPRESS, 10-4
statement, 10-59
switch<#106>status condition, 3-83
symbolic character, 2-14
SYNC clause, 3-45
SYNCHRONIZED (SYNC) clause, 3-45
elementary data item, 3-17
syntax rules, 2-11
system names, 2-16

T

table handling, 2-5
table definition, 2-5
table items, 2-7
table references, 2-7
TALLYING phrase
INSPECT statement, 3-126
UNSTRING statement, 3-164
TAN (tangent) function, 12-32
TAPE
ASSIGN clause, 4-6
TERMINATE, 10-4
TERMINATE statement, 10-59
INITIATE statement, 10-56
SUM clause, 10-43
USE statement, 10-62
text name, 2-14, 9-2
THROUGH (THRU)
MERGE statement, 8-8
PERFORM statement, 3-139
RENAMES clause, 3-42
SORT statement, 8-14
VALUE clause, 3-53
TIME, 3-103
TRAILING, 3-44
TYPE clause, 10-45

U

Unary
arithmetic operator, 3-71
minus, 3-71
plus, 3-71
UNSTRING statement, 3-164
UNTIL phrase, 3-142
UP BY, 3-153
UPON phrase
DISPLAY statement, 3-109
SUM clause, 10-42
UPPER<#106>CASE function, 12-33
USAGE clause, 3-47, 10-49
DISPLAY, 3-47
INSPECT statement, 3-126
relation condition, 3-75
SIGN clause, 3-44
STRING statement, 3-158
UNSTRING statement, 3-164
USAGE IS INDEX clause
MOVE statement, 3-134
SEARCH statement, 3-149
Working<#106>Storage Section, 3-15
USE BEFORE REPORTING statement, 7-41, 10-62
USE statement
DELETE statement, 5-14, 6-18
Indexed I<#106>O module, 6-31
Interprogram Communication module, 7-40
invalid key condition, 6-6
READ statement, 6-23
Relative I<#106>O module, 5-26
REWRITE statement, 5-23, 6-27
Sequential I<#106>O module, 4-42
START statement, 5-25, 6-30
WRITE statement, *4-46*, 5-28, 6-33
user<#106>defined words, 2-14
USING phrase
CALL statement, 7-30

Linkage Section, 7-18
MERGE statement, 8-7
Procedure Division header, 7-29
SORT statement, 8-14

V

VALUE clause, 3-53, 10-50
VALUE OF clause, 4-28
VARIANCE function, 12-33
VARYING phrase
PERFORM statement, 3-144

W

WHEN, 3-149
WHEN<#106>COMPILED function, 12-34
WITH ENTRY POINTS phrase, 7-29
words
COBOL, 2-10
optional, 2-17
required, 2-17
reserved, 2-16, A-1
special purpose, 2-17
user<#106>defined, 2-14
Working<#106>Storage records, 3-15
Working<#106>Storage Section, 3-14
WRITE statement
Indexed I<#106>O module, 6-33
OPEN statement, 5-16, 6-20
Relative I<#106>O module, 5-28
Sequential I<#106>O module, 4-44

Z

Z
PICTURE symbol, 3-30
zero suppression editing, 3-36
ZERO/ZEROS/ZEROES
restriction, 3-2

Special Characters

- operator, 3-71
* operator, 3-71
** operator, 3-71
/ operator, 3-71
‘77’ item description entry, 3-15
+ operator, 3-71
= relation, 3-75
> relation, 3-75

© 2015 Unisys Corporation.
All rights reserved.

7831 0448–005